LLD Handbook — Vol 1: OOP, UML, SOLID, Design Principles, Clean Code, Refactoring

OOP · UML · SOLID · Clean Code · Refactoring · Design Patterns · Concurrency · 45+ case studies in Java

6 parts · one place to study LLD: DesignGurus courses + classic books + GitHub problem sets, combined

what it is | problem | how it works | trade-offs | real world | interview | self-check | TL;DR

Personal study notes. Course diagrams belong to DesignGurus. Not for redistribution.

Contents

1. **Chapter 2 — OOP Fundamentals**
   - 2.1 What object-oriented programming is
   - 2.2 Classes, objects, constructors, `this`, `static` and `final`
   - 2.3 Encapsulation
   - 2.4 Abstraction
   - 2.5 Inheritance
   - 2.6 Polymorphism (and the full bank example)
   - 2.7 Compile-time vs runtime polymorphism: overloading vs overriding
   - 2.8 Interfaces (default, static, private methods, functional interfaces)
   - 2.9 Abstract classes vs interfaces
   - 2.10 Enums with fields and behaviour
   - 2.11 Composition over inheritance
   - 2.12 Object identity vs equality: the `equals` / `hashCode` contract
   - 2.13 Immutability and records
   - 2.14 Access modifiers and packages
   - 2.15 Generics basics for LLD
   - 2.16 Coupling and cohesion (the yardsticks)
   - 2.17 Object-oriented analysis and design (OOA / OOD / OOP)
   - Chapter summary: in the interview
   - Self-check (whole chapter)
   - TL;DR
2. **Chapter 3 — Class Relationships & UML**
3. **Part A — Class Relationships**
   - 3.1 The big picture: what a "relationship" really claims
   - 3.2 Association (uni-directional, bi-directional, multiplicity)
   - 3.3 Aggregation
   - 3.4 Composition
   - 3.5 Dependency
   - 3.6 Generalization (inheritance)
   - 3.7 Realization (interface implementation)
   - 3.8 Decision guide: choosing the relationship between two classes
4. **Part B — UML**
   - 3.9 What UML is (structural vs behavioural diagrams)
   - 3.10 Use-case diagrams
   - 3.11 Class diagrams
   - 3.12 Sequence diagrams
   - 3.13 Activity diagrams
   - 3.14 State machine diagrams
5. **Part C — UML in the interview**
   - 3.15 How much UML to draw in an interview
   - 3.16 Chapter self-check
   - 3.17 TL;DR
6. **Chapter 4 — SOLID Principles**
   - 4.1 What SOLID is and why you should learn it
   - 4.2 Single Responsibility Principle (SRP)
   - 4.3 Cohesion
   - 4.4 Coupling
   - 4.5 Separation of Concerns (SoC)
   - 4.6 SRP vs cohesion vs coupling vs SoC — one table
   - 4.7 Open/Closed Principle (OCP)
   - 4.8 Liskov Substitution Principle (LSP)
   - 4.9 Interface Segregation Principle (ISP)
   - 4.10 Dependency Inversion Principle (DIP)
   - 4.11 Refactoring one class through all five principles
   - 4.12 SOLID and design patterns
   - 4.13 SOLID in the design interview
   - 4.14 Common questions about SOLID (with short answers)
   - 4.15 Over-engineering: the warnings in one place
   - Chapter self-check
   - TL;DR
7. **Chapter 5 — More Design Principles**
   - 1. DRY — Don't Repeat Yourself
   - 2. KISS — Keep It Simple, Stupid
   - 3. YAGNI — You Aren't Gonna Need It
   - 4. Law of Demeter (Principle of Least Knowledge)
   - 5. Tell, Don't Ask
   - 6. Composition over Inheritance
   - 7. Program to an Interface, Not an Implementation
   - 8. Encapsulate What Varies
   - 9. Strive for Loosely Coupled Designs
   - 10. The Hollywood Principle (Inversion of Control)
   - 11. Separation of Concerns (SoC)
   - 12. Principle of Least Astonishment (POLA)
   - 13. Fail Fast
   - 14. Immutability
   - 15. Design by Contract (DbC)
   - 16. GRASP — General Responsibility Assignment Software Patterns
   - 16.1 Information Expert
   - 16.2 Creator
   - 16.3 Controller
   - 16.4 Low Coupling
   - 16.5 High Cohesion
   - 16.6 Polymorphism
   - 16.7 Pure Fabrication
   - 16.8 Indirection
   - 16.9 Protected Variations

- 17. Domain-Driven Design (DDD) Essentials

8. **Chapter 6 — Clean Code**
   - What is clean code, and why bother?
   - 1. Meaningful Names
   - 2. Functions
   - 3. Comments
   - 4. Formatting
   - 5. Objects and Data Structures
   - 6. Error Handling
   - 7. Boundaries
   - 8. Unit Tests
   - 9. Classes
   - 10. Systems

9. **Chapter 7 — Refactoring**
   - 7.1 Refactoring fundamentals
   - 7.2 Code smells
   - 7.3 The refactoring catalogue
   - 7.4 Smell → refactorings lookup table
   - Chapter summary — how the principles fit together

   - 11. Emergence — Kent Beck's Four Rules of Simple Design
   - 12. Concurrency
   - 13. Successive Refinement and the Case Studies (Chapters 14–16: how clean code is actually written)
   - 14. Smells and Heuristics (the checklist)
   - 15. Code Review Checklist (best practices)
   - 16. Error-Handling Strategy (best practices for real services)
   - 17. Testing Strategies (unit vs integration, test doubles, mocks)
   - Chapter summary

   - 7.5 Worked example — from messy to clean in six compiling steps
   - 7.6 Refactoring toward patterns
   - 7.7 Putting it together
   - 7.8 Full source code and output

Chapter 2 — OOP Fundamentals

What you will learn in this chapter

- What object-oriented programming (OOP) is and why the LLD round is really a test of it.
- Classes and objects, constructors, `this`, `static` vs instance members, and `final`.
- The four pillars — **encapsulation, abstraction, inheritance, polymorphism** — on one running bank example.
- Inheritance details (`super`, overriding rules) and polymorphism details (overloading vs overriding, dynamic dispatch).
- Interfaces (default, static, private methods, functional interfaces), abstract classes, and when to pick which.
- Enums with fields and behaviour.
- Composition over inheritance, identity vs equality (`equals` / `hashCode`), immutability and records.
- Access modifiers and packages, and the generics you actually need in LLD.
- Object-oriented analysis vs design vs programming (OOA / OOD / OOP), with a four-step process.

Every concept has a Java 21 example that compiles and runs; the real output is shown under each program.

How to run all examples: `cd chapters/02-oop-fundamentals/code && javac --release 21 -d /tmp/lld_02 $(find . -name '*.java') && java -cp /tmp/lld_02 lld.ch02.<package>.Main`

(`javac` is the Java compiler: it turns `.java` source files into `.class` files. `java` runs them on the **JVM**, the Java Virtual Machine, the program that executes compiled Java code; `-cp` says where the compiled classes live.)

Concept checklist (the commonly listed "must-know OOP concepts" and where each is covered):

Concept	Section	Concept	Section
Classes & objects	2.2	Interfaces (default/static/functional)	2.8
Constructors, constructor chaining	2.2	Abstract class vs interface	2.9
`this` keyword	2.2	Enums	2.10
`static` vs instance, `final`	2.2	Composition over inheritance	2.11
Encapsulation	2.3	Identity vs equality, `equals` / `hashCode`	2.12
Abstraction	2.4	Immutability & records	2.13
Inheritance, `super`, overriding rules	2.5	Access modifiers & packages	2.14
Polymorphism, overloading, dynamic dispatch	2.6–2.7	Generics basics	2.15
Coupling & cohesion	2.16	OOA / OOD / OOP	2.17

2.1 What object-oriented programming is

What it is

Object-oriented programming (OOP) is a style of programming that **bundles data with the code allowed to modify it**, packaging both into a single unit called an **object**.

Procedural programming separates the two: data sits in one place, and free-standing functions anywhere in the program can read and modify it. OOP instead **fences the data in**: the only code that can reach it is the object's own methods.

Analogy: a bank vault vs a pile of cash on a table. With the pile (procedural), anyone can take or add money and nobody enforces rules. With the vault (OOP), you must go through the teller (the methods), who checks every request.

The problem it solves

When any code can change any data, a rule such as "a balance can never go negative" has to be copied into every caller, and one forgotten copy is a bug. OOP puts each rule in **one place**, next to the data it protects, and lets a design grow by **adding new classes** instead of editing old ones.

Why this matters for the interview

Nobody in the LLD round will ask you to recite textbook definitions. What they want to see is you **breaking a problem into objects, assigning each one its responsibilities and data, and justifying that breakdown**. Every idea in this chapter is a tool for that.

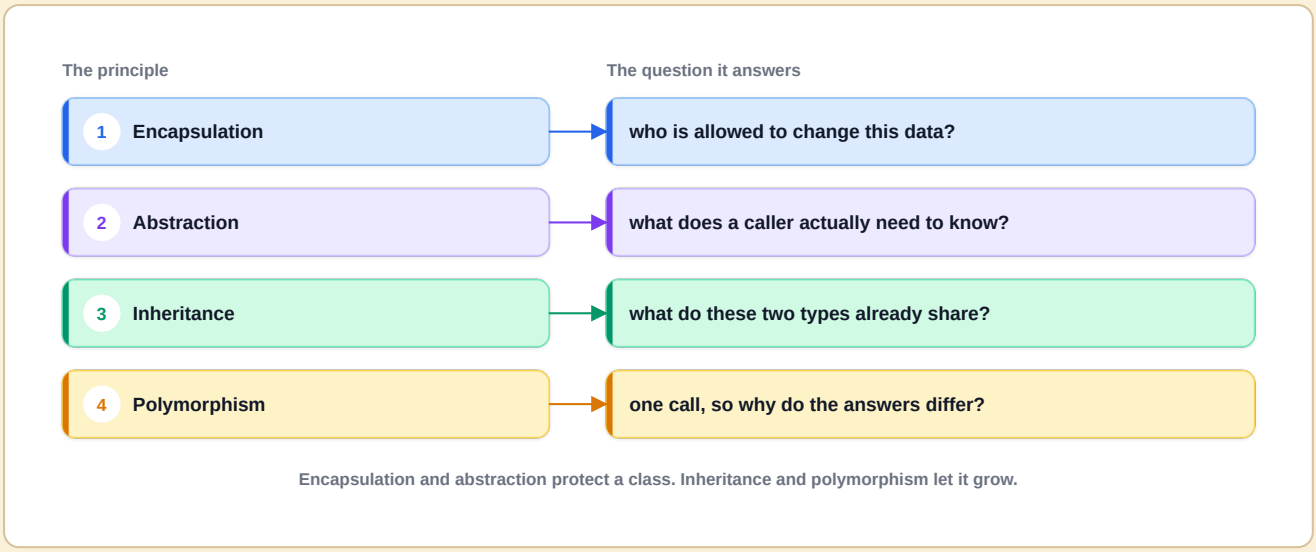


Figure — The four OOP principles and the question each one answers (course diagram)

As the figure above summarises, each pillar answers one design question:

Pillar	Question it answers
Encapsulation	Who is allowed to change this data?
Abstraction	What does a caller actually need to know?
Inheritance	What do these two types already share?
Polymorphism	One call, so why do the answers differ?

The first two pillars guard a single class; the last two let the design expand with new classes.

2.2 Classes, objects, constructors, `this`, `static` and `final`

What it is

- A **class** is the **blueprint**: it lists the fields a thing has and the operations it supports. You write it once.
- An **object** (instance) is **one concrete thing built from that blueprint**, holding actual values in its fields. You create as many as you need with `new`.
- A **field** is a variable that belongs to a class or object. A **method** is a function that belongs to a class or object.
- A **constructor** is special code that runs when an object is created. It has the class's name and **no return type**. Its job: put the new object into a **valid** state.

- `this` is a reference to **the current object** inside its own instance methods and constructors.
- `static` means "belongs to the **class**, shared by all objects". Without `static`, a member is an **instance** member: each object has its own copy.
- `final` means "cannot change": a `final` variable is assigned once, a `final` method cannot be overridden (replaced by a child class with its own version), a `final` class cannot be extended (used as the parent of another class; Section 2.5).

Examples: the `Account` class defines what any account looks like, while account `S-1` (owner Ada, balance 50,000 cents) is a single object of it. In an online shop, `Customer` is the class, and the particular shopper behind order 4192 from last Tuesday is an object.

How it works

1. `new Account("S-1", "Ada", 50_000)` allocates memory for the fields, sets them to defaults (0, `null` meaning "no object", `false`), then runs the constructor.
2. **Initialisation order:** static initialisers (`static` field assignments and `static { ... }` blocks of set-up code) run **once**, when the class is initialised (just before its first real use, such as the first `new` or the first static method call). For each `new`: the parent constructor runs first (`super(...)`, implicit if you do not write it), then instance field initialisers and instance initialiser blocks (plain `{ ... }` blocks inside the class body) in source order, then the rest of the constructor body.
3. If you write **no constructor**, the compiler adds a public no-argument **default constructor**. If you write any constructor, it does not.
4. **Overloaded constructors** (same name, different parameters) should **chain** to one canonical constructor (the single "main" constructor that all the others call) with `this(...)`, so validation lives in one place. In Java 21, `this(...)` or `super(...)` must be the **first statement**. (Java 25 relaxes this with "flexible constructor bodies", JEP 513 (a JDK Enhancement Proposal, the numbered documents that describe new Java features): statements that do not use the object under construction may come before the call. This book targets Java 21, so keep the call first.)
5. Uses of `this`: disambiguate a field from a parameter (`this.id = id`), pass the current object to another method, chain constructors (`this(...)`), and return the object for **fluent chaining** (`return this`).
6. `static` methods have **no** `this`: they cannot touch instance fields directly. Use them for utilities (small helper functions such as `Math.max`) and factories (methods that create and return objects).
7. Variables of class type hold **references** (like an address), not the object itself. `Account alias = a;` copies the reference: both names now point to the **same** object.

Java example

```java
// File: code/lld/ch02/classes/Account.java
package lld.ch02.classes;

/** A CLASS is the blueprint. Every Account OBJECT gets its own copy of the instance fields. */
public class Account {
    // static = belongs to the CLASS, shared by all objects
    private static int openedCount = 0;
    public static final long MIN_OPENING_CENTS = 0;

    // instance fields = belong to EACH object
    private final String id;
    private final String owner;
    private long balanceCents;

    /** Canonical constructor: does all validation in one place. */
    public Account(String id, String owner, long openingCents) {
        if (openingCents < MIN_OPENING_CENTS) throw new IllegalArgumentException("negative opening balance");
        this.id = id;                // 'this.id' is the field, 'id' is the parameter
        this.owner = owner;
        this.balanceCents = openingCents;
        openedCount++;               // shared counter
    }

    /** Overloaded constructor: chains to the canonical one with this(...). Must be the first statement. */
    public Account(String id, String owner) {
        this(id, owner, 0);
    }

    public Account deposit(long cents) {   // returns 'this' so calls can be chained
        if (cents <= 0) throw new IllegalArgumentException("deposit must be positive");
        this.balanceCents += cents;
        return this;
    }

    public long balanceCents() { return balanceCents; }
    public String id()         { return id; }
    public String owner()      { return owner; }

    public static int openedCount() { return openedCount; }  // static method: no 'this' available here

    @Override public String toString() { return id + "(" + owner + ", " + balanceCents + "c)"; }
}
```

```java
// File: code/lld/ch02/classes/Main.java
package lld.ch02.classes;

public class Main {
    public static void main(String[] args) {
        Account a = new Account("S-1", "Ada", 50_000);          // 3-arg constructor
        Account b = new Account("C-1", "Kai");                  // 2-arg constructor chains to 3-arg
        b.deposit(1_000).deposit(500);                          // chaining via 'return this'

        System.out.println("a = " + a);
        System.out.println("b = " + b);
        System.out.println("accounts opened (static, shared): " + Account.openedCount());

        Account alias = a;                                      // copies the REFERENCE, not the object
        alias.deposit(1);
        System.out.println("after alias.deposit(1), a = " + a + ", same object? " + (alias == a));

        try {
            new Account("X", "Bad", -5);
        } catch (IllegalArgumentException e) {
            System.out.println("constructor refused: " + e.getMessage());
        }
        System.out.println("accounts opened: " + Account.openedCount() + " (failed construction did not count)");
    }
}
```

Output:

```
a = S-1(Ada, 50000c)
b = C-1(Kai, 1500c)
accounts opened (static, shared): 2
after alias.deposit(1), a = S-1(Ada, 50001c), same object? true
constructor refused: negative opening balance
accounts opened: 2 (failed construction did not count)
```

Explaining each part: `openedCount` is `static`, so there is one counter for the class; `balanceCents` is an instance field, so each account has its own. The two-argument constructor calls `this(id, owner, 0)`, so validation is written once. `deposit` returns `this`, enabling `b.deposit(1_000).deposit(500)`. `alias == a` is `true` because both variables hold the same reference. The failing constructor threw **before** incrementing the counter, so no half-built object was counted: **validate first, then change state**.

Trade-offs

Choice	Benefit	Cost
Static members	Shared, no object needed	Global state (reachable from anywhere): hard to test and dangerous with threads (parts of a program running at the same time)
Many constructor overloads	Convenient	Confusing beyond 3–4; use a **Builder** (Chapter 8; a helper that collects settings step by step, then creates the object) or static factory methods
`final` fields	Safer, thread-friendlier	Must be set in every constructor

Common mistakes

- Putting per-object data in a `static` field (every object then shares it).
- Doing heavy work or calling overridable methods inside constructors (a subclass override may run before the subclass's fields are set).
- Forgetting `this.` so the parameter is assigned to itself (`id = id;`).
- Thinking `=` copies an object; it copies a reference.

Real-world examples

`Math.max`, `Integer.parseInt`, `List.of` are static methods; `StringBuilder.append` returns `this` for chaining; `String` is a `final` class.

In the interview

> **30-second answer:** "A class is the blueprint, an object is an instance with its own state. Constructors put an object into a valid state; I chain overloads with `this(...)` so validation lives once. Static members belong to the class and are shared, so I keep them for constants, utilities and factories and avoid mutable (changeable) static state."

- *"What happens if you define a constructor with arguments only?"* — The default no-arg constructor is no longer generated; `new X()` stops compiling.
- *"Can a static method use `this`?"* — No; there is no current object in a static context.
- *"Why avoid mutable static fields?"* — They are hidden global state shared by all threads and tests.

Self-check

- **Q:** How many copies of a `static` field exist for 1,000 objects? **A:** One.
- **Q:** What runs first: parent constructor or child field initialisers? **A:** Parent constructor.
- **Q:** What does `alias == a` compare? **A:** References (whether both point to the same object).

TL;DR

Class = blueprint, object = instance. Constructors create valid objects; chain them with `this(...)`. `static` = one per class; `final` = cannot change. Variables hold references.

2.3 Encapsulation

What it is

Encapsulation means an object hides its own data (**private**) and allows the outside world to modify it **only by calling methods the object itself defines and guards**. Analogy: a medicine bottle with a child-proof cap: the contents are reachable, but only through a mechanism that enforces the rules.

The problem it solves

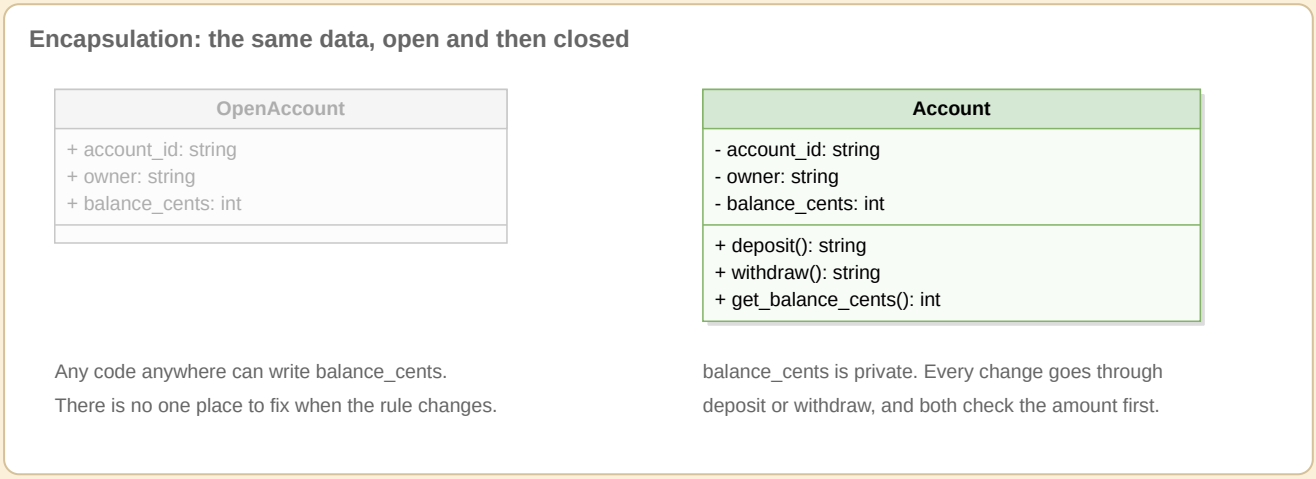


Figure — The same account data, first open (public fields anyone can write) and then closed (private fields changed only through deposit and withdraw) (course diagram)

The left side of the figure shows unprotected, public fields: **code in any file** may write a negative number into `balance`, and if the rule ever changes you have **nowhere central** to update it. (That open version exists only as a contrast; our code does not contain it.) The right side makes the balance private, and both `deposit` and `withdraw` validate the amount first. Overdrawing is therefore impossible:

every route to the balance goes through a method that can say no. (The default `monthlyUpdate` on `Account` does nothing yet; it becomes useful in Section 2.6.)

How it works

1. Make fields `private` (and `final` where possible).
2. Expose **behaviour** (`deposit`, `withdraw`), not raw setters (`setBalance`; methods that simply overwrite a field).
3. Validate inside those methods: the rule lives **once**.
4. Return copies or unmodifiable views (read-only wrappers that throw if someone tries to change them) of internal collections (never the live list).
5. For every field, settle **which code may modify it**. When the honest answer is "nobody but this class", state it aloud in the interview: it turns a list of fields into a visible design decision.

Java example

The `Account` class of the bank example in Section 2.6 (package `lld.ch02.bank`) is the encapsulation example: `balanceCents` is `private`; `deposit` rejects non-positive amounts; `withdraw` rejects amounts larger than the balance; both are `final` so a subclass cannot weaken the rule.

Trade-offs

Benefit	Cost
Invariants (rules that must always hold) enforced in one place	A little more code than public fields
Internals can change without breaking callers	Over-encapsulating trivial data carriers is noise (use records)

When not to use it (strictly)

Plain data carriers with no rules (DTOs, Data Transfer Objects that only carry data between parts of a system; events, records of something that happened) do not need hand-written getters and validation; a **record** gives you private final fields and accessors for free.

Common mistakes

- **Getter/setter for every field** (a getter is a method that just returns a field; "anemic" objects): the data is technically private but anyone can still set anything. That is not encapsulation.
- Returning a mutable internal list (`return items;`), so callers modify your state behind your back.

Real-world examples

`java.lang.String` hides its internal byte array; `ArrayList` hides its backing array (the plain array that actually stores the elements) and `size`; `LocalDate` exposes only validated factory methods (`LocalDate.of(2026, 2, 30)` throws).

In the interview

> **30-second answer:** "Encapsulation hides state behind methods, and those methods are the single home of every rule about that state. For each field I ask who may change it; if only this class, it is private and changed through intention-revealing methods (named after the business action they perform) like `withdraw`, not a setter."

- *"Are getters and setters encapsulation?"* — Only partly. A setter with no rule is a public field in disguise. Prefer behaviour methods that enforce rules.

Self-check

- **Q:** Where should the "no negative balance" rule live? **A:** Inside `Account.withdraw`, once.

TL;DR

Private data + methods that enforce the rules = encapsulation. Expose behaviour, not setters.

2.4 Abstraction

What it is

Abstraction means giving a caller **just the operations it requires** while concealing **how** they are carried out. Put simply: encapsulation conceals **state**, abstraction conceals **design choices**, and the short list of operations stays valid even if everything behind it is rewritten. Analogy: a car's steering wheel and pedals. You drive without knowing about fuel injection, and the engine can be replaced by an electric motor without changing how you drive.

The problem it solves

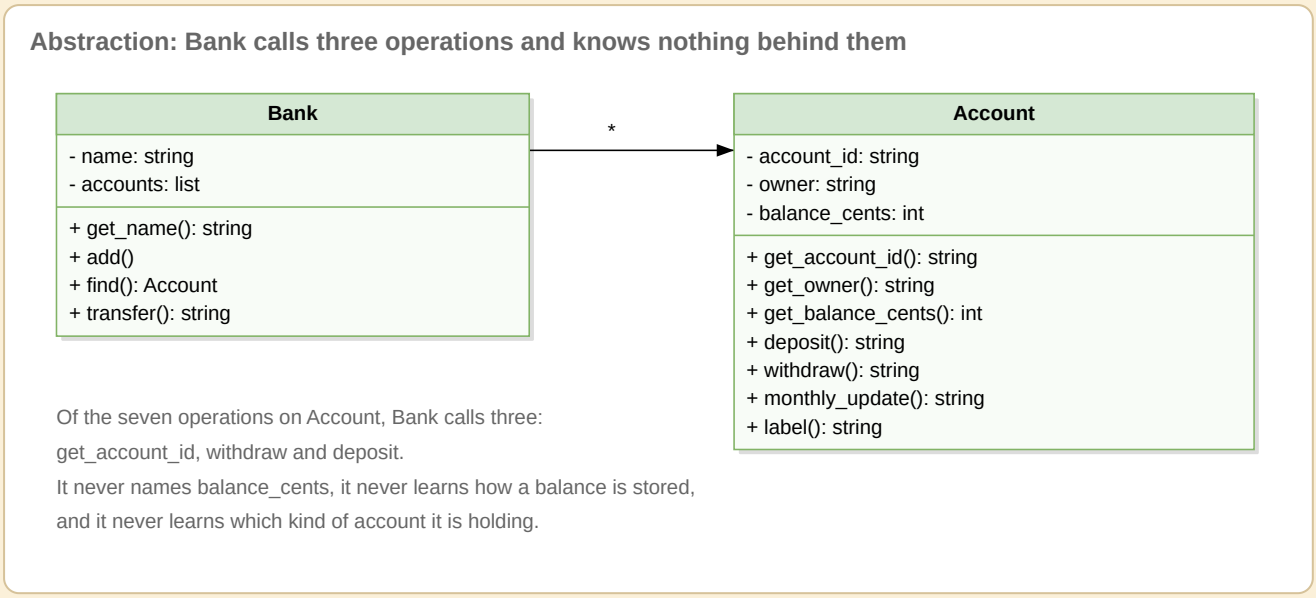


Figure — Of Account's seven public operations, Bank relies on three and sees nothing of what lies behind them (course diagram)

In the course diagram `Account` has seven public operations (it includes a `label()` helper; our Java `Account` has six: `id`, `owner`, `balanceCents`, `deposit`, `withdraw`, `monthlyUpdate`). Either way, `Bank.transfer` relies on only three of them: read the id, withdraw, deposit. `Bank` has no idea that balances are stored as whole cents, what checks an account performs, or that several account subclasses exist. That ignorance is the point: the balance could be moved from a `long` field into a database table and `transfer` would stay exactly as it is.

How it works

1. Decide the **smallest set of operations** a caller needs.
2. Name them after **what** they do (`transfer`), not how.
3. Put them on an interface (a named list of methods with no code, which classes promise to provide; Section 2.8) or the public surface of a class; keep everything else private.
4. Callers depend on that surface only.

Java example

`Bank.transfer` in Section 2.6 uses only `id()`, `withdraw()` and `deposit()`.

Trade-offs

Benefit	Cost
Callers are simpler and survive internal rewrites	A wrong abstraction is expensive to change later
Enables polymorphism (many implementations behind one surface)	Too many layers make code hard to follow

Common mistakes

- **Leaky abstraction:** the interface exposes implementation details (`getHashMapOfSeats()`).
- Abstracting too early, before two concrete cases exist.

Real-world examples

`java.util.List` (callers do not care whether it is an `ArrayList` or `LinkedList`); JDBC's `Connection`/`Statement` (JDBC, Java Database Connectivity, is Java's standard way to talk to SQL databases; same code for different databases); `java.io.InputStream`.

In the interview

> **30-second answer:** "Abstraction is exposing only what the caller needs. My test is the sentence 'the caller does not need to know that': whenever I can honestly say it, something moves behind the interface, the design shrinks, and I can change the implementation freely."

- *"Encapsulation vs abstraction?"* — Encapsulation hides and protects **data** with access control (keywords such as `private` that decide who may see a member, Section 2.14); abstraction hides **complexity and decisions** behind a small set of operations. They work together.

Self-check

- **Q:** Which three `Account` operations does `Bank.transfer` need? **A:** `id()`, `withdraw()`, `deposit()`.

TL;DR

Show the minimum surface; hide the how. "The caller does not need to know that."

2.5 Inheritance

What it is

Inheritance lets a new class (the **child**, subclass) start with everything an existing class (the **parent**, superclass) has, then **add** fields/methods or **replace** (override) some methods. In Java: `class SavingsAccount extends Account`. Analogy: a savings account **is a** bank account with one extra rule.

The problem it solves

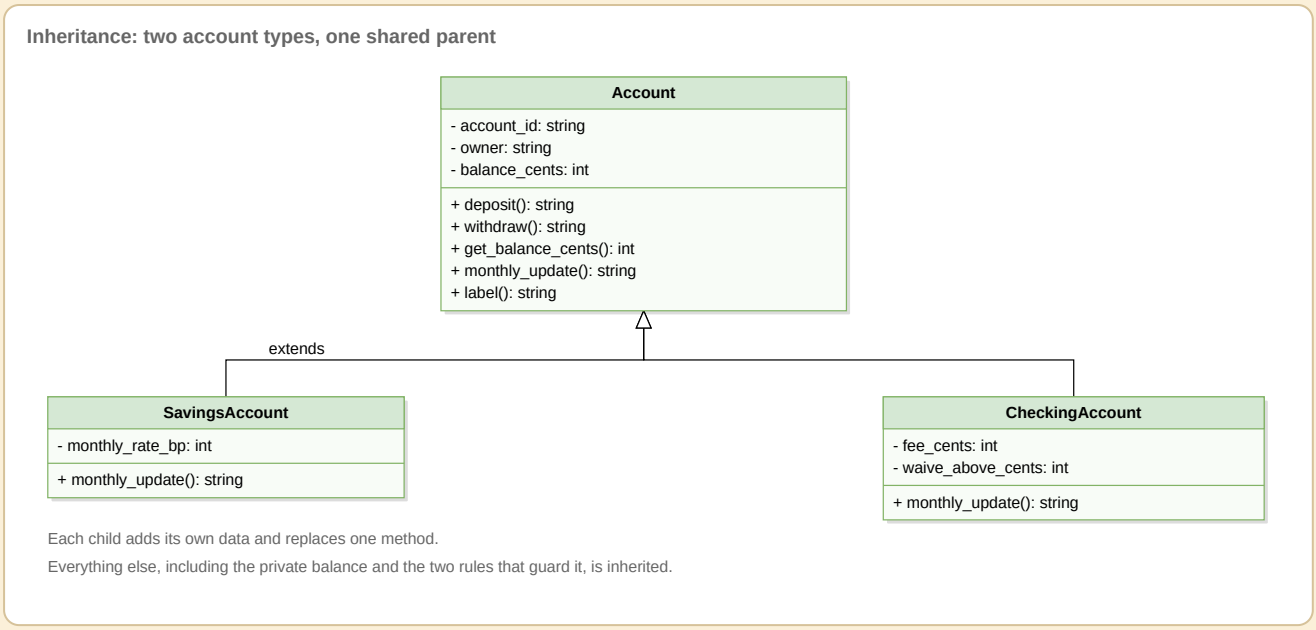


Figure — SavingsAccount and CheckingAccount extend Account; each adds its own data and replaces one method (course diagram)

Savings accounts earn interest; checking accounts take a monthly fee unless their balance clears a threshold. Apart from that, the two share everything: the balance, the deposit check and the withdrawal check, so all of that comes from the parent. In the figure, savings contributes one new field and checking two, and **each overrides a single method**; neither duplicates the balance or its rules. If the is-a sentence were false, the difference would belong in a **field**, not in a parent class.

How it works

1. Declare `class Child extends Parent`. Java allows **one** parent class (single inheritance of classes), but many interfaces.
2. The child inherits all **accessible** (non-private) members. Private fields still exist in the object but are reached only through the parent's methods. **Constructors are not inherited.**
3. The child constructor must call a parent constructor, `super(...)`, as its first statement (implicitly `super()` if omitted, which fails to compile if the parent has no no-arg constructor).
4. `super.method()` calls the parent's version from inside an override (reuse, then extend).
5. **Overriding rules:**

Rule	Detail
Same signature	Same method name and parameter types
Return type	Same, or a **subtype** (a type that extends or implements it) for reference types (*covariant return*); primitives (`int`, `long`, `boolean`...) must match exactly
Access	Same or **wider** (protected, i.e. visible to subclasses, → public is fine; public → protected is not)
Checked exceptions	May throw fewer or narrower checked exceptions, never new or broader ones; unchecked are free
Cannot override	`final` methods, `static` methods (they are **hidden**, not overridden), `private` methods (not inherited)
Annotation	Always write `@Override` (an annotation is an `@` tag that gives the compiler extra information): the compiler then catches typos and wrong signatures

(About the exceptions row: a **checked exception**, such as `IOException`, is one the compiler forces callers to catch or declare; an **unchecked exception**, such as `IllegalArgumentException`, extends `RuntimeException` and needs no declaration.)

6. Every class ultimately extends `java.lang.Object`, which is why every object has `toString`, `equals`, `hashCode` and `getClass`.
7. **The is-a test:** the sentence "a SavingsAccount **is an** Account" must be true in every situation. A car is not an engine, so `Car` should **hold** an `Engine` (Section 2.11).

Java example

See `SavingsAccount` and `CheckingAccount` in Section 2.6 (each calls `super(...)` and overrides `monthlyUpdate()`), and `Circle` in Section 2.7 (covariant return, `super.who()`, static-method hiding, field hiding).

Trade-offs

Benefit	Cost
Reuse of shared state and behaviour	**Tight coupling** (a change in one class forces changes in another): child depends on parent internals (fragile base class)
Enables polymorphism	Hierarchy is fixed at compile time (when the code is compiled, before it runs); single parent only
Natural for true "is-a" families	Deep hierarchies (more than 2–3 levels) are hard to reason about

When not to use it

- Only to **reuse code** when the is-a sentence is false (use composition).
- When the child would need to **disable** a parent method (throw `UnsupportedOperationException` or do nothing): that breaks the **Liskov Substitution Principle** (Chapter 4): the child can no longer stand in for the parent.
- For variations that differ **only in data** (use a field or enum).

Common mistakes

- `Square extends Rectangle` with independent `setWidth`/`setHeight`: code expecting a rectangle breaks.
- Forgetting `@Override` and silently **overloading** instead (e.g. `equals(Point p)` instead of `equals(Object o)`).
- Calling overridable methods from a parent constructor.

Real-world examples

Good: `FileInputStream extends InputStream`; Spring's (Spring is a popular Java application framework) `AbstractController`-style base classes. Known mistakes in the JDK (Java Development Kit: the compiler plus the standard library) itself: `java.util.Stack extends Vector` (a stack, which should only add and remove at the top, exposes random-access `insertElementAt`, which reaches any position), and `java.util.Properties extends Hashtable` (lets you put non-string entries).

In the interview

> **30-second answer:** "Inheritance lets a subclass reuse all of its parent's members, and it is only right when the sentence 'the child is a kind of the parent' holds in every situation. I use it for genuine type families, override with `@Override`, and if a child would have to disable a parent method I switch to composition, because that is a Liskov violation."

- *"Can you override a static method?"* — No. A static method with the same signature in the child **hides** the parent's; which one runs depends on the compile-time type (see Section 2.7 output).
- *"Can an override return a different type?"* — Only a subtype of the original (covariant return), e.g. `Circle copy()` overriding `Shape copy()`.

Self-check

- **Q:** Is `private` inherited? **A:** The field exists in the child object but the child cannot access it directly; private methods are not overridden.
- **Q:** Can an override reduce visibility from `public` to `protected`? **A:** No.

TL;DR

`extends` for true is-a families only. `super(...)` first, `@Override` always, never narrow access or disable parent behaviour.

2.6 Polymorphism (and the full bank example)

What it is

Polymorphism (Greek for "many forms") means **the same method call can do different things depending on the object it lands on**. The calling code works with a variable of the **parent type** and invokes a method by name; the concrete object behind that variable determines the behaviour. Analogy: a "start" button that starts a car, a washing machine or a laptop, each in its own way.

The problem it solves

Without it, code asks "what type are you?" everywhere: `if (a instanceof SavingsAccount) ... else if (a instanceof CheckingAccount) ...`. Every new type means finding and editing every such branch. With polymorphism, the branch disappears.

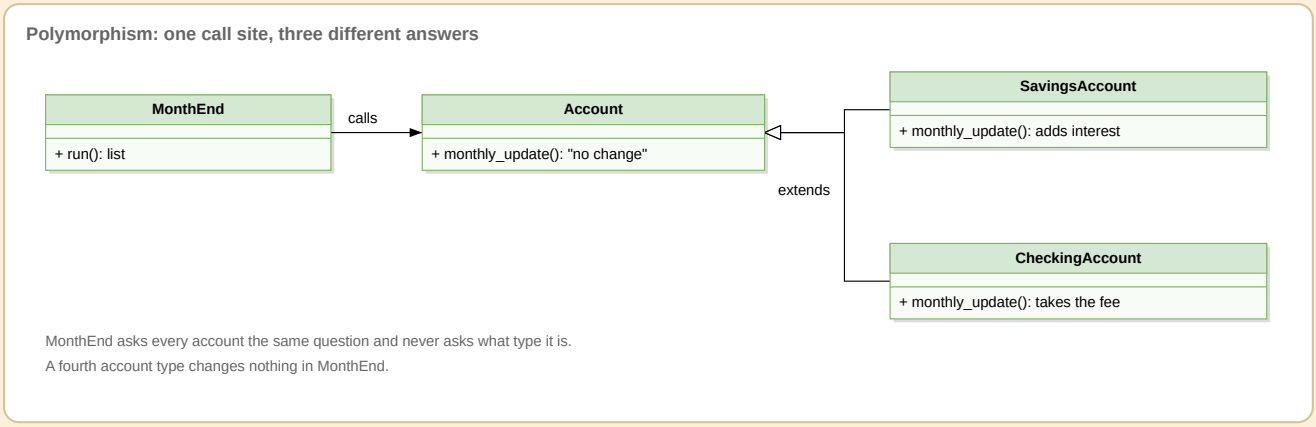


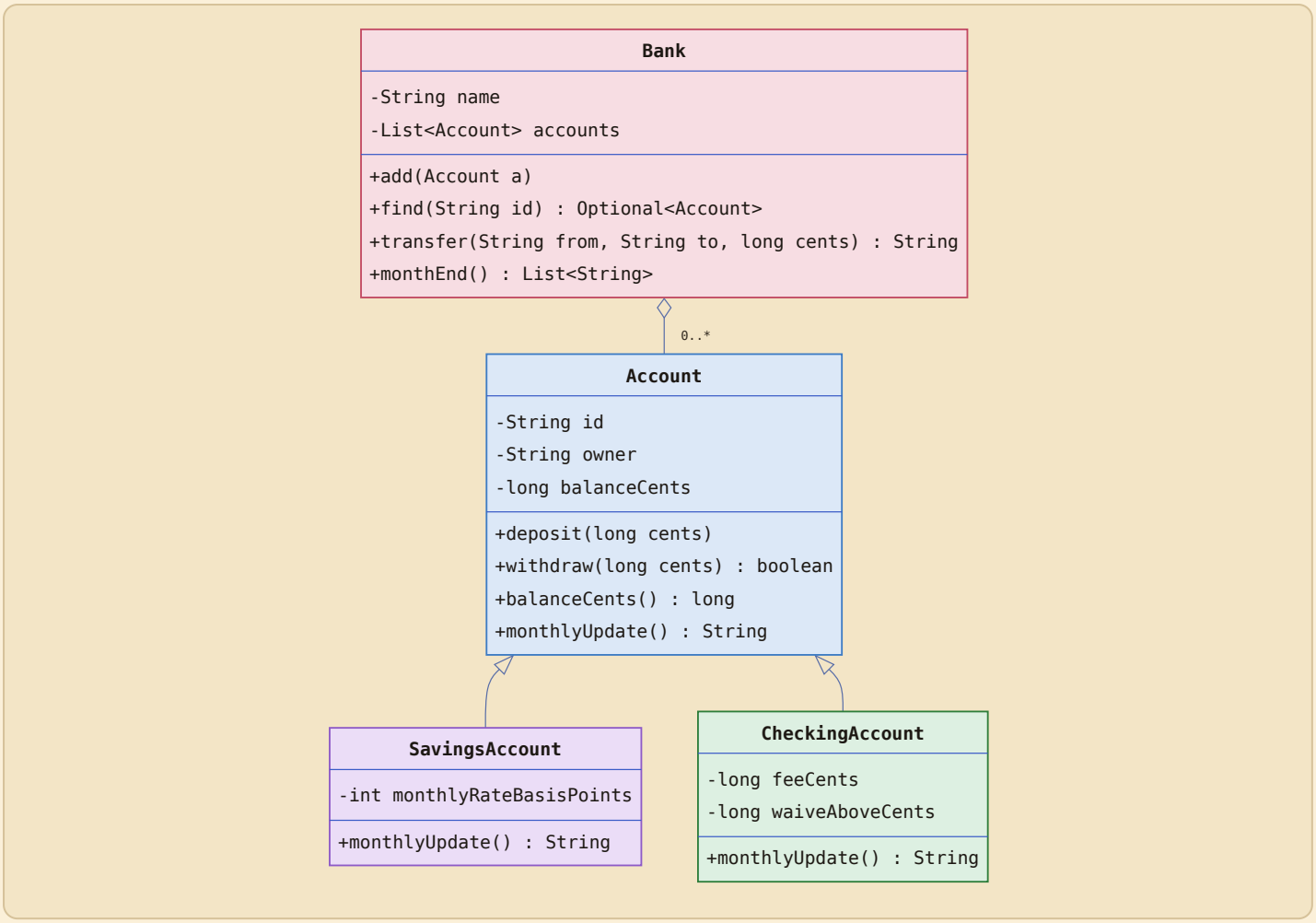
Figure — MonthEnd calls monthlyUpdate on every account and never asks which type it holds (course diagram)

In the figure, the month-end loop sends the identical request to each account and **never checks which kind it is**; it contains no `if` about account types. Introduce a fourth account type next month and the loop needs no edit.

How it works

1. Declare the method on the parent type (or interface).
2. Each child overrides it.
3. Callers hold the **parent type** (`Account a`) and call the method.
4. At run time (while the program is running) the JVM looks at the **actual class of the object** and runs that class's version: **dynamic dispatch** (also called late binding).

Java example: the bank (encapsulation + abstraction + inheritance + polymorphism)



```java
// File: code/lld/ch02/bank/Account.java
package lld.ch02.bank;

/**
 * ENCAPSULATION: the balance is private; the only way to change it is through deposit/withdraw,
 * and both enforce the rules. The parent also defines the hook monthlyUpdate() for POLYMORPHISM.
 */
public class Account {
    private final String id;
    private final String owner;
    private long balanceCents;                      // hidden: nobody outside can set it to -5

    public Account(String id, String owner, long openingCents) {
        if (openingCents < 0) throw new IllegalArgumentException("negative opening balance");
        this.id = id;
        this.owner = owner;
        this.balanceCents = openingCents;
    }

    public final void deposit(long cents) {         // final: children may not weaken the rule
        if (cents <= 0) throw new IllegalArgumentException("deposit must be positive");
        balanceCents += cents;
    }

    public final boolean withdraw(long cents) {
        if (cents <= 0 || cents > balanceCents) return false;   // the one place the rule lives
        balanceCents -= cents;
        return true;
    }

    public final long balanceCents() { return balanceCents; }
    public final String id()         { return id; }
    public final String owner()      { return owner; }

    /** Default month-end behaviour. Subclasses OVERRIDE it. */
    public String monthlyUpdate() { return "no change"; }
}
```

```java
// File: code/lld/ch02/bank/Bank.java
package lld.ch02.bank;

import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

/**
 * ABSTRACTION: Bank uses only id(), withdraw() and deposit(). It does not know how a balance is
 * stored, what rules an account applies, or which subclass it holds.
 */
public class Bank {
    private final String name;
    private final List<Account> accounts = new ArrayList<>();

    public Bank(String name) { this.name = name; }

    public void add(Account a) { accounts.add(a); }

    public Optional<Account> find(String id) {
        return accounts.stream().filter(a -> a.id().equals(id)).findFirst();
    }

    public String transfer(String fromId, String toId, long cents) {
        Account from = find(fromId).orElseThrow();
        Account to = find(toId).orElseThrow();
        if (!from.withdraw(cents)) return fromId + " rejected, not enough money";
        to.deposit(cents);
        return "moved " + Money.format(cents) + " from " + fromId + " to " + toId;
    }

    /** POLYMORPHISM: one call, a different result per object, and no 'if' on the account type. */
    public List<String> monthEnd() {
        List<String> lines = new ArrayList<>();
        for (Account a : accounts) {
            String what = a.monthlyUpdate();                // dynamic dispatch picks the override
            lines.add(a.id() + " (" + a.owner() + "): " + what + ", balance " + Money.format(a.balanceCents()));
        }
        return lines;
    }

    public List<Account> accounts() { return List.copyOf(accounts); }
    public String name() { return name; }
}
```

```java
// File: code/lld/ch02/bank/CheckingAccount.java
package lld.ch02.bank;

/** A checking account charges a monthly fee unless the balance is above a threshold. */
public class CheckingAccount extends Account {
    private final long feeCents;
    private final long waiveAboveCents;

    public CheckingAccount(String id, String owner, long openingCents, long feeCents, long waiveAboveCents) {
        super(id, owner, openingCents);
        this.feeCents = feeCents;
        this.waiveAboveCents = waiveAboveCents;
    }

    @Override
    public String monthlyUpdate() {
        if (balanceCents() >= waiveAboveCents) return "fee waived";
        return withdraw(feeCents) ? "fee " + Money.format(feeCents) : "fee could not be taken";
    }
}
```

```java
// File: code/lld/ch02/bank/Money.java
package lld.ch02.bank;

/** Utility: formats whole cents as dollars. Private constructor = cannot be instantiated. */
public final class Money {
    private Money() { }
    public static String format(long cents) { return String.format("$%d.%02d", cents / 100, cents % 100); }
}
```

```java
// File: code/lld/ch02/bank/SavingsAccount.java
package lld.ch02.bank;

/** INHERITANCE: a savings account IS an account; it adds a rate and replaces one method. */
public class SavingsAccount extends Account {
    private final int monthlyRateBasisPoints;      // 25 bp = 0.25 %

    public SavingsAccount(String id, String owner, long openingCents, int monthlyRateBasisPoints) {
        super(id, owner, openingCents);            // parent constructor runs first
        this.monthlyRateBasisPoints = monthlyRateBasisPoints;
    }

    @Override
    public String monthlyUpdate() {
        long interest = balanceCents() * monthlyRateBasisPoints / 10_000;
        if (interest > 0) deposit(interest);
        return "interest " + Money.format(interest);
    }
}
```

```java
// File: code/lld/ch02/bank/Main.java
package lld.ch02.bank;

public class Main {
    public static void main(String[] args) {
        Bank bank = new Bank("Gurus Bank");
        bank.add(new SavingsAccount("S-1", "Ada", 50_000, 25));
        bank.add(new CheckingAccount("C-1", "Kai", 8_000, 250, 50_000));
        bank.add(new Account("P-1", "Ravi", 3_000));

        System.out.println(bank.name() + ", opening balances:");
        for (Account a : bank.accounts()) {
            System.out.println("  " + a.id() + " (" + a.owner() + "): " + Money.format(a.balanceCents()));
        }
        System.out.println(bank.transfer("S-1", "C-1", 10_000));
        System.out.println(bank.transfer("S-1", "C-1", 90_000));
        System.out.println("month end:");
        bank.monthEnd().forEach(line -> System.out.println("  " + line));
    }
}
```

Output:

```
Gurus Bank, opening balances:
  S-1 (Ada): $500.00
  C-1 (Kai): $80.00
  P-1 (Ravi): $30.00
moved $100.00 from S-1 to C-1
S-1 rejected, not enough money
month end:
  S-1 (Ada): interest $1.00, balance $401.00
  C-1 (Kai): fee $2.50, balance $177.50
  P-1 (Ravi): no change, balance $30.00
```

Explaining each part:

- **Encapsulation:** `balanceCents` is private; `deposit` / `withdraw` are `final` and enforce the rules. The second transfer ($900 from an account holding $400) is refused inside `withdraw`.

- **Abstraction:** `Bank.transfer` only uses `id()`, `withdraw()`, `deposit()`.
- **Inheritance:** each child calls `super(...)` and adds one or two fields.
- **Polymorphism:** `monthEnd()` calls `a.monthlyUpdate()`: three accounts, one method name, three results (interest 0.25% of $400 = $1.00; checking balance $180 is below the $500 waiver so $2.50 is charged; the plain account uses the parent's "no change"). **No `if` asks which is which.**
- **Why `long` cents and not `BigDecimal` here:** money must never be a `double` (binary floating point cannot store 0.10 exactly, so sums drift). A `long` count of whole **cents** is exact too, and keeps this example about the four pillars rather than about arithmetic. Its limit is that it cannot hold a fraction of a cent: `balanceCents() * monthlyRateBasisPoints / 10_000` uses integer division, which **truncates** (rounds toward zero). With 40,000 cents at 25 bp the result is exactly 100 cents, but for 40,001 cents the 0.0025-cent remainder would be dropped. When a domain needs fractional amounts or an explicit rounding rule (interest, tax, currency conversion), use `BigDecimal` (Java's exact decimal-number type) with a stated `RoundingMode`, as the `Money` value object (a small object defined only by its values, Section 2.12) in Section 2.13 does. Interest is in **basis points** (1 bp = 0.01%).

How the four fit together

Encapsulation and abstraction guard a class: they settle **what is hidden** and what outsiders may see. Inheritance and polymorphism let the design expand: they make **a new requirement show up as an added class** rather than as surgery on an existing one. Designs that apply all four tend to look alike: hidden state, a small public API (application programming interface: the set of methods other code is allowed to call), a parent for the common parts, and subclasses that each override one method.

Trade-offs

Benefit	Cost
Removes type switches; open to new types	Behaviour is spread over several classes; you must look at the runtime type to know what runs
Enables Strategy, State, Command and most patterns (named, reusable designs; Chapters 8–11)	A virtual call (one resolved at run time) is marginally slower than a direct call (almost never matters)

Common mistakes

- Long `if/else` or `switch` over a type: almost always where polymorphism belongs. **Pointing this out to the interviewer earns more credit than actually refactoring (restructuring without changing behaviour) it.**
- Using `instanceof` chains to decide behaviour (fine for occasional data inspection, e.g. pattern matching in Java 21, which tests a type and binds a variable in one step, but not as the main dispatch mechanism in an OO model).

Real-world examples

`List.add` behaves differently for `ArrayList` and `LinkedList`; `Runnable.run()` executed by any `Executor` (a Java object that runs tasks, often on background threads); Spring injecting (handing in from outside) any implementation of an interface; `Comparator.compare` in `Collections.sort`.

In the interview

30-second answer: "Polymorphism lets one call behave differently depending on the object receiving it. I code against the parent type or an interface, each subtype overrides the method, and the JVM picks the right one at run time. It removes type switches, so a new type is a new class, not an edit."

- *"Where would you use it in a parking lot?"* — Pricing (`PricingStrategy.feeCents`), payments (`Payment.pay`), spot selection.

Self-check

- **Q:** Which version of `monthlyUpdate` runs for `Account a = new SavingsAccount(...)`? **A:** `SavingsAccount`'s, chosen at run time.

TL;DR

Hold the parent type, call the method, let the object decide. Type switches are the signal that polymorphism is missing.

2.7 Compile-time vs runtime polymorphism: overloading vs overriding

What it is

	Overloading (compile-time / static polymorphism)	**Overriding (runtime / dynamic polymorphism)**
What	Same method name, **different parameter lists**, usually in the same class	Same signature re-implemented in a **subclass**
Decided by	The **compiler**, using the **declared (static) types** of the arguments	The **JVM at run time**, using the **actual class** of the receiver object
Return type	May differ (but cannot be the only difference)	Same or covariant
Needs inheritance?	No	Yes (class or interface)
Example	`add(int,int)`, `add(double,double)`	`Circle.area()` overriding `Shape.area()`

How it works: overload resolution

The compiler picks an overload in three phases and stops at the first phase that finds a match:

1. Exact match or **widening** (automatic conversion to a bigger or more general type) of primitives/references (`int` → `long` → `double`, `Circle` → `Shape`), no boxing, no varargs.
2. Allow **boxing/unboxing** (`int` ↔ `Integer`: wrapping a primitive value in its object form, and back).
3. Allow **varargs** (`int...`: a parameter that accepts any number of arguments). If several candidates remain in a phase, the **most specific** wins; if none is most specific, it is a compile error (ambiguous call).

Not polymorphic: fields (resolved by the static type; a child field with the same name **hides** the parent's), `static` methods (hidden, chosen by the static type), and `private` methods.

Java example

```java
// File: code/lld/ch02/polymorphism/Calculator.java
package lld.ch02.polymorphism;

/** OVERLOADING: same name, different parameter lists. Chosen by the COMPILER from static types. */
public class Calculator {
    public String add(int a, int b)       { return "add(int,int) = " + (a + b); }
    public String add(long a, long b)     { return "add(long,long) = " + (a + b); }
    public String add(double a, double b) { return "add(double,double) = " + (a + b); }
    public String add(int... xs)          { int s = 0; for (int x : xs) s += x; return "add(int...) = " + s; }

    public String greet(Shape s)  { return "greet(Shape)"; }
    public String greet(Circle c) { return "greet(Circle)"; }
}
```

```java
// File: code/lld/ch02/polymorphism/Circle.java
package lld.ch02.polymorphism;

public class Circle extends Shape {
    public String kind = "circle";                                  // HIDES Shape.kind, does not override it
    private final double r;

    public Circle(double r) {
        super();                                                    // implicit anyway; parent constructed first
        this.r = r;
    }

    @Override public double area() { return Math.PI * r * r; }  // OVERRIDE: same signature

    @Override public Circle copy() { return new Circle(r); }    // covariant return type is allowed

    public static String describe() { return "static Circle.describe"; }

    @Override public String who() { return super.who() + " (radius " + r + ")"; }  // super.method() reuses parent
code
}
```

```java
// File: code/lld/ch02/polymorphism/Shape.java
package lld.ch02.polymorphism;

public class Shape {
    public String kind = "shape";                                   // fields are NOT polymorphic

    public double area() { return 0; }

    public Shape copy() { return new Shape(); }

    public static String describe() { return "static Shape.describe"; }   // static: hidden, not overridden

    public String who() { return "I am a " + kind + " with area " + String.format("%.2f", area()); }
}
```

```java
// File: code/lld/ch02/polymorphism/Main.java
package lld.ch02.polymorphism;

import java.util.List;

public class Main {
    public static void main(String[] args) {
        Calculator calc = new Calculator();
        System.out.println(calc.add(2, 3));           // exact match int,int
        System.out.println(calc.add(2L, 3));          // widening to long,long
        System.out.println(calc.add(2.5, 3));         // widening to double,double
        System.out.println(calc.add(1, 2, 3));        // varargs, chosen last

        Shape s = new Circle(1.0);                    // static type Shape, runtime type Circle
        System.out.println("s.area() -> " + String.format("%.2f", s.area()) + "  (runtime override: dynamic
dispatch)");
        System.out.println("calc.greet(s) -> " + calc.greet(s) + "  (overload picked at compile time from static
type)");
        System.out.println("s.kind -> " + s.kind + "  (fields use the static type)");
        System.out.println("((Circle) s).kind -> " + ((Circle) s).kind);
        System.out.println("Shape.describe() -> " + Shape.describe() + ", Circle.describe() -> " +
Circle.describe());
        System.out.println("s.copy() class -> " + s.copy().getClass().getSimpleName());
        System.out.println(s.who());

        List<Shape> shapes = List.of(new Shape(), new Circle(2), new Circle(0.5));
        double total = 0;
        for (Shape sh : shapes) total += sh.area();     // one call site, many behaviours
        System.out.println("total area = " + String.format("%.2f", total));

        if (s instanceof Circle c) {                    // pattern matching instanceof (Java 16+)
            System.out.println("pattern-matched a circle: " + c.who());
        }
    }
}
```

Output:

```
add(int,int) = 5
add(long,long) = 5
add(double,double) = 5.5
add(int...) = 6
s.area() -> 3.14  (runtime override: dynamic dispatch)
calc.greet(s) -> greet(Shape)  (overload picked at compile time from static type)
s.kind -> shape  (fields use the static type)
((Circle) s).kind -> circle
Shape.describe() -> static Shape.describe, Circle.describe() -> static Circle.describe
s.copy() class -> Circle
I am a shape with area 3.14 (radius 1.0)
total area = 13.35
pattern-matched a circle: I am a shape with area 3.14 (radius 1.0)
```

Explaining each part:

- `add(2L, 3)`: no `(long,int)` overload, so `3` widens to `long` (phase 1).
- `s.area()` runs `Circle.area()` although `s` is declared `Shape`: **dynamic dispatch**.
- `calc.greet(s)` prints `greet(Shape)`, not `greet(Circle)`: overloads are chosen **at compile time from the declared type** of `s`. This surprises many people and is a favourite interview question.
- `s.kind` is `"shape"`: fields are not polymorphic. Inside `Shape.who()`, `kind` also means `Shape.kind`, which is why the output says "I am a shape". Lesson: **never hide fields**; use private fields plus overridable methods.
- `Circle copy()` overrides `Shape copy()` with a **covariant return type**.
- `super.who()` reuses the parent's text and appends the radius.
- `s instanceof Circle c` is **pattern matching for instanceof** (Java 16+): test and cast (treat the value as the more specific type) in one step.

Common mistakes

- Expecting overloads to be picked by runtime type (they are not; that would require double dispatch, choosing a method by the runtime types of two objects; see the Visitor pattern in Chapter 11).
- Writing `public boolean equals(Point p)`: an **overload**, not an override of `equals(Object)`, so collections (Java's built-in containers such as `List`, `Set` and `Map`) never call it. `@Override` would have caught it.

In the interview

> **30-second answer:** "Overloading is compile-time: same name, different parameters, chosen by declared types. Overriding is runtime: a subclass re-implements the same signature and the JVM dispatches on the actual object. Fields and static methods are not polymorphic."

- *"Why does `greet(s)` call `greet(Shape)`?"* — Overload resolution uses the static type of `s`, which is `Shape`.

Self-check

- **Q:** Can two methods differ only in return type? **A:** No; that is a compile error.
- **Q:** Which phase allows varargs? **A:** The third (last).

TL;DR

Overloading = compiler picks by declared types. Overriding = JVM picks by actual object. Fields and statics never dispatch dynamically.

2.8 Interfaces (default, static, private methods, functional interfaces)

What it is

An **interface** is a **contract**: a named list of methods a class promises to provide, without (necessarily) saying how. `class EmailNotifier implements Notifier`. Analogy: a wall socket standard. Any appliance with the right plug works with any socket, whoever made them.

Key facts:

- Abstract methods are implicitly `public abstract`; fields are implicitly `public static final` (constants).
- A class can **implement many interfaces** (but extend only one class).
- **Default methods** (Java 8+): a method with a body that implementers inherit and may override. Added so interfaces can evolve without breaking every implementer (for example `Collection.stream()`, `Iterable.forEach`).
- **Static methods** (Java 8+): belong to the interface; called as `Notifier.console()`; **not inherited** by implementing classes.
- **Private methods** (Java 9+): helpers shared by default methods, invisible to implementers.
- A **functional interface** has **exactly one abstract method** (methods inherited from `Object`, like `equals`, do not count; default and static methods do not count). It can be implemented by a **lambda** (a short unnamed function written inline, such as `(to, msg) -> ...`) or **method reference** (a pointer to an existing method, such as `String::isBlank`). `@FunctionalInterface` asks the compiler to enforce the rule.

The problem it solves

Callers depend on **what** can be done, not on a concrete class. New implementations can be added without touching callers (open/closed: open to extension, closed to modification), and tests can pass fakes, simple stand-in objects (dependency inversion: depend on interfaces, not concrete classes; both are covered in Chapter 4). Interfaces also give Java a safe form of multiple inheritance (one class inheriting from several parents; here of **type**, not state).

How it works

1. Define the interface with the smallest set of methods a caller needs.
2. Implement it in one or more classes (or with a lambda if it is functional).
3. Callers declare variables and parameters of the interface type.
4. If two interfaces supply the same default method, the class **must** override it and may pick one with `A.super.method()`. If a superclass declares the method, **the class wins** over interface defaults.

Java example

```java
// File: code/lld/ch02/interfaces/DiscountRule.java
package lld.ch02.interfaces;

/** A FUNCTIONAL interface has exactly one abstract method, so a lambda can implement it. */
@FunctionalInterface
public interface DiscountRule {
    long apply(long priceCents);

    default DiscountRule then(DiscountRule next) { return p -> next.apply(apply(p)); }  // defaults are allowed
}
```

```java
// File: code/lld/ch02/interfaces/EmailNotifier.java
package lld.ch02.interfaces;

public class EmailNotifier implements Notifier {
    @Override public String send(String to, String message) { return "[email] to " + to + ": " + message; }
}
```

```java
// File: code/lld/ch02/interfaces/Notifier.java
package lld.ch02.interfaces;

/** An interface is a contract: WHAT can be done, not HOW. */
public interface Notifier {
    String send(String to, String message);        // abstract method: every implementer must provide it

    /** default method: implementers inherit it and may override it (added in Java 8). */
    default String sendAll(java.util.List<String> recipients, String message) {
        StringBuilder sb = new StringBuilder();
        for (String r : recipients) sb.append(send(r, clean(message))).append('\n');
        return sb.toString().stripTrailing();
    }

    /** static method: belongs to the interface itself; called as Notifier.console(). */
    static Notifier console() { return (to, msg) -> "[console] " + to + ": " + msg; }

    /** private method (Java 9+): a helper shared by default methods, hidden from implementers. */
    private String clean(String message) { return message.strip(); }
}
```

```java
// File: code/lld/ch02/interfaces/Report.java
package lld.ch02.interfaces;

/**
 * An ABSTRACT class can hold state and constructors, and mix finished and unfinished methods.
 * Here it fixes the skeleton of an algorithm (Template Method) and lets subclasses fill the steps.
 */
public abstract class Report {
    private final String title;                    // state: interfaces cannot have instance fields

    protected Report(String title) { this.title = title; }

    public final String render() {                 // final: subclasses cannot change the skeleton
        return header() + "\n" + body() + "\n-- end of " + title + " --";
    }

    protected String header() { return "== " + title + " =="; }   // concrete, overridable
    protected abstract String body();                              // must be supplied
}
```

```java
// File: code/lld/ch02/interfaces/SalesReport.java
package lld.ch02.interfaces;

public class SalesReport extends Report {
    private final long totalCents;
    public SalesReport(long totalCents) { super("Sales"); this.totalCents = totalCents; }
    @Override protected String body() { return "total: " + totalCents + " cents"; }
}
```

```java
// File: code/lld/ch02/interfaces/Main.java
package lld.ch02.interfaces;

import java.util.List;
import java.util.function.Predicate;

public class Main {
    public static void main(String[] args) {
        Notifier email = new EmailNotifier();                 // class implementing the interface
        Notifier sms = (to, msg) -> "[sms] " + to + ": " + msg; // Notifier is functional too (one abstract
method)
        for (Notifier n : List.of(email, sms, Notifier.console())) {
            System.out.println(n.send("ada", "your order shipped"));
        }
        System.out.println(email.sendAll(List.of("kai", "ravi"), "  sale starts today  "));

        DiscountRule tenPercent = p -> p * 90 / 100;
        DiscountRule minusFiveDollars = p -> Math.max(0, p - 500);
        System.out.println("10% then -$5 on 10000c = " + tenPercent.then(minusFiveDollars).apply(10_000) + "c");

        Predicate<String> blank = String::isBlank;              // JDK functional interface + method reference
        System.out.println("'  ' blank? " + blank.test("  "));

        Report r = new SalesReport(123_45);
        System.out.println(r.render());
    }
}
```

Output:

```
[email] to ada: your order shipped
[sms] ada: your order shipped
[console] ada: your order shipped
[email] to kai: sale starts today
[email] to ravi: sale starts today
10% then -$5 on 10000c = 8500c
'  ' blank? true
== Sales ==
total: 12345 cents
-- end of Sales --
```

Explaining each part: `Notifier` has one abstract method, so it is also usable with lambdas (`sms`). `sendAll` is a default method that uses the private helper `clean`. `Notifier.console()` is a static factory on the interface. `DiscountRule` is an explicit `@FunctionalInterface` with a default combinator (a method that joins two functions into one) `then` (like `Function.andThen`). `Predicate<String>` is one of the JDK's built-in functional interfaces (`java.util.function`: `Function`, `Supplier`, `Consumer`, `Predicate`, `BiFunction`, `UnaryOperator`...). `Report`/`SalesReport` show an **abstract class** (Section 2.9).

Trade-offs

Benefit	Cost
Loose coupling (classes know little about each other), easy testing, multiple implementations	One more type to read; pointless if only one implementation will ever exist
Multiple inheritance of type	Cannot hold instance state
Default methods allow evolution	Overusing defaults turns interfaces into mini base classes

When not to use it

An interface with a single implementation and no realistic second one (and no test-double need, i.e. no fake version for tests) is indirection (an extra layer between caller and code) without benefit. Add it when the second implementation appears.

Common mistakes

- **Fat interfaces** that force implementers to leave methods empty (Interface Segregation, Chapter 4).
- "Constant interfaces" used only to hold constants (use a final class or enum).

Real-world examples

`Comparable`, `Comparator`, `Runnable`, `Callable`, `Iterable`, `AutoCloseable` (enables try-with-resources, a `try (...)` block that closes files or connections automatically), `List`/`Map`; Spring's `Repository` interfaces; SLF4J's (a common Java logging library) `Logger`.

In the interview

30-second answer: "An interface is a contract of methods. I make services depend on interfaces such as `PaymentGateway` or `PricingStrategy`, so I can add implementations without editing callers and test with fakes. Since Java 8 interfaces can have default and static methods, and an interface with one abstract method is functional, so a lambda can implement it."

- *"Why were default methods added?"* — To add methods to widely-implemented interfaces (like `Collection.stream`) without breaking existing implementations.
- *"Two interfaces with the same default method?"* — Compile error until the class overrides it; it can delegate with `X.super.m()`.

Self-check

- **Q:** Is `Comparator` functional although it has many methods? **A:** Yes; only `compare` is abstract (`equals` comes from `Object`, the rest are default/static).

TL;DR

Interfaces are contracts; depend on them. Defaults evolve them, statics provide factories/helpers, one abstract method = lambda-friendly.

2.9 Abstract classes vs interfaces

What it is

An **abstract class** is a class marked `abstract` that **cannot be instantiated**. It may contain fields, constructors, finished methods and **abstract methods** (declared without a body) that subclasses must implement. Analogy: a form with some fields pre-printed and some left blank for each department to fill in.

How it works

In the `interfaces` package above, `Report` holds state (`title`), has a constructor, a `final` skeleton method `render()`, a concrete overridable `header()`, and an abstract `body()`. `SalesReport` supplies only `body()`. This "fixed skeleton, variable steps" shape is the **Template Method** pattern (Chapter 11).

	Abstract class	**Interface**
Instantiable?	No	No
Instance fields (state)	Yes	No (only `public static final` constants)
Constructors	Yes	No
Method bodies	Any	`default`, `static`, `private` only
Access modifiers on members	Any	Abstract/default/static are public; helpers can be private
How many can a class extend/implement?	**One**	**Many**
Relationship expressed	"is a" (shared identity and state)	"can do" (capability / role)
Evolution	Add concrete methods freely	Add `default` methods
Typical use	Shared state + partial implementation for a close family	Contracts, roles, strategies, test seams (points where a test can swap in a fake)

Rule of thumb

Start with an **interface** for the contract. Add an **abstract class** underneath when several implementations share state or a fixed algorithm (JDK example: `List` interface + `AbstractList` skeleton + `ArrayList`). Callers still depend on the interface.

Common mistakes

- Using an abstract class just to share one helper method (use composition or a static helper).
- Making everything abstract "for flexibility".

In the interview

> **30-second answer:** "Interfaces define capabilities and allow multiple implementation; abstract classes share state and partial implementation but allow only one parent. I default to interfaces and add an abstract base class when implementations share real state or a template algorithm."

Self-check

- **Q:** Can an abstract class have no abstract methods? **A:** Yes; it just cannot be instantiated.
- **Q:** Can an interface have a constructor? **A:** No.

TL;DR

Interface = contract, many allowed, no state. Abstract class = partial implementation with state, one allowed. Interface first; abstract base class when sharing real code.

2.10 Enums with fields and behaviour

What it is

An **enum** (enumerated type) is a type with a **fixed set of named instances**, such as `Coin.PENNY` or `OrderStatus.SHIPPED`. In Java an enum is a full class: it can have **fields, constructors, methods, implement interfaces**, and even give each constant its own method body. Analogy: the fixed denominations of coins; you cannot invent a 7-cent coin.

The problem it solves

String or int constants (`"PAID"`, `2`) allow typos and invalid values and carry no behaviour. Enums are **type-safe** (only valid values compile), readable, and can hold the data and rules that belong to each value, which removes `switch` statements scattered around the code. In LLD, enums are the natural home for "**different data, not different behaviour**" (spot types, vehicle types, coin values, statuses).

How it works

1. Every enum implicitly extends `java.lang.Enum` and cannot extend anything else; it can implement interfaces.
2. Constructors are always private; constants are created once when the enum class is initialised.
3. Built-ins: `values()`, `valueOf(String)` (throws `IllegalArgumentException` for unknown names), `name()`, `ordinal()` (position, 0-based; do **not** persist it, i.e. save it to a database or file, since reordering changes it), `compareTo` (by ordinal).
4. Compare with `==` (each constant is a single instance).
5. Constant-specific bodies (`PAID { ... }`) let each constant override an abstract method: a tiny state machine (a fixed set of states plus the allowed moves between them) or strategy.
6. `EnumMap` / `EnumSet` are compact, fast collections keyed by enums.
7. `switch` expressions over an enum that cover every constant are **exhaustive**: no `default` needed, and adding a constant makes the compiler flag missing cases.
8. A single-element enum is the simplest thread-safe, serialization-safe (serialization turns an object into bytes to store or send) **Singleton** (a class allowed exactly one instance; Chapter 8).

Java example

```java
// File: code/lld/ch02/enums/Coin.java
package lld.ch02.enums;

/** An enum with a FIELD, a constructor and a method. Enum constructors are always private. */
public enum Coin {
    PENNY(1), NICKEL(5), DIME(10), QUARTER(25);

    private final int cents;

    Coin(int cents) { this.cents = cents; }

    public int cents() { return cents; }
}
```

```java
// File: code/lld/ch02/enums/OrderStatus.java
package lld.ch02.enums;

import java.util.Set;

/** An enum with BEHAVIOUR: each constant says which statuses it may move to (a tiny state machine). */
public enum OrderStatus {
    PLACED    { @Override Set<OrderStatus> next() { return Set.of(PAID, CANCELLED); } },
    PAID      { @Override Set<OrderStatus> next() { return Set.of(SHIPPED, CANCELLED); } },
    SHIPPED   { @Override Set<OrderStatus> next() { return Set.of(DELIVERED); } },
    DELIVERED { @Override Set<OrderStatus> next() { return Set.of(); } },
    CANCELLED { @Override Set<OrderStatus> next() { return Set.of(); } };

    abstract Set<OrderStatus> next();          // constant-specific method bodies

    public boolean canMoveTo(OrderStatus target) { return next().contains(target); }
    public boolean isFinal() { return next().isEmpty(); }
}
```

```java
// File: code/lld/ch02/enums/Main.java
package lld.ch02.enums;

import java.util.EnumMap;
import java.util.List;
import java.util.Map;

public class Main {
    public static void main(String[] args) {
        List<Coin> inserted = List.of(Coin.QUARTER, Coin.QUARTER, Coin.DIME, Coin.PENNY);
        int total = inserted.stream().mapToInt(Coin::cents).sum();
        System.out.println("inserted " + inserted + " = " + total + " cents");

        Map<Coin, Integer> counts = new EnumMap<>(Coin.class);   // compact, ordered by declaration
        for (Coin c : inserted) counts.merge(c, 1, Integer::sum);
        System.out.println("counts: " + counts);

        System.out.println("valueOf(\"DIME\") = " + Coin.valueOf("DIME") + ", ordinal " + Coin.DIME.ordinal());

        OrderStatus s = OrderStatus.PLACED;
        for (OrderStatus target : List.of(OrderStatus.SHIPPED, OrderStatus.PAID, OrderStatus.SHIPPED,
OrderStatus.DELIVERED)) {
            if (s.canMoveTo(target)) { s = target; System.out.println("moved to " + s); }
            else System.out.println("refused " + s + " -> " + target);
        }
        System.out.println(s + " is final? " + s.isFinal());

        String label = switch (s) {                          // exhaustive switch: no default needed
            case PLACED, PAID -> "in progress";
            case SHIPPED -> "on the way";
            case DELIVERED -> "done";
            case CANCELLED -> "stopped";
        };
        System.out.println("label: " + label);
    }
}
```

Output:

```
inserted [QUARTER, QUARTER, DIME, PENNY] = 61 cents
counts: {PENNY=1, DIME=1, QUARTER=2}
valueOf("DIME") = DIME, ordinal 2
refused PLACED -> SHIPPED
moved to PAID
moved to SHIPPED
moved to DELIVERED
DELIVERED is final? true
label: done
```

Explaining each part: `Coin` stores its value in a field, so summing is `Coin::cents`, with no switch. `EnumMap` prints in declaration order. `OrderStatus` gives each constant its own `next()` set, so illegal transitions (`PLACED → SHIPPED`) are refused in one method. This is the "enum plus a guarded method" alternative to the full State pattern that Chapter 1 recommends for small lifecycles.

Trade-offs

Benefit	Cost
Type-safe, self-documenting, can hold data/behaviour	Closed set: adding a value means recompiling
Great for small state machines and lookup tables	Large per-constant behaviour becomes hard to read; move to State/Strategy classes

When not to use it

When values are **open-ended or user-defined at run time** (product categories created by an admin) or when each value needs substantial, independently evolving behaviour (use classes).

Common mistakes

- Persisting `ordinal()` in a database (store `name()` or an explicit code field).
- Giant `switch` statements over an enum in many places instead of a method on the enum.

Real-world examples

`java.time.DayOfWeek` and `Month` (with methods like `plus`), `java.util.concurrent.TimeUnit` (each constant converts units: `TimeUnit.SECONDS.toMillis(5)`), `java.nio.file.StandardOpenOption`, `Thread.State`.

In the interview

> **30-second answer:** "I use enums for fixed sets like vehicle types, spot types and statuses. Java enums can carry fields and methods, so data like 'which spots fit this vehicle' or 'which statuses can follow this one' lives on the enum rather than in switches around the code."

- *"Enum or class hierarchy for vehicle types?"* — Enum if they differ only in data; classes if they behave differently.

Self-check

- **Q:** What does `Coin.valueOf("dime")` do? **A:** Throws `IllegalArgumentException` (names are case-sensitive).

TL;DR

Enums are type-safe fixed sets that can carry data and behaviour. Use them for "different data, same behaviour"; compare with `==`; never persist ordinals.

2.11 Composition over inheritance

What it is

Composition means building a class by **holding references to other objects** and delegating work to them ("has-a"), rather than inheriting from them ("is-a"). The principle **"favour composition over inheritance"** comes from the Gang of Four book and is a core idea in *Head First Design Patterns*. Analogy: a phone **has** a camera module; it is not a kind of camera. The module can be upgraded without redesigning the phone.

The problem it solves

Inheritance couples a child to the **internal implementation** of its parent (the **fragile base class problem**): a change in the parent, or a hidden detail of how it works, can silently break the child. Inheritance is also fixed at compile time and single-parent. Composition keeps classes independent, lets you **swap parts at run time**, and combine behaviours freely.

How it works

1. Identify the behaviour you want to reuse or vary.
2. Put it behind an interface (`Engine`, `Set`).
3. Hold it in a private field, received through the constructor (so it can be replaced or faked).
4. **Forward** (delegate) calls to it, adding your own logic around them.

Java example

```java
// File: code/lld/ch02/composition/Car.java
package lld.ch02.composition;

/** A car IS NOT an engine; it HAS an engine. The part can be swapped at run time. */
public class Car {
    private final String model;
    private Engine engine;

    public Car(String model, Engine engine) { this.model = model; this.engine = engine; }

    public void replaceEngine(Engine e) { this.engine = e; }

    public String drive() { return model + ": " + engine.start(); }
}
```

```java
// File: code/lld/ch02/composition/CountingSet.java
package lld.ch02.composition;

import java.util.Collection;
import java.util.Set;

/**
 * FIXED with composition: HAS-A Set and forwards to it. Works with ANY Set implementation
 * and does not care how that set implements addAll internally.
 */
public class CountingSet<E> {
    private final Set<E> inner;
    private int attempts = 0;

    public CountingSet(Set<E> inner) { this.inner = inner; }

    public boolean add(E e) { attempts++; return inner.add(e); }

    public boolean addAll(Collection<? extends E> c) {
        attempts += c.size();
        boolean changed = false;
        for (E e : c) changed |= inner.add(e);     // talk to the inner set, not to our own add()
        return changed;
    }

    public int size()     { return inner.size(); }
    public int attempts() { return attempts; }
}
```

```java
// File: code/lld/ch02/composition/Engine.java
package lld.ch02.composition;

public interface Engine {
    String start();
}
```

```java
// File: code/lld/ch02/composition/InheritingCountingSet.java
package lld.ch02.composition;

import java.io.Serial;
import java.util.Collection;
import java.util.HashSet;

/**
 * BROKEN on purpose: counts attempted additions by EXTENDING HashSet.
 * HashSet.addAll is inherited from AbstractCollection, which calls add() once per element,
 * so every element of addAll is counted twice. The subclass silently depends on how the parent
 * uses its own methods internally ("self-use"): the "fragile base class" problem.
 */
public class InheritingCountingSet<E> extends HashSet<E> {
    @Serial private static final long serialVersionUID = 1L;   // HashSet is Serializable, so subclasses should declare one

    private int attempts = 0;

    @Override public boolean add(E e) { attempts++; return super.add(e); }

    @Override public boolean addAll(Collection<? extends E> c) { attempts += c.size(); return super.addAll(c); }

    public int attempts() { return attempts; }
}
```

```java
// File: code/lld/ch02/composition/Main.java
package lld.ch02.composition;

import java.util.List;
import java.util.TreeSet;

public class Main {
    public static void main(String[] args) {
        InheritingCountingSet<String> bad = new InheritingCountingSet<>();
        bad.addAll(List.of("a", "b", "c"));
        System.out.println("inheritance: added 3, attempts counted = " + bad.attempts());

        CountingSet<String> good = new CountingSet<>(new TreeSet<>());
        good.addAll(List.of("a", "b", "c"));
        good.add("a");
        System.out.println("composition: 4 attempts counted = " + good.attempts() + ", size = " + good.size());

        Car car = new Car("City", () -> "petrol engine vroom");
        System.out.println(car.drive());
        car.replaceEngine(() -> "electric motor hum");
        System.out.println(car.drive());
    }
}
```

Output:

```
inheritance: added 3, attempts counted = 6
composition: 4 attempts counted = 4, size = 3
City: petrol engine vroom
City: electric motor hum
```

Explaining each part: `InheritingCountingSet` counts 6 for 3 elements because `HashSet` inherits `addAll` from `AbstractCollection`, which calls `add` once per element, and `add` is overridden to count again. The subclass depended on **how the parent uses its own methods internally** ("self-use"). `AbstractCollection` describes this in an implementation note, but `HashSet` does not promise to keep it, so a future version could change it and silently fix or break the count. `CountingSet` **has a** `Set`, forwards to it, and works with any `Set` implementation (here a `TreeSet`). `Car` has an `Engine`, and the engine is swapped at run time, which inheritance cannot do.

Trade-offs

	Inheritance	Composition
Relationship	is-a	has-a
Coupling	Tight (to parent internals)	Loose (to an interface)
Change at run time	No	Yes
Boilerplate (repetitive filler code)	Low	Forwarding methods
Polymorphism	Built in	Via the interface the parts implement

When not to use it

When there is a **genuine is-a relationship** designed for extension (e.g. extending an abstract template class meant to be subclassed), inheritance is fine and simpler.

Common mistakes

- Extending a concrete library class (`HashMap`, `ArrayList`) to add a feature.
- Composing but then exposing the inner object directly (`getInner()`), losing control again.

Real-world examples

`java.io.BufferedReader` wraps any `Reader` (Decorator, a wrapper that adds behaviour around an object = composition); `Collections.unmodifiableList` wraps a list; Spring beans (objects the framework creates) are composed via dependency injection (the framework hands each object the collaborators it needs); strategy objects such as `Comparator` passed into `TreeMap`.

In the interview

> **30-second answer:** "I prefer composition: a class holds an interface-typed collaborator and delegates to it. It avoids the fragile base class problem, lets me swap behaviour at run time, and most patterns such as Strategy, Decorator and State are built on it. I use inheritance only for true is-a relationships."

- *"Show a case where inheritance breaks."* — The counting set above: overriding `add` and `addAll` double counts because the parent's `addAll` calls `add`.

Self-check

- **Q:** Why does `CountingSet.addAll` call `inner.add`, not `this.add`? **A:** To avoid double counting and to depend only on the inner set's public contract.

TL;DR

Has-a + delegate beats is-a for reuse. Use inheritance only for real, designed-for-extension type families.

2.12 Object identity vs equality: the `equals` / `hashCode` contract

What it is

- **Identity:** are these two references pointing at the **same object**? Tested with `==`.
- **Equality:** do these two objects **represent the same value**? Tested with `equals`. By default `Object.equals` is identity; classes override it to mean value equality (`String`, `Integer`, `LocalDate`, records).

Analogy: two printed copies of the same book are **equal** (same content) but not **identical** (two physical objects).

The problem it solves

Hash-based collections (`HashMap`, `HashSet`) first use `hashCode()` to pick a bucket, then `equals()` to compare inside the bucket. If you get either wrong, objects "disappear" from sets and maps.

> **Key term — hash table:** A `HashMap` or `HashSet` keeps its entries in an array of slots called **buckets**. To store or find an object it computes the object's **hash code** (an `int` derived from its fields), uses that number to jump straight to one bucket, and only then compares the few objects in that bucket with `equals`. This is why lookups are fast, and why two equal objects must produce the same hash code: otherwise the search starts in the wrong bucket.

How it works: the contracts

`equals` must be:

1. **Reflexive:** `x.equals(x)` is true.
2. **Symmetric:** `x.equals(y)` ⇔ `y.equals(x)`.
3. **Transitive:** if `x.equals(y)` and `y.equals(z)` then `x.equals(z)`.
4. **Consistent:** repeated calls give the same answer if nothing used in the comparison changed.
5. **Non-null:** `x.equals(null)` is false.

`hashCode` must:

1. Return the same value on repeated calls in one run, if the fields used by `equals` do not change.
2. Return **equal hash codes for equal objects**.
3. Unequal objects **may** share a hash code (collisions are allowed, just slower).

So: whenever you override `equals`, override `hashCode` using the same fields. Records and IDE generators (the "generate equals/hashCode" command in a code editor) do this for you.

Java example

```java
// File: code/lld/ch02/equality/BrokenPoint.java
package lld.ch02.equality;

/**
 * WRONG on purpose: overrides equals but NOT hashCode, breaking the contract "equal objects -> equal hash codes".
 * javac -Xlint warns about exactly this bug ([overrides]); the warning is silenced ONLY because this class
 * exists to demonstrate the bug. In real code, treat that warning as an error and add hashCode().
 */
@SuppressWarnings("overrides")
public final class BrokenPoint {
    private final int x, y;

    public BrokenPoint(int x, int y) { this.x = x; this.y = y; }

    @Override
    public boolean equals(Object o) {
        return o instanceof BrokenPoint p && x == p.x && y == p.y;
    }
    // hashCode() inherited from Object: usually different for two distinct objects
}
```

```java
// File: code/lld/ch02/equality/Point.java
package lld.ch02.equality;

import java.util.Objects;

/** Correct value-equality: equals and hashCode are overridden TOGETHER, on the same fields. */
public final class Point {
    private final int x, y;

    public Point(int x, int y) { this.x = x; this.y = y; }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;                          // reflexive fast path
        if (!(o instanceof Point p)) return false;           // also handles null
        return x == p.x && y == p.y;
    }

    @Override public int hashCode() { return Objects.hash(x, y); }

    @Override public String toString() { return "(" + x + "," + y + ")"; }
}
```

```java
// File: code/lld/ch02/equality/Main.java
package lld.ch02.equality;

import java.util.HashSet;
import java.util.Set;

public class Main {
    public static void main(String[] args) {
        Point a = new Point(1, 2), b = new Point(1, 2);
        System.out.println("a == b (identity): " + (a == b));
        System.out.println("a.equals(b) (equality): " + a.equals(b));
        System.out.println("hash codes equal: " + (a.hashCode() == b.hashCode()));
        Set<Point> good = new HashSet<>();
        good.add(a); good.add(b);
        System.out.println("HashSet<Point> size after adding a and b: " + good.size() + ", contains (1,2)? " +
good.contains(new Point(1, 2)));

        BrokenPoint x = new BrokenPoint(1, 2), y = new BrokenPoint(1, 2);
        Set<BrokenPoint> bad = new HashSet<>();
        bad.add(x);
        System.out.println("BrokenPoint equals: " + x.equals(y) + ", but set.contains(equal one): " +
bad.contains(y));

        String s1 = "hi", s2 = new String("hi");
        System.out.println("\"hi\" == new String(\"hi\"): " + (s1 == s2) + ", equals: " + s1.equals(s2));

        Integer i1 = 1000, i2 = 1000;
        System.out.println("Integer 1000 equals: " + i1.equals(i2) + "  (never compare boxed numbers with ==)");
    }
}
```

Output:

```
a == b (identity): false
a.equals(b) (equality): true
hash codes equal: true
HashSet<Point> size after adding a and b: 1, contains (1,2)? true
BrokenPoint equals: true, but set.contains(equal one): false
"hi" == new String("hi"): false, equals: true
Integer 1000 equals: true  (never compare boxed numbers with ==)
```

Explaining each part: `Point` overrides both methods on the same fields, so the set treats `a` and `b` as one element. `BrokenPoint` overrides only `equals`; its inherited identity-based `hashCode` (almost always different for two distinct objects) sends an equal object to a different bucket, so `contains` fails. `javac -Xlint` (the compiler run with extra warnings switched on) even warns about this (`[overrides] ...`

`overrides equals, but neither it nor any superclass overrides hashCode`); `BrokenPoint` carries `@SuppressWarnings("overrides")` only because it exists to show the bug. In real code, treat that warning as an error. Strings and boxed numbers must be compared with `equals`: `Integer` caches (keeps ready-made, reused objects for) only small values (at least −128 to 127), so `==` on boxed numbers happens to work for small values and fails for large ones.

Trade-offs / design choice

Kind of class	Equality
Value object (Money, Point, Coordinate)	Value equality on all fields; ideally immutable
Entity (User, Order, ParkingSpot)	Identity or equality by a stable **ID** only

Common mistakes

- Overriding `equals` without `hashCode` (above).
- `equals(Point p)` instead of `equals(Object o)` (an overload, never called by collections).
- Using **mutable fields** in `hashCode` and then mutating an object that is already a key in a `HashMap` (it becomes unfindable).
- Using `getClass() !=` vs `instanceof` inconsistently across a hierarchy, breaking symmetry.

Real-world examples

`String`, `Integer`, `BigDecimal` (note: `new BigDecimal("1.0").equals(new BigDecimal("1.00"))` is false because scale (the number of decimal places) is part of equality; use `compareTo` for numeric comparison), `LocalDate`, `List.equals` (element-wise).

In the interview

> **30-second answer:** "`==` compares identity, `equals` compares logical value. If I override `equals` I must override `hashCode` on the same fields, because hash collections use the hash to find the bucket and `equals` inside it. For value objects I use records; for entities I compare by ID."

- *"What happens if only `equals` is overridden?"* — Equal objects can land in different buckets; `HashSet.contains` and `HashMap.get` fail.

Self-check

- **Q:** Can two unequal objects have the same hash code? **A:** Yes; that is a collision and is allowed.
- **Q:** Can two equal objects have different hash codes? **A:** No; that breaks the contract.

TL;DR

`==` = same object; `equals` = same value. Override `equals` and `hashCode` together, on the same (preferably immutable) fields.

2.13 Immutability and records

What it is

An **immutable** object cannot change after construction. "Changes" produce **new** objects. A **record** (Java 16+) is a concise way to declare an immutable **data carrier**: `record Order(String id, List<String> items, Money total) {}`. Analogy: a printed receipt. To correct it, you print a new receipt; you do not edit the old one.

The problem it solves

Mutable shared objects cause bugs when one part of the program changes something another part relies on, and they need locking when shared between threads. Immutable objects (all fields `final`, `this` not

leaked during construction) are **thread-safe without locks**, safe as `HashMap` keys and `HashSet` elements, easy to cache and share, and easy to reason about.

How it works

Hand-written immutable class (checklist):

1. Make the class `final` (or give it private constructors and static factories) so no subclass can add mutability.
2. Make all fields `private final`.
3. Provide no setters; methods that "modify" return a new object (`plus`, `withItem`).
4. **Defensive copies:** copy mutable inputs in the constructor and never hand out internal mutable objects (return copies or unmodifiable views).
5. Do not let `this` escape during construction.

Records generate: `private final` fields for each component, a canonical constructor, accessors named after the components (`id()`, not `getId()`), and `equals`, `hashCode`, `toString` based on all components. Records are implicitly `final`, extend `java.lang.Record`, cannot declare extra **instance** fields (static fields and extra methods are fine), and can implement interfaces. A **compact constructor** (`public Order { ... }`) validates or normalises the parameters before they are assigned.

Records are only shallowly immutable: if a component is a mutable `List`, the list can still change unless you copy it (`List.copyOf`).

Java example

```java
// File: code/lld/ch02/immutability/Money.java
package lld.ch02.immutability;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.Objects;

/**
 * A hand-written immutable VALUE OBJECT: final class, private final fields, no setters,
 * "changes" return NEW objects. The amount is a BigDecimal (exact decimal arithmetic, never double),
 * normalised to 2 decimal places so that 5 and 5.00 are the same Money.
 */
public final class Money {
    private final BigDecimal amount;
    private final String currency;

    private Money(BigDecimal amount, String currency) {
        this.currency = Objects.requireNonNull(currency);
        // UNNECESSARY = refuse (ArithmeticException) anything finer than a cent instead of silently rounding it
        this.amount = amount.setScale(2, RoundingMode.UNNECESSARY);
    }

    public static Money of(String amount, String currency) { return new Money(new BigDecimal(amount), currency); }

    public Money plus(Money other) {
        if (!currency.equals(other.currency)) throw new IllegalArgumentException("currency mismatch");
        return new Money(amount.add(other.amount), currency);   // new object; 'this' is untouched
    }

    public BigDecimal amount() { return amount; }

    @Override public boolean equals(Object o) {        // safe: the scale is always 2, so equals == numeric
equality
        return o instanceof Money m && amount.equals(m.amount) && currency.equals(m.currency);
    }
    @Override public int hashCode() { return Objects.hash(amount, currency); }
    @Override public String toString() { return amount.toPlainString() + " " + currency; }
}
```

```java
// File: code/lld/ch02/immutability/Order.java
package lld.ch02.immutability;

import java.util.List;

/**
 * A RECORD: the compiler generates the private final fields, canonical constructor, accessors,
 * equals, hashCode and toString. The compact constructor validates and makes a DEFENSIVE COPY,
 * because a record is only shallowly immutable.
 */
public record Order(String id, List<String> items, Money total) {
    public Order {
        if (id == null || id.isBlank()) throw new IllegalArgumentException("id required");
        items = List.copyOf(items);                // unmodifiable copy: caller's list cannot leak in
    }

    public Order withItem(String item, Money price) {  // "wither": returns a modified copy
        var more = new java.util.ArrayList<>(items);
        more.add(item);
        return new Order(id, more, total.plus(price));
    }
}
```

```java
// File: code/lld/ch02/immutability/Main.java
package lld.ch02.immutability;

import java.util.ArrayList;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        Money five = Money.of("5.00", "USD");
        Money seven = five.plus(Money.of("2.00", "USD"));
        System.out.println("five = " + five + ", seven = " + seven + "  (five unchanged)");
        System.out.println("Money.of(\"5\") equals Money.of(\"5.00\")? " + Money.of("5", "USD").equals(five) + "
(scale normalised)");

        List<String> callerList = new ArrayList<>(List.of("pen"));
        Order o1 = new Order("O-1", callerList, five);
        callerList.add("SNEAKY");                          // does not reach the record
        System.out.println("o1 = " + o1);

        Order o2 = o1.withItem("ink", Money.of("1.50", "USD"));
        System.out.println("o2 = " + o2);
        System.out.println("o1 still = " + o1.items() + ", equals copy? " + o1.equals(new Order("O-1",
List.of("pen"), Money.of("5.00", "USD"))));

        try {
            o1.items().add("hack");
        } catch (UnsupportedOperationException e) {
            System.out.println("items() is unmodifiable: " + e.getClass().getSimpleName());
        }
    }
}
```

Output:

```
five = 5.00 USD, seven = 7.00 USD  (five unchanged)
Money.of("5") equals Money.of("5.00")? true  (scale normalised)
o1 = Order[id=O-1, items=[pen], total=5.00 USD]
o2 = Order[id=O-1, items=[pen, ink], total=6.50 USD]
o1 still = [pen], equals copy? true
items() is unmodifiable: UnsupportedOperationException
```

Explaining each part: `Money.plus` returns a new object; `five` stays 5.00. The amount is a `BigDecimal` (exact decimal arithmetic; `double` cannot represent 0.10 exactly, so money in `double` drifts). The constructor fixes the **scale** (number of decimal places) at 2, because `BigDecimal.equals` compares scale too (`5` and `5.00` would otherwise be unequal, see Section 2.12); `RoundingMode.UNNECESSARY` makes it throw

instead of silently rounding away a fraction of a cent. The constructor is private and `Money.of(...)` is a **static factory**, one of the options in the checklist above. The record's compact constructor copies the caller's list with `List.copyOf`, so adding `"SNEAKY"` to the caller's list later has no effect, and `items()` is unmodifiable. `withItem` is a "wither" returning a changed copy. Two records with equal components are `equals`.

Trade-offs

Benefit	Cost
Thread-safe without locks; safe hash keys; no defensive reasoning needed	New object per change (usually cheap; can matter in hot loops, i.e. code that runs millions of times)
Records remove boilerplate	Records cannot hide their components or extend classes

When not to use it

Entities whose identity persists while state changes (a `ParkingSpot` becoming occupied, an `Account` balance) are naturally **mutable**, with encapsulated changes. Make them immutable only if you adopt an event-sourced (state rebuilt from a stored list of past events) or copy-on-write (every change makes a new copy) style deliberately.

Common mistakes

- Forgetting defensive copies (a "final" list field whose contents anyone can change).
- Assuming `final` makes the referenced object immutable (it only fixes the reference).
- Using records for entities that need controlled mutation.

Real-world examples

`String`, boxed primitives (`Integer`, `Long`), `BigDecimal`, `BigInteger`, all of `java.time` (`LocalDate`, `Instant`, `Duration`), `List.of`/`Map.of` collections; Kafka (a widely used message-streaming system) messages and event-sourcing events are treated as immutable facts.

In the interview

> **30-second answer:** "For value objects like Money, Coordinate or Ticket I use immutable records: final fields, no setters, validation in the compact constructor, and defensive copies of collections. They are thread-safe and safe as map keys. Entities whose state changes stay mutable but encapsulated."

- *"Is a record fully immutable?"* — Shallowly. Mutable components must be copied.

Self-check

- **Q:** What does `final List<String> items` guarantee? **A:** The reference never changes; the list contents still can.

TL;DR

Immutable = final class, private final fields, no setters, defensive copies. Records give you this (shallowly) in one line. Use for value objects; keep entities mutable but encapsulated.

2.14 Access modifiers and packages

What it is

A **package** is a named group of related classes (a namespace and a folder), e.g. `lld.ch02.access.core`. **Access modifiers** control who can see a class or member.

Modifier	Same class	Same package	Subclass in another package	Everyone
`private`	✔	✘	✘	✘
(none) package-private	✔	✔	✘	✘
`protected`	✔	✔	✔ (through inheritance)	✘
`public`	✔	✔	✔	✔

Top-level classes can only be `public` or package-private. Analogy: rooms in an office: your desk drawer (private), the team room (package), the company floor accessible to affiliated teams (protected), the lobby (public).

The problem it solves

Access modifiers make encapsulation **enforceable by the compiler**. Packages group classes by feature and let you hide implementation classes, exposing only a small public API.

How it works

1. Start everything as `private`; widen only when needed.
2. Group by **feature** (`parking`, `payment`) rather than by layer only.
3. Keep helper classes package-private; expose a small `public` facade (one simple front class that hides the others).
4. `protected` is for members designed for subclasses; note that it **also** grants access to the whole package.
5. Since Java 9, **modules** (`module-info.java`) add a further level: only exported packages are visible to other modules.

Java example

```java
// File: code/lld/ch02/access/client/PremiumWallet.java
package lld.ch02.access.client;

import lld.ch02.access.core.Wallet;

/** A subclass in ANOTHER package: may use protected members, not package-private or private ones. */
public class PremiumWallet extends Wallet {
    public PremiumWallet(String owner) { super(owner); }

    public void grantBonus(long cents) {
        bonusCents += cents;                    // OK: protected, accessed through inheritance
        // auditCount++;                        // would NOT compile: package-private in another package
        // balanceCents += cents;               // would NOT compile: private
    }

    @Override public String describe() {       // may WIDEN protected -> public when overriding
        return "[premium] " + super.describe();
    }
}
```

```java
// File: code/lld/ch02/access/client/Main.java
package lld.ch02.access.client;

import lld.ch02.access.core.Audits;
import lld.ch02.access.core.Wallet;

public class Main {
    public static void main(String[] args) {
        Wallet basic = new Wallet("Ada");
        basic.add(1_000);
        PremiumWallet premium = new PremiumWallet("Kai");
        premium.add(2_000);
        premium.add(500);
        premium.grantBonus(300);

        System.out.println("public field owner: " + basic.owner + ", public method balance: " +
basic.balanceCents());
        System.out.println(premium.describe());
        System.out.println("audit lines: " + Audits.all());
        System.out.println("audits for Kai: " + Audits.countFor(premium));
        // new lld.ch02.access.core.AuditLog();  // would NOT compile: class is package-private
    }
}
```

```java
// File: code/lld/ch02/access/core/AuditLog.java
package lld.ch02.access.core;

import java.util.ArrayList;
import java.util.List;

/** Package-private CLASS: an implementation detail of the core package. */
final class AuditLog {
    private static final List<String> LINES = new ArrayList<>();

    private AuditLog() { }

    static void record(Wallet w, String what) {
        w.auditCount++;                          // allowed: same package
        LINES.add(w.owner + ": " + what);
    }

    static List<String> lines() { return List.copyOf(LINES); }
}
```

```java
// File: code/lld/ch02/access/core/Audits.java
package lld.ch02.access.core;

import java.util.List;

/** The PUBLIC face of the package: exposes only what outsiders are allowed to read. */
public final class Audits {
    private Audits() { }
    public static List<String> all() { return AuditLog.lines(); }
    public static long countFor(Wallet w) { return w.auditCount; }
}
```

```java
// File: code/lld/ch02/access/core/Wallet.java
package lld.ch02.access.core;

/** Shows all four access levels on one class. */
public class Wallet {
    public final String owner;                // public: visible everywhere
    protected long bonusCents;                // protected: same package + subclasses anywhere
    long auditCount;                          // package-private (no keyword): same package only
    private long balanceCents;                // private: this class only

    public Wallet(String owner) { this.owner = owner; }

    public void add(long cents) {
        if (cents <= 0) throw new IllegalArgumentException("positive only");
        balanceCents += cents;
        AuditLog.record(this, "add " + cents);   // package-private helper class, invisible outside core
    }

    public long balanceCents() { return balanceCents; }

    protected String describe() { return owner + " has " + balanceCents + "c (+" + bonusCents + "c bonus)"; }
}
```

(`Main` is in `lld.ch02.access.client`; run `java -cp /tmp/lld_02 lld.ch02.access.client.Main`.)

Output:

```
public field owner: Ada, public method balance: 1000
[premium] Kai has 2500c (+300c bonus)
audit lines: [Ada: add 1000, Kai: add 2000, Kai: add 500]
audits for Kai: 2
```

Explaining each part: `AuditLog` is a package-private class and `auditCount` a package-private field, so only code in `core` touches them; outsiders read audit data only through the public `Audits` facade. `PremiumWallet` in another package can update `protected bonusCents` but the commented lines (package-private and private access) would not compile. The override of `describe()` widens `protected` to `public`, which overriding allows.

Common mistakes

- Making everything `public` "to be safe".
- Using `protected` fields as a shortcut instead of protected methods (subclasses then break invariants).
- Putting every class of a machine-coding solution in one package.

In the interview

30-second answer: "Fields are private by default, collaborators within a feature package can be package-private, protected is only for members designed for subclassing, and public is the deliberate API. I organise machine-coding solutions into packages such as model, repository, service and strategy."

Self-check

- **Q:** Can a class in another package, which is not a subclass, read a `protected` field? **A:** No.

TL;DR

private < package-private < protected < public. Default to private; expose a small public API per package.

2.15 Generics basics for LLD

What it is

Generics let a class, interface or method work with **any type while keeping compile-time type safety**. `List<String>` holds only strings; `Repository<String, User>` stores only users. The letters in angle brackets (`T`, `ID`, `E`, `K`, `V`) are **type parameters**. Analogy: a labelled storage box: "SHOES ONLY". The label is checked when you put things in, so you never need to check when you take them out.

The problem it solves

Without generics you store `Object` (the root type of every class) and cast on every read, discovering mistakes at run time (`ClassCastException`). With generics, mistakes are **compile errors**, and one class (`Repository<ID, T>`) serves every entity, which is extremely handy in machine-coding rounds.

How it works

1. **Generic class/interface:** `class Repository<ID, T extends Identifiable<ID>>`.
2. **Bounded type parameter:** `T extends X` means T must be X or a subtype, so you can call X's methods on T.
3. **Generic method:** `static <T extends Comparable<? super T>> T max(List<? extends T> items)`.
4. **Wildcards:** `? extends T` (read-only producer), `? super T` (write-capable consumer), `?` (unknown). Mnemonic **PECS**: *Producer Extends, Consumer Super* (from *Effective Java*, Joshua Bloch's well-known Java best-practice book).
5. **Diamond operator:** `new Repository<>()` infers the type arguments.
6. **Type erasure:** type arguments exist only at compile time; at run time `List<String>` and `List<Integer>` are the same class. Consequences: no `new T()`, no `instanceof List<String>`, no primitive type arguments (use `Integer`, not `int`).
7. **Avoid raw types** (`List` without `<...>`); they switch the checks off.

Java example

```java
// File: code/lld/ch02/generics/Identifiable.java
package lld.ch02.generics;

/** Generic interface: ID is a TYPE PARAMETER filled in by each implementer. */
public interface Identifiable<ID> {
    ID id();
}
```

```java
// File: code/lld/ch02/generics/Product.java
package lld.ch02.generics;

public record Product(Integer id, String title, long priceCents)
        implements Identifiable<Integer>, Comparable<Product> {
    @Override public int compareTo(Product o) { return Long.compare(priceCents, o.priceCents); }
}
```

```java
// File: code/lld/ch02/generics/Repository.java
package lld.ch02.generics;

import java.util.*;

/**
 * A generic in-memory repository, a very common helper in machine-coding rounds.
 * T is BOUNDED: it must be Identifiable by the same ID type.
 */
public class Repository<ID, T extends Identifiable<ID>> {
    private final Map<ID, T> store = new LinkedHashMap<>();

    public void save(T item)            { store.put(item.id(), item); }
    public Optional<T> findById(ID id)  { return Optional.ofNullable(store.get(id)); }
    public List<T> findAll()            { return List.copyOf(store.values()); }
}
```

```java
// File: code/lld/ch02/generics/User.java
package lld.ch02.generics;

public record User(String id, String name) implements Identifiable<String> { }
```

```java
// File: code/lld/ch02/generics/Main.java
package lld.ch02.generics;

import java.util.ArrayList;
import java.util.List;

public class Main {
    /** Generic METHOD with a bounded type parameter. */
    static <T extends Comparable<? super T>> T max(List<? extends T> items) {
        T best = items.get(0);
        for (T t : items) if (t.compareTo(best) > 0) best = t;
        return best;
    }

    /** PECS: Producer Extends (we read from src), Consumer Super (we write into dst). */
    static <T> void copy(List<? extends T> src, List<? super T> dst) {
        dst.addAll(src);
    }

    public static void main(String[] args) {
        Repository<String, User> users = new Repository<>();
        users.save(new User("u1", "Ada"));
        users.save(new User("u2", "Kai"));
        System.out.println("user u2: " + users.findById("u2").map(User::name).orElse("none"));
        System.out.println("user u9: " + users.findById("u9").map(User::name).orElse("none"));

        Repository<Integer, Product> products = new Repository<>();
        products.save(new Product(1, "pen", 150));
        products.save(new Product(2, "lamp", 2_500));
        products.save(new Product(3, "mug", 800));
        System.out.println("most expensive: " + max(products.findAll()).title());

        List<Integer> ints = List.of(3, 1, 2);
        List<Number> numbers = new ArrayList<>();
        copy(ints, numbers);                            // Integer list into a Number list
        System.out.println("copied into List<Number>: " + numbers + ", max int = " + max(ints));

        // users.save(new Product(4, "x", 1));          // would NOT compile: type safety at compile time
        List<String> a = new ArrayList<>();
        List<Integer> b = new ArrayList<>();
        System.out.println("same runtime class (type erasure)? " + (a.getClass() == b.getClass()));
    }
}
```

Output:

```
user u2: Kai
user u9: none
most expensive: lamp
copied into List<Number>: [3, 1, 2], max int = 3
same runtime class (type erasure)? true
```

Explaining each part: one `Repository` class stores `User`s keyed by `String` and `Product`s keyed by `Integer`; `findById` returns `Optional` (a box that either holds a value or is empty, used instead of `null`) for a normal "not found". `max` works for any comparable type (`Product` compares by price). `copy` accepts a `List<Integer>` as a producer and a `List<Number>` as a consumer (PECS). The final line demonstrates erasure.

Common mistakes

- Raw types and unchecked casts.
- Thinking `List<Integer>` is a subtype of `List<Number>` (it is not; use `List<? extends Number>`).
- Over-generic designs with five type parameters nobody can read.

Real-world examples

The entire Collections framework (`List<E>`, `Map<K,V>`), `Optional<T>`, `CompletableFuture<T>` (a result that will arrive later from background work), `Comparator<T>`, Spring Data's `CrudRepository<T, ID>` (the same idea as our `Repository`).

In the interview

30-second answer: "I use generics for reusable infrastructure like an in-memory `Repository<ID, T>` or an `EventBus<E>`, so type errors are caught at compile time. I know bounded types, wildcards with PECS, and that erasure removes type arguments at run time."

Self-check

- **Q:** Why can't you write `new T()`? **A:** Type erasure: the runtime does not know T; pass a `Supplier<T>` instead.

TL;DR

Generics = type-safe reuse. Bounds let you call methods on T; PECS for wildcards; erasure means no runtime type info.

2.16 Coupling and cohesion (the yardsticks)

- **Cohesion** — how closely the responsibilities **inside** one class belong together. **High cohesion is good**: every method works on the same data toward one purpose (the "one sentence, no 'and'" test from Chapter 1).
- **Coupling** — how much one class **depends on** others' details. **Low coupling is good**: classes talk through small interfaces, so a change in one does not ripple through many.

Every idea in this chapter pushes toward **high cohesion, low coupling**: encapsulation and access modifiers reduce coupling; single-responsibility classes raise cohesion; interfaces and composition decouple callers from implementations. Chapters 4–5 turn these yardsticks into named principles (SOLID, Law of Demeter, GRASP).

2.17 Object-oriented analysis and design (OOA / OOD / OOP)

What it is

Object-oriented analysis and design (OOAD) is a long-established, standard method for converting a written description of a system into objects plus the links between them. Most of the vocabulary used in an LLD interview originates here. The name covers two separate activities, analysis and design, followed by implementation; each asks a different question and produces a different result:

Activity	Question it answers	What exists at the end	Talks about
Analysis (OOA)	What does the system consist of, and what is it required to do?	The things in the problem, named in **business (domain) terms**	books, members, loans
Design (OOD)	Which objects, with which responsibilities, will provide that behaviour?	Classes, what each is responsible for, and how they relate	`Book`, `Member`, `Loan` and who holds whom
Implementation (OOP)	How do we express it in a programming language?	Running code	`ArrayList`, `HashMap`, `synchronized`

If you are talking about a `HashMap` two minutes in, you have leapt straight to implementation and **left out the two activities the interviewer wanted to watch**.

The problem it solves

Blending the activities is the classic time sink: if you pick operations before the objects are settled, operations end up on the wrong class, and moving them later is expensive.

How it works: the four steps

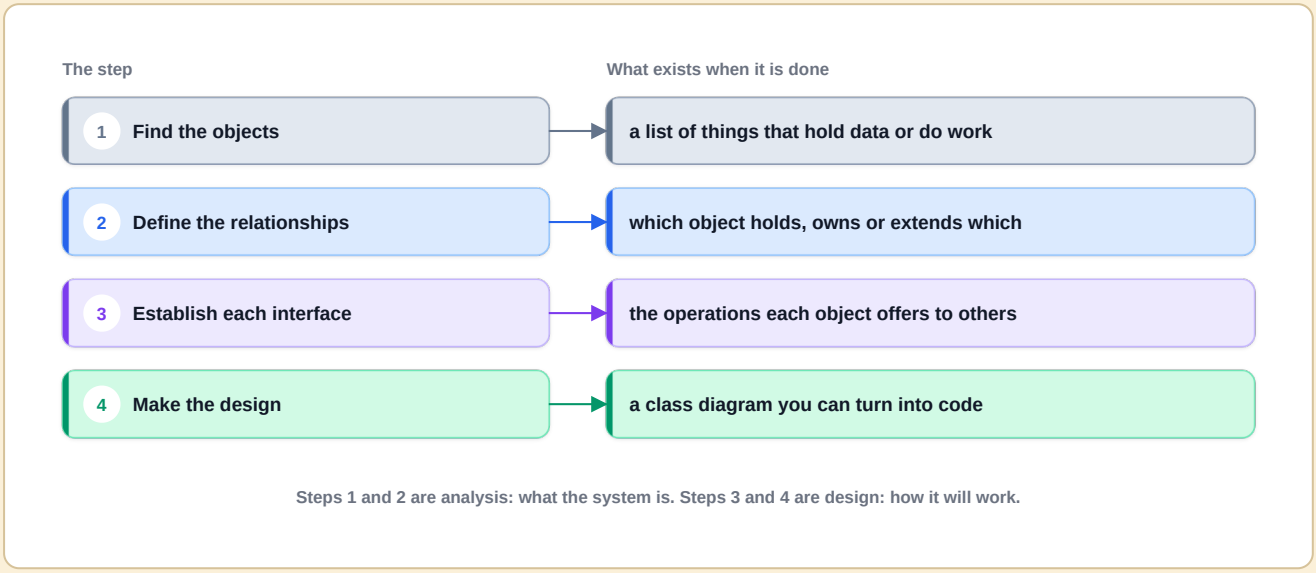


Figure — The four OOAD steps and what exists after each; steps 1–2 are analysis and steps 3–4 are design (course diagram)

1. **Find the objects.** Go through the description and note everything that **stores information or performs work**; most candidates are nouns. Anything that carries only a single value (a colour, a date) becomes a **field** of some object, not an object itself.
2. **Define the relationships.** For every pair of objects mentioned together, answer three questions: which one holds a reference to the other, does it **own** it, and **how many** of each are involved? (E.g. a library holds many books; a loan points at one book and one member.)
3. **Establish each interface.** Here *interface* means the set of operations an object makes available **to other objects**. Write that set down per object. Whatever is left off becomes **private**, so this step is where encapsulation really gets decided.

4. **Make the design.** Put classes, fields, operations and relationships into a single drawing (a UML class diagram) and verify it against the description you began with. That drawing is the design; the code is derived from it.

In the figure, steps 1–2 form the analysis (what the system is) and steps 3–4 form the design (how it will work). **The order is fixed because each step consumes the output of the one before.**

Worked example: "library members borrow books; each book on loan must be returned by a due date"

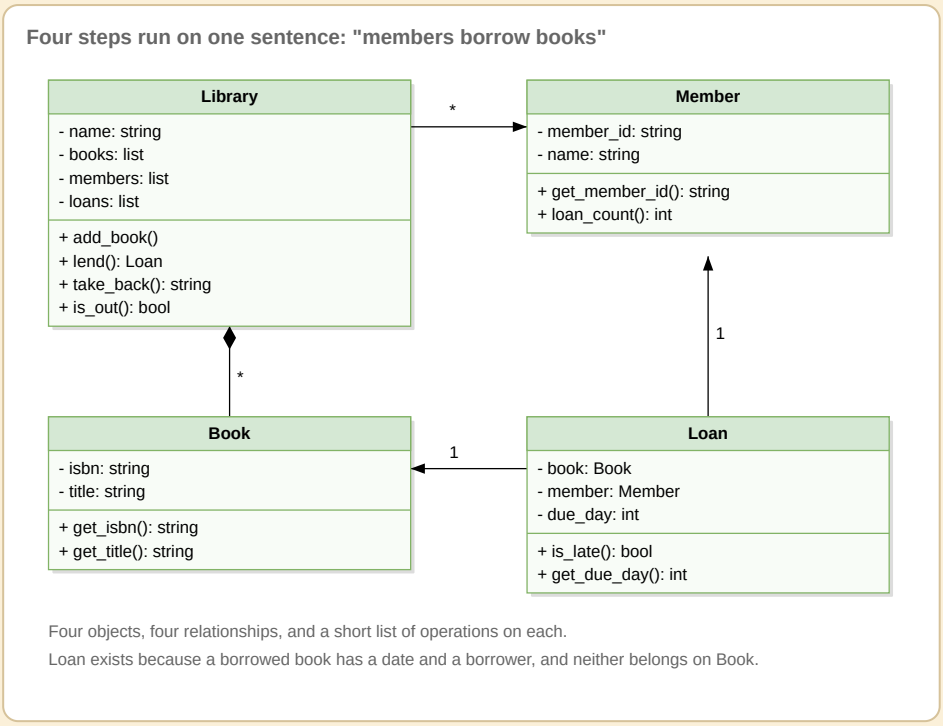


Figure — The four steps applied to one sentence: Library, Member, Book and the connection object Loan (course diagram)

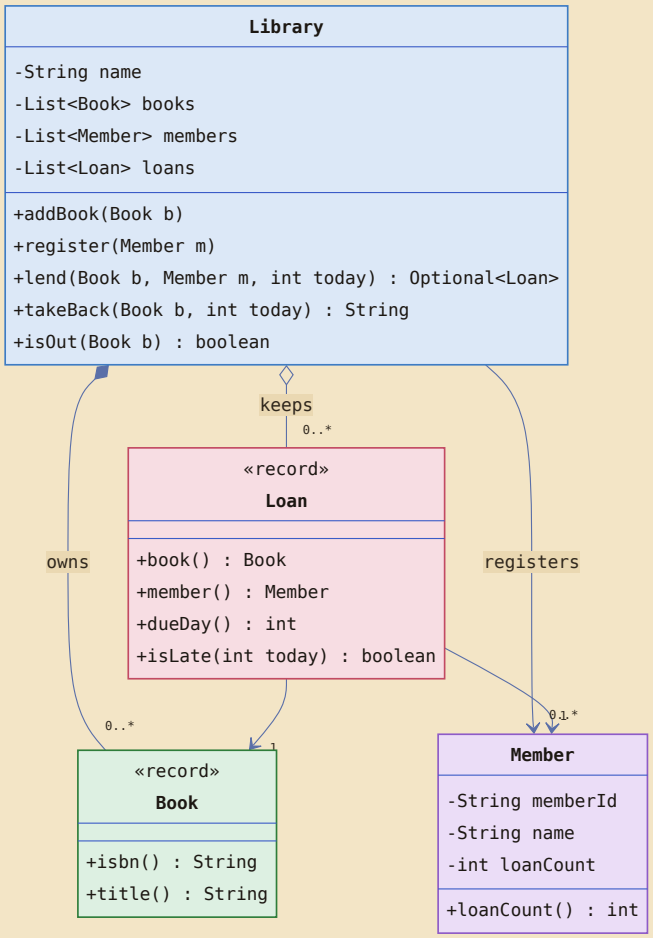
Step 1, objects. Candidate nouns: member, library, book, date, borrowed book. A *date* is one value, so it becomes a field. That leaves four objects: `Library`, `Member`, `Book`, and the borrowed book, which reads better as **`Loan`**. `Loan` is the one candidates overlook most often, since the sentence never uses it as a noun.

> **General rule:** if two objects are linked and **the link has data of its own**, model the link as an object. A lent book comes with someone who borrowed it and a date it must be back by; neither can live on `Book`, because a book sitting on the shelf has neither. Other examples: `Reservation` in a hotel, `Ticket` in a parking lot, `Enrollment` between student and course.

Step 2, relationships. The library **owns** its many book records in the strong sense (delete the library and its book records disappear with it): that is **composition**, drawn as the filled diamond. The library also **registers** many members without owning them (people go on existing without the library): a plain **association** arrow. Each loan **points to** a single book and a single member, owning neither.

Step 3, interfaces. Library: lend a book, accept a return, report whether a copy is currently out. Book: expose its title and ISBN (the International Standard Book Number printed on every book). Member: report its number of loans. Loan: report whether it is overdue. All other members, every field included, stay private. Pay attention to **which class answers "is this book out?"**: `Book` cannot, since it has no knowledge of loans; `Library` keeps the loans, so only it can answer.

Step 4, design. The figure fully answers the original sentence in four steps without a line of code. Our Java version (which adds a `register` operation so members can be checked):




```java
// File: code/lld/ch02/ooad/Book.java
package lld.ch02.ooad;

/** A book on the shelf: knows its ISBN and title, and nothing about who borrowed it. */
public record Book(String isbn, String title) { }
```

```java
// File: code/lld/ch02/ooad/Library.java
package lld.ch02.ooad;

import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

/** Owns its books (composition), refers to members (association), and keeps the loans. */
public final class Library {
    public static final int LOAN_DAYS = 14;
    public static final int MAX_LOANS_PER_MEMBER = 5;

    private final String name;
    private final List<Book> books = new ArrayList<>();
    private final List<Member> members = new ArrayList<>();
    private final List<Loan> loans = new ArrayList<>();

    public Library(String name) { this.name = name; }

    public void addBook(Book b)     { books.add(b); }
    public void register(Member m)  { members.add(m); }

    /** Only the library can answer this: a Book does not know about loans. */
    public boolean isOut(Book b) { return loans.stream().anyMatch(l -> l.book().equals(b)); }

    public Optional<Loan> lend(Book b, Member m, int today) {
        if (!books.contains(b) || !members.contains(m)) return Optional.empty();
        if (isOut(b) || m.loanCount() >= MAX_LOANS_PER_MEMBER) return Optional.empty();
        Loan loan = new Loan(b, m, today + LOAN_DAYS);
        loans.add(loan);
        m.loanStarted();
        return Optional.of(loan);
    }

    public String takeBack(Book b, int today) {
        for (Loan l : loans) {
            if (l.book().equals(b)) {
                loans.remove(l);
                l.member().loanEnded();
                return b.title() + " returned " + (l.isLate(today) ? "LATE" : "on time");
            }
        }
        return b.title() + " was not on loan";
    }

    public String name() { return name; }
}
```

```java
// File: code/lld/ch02/ooad/Loan.java
package lld.ch02.ooad;

/**
 * The CONNECTION OBJECT: "a borrowed book" carries its own data (borrower, due day),
 * which belongs neither on Book nor on Member.
 */
public record Loan(Book book, Member member, int dueDay) {
    public boolean isLate(int today) { return today > dueDay; }
}
```

```java
// File: code/lld/ch02/ooad/Member.java
package lld.ch02.ooad;

/** A member exists independently of the library (the library refers to it, it does not own it). */
public final class Member {
    private final String memberId;
    private final String name;
    private int loanCount;

    public Member(String memberId, String name) { this.memberId = memberId; this.name = name; }

    public String memberId() { return memberId; }
    public String name()     { return name; }
    public int loanCount()   { return loanCount; }

    void loanStarted() { loanCount++; }        // package-private: only Library changes it
    void loanEnded()   { loanCount--; }
}
```

```java
// File: code/lld/ch02/ooad/Main.java
package lld.ch02.ooad;

public class Main {
    public static void main(String[] args) {
        Library lib = new Library("City Library");
        Book ddd = new Book("978-0321125217", "Domain-Driven Design");
        Book gof = new Book("978-0201633610", "Design Patterns");
        lib.addBook(ddd);
        lib.addBook(gof);
        Member ada = new Member("M1", "Ada");
        Member kai = new Member("M2", "Kai");
        lib.register(ada);
        lib.register(kai);

        System.out.println("Ada borrows DDD on day 1: " + lib.lend(ddd, ada, 1).map(l -> "due day " +
l.dueDay()).orElse("refused"));
        System.out.println("Kai borrows DDD on day 2: " + lib.lend(ddd, kai, 2).map(l -> "due day " +
l.dueDay()).orElse("refused (already out)"));
        System.out.println("Kai borrows GoF on day 2: " + lib.lend(gof, kai, 2).map(l -> "due day " +
l.dueDay()).orElse("refused"));
        System.out.println("is DDD out? " + lib.isOut(ddd) + ", Ada's loans: " + ada.loanCount());
        System.out.println(lib.takeBack(ddd, 10));
        System.out.println(lib.takeBack(gof, 20));
        System.out.println(lib.takeBack(gof, 21));
        System.out.println("is DDD out? " + lib.isOut(ddd));
    }
}
```

Output:

```
Ada borrows DDD on day 1: due day 15
Kai borrows DDD on day 2: refused (already out)
Kai borrows GoF on day 2: due day 16
is DDD out? true, Ada's loans: 1
Domain-Driven Design returned on time
Design Patterns returned LATE
Design Patterns was not on loan
is DDD out? false
```

Explaining each part: limits from the requirements became constants (`LOAN_DAYS = 14`, `MAX_LOANS_PER_MEMBER = 5`). `Loan` is an immutable record holding the connection data. `Member.loanStarted/loanEnded` are package-private so only `Library` changes loan counts. `isOut` lives on `Library`, the only object with the information.

Why UML

A design only helps if another engineer can read it. **UML (Unified Modeling Language)** is the standard drawing notation for that; it displaced several rival notations decades ago because it can express exactly the OOAD ideas above: a **filled diamond** marks ownership (composition), a **plain arrow** marks one object referring to another, and **+ / −** mark public / private. No symbol is decorative; each records a design decision. That is also why the concepts are taught before the notation: a shape means nothing until you know the decision behind it. Chapter 3 teaches UML in full.

Three ways this goes wrong

1. **Naming classes after tables** (`UserRow`, `OrderTable`, `BookData`). That is a database schema (the layout of tables and columns) wearing OO clothes. An object pairs data with the behaviour that modifies it, so a class holding only fields and getters should prompt the question: what is it for, what does it **do**?
2. **Missing the connection object** (`Loan`, `Reservation`, `Ticket`). If `borrower` and `dueDate` are drifting onto `Book`, that is the signal that a connection class is missing.
3. **Listing every operation you can imagine.** Step 3 is about what an object provides **to other objects**, not every conceivable action. Long method lists on every class show that no decisions were made.

In the interview

30-second answer: "I separate analysis from design. First I find the objects from the nouns, dropping single values into fields, and define relationships with ownership and multiplicity. Then I decide each object's public operations, which also decides what is private, and draw the class diagram. When the link between two objects has data of its own, like a loan's due date, that link becomes a class."

- *"Where does `isOut(book)` belong?"* — On `Library`, because it owns the loans; a `Book` does not know about loans.
- *"Why is `date` not a class?"* — It holds a single value with no behaviour of its own in this problem; it is a field.

Self-check

- **Q:** Which OOAD steps are analysis? **A:** Find the objects, define the relationships.
- **Q:** What is the connection object in a parking lot? **A:** The parking ticket (vehicle + spot + entry time).
- **Q:** When is encapsulation actually decided? **A:** In step 3, when you list each object's public operations; everything unlisted is private.

TL;DR

Analysis = what the system is (objects, relationships). Design = how objects deliver it (interfaces, class diagram). Implementation = code. Connections with data become objects. Don't name a `HashMap` in minute two.

Chapter summary: in the interview

30-second answer (to "Explain the pillars of OOP"): "Encapsulation hides data and gathers the rules about it into the class that owns it. Abstraction exposes only what callers need. Inheritance lets a child reuse a parent when 'child is a parent' is truly correct. Polymorphism lets one call behave differently per object, which removes type switches. In design I prefer interfaces and composition over deep inheritance, use enums for fixed sets, records for immutable values, and I keep equals and hashCode consistent."

Common follow-ups:

- *"Abstract class or interface?"* — Interface for the contract (many allowed, no state); abstract class when implementations share state or a template algorithm.
- *"Overloading vs overriding?"* — Compile-time by declared parameter types vs run-time by actual object.
- *"Why composition over inheritance?"* — Avoids fragile base classes, allows run-time swapping, keeps coupling low.
- *"What breaks if `hashCode` is not overridden with `equals`?"* — Hash collections fail to find equal objects.
- *"How do you make a class immutable?"* — Final class, private final fields, no setters, defensive copies; or a record with a compact constructor that copies collections.

Self-check (whole chapter)

- **Q:** Class vs object? **A:** Blueprint vs one instance with real values.
- **Q:** What does `this(...)` do? **A:** Calls another constructor of the same class (constructor chaining).
- **Q:** Encapsulation vs abstraction? **A:** Hiding data behind rule-enforcing methods vs hiding complexity behind a minimal set of operations.
- **Q:** What is dynamic dispatch? **A:** The JVM choosing an overridden method's implementation from the object's actual class at run time.
- **Q:** Are static methods overridden? **A:** No, hidden.
- **Q:** Can an interface have a method with a body? **A:** Yes: default, static and private methods.
- **Q:** Where should "which spot types fit a truck" live? **A:** As data on the `VehicleType` enum.
- **Q:** State the hashCode rule in one line. **A:** Equal objects must have equal hash codes.
- **Q:** Are records deeply immutable? **A:** No, shallowly; copy mutable components.
- **Q:** Who can see a package-private member? **A:** Only classes in the same package.
- **Q:** What does PECS stand for? **A:** Producer `extends`, consumer `super`.
- **Q:** Name the OOAD steps. **A:** Find objects, define relationships, establish interfaces, make the design.

TL;DR

- OOP = data bundled with the only code allowed to modify it.
- Class = blueprint; object = instance. Constructors create valid objects; `this` = current object; `static` = one per class.
- Four pillars: encapsulation (who may change data), abstraction (what callers need), inheritance (true is-a only), polymorphism (one call, many behaviours; kills type switches).
- Overloading is compile-time; overriding is run-time; fields and statics are not polymorphic.
- Interfaces = contracts (default/static/private methods, functional interfaces for lambdas); abstract classes = shared state and templates.
- Enums hold data and behaviour for fixed sets; composition beats inheritance for reuse.
- `==` identity vs `equals` value; override `hashCode` with `equals`.
- Immutable value objects (records + defensive copies); mutable but encapsulated entities.
- Default to `private`; group by feature packages; generics for type-safe reusable infrastructure.
- OOA (what) → OOD (how, with objects) → OOP (code). Connections that carry data become objects.

Chapter 3 — Class Relationships & UML

> **What you will learn.** How two classes can be connected (association, aggregation, composition, dependency, generalization, realization), how each connection looks in Java, how to pick the right one in ten seconds, and how to *draw* a design with UML (Unified Modeling Language, the standard picture notation for software designs; Section 3.9): use-case, class, sequence, activity and state machine diagrams. The chapter ends with how much UML you should actually draw in a 45-minute interview.

How this chapter is organised

Part	Topics	Java package
A. Relationships	3.1 Big picture · 3.2 Association (uni/bi, multiplicity) · 3.3 Aggregation · 3.4 Composition · 3.5 Dependency · 3.6 Generalization · 3.7 Realization · 3.8 Decision guide	`lld.ch03.association`, `aggregation`, `composition`, `dependency`, `generalization`, `realization`
B. UML	3.9 What UML is · 3.10 Use-case · 3.11 Class · 3.12 Sequence · 3.13 Activity · 3.14 State machine	`lld.ch03.library` (use case → class → sequence → activity), `lld.ch03.statemachine`
C. Interview	3.15 How much UML to draw · 3.16 Chapter self-check · 3.17 TL;DR	—

Every package has a `Main` that you can run (`javac` is the Java compiler, which turns `.java` files into runnable `.class` files; `java` runs them, and `-cp` says where the compiled classes are):

```
cd chapters/03-relationships-and-uml/code
javac --release 21 -d /tmp/lld_03 $(find . -name '*.java')
java -cp /tmp/lld_03 lld.ch03.association.Main     # or aggregation, composition, dependency,
                                                   # generalization, realization, statemachine, library
```

Part A — Class Relationships

3.1 The big picture: what a "relationship" really claims

What it is

A **relationship** is a line between two classes on a class diagram. It says that objects of one class are connected to objects of the other. That is all a beginner usually hears. But the line actually makes two precise claims:

1. **A claim about lifetime.** If object A is destroyed, is object B destroyed too? ("Lifetime" = the time between an object being created and it no longer being used.)
2. **A claim about substitution.** Can an object of class B be used anywhere an object of class A is expected? ("Substitution" = handing a B to code that asked for an A and having everything still work.)

Everyday analogy: your **body** and your **heart** share a lifetime (composition). A **car** and its **driver** do not — the driver walks away after parking (association). A **sports car** *is a* car and can go anywhere a car can (generalization).

There are six relationships you must know. Three of them are "has-a", one is "uses-a", and two are "is-a":

Family	Relationship	One-line meaning	Coupling (rough order)
uses-a	**Dependency**	A uses B briefly (parameter, local variable, return type)	weakest
has-a	**Association**	A holds a reference to B and can call it; nobody owns anybody	↓
has-a	**Aggregation**	A is a whole holding parts B; the parts can outlive A	↓
has-a	**Composition**	A is a whole *owning* parts B; the parts die with A	↓
is-a	**Realization**	class A implements interface B	↓
is-a	**Generalization**	class A extends class B (inheritance)	strongest coupling

"Coupling" means how much one class must change when the other changes. Inheritance couples a subclass to every detail of its parent, which is why it is at the bottom of the list. The order is a rule of thumb for discussion, not something the UML standard defines.

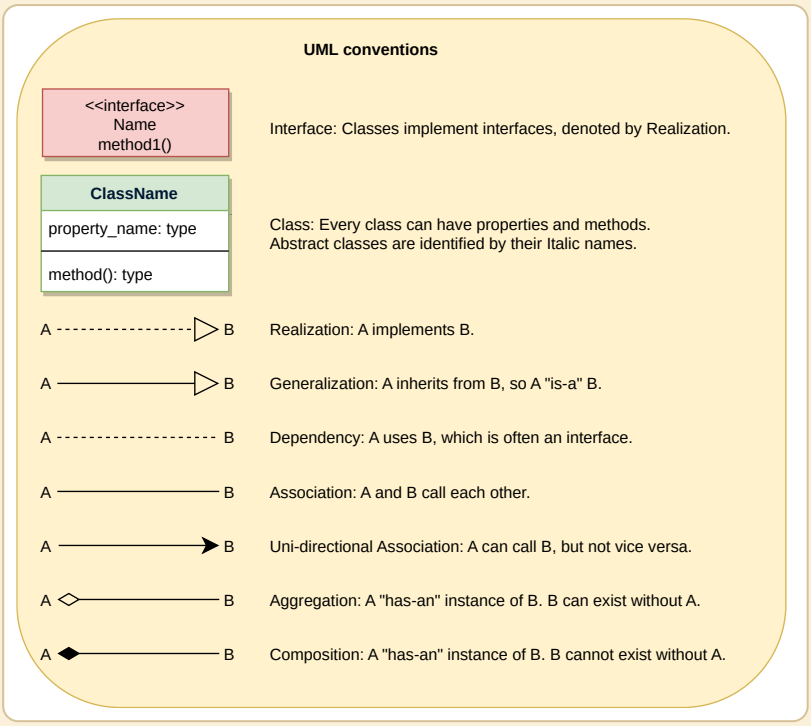


Figure — The UML conventions legend used throughout this book: class and interface boxes, and the line style for realization, generalization, dependency, association, uni-directional association, aggregation and composition (course diagram)

In the figure above, look at the *ends* of each line. The shape at the end (triangle, diamond, arrowhead) and whether the line is solid or dashed is the entire vocabulary. The rest of Part A goes through each line one at a time.

Two small differences between this legend and strict UML, so you are not confused later: the legend draws the **uni-directional association** with a *filled* arrowhead and the **dependency** as a dashed line with *no* head. Standard UML (and this book's Mermaid diagrams; Mermaid is a text-based diagram language that renders into pictures) uses an **open** arrowhead (`→`) for both — solid line for association, dashed line for dependency. Either drawing is understood in an interview if you say what you mean.

The problem it solves

Without a precise vocabulary, "Order has a Customer" and "ParkingFloor has ParkingSpots" sound the same. In code they are very different: one only points at an object that lives elsewhere, the other creates, owns and deletes its parts. If you mix them up you get bugs such as deleting data you did not own, or leaking objects nobody is responsible for. In an interview the mistake is visible on the whiteboard before a single line of code is written.

TL;DR

A relationship line is a claim about **who owns whom (lifetime)** and **who can stand in for whom (substitution)**. Learn the six lines and the one question that separates each.

3.2 Association (uni-directional, bi-directional, multiplicity)

What it is

An **association** means one class keeps a *reference* (a field) to another class and can call its methods over a long period — longer than one method call. Neither side owns the other. Destroy one and the other is unaffected.

Analogy: a **ticket** refers to a **parking spot**. Throw the ticket away and the spot is still in the building.

Three sub-ideas come with it:

- **Direction (navigability).** "Navigable" means "can be reached from". In a **uni-directional** association only one side has a field pointing at the other. In a **bi-directional** association both sides have fields pointing at each other.
- **Multiplicity.** How many objects take part at each end: `1`, `0..1` (optional), `*` or `0..*` (any number), `1..*` (at least one), `2` (exactly two), `2..4` (a range).
- **Role name / association name.** An optional label on the line such as `assigned to` or `borrower`.

The problem it solves

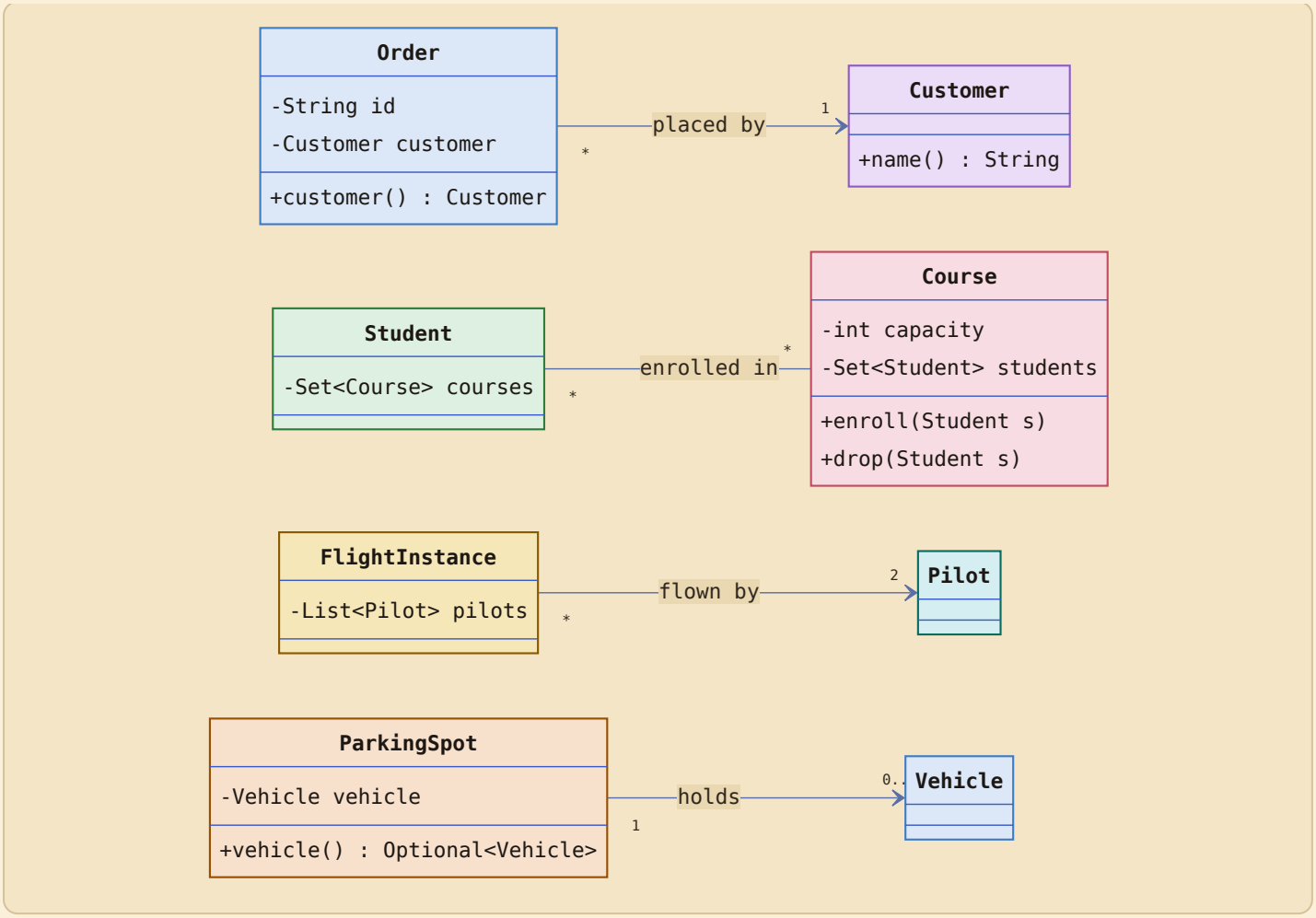
Objects need to collaborate over time. An `Order` must always know which `Customer` placed it. Association is the plain, honest way to record "A knows B" without claiming ownership. It is the most common relationship in a real design, and that is correct.

How it works

1. Decide who needs to call whom. If only `Order` ever asks for its customer, make it **uni-directional**: put a `customer` field in `Order` only.
2. Only make it **bi-directional** if *both* directions are needed by real use cases (e.g. "list a student's courses" *and* "list a course's students").
3. For a bi-directional link, choose **one side to own the update** and make it update both fields in one method, so the two ends can never disagree.
4. Write the **multiplicity** at each end of the line, and then **enforce it in code**:
    - `1` → the field is required: `Objects.requireNonNull` (throws an error if the value is `null`, i.e. no object) in the constructor.
    - `0..1` → the field may be empty: expose it as `Optional<T>` (a box that either holds one value or is empty, so callers must check).
    - `*` → a collection; `1..*` → a collection that must never be empty.
    - exact numbers like `2` → check the size in the constructor.

UML notation

- Plain solid line = association (UML reads a plain line as "navigability unspecified"; in interviews it is read as "both know each other").
- Solid line with an **open arrowhead** = uni-directional; the class at the tail knows the class at the head.
- An **×** at an end (rarely drawn) = explicitly not navigable in that direction.
- Numbers at each end = multiplicity. The number sits next to the class it counts. `Order "*" --> "1" Customer` reads "many orders, each with exactly one customer".



Java example

```java
// File: code/lld/ch03/association/Customer.java
package lld.ch03.association;

/** Knows nothing about orders: the association Order -> Customer is one-way. */
public record Customer(String name) { }
```

```java
// File: code/lld/ch03/association/Order.java
package lld.ch03.association;

import java.util.Objects;

/**
 * Uni-directional association, multiplicity "many orders to exactly 1 customer".
 * Order holds a reference to Customer; Customer has no field pointing back.
 */
public final class Order {
    private final String id;
    private final Customer customer;   // "1" end: never null

    public Order(String id, Customer customer) {
        this.id = id;
        this.customer = Objects.requireNonNull(customer, "an order needs exactly one customer");
    }

    public String id() { return id; }
    public Customer customer() { return customer; }
}
```

```java
// File: code/lld/ch03/association/Student.java
package lld.ch03.association;

import java.util.Collections;
import java.util.LinkedHashSet;
import java.util.Set;

/** One side of a bi-directional many-to-many association (Student * -- * Course). */
public final class Student {
    private final String name;
    private final Set<Course> courses = new LinkedHashSet<>();

    public Student(String name) { this.name = name; }

    public String name() { return name; }
    public Set<Course> courses() { return Collections.unmodifiableSet(courses); }

    /** Package-private: only Course.enroll/drop may touch this side, so both ends stay in sync. */
    void linkCourse(Course c)   { courses.add(c); }
    void unlinkCourse(Course c) { courses.remove(c); }

    @Override public String toString() { return name; }
}
```

```java
// File: code/lld/ch03/association/Course.java
package lld.ch03.association;

import java.util.Collections;
import java.util.LinkedHashSet;
import java.util.Set;

/**
 * Other side of the bi-directional association. Course is the "owner" of the link:
 * enroll/drop update BOTH ends in one place, which is the main rule for two-way links.
 * Multiplicity on this end: a course holds 0..capacity students.
 */
public final class Course {
    private final String code;
    private final int capacity;
    private final Set<Student> students = new LinkedHashSet<>();

    public Course(String code, int capacity) {
        this.code = code;
        this.capacity = capacity;
    }

    public void enroll(Student s) {
        if (students.contains(s)) return;
        if (students.size() == capacity) {
            throw new IllegalStateException(code + " is full (max " + capacity + ")");
        }
        students.add(s);
        s.linkCourse(this);
    }

    public void drop(Student s) {
        if (students.remove(s)) s.unlinkCourse(this);
    }

    public Set<Student> students() { return Collections.unmodifiableSet(students); }
    public String code() { return code; }

    @Override public String toString() { return code; }
}
```

```java
// File: code/lld/ch03/association/Pilot.java
package lld.ch03.association;

public record Pilot(String name) { }
```

```java
// File: code/lld/ch03/association/FlightInstance.java
package lld.ch03.association;

import java.util.List;

/** Multiplicity "exactly 2": the constructor enforces the number written on the diagram. */
public final class FlightInstance {
    private final String flightNo;
    private final List<Pilot> pilots;

    public FlightInstance(String flightNo, List<Pilot> pilots) {
        if (pilots.size() != 2) {
            throw new IllegalArgumentException("a flight needs exactly 2 pilots, got " + pilots.size());
        }
        this.flightNo = flightNo;
        this.pilots = List.copyOf(pilots);
    }

    public List<Pilot> pilots() { return pilots; }
    public String flightNo() { return flightNo; }
}
```

```java
// File: code/lld/ch03/association/Vehicle.java
package lld.ch03.association;

public record Vehicle(String plate) { }
```

```java
// File: code/lld/ch03/association/ParkingSpot.java
package lld.ch03.association;

import java.util.Optional;

/** Multiplicity "0..1": the field may be empty, so the getter returns Optional and callers must check. */
public final class ParkingSpot {
    private final int number;
    private Vehicle vehicle;   // null means "0" of 0..1

    public ParkingSpot(int number) { this.number = number; }

    public void park(Vehicle v) {
        if (vehicle != null) throw new IllegalStateException("spot " + number + " is taken");
        vehicle = v;
    }

    public void leave() { vehicle = null; }

    public Optional<Vehicle> vehicle() { return Optional.ofNullable(vehicle); }
    public int number() { return number; }
}
```

```java
// File: code/lld/ch03/association/Main.java
package lld.ch03.association;

import java.util.List;

public class Main {
    public static void main(String[] args) {
        System.out.println("== Uni-directional: Order -> Customer ==");
        Customer asha = new Customer("Asha");
        Order o1 = new Order("O-1", asha);
        Order o2 = new Order("O-2", asha);
        System.out.println(o1.id() + " belongs to " + o1.customer().name());
        System.out.println(o2.id() + " belongs to " + o2.customer().name());
        System.out.println("Customer has no way to list its orders (one-way link).");

        System.out.println("\n== Bi-directional many-to-many: Student <-> Course ==");
        Student ravi = new Student("Ravi");
        Student meera = new Student("Meera");
        Course java = new Course("JAVA101", 2);
        Course uml = new Course("UML201", 30);
        java.enroll(ravi);
        java.enroll(meera);
        uml.enroll(ravi);
        System.out.println("JAVA101 students: " + java.students());
        System.out.println("Ravi's courses:   " + ravi.courses());
        try {
            java.enroll(new Student("Kiran"));
        } catch (IllegalStateException e) {
            System.out.println("Multiplicity 0..2 enforced: " + e.getMessage());
        }
        java.drop(ravi);
        System.out.println("After drop, JAVA101: " + java.students() + ", Ravi: " + ravi.courses());

        System.out.println("\n== Multiplicity 2 and 0..1 ==");
        FlightInstance f = new FlightInstance("AI-202", List.of(new Pilot("Capt. Rao"), new Pilot("F/O Das")));
        System.out.println(f.flightNo() + " pilots: " + f.pilots());
        try {
            new FlightInstance("AI-303", List.of(new Pilot("Solo")));
        } catch (IllegalArgumentException e) {
            System.out.println("Rejected: " + e.getMessage());
        }
        ParkingSpot spot = new ParkingSpot(7);
        System.out.println("Spot 7 vehicle: " + spot.vehicle().map(Vehicle::plate).orElse("<empty>"));
        spot.park(new Vehicle("KA-01-1234"));
        System.out.println("Spot 7 vehicle: " + spot.vehicle().map(Vehicle::plate).orElse("<empty>"));
    }
}
```

What each part shows:

- `Order` has a `final Customer customer` field and `Customer` has nothing pointing back → **uni-directional**, multiplicity `1` enforced with `requireNonNull`.
- `Student` and `Course` each hold a `Set` of the other → **bi-directional many-to-many**. Only `Course.enroll/drop` may change the link; `Student.linkCourse` is package-private (no access keyword, so only classes in the same Java package can call it). This single "owning side" rule is what keeps both ends consistent.
- `Course(capacity)` turns multiplicity `0..capacity` into a runtime check.
- `FlightInstance` rejects anything but exactly **2** pilots; `ParkingSpot` exposes **0..1** as `Optional<Vehicle>`.
- Lifetimes: every object here is created by `Main` and merely *referenced* by the others. Dropping a course does not delete the student.

Output:

```
== Uni-directional: Order -> Customer ==
O-1 belongs to Asha
O-2 belongs to Asha
Customer has no way to list its orders (one-way link).

== Bi-directional many-to-many: Student <-> Course ==
JAVA101 students: [Ravi, Meera]
Ravi's courses:   [JAVA101, UML201]
Multiplicity 0..2 enforced: JAVA101 is full (max 2)
After drop, JAVA101: [Meera], Ravi: [UML201]

== Multiplicity 2 and 0..1 ==
AI-202 pilots: [Pilot[name=Capt. Rao], Pilot[name=F/O Das]]
Rejected: a flight needs exactly 2 pilots, got 1
Spot 7 vehicle: <empty>
Spot 7 vehicle: KA-01-1234
```

Trade-offs

Choice	Pros	Cons
Uni-directional	Simple; one field; no sync bugs; lower coupling	Reverse question ("which orders does this customer have?") needs a query or repository (a class that stores and looks up objects)
Bi-directional	Fast navigation both ways	Two fields to keep in sync; tighter coupling; risk of infinite loops in `toString` / `equals` /JSON serialisation (turning objects into JSON, a common text data format)
Strict multiplicity checks	Invalid states cannot be created	A little more constructor code

When not to use it

- Do not store a long-lived field if you only need the object inside one method — that is a **dependency** (3.5), which couples less.
- Do not default to bi-directional. Most associations should be uni-directional.

Common mistakes

- Making every line bi-directional "just in case".
- Updating only one side of a bi-directional link (`student.courses.add(c)` without `course.students.add(s)`).
- Forgetting multiplicity on the line, then forgetting the null-check / non-empty check in the code.
- Including both sides in `toString()` or `equals()` → stack overflow from infinite recursion (each object's method calls the other's, forever, until Java runs out of room to track the calls).

Real-world examples

- **JPA / Hibernate** (Java's standard API for saving objects into SQL database tables, and its most popular implementation): `@ManyToOne` on `Order.customer` is a uni-directional association. A bi-directional `@OneToMany(mappedBy = "customer")` needs an *owning side* (the side with the foreign key, the column that stores the other row's ID) and helper methods that set both ends — exactly the `Course.enroll` rule above.
- **Spring** (a popular Java application framework): a service with a constructor-injected repository (one handed in through the constructor) holds an association to it; the Spring container (the part of Spring that creates and wires objects), not the service, creates and destroys the repository.
- **E-commerce:** an order row stores `customer_id` (Order → Customer); a customer does not store a list of order ids.

In the interview

> **30-second answer:** "Association means one class holds a reference to another and can call it, with no ownership. I make it uni-directional unless both directions are needed, and I write multiplicity on every line because it changes the code: `1` is a required field, `0..1` is an Optional, `1..*` is a non-empty collection."

Follow-ups:

- *Q: How do you keep a bi-directional association consistent?* — A: Pick one owning side and put a single method there (`enroll`) that updates both ends; make the other side's mutators (methods that change state) package-private.
- *Q: What changes if a parking spot can hold two motorbikes?* — A: The multiplicity changes from `0..1` to `0..2`, so the field becomes a small collection with a size check instead of an `Optional`.
- *Q: Why not make everything bi-directional?* — A: It doubles the places that must change together and increases coupling; the reverse direction can usually be answered by a repository query.

Self-check

- *Q: `Order "*" --> "1" Customer` — which class has the field?* → A: `Order`; the arrow points at the class being referenced.
- *Q: What Java type fits multiplicity `0..1`?* → A: A nullable field exposed as `Optional<T>`.
- *Q: Is a many-to-many association always bi-directional?* → A: No. "Many-to-many" is about counts; direction is a separate choice.

TL;DR

Association = "A knows B", no ownership. Prefer one-way. Put multiplicity on every line and enforce it in the constructor/types.

3.3 Aggregation

What it is

Aggregation is a special association that says "A is a *whole* made of parts B, but the parts have their own life". The parts exist before the whole, can move to another whole, and survive when the whole is destroyed. It is sometimes called *shared aggregation* because a part may even belong to several wholes.

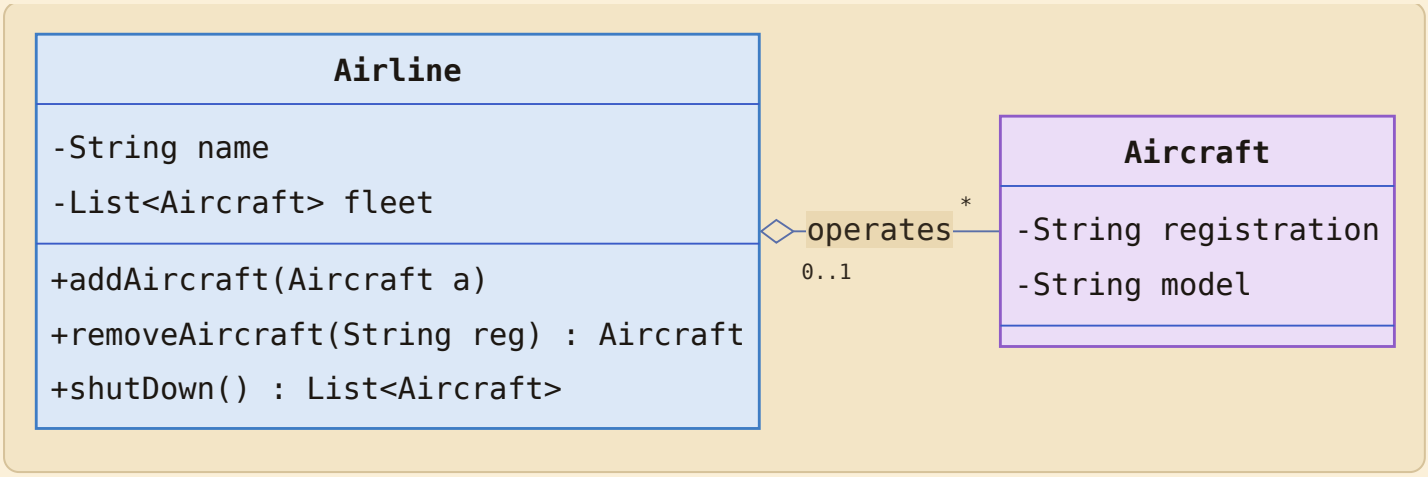
Analogy: an **airline** and its **aircraft**. If the airline shuts down, the planes are sold to another airline — they are not scrapped. A **team** and its **players** is another: disband the team and the players still exist.

The problem it solves

It lets you say "these objects form a group" (a whole–part meaning, stronger than a plain association) while making clear that the group does **not** control their lifetime. That tells readers: "do not delete the parts when you delete the whole".

How it works

1. The part is **created outside** the whole (by the caller, a factory, or a repository).
2. The part is **passed in** (`addAircraft(a)`), and the whole stores a reference.
3. Removing the part from the whole **returns it alive**; destroying the whole **releases** its parts.
4. In UML, draw a **hollow diamond at the whole's end** of the line.



Java example

```java
// File: code/lld/ch03/aggregation/Aircraft.java
package lld.ch03.aggregation;

/** A part that has its own life: it is created outside any airline and can move between airlines. */
public final class Aircraft {
    private final String registration;
    private final String model;

    public Aircraft(String registration, String model) {
        this.registration = registration;
        this.model = model;
    }

    public String registration() { return registration; }

    @Override public String toString() { return registration + " (" + model + ")"; }
}
```

```java
// File: code/lld/ch03/aggregation/Airline.java
package lld.ch03.aggregation;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

/**
 * Aggregation (hollow diamond at Airline): Airline 0..1 o-- * Aircraft
 * (an aircraft belongs to at most one airline at a time and may belong to none).
 * The airline HOLDS aircraft that were created elsewhere and handed in.
 * It never creates them and never destroys them.
 */
public final class Airline {
    private final String name;
    private final List<Aircraft> fleet = new ArrayList<>();

    public Airline(String name) { this.name = name; }

    public void addAircraft(Aircraft a) { fleet.add(a); }        // part is passed in

    public Aircraft removeAircraft(String registration) {         // part is handed back, still alive
        for (var it = fleet.iterator(); it.hasNext(); ) {
            Aircraft a = it.next();
            if (a.registration().equals(registration)) { it.remove(); return a; }
        }
        throw new IllegalArgumentException("no aircraft " + registration + " in " + name);
    }

    /** Shutting down the airline releases the parts instead of destroying them. */
    public List<Aircraft> shutDown() {
        List<Aircraft> released = List.copyOf(fleet);
        fleet.clear();
        return released;
    }

    public List<Aircraft> fleet() { return Collections.unmodifiableList(fleet); }
    public String name() { return name; }
}
```

```java
// File: code/lld/ch03/aggregation/Main.java
package lld.ch03.aggregation;

import java.util.List;

public class Main {
    public static void main(String[] args) {
        // The parts exist BEFORE any whole.
        Aircraft a1 = new Aircraft("VT-AAA", "A320");
        Aircraft a2 = new Aircraft("VT-BBB", "B737");
        Aircraft a3 = new Aircraft("VT-CCC", "ATR72");

        Airline skyways = new Airline("Skyways");
        skyways.addAircraft(a1);
        skyways.addAircraft(a2);
        Airline blueJet = new Airline("BlueJet");
        blueJet.addAircraft(a3);

        System.out.println(skyways.name() + " fleet: " + skyways.fleet());
        System.out.println(blueJet.name() + " fleet: " + blueJet.fleet());

        // Transfer one part between wholes.
        blueJet.addAircraft(skyways.removeAircraft("VT-BBB"));
        System.out.println("After transfer, BlueJet fleet: " + blueJet.fleet());

        // Destroy a whole: its parts survive and can join another whole.
        List<Aircraft> released = skyways.shutDown();
        skyways = null;   // the Airline object is now unreachable
        System.out.println("Skyways shut down. Released aircraft still exist: " + released);
        released.forEach(blueJet::addAircraft);
        System.out.println("BlueJet fleet now: " + blueJet.fleet());
    }
}
```

What each part shows: `Aircraft` objects are built in `Main` *before* any airline exists. `Airline.addAircraft` receives them; `removeAircraft` and `shutDown` hand them back. After `skyways = null` the airline is garbage (nothing points to it any more, so Java's garbage collector, which frees the memory of unreachable objects, will reclaim it), but its aircraft are still referenced and join `BlueJet`.

Output:

```
Skyways fleet: [VT-AAA (A320), VT-BBB (B737)]
BlueJet fleet: [VT-CCC (ATR72)]
After transfer, BlueJet fleet: [VT-CCC (ATR72), VT-BBB (B737)]
Skyways shut down. Released aircraft still exist: [VT-AAA (A320)]
BlueJet fleet now: [VT-CCC (ATR72), VT-BBB (B737), VT-AAA (A320)]
```

Trade-offs

	Aggregation	Plain association
Extra meaning	"whole–part" grouping	just "knows"
Lifetime claim	parts outlive whole	none
Code difference	usually **none** — both are a field holding a reference	—

The honest trade-off: aggregation adds meaning for the reader but almost nothing in code. The UML specification itself leaves the precise meaning of shared aggregation to the modeller, and several well-known UML authors (Martin Fowler's *UML Distilled* among them) advise using it sparingly. In interviews, it is still expected when there is a clear whole–part grouping.

When not to use it

- If there is no "whole made of parts" idea (a ticket and a spot), use plain association.
- If the parts are created by and die with the whole, it is composition.

Common mistakes

- Putting the diamond at the **part's** end. The diamond always sits at the **whole**.
- Choosing aggregation vs composition at random. Use the test: *imagine the whole is gone — is the part still meaningful by itself?* Yes → aggregation.
- Deleting parts in the whole's "close/delete" method even though the diagram says aggregation.

Real-world examples

- A `java.util.List<Customer>`: clearing the list does not destroy the customers; they are still referenced elsewhere.
- A Slack/Teams **channel** and its **members**: archiving a channel does not delete user accounts.
- A **playlist** and **songs** (Spotify): deleting your playlist does not delete the songs from the catalogue.
- A **department** and its **professors** in a university model.

In the interview

> **30-second answer:** "Aggregation is a whole–part association where the parts can outlive the whole — hollow diamond at the whole. An airline aggregates aircraft: shut the airline down and the aircraft still exist. In Java the parts are created outside and passed in, and the whole never destroys them."

Follow-ups:

- *Q: Can an airline exist without aircraft?* — A: That is a multiplicity question, not an aggregation question. If the business says an airline must have at least one aircraft, write `1..*` on the aircraft end; it is still aggregation because the aircraft survive the airline.
- *Q: How is aggregation different in code from association?* — A: Usually it is not; the difference is the modelling intent (whole–part). That is why you say the lifetime test out loud.

Self-check

- *Q: Where does the hollow diamond go?* → A: At the whole (`Airline`).
- *Q: A library closes; do its members disappear?* → A: No — a person exists either way, so Library ◇— Member is aggregation.

TL;DR

Aggregation = "has-a, parts live on". Parts are passed in, never destroyed by the whole. Hollow diamond at the whole.

3.4 Composition

What it is

Composition is the strongest has-a relationship: the whole **owns** its parts. A part belongs to **exactly one** whole at a time, is usually **created by** the whole, and is **destroyed with** it. (UML also allows a part to be explicitly removed from the whole *before* the whole is deleted.)

Analogy: a **house** and its **rooms**. Demolish the house and the rooms are gone — a "room of a house that does not exist" means nothing. Likewise a **parking floor** and its **spots**, a **parking lot** and its **floors**, a **flight** and its **weekly schedule**.

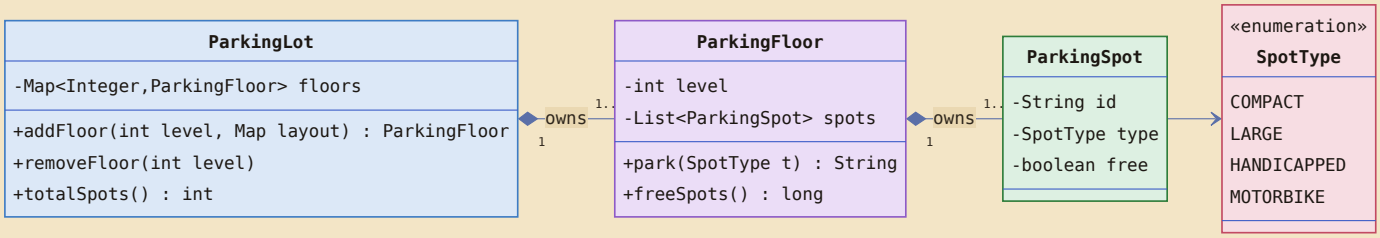
The problem it solves

Some objects make no sense on their own, and letting outsiders create or share them causes bugs (two floors pointing at the same spot, orphan spots after a floor is removed). Composition makes the owner responsible for the parts' full life cycle, so there is one place that creates, changes and removes them.

How it works

1. The whole **creates** the parts itself (in its constructor or an `addX(...)` method that takes *data*, not a ready-made part).
2. The part's constructor is **not public** (package-private or a private nested class, i.e. a class declared inside the whole's class) so nobody else can make one.
3. The whole **never hands out** its internal collection — only read-only views or copies.
4. **Removing** the whole (or removing a part from it) drops the only reference, so the part becomes garbage.
5. In UML, draw a **filled (black) diamond at the whole's end**.

A note on Java: Java has no destructors (methods that run automatically when an object is destroyed, as in C++). "The part dies with the whole" means *logically*: nobody else holds a reference, so when the whole is removed the part becomes unreachable and the garbage collector reclaims it. If you need clean-up (closing files, sockets, i.e. open network connections), the whole's `close()` must close its parts too — for example, closing a `BufferedReader` also closes the `FileReader` it wraps.



Java example

```java
// File: code/lld/ch03/composition/SpotType.java
package lld.ch03.composition;

/** A spot "kind" is a value, not a subclass (see the generalization chapter section). */
public enum SpotType { COMPACT, LARGE, HANDICAPPED, MOTORBIKE }
```

```java
// File: code/lld/ch03/composition/ParkingSpot.java
package lld.ch03.composition;

/**
 * A part of a composition. The constructor is package-private, so only ParkingFloor
 * (same package) can create a spot. Outside code can read a spot but never make one.
 */
public final class ParkingSpot {
    private final String id;
    private final SpotType type;
    private boolean free = true;

    ParkingSpot(String id, SpotType type) {
        this.id = id;
        this.type = type;
    }

    public String id() { return id; }
    public SpotType type() { return type; }
    public boolean isFree() { return free; }
    void occupy() { free = false; }

    @Override public String toString() { return id + ":" + type; }
}
```

```java
// File: code/lld/ch03/composition/ParkingFloor.java
package lld.ch03.composition;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.Map;

/**
 * Composition (filled diamond at ParkingFloor): ParkingFloor 1 *-- 1..* ParkingSpot.
 * The floor CREATES its spots in its own constructor, never shares them with another floor,
 * and when the floor is removed the spots go with it.
 */
public final class ParkingFloor {
    private final int level;
    private final List<ParkingSpot> spots = new ArrayList<>();

    ParkingFloor(int level, Map<SpotType, Integer> layout) {
        this.level = level;
        int n = 1;
        for (var e : layout.entrySet()) {
            for (int i = 0; i < e.getValue(); i++) {
                spots.add(new ParkingSpot("L" + level + "-" + n++, e.getKey()));
            }
        }
        if (spots.isEmpty()) throw new IllegalArgumentException("a floor needs 1..* spots");
    }

    public int level() { return level; }
    public List<ParkingSpot> spots() { return Collections.unmodifiableList(spots); }

    public long freeSpots() { return spots.stream().filter(ParkingSpot::isFree).count(); }

    /** Parks in the first free spot of the type; the spot itself never leaves the floor. */
    public String park(SpotType type) {
        for (ParkingSpot s : spots) {
            if (s.isFree() && s.type() == type) { s.occupy(); return s.id(); }
        }
        return null;
    }
}
```

```java
// File: code/lld/ch03/composition/ParkingLot.java
package lld.ch03.composition;

import java.util.Collections;
import java.util.Map;
import java.util.TreeMap;

/** Composition again: ParkingLot 1 *-- 1..* ParkingFloor. The lot builds and removes its floors. */
public final class ParkingLot {
    private final String name;
    private final Map<Integer, ParkingFloor> floors = new TreeMap<>();

    public ParkingLot(String name) { this.name = name; }

    /** The caller describes the floor; the lot constructs it. No ParkingFloor object is passed in. */
    public ParkingFloor addFloor(int level, Map<SpotType, Integer> layout) {
        ParkingFloor f = new ParkingFloor(level, layout);
        floors.put(level, f);
        return f;
    }

    /** Removing the whole removes its parts: no one else holds the spots, so they become garbage. */
    public void removeFloor(int level) { floors.remove(level); }

    public int totalSpots() {
        return floors.values().stream().mapToInt(f -> f.spots().size()).sum();
    }

    public Map<Integer, ParkingFloor> floors() { return Collections.unmodifiableMap(floors); }
    public String name() { return name; }
}
```

```java
// File: code/lld/ch03/composition/Main.java
package lld.ch03.composition;

import java.util.LinkedHashMap;
import java.util.Map;

public class Main {
    public static void main(String[] args) {
        ParkingLot lot = new ParkingLot("City Centre");
        Map<SpotType, Integer> layout = new LinkedHashMap<>();
        layout.put(SpotType.COMPACT, 3);
        layout.put(SpotType.LARGE, 1);

        ParkingFloor ground = lot.addFloor(0, layout);
        lot.addFloor(1, Map.of(SpotType.MOTORBIKE, 4));

        System.out.println("Floor 0 spots: " + ground.spots());
        System.out.println("Total spots in lot: " + lot.totalSpots());
        System.out.println("Parked compact car at " + ground.park(SpotType.COMPACT));
        System.out.println("Free spots on floor 0: " + ground.freeSpots());

        try {
            ground.spots().clear();
        } catch (UnsupportedOperationException e) {
            System.out.println("Outsiders cannot change the parts: spots() is read-only");
        }

        lot.removeFloor(1);
        System.out.println("After removing floor 1, total spots: " + lot.totalSpots()
                + " (its 4 spots are gone with it)");
        System.out.println("Floors left: " + lot.floors().keySet());
    }
}
```

What each part shows: `ParkingLot.addFloor` receives a *layout description* and builds the `ParkingFloor` itself. `ParkingFloor`'s constructor builds its `ParkingSpot`s; `ParkingSpot`'s constructor is package-

private, so code outside the package cannot create a spot. `spots()` returns an unmodifiable view, so outsiders cannot add or remove parts. `removeFloor(1)` drops the floor and, with it, its four spots.

Output:

```
Floor 0 spots: [L0-1:COMPACT, L0-2:COMPACT, L0-3:COMPACT, L0-4:LARGE]
Total spots in lot: 8
Parked compact car at L0-1
Free spots on floor 0: 3
Outsiders cannot change the parts: spots() is read-only
After removing floor 1, total spots: 4 (its 4 spots are gone with it)
Floors left: [0]
```

Trade-offs

	Composition	Aggregation
Who creates the part	the whole	someone else
Can the part be shared?	no (one owner at a time)	yes
Part outlives whole?	no	yes
Encapsulation	strong — whole controls all changes	weaker
Flexibility	lower — parts cannot be reused elsewhere	higher

When not to use it

- When the part has an independent identity the business cares about (a *person*, an *aircraft*, a *song*) — use aggregation/association.
- When the part must be shared between wholes.

Common mistakes

- Accepting a ready-made part from outside (`addSpot(ParkingSpot s)`) and calling it composition — the caller may still hold and share it.
- Returning the internal mutable list (`return spots;`) — outsiders can now add/remove parts behind the owner's back.
- Mixing up composition (object design) with "favour composition over inheritance" (3.6): in that slogan "composition" means *holding* an object instead of extending its class, and it covers association/aggregation too.

Real-world examples

- `java.util.HashMap` creates and manages its internal `Node` entries (a package-private nested class); they are never shared with another map and go away with it.
- **JPA:** `@OneToMany(cascade = CascadeType.ALL, orphanRemoval = true)` models composition — delete the `Order` and its `OrderLine` rows are deleted too.
- **Documents:** deleting a Google Doc deletes its comments and revision history.
- **Operating systems:** killing a process (a running program) frees its threads (its parallel lines of execution) and memory pages (the blocks of memory the OS gave it).

In the interview

30-second answer: "Composition means the whole owns the parts: it creates them, they belong to only it, and they die with it — filled diamond at the whole. A parking floor composes its spots: remove the floor and the spots are gone. In Java I build the parts inside the owner, hide their constructor, and only expose read-only views."

Follow-ups:

- *Q: Java has garbage collection — how can a part 'die'?* — A: It dies logically: the whole holds the only reference, so removing the whole makes the part unreachable. If the part holds resources, the whole's `close()` closes it.
- *Q: Is `Flight` → `WeeklySchedule` composition?* — A: Yes; a schedule of a flight that no longer exists is meaningless.

Self-check

- *Q: The test that separates aggregation from composition?* → A: Imagine the whole is gone — is the part still meaningful by itself? No → composition.
- *Q: Why is `ParkingSpot`'s constructor package-private?* → A: So only the owning floor can create spots.

TL;DR

Composition = "owns, parts die with it". Whole creates parts, never shares them, exposes read-only views. Filled diamond at the whole.

3.5 Dependency

What it is

A **dependency** means class A *uses* class B for a short time — inside one method — without keeping a field that points at B. B appears as a **method parameter**, a **local variable**, a **return type**, or a **static method call**. When the method returns, the relationship is over.

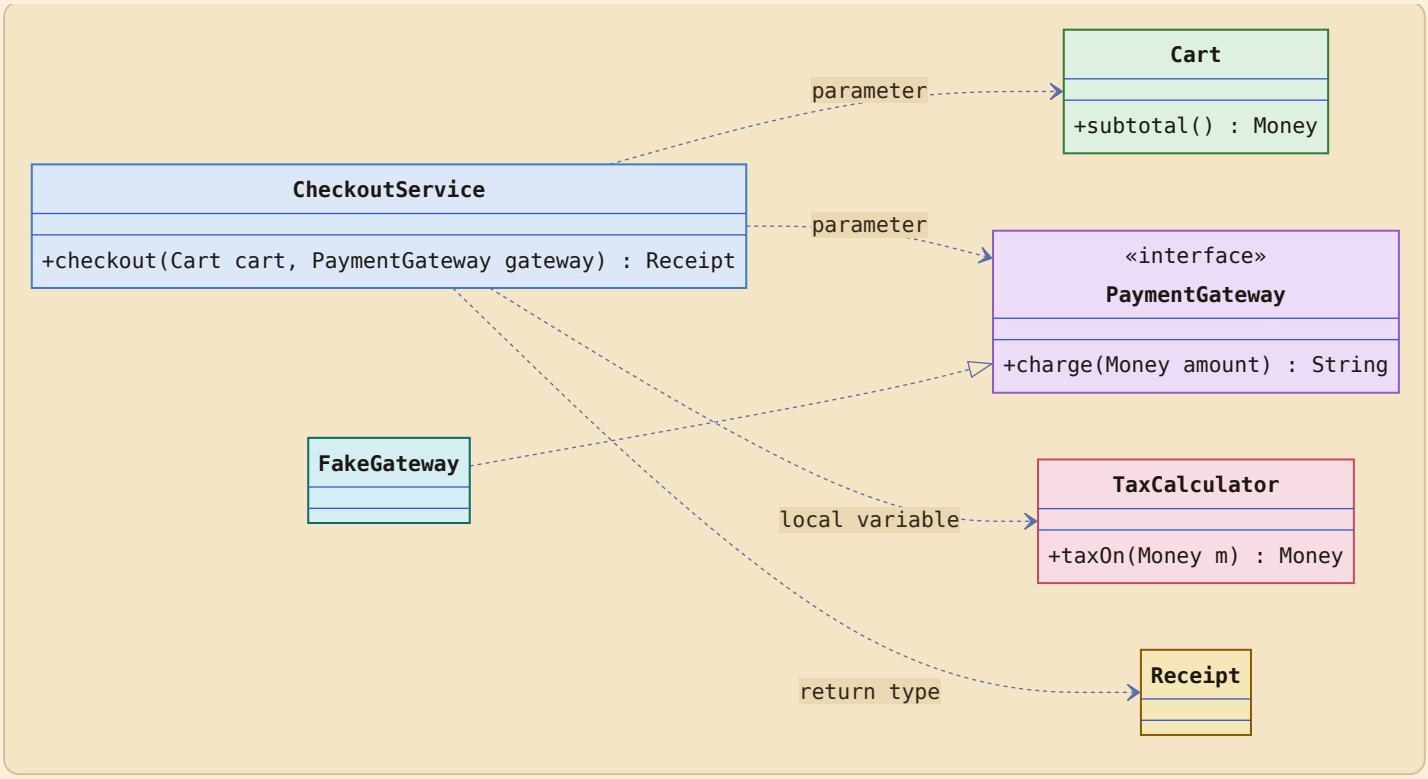
Analogy: a **chef** *uses* a **delivery van** to receive groceries this morning. The chef does not own the van and does not keep it in the kitchen; tomorrow it may be a different van.

The problem it solves

It records the weakest possible link: "if B's interface changes, A may have to change", nothing more. Recognising a dependency (instead of an association) keeps classes stateless (holding no data between calls) and loosely coupled — the caller can pass in a different collaborator on every call.

How it works

1. A method receives B as a parameter (`checkout(Cart cart, PaymentGateway gateway)`), or
2. creates B as a local variable (`new TaxCalculator()`), or
3. returns a B (`Receipt`), or calls a static method on B.
4. No field of type B exists in A.
5. In UML, draw a **dashed line with an open arrowhead** from the user (A) to the used class (B). You may add a keyword such as `«use»` or `«create»`.



Java example

```java
// File: code/lld/ch03/dependency/Money.java
package lld.ch03.dependency;

import java.math.BigDecimal;
import java.math.RoundingMode;

/** A small value type. Anything that returns or accepts Money depends on it. */
public record Money(BigDecimal amount) {
    public static Money of(String s) { return new Money(new BigDecimal(s).setScale(2, RoundingMode.HALF_UP)); }
    public Money plus(Money o) { return new Money(amount.add(o.amount)); }
    public Money times(BigDecimal f) { return new Money(amount.multiply(f).setScale(2, RoundingMode.HALF_UP)); }
    @Override public String toString() { return "Rs " + amount.toPlainString(); }
}
```

```java
// File: code/lld/ch03/dependency/Cart.java
package lld.ch03.dependency;

import java.util.List;

public record Cart(List<Money> itemPrices) {
    public Money subtotal() {
        return itemPrices.stream().reduce(Money.of("0"), Money::plus);
    }
}
```

```java
// File: code/lld/ch03/dependency/PaymentGateway.java
package lld.ch03.dependency;

public interface PaymentGateway {
    String charge(Money amount);   // returns a transaction id
}
```

```java
// File: code/lld/ch03/dependency/FakeGateway.java
package lld.ch03.dependency;

public final class FakeGateway implements PaymentGateway {
    private int next = 1;
    @Override public String charge(Money amount) {
        System.out.println("  [gateway] charging " + amount);
        return "TXN-" + next++;
    }
}
```

```java
// File: code/lld/ch03/dependency/TaxCalculator.java
package lld.ch03.dependency;

import java.math.BigDecimal;

public final class TaxCalculator {
    private static final BigDecimal GST = new BigDecimal("0.18");
    public Money taxOn(Money m) { return m.times(GST); }
}
```

```java
// File: code/lld/ch03/dependency/CheckoutService.java
package lld.ch03.dependency;

/**
 * Dependency (dashed arrow, "uses"): CheckoutService has NO fields pointing at
 * Cart, PaymentGateway, TaxCalculator or Receipt. It only touches them for the
 * duration of one method call:
 *   - Cart and PaymentGateway arrive as PARAMETERS,
 *   - TaxCalculator is a LOCAL variable,
 *   - Receipt is the RETURN type.
 * When checkout() returns, the relationship is over.
 */
public final class CheckoutService {

    public Receipt checkout(Cart cart, PaymentGateway gateway) {
        TaxCalculator tax = new TaxCalculator();            // local-variable dependency
        Money subtotal = cart.subtotal();                   // parameter dependency
        Money total = subtotal.plus(tax.taxOn(subtotal));
        String txn = gateway.charge(total);                 // parameter dependency (interface)
        return new Receipt(txn, subtotal, total);           // return-type dependency
    }

    public record Receipt(String txnId, Money subtotal, Money total) { }
}
```

```java
// File: code/lld/ch03/dependency/Main.java
package lld.ch03.dependency;

import java.util.List;

public class Main {
    public static void main(String[] args) {
        CheckoutService service = new CheckoutService();   // holds no collaborators at all
        Cart cart = new Cart(List.of(Money.of("250"), Money.of("99.50")));

        // The caller decides which gateway to use on every call.
        CheckoutService.Receipt r = service.checkout(cart, new FakeGateway());
        System.out.println("Receipt: " + r);

        PaymentGateway loggingGateway = amount -> {
            System.out.println("  [lambda gateway] charging " + amount);
            return "LMB-42";
        };
        System.out.println("Receipt: " + service.checkout(cart, loggingGateway));
    }
}
```

What each part shows: `CheckoutService` has **no fields at all**. It depends on `Cart` and `PaymentGateway` through parameters, on `TaxCalculator` through a local variable, and on `Receipt` through its return type. `Main` passes a different gateway (even a lambda, a short unnamed function written inline) on each call — something you cannot do as easily with a stored field. `Money` uses `BigDecimal` (Java's exact decimal-number type), not `double` (a binary floating-point number that cannot hold values like 0.10 exactly), because currency needs exact decimal arithmetic.

Output:

```
  [gateway] charging Rs 412.41
Receipt: Receipt[txnId=TXN-1, subtotal=Rs 349.50, total=Rs 412.41]
  [lambda gateway] charging Rs 412.41
Receipt: Receipt[txnId=LMB-42, subtotal=Rs 349.50, total=Rs 412.41]
```

Trade-offs

	Dependency (parameter)	Association (field)
Coupling	lowest	higher
Object state	can be stateless	holds state
Swapping collaborator	per call	per object (or via setter)
Call-site noise	caller passes B every time	caller passes B once

When not to use it

If A needs B across many methods and many calls (e.g. a service always uses the same repository), a stored field (association via constructor injection, where the collaborator is passed into the constructor once) is cleaner than passing B to every method.

Common mistakes

- Drawing a solid association line for something that is only a parameter — it overstates coupling.
- Drawing *every* dependency: a real class depends on `String`, `List`, `LocalDate`… Draw only the ones that matter to the design.
- Hiding a dependency with `new ConcreteThing()` deep inside a method when the caller should decide (this blocks testing). Prefer taking an interface as a parameter.

Real-world examples

- `Collections.sort(list, comparator)` depends on the `Comparator` only during the call.
- A Spring MVC controller method (the code that answers one web request) `handle(HttpServletRequest req)` depends on the request only for that call.
- `LocalDate.now(Clock clock)` depends on a `Clock` parameter — the JDK's way to make time testable.
- In build tools (Maven/Gradle, programs that download libraries and compile a Java project) a "dependency" is the same idea at package level: your code uses a library's classes.

In the interview

30-second answer: "Dependency is 'uses briefly': B is a parameter, local variable or return type of A's method, and A keeps no field for it. Dashed arrow from A to B. It is the loosest coupling, so when a collaborator is only needed in one method I pass it in rather than store it."

Follow-ups:

- *Q: `FlightReservation` takes a `Payment` in `pay(Payment p)` — association or dependency?* — A: Dependency, unless the reservation stores the payment in a field; then it becomes an association.
- *Q: How is this related to Dependency Injection?* — A: DI is about *who supplies* the collaborator (from outside, not `new` inside). The supplied object may be stored (association) or passed per call

(dependency). The Dependency Inversion Principle adds: depend on interfaces, not concrete classes (Chapter 4).

Self-check

- *Q: Which UML line: dashed + open arrow, or dashed + hollow triangle?* → A: Dashed + open arrow is dependency; dashed + hollow triangle is realization.
- *Q: A method returns a `Receipt`. Does that create a relationship?* → A: Yes, a dependency on `Receipt`.

TL;DR

Dependency = "uses for one call". Parameter, local variable, return type or static call; no field. Dashed arrow.

3.6 Generalization (inheritance)

What it is

Generalization is UML's word for **inheritance**: class B `extends` class A, so B *is a kind of* A. A is the **parent** (superclass, general); B is the **child** (subclass, specialised). B inherits A's fields and methods and may override them.

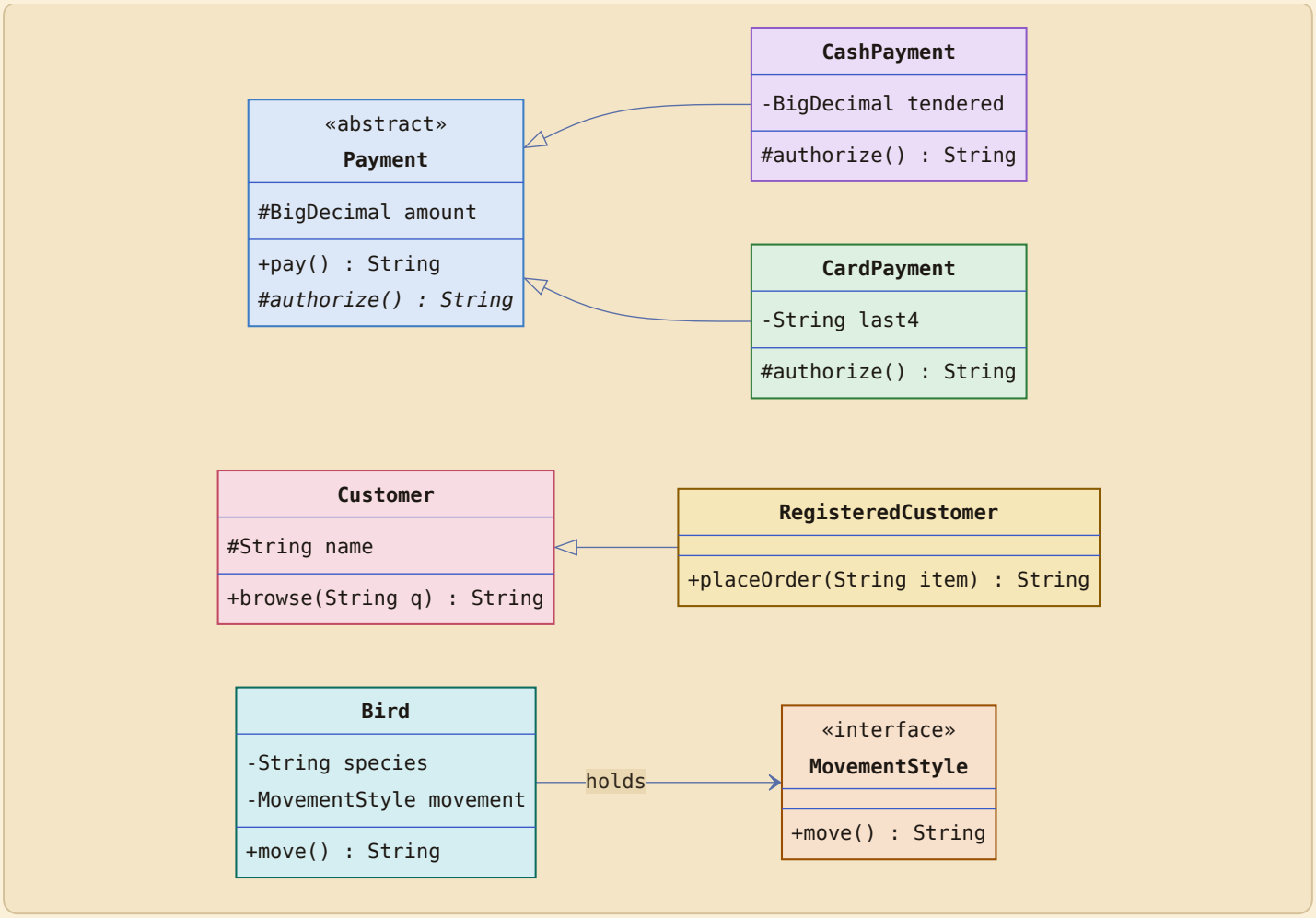
Analogy: a **credit-card payment** *is a* **payment**. Anywhere the shop's code can handle "a payment", it can handle a credit-card payment.

The problem it solves

When several classes truly share a contract **and** some behaviour, inheritance puts the shared parts in one place and lets calling code work with the general type (`Payment`) without knowing the specific one (**polymorphism** — one call, different behaviour depending on the actual object).

How it works

1. Before extending, run the **substitution test**: *can B be used anywhere A is expected, with no surprises?* If there is any method of A that B would have to refuse or break, the answer is no. (This is the Liskov Substitution Principle — Chapter 4.)
2. If yes, put the shared state and workflow in A (often `abstract`: a class that cannot be instantiated and may leave some methods for subclasses to fill in), and let subclasses override only the varying steps.
3. If no, **hold instead of extend**: give A a field of an interface type and plug in different behaviour objects.
4. In UML, draw a **solid line with a hollow triangle pointing at the parent**. An abstract class is shown with its name in *italics* (or `{abstract}` / `«abstract»`); abstract operations are italic too.



Java example

```java
// File: code/lld/ch03/generalization/Payment.java
package lld.ch03.generalization;

import java.math.BigDecimal;

/**
 * Generalization parent (abstract, drawn in italics or with {abstract}).
 * Every subclass genuinely "is a" Payment: each one really can take money, in its own way.
 * Money is BigDecimal, never double: double cannot represent amounts like 0.10 exactly.
 */
public abstract class Payment {
    protected final BigDecimal amount;   // '#' protected: visible to subclasses

    protected Payment(BigDecimal amount) {
        if (amount.signum() <= 0) throw new IllegalArgumentException("amount must be positive");
        this.amount = amount;
    }

    /** Shared, final workflow; the varying step is abstract. */
    public final String pay() {
        String ref = authorize();
        return getClass().getSimpleName() + " of " + amount + " -> " + ref;
    }

    protected abstract String authorize();   // abstract operation (italics in UML)
}
```

```java
// File: code/lld/ch03/generalization/CashPayment.java
package lld.ch03.generalization;

import java.math.BigDecimal;

public final class CashPayment extends Payment {
    private final BigDecimal tendered;

    public CashPayment(BigDecimal amount, BigDecimal tendered) {
        super(amount);
        this.tendered = tendered;
    }

    @Override protected String authorize() {
        if (tendered.compareTo(amount) < 0) return "DECLINED (short by " + amount.subtract(tendered) + ")";
        return "OK, change " + tendered.subtract(amount);
    }
}
```

```java
// File: code/lld/ch03/generalization/CardPayment.java
package lld.ch03.generalization;

import java.math.BigDecimal;

public final class CardPayment extends Payment {
    private final String last4;

    public CardPayment(BigDecimal amount, String last4) {
        super(amount);
        this.last4 = last4;
    }

    @Override protected String authorize() { return "OK, card ****" + last4 + " approved"; }
}
```

```java
// File: code/lld/ch03/generalization/Customer.java
package lld.ch03.generalization;

/** Base class holds only what EVERY customer can do, so no subclass must refuse anything. */
public class Customer {
    protected final String name;
    public Customer(String name) { this.name = name; }
    public String browse(String query) { return name + " browses '" + query + "'"; }
}
```

```java
// File: code/lld/ch03/generalization/RegisteredCustomer.java
package lld.ch03.generalization;

/** Ordering lives here, not on Customer, because a guest may not order. */
public class RegisteredCustomer extends Customer {
    public RegisteredCustomer(String name) { super(name); }
    public String placeOrder(String item) { return name + " orders " + item; }
}
```

```java
// File: code/lld/ch03/generalization/MovementStyle.java
package lld.ch03.generalization;

/** "Prefer holding over extending": behaviour that varies is held, not inherited. */
public interface MovementStyle {
    String move();

    MovementStyle FLY = () -> "flies";
    MovementStyle SWIM = () -> "swims";
    MovementStyle WALK = () -> "walks";
}
```

```java
// File: code/lld/ch03/generalization/Bird.java
package lld.ch03.generalization;

public final class Bird {
    private final String species;
    private MovementStyle movement;   // held, so it can be swapped at run time

    public Bird(String species, MovementStyle movement) {
        this.species = species;
        this.movement = movement;
    }

    public void setMovement(MovementStyle m) { this.movement = m; }
    public String move() { return species + " " + movement.move(); }
}
```

```java
// File: code/lld/ch03/generalization/Main.java
package lld.ch03.generalization;

import java.math.BigDecimal;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        System.out.println("== Good hierarchy: substitution works ==");
        List<Payment> payments = List.of(
                new CashPayment(new BigDecimal("500.00"), new BigDecimal("600.00")),
                new CardPayment(new BigDecimal("1200.00"), "4242"),
                new CashPayment(new BigDecimal("300.00"), new BigDecimal("200.00")));
        for (Payment p : payments) System.out.println(p.pay());   // caller only knows Payment

        System.out.println("\n== Guest vs registered customer ==");
        Customer guest = new Customer("Guest-17");
        RegisteredCustomer reg = new RegisteredCustomer("Asha");
        List<Customer> visitors = List.of(guest, reg);   // a RegisteredCustomer IS a Customer
        visitors.forEach(c -> System.out.println(c.browse("headphones")));
        System.out.println(reg.placeOrder("headphones"));
        System.out.println("guest.placeOrder(...) does not compile: guests never inherit ordering");

        System.out.println("\n== Holding instead of extending ==");
        Bird eagle = new Bird("Eagle", MovementStyle.FLY);
        Bird penguin = new Bird("Penguin", MovementStyle.SWIM);
        System.out.println(eagle.move());
        System.out.println(penguin.move());
        penguin.setMovement(MovementStyle.WALK);   // impossible with a subclass
        System.out.println(penguin.move() + " (behaviour swapped at run time)");
    }
}
```

What each part shows:

- `Payment` is abstract, its `pay()` is `final` (the shared workflow), and `authorize()` is abstract (the step that varies). Both subclasses honour every promise of `Payment`, so `Main` loops over `List<Payment>` without caring which is which. Amounts are `BigDecimal` and compared with `compareTo`, because `double` cannot store values such as `0.10` exactly and money must not drift.
- **Wrong-inheritance fix #1 — the subclass that must refuse something.** If `Guest extends Customer` and `Customer` can place orders, a guest could order, which the requirement forbids. The fix: `Customer` only has what *every* customer can do (browse); ordering moves down to `RegisteredCustomer`.
- **Wrong-inheritance fix #2 — the subclass that only carries a constant.** `CompactSpot`, `LargeSpot`, `HandicappedSpot` whose constructors only pass a different value up are three classes with no new behaviour. That is a field: see the `SpotType` enum in the composition example. A fourth type becomes one enum constant, not a new class.
- **Prefer holding over extending.** `FlyingBird` / `Penguin` subclasses break when a penguin arrives (penguins don't fly). `Bird` *holds* a `MovementStyle` instead, and it can even be swapped at run time — impossible with `extends`.

Output:

```
== Good hierarchy: substitution works ==
CashPayment of 500.00 -> OK, change 100.00
CardPayment of 1200.00 -> OK, card ****4242 approved
CashPayment of 300.00 -> DECLINED (short by 100.00)

== Guest vs registered customer ==
Guest-17 browses 'headphones'
Asha browses 'headphones'
Asha orders headphones
guest.placeOrder(...) does not compile: guests never inherit ordering

== Holding instead of extending ==
Eagle flies
Penguin swims
Penguin walks (behaviour swapped at run time)
```

Trade-offs

	Inheritance (extends)	Holding an object (composition/association)
Reuse	automatic	explicit delegation
Change at run time	no — fixed at compile time	yes — swap the held object
Coupling	tight: subclass sees parent internals ("fragile base class")	loose: only the interface
Number of parents in Java	one class	many interfaces / many fields
Best for	true is-a with different behaviour	varying behaviour, features that combine

When not to use it

- To share a few fields or a helper method ("code reuse" alone is not a reason).
- When a subclass would have to throw `UnsupportedOperationException` (Java's standard "this operation is not supported" error) for an inherited method.
- When the variation is just data (use a field or enum).
- When behaviours need to be mixed (fly + swim + walk) — the class count explodes.

Common mistakes

- Deep hierarchies (more than 2–3 levels) that nobody can follow.
- `Stack extends Vector` style mistakes: the child inherits methods that break its own rules (a JDK `Stack` lets you insert in the middle).
- Arrow direction: the triangle points **at the parent**, not the child.
- Using inheritance between two classes when one can do what the other does only in *some* situations. Generalization needs "always"; for "sometimes", hold a shared interface or a role object instead.

Real-world examples

- `java.util.ArrayList extends AbstractList` (reuses the skeleton implementation).
- `java.io.FileInputStream extends InputStream`; any code taking an `InputStream` accepts it.
- Every Java exception: `IOException extends Exception extends Throwable`, which is why `catch (Exception e)` catches an `IOException`.
- **Counter-example in the JDK:** `java.util.Stack extends Vector` and `java.util.Properties extends Hashtable` are widely cited as inheritance that should have been composition.

In the interview

> **30-second answer:** "Generalization is inheritance — solid line with a hollow triangle at the parent. I use it only when the child passes the substitution test: it can go anywhere the parent goes and never refuses a parent method. A subclass that only passes a constant up becomes an enum field; varying behaviour I hold as an interface instead of extending."

Follow-ups:

- *Q: Why not* `CompactSpot extends ParkingSpot`*?* — A: It adds no behaviour; a `SpotType` field does the same job and new types are one enum constant.
- *Q: When is inheritance right?* — A: When subclasses behave differently but all keep the parent's promises, like `Payment` → cash/card: both really take money, differently.
- *Q: Can a Java class have two parents?* — A: It can extend only one class but implement many interfaces.

Self-check

- *Q: Which way does the hollow triangle point?* → A: At the parent (general class).
- *Q: Guest cannot order, but* `Guest extends Customer` *and* `Customer.placeOrder()` *exists. What principle is broken and how do you fix it?* → A: Substitution (LSP). Move `placeOrder` to `RegisteredCustomer`.

TL;DR

Generalization = "is-a", `extends`. Only after the substitution test. Constants → fields; varying behaviour → hold an interface. Triangle points at the parent.

3.7 Realization (interface implementation)

What it is

Realization means a class *implements* an **interface**. An interface is a named list of operations (a contract) with no instance state. The class promises to provide those operations. In Java: `class EmailNotifier implements Notifier`.

Analogy: a **wall socket standard**. Any appliance whose plug "realizes" the standard works in any socket; the socket does not care who made the appliance.

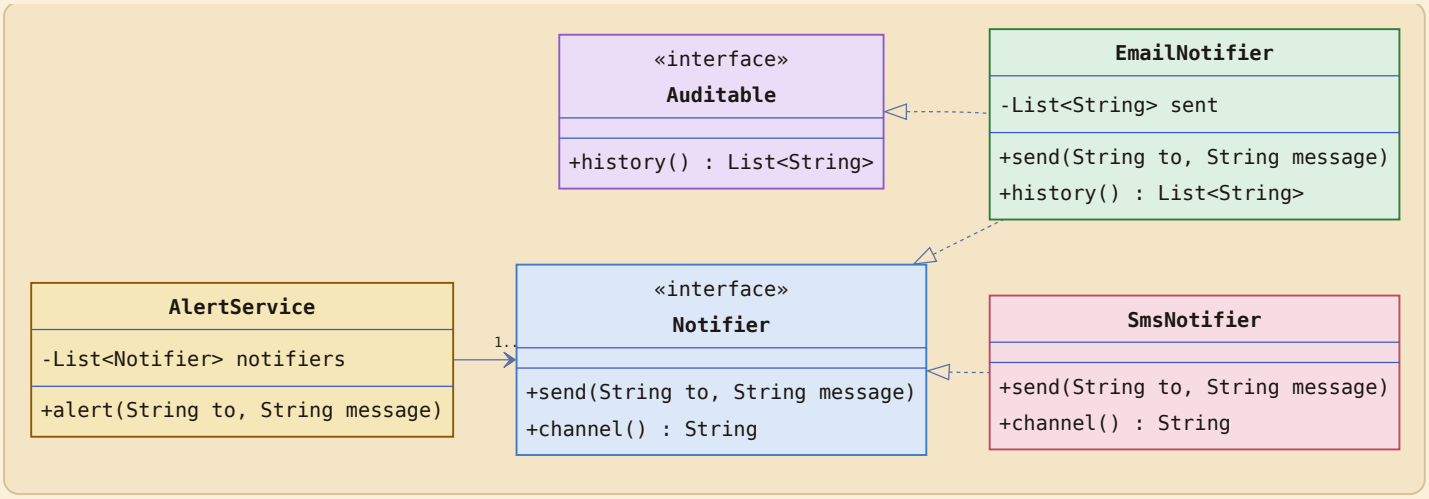
The problem it solves

Callers can depend on the **contract**, not the concrete class. You can add new implementations (SMS, push, WhatsApp) without changing the caller, test with fakes, and give one class several roles (a class may implement many interfaces but extend only one class).

How it works

1. Define an interface with the operations callers need (`send(to, message)`).
2. Each concrete class `implements` it and provides the method bodies.
3. Callers hold / receive the **interface type** (`List<Notifier>`).
4. Since Java 8 an interface may have `default` methods (with a body) and `static` methods; implementers may override defaults.
5. In UML, draw a **dashed line with a hollow triangle pointing at the interface**, and mark the interface with `«interface»`. (A compact alternative is the "lollipop": a small circle on a stick labelled with the interface name.)

Realization looks like generalization on purpose: it is the same "is-a" idea, but nothing but the contract is inherited.



Java example

```java
// File: code/lld/ch03/realization/Notifier.java
package lld.ch03.realization;

/** An interface: a contract of operations with no state. Drawn as <<interface>>. */
public interface Notifier {
    void send(String to, String message);

    /** Java 8+ default method: implementers inherit this unless they override it. */
    default String channel() {
        String n = getClass().getSimpleName();
        return n.contains("$$Lambda") ? "lambda" : n;   // lambdas get generated class names
    }
}
```

```java
// File: code/lld/ch03/realization/Auditable.java
package lld.ch03.realization;

import java.util.List;

public interface Auditable {
    List<String> history();
}
```

```java
// File: code/lld/ch03/realization/EmailNotifier.java
package lld.ch03.realization;

import java.util.ArrayList;
import java.util.List;

/** Realizes TWO interfaces. A class may implement many interfaces but extend only one class. */
public final class EmailNotifier implements Notifier, Auditable {
    private final List<String> sent = new ArrayList<>();

    @Override public void send(String to, String message) {
        String line = "EMAIL to " + to + ": " + message;
        sent.add(line);
        System.out.println("  " + line);
    }

    @Override public List<String> history() { return List.copyOf(sent); }
}
```

```java
// File: code/lld/ch03/realization/SmsNotifier.java
package lld.ch03.realization;

public final class SmsNotifier implements Notifier {
    @Override public void send(String to, String message) {
        String text = message.length() > 20 ? message.substring(0, 20) + "..." : message;
        System.out.println("  SMS to " + to + ": " + text);
    }

    @Override public String channel() { return "SMS"; }   // overrides the default
}
```

```java
// File: code/lld/ch03/realization/AlertService.java
package lld.ch03.realization;

import java.util.List;

/**
 * AlertService is associated with the INTERFACE, not with any concrete class.
 * Adding a PushNotifier later needs no change here.
 */
public final class AlertService {
    private final List<Notifier> notifiers;

    public AlertService(List<Notifier> notifiers) { this.notifiers = List.copyOf(notifiers); }

    public void alert(String to, String message) {
        for (Notifier n : notifiers) {
            System.out.println("via " + n.channel());
            n.send(to, message);
        }
    }
}
```

```java
// File: code/lld/ch03/realization/Main.java
package lld.ch03.realization;

import java.util.List;

public class Main {
    public static void main(String[] args) {
        EmailNotifier email = new EmailNotifier();
        Notifier sms = new SmsNotifier();
        Notifier console = (to, msg) -> System.out.println("  CONSOLE " + to + ": " + msg); // lambda realizes it
too

        AlertService alerts = new AlertService(List.of(email, sms, console));
        alerts.alert("ops@shop.example", "Disk usage above 90 percent on db-1");

        Auditable audit = email;   // same object, seen through its other interface
        System.out.println("Email audit trail: " + audit.history());
    }
}
```

What each part shows: `EmailNotifier` realizes **two** interfaces; `SmsNotifier` realizes one and overrides the `default` method; a **lambda** also realizes `Notifier`, because it has exactly one abstract method (a *functional interface*). `AlertService` holds `List<Notifier>` — an association to the interface — so a new channel needs no change in it. The same `email` object is seen as a `Notifier` by `AlertService` and as an `Auditable` by `Main`.

Output:

```
via EmailNotifier
  EMAIL to ops@shop.example: Disk usage above 90 percent on db-1
via SMS
  SMS to ops@shop.example: Disk usage above 90 ...
via lambda
  CONSOLE ops@shop.example: Disk usage above 90 percent on db-1
Email audit trail: [EMAIL to ops@shop.example: Disk usage above 90 percent on db-1]
```

Trade-offs

	Interface + realization	Abstract class + generalization
Multiple per class	yes	no (single inheritance)
Instance fields	no	yes
Shared code	`default` methods only	any methods, constructors
Best for	roles/capabilities, plug-ins, test seams	a family sharing state and a workflow

When not to use it

- An interface with exactly one implementation that will never vary, and no testing need, is ceremony. Add it when a second implementation or a test double (a fake stand-in used in tests) appears (YAGNI: "you aren't gonna need it", so don't build things before they are needed).
- "Marker" interfaces with no methods are rarely needed now; annotations (`@` tags such as `@Override` that attach information to code) usually serve better.

Common mistakes

- Solid line instead of dashed (that would mean class inheritance).
- Fat interfaces that force implementers to stub (write empty placeholder versions of) methods they do not need (breaks the Interface Segregation Principle, Chapter 4).
- Callers declaring variables with the concrete type (`EmailNotifier n = ...`) and losing the benefit.

Real-world examples

- `java.util.List` realized by `ArrayList` and `LinkedList`; `Comparable`, `Runnable`, `AutoCloseable`.
- **JDBC** (Java Database Connectivity, Java's standard way to talk to SQL databases): `java.sql.Connection` is an interface; each database vendor's driver realizes it, and your code never names the vendor class.
- **Spring:** you inject `PaymentGateway`; the bean (an object Spring creates) may be a Stripe, Razorpay (online payment providers) or mock (fake, for testing) implementation.
- **SLF4J** (logging facade, i.e. one simple front for several logging libraries): your code calls the `Logger` interface; Logback or Log4j (two logging libraries) realizes it.

In the interview

30-second answer: "Realization is a class implementing an interface — dashed line, hollow triangle at the interface. I put an interface wherever behaviour can vary or I need a test double (payment gateway, notifier, pricing strategy), and the rest of the design depends on the interface only."

Follow-ups:

- *Q: Interface or abstract class?* — A: Interface for a capability many unrelated classes can have, or when a class needs several; abstract class when subclasses share state and a fixed workflow. They combine well: interface on top, abstract skeleton class below (like `List` → `AbstractList` → `ArrayList`).
- *Q: Is a lambda a realization?* — A: Yes; a lambda is an instance of a functional interface.

Self-check

- *Q: Draw the difference between generalization and realization.* → A: Same hollow triangle; generalization has a solid line, realization a dashed line.
- *Q: How many interfaces can a Java class implement?* → A: Any number.

TL;DR

Realization = "implements". Dashed line + hollow triangle at the `«interface»`. Depend on the interface, plug in implementations.

3.8 Decision guide: choosing the relationship between two classes

Ask these questions **in order** and stop at the first "yes":

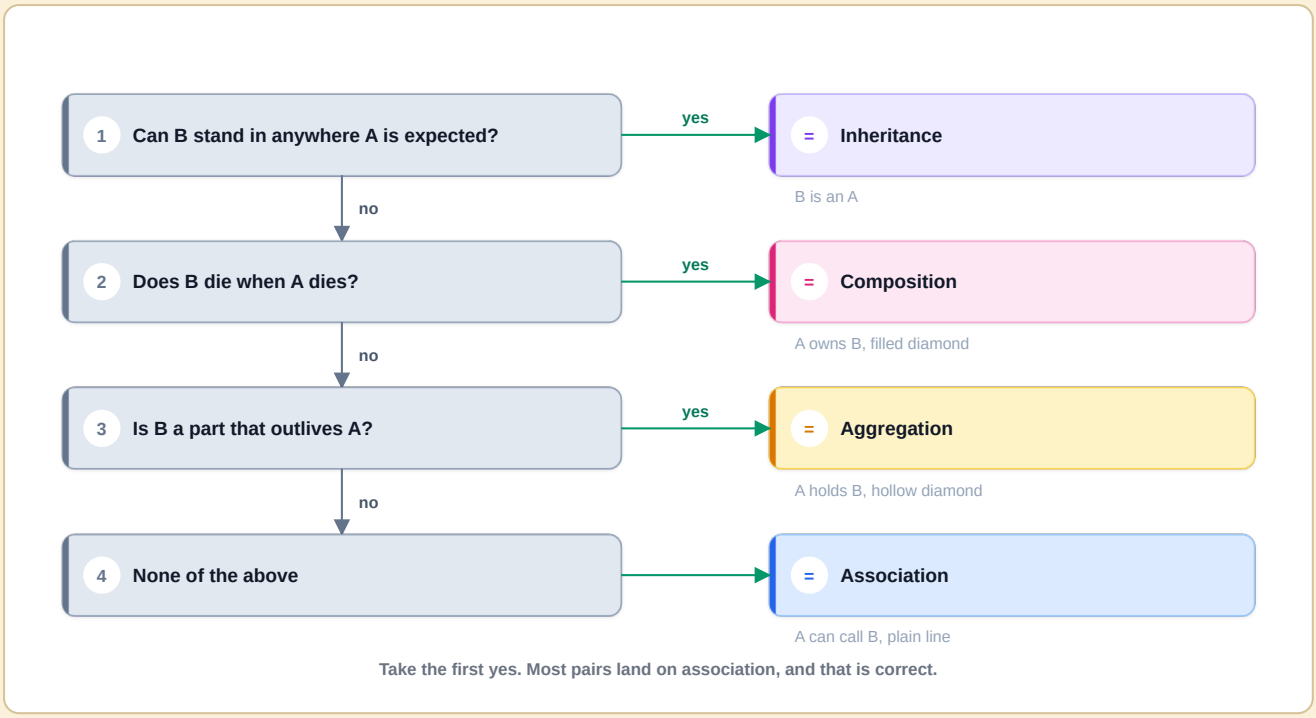
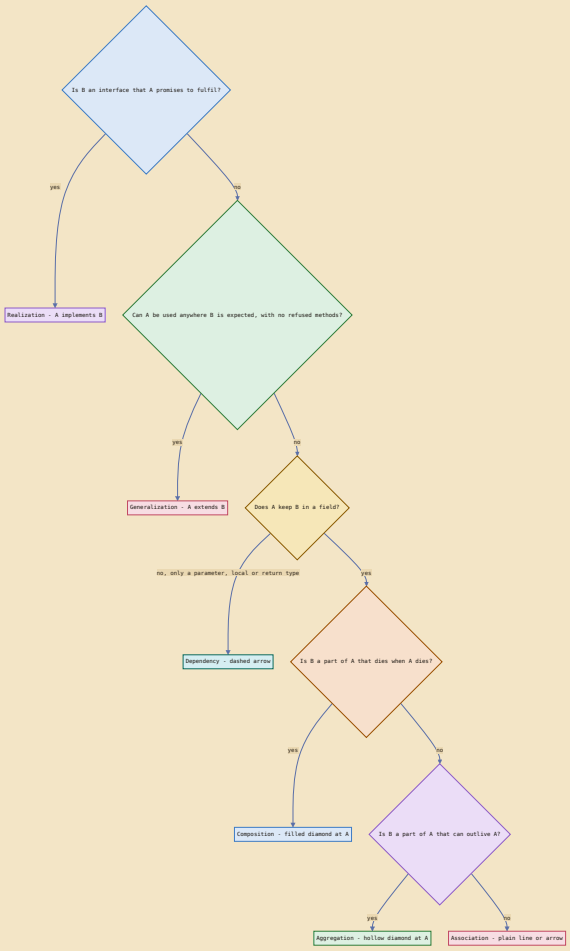


Figure — The four-question decision: inheritance, then composition, then aggregation, else association (course diagram)

As the figure above shows, inheritance is asked first only because it is the easiest to *rule out*: it needs a strict yes to the substitution test, and most pairs fail it. Most pairs end up as plain association, which is healthy. If nearly every box in your diagram extends another, the design needs another look.

The full guide including the two "light" relationships:



Summary table (keep this one)

Relationship	Phrase	UML line	Java shape	Lifetime	Example
Dependency	A *uses* B	dashed, open arrow → B	B as parameter / local / return / static call	none	`CheckoutService` uses `PaymentGateway`
Association	A *knows* B	solid line (arrow for one-way)	field of type B, B created elsewhere	independent	`Order` → `Customer`
Aggregation	A *has* B (parts live on)	solid, hollow ◇ at A	field/collection, parts passed in	B outlives A	`Airline` ◇— `Aircraft`
Composition	A *owns* B	solid, filled ◆ at A	A creates B, hides it, B not shared	B dies with A	`ParkingFloor` ◆— `ParkingSpot`
Realization	A *implements* B	dashed, hollow △ at B	`implements`	—	`EmailNotifier` ⇢ `Notifier`
Generalization	A *is a* B	solid, hollow △ at B	`extends`	—	`CardPayment` → `Payment`

Mermaid cheat-sheet for these lines (used in all later chapters):

Mermaid	Meaning
`A ..> B`	A depends on B
`A --> B`	A associated with B, navigable A to B
`A -- B`	association (both ways / unspecified)
`A o-- B`	aggregation, A is the whole
`A *-- B`	composition, A is the whole
`B <|.. A`	A realizes interface B
`B <|-- A`	A extends B
`A "1" *-- "1..*" B`	with multiplicities

Common mistakes (all relationships)

- Diamond at the wrong end (it goes at the **whole**).
- Triangle at the wrong end (it goes at the **parent / interface**).
- Lines without multiplicity. A line with no type or count means the design question behind it is still open. Candidates who write the numbers on every line from the very first box report that the interviewer simply stops asking about them — it is cheap to add and removes a whole class of follow-up questions.
- Inheritance used to share data.

In the interview

Say the test **out loud** as you draw each line, e.g. *"When a floor is removed its spots go too, so this is composition"* or *"Cash and card can both be asked to take money, so inheriting from Payment is honest."* If you spot a subclass that only carries a constant, say so and collapse it into a field yourself. Expect two follow-ups: "why not inheritance here?" (answer with the substitution test) and "what if a spot can hold two motorbikes?" (change the multiplicity, then say what it does to the code).

Self-check

- *Q:* `Library` *and* `Member` *— which relationship?* → A: Aggregation (or plain association): members exist whether or not the library does.
- *Q:* `Flight` *and* `WeeklySchedule`*?* → A: Composition.
- *Q:* `ParkingTicket` *and* `ParkingSpot`*?* → A: Association: the ticket refers to the spot but is not a whole made of spots.
- *Q:* `Report.render(Formatter f)`*?* → A: Dependency.

TL;DR

Realization/generalization for "is-a" (only after the substitution test), composition when parts die with the whole, aggregation when they live on, association when one just knows the other, dependency when it is only used inside a method.

Part B — UML

3.9 What UML is (structural vs behavioural diagrams)

What it is

UML (Unified Modeling Language) is a standard set of symbols for drawing software designs. Because everybody uses the same symbols, a diagram drawn by one engineer can be read by another without a legend. It is maintained by the OMG (Object Management Group); the current version is UML 2.5.1.

UML is **not** a programming language. It does not run and you do not need a tool to use it — a whiteboard is enough. It is an agreement about what, for example, "a line with a filled diamond on one end" means.

A little history explains its size: in the 1990s three competing object-oriented methods (Grady Booch's, James Rumbaugh's OMT and Ivar Jacobson's OOSE) each had their own notation. They were merged into UML so that the industry had one language. Because it had to satisfy three groups of users, UML is large — and every team uses only the subset it needs.

The problem it solves

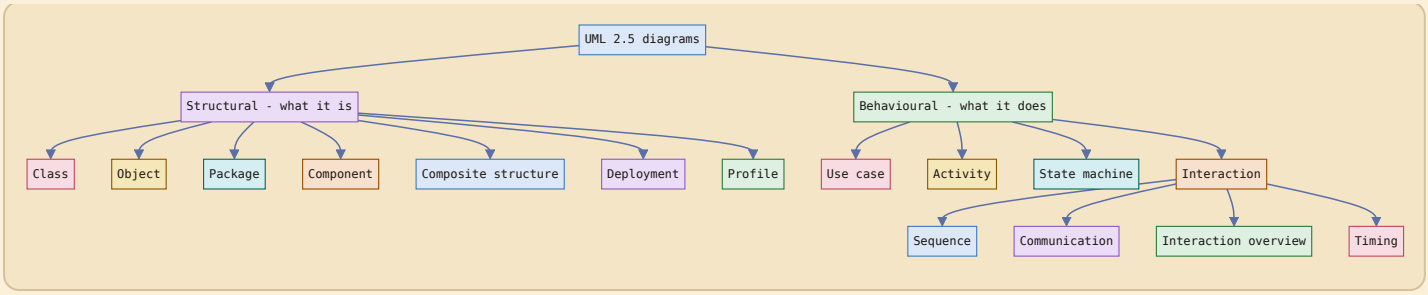
1. **It makes a design visible.** Twelve classes in sentences are hard to hold in your head; twelve boxes with lines are readable in seconds.
2. **It makes decisions explicit.** In a sentence you can stay vague about whether A owns B. On a diagram you must pick a diamond — filled or hollow.
3. **It makes disagreement cheap.** The interviewer can put a finger on one line and ask "why composition here?". That kind of back-and-forth is exactly what a good interview looks like.

How it works — the 14 diagram types

UML 2.5 defines **14 diagram types** in two families:

- **Structural diagrams** show what the system *is made of* — the static parts that exist at any moment.
- **Behavioural diagrams** show what the system *does* over time. A subgroup of behavioural diagrams, **interaction diagrams**, focus on messages between objects.

Family	Diagram	Shows	Interview use
Structural	**Class**	classes, attributes, operations, relationships	**always**
Structural	Object	a snapshot of real objects and their links	rarely (to explain an example)
Structural	Package	how classes group into packages/modules	rarely
Structural	Component	deployable parts and their interfaces	system design rounds
Structural	Composite structure	internal parts of one class	rarely
Structural	Deployment	software on hardware nodes (machines)	system design rounds
Structural	Profile	extensions to UML itself	never
Behavioural	**Use case**	actors and what they can achieve	early, often spoken
Behavioural	**Activity**	steps of one flow with branches	one per key flow
Behavioural	**State machine**	states of one object and allowed transitions	when an object has a lifecycle
Behavioural → interaction	**Sequence**	who calls whom, in time order	on request
Behavioural → interaction	Communication	same messages, drawn on a network of objects	rarely
Behavioural → interaction	Interaction overview	an activity diagram whose nodes are interactions	rarely
Behavioural → interaction	Timing	state changes against a time axis	embedded (software inside devices) / hardware



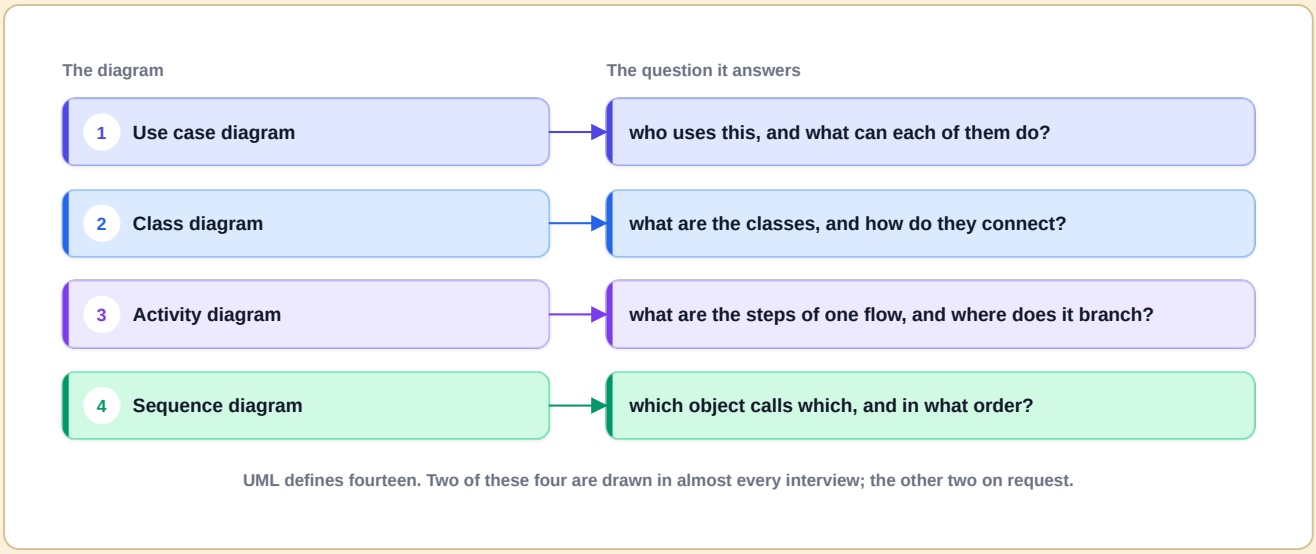


Figure — The four diagrams an OOD interview uses and the question each answers (course diagram)

As the figure above shows, four diagrams carry an object-oriented design interview, and this book adds a fifth that often helps:

1. **Use-case diagram** — who uses the system and what can each of them do? Drawn first, before any class exists.
2. **Class diagram** — what are the classes and how do they connect? *The* answer; the only one that maps directly to code.
3. **Activity diagram** — what are the steps of one flow and where does it branch?
4. **Sequence diagram** — which object calls which, in what order?
5. **State machine diagram** — what states can one object be in, and what moves it between them? (Parking spot, order, elevator, vending machine…)

Trade-offs

Benefit	Cost
Shared vocabulary, no legend needed	Large spec; easy to over-draw
Forces explicit decisions	Takes whiteboard time from code/discussion
Tool support (PlantUML, Mermaid, draw.io, StarUML: diagram-drawing tools)	Diagrams go stale if not kept with code

When not to use it

As documentation that nobody maintains, or in full formal detail on a whiteboard. Many teams today use lightweight "sketch" UML, the C4 model for architecture (a simple style that draws a system at four zoom levels), or text-based tools (Mermaid/PlantUML) kept next to the code. For an interview, UML-lite is exactly right.

Common mistakes

- Thinking the interviewer grades notation. They grade whether your **diagram and your words agree**. If you forget the realization arrow, say "this one implements that one" and draw something. If you draw a hollow diamond where you meant a filled one and then say which you meant, that is a discussion, not a lost point.
- Drawing a line you cannot justify. It hurts more than a missing line.

Real-world examples

- Mermaid diagrams render natively in GitHub and GitLab Markdown (a plain-text formatting language), which is why many design docs (RFCs, written design proposals; ADRs, Architecture Decision Records) embed sequence and class diagrams as text.
- PlantUML is widely used in enterprise wikis (Confluence, a company wiki tool, via plug-ins) for sequence diagrams of service calls.

In the interview

> **30-second answer:** "UML is a standard notation for designs. It has 14 diagrams: structural ones (class, object, component…) describe what the system is made of; behavioural ones (use case, activity, state machine, and the interaction diagrams such as sequence) describe what it does. In an OOD round I use a quick use-case list, one class diagram, and an activity or sequence diagram for the key flow."

Follow-ups:

- *Q: Is UML outdated?* — A: The full formal UML is rarely used, but its core notations (class, sequence, state) remain the shared language of design discussions and appear in Mermaid/PlantUML in modern repos (code repositories). You only need the subset.
- *Q: Which diagram maps to code?* — A: The class diagram.

Self-check

- *Q: Structural or behavioural: sequence diagram?* → A: Behavioural (an interaction diagram).
- *Q: How many UML diagram types exist?* → A: 14 (7 structural, 7 behavioural).

TL;DR

UML = standard symbols, not a language that runs. 14 diagrams: structural (what it is) vs behavioural (what it does). Interviews use class (always), use case, activity, sequence, and state machine when an object has a lifecycle.

3.10 Use-case diagrams

What it is

A **use-case diagram** shows **who** uses the system and **what** each of them can achieve. It has no classes, no methods, no data and no ordering.

- An **actor** is a *role* that interacts with the system, drawn as a stick figure outside the system. Roles, not people: one person can be both a Member and a Librarian; one actor can stand for thousands of users. An actor need not be human — a payment gateway, an email service or a timer ("System") is an actor too.
- A **use case** is one goal an actor can achieve that produces a result the actor cares about, drawn as an oval with a verb phrase: "Borrow a book". "Click the search button" is not a use case; it is a step inside one.
- The **system boundary** is a rectangle around all use cases, labelled with the system's name. Inside = what you build. Outside = everything else.

Analogy: the menu board at a restaurant. It lists what customers can order (use cases) and who can order what (a kids' menu for children), without saying anything about the kitchen.

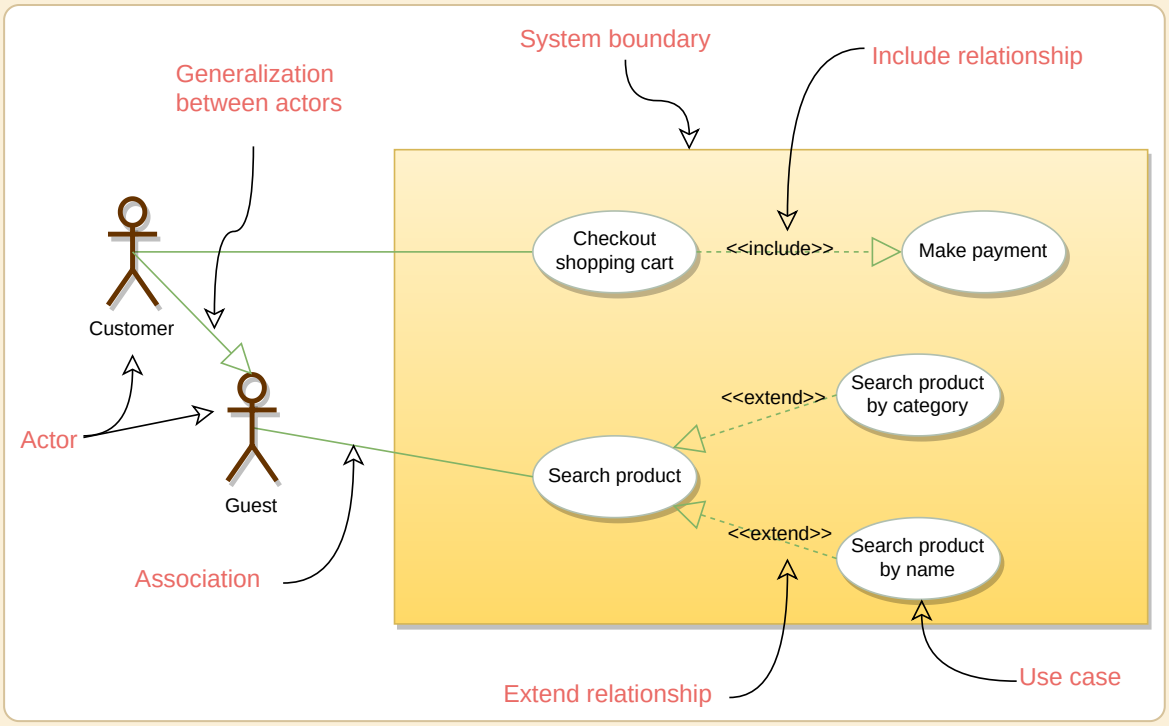


Figure — The parts of a use-case diagram for online shopping: actors (Customer, Guest), use cases, system boundary, generalization between actors, and «include» / «extend» relationships (course diagram)

The problem it solves

It turns a vague one-line question ("design a library") into an **agreed scope**. Drawing the boundary lets you say "these nine things are in; refunds, seasonal pricing and reporting are out" — and the interviewer, who judges whether your answer is complete, has agreed to that list. It also hands you your first classes: **actors usually become classes and use cases usually become methods**.

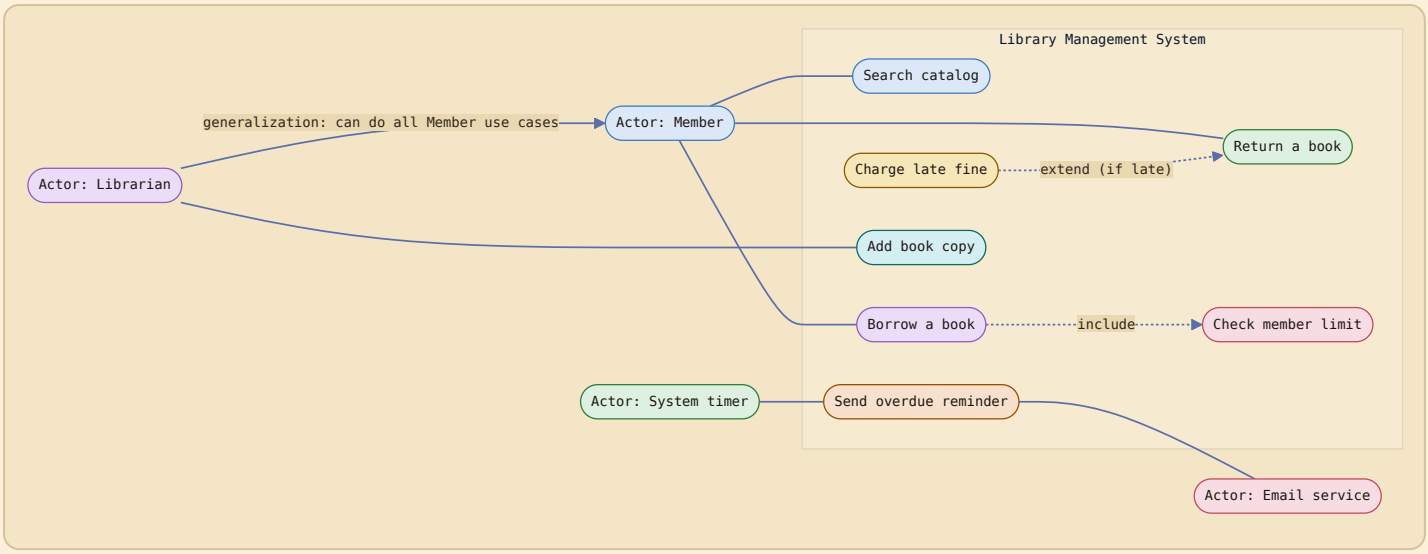
How it works

1. **Find the actors** with four questions:
   - *Who starts something?* (customer, member, player)
   - *Who administers it?* (adds books, sets rates, blocks accounts — the most-forgotten actor, and it brings a whole column of use cases)
   - *Who gets told?* (anyone receiving notifications)
   - *What acts with nobody present?* (timers, background jobs, external services — draw as "System")
2. Put **primary actors** (who start use cases) on the left and **secondary/supporting actors** (whom the system calls, e.g. payment gateway) on the right.
3. Draw the **system boundary** and put each **use case** (verb phrase) inside.
4. Connect actors to their use cases with **plain lines (associations)**.
5. Add relationships between use cases only where they help:
   - **«include»** — *always*. The base use case always performs the included one. Dashed arrow **from the base to the included** case. E.g. *Borrow a book* «include» *Check member's limit*. In code: one method always calls another.
   - **«extend»** — *sometimes*. An optional behaviour that is inserted into the base under a condition, at a named **extension point**. Dashed arrow **from the extending case back to the base**. E.g. *Charge fine* «extend» *Return a book* (only if late). Test: can the base use case complete without it? Yes → extend; no → include.
   - **Generalization** — solid line with a hollow triangle: between actors (a Librarian can do everything a Member can) or between use cases (*Pay by card* is a kind of *Make payment*).

6. Spell the keywords singular: «include», «extend». Older material uses «includes», «extends» or «uses»; «uses» was replaced by «include» long ago.

Reading the course figure above: the Customer→Guest line has its hollow triangle at **Guest**, so it reads "a Customer is a kind of Guest — a Customer can do everything a Guest can, plus check out". The figure also puts hollow triangles on its «include»/«extend» arrows; strict UML draws both as a dashed line with an **open** arrowhead (the triangle is reserved for generalization). What matters is the direction and the label: «include» points *Checkout shopping cart → Make payment*; both «extend» arrows point from the search variants back to *Search product*.

Mermaid has no native use-case diagram, so this book draws them as flowcharts (stadium shapes = use cases, a subgraph = the boundary):



From use cases to code (see `lld.ch03.library`)

Use-case element	Becomes in Java
Actor *Member*	class `Member`
Actor *Librarian* (generalizes Member)	`class Librarian extends Member`
Borrow a book	`Member.checkout(BookItem, LocalDate)`
«include» *Check member limit*	private `checkLimit()` called on **every** checkout
Return a book	`Member.returnBook(BookItem, LocalDate)`
«extend» *Charge late fine*	`chargeFine(...)` called **only if** the book is late
Add book copy	`Librarian.addCopy(...)`

This is a starting point, not a rule: a use-case method sometimes belongs on a service class instead of the actor class.

Trade-offs

Pro	Con
Fast scope agreement; finds forgotten actors	No ordering, no data — it cannot show *how*
Maps to classes and methods	Easy to bloat with UI (user interface) steps

When not to use it

When the interviewer has already given precise requirements — then a spoken list of actors and their use cases is enough.

Common mistakes

- **Use cases that are UI steps.** Typing a password, pressing submit and seeing a confirmation are all part of a single use case, "Log in". A diagram with dozens of ovals is usually a list of screens, not goals.
- **One actor per person.** A librarian who also borrows is *two roles*. Draw two actors, linked by generalization if one can do everything the other can.
- Reversing the «extend» arrow (it points from the optional case to the base).
- Using «include» for something that only happens sometimes.
- Leaving off the system boundary — the most useful part.

Real-world examples

- **An e-commerce checkout (Amazon-style), modelled:** *Place order* always «include»s *Make payment*; *Apply gift card* «extend»s it only when the customer has one.
- **BookMyShow:** Customer, Theatre admin, Payment gateway and a timer that releases held seats are the actors.

In the interview

> **30-second answer:** "Before classes, I list actors and what each can do, and draw a boundary to agree scope. Actors are roles — including admins, external services and timers. «include» is always, «extend» is sometimes. Actors become my first classes and use cases become methods."

Follow-ups:

- *Q: Is 'make payment' an include or an extend of 'checkout'?* — A: If every checkout requires payment, it is «include». Something like "apply coupon", which checkout can complete without, is «extend».
- *Q: How long should this take?* — A: Three to five minutes, early.

Self-check

- *Q: Which direction does the «include» arrow point?* → A: From the base use case to the included one.
- *Q: Is a payment gateway an actor?* → A: Yes — an external system that interacts with ours (a secondary actor).
- *Q: "Search product by name" and "by category" relate to "Search product" how?* → A: As in the course diagram: optional variants, drawn as «extend» (some modellers draw them as use-case generalization — both are defensible if explained).

TL;DR

Actors (roles, outside) + use cases (goals, inside) + boundary (scope). «include» = always, arrow base→included; «extend» = sometimes, arrow extension→base. Actors → classes, use cases → methods.

3.11 Class diagrams

What it is

A **class diagram** shows the **static structure** of a design: which classes exist, what each holds (attributes), what each can do (operations), and how they connect (relationships with multiplicity). It is the answer to an OOD question and the only UML diagram that maps directly to code.

The problem it solves

It captures the whole design on one page so it can be reviewed, challenged and turned into code. Every other diagram exists either to get you to it (use case) or to check/explain it (activity, sequence, state).

How it works — the class box

A class is a rectangle with three compartments: **name** / **attributes** / **operations**.

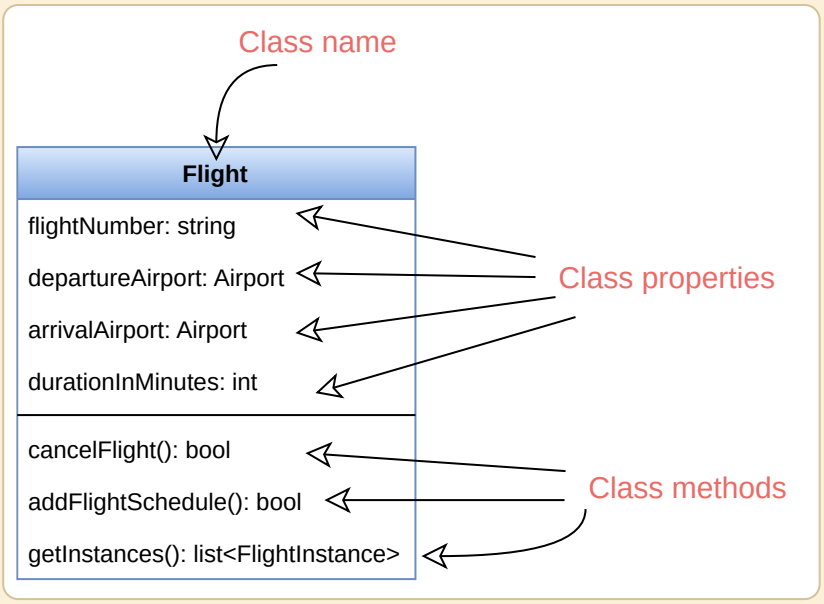


Figure — A class box: name on top, properties (attributes) in the middle, methods (operations) at the bottom, using a Flight class (course diagram)

As in the figure above, attributes are written `name: Type` and operations `name(params): ReturnType`. The full UML syntax is:

```
visibility name : Type [multiplicity] = default      e.g.  - spots : ParkingSpot [1..*]
visibility name(param : Type, ...) : ReturnType      e.g.  + park(v : Vehicle) : Ticket
```

Visibility markers (mapped to Java):

UML	Meaning	Java
`+`	public — anyone	`public`
`-`	private — only this class	`private`
`#`	protected — this class and subclasses	`protected` (Java also allows the same package)
`~`	package — same package only	no modifier (package-private)

Other notation:

Notation	Meaning	Java
Italic name, or `{abstract}` / `«abstract»`	abstract class (one you never create objects of directly) / abstract operation (no body)	`abstract`
`«interface»` above the name	interface	`interface`
`«enumeration»` + literals	enum	`enum`
underlined member	static (class-level)	`static`
`{readOnly}`	cannot change after creation	`final` field
`List<T>` / `T[*]`	collection attribute	`List<T>`

You do **not** have to fill all three compartments. Early in an interview a box with only a name is a perfectly good class. Attributes and operations arrive as the design settles.

In Mermaid, visibility uses the same symbols; `*` after a method means abstract and `$` means static; `<<interface>>`, `<<abstract>>`, `<<enumeration>>` are annotations; generics are written `List~T~`.

How it works — the relationships (all arrow types)

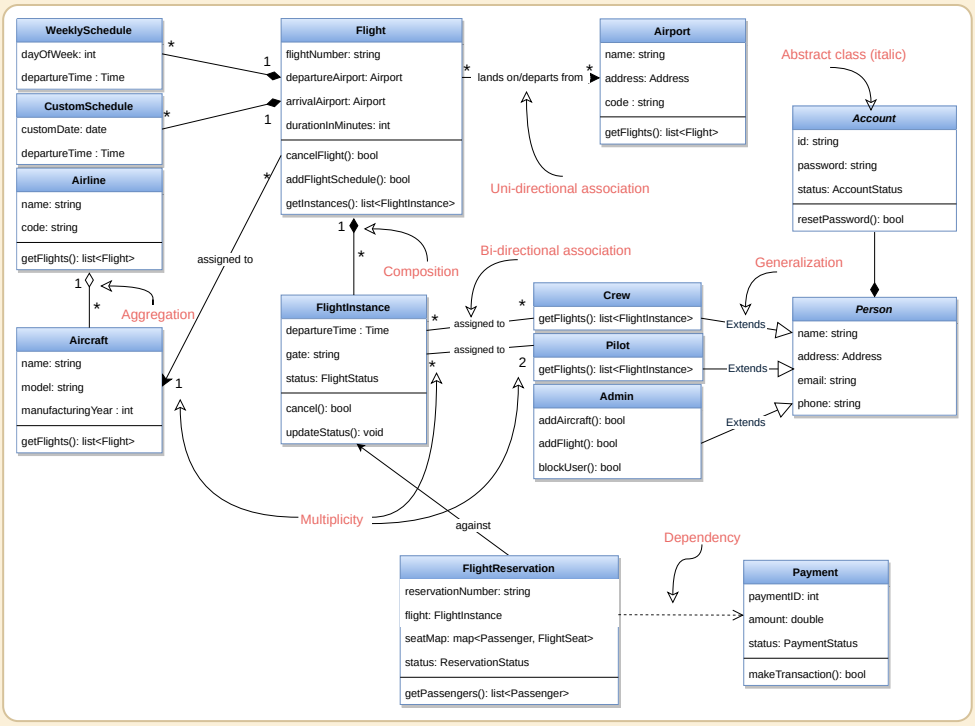


Figure — An airline-system class diagram using every relationship: associations (e.g. Flight to Aircraft, uni-directional), multiplicities (FlightInstance has 2 Pilots), aggregation (Airline–Aircraft), composition (Flight–WeeklySchedule), generalization (Person to Crew/Pilot/Admin) and dependency (course diagram)

The figure above uses the complete vocabulary: the six relationships of Part A (association appearing in both its two-way and one-way form) plus multiplicity. Those are the only symbols a class diagram needs:

Line	Drawn as	Read as	Example from the figure
Association (bi-directional)	solid line	"A and B know each other"	FlightInstance — Pilot, FlightInstance — Crew ("assigned to")
Uni-directional association	solid line, arrowhead at B	"A can call B, not vice versa"	Flight → Airport ("lands on/departs from"), Flight → Aircraft ("assigned to")
Multiplicity	numbers at the line ends: `1`, `*`, `0..1`, `2..4` (a range)	"how many"	FlightInstance `*` — `2` Pilot: each flight instance has exactly 2 pilots, each pilot flies many instances
Aggregation	hollow diamond at the whole	"has-a, part lives on"	Airline `1` ◇— `*` Aircraft
Composition	filled diamond at the whole	"owns, part dies with it"	Flight ◆— WeeklySchedule, Flight ◆— CustomSchedule, Flight ◆— FlightInstance
Generalization	solid line, hollow triangle at parent	"is-a"	Pilot, Crew, Admin → Person
Realization	dashed line, hollow triangle at interface	"implements"	(any `«interface»`)
Dependency	dashed line, open arrowhead	"uses"	FlightReservation ⇢ Payment

(The figure draws its one-way arrowheads filled; strict UML uses an open arrowhead — see the legend note in 3.1.)

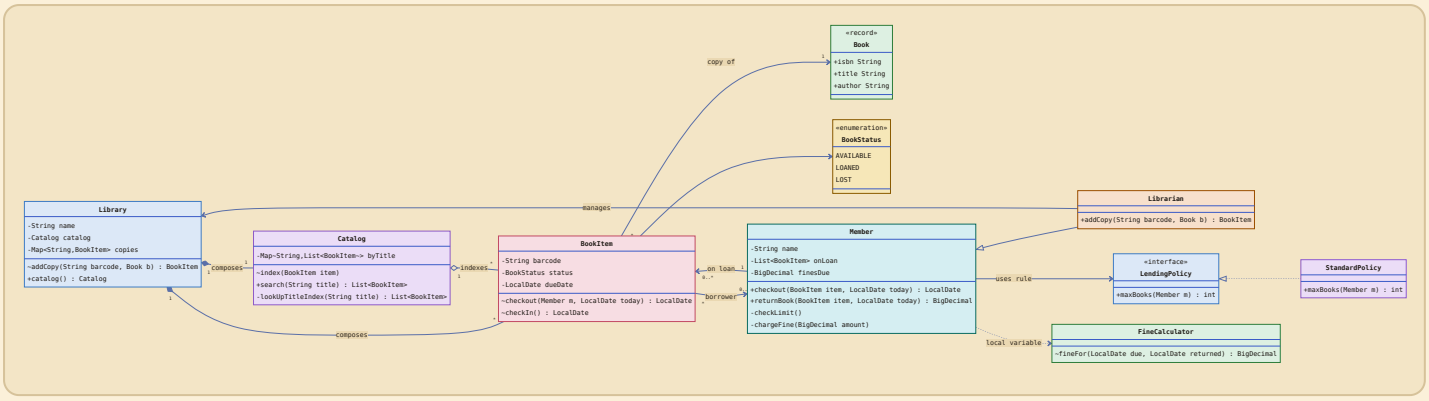
A note on reading the figure: some of its classes carry getters for the "other" direction (e.g. `Aircraft.getFlights()`) even though the line is drawn one-way. Strictly, such a getter makes the

association navigable both ways; treat the arrow as "the primary direction", and in your own diagrams keep arrows and fields consistent.

Two extra notations worth recognising:

- **Association class** — a class hanging off an association by a dashed line, holding data about the *link itself* (e.g. `Enrollment{grade}` on Student–Course).
- **Qualified association** — a small box on one end with a key (e.g. `Library [isbn] → Book`), meaning "look up by this key" — in Java a `Map<Isbn, Book>`.

Worked example — the library design (code in `lld.ch03.library`)



Every line type appears: composition (Library ◆ Catalog, Library ◆ BookItem), aggregation (Catalog ◇ BookItem — the index points at copies it does not own), associations with multiplicity, generalization (Librarian → Member), realization (StandardPolicy ⇢ LendingPolicy) and dependency (Member ⇢ FineCalculator). Visibility `~` marks package-private methods such as `BookItem.checkout`, which only `Member` (same package) may call. The Java code is listed in full in 3.12.

How much detail to draw

- **Every class gets a box and a name.** 10–15 boxes is a full answer.
- **Attributes only where they matter** — the two or three fields that carry the design.
- **Operations only where they are interesting** — the methods that appear in your flows.
- **Every line gets a relationship type and a multiplicity.** This is the part not to skip.
- If short of time, **drop attributes before relationships**. Boxes with only names but precise, labelled lines already describe a design; richly detailed boxes joined by vague lines do not.

Trade-offs

Detailed diagram	Sketch diagram
Near code-ready; fewer surprises later	Faster; easier to change during discussion
Takes 10+ minutes	Leaves details for the coding phase

When not to use it

It does not show *order* or *time* — use sequence/activity/state diagrams for behaviour.

Common mistakes

- Unlabelled lines (no type, no multiplicity).
- Diamonds/triangles at the wrong end.
- Listing every getter/setter and `String` field.
- Associations drawn in the diagram that do not exist as fields in the code (or vice versa).

Real-world examples

- IntelliJ IDEA (Ultimate), a popular Java IDE (code editor with build and run tools), can generate class diagrams from code; many teams use them to explore unfamiliar modules.

- JPA entity diagrams (ER-style, i.e. entity-relationship pictures of database tables) are essentially class diagrams with multiplicities (`@OneToMany` = `1..*`).

In the interview

> **30-second answer:** "I start with named boxes from the nouns and actors, then draw every line with a type and multiplicity, saying the lifetime/substitution test aloud. I add only the attributes and methods that drive the key flows. `+ - # ~` for visibility, italics or «abstract» for abstract, «interface» for interfaces."

Follow-ups:

- *Q: What does `#` mean and what is the Java nuance?* — A: Protected. In Java, protected is also visible to classes in the same package, which UML's `#` does not imply.
- *Q: How do you show a static method?* — A: Underline it (Mermaid: `$` suffix).

Self-check

- *Q: `~` means?* → A: Package visibility (Java's default, no modifier).
- *Q: What is the one test for aggregation vs composition?* → A: Imagine the whole is gone — is the part still meaningful by itself?

TL;DR

Box = name / attributes / operations with `+ - # ~`. Seven lines: association (± arrow), aggregation ◇, composition ◆, generalization (solid △), realization (dashed △), dependency (dashed arrow), plus multiplicity on every line.

3.12 Sequence diagrams

What it is

A **sequence diagram** shows **which object calls which, and in what order**, for one scenario. Objects run across the top; **time runs down the page**.

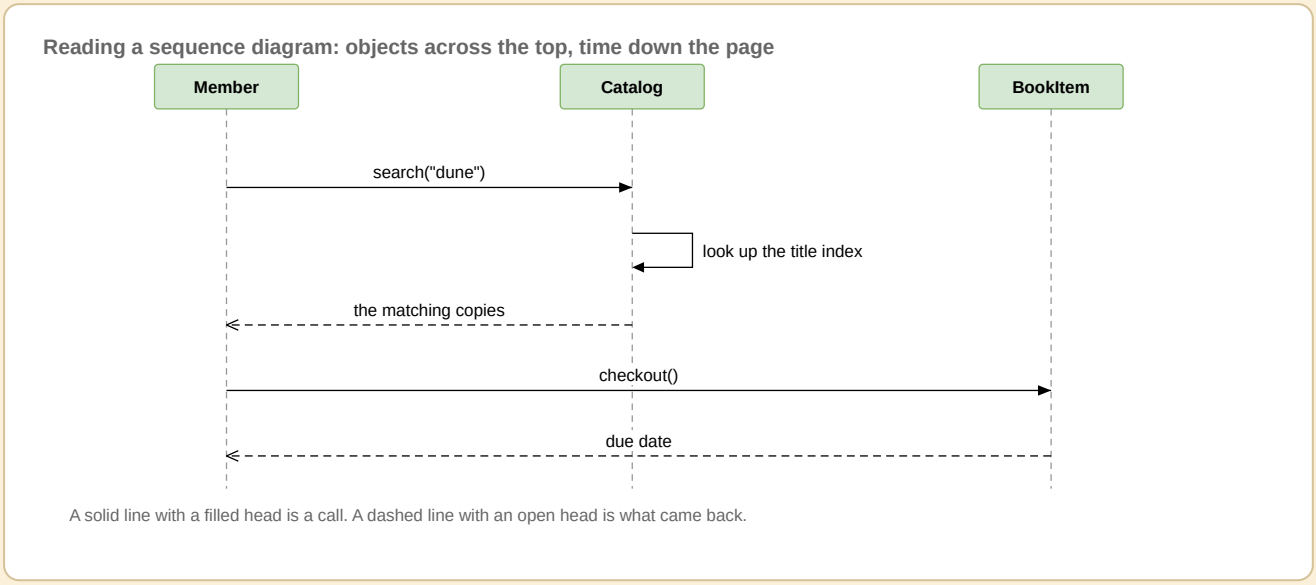


Figure — A member searches the catalog for "dune" and borrows a copy: lifelines, a self-call on Catalog, calls with filled arrowheads and dashed returns (course diagram)

The problem it solves

A class diagram says objects *can* talk; a sequence diagram shows what they *actually say* and in which order. It is the right diagram when the order is easy to get wrong (reserve seat → take payment →

confirm — any other order is a bug, and the diagram is the argument for this one), when several objects take part in one flow (two objects need no diagram; five usually do), or when you are explaining a class diagram you have already drawn.

How it works — the notation

Element	Drawn as	Meaning
Participant / lifeline	box at top + dashed vertical line	one *object* (strictly `catalog : Catalog`; most people just write the class name)
Actor	stick figure at top	a user or external system starting the flow
Synchronous message	solid line, **filled** arrowhead	a call; the caller waits for it to finish
Asynchronous message	solid line, **open** arrowhead	fire-and-forget; the caller continues (events, queues: lists of messages waiting to be processed)
Return (reply)	dashed line, open arrowhead	the value coming back — draw only when it matters
Self-message	arrow looping back to the same lifeline	the object calls its own method
Activation bar (execution specification)	thin rectangle on a lifeline	the object is busy executing
Create message	dashed arrow to the new object's box, often labelled «create»	the object is created at that moment
Destruction	**X** at the end of a lifeline	the object is destroyed
Combined fragment	labelled box around messages	control logic: see below

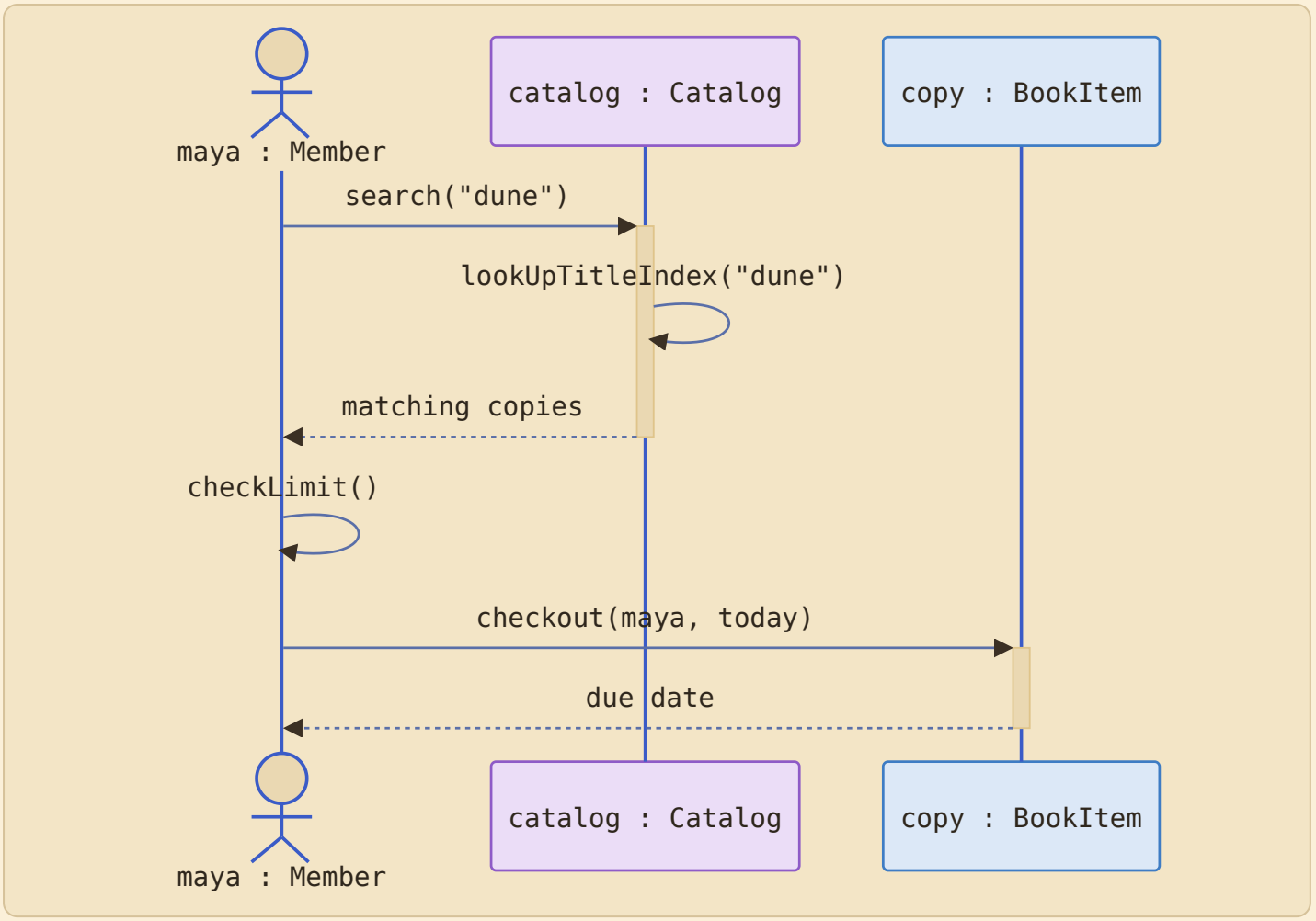
Combined fragments (the box's top-left label is the *operator*; conditions called *guards* are in `[ ]`):

Operator	Meaning	Java equivalent
`alt`	alternatives, one runs (sections split by dashed lines, each with a guard, last can be `[else]`)	`if / else if / else`
`opt`	optional, runs only if the guard is true	`if` without `else`
`loop`	repeats while the guard holds (`loop [for each item]`)	`for` / `while`
`par`	sections run in parallel	threads (independent lines of execution) / `CompletableFuture` (a result that arrives later from background work)
`break`	if guard true, run this and leave the enclosing fragment	early `return` / `break`
`critical`	atomic region (runs as one indivisible step), no interleaving	`synchronized` (Java keyword: one thread at a time) / lock
`ref`	reference to another diagram	calling a helper flow

(UML also has `neg`, `strict`, `seq`, `assert`, `ignore` and `consider`; you will not need them in an interview.)

On a whiteboard, activation bars and fragments are both optional and usually skipped: lifelines, calls, the returns that matter and self-calls are enough. They are shown in full here so you can read them when you meet them.

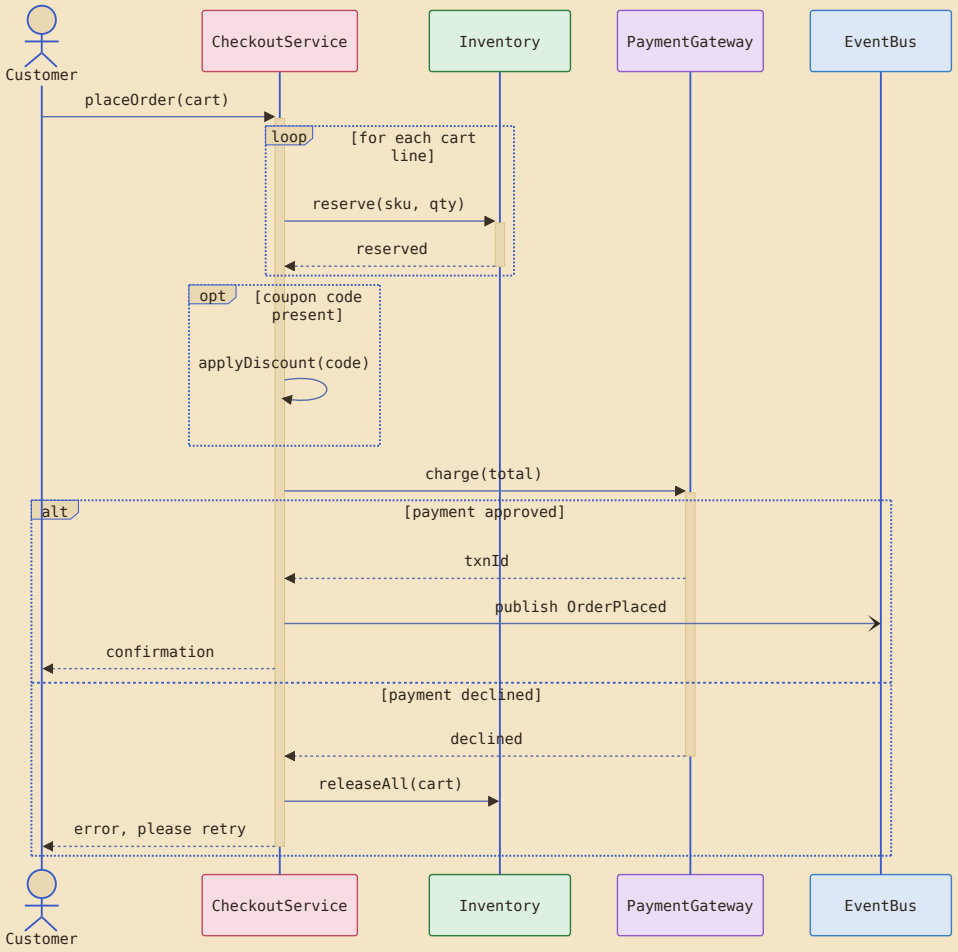
Example 1 — the library checkout, matching the Java code



Four things are worth noticing (the same as in the course figure above):

1. **The first message starts the flow** — a flow is always kicked off from outside the objects shown; here the member calls `search`.
2. **The catalog calls itself** — looking up the title index needs no other object, so the arrow loops back.
3. **Only two returns are drawn** — the copies and the due date; other calls return nothing worth naming.
4. **The member talks to the copy directly** — borrowing goes straight to `BookItem`, not back through `Catalog`. No other diagram shows this choice, and an interviewer can question it simply by tapping that arrow.

Example 2 — all fragments and async messages (checkout with payment)



Here `->>` is a synchronous call, `-->>` a reply, and `-)` an **asynchronous** message: publishing an event does not wait for subscribers. (An `EventBus` is an object that delivers each published event to everyone subscribed to it; `sku` is a stock-keeping unit, a product's ID. Queues and asynchronous messaging are explained in System Design Fundamentals.) The order is the point of the diagram: reserve stock *before* charging, and release it if the charge fails.

Java code for Example 1 (`lld.ch03.library`)

```java
// File: code/lld/ch03/library/Book.java
package lld.ch03.library;

/** The title (one per ISBN). Many physical copies (BookItem) point at one Book. */
public record Book(String isbn, String title, String author) { }
```

```java
// File: code/lld/ch03/library/BookStatus.java
package lld.ch03.library;

public enum BookStatus { AVAILABLE, LOANED, LOST }
```

```java
// File: code/lld/ch03/library/BookItem.java
package lld.ch03.library;

import java.time.LocalDate;
import java.util.Optional;

/** One physical copy. Association BookItem * --> 1 Book, and 0..1 borrower. */
public final class BookItem {
    static final int LOAN_DAYS = 14;

    private final String barcode;
    private final Book book;
    private BookStatus status = BookStatus.AVAILABLE;
    private Member borrower;      // 0..1
    private LocalDate dueDate;    // set only while LOANED

    public BookItem(String barcode, Book book) {
        this.barcode = barcode;
        this.book = book;
    }

    /** Called by Member. Returns the due date: the one return the sequence diagram draws. */
    LocalDate checkout(Member m, LocalDate today) {
        if (status != BookStatus.AVAILABLE) {
            throw new IllegalStateException(barcode + " is " + status);
        }
        status = BookStatus.LOANED;
        borrower = m;
        dueDate = today.plusDays(LOAN_DAYS);
        return dueDate;
    }

    LocalDate checkIn() {
        if (status != BookStatus.LOANED) throw new IllegalStateException(barcode + " is not on loan");
        LocalDate due = dueDate;
        status = BookStatus.AVAILABLE;
        borrower = null;
        dueDate = null;
        return due;
    }

    public String barcode() { return barcode; }
    public Book book() { return book; }
    public BookStatus status() { return status; }
    public Optional<Member> borrower() { return Optional.ofNullable(borrower); }

    @Override public String toString() { return barcode + "[" + book.title() + ", " + status + "]"; }
}
```

```java
// File: code/lld/ch03/library/Catalog.java
package lld.ch03.library;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Locale;
import java.util.Map;

/** An index over copies. It aggregates BookItems: it points at them but does not create or own them. */
public final class Catalog {
    private final Map<String, List<BookItem>> byTitle = new HashMap<>();

    void index(BookItem item) {
        byTitle.computeIfAbsent(key(item.book().title()), k -> new ArrayList<>()).add(item);
    }

    public List<BookItem> search(String title) {
        List<BookItem> hits = lookUpTitleIndex(title);   // self-call in the sequence diagram
        return List.copyOf(hits);
    }

    private List<BookItem> lookUpTitleIndex(String title) {
        return byTitle.getOrDefault(key(title), List.of());
    }

    private static String key(String title) { return title.toLowerCase(Locale.ROOT); }
}
```

```java
// File: code/lld/ch03/library/LendingPolicy.java
package lld.ch03.library;

/** Realized by concrete policies. Member depends on this interface, not on a class. */
public interface LendingPolicy {
    int maxBooks(Member m);
}
```

```java
// File: code/lld/ch03/library/StandardPolicy.java
package lld.ch03.library;

public final class StandardPolicy implements LendingPolicy {
    @Override public int maxBooks(Member m) { return m instanceof Librarian ? 10 : 2; }
}
```

```java
// File: code/lld/ch03/library/FineCalculator.java
package lld.ch03.library;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.time.temporal.ChronoUnit;

/** Used as a local helper inside Member.returnBook: a dependency, not a field. */
final class FineCalculator {
    private static final BigDecimal PER_DAY = new BigDecimal("5.00");

    BigDecimal fineFor(LocalDate due, LocalDate returned) {
        long late = ChronoUnit.DAYS.between(due, returned);
        return late > 0 ? PER_DAY.multiply(BigDecimal.valueOf(late)) : BigDecimal.ZERO.setScale(2);
    }
}
```

```java
// File: code/lld/ch03/library/Member.java
package lld.ch03.library;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

/**
 * Actor "Member" becomes a class; its use cases become methods:
 *   Borrow a book  -> checkout(...)  which always <<include>>s the limit check
 *   Return a book  -> returnBook(...) which is sometimes <<extend>>ed by "charge a fine"
 */
public class Member {
    private final String name;
    private final LendingPolicy policy;             // association to an interface
    private final List<BookItem> onLoan = new ArrayList<>();
    private BigDecimal finesDue = BigDecimal.ZERO.setScale(2);   // money: BigDecimal, not double

    public Member(String name, LendingPolicy policy) {
        this.name = name;
        this.policy = policy;
    }

    public LocalDate checkout(BookItem item, LocalDate today) {
        checkLimit();                               // <<include>>: happens every time
        LocalDate due = item.checkout(this, today);
        onLoan.add(item);
        return due;
    }

    public BigDecimal returnBook(BookItem item, LocalDate today) {
        if (!onLoan.remove(item)) throw new IllegalStateException(name + " does not hold " + item.barcode());
        LocalDate due = item.checkIn();
        BigDecimal fine = new FineCalculator().fineFor(due, today);
        if (fine.signum() > 0) chargeFine(fine);       // <<extend>>: only when late
        return fine;
    }

    private void checkLimit() {
        int max = policy.maxBooks(this);
        if (onLoan.size() >= max) {
            throw new IllegalStateException(name + " already has " + max + " books (limit)");
        }
    }

    private void chargeFine(BigDecimal amount) { finesDue = finesDue.add(amount); }

    public String name() { return name; }
    public BigDecimal finesDue() { return finesDue; }
    public List<BookItem> onLoan() { return Collections.unmodifiableList(onLoan); }
}
```

```java
// File: code/lld/ch03/library/Librarian.java
package lld.ch03.library;

/** Generalization between actors: a librarian can do everything a member can, plus more. */
public final class Librarian extends Member {
    private final Library library;

    public Librarian(String name, LendingPolicy policy, Library library) {
        super(name, policy);
        this.library = library;
    }

    public BookItem addCopy(String barcode, Book book) { return library.addCopy(barcode, book); }
}
```

```java
// File: code/lld/ch03/library/Library.java
package lld.ch03.library;

import java.util.LinkedHashMap;
import java.util.Map;

/**
 * The system boundary made concrete. Library composes its Catalog (created here,
 * never shared) and composes its copies (only a Library creates a BookItem it registers).
 */
public final class Library {
    private final String name;
    private final Catalog catalog = new Catalog();                  // composition
    private final Map<String, BookItem> copies = new LinkedHashMap<>();

    public Library(String name) { this.name = name; }

    BookItem addCopy(String barcode, Book book) {
        BookItem item = new BookItem(barcode, book);
        copies.put(barcode, item);
        catalog.index(item);
        return item;
    }

    public Catalog catalog() { return catalog; }
    public String name() { return name; }
}
```

```java
// File: code/lld/ch03/library/Main.java
package lld.ch03.library;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        LendingPolicy policy = new StandardPolicy();
        Library lib = new Library("Town Library");
        Librarian lee = new Librarian("Lee", policy, lib);

        Book dune = new Book("978-0441013593", "Dune", "Frank Herbert");
        Book hobbit = new Book("978-0547928227", "The Hobbit", "J.R.R. Tolkien");
        lee.addCopy("B-1", dune);
        lee.addCopy("B-2", dune);
        lee.addCopy("B-3", hobbit);
        lee.addCopy("B-4", hobbit);

        Member maya = new Member("Maya", policy);
        LocalDate day1 = LocalDate.of(2026, 3, 1);

        // Sequence: member -> catalog.search -> (self) lookUpTitleIndex -> copies; member -> copy.checkout -> due
        List<BookItem> hits = lib.catalog().search("dune");
        System.out.println("search(\"dune\") -> " + hits);
        LocalDate due = maya.checkout(hits.get(0), day1);
        System.out.println("Maya borrowed B-1, due " + due);
        maya.checkout(lib.catalog().search("the hobbit").get(0), day1);
        System.out.println("Maya on loan: " + maya.onLoan());

        try {
            maya.checkout(hits.get(1), day1);   // <<include>> limit check fires
        } catch (IllegalStateException e) {
            System.out.println("Blocked: " + e.getMessage());
        }

        try {
            new Member("Omar", policy).checkout(hits.get(0), day1);   // copy already LOANED
        } catch (IllegalStateException e) {
            System.out.println("Blocked: " + e.getMessage());
        }

        BigDecimal onTime = maya.returnBook(hits.get(0), day1.plusDays(10));
        System.out.println("Returned B-1 on time, fine = " + onTime);
        BookItem hobbitCopy = maya.onLoan().get(0);
        BigDecimal late = maya.returnBook(hobbitCopy, day1.plusDays(17));   // <<extend>> fine applies
        System.out.println("Returned " + hobbitCopy.barcode() + " 3 days late, fine = " + late
                + ", Maya owes " + maya.finesDue());
        System.out.println("Librarian limit: " + policy.maxBooks(lee) + ", member limit: " +
policy.maxBooks(maya));
    }
}
```

Output:

```
search("dune") -> [B-1[Dune, AVAILABLE], B-2[Dune, AVAILABLE]]
Maya borrowed B-1, due 2026-03-15
Maya on loan: [B-1[Dune, LOANED], B-3[The Hobbit, LOANED]]
Blocked: Maya already has 2 books (limit)
Blocked: B-1 is LOANED
Returned B-1 on time, fine = 0.00
Returned B-3 3 days late, fine = 15.00, Maya owes 15.00
Librarian limit: 10, member limit: 2
```

Check the diagram against the code: every arrow is a real method — `Catalog.search`, `Catalog.lookUpTitleIndex`, `Member.checkLimit`, `BookItem.checkout`. **If a message has no method, your class diagram is missing one** — the same check the activity diagram does, from another direction.

Trade-offs

Good at	Poor at
Order of calls, who talks to whom	Branch-heavy flows (nested `alt` becomes unreadable)
Showing sync vs async	Showing the whole design
Spotting missing methods	Being quick to draw — each message is a line

When not to use it

- For a flow with three or more decisions: use an **activity diagram**. A second `alt`/`opt` box on the same diagram is the warning sign.
- Unprompted in most OOD interviews — it costs time you need for the class diagram.

Common mistakes

- Drawing a return for every call — the diagram doubles in height and says nothing more.
- Messages that are not methods on the receiving class.
- Mixing up arrowheads: filled = sync call, open solid = async, dashed = reply.
- Drawing classes instead of the specific objects when two objects of the same class take part (e.g. `from : Account`, `to : Account`).

Real-world examples

- **OAuth 2.0 / OpenID Connect** (standard protocols for "log in with another account") login flows (browser → app → authorization server → token endpoint) are almost always documented as sequence diagrams.
- **Payment integrations** (Stripe, Razorpay) document "create order → redirect → webhook" (a webhook is an HTTP call the provider makes to your server when something happens) as sequence diagrams, with the webhook as an asynchronous message.
- **Microservice sagas** (microservices are small, separately deployed services; a saga is a multi-step transaction in which every step has an "undo"; e.g. a travel booking that reserves a flight, a hotel and a car) are usually explained with sequence diagrams that show the compensating "undo" calls when one step fails.

In the interview

> **30-second answer:** "A sequence diagram shows one scenario: objects across the top, time downwards. Solid filled arrows are sync calls, open arrows async, dashed arrows returns. `alt` / `opt` / `loop` fragments show if/else, if, and loops. I keep it to three or four objects and six to eight messages, and check that every arrow is a method on my class diagram."

Follow-ups:

- *Q: Sync vs async message?* — A: Sync: caller blocks until the callee returns (a normal method call). Async: caller continues immediately (a queue message, event, `CompletableFuture`).
- *Q: When would you use a sequence diagram over an activity diagram?* — A: When the interesting part is *who calls whom* in what order; activity when the interesting part is branching.

Self-check

- *Q: What does a thin rectangle on a lifeline mean?* → A: An activation bar — the object is executing.
- *Q: Which fragment models "retry up to 3 times"?* → A: `loop [attempt <= 3]`.
- *Q: How do you show that `Order` creates a `Payment` mid-flow?* → A: A create message (dashed arrow, «create») pointing to the new lifeline's box, which starts lower on the page.

TL;DR

Objects across, time down. Filled = sync call, open = async, dashed = return. `alt` = if/else, `opt` = if, `loop` = loop, `par` = parallel. Keep it small and check each arrow is a real method.

3.13 Activity diagrams

What it is

An **activity diagram** traces **a single flow end to end**, marking each point where it **branches** (and any steps that run in parallel). Where a class diagram captures the design at rest, an activity diagram follows one route through it. Among UML diagrams it is the closest thing to a flowchart, so nearly everyone can follow it — which also makes a careless one easy to produce.

The problem it solves

It makes the business logic of a flow — every `if`, `unless`, `otherwise` in the requirements — visible and checkable. Its best interview use is as a **test of your class diagram**: walk the flow action by action and ask *which class does this?* If an action has no home, you have found a missing class — on the whiteboard, not in the code.

How it works — the symbols

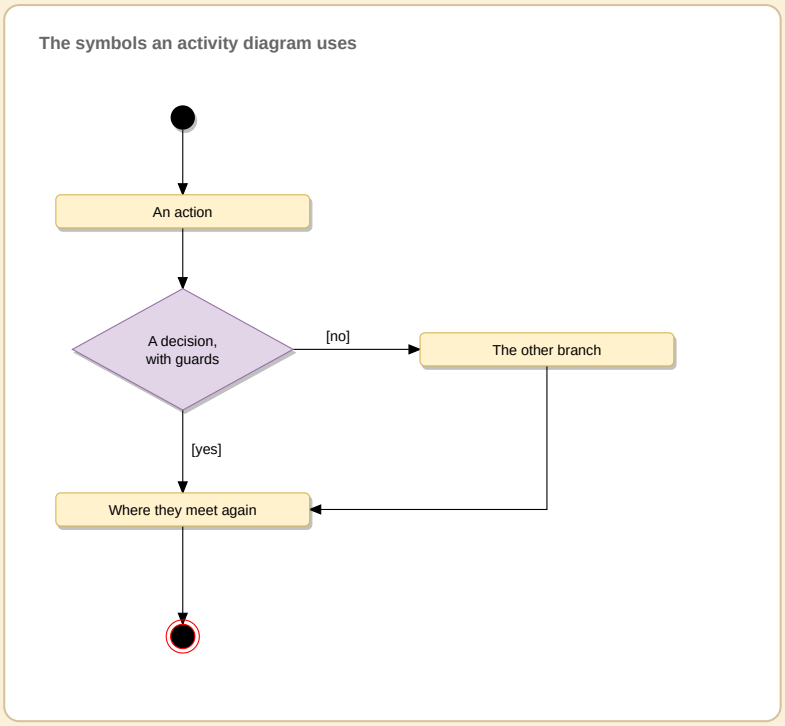


Figure — The core activity-diagram symbols: start, action, a decision with [yes]/[no] guards, the other branch, where the branches meet again, and the end (course diagram)

Symbol	Drawn as	Meaning
Initial node (start)	filled black circle	where the flow begins (draw exactly one; UML technically allows several, but one keeps the diagram readable)
Action	rounded rectangle with a verb phrase	one step: "System calculates the parking fee"
Control flow	arrow	time runs along the arrows
Decision	diamond, each outgoing arrow has a **guard** `[condition]`	a branch; guards must cover every case (use `[else]`)
Merge	diamond (or arrows meeting one node)	branches come back together
Fork	thick bar, one arrow in, many out	the following actions run **in parallel**
Join	thick bar, many in, one out	wait until **all** parallel actions finish
Activity final (end)	filled circle inside a ring (bullseye)	the whole activity stops; there may be several ("payment failed", "gate opened")
Flow final	circle with an X	only *this* branch stops; others continue
Swimlane (partition)	vertical or horizontal lanes labelled with an actor/class	*who* performs each action
Object node	rectangle	data passed between actions (e.g. `[Ticket]`)
Send / receive signal	convex / concave pentagon	sending or waiting for an event

Steps to draw one:

1. Pick **one** flow, ideally the one with an interesting branch. A requirement containing "if", "unless" or "otherwise" describes a branch — draw that.
2. Start node → actions in the language of the problem ("The system calculates the fee", **not** "call `calculateFee()`").
3. At each decision, draw every outcome with a guard.
4. Use fork/join only when steps really happen at the same time. Most interview flows do not need them.
5. Add swimlanes if it helps to show which actor/class does each step.
6. End node(s).

Example 1 — online shopping (course)

Every activity diagram has the shape of the small symbols figure above, and it reads as a single sentence: *begin, perform an action, ask a question, follow one of two paths, merge, end.* Here is a bigger example:

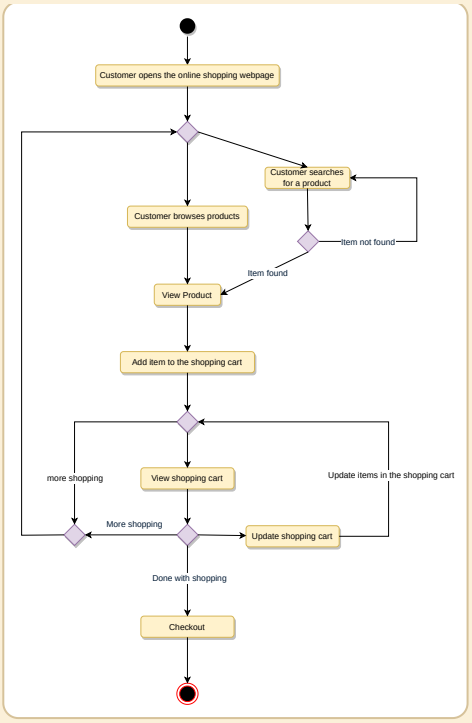
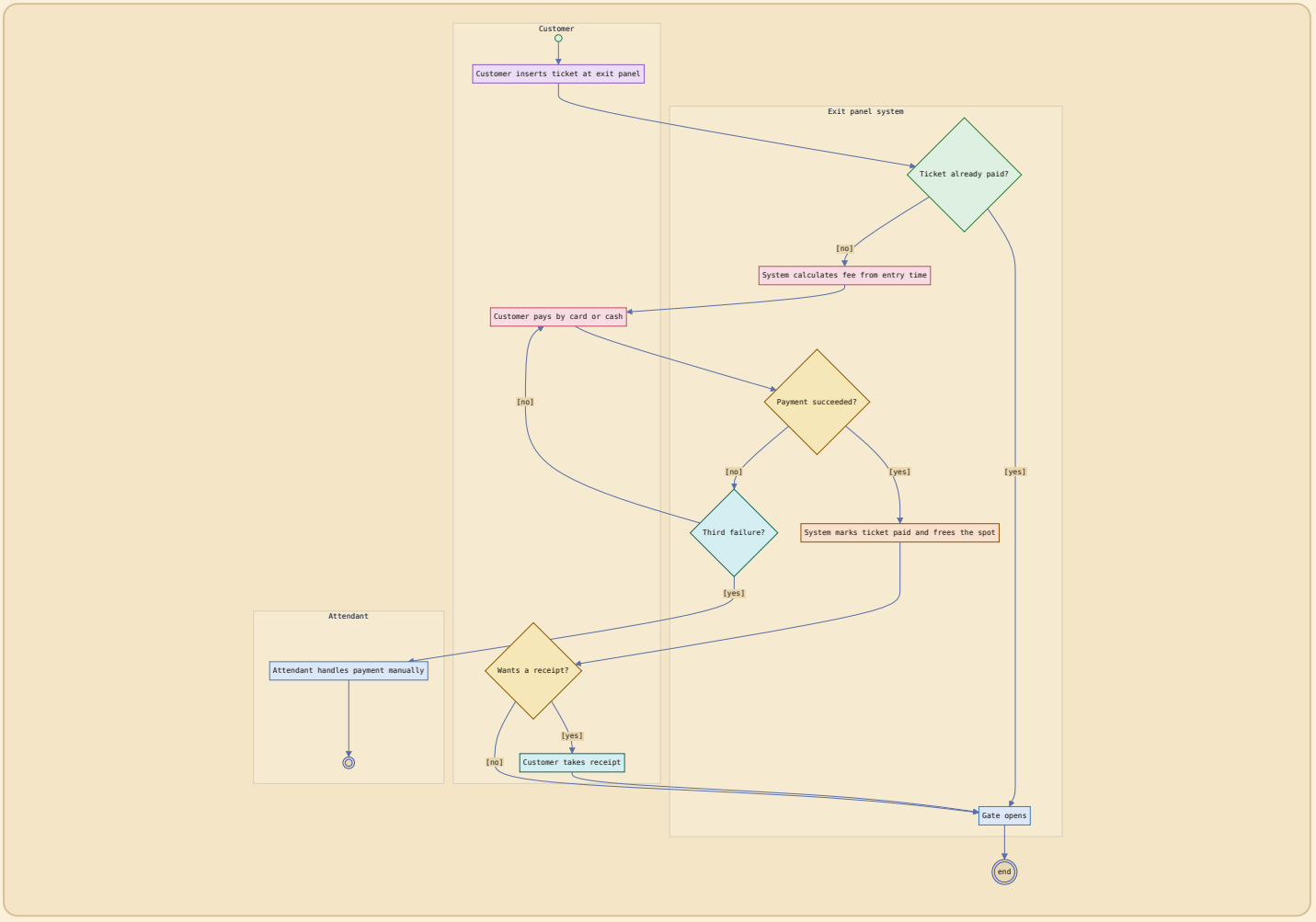


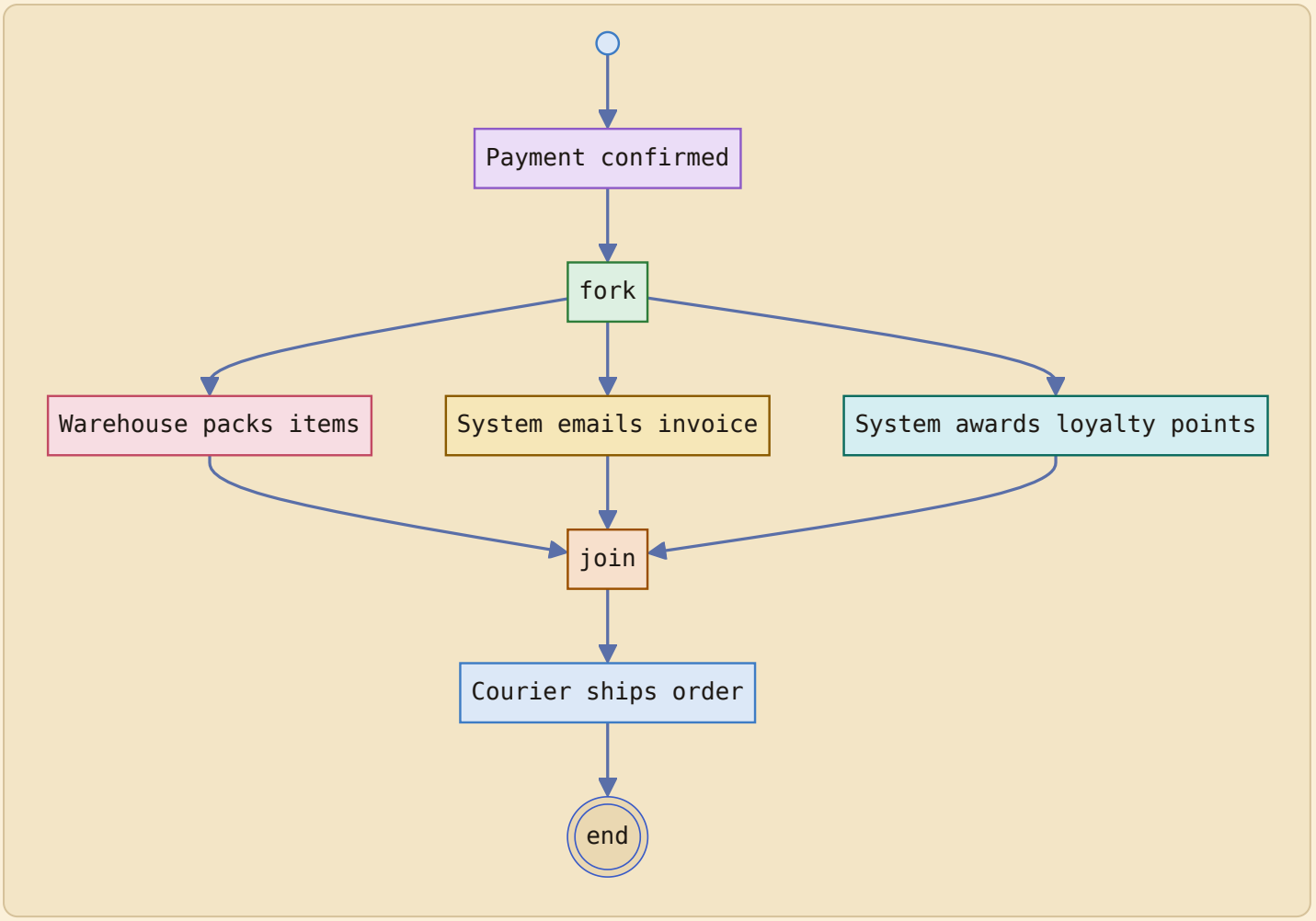
Figure — A customer shopping online: search, item found or not found, add to cart, view/update the cart, more shopping or checkout (course diagram)

Example 2 — pay at the parking exit (three decisions, with swimlanes)

Three decisions appear in the Parking Lot case study (Chapter 14): *is the ticket paid already?*, *did payment go through?* and *does the customer ask for a receipt?* Every one traces back to a requirement. Below it is drawn with swimlanes (subgraphs), a retry loop, and a hand-off to an attendant once three attempts fail:



Example 3 — fork and join (order fulfilment)



After the **fork** bar the three actions happen in parallel; the **join** waits for all three before shipping.

Activity vs sequence diagram

	Activity diagram	Sequence diagram
Shows	a process as a series of steps	objects sending messages to each other
Branches	natural: decisions with guards	awkward: nested `alt` fragments
Names	actions, in business language	objects and method calls
Use when	the flow has interesting branches	the order of calls is the interesting part
Answers	"what happens?"	"who calls whom?"

Rule of thumb: a flow with **three branches** calls for an activity diagram; **four objects** whose call order is easy to confuse call for a sequence diagram.

Trade-offs

Pro	Con
Anyone (including non-engineers) can read it	Does not show which object does what unless you add swimlanes
Great at branches and parallelism	Big flows sprawl quickly

When not to use it

For a straight-line flow ("customer pays and leaves") — there is nothing to decide, so the picture adds nothing. For object-to-object call order, use a sequence diagram.

Common mistakes

- **A diamond with only one exit.** If only one outcome is possible, nothing is being decided — draw the missing branch or remove the diamond.
- **Actions written as code** ("call `calculateFee()`"). Keep it in the language of the problem so it can be checked against the requirements.
- Guards that do not cover every case (`[amount > 100]` and `[amount < 100]` — what about exactly 100?).
- Using fork/join for steps that are merely sequential.

Real-world examples

- **BPMN** (Business Process Model and Notation), used by workflow engines (software that runs business processes step by step) such as Camunda, is a close cousin of UML activity diagrams, with lanes and gateways.
- **CI/CD pipelines** (Continuous Integration / Continuous Delivery: automatically building, testing and deploying code on every change; e.g. GitHub Actions, Jenkins) are activity diagrams in practice: parallel jobs (fork), a job that `needs:` several others (join), conditional steps (guards).
- Food-delivery apps (Swiggy, Zomato, DoorDash) typically let the restaurant prepare the food while a delivery partner is being assigned — two parallel branches that join at "order picked up".

In the interview

> **30-second answer:** "An activity diagram walks one flow: start, actions, decisions with guards covering every case, merges, optional fork/join for parallel steps, swimlanes for who does what, and one or more ends. I draw the flow with the interesting branch, then check that every action has a class to live in."

Follow-ups:

- *Q: Activity final vs flow final?* — A: Activity final ends the whole activity; flow final ends only that one path while other parallel paths continue.
- *Q: Which flow do you draw?* — A: The one whose requirement contains "if/unless/otherwise" — e.g. payment failure and retry.

Self-check

- *Q: What must every arrow leaving a decision have?* → A: A guard in square brackets; together they must cover all cases.
- *Q: How do you show two steps that happen at the same time?* → A: A fork bar before them and a join bar after them.

TL;DR

One flow, start → actions → decisions with guards → merge → end. Fork/join for parallel, swimlanes for who. Draw the branching flow and use it to test your class diagram.

3.14 State machine diagrams

What it is

A **state machine diagram** (also called a *statechart* or *state diagram*) shows the **states one object can be in** during its life and the **transitions** that move it from one state to another in response to **events**.

- **State** — a condition during which the object behaves in a certain way or waits (rounded rectangle): `CREATED`, `PAID`, `SHIPPED`.
- **Initial pseudostate** — filled circle, arrow to the first state.
- **Final state** — bullseye; the object's life (in this machine) is over.
- **Transition** — arrow labelled `event [guard] / action`:

    - **event (trigger)** — what happens: `pay(amount)`.
    - **guard** — boolean condition that must be true for the transition to fire: `[amount == total]`.
    - **action (effect)** — what the object does while transitioning: `/ refund()`.
- **Internal activities** inside a state: `entry / ...` (on entering), `exit / ...` (on leaving), `do / ...` (while in the state).
- **Self-transition** — an arrow from a state back to itself (exit and re-entry actions run).
- **Choice pseudostate** — a small diamond that picks the next state based on guards.
- **Composite state** — a state containing sub-states (e.g. `Active` containing `Idle` and `Moving`); **history** `(H)` remembers the last sub-state.
- **Orthogonal regions** — a state split by dashed lines into parts that are active at the same time.

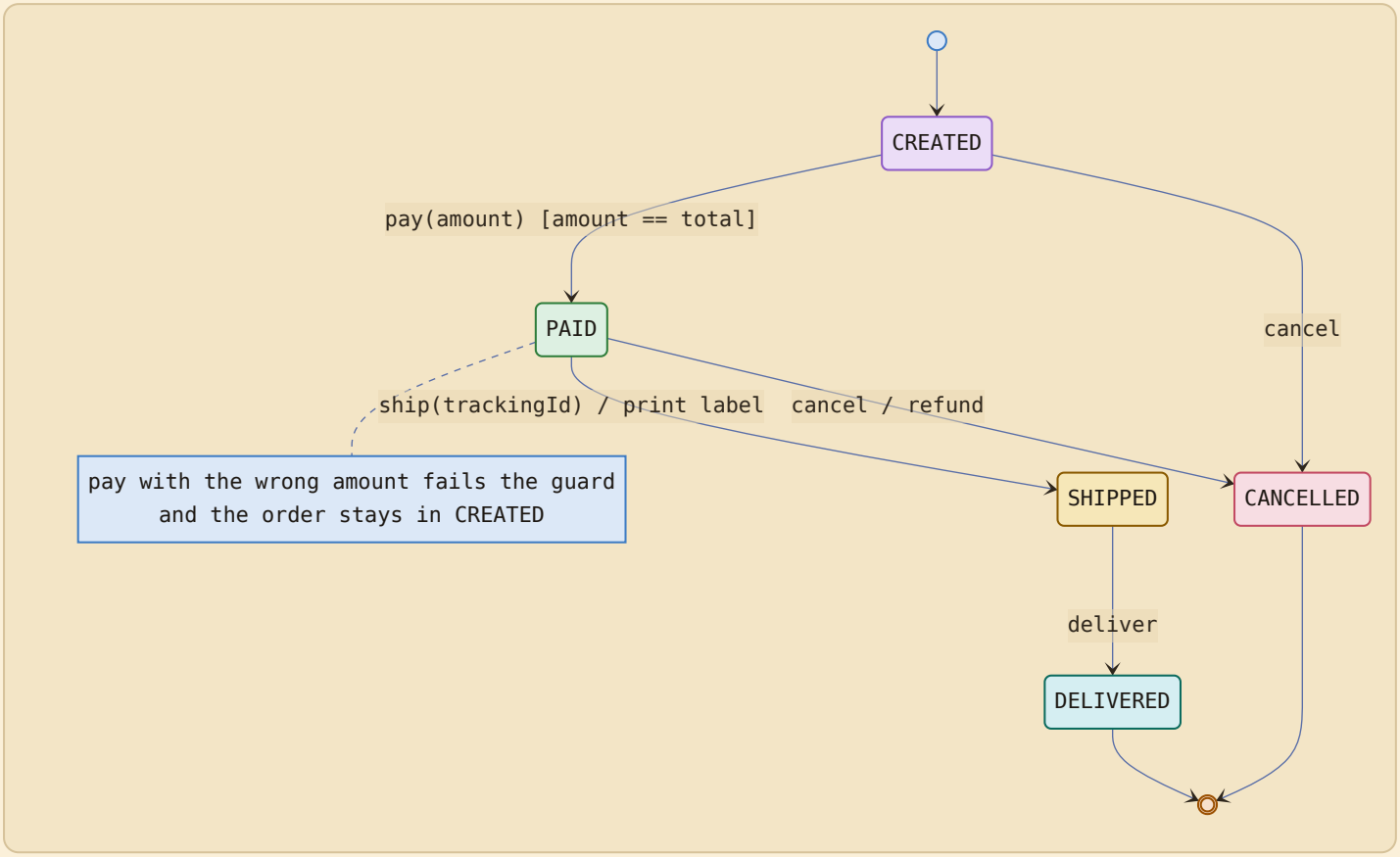
Analogy: a **traffic light**. It is always in exactly one state (red, green, yellow), a timer event moves it on, and the allowed order is fixed — it never goes straight from green to red.

The problem it solves

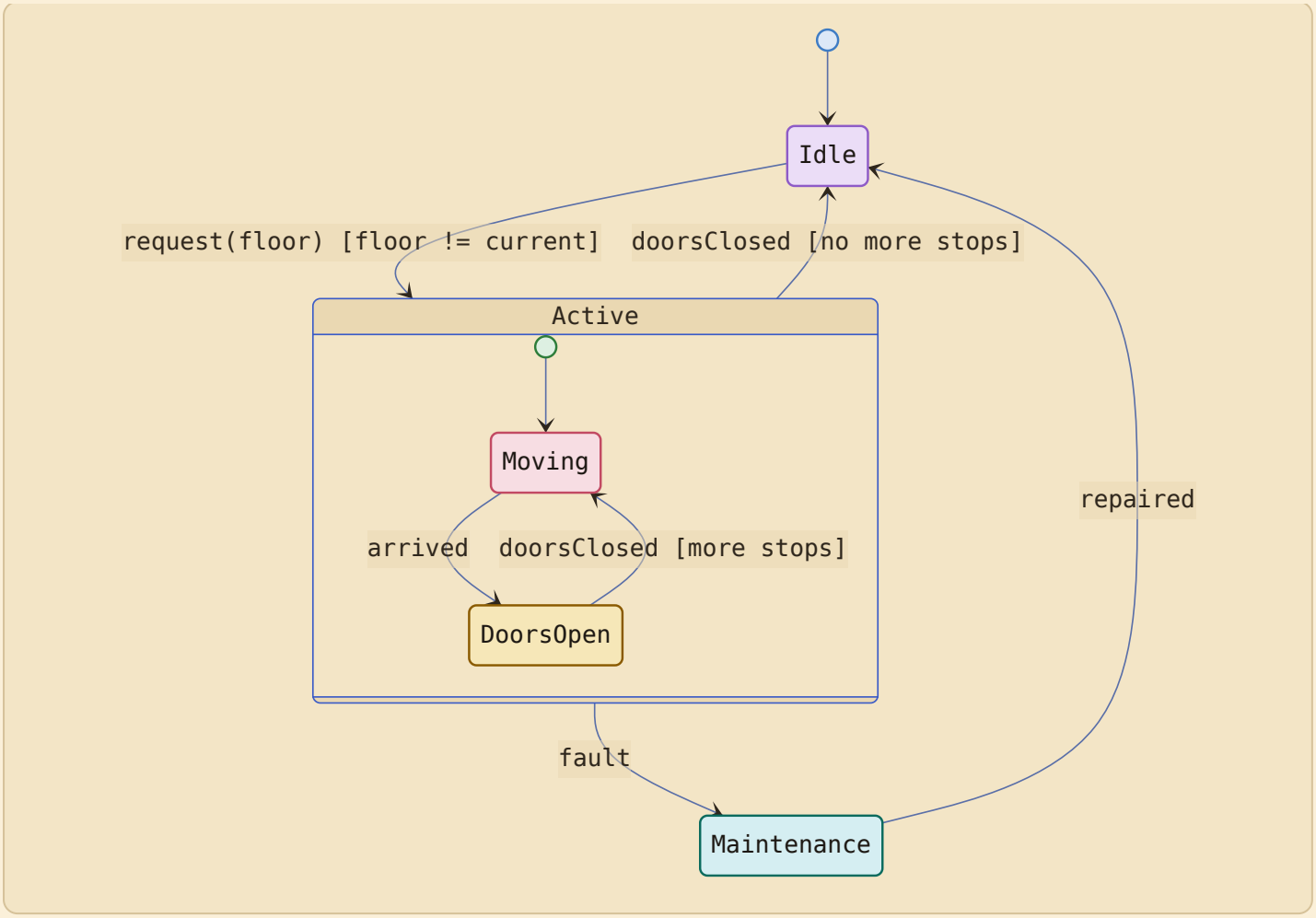
Objects with a lifecycle (orders, tickets, elevators, vending machines, parking spots, TCP connections, TCP being the network protocol for reliable connections, see System Design Fundamentals) are the source of many bugs: shipping a cancelled order, paying twice, refunding an undelivered item. A state diagram lists **every legal transition** so that everything else can be rejected. It also maps directly to code (an `enum` of states plus a transition method, or the **State pattern**, Chapter 11).

How it works

1. Pick **one** class with a lifecycle.
2. List its states (usually an `enum`).
3. For each state, list the events it accepts, the guard (if any), the target state and the action.
4. Everything not listed is **illegal** — the code must reject it.
5. Mark the initial and final states.



A composite-state example (an elevator car) for recognition:



Java example (`lld.ch03.statemachine`)

```java
// File: code/lld/ch03/statemachine/OrderState.java
package lld.ch03.statemachine;

public enum OrderState { CREATED, PAID, SHIPPED, DELIVERED, CANCELLED;
    public boolean isFinal() { return this == DELIVERED || this == CANCELLED; }
}
```

```java
// File: code/lld/ch03/statemachine/OrderEvent.java
package lld.ch03.statemachine;

import java.math.BigDecimal;

/** Events that can trigger transitions. Sealed, so the switch in Order is checked for completeness. */
public sealed interface OrderEvent {
    record Pay(BigDecimal amount) implements OrderEvent { }
    record Ship(String trackingId) implements OrderEvent { }
    record Deliver() implements OrderEvent { }
    record Cancel(String reason) implements OrderEvent { }
}
```

```java
// File: code/lld/ch03/statemachine/Order.java
package lld.ch03.statemachine;

import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.List;

/**
 * Code for the state machine diagram. Each transition is written as
 *   event [guard] / action
 * An event that has no transition from the current state is rejected.
 */
public final class Order {
    private final String id;
    private final BigDecimal total;   // money: BigDecimal, not double
    private OrderState state = OrderState.CREATED;   // initial pseudostate -> CREATED
    private final List<String> log = new ArrayList<>();

    public Order(String id, BigDecimal total) {
        this.id = id;
        this.total = total;
        log.add("[*] -> CREATED");
    }

    public OrderState state() { return state; }
    public List<String> log() { return List.copyOf(log); }

    public void handle(OrderEvent event) {
        OrderState from = state;
        OrderState to = switch (event) {
            case OrderEvent.Pay p when state == OrderState.CREATED && p.amount().compareTo(total) == 0 ->
OrderState.PAID;
            case OrderEvent.Pay p when state == OrderState.CREATED ->
                    reject(event, "guard failed: paid " + p.amount() + " but total is " + total);
            case OrderEvent.Ship s when state == OrderState.PAID -> {
                log.add("  action: print label " + s.trackingId());
                yield OrderState.SHIPPED;
            }
            case OrderEvent.Deliver d when state == OrderState.SHIPPED -> OrderState.DELIVERED;
            case OrderEvent.Cancel c when state == OrderState.CREATED -> OrderState.CANCELLED;
            case OrderEvent.Cancel c when state == OrderState.PAID -> {
                log.add("  action: refund " + total);
                yield OrderState.CANCELLED;
            }
            default -> reject(event, "no transition from " + state);
        };
        state = to;
        log.add(from + " --" + event.getClass().getSimpleName() + "--> " + to);
    }

    private OrderState reject(OrderEvent e, String why) {
        throw new IllegalStateException(id + ": " + e.getClass().getSimpleName() + " rejected, " + why);
    }
}
```

```java
// File: code/lld/ch03/statemachine/Main.java
package lld.ch03.statemachine;

import java.math.BigDecimal;

public class Main {
    public static void main(String[] args) {
        Order happy = new Order("ORD-1", new BigDecimal("499.00"));
        happy.handle(new OrderEvent.Pay(new BigDecimal("499.00")));
        happy.handle(new OrderEvent.Ship("TRK-77"));
        happy.handle(new OrderEvent.Deliver());
        System.out.println("ORD-1 path:");
        happy.log().forEach(l -> System.out.println("  " + l));
        System.out.println("  final? " + happy.state().isFinal());

        Order refund = new Order("ORD-2", new BigDecimal("120.00"));
        tryEvent(refund, new OrderEvent.Pay(new BigDecimal("100.00")));   // guard fails
        tryEvent(refund, new OrderEvent.Pay(new BigDecimal("120.00")));
        tryEvent(refund, new OrderEvent.Cancel("changed mind"));
        tryEvent(refund, new OrderEvent.Ship("TRK-99"));                        // no transition from CANCELLED
        System.out.println("ORD-2 path:");
        refund.log().forEach(l -> System.out.println("  " + l));
    }

    private static void tryEvent(Order o, OrderEvent e) {
        try {
            o.handle(e);
        } catch (IllegalStateException ex) {
            System.out.println("Rejected: " + ex.getMessage());
        }
    }
}
```

What each part shows: `OrderState` is the list of states. Amounts are `BigDecimal` (never `double` for money), so the guard `[amount == total]` is written `p.amount().compareTo(total) == 0` — `BigDecimal.equals` would treat `120.0` and `120.00` as different because it also compares scale. `OrderEvent` is a **sealed** interface (only the types listed inside it may implement it) of event records, so the `switch` in `Order.handle` uses Java 21 **pattern matching with `when` guards** — each `case` line is literally one arrow of the diagram: `event [guard] -> target` with its action. The `default` branch rejects every transition that is not on the diagram.

Output:

```
ORD-1 path:
  [*] -> CREATED
  CREATED --Pay--> PAID
    action: print label TRK-77
  PAID --Ship--> SHIPPED
  SHIPPED --Deliver--> DELIVERED
  final? true
Rejected: ORD-2: Pay rejected, guard failed: paid 100.00 but total is 120.00
Rejected: ORD-2: Ship rejected, no transition from CANCELLED
ORD-2 path:
  [*] -> CREATED
  CREATED --Pay--> PAID
    action: refund 120.00
  PAID --Cancel--> CANCELLED
```

Trade-offs

enum + switch (this example)	State pattern (class per state, Ch. 11)
All transitions visible in one place	Each state's behaviour in its own class
Best for few states, simple actions	Best when states have lots of different behaviour
Adding a state edits the switch	Adding a state adds a class (Open/Closed)

When not to use it

For objects with only a flag or two and no real lifecycle (`active/inactive`). For flows that span many objects, use an activity diagram instead — a state machine describes **one** object.

Common mistakes

- Confusing states with actions: "Paying" (an action) vs "PAID" (a state).
- Missing transitions (e.g. forgetting that a paid order can be cancelled with a refund) or allowing illegal ones by default.
- Guards that overlap, so two transitions could fire for the same event.
- Drawing a state machine for the *system* rather than for one object.

Real-world examples

- `java.lang.Thread.State`: `NEW`, `RUNNABLE`, `BLOCKED`, `WAITING`, `TIMED_WAITING`, `TERMINATED`.
- **TCP** connection states (`LISTEN`, `SYN_SENT`, `ESTABLISHED`, `FIN_WAIT_1`, …) — the classic protocol state machine.
- **Amazon / Flipkart order tracking** (ordered → packed → shipped → out for delivery → delivered; cancel allowed only before shipping).
- **Uber trip** (requested → driver assigned → driver arrived → in progress → completed / cancelled).
- **Spring Statemachine** and AWS Step Functions (Amazon's cloud workflow service) are libraries/services built around state machines.

In the interview

> **30-second answer:** "When an object has a lifecycle — order, ticket, elevator, vending machine — I draw its states with transitions labelled `event [guard] / action`. Anything not on the diagram is illegal and the code rejects it. In Java it is an enum of states with one transition method, or the State pattern if each state has a lot of behaviour."

Follow-ups:

- *Q: What is a guard?* — A: A boolean condition in `[ ]` that must be true for the transition to fire, e.g. `[amount == total]`.
- *Q: State machine vs activity diagram?* — A: A state machine is about one object's states over its lifetime; an activity diagram is about the steps of a process, possibly across many objects.
- *Q: How do you make transitions thread-safe?* — A: Make the transition method `synchronized` (or use a lock / compare-and-set on an `AtomicReference<State>`; compare-and-set changes a value only if it still equals what you last read, in one atomic step), so two events cannot both read the old state and both move on.

Self-check

- *Q: Read* `PAID --> CANCELLED : cancel / refund`. → A: In PAID, the event `cancel` moves to CANCELLED and performs the action `refund`.
- *Q: What happens to an event that has no transition from the current state?* → A: It is ignored or rejected — in our code, rejected with an exception.

TL;DR

One object, its states, and `event [guard] / action` arrows. Initial dot, final bullseye. Everything not drawn is illegal. Code: enum + guarded switch, or the State pattern.

Part C — UML in the interview

3.15 How much UML to draw in an interview

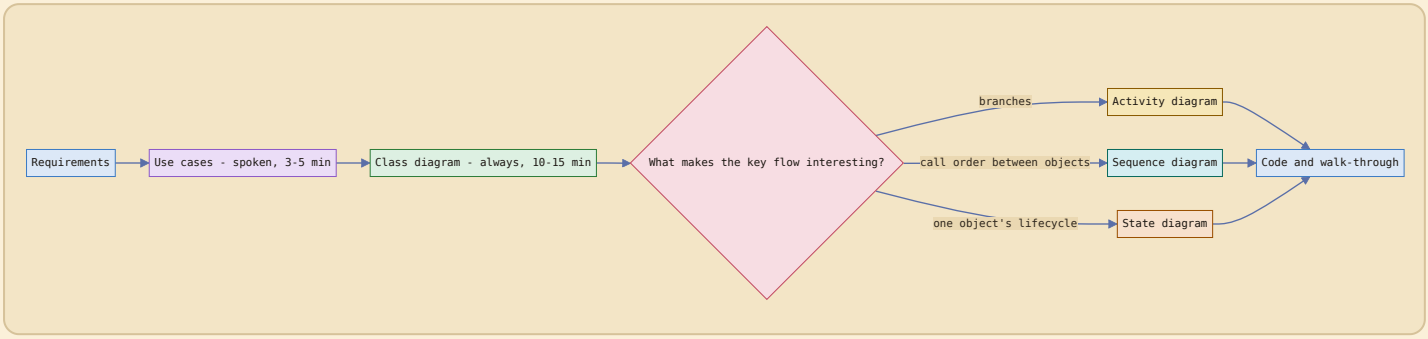
The interviewer is not marking your arrowheads. What is checked is whether **your diagram and your words agree**. Leaving a line out costs you less than drawing one you cannot justify.

Suggested budget for a 45-minute OOD round:

Minute	Diagram	Depth
0–5	**Use case**	Usually *spoken* or a quick list: actors + what each does + what is out of scope (the boundary). Draw it only if asked.
5–20	**Class diagram**	*Always drawn.* 10–15 boxes, every line typed with multiplicity; key attributes and methods only.
20–25	**Activity** *or* **sequence** *or* **state**	One diagram for the one flow that carries the design: activity if it branches, sequence if call order matters, state if one object has a lifecycle.
25–45	Code + discussion	Code the core classes; walk the flow against your diagrams.

Rules of thumb

1. **If you only have one picture, make it the class diagram.** It is the answer.
2. **Drop attributes before relationships** when short of time.
3. **Never leave a line unlabelled** — type + multiplicity.
4. **Say the test aloud** for every diamond and triangle.
5. **Use the other diagrams as tests:** every action in the activity diagram and every arrow in the sequence diagram must be a method on a class in the class diagram.
6. **Keep sequence diagrams small:** 3–4 objects, 6–8 messages, no branches.
7. **Machine-coding rounds** (90–120 min, working code expected): a 2-minute class sketch is enough — spend the time on code.
8. If you forget a symbol, **say what you mean** ("this implements that") and move on.



In the interview

> **30-second answer:** "I list use cases verbally to fix scope, spend most of my diagram time on one class diagram with typed, multiplicity-labelled lines, then draw one behaviour diagram for the key flow and use it to check that every step has a class and method."

3.16 Chapter self-check

1. *Q: Name the six relationships from weakest to strongest coupling.* → A: Dependency, association, aggregation, composition, realization, generalization.
2. *Q: Where does the diamond go, and what does filled vs hollow mean?* → A: At the whole; filled = composition (parts die with it), hollow = aggregation (parts live on).
3. *Q: Code smell (a surface sign of a likely design problem):* `CompactSpot`, `LargeSpot` *subclasses that only pass a constant to* `super`*. Fix?* → A: One `ParkingSpot` with a `SpotType` enum field.

4. *Q: Java type for multiplicity `0..1` and `1..*`?* → A: `Optional<T>` (nullable field); a non-empty collection checked in the constructor.
5. *Q: «include» vs «extend»?* → A: Include = always, arrow from base to included; extend = sometimes (conditional), arrow from extension to base.
6. *Q: `+ - # ~`?* → A: public, private, protected, package.
7. *Q: Arrowheads in sequence diagrams?* → A: Solid + filled = sync call; solid + open = async; dashed + open = return.
8. *Q: `alt` vs `opt`?* → A: `alt` = if/else with several guarded sections; `opt` = a single guarded section (if without else).
9. *Q: Fork/join?* → A: Fork splits one flow into parallel flows; join waits for all of them.
10. *Q: Transition label syntax in a state machine?* → A: `event [guard] / action`.
11. *Q: 14 UML diagrams split how?* → A: 7 structural (class, object, package, component, composite structure, deployment, profile) and 7 behavioural (use case, activity, state machine, sequence, communication, interaction overview, timing).
12. *Q: If you can draw only one diagram?* → A: The class diagram.

3.17 TL;DR

- **Relationships are claims about lifetime and substitution.** Dependency (uses, dashed arrow) · Association (knows, line/arrow) · Aggregation (has, parts live on, ◇ at whole) · Composition (owns, parts die, ◆ at whole) · Realization (implements, dashed △) · Generalization (extends, solid △).
- **Choose in order:** substitution test → dies with whole? → part that outlives? → else association; parameter-only → dependency. Inheritance last.
- **Multiplicity changes code:** `1` required, `0..1` Optional, `1..*` non-empty collection, exact numbers checked in constructors.
- **UML:** 14 diagrams, structural vs behavioural. Interviews: use case (scope), class (the answer), activity (branches), sequence (call order), state machine (lifecycle).
- **Draw little, label everything, say the test out loud,** and use behaviour diagrams to test the class diagram.

Chapter 4 — SOLID Principles

> **What you will learn.** The five SOLID principles, one at a time and then all together. For each one you get a plain definition, an everyday analogy, a broken Java example, the refactored Java example, the rules behind it, the cost of applying it, and how to talk about it in a design interview. Along the way you also learn the ideas SOLID is built on: **cohesion**, **coupling** and **separation of concerns**.
>
> **How to use this chapter.** Read 4.1 first. Then read the principles in order, because each one sets up the next. Section 4.11 runs one class through all five principles, and it is the best single thing to rehearse before an interview. Sections 4.13 and 4.14 are about saying all this out loud.

Run the code. All code for this chapter is under `chapters/04-solid-principles/code/`, one package per topic. From that folder (`javac` is the Java compiler; `java` runs the compiled classes, and `-cp` tells it where they are):

```
javac --release 21 -d /tmp/lld_04 $(find . -name '*.java')
java -cp /tmp/lld_04 lld.ch04.srp.Main       # also: cohesion, coupling, soc, ocp, lsp, isp, dip, checkout
```

Package	What it shows
`lld.ch04.srp`	SRP: an `Employee` class split by actor
`lld.ch04.cohesion`	Low cohesion vs high cohesion
`lld.ch04.coupling`	Tight coupling vs loose coupling, in three steps
`lld.ch04.soc`	Separation of concerns: rules vs storage
`lld.ch04.ocp`	OCP: invoice formats behind an interface
`lld.ch04.lsp`	LSP: cars, the four contract rules, the history rule, birds and composition
`lld.ch04.isp`	ISP: fat printer interface, over-splitting, role interfaces
`lld.ch04.dip`	DIP: notifications, dependency injection, service locator, plug-ins
`lld.ch04.checkout`	One checkout class refactored through all five principles

A note on money. Every amount of money in this chapter's code (pay, invoice totals, prices, balances, order totals) is a `java.math.BigDecimal` (Java's exact decimal-number type) created from a string (`new BigDecimal("19.99")`), and every result that could gain extra decimal places (a multiplication by a rate) is rounded explicitly with `setScale(2, RoundingMode.HALF_EVEN)` (two decimal places, "banker's rounding": a value exactly halfway, such as 2.345, goes to the neighbour whose last digit is even, 2.34, so rounding errors do not all lean one way). Adding or subtracting two-decimal amounts stays at two decimals on its own. Never use `double` for money: binary floating point cannot store values such as 0.1 exactly, so `0.1 + 0.2` prints `0.30000000000000004`, and those tiny errors add up in totals. `BigDecimal` also has one trap: `equals` compares the scale (the number of decimal places) too (`2.0` is not equal to `2.00`), so compare amounts with `compareTo` or `signum()`, as the code here does.

4.1 What SOLID is and why you should learn it

What it is

SOLID is a set of five design guidelines for object-oriented code. Each letter names one principle:

Letter	Principle	The one-line rule
S	Single Responsibility Principle (SRP)	A class should have one reason to change.
O	Open/Closed Principle (OCP)	Add new behaviour by adding code, not by editing working code.
L	Liskov Substitution Principle (LSP)	A subtype must be usable anywhere its parent type is expected.
I	Interface Segregation Principle (ISP)	No client should be forced to depend on methods it does not use.
D	Dependency Inversion Principle (DIP)	Depend on abstractions, not on concrete details.

A few terms first, because the rest of the chapter uses them:

- A **class** is a blueprint for objects. A **module** is a bigger unit: a package, a library, a service.
- A **client** (or **caller**) is any code that uses another piece of code.
- An **interface** is a list of methods a class promises to provide, with no bodies. In Java it is the `interface` keyword. In general design talk it also means "the set of methods a caller can see".
- An **abstraction** is an interface or abstract class (a class marked `abstract` that cannot be created directly and may leave some methods for subclasses to write): something that says *what* is done without saying *how*.
- A **concrete class** is a class you can actually create with `new`.
- A **subtype** is a class that extends another class or implements an interface; the type it extends or implements is its **parent** (or base) type.

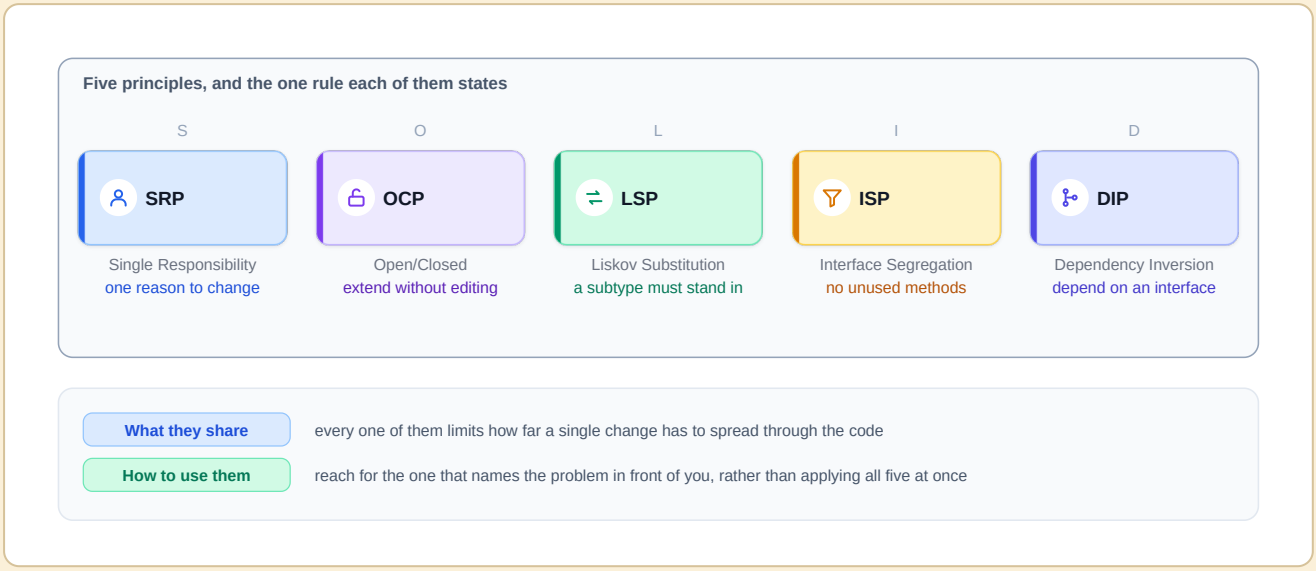


Figure — the five principles and the single rule each one states (course diagram)

In the figure above, look at the bottom line: every principle **limits how far one change spreads through the code**. That is the common thread. SRP limits it by giving each class one owner. OCP limits it by letting you add instead of edit. LSP makes sure a new subclass does not break old callers. ISP stops a change in one method from reaching callers that never use it. DIP stops a change in a low-level detail (a technical part such as a database driver) from reaching business logic (the rules the application exists for, such as pricing).

A little history. Robert C. Martin ("Uncle Bob") collected these principles in his writing around 2000, most fully in the paper *Design Principles and Design Patterns* and the book *Agile Software Development: Principles, Patterns, and Practices* (2002). Michael Feathers noticed the initials spelled SOLID. Two of the ideas are older: the Open/Closed Principle comes from Bertrand Meyer (1988), and the Substitution Principle from Barbara Liskov (1987).

The problem it solves

Code that works today still has to change tomorrow. Without guidelines, classes grow by small additions until one edit breaks three unrelated features. SOLID gives you:

- **Cleaner code.** Small, focused classes are easier to read and to hand to a teammate.
- **Flexibility.** New features plug in without breaking old ones.
- **Fewer bugs.** Less code is touched per change, so there is less to break.
- **Reuse.** A class with one job can be used in places its author never imagined.
- **Easier change.** When requirements shift, the change lands in one place.
- **A shared vocabulary.** This is the benefit people forget. When a teammate says "this class breaks ISP", everybody knows what the problem is, and a disagreement about design can be decided by an argument rather than by personal preference.

How to use SOLID (the honest version)

1. Treat each principle as a **name for a problem**, not a rule to obey everywhere.
2. When code is painful to change, find the principle that names the pain.
3. Apply the fix **when a real change asks for it**, not in advance.
4. Always say what the fix costs (more classes, more interfaces, more wiring, i.e. code that creates objects and connects them).

Applied without judgement, SOLID produces a maze of interfaces that each have one implementation. This chapter shows the cost of every recommendation along with its benefit.

> **TL;DR**
>
> SOLID = five guidelines that keep one change from spreading. Use them to **diagnose** code, apply them when a real change needs them, and always state the cost.

4.2 Single Responsibility Principle (SRP)

What it is

Robert C. Martin's first wording was roughly: **a class should have only one reason to change.** Later (in *Clean Architecture*, 2017) he sharpened it to: **a module should be responsible to one, and only one, actor.** An **actor** is a person or group who can ask for a change to the code.

The everyday version you will often hear is "every component should have one job". That version is a useful start, but it is hard to measure, as we will see.

> **Analogy: army knife vs kitchen knife.** A Swiss army knife has scissors, a bottle opener and a screwdriver. It does many jobs and is not the best tool for any of them. A kitchen knife has one job, cutting, and it is excellent at it.

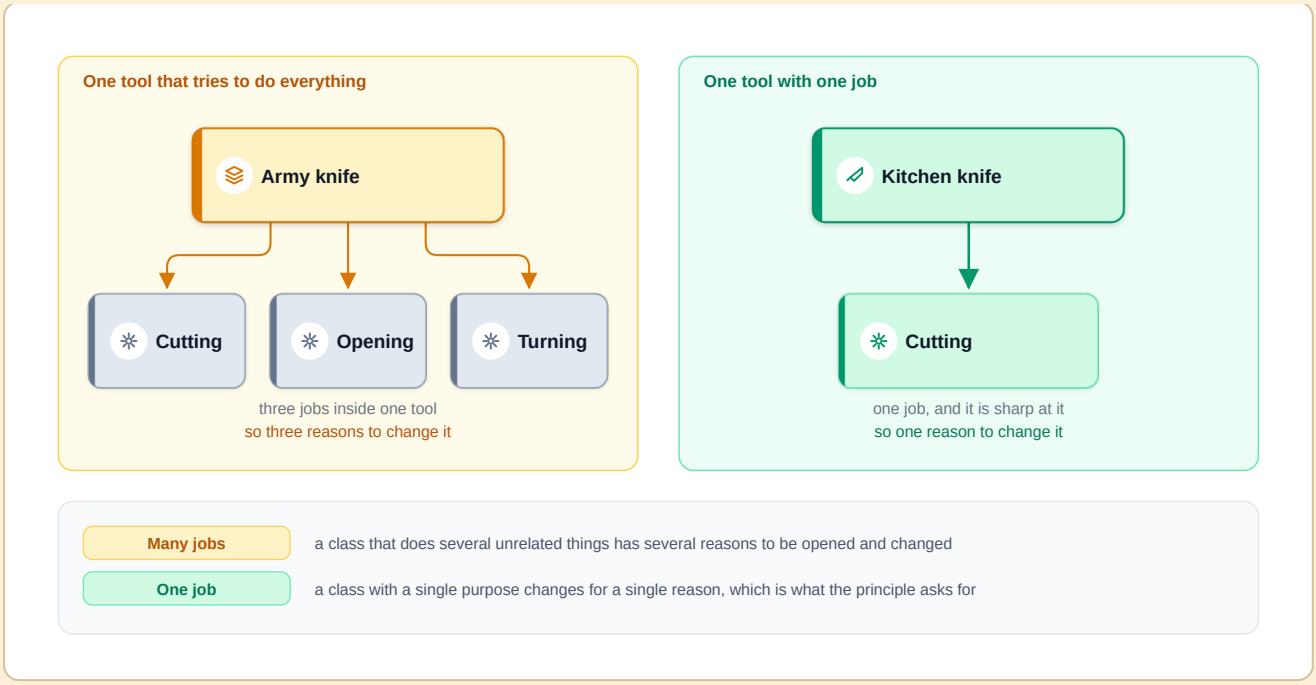


Figure — an army knife with three jobs (three reasons to change) next to a knife with one job (course diagram)

In the figure above, the army knife is the class that breaks SRP: three jobs, three reasons to open it and change it. The kitchen knife follows SRP.

The "one job" idea is often applied at every size of code:

- a **method** should perform one action,
- a **class** should serve one purpose,
- a **module** should cover one area of functionality.

Be precise about the names, though. Martin's SRP is a rule about **classes and modules** and their reasons to change. "A function should do one thing" is a separate (good) rule about functions from *Clean Code*. Both push in the same direction, but only the first is SRP.

The problem it solves

When one class serves several groups, every edit made for one group ships inside code that the other groups rely on, so a payroll tweak can break an operations report. Unrelated logic is also mixed together, which makes it slow to find the right place for a change. With one responsibility per class, you know exactly which class to open.

What counts as one responsibility?

"Do one thing" has no fixed size. Is "register a user" one thing? Is checking the email address part of it, or a job of its own? Reasonable people can debate that without end, because the phrase offers no yardstick.

The **actor** view gives you something to measure. A reason to change always comes from someone. So ask: **who can send me a change request for this code?** If two different groups can, the class has two responsibilities, however small it looks.

Here is an `Employee` class. Every method is "about employees", so the "one thing" test passes. Now ask who can demand a change to each method:

Method	Who asks for the change	Example request
`calculatePay`	Finance	"Overtime starts after 38 hours, not 40."
`reportHours`	Operations	"Show the week-ending date in the report."
`save`	Database team	"Move from SQL (table database) to a document store (stores JSON-like documents)."

So one class has three owners and three independent reasons to change. A new overtime rule from finance means editing `Employee`, which also ships the report code and the storage code again, although neither of those teams wanted anything. If the edit accidentally breaks the report layout, operations pays for a decision it had no part in.

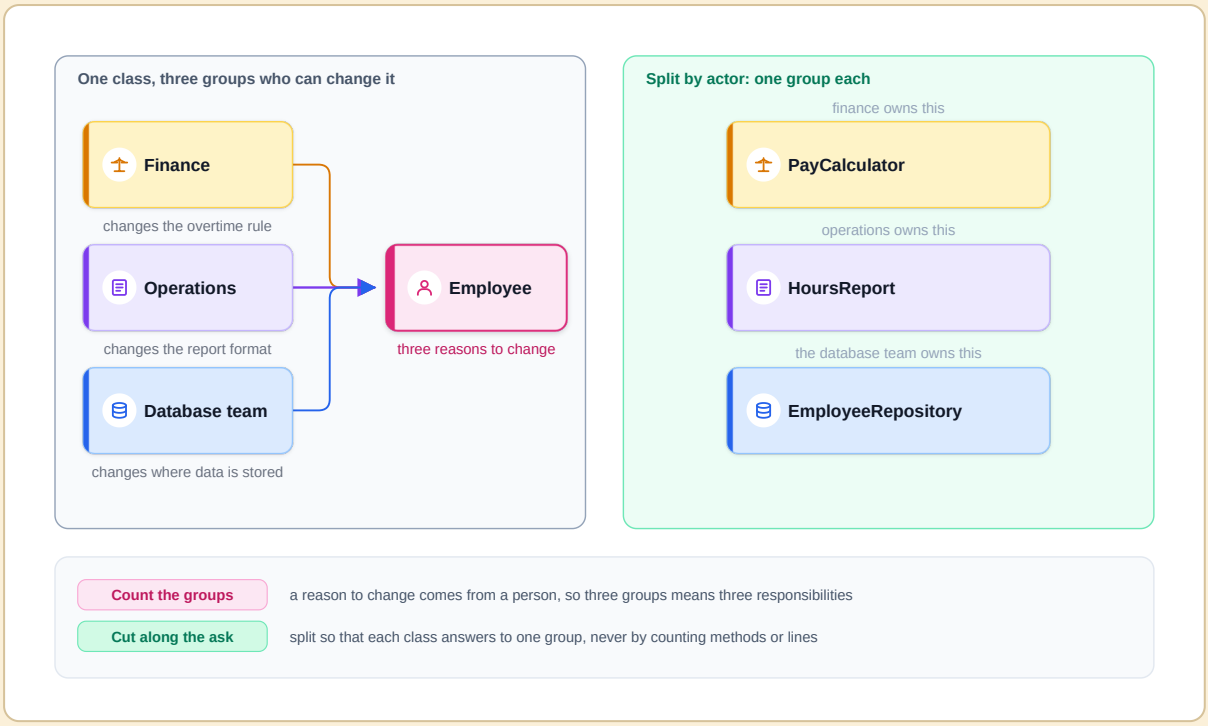


Figure — finance, operations and the database team all reach into one Employee class; splitting by actor gives each group its own class (course diagram)

Martin tells a sharper version of this story in *Clean Architecture*. Suppose `calculatePay` and `reportHours` both call a shared private helper, `regularHours()`, because the two calculations happen to match today. Finance changes the helper for its own reasons. The operations report now shows wrong numbers, and nobody on the operations side was told. The code looked like good reuse, but the two uses belonged to different actors. That is **accidental duplication**: two pieces of code that look the same but change for different reasons should not be merged.

How it works

1. List the methods of the class.
2. For each one, name **who** would ask for it to change.
3. Group the methods by actor.
4. Move each group into its own class, named after its job.
5. Keep plain data in a simple data class (a Java `record` is ideal: a compact class that only holds a fixed set of fields).
6. Check the split with the three checks below.

Java example: splitting `Employee` by actor

The violation. Three actors share one class:

```java
// File: code/lld/ch04/srp/before/Employee.java
package lld.ch04.srp.before;

import java.math.BigDecimal;
import java.math.RoundingMode;

/**
 * VIOLATION: three different groups (actors) can each demand a change here.
 * Finance owns the pay rule, operations owns the report, the database team owns storage.
 */
public class Employee {
    private final String name;
    private final int hoursWorked;
    private final BigDecimal hourlyRate; // money: BigDecimal, never double

    public Employee(String name, int hoursWorked, BigDecimal hourlyRate) {
        this.name = name;
        this.hoursWorked = hoursWorked;
        this.hourlyRate = hourlyRate;
    }

    // Actor 1: Finance -- "overtime should start after 38 hours, not 40"
    public BigDecimal calculatePay() {
        int overtime = Math.max(0, hoursWorked - 40);
        int normal = hoursWorked - overtime;
        BigDecimal normalPay = hourlyRate.multiply(BigDecimal.valueOf(normal));
        BigDecimal overtimePay = hourlyRate.multiply(new BigDecimal("1.5"))
                .multiply(BigDecimal.valueOf(overtime));
        return normalPay.add(overtimePay).setScale(2, RoundingMode.HALF_EVEN);
    }

    // Actor 2: Operations -- "show the week-ending date in the report"
    public String reportHours() {
        return name + " worked " + hoursWorked + " hours";
    }

    // Actor 3: Database team -- "move from SQL to a document store"
    public void save() {
        System.out.println("INSERT INTO employees VALUES ('" + name + "')");
    }
}
```

The refactor. `Employee` keeps only the data. Each actor gets its own class:

```java
// File: code/lld/ch04/srp/after/Employee.java
package lld.ch04.srp.after;

import java.math.BigDecimal;

/** Just the data. No group owns a business rule in here. */
public record Employee(String name, int hoursWorked, BigDecimal hourlyRate) { }
```

```java
// File: code/lld/ch04/srp/after/PayCalculator.java
package lld.ch04.srp.after;

import java.math.BigDecimal;
import java.math.RoundingMode;

/** Only Finance asks for changes to this class. */
public class PayCalculator {
    private static final int OVERTIME_THRESHOLD = 40;
    private static final BigDecimal OVERTIME_MULTIPLIER = new BigDecimal("1.5");

    public BigDecimal calculatePay(Employee e) {
        int overtime = Math.max(0, e.hoursWorked() - OVERTIME_THRESHOLD);
        int normal = e.hoursWorked() - overtime;
        BigDecimal normalPay = e.hourlyRate().multiply(BigDecimal.valueOf(normal));
        BigDecimal overtimePay = e.hourlyRate().multiply(OVERTIME_MULTIPLIER)
                .multiply(BigDecimal.valueOf(overtime));
        return normalPay.add(overtimePay).setScale(2, RoundingMode.HALF_EVEN); // cents
    }
}
```

```java
// File: code/lld/ch04/srp/after/HoursReport.java
package lld.ch04.srp.after;

/** Only Operations asks for changes to this class. */
public class HoursReport {
    public String format(Employee e) {
        return e.name() + " worked " + e.hoursWorked() + " hours";
    }
}
```

```java
// File: code/lld/ch04/srp/after/EmployeeRepository.java
package lld.ch04.srp.after;

/** Only the database team asks for changes to this class. */
public class EmployeeRepository {
    public void save(Employee e) {
        System.out.println("INSERT INTO employees VALUES ('" + e.name() + "')");
    }
}
```

```java
// File: code/lld/ch04/srp/Main.java
package lld.ch04.srp;

import java.math.BigDecimal;
import lld.ch04.srp.after.EmployeeRepository;
import lld.ch04.srp.after.HoursReport;
import lld.ch04.srp.after.PayCalculator;

public class Main {
    public static void main(String[] args) {
        System.out.println("--- Before: one class, three actors ---");
        var before = new lld.ch04.srp.before.Employee("Ada", 45, new BigDecimal("20.00"));
        System.out.println("Pay: " + before.calculatePay());
        System.out.println(before.reportHours());
        before.save();

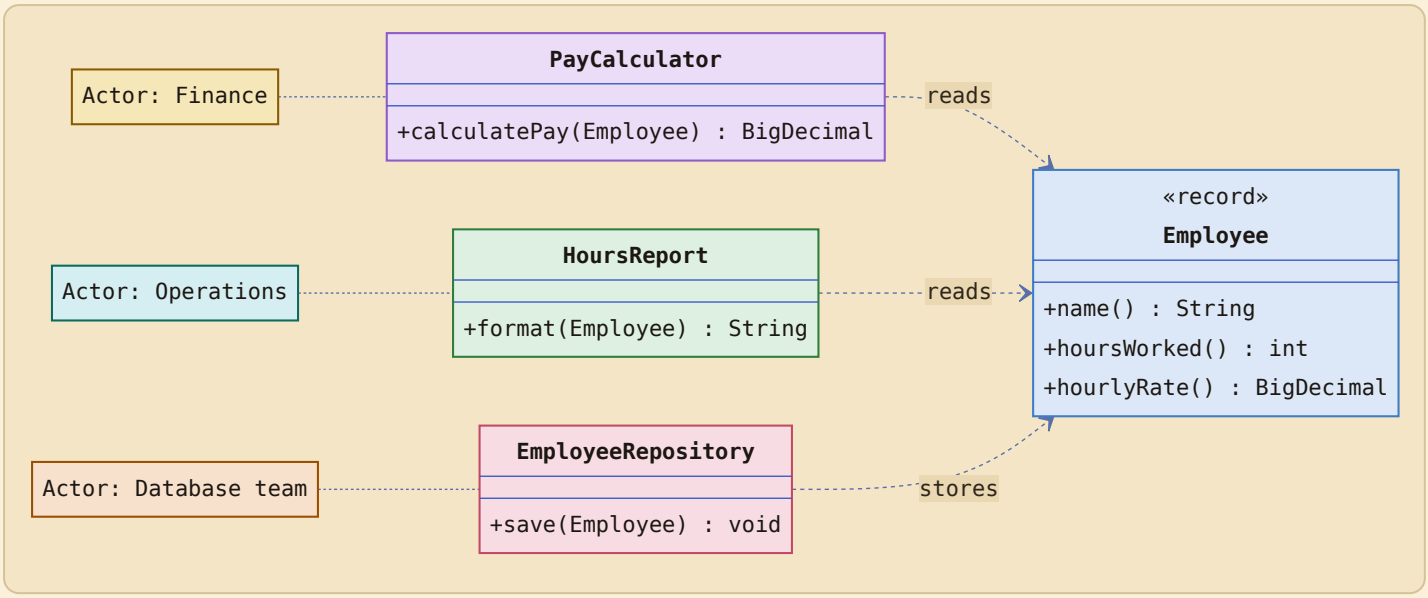
        System.out.println("--- After: split by actor ---");
        var ada = new lld.ch04.srp.after.Employee("Ada", 45, new BigDecimal("20.00"));
        System.out.println("Pay: " + new PayCalculator().calculatePay(ada));
        System.out.println(new HoursReport().format(ada));
        new EmployeeRepository().save(ada);
    }
}
```

Output:

```
--- Before: one class, three actors ---
Pay: 950.00
Ada worked 45 hours
INSERT INTO employees VALUES ('Ada')
--- After: split by actor ---
Pay: 950.00
Ada worked 45 hours
INSERT INTO employees VALUES ('Ada')
```

What each part does:

- `Employee` is a `record`: an immutable (cannot change after creation) data carrier. No business rule lives there, so no actor needs to edit it for a rule change.
- `PayCalculator` belongs to finance. The overtime threshold and multiplier are named constants, so the next finance request is a one-line change in one class.
- `HoursReport` belongs to operations. `EmployeeRepository` belongs to the database team.
- The output is **identical** before and after. Behaviour did not change. What changed is who must be consulted before an edit. Finance can now change the overtime rule without anyone touching the report or the database code.



Where splitting goes wrong

A badly applied SRP leaves the code in a **worse** state than no SRP at all. The typical mistake: a large class is chopped up by verb, one class per method or per step, without asking who owns what. The cuts fall in arbitrary places, and the result is a handful of tiny classes that must always be edited as a group, so each change now spreads over half a dozen files. Three checks catch this:

1. **Do the pieces change together?** After a few months, read the version-control log (the history kept by a tool such as Git, where each **commit** is one saved change). Two classes that show up in nearly every commit side by side were really one responsibility that got cut in two.
2. **Does each new class have a name that stands on its own?** `EmployeePayCalculator` describes a job. `EmployeeHelper` and `EmployeeUtils` describe nothing. If the name is meaningless without the original class next to it, the class usually is too.
3. **Could the new class be used from somewhere else?** A pay calculator could serve a payroll export, a quote for a contractor, or a unit test (a small automated test of one class on its own). If you cannot imagine any second caller, question why you split it off.

Splitting does not automatically reduce dependencies. After an extraction the two classes still call each other. If the extracted class was designed around the precise needs of the class it came from, every change to one will drag the other along. So the split is only step one; step two is deciding what each class is allowed to know about the other. Those two questions have names: **cohesion** (4.3) and **coupling** (4.4). SRP usually helps both, but promises neither.

Why SRP is important

- **Simpler maintenance.** A class with one job is easy to understand, debug and change.
- **Reuse.** A focused class fits in more places.
- **Scales with the codebase.** New features add classes instead of bloating old ones.
- **High cohesion.** Everything in the class serves one purpose.
- **Opens the door to low coupling, but does not finish the job.** Separating responsibilities gives you a seam (a boundary where one side can be swapped without editing the other). Coupling drops only when the dependency across that seam points at an abstraction (see 4.4).

Trade-offs

You gain	You pay
A change requested by one group touches only that group's code	More classes and files to navigate
Smaller classes are easier to test in isolation	The caller now wires several objects together
Clear ownership (who reviews what)	Risk of over-splitting into classes that always change together

When not to use it

- When you cannot name two different actors. If you cannot name the groups, you do not yet know where the seam is.
- In short-lived code such as a one-off script.
- When the pieces would always change together anyway.

Common mistakes

- **Splitting by counting methods or verbs** instead of by actor.
- **Helper/Utils classes** that exist only because something had to be moved somewhere.
- **Merging "duplicate" code that belongs to different actors** (the `regularHours()` trap).
- **Thinking SRP means "a class has one method".** A class can have ten methods and one responsibility.
- **Confusing SRP with "a function should do one thing".** That is a separate (good) rule about functions from *Clean Code*. SRP is about reasons to change.

Code smells that point to SRP (from *Refactoring*)

A **code smell** is a surface sign that the design is probably wrong somewhere; *Refactoring* is Martin Fowler's book that names the common smells and the step-by-step fixes (called refactorings) for each.

- **Divergent Change** — one class is edited for many different kinds of reasons. This is SRP broken.
- **Large Class** — too many fields and methods, often a sign of several jobs.
- **Shotgun Surgery** — one change forces small edits in many classes. This is often the result of **over-splitting**, the opposite mistake.
- The usual fix is the **Extract Class** refactoring (and **Move Function**/**Move Field**).

Real-world examples

- **Spring layers** (Spring is a popular Java application framework): `@Controller` handles HTTP (the web's request/response protocol, see System Design Fundamentals), `@Service` holds business rules, `@Repository` handles persistence (saving data permanently, usually in a database). Each layer answers to a different kind of change.
- **JDK**: `java.time.format.DateTimeFormatter` formats and parses dates; `LocalDate` holds the date. Formatting rules can change without touching the date value type.
- **Logging frameworks** (Logback, Log4j 2: libraries that write a program's log messages) split the *logger* (what to log), the *appender* (where it goes) and the *layout* (how it looks). A new output format does not touch the destination code.

In the interview

30-second answer: "SRP says a class should have one reason to change. The sharp version is: one actor, one group of people who can request changes. I find the actors by asking who would ask for each method to change. If finance, operations and the DBAs (database administrators) can all ask for edits to `Employee`, I split it into `PayCalculator`, `HoursReport` and `EmployeeRepository`, and keep `Employee` as data. The cost is more classes and some wiring, so I only split when I can name the different groups."

Follow-ups:

- *"Would you always split?"* — No. If the two halves always change together, I have turned one file into two for nothing. I split when two different people can ask for two different changes.
- *"How do you know you split too far?"* — The pieces show up in the same commits, their names only make sense next to each other (`XHelper`), and nothing else could use them.
- *"Is SRP about methods?"* — It applies to methods, classes and modules, but the test is always reasons to change, not the number of methods.

Self-check

1. **Q:** What is an "actor" in SRP? **A:** A person or group who can request a change to the code, for example finance or the database team.
2. **Q:** A `ReportService` formats a report and emails it. Is that an SRP problem? **A:** Yes, if different people own the wording and the delivery. Formatting and delivery change for different reasons.
3. **Q:** Name the three checks for a good split. **A:** Do the pieces change together? Does each name stand alone? Could each piece be used elsewhere?
4. **Q:** Does splitting a class always lower coupling? **A:** No. If the new class names a concrete collaborator, the dependency just moved. Point it at an abstraction to loosen it.

TL;DR

One class, one actor. Find the actors, split along them, keep data in a record, check the split, and do not split what always changes together.

4.3 Cohesion

What it is

Cohesion is how closely the contents of one class (or module, or method) belong together. **High cohesion**: every method works toward the same purpose. **Low cohesion**: unrelated jobs are bundled into one place.

Analogy: a garbage dump vs sorted bins. A dump holds plastic, paper and food waste in one pile. Nothing in it has a clear purpose, and finding anything is hard. Separate bins, one per material, each have one job.

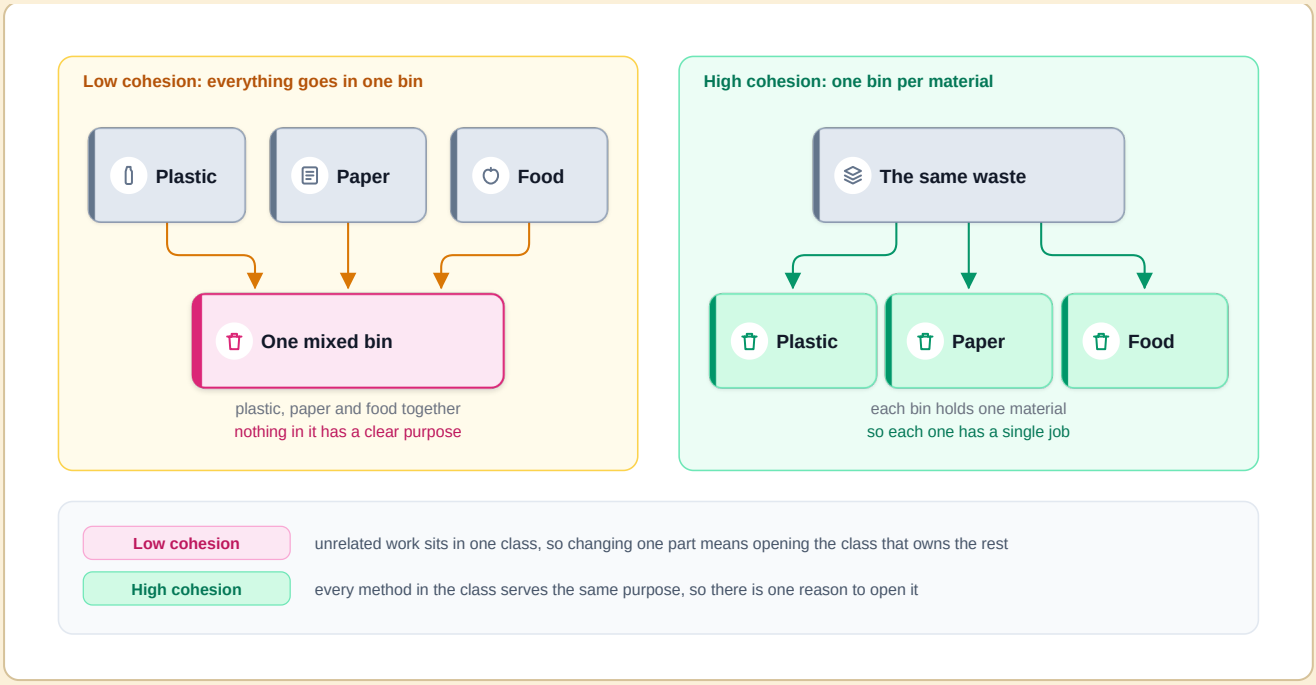


Figure — one mixed bin (low cohesion) next to one bin per material (high cohesion) (course diagram)

In the figure above, the mixed bin is a class whose methods have nothing to do with each other. The sorted bins are classes where every method serves the same purpose, so each one has a single reason to change.

The problem it solves

In a low-cohesion class, someone who wants to change one job has to open, read and redeploy (release to production again) a class that also owns other jobs. Low cohesion also works in the other direction: if the code for **one** job is **scattered** over several classes (say `addUser` in one class and `deleteUser` in another), a change to that job has to hunt for all the pieces. Both are cohesion problems: the first mixes unrelated things, the second separates related things.

The classic types of cohesion

Structured-design authors (Larry Constantine, Ed Yourdon, Glenford Myers, 1970s) ranked cohesion from worst to best. The names are still useful for describing what you see:

Type (worst → best)	What groups the code	Java-flavoured example
Coincidental	Nothing, it just ended up together	`Utils` with `formatDate`, `sendEmail`, `parseCsv`
Logical	Same category, chosen by a flag	`handle(type)` with a big `switch` over unrelated actions
Temporal	Things that run at the same time	`startup()` that opens the DB, warms caches (fills fast temporary copies of data) and prints a banner
Procedural	Steps that follow an order	A method that validates, then logs, then retries
Communicational	Work on the same data	`ReportBuilder` methods that all read the same `Order` list
Sequential	Output of one step is input to the next	`parse → validate → transform` pipeline in one class
Functional (best)	Everything contributes to one well-defined task	`PayCalculator`

You do not need to memorise the list. Aim for **functional** cohesion and recognise **coincidental** cohesion (the "Utils" class) as a warning sign.

How it works

1. Look at each method in the class. Which fields does it use?
2. If the methods fall into groups that use **different** fields for **different** purposes, the class has low cohesion.
3. Move each group into its own class.
4. If the code for one job is scattered over several classes, gather it into one.

Java example

Before: user bookkeeping and audit-file work share a class. `addUser`/`deleteUser` use `users`; `writeToFile`/`readFromFile` use `auditLines`. Two disjoint groups:

```
// File: code/lld/ch04/cohesion/before/UserManager.java
package lld.ch04.cohesion.before;

import java.util.ArrayList;
import java.util.List;

/** LOW cohesion: user bookkeeping and audit-file work share one class but not one purpose. */
public class UserManager {
    private final List<String> users = new ArrayList<>();
    private final List<String> auditLines = new ArrayList<>();

    // --- user work ---
    public void addUser(String user) {
        users.add(user);
        System.out.println("user added: " + user);
    }

    public void deleteUser(String user) {
        users.remove(user);
        System.out.println("user deleted: " + user);
    }

    // --- file work, unrelated to the two methods above ---
    public void writeToFile(String line) {
        auditLines.add(line);
        System.out.println("wrote a line to the audit file: " + line);
    }

    public List<String> readFromFile() {
        System.out.println("read " + auditLines.size() + " line(s) back from the audit file");
        return List.copyOf(auditLines);
    }
}
```

After: two classes, each with one purpose:

```java
// File: code/lld/ch04/cohesion/after/UserManager.java
package lld.ch04.cohesion.after;

import java.util.ArrayList;
import java.util.List;

/** HIGH cohesion: every method works on the same field for the same purpose. */
public class UserManager {
    private final List<String> users = new ArrayList<>();

    public void addUser(String user) {
        users.add(user);
        System.out.println("user added: " + user);
    }

    public void deleteUser(String user) {
        users.remove(user);
        System.out.println("user deleted: " + user);
    }
}
```

```java
// File: code/lld/ch04/cohesion/after/AuditFile.java
package lld.ch04.cohesion.after;

import java.util.ArrayList;
import java.util.List;

/** HIGH cohesion: only audit-log work lives here. */
public class AuditFile {
    private final List<String> lines = new ArrayList<>();

    public void writeLine(String line) {
        lines.add(line);
        System.out.println("wrote a line to the audit file: " + line);
    }

    public List<String> readAll() {
        System.out.println("read " + lines.size() + " line(s) back from the audit file");
        return List.copyOf(lines);
    }
}
```

```java
// File: code/lld/ch04/cohesion/Main.java
package lld.ch04.cohesion;

import lld.ch04.cohesion.after.AuditFile;
import lld.ch04.cohesion.after.UserManager;

public class Main {
    public static void main(String[] args) {
        System.out.println("--- Before: one class, two unrelated jobs ---");
        var manager = new lld.ch04.cohesion.before.UserManager();
        manager.addUser("ada");
        manager.writeToFile("created ada");
        manager.deleteUser("ada");
        manager.readFromFile();

        System.out.println("--- After: two classes, one job each ---");
        var users = new UserManager();
        var audit = new AuditFile();
        users.addUser("ada");
        audit.writeLine("created ada");
        users.deleteUser("ada");
        audit.readAll();
    }
}
```

Output:

```
--- Before: one class, two unrelated jobs ---
user added: ada
wrote a line to the audit file: created ada
user deleted: ada
read 1 line(s) back from the audit file
--- After: two classes, one job each ---
user added: ada
wrote a line to the audit file: created ada
user deleted: ada
read 1 line(s) back from the audit file
```

Both halves of the output are identical. The difference is structural: each class now has a single purpose, so a new audit-log format is an edit to `AuditFile` alone and never touches user management. The **cost** is that the caller must create and hold two objects instead of one, which becomes worth it as soon as the two jobs change for different reasons.

Cohesion and SRP

- Low cohesion usually means several unrelated jobs in one class, which breaks SRP.
- High cohesion means the class is built around one job, which is what SRP asks for.
- So aiming for high cohesion tends to give you SRP for free.

Trade-offs

High cohesion gains	Costs
Easy to name, read and test each class	More classes to wire
A change lands in one class	Callers may hold several objects

When not to use it

Do not split a class that already has functional cohesion just to make it smaller. Five tiny classes can be harder to follow than one clear one.

Common mistakes

- Calling every mixed class a "coupling problem". A single class mixing jobs is a **cohesion** problem; coupling needs two classes.
- Creating a `Utils` dumping ground (coincidental cohesion).

Real-world examples

- `java.lang.Math` is a set of static math functions: functional cohesion around "numeric calculations".
- `java.util.Collections` is looser: sorting, searching, wrapping and synchronising collections all live together. It is still "operations on collections", but it shows how a utility class drifts toward logical cohesion.

In the interview

30-second answer: "Cohesion is how much the things inside one class belong together. I check which fields each method uses. If the methods split into two groups that touch different fields for different reasons, the class has low cohesion and I split it. High cohesion is basically SRP seen from inside the class."

Follow-up: *"Is a big class always low-cohesion?"* — No. If every method serves one task, it is cohesive even if it is large. Size is a hint, not the measure.

Self-check

1. **Q:** What is the worst kind of cohesion? **A:** Coincidental: code grouped for no reason, like a `Utils` class.

2. **Q:** Is "the code for one job is spread over three classes" a cohesion problem? **A:** Yes. Related code should live together.

> **TL;DR**

Cohesion = do the parts of this class belong together? Aim for one purpose per class; split disjoint groups; gather scattered pieces.

4.4 Coupling

What it is

Coupling is how much one class depends on another. It always describes a pair: a lone class cannot be "tightly coupled", because there is nothing else in the picture for it to depend on.

- **Low (loose) coupling**: you can change, replace or test one class without changing the other.
- **High (tight) coupling**: a change to one forces a change to the other.

> **Analogy: TV and remote.** A remote that controls only one specific TV, and a TV that accepts only that remote, are tightly coupled: replace one and you must replace both. A universal remote and a TV that accepts any remote are loosely coupled: each follows a shared standard (the infrared signal), and each can be swapped on its own.

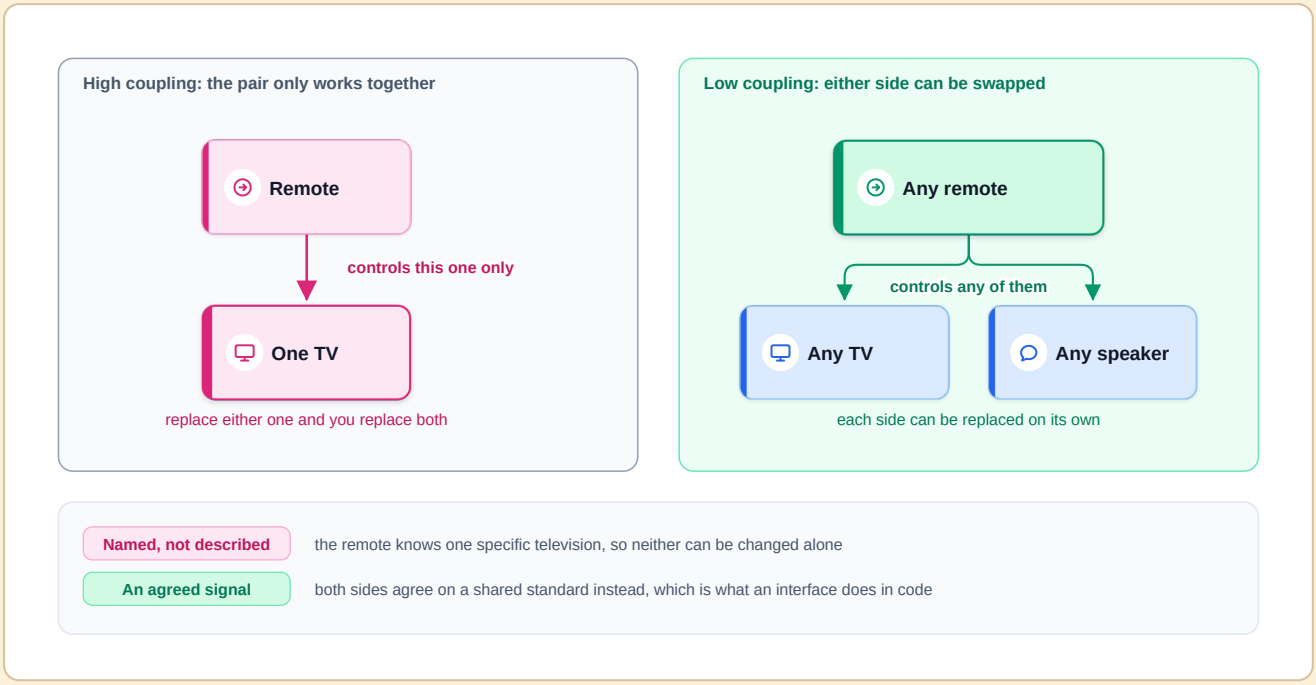


Figure — a remote tied to one TV (high coupling) vs any remote working with any device through an agreed signal (low coupling) (course diagram)

In the figure above, the "agreed signal" on the right is exactly what an **interface** does in code: both sides agree on a contract instead of knowing each other.

The problem it solves

With tight coupling:

- you cannot swap an implementation (MySQL → Postgres, two popular SQL databases) without editing the caller,
- you cannot unit-test the caller without the real dependency (a running database),
- a change in how the dependency is **constructed** ripples into every class that builds it.

The classic types of coupling

From structured design again, from worst to best:

Type (worst → best)	What A knows about B	Java-flavoured example
Content	A reaches into B's internals	Reflection (Java's ability to inspect and change classes while running) that sets B's private field
Common	A and B share global mutable data	Both read/write a `public static Map`
External	Both depend on an external format or device	Both parse the same hand-rolled file format
Control	A passes a flag telling B *how* to work	`print(doc, boolean asPdf)`
Stamp	A passes a whole structure when B needs one field	`sendInvoice(Customer c)` that only reads `c.email()`
Data	A passes only the values B needs	`sendInvoice(String email, Money total)`
Message (lowest)	A sends a message through an interface	`notifier.send(message)` on an interface

In object-oriented design, the practical goal is: **depend on interfaces, pass only what is needed, never share mutable globals.**

How it works

1. Find where class A **names** class B (a field type, a `new B()`, a static call `B.x()`).
2. Ask: does A need *this particular* B, or just *something that does the job*?
3. If the latter, describe the job as an interface.
4. Let A accept any object that implements the interface, usually through its constructor.
5. Let the caller (or the program's entry point, the `main` method where it starts) choose which implementation to pass.

Java example: three steps with `Student`

Start: one class, two jobs. `Student` holds data *and* talks to MySQL. With a single class in play this is a **cohesion** problem; coupling needs a second class:

```java
// File: code/lld/ch04/coupling/start/Student.java
package lld.ch04.coupling.start;

/** Starting point: ONE class, two jobs. A cohesion problem -- nothing to be coupled to yet. */
public class Student {
    private final String studentId;

    public Student(String studentId) {
        this.studentId = studentId;
    }

    public String studentId() {
        return studentId;
    }

    // Student holds the data AND knows how to store it
    public void save() {
        System.out.println("Connecting to MySQL...");
        System.out.println("Saving student with ID: " + studentId);
    }
}
```

Step 1: extract the database work. Cohesion is fixed. But now there are two classes, and the coupling is **tight**: `Student` names `MySqlDatabase` directly.

```java
// File: code/lld/ch04/coupling/extracted/MySqlDatabase.java
package lld.ch04.coupling.extracted;

public class MySqlDatabase {
    public void saveStudent(String studentId) {
        System.out.println("Connecting to MySQL...");
        System.out.println("Saving student with ID: " + studentId);
    }
}
```

```java
// File: code/lld/ch04/coupling/extracted/Student.java
package lld.ch04.coupling.extracted;

/** Step 1: cohesion fixed, but Student now names one concrete class -> TIGHT coupling. */
public class Student {
    private final String studentId;
    private final MySqlDatabase database = new MySqlDatabase(); // hard-wired

    public Student(String studentId) {
        this.studentId = studentId;
    }

    public void save() {
        database.saveStudent(studentId);
    }
}
```

Three costs follow: storing students in any other place requires editing `Student`; every test of `Student` needs a live MySQL server; and if `MySqlDatabase` ever needs constructor arguments, `Student` must change too. The extraction relocated the database code, but `Student` still **depends** on it just as much as before.

Step 2: depend on an interface. Describe what `Student` needs as `StudentRepository`, and let the caller choose the implementation:

```java
// File: code/lld/ch04/coupling/decoupled/StudentRepository.java
package lld.ch04.coupling.decoupled;

/** What Student needs, described as a contract rather than a working class. */
public interface StudentRepository {
    void save(String studentId);
}
```

```java
// File: code/lld/ch04/coupling/decoupled/MySqlStudentRepository.java
package lld.ch04.coupling.decoupled;

public class MySqlStudentRepository implements StudentRepository {
    @Override
    public void save(String studentId) {
        System.out.println("Saving student " + studentId + " to MySQL");
    }
}
```

```java
// File: code/lld/ch04/coupling/decoupled/InMemoryStudentRepository.java
package lld.ch04.coupling.decoupled;

import java.util.ArrayList;
import java.util.List;

/** A stand-in that a unit test can use -- no database server needed. */
public class InMemoryStudentRepository implements StudentRepository {
    private final List<String> saved = new ArrayList<>();

    @Override
    public void save(String studentId) {
        saved.add(studentId);
        System.out.println("Keeping student " + studentId + " in memory for a test");
    }

    public List<String> saved() {
        return List.copyOf(saved);
    }
}
```

```java
// File: code/lld/ch04/coupling/decoupled/Student.java
package lld.ch04.coupling.decoupled;

import java.util.Objects;

/** Step 2: Student names no database at all -> LOOSE coupling. */
public class Student {
    private final String studentId;
    private final StudentRepository repository;

    public Student(String studentId, StudentRepository repository) {
        this.studentId = studentId;
        this.repository = Objects.requireNonNull(repository);
    }

    public void save() {
        repository.save(studentId);
    }
}
```

```java
// File: code/lld/ch04/coupling/Main.java
package lld.ch04.coupling;

import lld.ch04.coupling.decoupled.InMemoryStudentRepository;
import lld.ch04.coupling.decoupled.MySqlStudentRepository;
import lld.ch04.coupling.decoupled.Student;

public class Main {
    public static void main(String[] args) {
        System.out.println("--- Start: one class holds data and SQL ---");
        new lld.ch04.coupling.start.Student("S-1024").save();

        System.out.println("--- Step 1: extracted, but tightly coupled to MySqlDatabase ---");
        new lld.ch04.coupling.extracted.Student("S-1024").save();

        System.out.println("--- Step 2: depends on the StudentRepository interface ---");
        new Student("S-1024", new MySqlStudentRepository()).save();
        var fake = new InMemoryStudentRepository();
        new Student("S-2048", fake).save();
        System.out.println("test double recorded: " + fake.saved());
    }
}
```

Output:

```
--- Start: one class holds data and SQL ---
Connecting to MySQL...
Saving student with ID: S-1024
--- Step 1: extracted, but tightly coupled to MySqlDatabase ---
Connecting to MySQL...
Saving student with ID: S-1024
--- Step 2: depends on the StudentRepository interface ---
Saving student S-1024 to MySQL
Keeping student S-2048 in memory for a test
test double recorded: [S-2048]
```

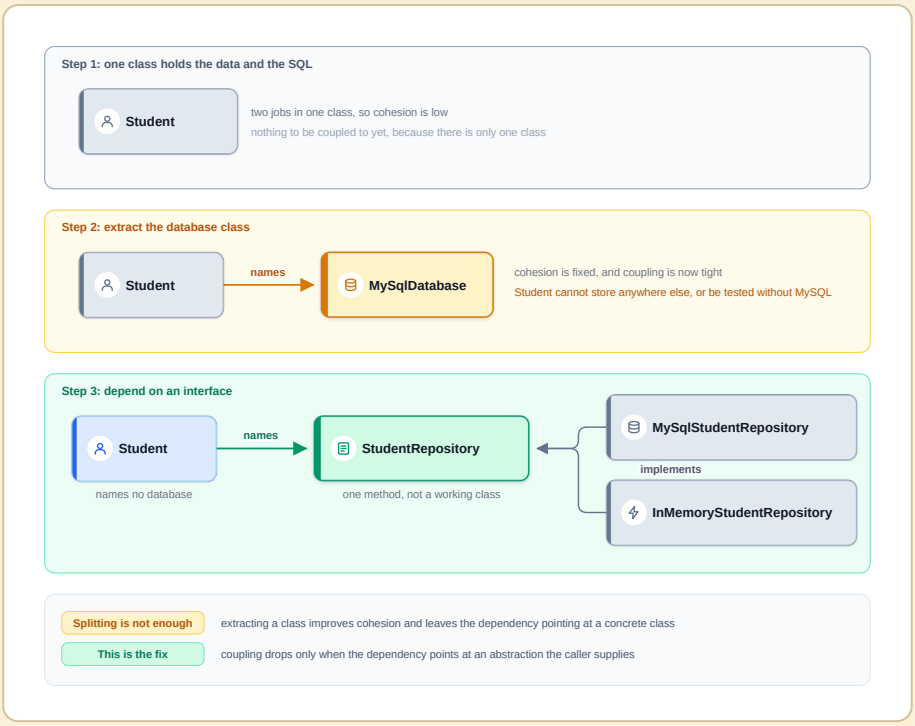


Figure — the three steps: one class with two jobs, an extracted but tightly coupled class, and a class that depends on an interface (course diagram)

In the figure above, follow the arrows. In the middle panel `Student` *names* `MySqlDatabase`. In the last panel it names only `StudentRepository`, a one-method contract, and the concrete repositories point *toward* that contract. Moving to Postgres is now a new repository class handed in by the caller, with no edit to `Student`, and a unit test simply hands in the in-memory version.

This final move is the rule **depend on abstractions, not on concrete classes**, the D in SOLID (4.10). DIP is, in effect, the standard cure for tight coupling.

Coupling and SRP

- **Separating the jobs** (SRP) repaired cohesion, yet by itself it left `Student` bound tightly to `MySqlDatabase`.
- **Aiming the dependency at an interface** is what repaired coupling, and SRP alone never requires that step.
- Hence SRP *usually* reduces coupling (a single-job class needs fewer collaborators) without *guaranteeing* it. After each split, inspect what the two resulting classes depend on; a class carved out to serve its parent's exact needs can remain welded to it.

Trade-offs

Loose coupling gains	Costs
Swap implementations without editing callers	An extra interface and a constructor parameter
Unit tests with in-memory fakes (simple stand-ins that keep data in a list instead of a database)	Someone must do the wiring (the entry point or a framework)
Changes stop rippling	"Go to definition" lands on an interface, one more hop to the real code

When not to use it

- Do not put an interface in front of a **stable** class that will never vary and never needs faking (`String`, `LocalDate`, a pure calculation). Coupling to stable things is harmless.
- An interface with one implementation and no test double (a fake stand-in object used in tests) is usually just an extra file.

Common mistakes

- Thinking "I extracted a class, so coupling is lower". Only depending on an abstraction lowers it.
- Hiding `new ConcreteThing()` inside a constructor. The dependency is still hard-wired.
- Passing whole objects when only one field is needed (stamp coupling).

Real-world examples

- **JDBC** (Java Database Connectivity, Java's standard way to talk to SQL databases): your code talks to `java.sql.Connection`, `Statement` and `ResultSet`, which are interfaces. The MySQL or Postgres driver (the library that speaks one database's protocol) supplies the implementation.
- **SLF4J** (a common Java logging facade, i.e. one front for several logging libraries): code logs through the `org.slf4j.Logger` interface; Logback or Log4j is plugged in at deployment (when the program is installed and started).
- **USB, HTTP, SQL** themselves are "agreed signals", the real-world version of an interface.

In the interview

> **30-second answer:** "Coupling is how much one class depends on another. I check where a class names a concrete collaborator. If it only needs something that does the job, I describe the job as an interface and pass the implementation in. That lets me swap MySQL for Postgres and test with an in-memory fake. I do not add interfaces in front of stable things like `String`."

Follow-up: *"Can SRP increase coupling?"* — It can. Extract a class built around the old class's exact needs and the two are tightly bound. Check the dependencies after every split.

Self-check

1. **Q:** Can a single class be "tightly coupled"? **A:** Not to anything inside itself. Coupling is between two things. A single class with two jobs has a cohesion problem.
2. **Q:** `Student` does `new MySqlDatabase()` in its constructor. Tight or loose? **A:** Tight. It names a concrete class and creates it.
3. **Q:** What is "control coupling"? **A:** Passing a flag that tells the other code how to behave, e.g. `print(doc, asPdf)`.

TL;DR

Coupling = how much A must know about B. Loosen it by depending on an interface and passing the implementation in. Extracting a class alone is not enough.

4.5 Separation of Concerns (SoC)

What it is

Separation of Concerns means splitting a program into parts where each part handles one **concern**. A concern is one area of the work: storing data is one concern, deciding whether an email address is acceptable is another, drawing the screen is a third. The term comes from Edsger Dijkstra (1974).

> **Analogy:** in a restaurant, the kitchen cooks, the waiters serve, and the cashier takes payment. A new menu does not change how payment works.

The problem it solves

When concerns are mixed, one change forces you to read and edit code about other concerns. When they are separate, each change lands in one module.

Benefits:

- **Modularity** — each part covers one area, so the system is easier to follow.
- **Maintainability** — a change in one part rarely affects the others.
- **Reusability** — a well-defined module can be reused elsewhere.
- **Testability** — test one concern with the others replaced by simple stand-ins.

How it works

1. List the concerns in the feature (for registration: storage, business rules).
2. Give each concern its own module.
3. Let modules talk only through small, clear methods.
4. Test the result: for each likely requirement change, **count the modules you would have to edit**. The ideal answer is one.

Java example: user registration

Two concerns. **Storage** keeps users and never judges whether a user is valid. **Business rules** decide what is acceptable and ask storage to save it, without knowing how storage works.

```java
// File: code/lld/ch04/soc/UserStore.java
package lld.ch04.soc;

import java.util.ArrayList;
import java.util.List;

/** Storage concern only. It has no opinion about what a valid user is. */
public class UserStore {
    public record User(String username, String email) { }

    private final List<User> users = new ArrayList<>();

    public void addUser(String username, String email) {
        users.add(new User(username, email));
        System.out.println("Stored user: " + username);
    }

    public int count() {
        return users.size();
    }
}
```

```java
// File: code/lld/ch04/soc/UserService.java
package lld.ch04.soc;

/** Business-rules concern only. Storage is somebody else's job. */
public class UserService {
    private final UserStore store;

    public UserService(UserStore store) {
        this.store = store;
    }

    boolean isValidUsername(String username) {
        return username.length() >= 3;
    }

    boolean isValidEmail(String email) {
        return email.contains("@") && email.contains(".");
    }

    public boolean registerUser(String username, String email) {
        if (!isValidUsername(username)) {
            System.out.println("Rejected: " + username + " is too short");
            return false;
        }
        if (!isValidEmail(email)) {
            System.out.println("Rejected: " + email + " is not an email address");
            return false;
        }
        store.addUser(username, email);
        return true;
    }
}
```

```java
// File: code/lld/ch04/soc/Main.java
package lld.ch04.soc;

public class Main {
    public static void main(String[] args) {
        UserStore store = new UserStore();
        UserService service = new UserService(store);
        service.registerUser("ada", "ada@example.com");
        service.registerUser("jo", "jo@example.com");
        service.registerUser("grace", "grace-at-example");
        System.out.println("Users in store: " + store.count());
    }
}
```

Output:

```
Stored user: ada
Rejected: jo is too short
Rejected: grace-at-example is not an email address
Users in store: 1
```

Of the three attempts, only `ada` made it into storage: `jo` failed the length rule and `grace-at-example` failed the email rule. Every accept/reject decision was made by `UserService`, which never touched the list itself; `UserStore` simply kept whatever it was handed, without judging it.

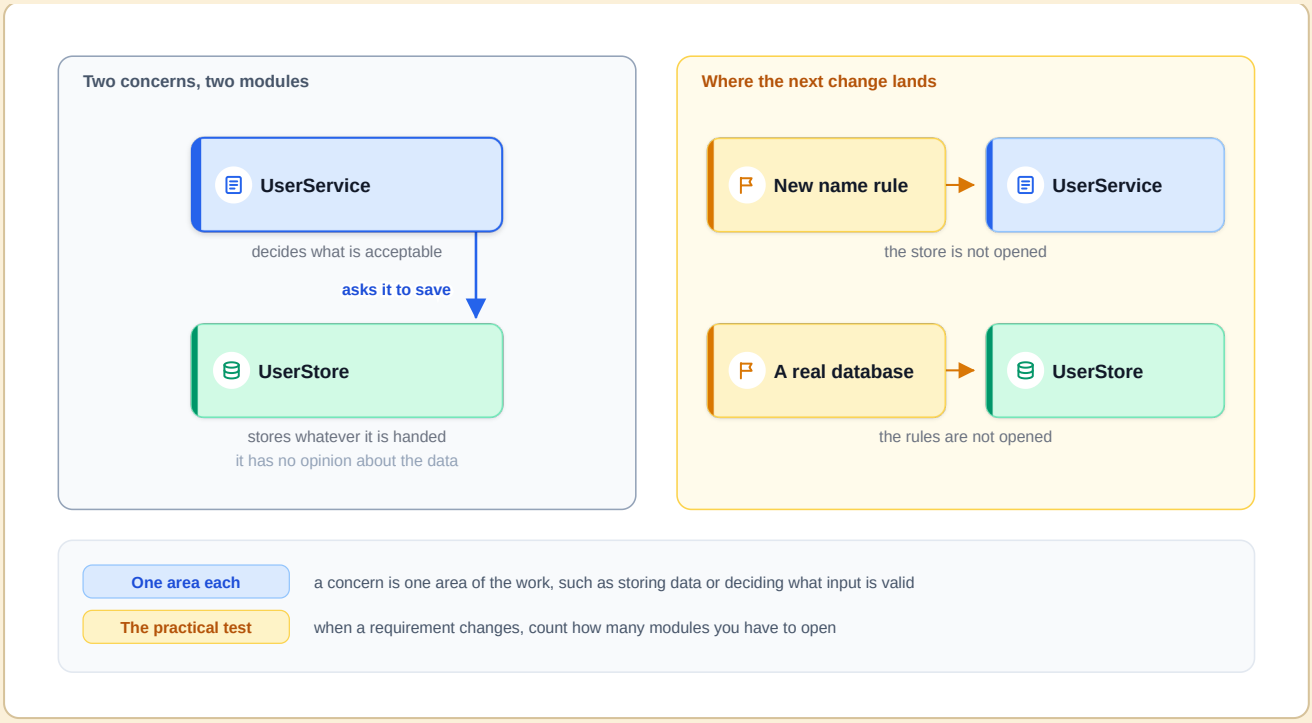


Figure — UserService decides what is acceptable and asks UserStore to save it; each change lands in one module (course diagram)

As the figure above shows, each likely change lands in exactly one place:

Change	Module you open
Move from an in-memory list to a real database	`UserStore` only
Require usernames of at least eight characters	`UserService` only
Test the rules without a database	Pass `UserService` a stand-in store

(In a bigger system you would also put `UserStore` behind an interface, which is the coupling fix from 4.4.)

SoC and SRP

The two are the same instinct applied at different scales. **SRP** is normally discussed for a single class; **SoC** for modules, layers and whole subsystems. Giving every class in a module a single responsibility is one route to well-separated concerns.

Trade-offs

Gains	Costs
Changes stay local	More modules and boundaries to design
Each concern tested alone	Cross-cutting concerns (logging, security, transactions, i.e. groups of database changes that succeed or fail together) still touch many modules; they need special tools such as aspects (code woven automatically around many methods) or decorators (wrappers that add behaviour)

When not to use it

A 50-line script does not need layers. Separate concerns when they change for different reasons or at different speeds.

Common mistakes

- Validation that quietly happens inside the storage class ("the DAO also checks the email"; a DAO, Data Access Object, is a class that reads and writes one kind of data in the database).

- Layers that exist on paper but call each other in every direction.

Real-world examples

- **MVC** (Model–View–Controller): model = data and rules, view = display, controller = input handling. Spring MVC and most web frameworks (libraries that handle the plumbing of web applications) follow it.
- **HTML / CSS / JavaScript**: structure, style and behaviour are three concerns in three languages.
- **Layered architecture**: controller → service → repository.
- **Spring** `@Transactional`: transaction handling (a cross-cutting concern) is separated from business code by a proxy (a stand-in object that wraps the real one and adds behaviour before and after each call).

In the interview

> **30-second answer:** "Separation of concerns means each module handles one area of the work, like storage vs business rules. My test is: when a requirement changes, how many modules do I open? Ideally one. It is SRP at module size."

Self-check

1. **Q:** What is a "concern"? **A:** One area of the work, such as storage, validation or presentation.
2. **Q:** How do SoC and SRP differ? **A:** Mostly in size: SRP is about a class, SoC about modules and layers.

TL;DR

One module per area of work; measure it by how many modules a change forces you to open.

4.6 SRP vs cohesion vs coupling vs SoC — one table

Idea	Question it asks	Scope	How it relates to SRP
SRP	How many actors can ask this class to change?	Class / module	—
Cohesion	Do the parts of this class belong together?	Inside one unit	Applying SRP usually raises cohesion, because one job means related parts.
Coupling	How much does A depend on B?	Between two units	SRP tends to lower coupling but does not guarantee it; you still need to depend on abstractions.
SoC	Is each area of work in its own module?	Modules, layers	SRP applied to every class is one way to achieve SoC.

The rule of thumb every textbook repeats, **"high cohesion, low coupling"**, is the goal. SRP is the tool that gets you the first half; depending on abstractions (DIP) gets you the second.

4.7 Open/Closed Principle (OCP)

What it is

Software entities (classes, modules, functions) should be open for extension but closed for modification. In plain words: you should be able to add new behaviour **by adding new code**, without editing code that already works.

- **Closed for modification**: once a class is written and tested, adding the next variation should not require opening it. Every edit to working code risks breaking it.

- **Open for extension**: new behaviour arrives as new parts, usually through **interfaces**, **polymorphism** (one call, many implementations) or **inheritance**.

Where it comes from. Bertrand Meyer stated the principle in 1988 (*Object-Oriented Software Construction*). In **Meyer's OCP** a class is *closed* once it is compiled, published and used by clients (it is changed only to fix bugs), and it stays *open* because a new class can **inherit** from it and add or redefine features. That is extension by implementation inheritance. Robert C. Martin's 1996 article restated it as the **polymorphic OCP**: the stable code depends on an **abstract interface**, and new behaviour arrives as new implementations of that interface, which can be swapped for one another. The polymorphic version is the one people usually mean today, and it is the one used in this chapter.

> **Analogy: a game console and its accessories.** You buy a console with a standard controller. Later you add a racing wheel, a motion controller and a VR headset. Nobody opens the console and rewires it. The console is **closed** (its internals stay sealed) and **open** (it offers a standard port that new accessories plug into).

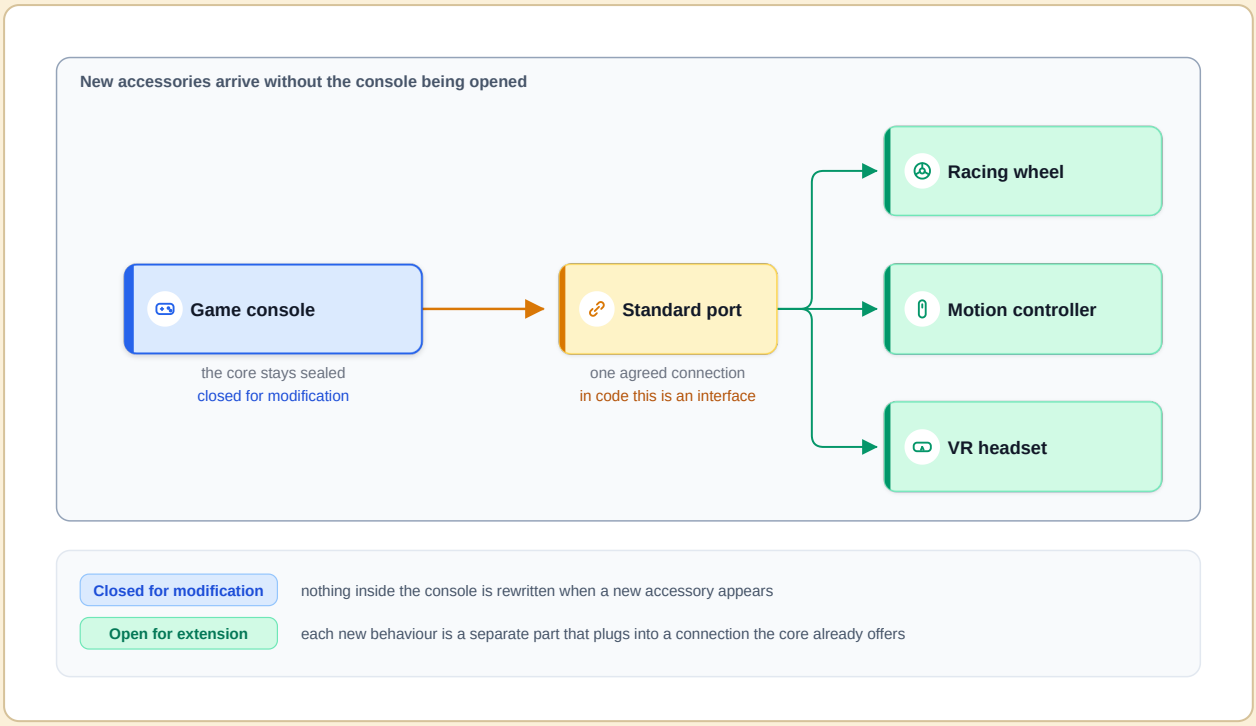


Figure — new accessories plug into a standard port; the console itself is never opened (course diagram)

In the figure above, the **standard port** is the interface. The accessories are new classes that implement it.

The problem it solves

The textbook symptom is an `if/else` chain or `switch` on a type code that keeps growing. Each new case is an edit to a tested, working method, and each edit can break the cases that were fine. Often the same `switch` also appears in several places, and one copy gets missed.

Quick test: *can I add the next case without opening this file?* If not, the class is not closed against that kind of change.

How it works

1. Wait until you have seen the variation happen (at least a second case).
2. Identify **the part that varies** (here: the invoice format).
3. Describe it as an interface.
4. Move each variant into its own class that implements the interface.
5. Make the stable class **delegate** (hand the work over) to the interface instead of branching.
6. Add the next variant as a new class. Nothing existing is opened.

7. Choose the variant in one place, at the edge of the program (the entry point, a factory, i.e. a method or class whose job is creating objects, a map, or configuration).

Java example: invoice formats

Before. While `Invoice` had a single format, there was nothing to fix: OCP only matters for code that actually receives new variations. The trouble starts when sales wants **international invoices** and the quickest route is a type parameter and a branch:

```java
// File: code/lld/ch04/ocp/before/Invoice.java
package lld.ch04.ocp.before;

import java.math.BigDecimal;

/** VIOLATION: every new invoice type means opening this working, tested method. */
public class Invoice {
    private final BigDecimal amount;

    public Invoice(BigDecimal amount) {
        this.amount = amount;
    }

    public void generate(String invoiceType) {
        if (invoiceType.equals("basic")) {
            System.out.println("Generating basic invoice for amount: " + amount);
        } else if (invoiceType.equals("international")) {
            System.out.println("Generating international invoice for amount: " + amount);
        } else {
            throw new IllegalArgumentException("Unknown invoice type: " + invoiceType);
        }
    }

    public void applyDiscount() {
        System.out.println("Applying discount on invoice: " + amount);
    }
}
```

Note exactly what is wrong. Doing two jobs would be an SRP complaint; the OCP complaint is that **each new invoice type forces an edit to code that already works**.

After. Step 1: turn the part that varies into an interface. Step 2: give each format its own class. These classes *produce* invoices, so they are named *generators*; a name like `BasicInvoice` would falsely imply a subtype of invoice. Step 3: `Invoice` hands the work to its generator. Step 4: a third format costs one new class plus one line at the place where formats are chosen.

```java
// File: code/lld/ch04/ocp/after/InvoiceGenerator.java
package lld.ch04.ocp.after;

import java.math.BigDecimal;

/** Step 1: the part that varies (the invoice format) gets its own abstraction. */
@FunctionalInterface
public interface InvoiceGenerator {
    void generate(BigDecimal amount);
}
```

```java
// File: code/lld/ch04/ocp/after/BasicInvoiceGenerator.java
package lld.ch04.ocp.after;

import java.math.BigDecimal;

public class BasicInvoiceGenerator implements InvoiceGenerator {
    @Override
    public void generate(BigDecimal amount) {
        System.out.println("Generating basic invoice for amount: " + amount);
    }
}
```

```java
// File: code/lld/ch04/ocp/after/InternationalInvoiceGenerator.java
package lld.ch04.ocp.after;

import java.math.BigDecimal;

public class InternationalInvoiceGenerator implements InvoiceGenerator {
    @Override
    public void generate(BigDecimal amount) {
        System.out.println("Generating international invoice for amount: " + amount);
        System.out.println("  including customs declaration and currency conversion");
    }
}
```

```java
// File: code/lld/ch04/ocp/after/DetailedInvoiceGenerator.java
package lld.ch04.ocp.after;

import java.math.BigDecimal;

/** Step 4: the ONLY new code for a third format. No existing class was opened. */
public class DetailedInvoiceGenerator implements InvoiceGenerator {
    @Override
    public void generate(BigDecimal amount) {
        System.out.println("Generating detailed invoice for amount: " + amount);
        System.out.println("  line items, tax breakdown and payment terms");
    }
}
```

```java
// File: code/lld/ch04/ocp/after/Invoice.java
package lld.ch04.ocp.after;

import java.math.BigDecimal;
import java.util.Objects;

/** Step 3: Invoice delegates. It no longer knows how many formats exist. */
public class Invoice {
    private final BigDecimal amount;
    private final InvoiceGenerator generator;

    public Invoice(BigDecimal amount, InvoiceGenerator generator) {
        this.amount = Objects.requireNonNull(amount);
        this.generator = Objects.requireNonNull(generator);
    }

    public void generate() {
        generator.generate(amount); // no branching
    }

    // Still closed? No -- a new discount rule would still edit this class.
    // That is fine until discounts actually start to vary.
    public void applyDiscount() {
        System.out.println("Applying discount on invoice: " + amount);
    }
}
```

```java
// File: code/lld/ch04/ocp/Main.java
package lld.ch04.ocp;

import java.math.BigDecimal;
import java.util.List;
import lld.ch04.ocp.after.BasicInvoiceGenerator;
import lld.ch04.ocp.after.DetailedInvoiceGenerator;
import lld.ch04.ocp.after.InternationalInvoiceGenerator;
import lld.ch04.ocp.after.Invoice;
import lld.ch04.ocp.after.InvoiceGenerator;

public class Main {
    public static void main(String[] args) {
        BigDecimal amount = new BigDecimal("120.00"); // money: BigDecimal with 2 decimals

        System.out.println("--- Before: branch per type ---");
        var old = new lld.ch04.ocp.before.Invoice(amount);
        old.generate("basic");
        old.generate("international");

        System.out.println("--- After: one class per format ---");
        List<InvoiceGenerator> generators = List.of(
                new BasicInvoiceGenerator(),
                new InternationalInvoiceGenerator(),
                new DetailedInvoiceGenerator(),
                // A lambda is also an extension: no new file, no edited file.
                a -> System.out.println("Generating pro-forma invoice for amount: " + a));
        for (InvoiceGenerator g : generators) {
            new Invoice(amount, g).generate();
        }
    }
}
```

Output:

```
--- Before: branch per type ---
Generating basic invoice for amount: 120.00
Generating international invoice for amount: 120.00
--- After: one class per format ---
Generating basic invoice for amount: 120.00
Generating international invoice for amount: 120.00
  including customs declaration and currency conversion
Generating detailed invoice for amount: 120.00
  line items, tax breakdown and payment terms
Generating pro-forma invoice for amount: 120.00
```

What to notice:

- `Invoice` has **no branch**. It does not know how many formats exist.
- `DetailedInvoiceGenerator` is the only new code for format three. `Invoice`, `BasicInvoiceGenerator` and `InternationalInvoiceGenerator` were not opened.
- `InvoiceGenerator` is a `@FunctionalInterface` (an interface with exactly one abstract method), so in Java a **lambda** (a short unnamed function written inline, such as `a -> ...`) is also an extension (the pro-forma line in `Main`). That is OCP with zero new files.

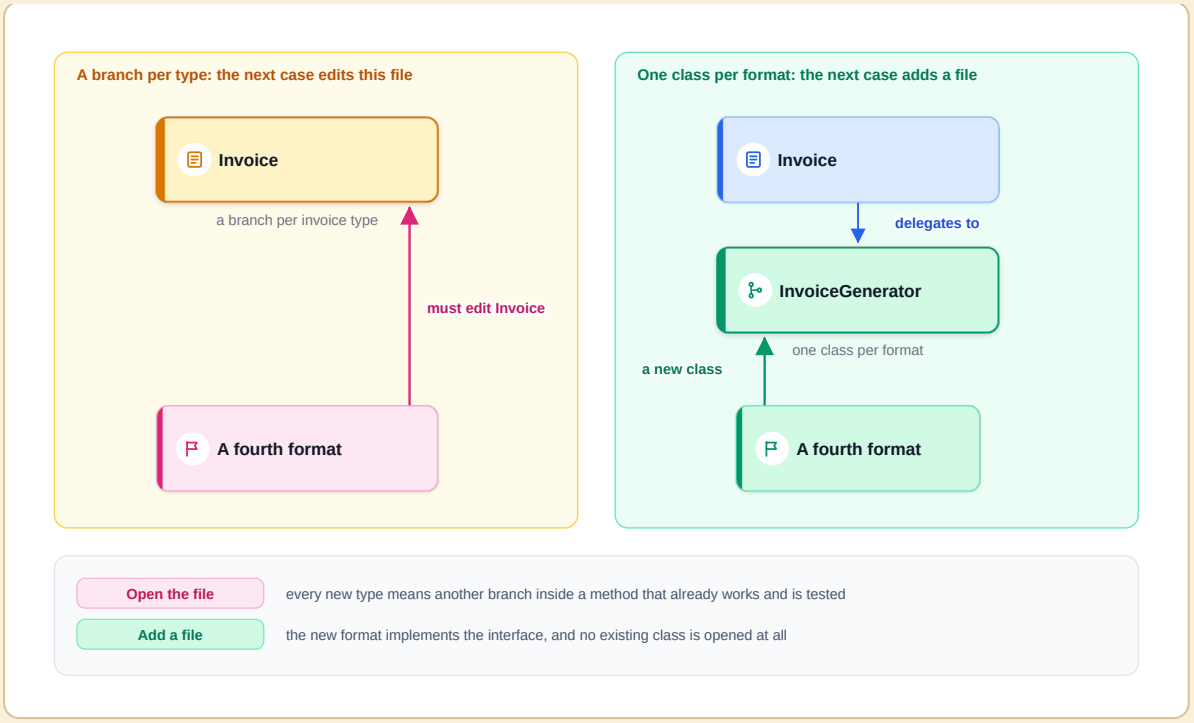
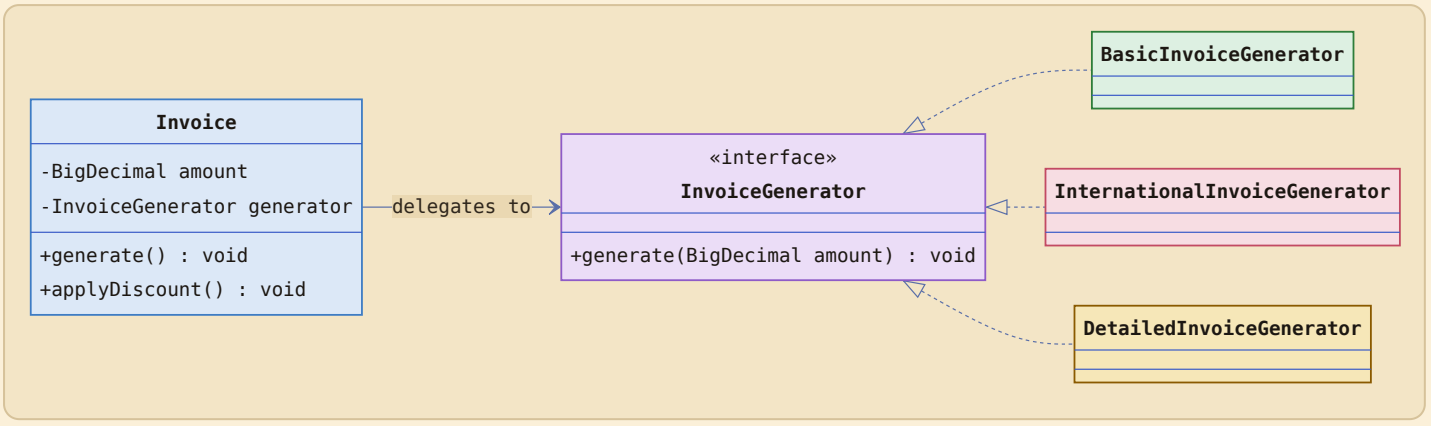


Figure — a branch per type forces an edit to Invoice; one class per format lets a new format be added as a file (course diagram)



This shape (a context object holding an interface-typed field, with interchangeable implementations) is the **Strategy pattern** (a design pattern, i.e. a named reusable design; Chapters 8–11 cover them all). Strategy is OCP applied to an algorithm that varies.

Why not just subclass `Invoice`?

Most readers ask this. Writing `class InternationalInvoice extends Invoice` and overriding `generate()` is also OCP-compliant (it is Meyer's original style): a class is added, nothing is edited. In a small program that is perfectly acceptable. Delegation wins as the system grows, for two reasons:

- **A second dimension of change.** If invoices differ by format **and** by currency, subclassing needs a class for every pairing (3 formats × 4 currencies = 12 classes), while delegation combines two independent objects (3 + 4 = 7 classes).
- **Deciding while the program runs.** An object's class is fixed at `new`. A generator can be picked once the customer's country is known, and even replaced later on the same invoice.

Inheritance is therefore a legitimate option; delegation simply copes with more situations. (*Head First Design Patterns*, a popular introductory book, calls this "favour composition over inheritance".)

What is still not closed

Only invoice *generation* was opened up. `applyDiscount` still lives inside `Invoice`, so a new kind of discount still means an edit there, and that is fine. **"Closed" is never absolute: a class is closed against a specific kind of change**, normally one you have already watched happen. Discount rules have

not varied yet, so there is no reason to abstract them yet. Martin's term for this is **strategic closure**: since no real program can be closed against every possible change, the designer picks which kinds of change to close against, guided by experience.

Approaches to OCP

Approach	How it extends	Pattern / Java feature	Good when
Interface + delegation	New class implements the interface	Strategy, `Comparator`	The algorithm varies, choose at run time
Inheritance + overriding	Subclass overrides a hook method (an overridable step the parent calls)	Template Method, abstract classes	Fixed skeleton, a few steps vary
Wrapping	New class wraps the old and adds behaviour	Decorator (`BufferedInputStream`)	Add features around existing behaviour
Listening	New subscriber registers	Observer, event listeners	New reactions to existing events
Plug-ins / registry	New implementation registers itself in a map or via `ServiceLoader` (Java's built-in way to discover implementations at start-up)	Plug-in architecture	Third parties add features
Functions	Pass a lambda	`java.util.function`, `@FunctionalInterface`	Small, one-method variations
Data / configuration	New entry in a table or config file	Rules engines, feature flags (on/off switches kept in configuration)	Variations that are just different numbers

The `switch` does not always disappear. It moves to **one** place at the edge of the program, often a factory or a `Map<String, InvoiceGenerator>`, instead of being repeated inside business logic.

Importance of OCP

- **Extensibility** — features are added without touching the tested core.
- **Faster testing** — you test the new class, and the core's tests stay valid.
- **Fewer regressions** (old features that break after a change) — working code is not edited, so it is not broken.
- **Adaptability** — the software absorbs new requirements without rewrites.

Caution: do not over-engineer

- **Start simple.** Write the `if` first. Introduce the interface when the second or third case actually arrives (the "rule of three").
- **Balance.** Too many abstraction layers make the code harder to read, which defeats the purpose.
- **Think about real needs.** Extension points for futures that never arrive are wasted effort. *Refactoring* calls this smell **Speculative Generality**.

How OCP relates to SRP

The SOLID principles reinforce one another. OCP pulls the parts that change away from the core that stays put, and each changing part ends up in a class with a single job, which is SRP. In practice SRP usually has to happen first, because the extension point stays hidden until the class is down to one job.

Trade-offs

You gain	You pay
New variants without editing old code	An interface plus one class per variant
Each variant tested on its own	Behaviour is spread over files; reading the whole flow takes more hops
Variants selectable at run time	Someone must choose the variant (factory, map, DI, i.e. dependency injection, Section 4.10)

When not to use it

- Nothing has varied yet.
- The "variants" are just different numbers (use data, not classes).
- The set of cases is fixed and small and will not grow (e.g. the two values of a boolean). Java 21's **sealed interfaces with pattern-matching** `switch` (a sealed interface lists the only types allowed to implement it, so a `switch` can check it has covered them all) are a good fit here: the compiler checks every case is handled, which is the opposite trade (easy to add operations, hard to add types).

Common mistakes

- Thinking OCP means "never edit a file". Bug fixes, refactoring and changes to the extension point itself all involve editing.
- Building plug-in systems for one implementation.
- Replacing a `switch` in business logic with a `switch` in five factories.

Refactorings and smells

- Smell: **Repeated Switches** (called *Switch Statements* in the first edition of *Refactoring*).
- Fix: **Replace Conditional with Polymorphism**; also **Replace Type Code with Subclasses**.

Real-world examples

- `Comparator`: `list.sort(comparator)` never changes; you pass a new comparator for a new ordering.
- **Java I/O decorators**: `new BufferedInputStream(new FileInputStream(f))` adds buffering without editing `FileInputStream`.
- **Servlet filters / Spring** `HandlerInterceptor` (code that runs before and after every web request in a Java web server): new cross-cutting behaviour is a new filter registered in a chain.
- **JDBC drivers** via `ServiceLoader`: a new database is supported by adding a driver JAR (a zipped bundle of compiled Java classes), not by editing `DriverManager`.
- **IDE plug-ins** (IntelliJ, VS Code extensions) and **browser extensions**: the core is closed, the plug-in API is the port.

In the interview

> **30-second answer:** "OCP says you should be able to add the next variation by adding code, not editing working code. When I see a `switch` over a type that keeps growing, I put the varying behaviour behind an interface, one class per variant, and the stable class delegates to it. That is Strategy. I only do it once the variation has actually appeared, and I say that a class is closed against a specific kind of change, not against everything."

Follow-ups:

- *"Does OCP mean you never edit a class?"* — No. It means the next variation of a known kind is an addition. Bug fixes and refactoring still edit.
- *"Inheritance or delegation?"* — Both satisfy OCP. Delegation handles several axes of variation and run-time choice better.

- *"Would you add an interface for pricing up front?"* — Only if pricing has already varied or is clearly about to. Otherwise I leave the `if` and refactor when the second case shows up.
- *"The interviewer adds: 'trucks are charged a different rate'."* — A design that answers "add a class" beats one that answers "add a branch in this switch".

Self-check

1. **Q:** What does "closed for modification" mean? **A:** Adding the next variation does not require editing tested code.
2. **Q:** After the refactor, is `Invoice` closed against new discount rules? **A:** No, only against new formats. That is fine until discounts vary.
3. **Q:** Which pattern is "OCP for an algorithm"? **A:** Strategy.
4. **Q:** Give a quick OCP test. **A:** "To add the next case, must I open this file?"

TL;DR

Put the part that varies behind an interface, add variants as new classes, choose them at the edge. Do it when variation is real, and remember closure is always against a specific kind of change.

4.8 Liskov Substitution Principle (LSP)

What it is

Objects of a subtype must be usable wherever objects of the parent type are expected, without making the program incorrect. Barbara Liskov introduced the idea in a 1987 keynote; with Jeannette Wing she later (1994) defined it precisely as *behavioural subtyping*. Robert C. Martin's short form is: **subtypes must be substitutable for their base types.**

"Substitutable" means more than "it compiles". Code written against the parent type must still get **correct results** when handed a child.

> **Analogy:** a hire-car company promises "a car that takes petrol". If they hand you an electric car and you drive to a petrol station, the promise was broken, even though an electric car is certainly a car.

The problem it solves

Polymorphism only works if callers can trust the base type. If some subclass secretly behaves differently, every caller has to check which subclass it got (`instanceof`), and the benefit of polymorphism, and of OCP, is gone.

Example 1: `Car` and `ElectricCar`

`Car` has `start()`, `stop()` and `refuel()`. `ElectricCar extends Car`. An electric car starts and stops fine, but `refuel()` makes no sense, so it throws. It also adds its own `recharge()`.

```java
// File: code/lld/ch04/lsp/car/before/Car.java
package lld.ch04.lsp.car.before;

public class Car {
    public void start() {
        System.out.println("Car is starting");
    }

    public void stop() {
        System.out.println("Car is stopping");
    }

    /** Names a MECHANISM (liquid fuel) instead of a goal. */
    public void refuel() {
        System.out.println("Car is refueling");
    }
}
```

```java
// File: code/lld/ch04/lsp/car/before/ElectricCar.java
package lld.ch04.lsp.car.before;

/** VIOLATION: inherits a promise (refuel) it cannot keep. */
public class ElectricCar extends Car {
    @Override
    public void refuel() {
        throw new UnsupportedOperationException("Electric cars don't need refueling");
    }

    public void recharge() {
        System.out.println("Electric car is recharging");
    }
}
```

Code that *knows* it holds an `ElectricCar` can call `recharge()` and never notice a problem. But the new method does not make `refuel()` go away; the broken method is still part of the object's public face. LSP is tested by code that does **not** know the subclass, such as `service(Car car)` in `Main` below: it calls `start`, `refuel` and `stop` on whatever `Car` it receives, and it crashes when that car is electric.

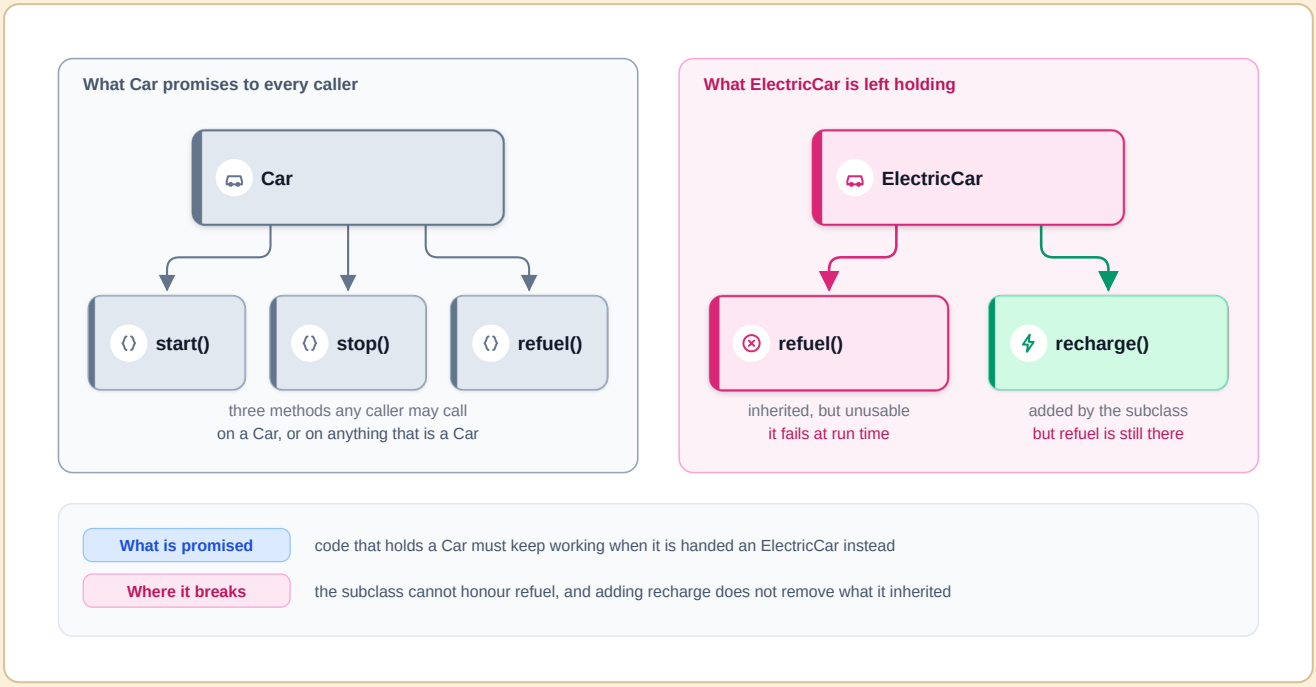


Figure — what Car promises every caller, and the unusable refuel() that ElectricCar is left holding (course diagram)

As the figure above shows, `ElectricCar` cannot fulfil the contract of `Car`, so it is not a valid substitute.

Name the real mistake. "`ElectricCar` should not extend `Car`" is only half the diagnosis. The root cause is the method itself: `refuel()` describes one **mechanism** (pouring in liquid fuel) that only some vehicles use. A method on a base class should describe the **goal** that all subclasses have in common. Every vehicle must take on energy before driving, so the common goal is `replenishEnergy()`.

The fix: raise a general parent. `Vehicle` declares the goal. `Car` and `ElectricCar` both extend `Vehicle`, and **neither extends the other**:

```
// File: code/lld/ch04/lsp/car/after/Vehicle.java
package lld.ch04.lsp.car.after;

/** The base type promises the GOAL every vehicle shares, not one mechanism. */
public abstract class Vehicle {
    public void start() {
        System.out.println(getClass().getSimpleName() + " is starting");
    }

    public void stop() {
        System.out.println(getClass().getSimpleName() + " is stopping");
    }

    public abstract void replenishEnergy();
}
```

```
// File: code/lld/ch04/lsp/car/after/Car.java
package lld.ch04.lsp.car.after;

public class Car extends Vehicle {
    @Override
    public void replenishEnergy() {
        System.out.println("Car is pumping gasoline");
    }
}
```

```
// File: code/lld/ch04/lsp/car/after/ElectricCar.java
package lld.ch04.lsp.car.after;

/** A sibling of Car, not a child of it. Both can stand in for Vehicle. */
public class ElectricCar extends Vehicle {
    @Override
    public void replenishEnergy() {
        System.out.println("Electric car is charging its battery");
    }
}
```

Three points worth stating carefully:

- **The easy workaround is still wrong.** You could keep `refuel()` and have `ElectricCar.refuel()` secretly charge the battery. The exception disappears, but the method name keeps telling callers something untrue about half the fleet. Renaming fixes the promise itself rather than disguising it.
- **Why not just add `recharge()` to `ElectricCar`?** Code typed as `Vehicle` has no way to call it. A caller may depend only on what the base type declares, never on extras one subclass happens to offer.
- **Substitution goes from child to parent, never between siblings.** `Car` and `ElectricCar` can each replace a `Vehicle`. An `ElectricCar` was never meant to replace a `Car`; the two are siblings.

When to break a hierarchy, watch for:

- a subclass that **cannot implement** a parent method (it throws or does nothing),
- a base method that describes a **mechanism** rather than a goal,
- callers that **test the type** (`if (v instanceof ElectricCar)`), a sign that the base type promises too little or promises the wrong thing,
- subclasses with almost nothing in common, where the shared parent invents a relationship that is not really there.

The fix is usually one of two moves: **raise a general parent** (as above) or **take the varying behaviour out of the hierarchy** with composition (next).

Example 2: birds, and composition instead of inheritance

Here `fly()` is declared on `Bird`, so penguins inherit it and have to reject it. The fault is not the penguin; it is that `Bird` made a promise only part of its family can keep. (In *Refactoring*, a subclass that does not want what it inherits is the **Refused Bequest** smell.)

Two ways to connect classes:

- **Inheritance is an is-a relationship.** `Sparrow extends Bird`: a sparrow is a bird and receives **every** method of `Bird`, whether it suits it or not.
- **Composition is a has-a relationship.** `Bird` holds a field of type `MovementStyle`: a bird *has* a way of moving and calls it. It inherits nothing from it.

The fix: turn the varying behaviour (how a bird moves) into its own small type, and give each bird one through its constructor. There are **no bird subclasses** at all. A sparrow and a penguin are both `Bird`, built with a different `MovementStyle`.

```java
// File: code/lld/ch04/lsp/birds/inheritance/Bird.java
package lld.ch04.lsp.birds.inheritance;

/** Promises fly() to EVERY bird -- a promise only some birds can keep. */
public class Bird {
    public void fly() {
        System.out.println(getClass().getSimpleName() + " is flying");
    }
}
```

```java
// File: code/lld/ch04/lsp/birds/inheritance/Sparrow.java
package lld.ch04.lsp.birds.inheritance;

public class Sparrow extends Bird { }
```

```java
// File: code/lld/ch04/lsp/birds/inheritance/Penguin.java
package lld.ch04.lsp.birds.inheritance;

/** VIOLATION ("refused bequest"): inherits fly() and has to refuse it. */
public class Penguin extends Bird {
    @Override
    public void fly() {
        throw new UnsupportedOperationException("Penguins can't fly");
    }
}
```

```java
// File: code/lld/ch04/lsp/birds/composition/MovementStyle.java
package lld.ch04.lsp.birds.composition;

/** The behaviour that varies, pulled out into its own small type. */
@FunctionalInterface
public interface MovementStyle {
    void move(String birdName);
}
```

```java
// File: code/lld/ch04/lsp/birds/composition/Flying.java
package lld.ch04.lsp.birds.composition;

public class Flying implements MovementStyle {
    @Override
    public void move(String birdName) {
        System.out.println(birdName + " is flying through the air");
    }
}
```

```java
// File: code/lld/ch04/lsp/birds/composition/Swimming.java
package lld.ch04.lsp.birds.composition;

public class Swimming implements MovementStyle {
    @Override
    public void move(String birdName) {
        System.out.println(birdName + " is swimming through the water");
    }
}
```

```java
// File: code/lld/ch04/lsp/birds/composition/Running.java
package lld.ch04.lsp.birds.composition;

public class Running implements MovementStyle {
    @Override
    public void move(String birdName) {
        System.out.println(birdName + " is running along the ground");
    }
}
```

```java
// File: code/lld/ch04/lsp/birds/composition/Bird.java
package lld.ch04.lsp.birds.composition;

import java.util.Objects;

/** A bird HAS-A movement style. No subclasses, so nothing is inherited and refused. */
public final class Bird {
    private final String name;
    private MovementStyle movement;

    public Bird(String name, MovementStyle movement) {
        this.name = name;
        this.movement = Objects.requireNonNull(movement);
    }

    public void eat() {
        System.out.println(name + " is eating");
    }

    /** Tell, don't ask: callers say "travel", the bird decides how. */
    public void travel() {
        movement.move(name);
    }

    /** Composition can change behaviour at run time (e.g. an injured bird). */
    public void changeMovement(MovementStyle movement) {
        this.movement = Objects.requireNonNull(movement);
    }
}
```

Why this satisfies LSP: `Bird` now promises only what every bird can do (`eat()` and `travel()`), so there is nothing any bird must refuse. Flight has become one optional object that some birds are given, not a promise to all of them. Supporting the ostrich took one new movement style instead of an `Ostrich` subclass stuck with a meaningless `fly()`. And since the style is just a field, it can be replaced while the program runs (`changeMovement`).

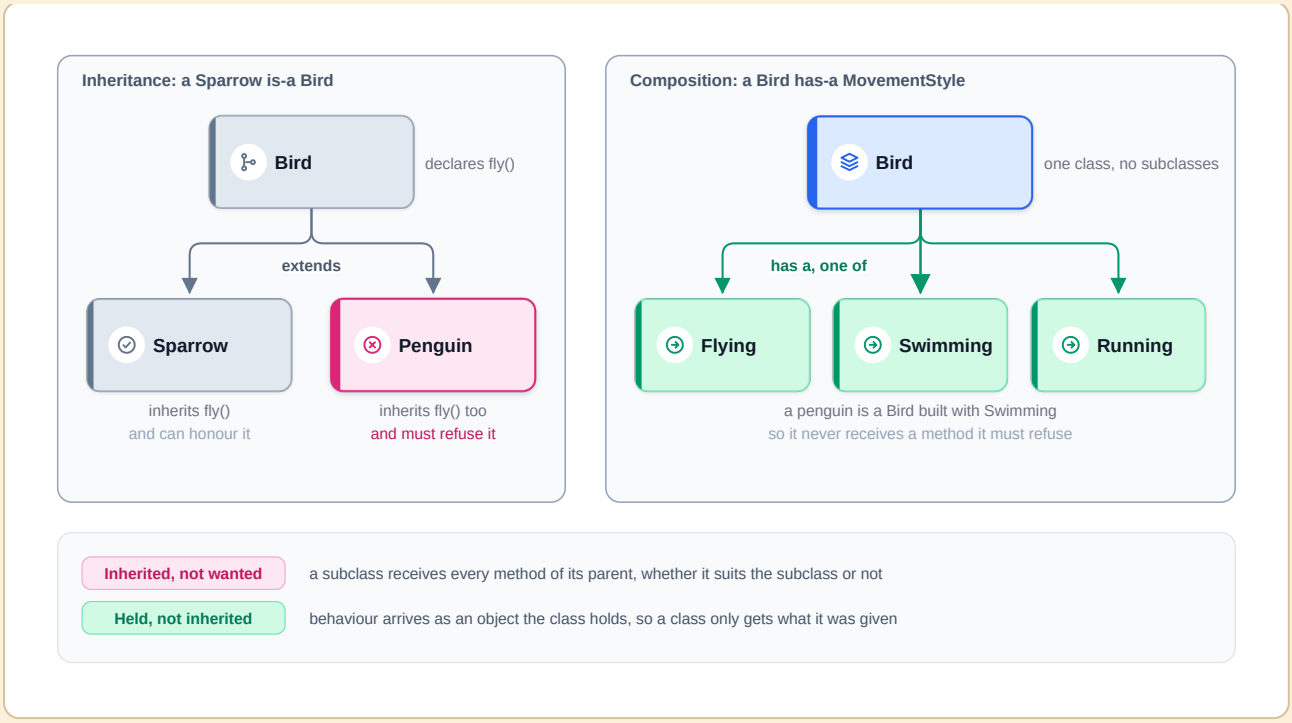
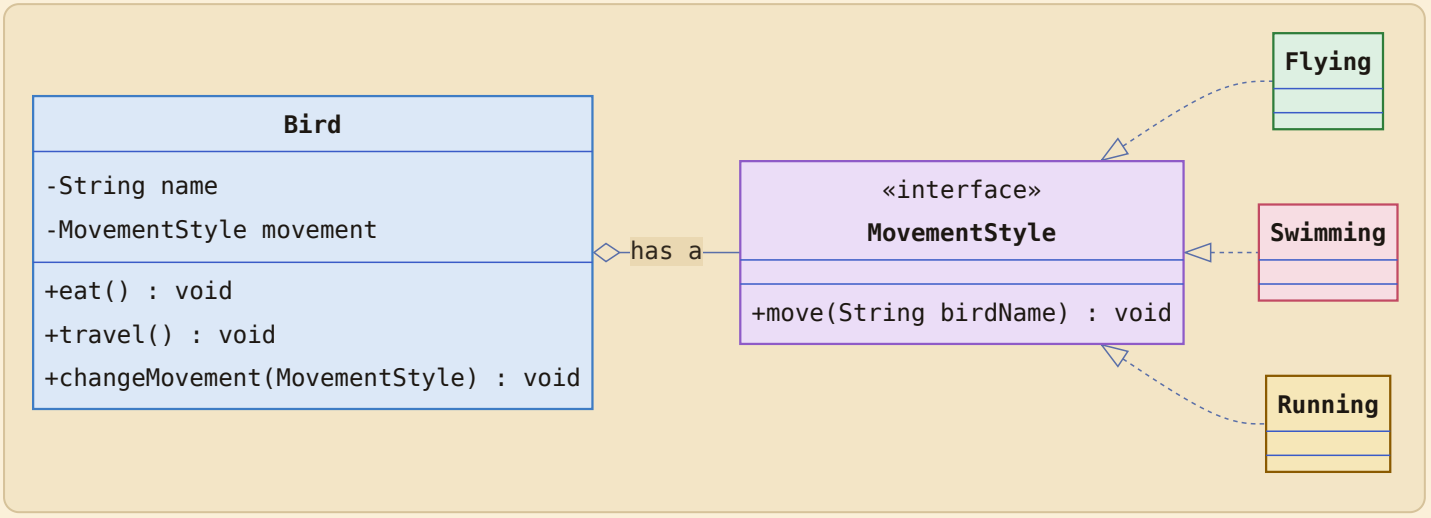


Figure — inheritance (Penguin inherits fly() and refuses it) vs composition (a Bird has a MovementStyle) (course diagram)

In the figure above, the left side is the inheritance tree where the penguin receives a method it does not want. The right side is composition: one `Bird` class holding one of `Flying`, `Swimming`, `Running`.



Is-a and has-a side by side:

	Inheritance	Composition
Kind of link	Penguin **is a** Bird	Bird **has a** MovementStyle
In code	`class Penguin extends Bird`	`Bird` holds a `MovementStyle` field
What it gets	all of the parent's methods	just the object handed to it
To change behaviour	add another subclass	supply another object
Change while running?	no, fixed at creation	yes, replace the field
LSP risk	high: inherited methods may not suit	low: nothing is inherited

When to use which. Choose **inheritance** when the child genuinely is a kind of the parent *and* can keep every inherited promise. Choose **composition** when what differs between objects is a behaviour, or the moment you find a subclass rejecting something it inherited. That rejection is the tell: an overridden

method that throws, silently does nothing, or carries a "never call this on X" comment should become a separate object.

Tell, Don't Ask

The composition design above follows a related guideline, popularised by Andy Hunt and Dave Thomas (*The Pragmatic Programmer* authors): **tell an object what to do; do not ask it about its state and then decide for it.**

```
// Asking: the caller inspects the object and makes the decision.
if (bird instanceof Penguin) { swim(bird); } else { fly(bird); }

// Telling: the caller states the goal, the object decides how.
bird.travel();
```

Asking code has to know every subtype, so it breaks OCP and hides LSP problems behind `instanceof` checks. Telling code works with any substitute. In the composition design, `bird.travel()` is the "tell". A caller that needs `instanceof` on a base type is a strong hint that the base type promises the wrong thing.

The rules a subclass must follow

So far every violation crashed with an exception, which makes it easy to spot. The harmful ones are those that **quietly produce a wrong result and keep running**.

Example: `Square extends Rectangle`. Geometry says every square is a rectangle, so the subclass looks natural. To remain square, `Square` keeps both sides equal: setting the width also changes the height. No exception is thrown. Yet a method written for `Rectangle` that sets width 5 and height 4 expects an area of 20 and receives 16. In production that kind of error becomes an incorrect bill or inventory figure that nobody notices for weeks.

The rule that `Rectangle` implied without ever stating is **width and height can be changed independently**; the two separate setters invited every caller to rely on it. Mathematically a square is a rectangle, but it is **not a valid subtype of this `Rectangle` class**, because the class makes a promise a square cannot honour. **Base inheritance on the promises written in the code, not on how things are classified in the real world.**

A subclass is free to do the job *differently*. What it may not do is **require more** from its callers, **give them less**, or **break an assumption they depend on**. The rules below spell that out. Their vocabulary comes from Bertrand Meyer's **Design by Contract**: a method has **preconditions** (what the caller must provide), **postconditions** (what the method guarantees afterwards) and **invariants** (what is always true of the object).

Rule 1 — Preconditions may not be strengthened. A precondition is what must be true before the call. An overriding method may accept a *wider* range of inputs than the parent, never a narrower one. In the example, `Discount` accepts any amount of zero or more, while `SmallOrderDiscount` refuses anything from 100 upward. A caller that passes 250, which the parent allows, now fails, and would have to find out which discount it was given, the very check substitution was meant to make unnecessary.

Rule 2 — Postconditions may not be weakened. A postcondition is what the method guarantees when it returns. An override may guarantee *more*, never less. `Account.withdraw` guarantees the balance stays at zero or above. `OverdraftAccount` lets it go negative: a reasonable banking feature, but a broken guarantee. Every caller that relied on "balance ≥ 0" is now silently wrong. Nothing crashes; a payment job might, for example, keep paying suppliers from an account that is already overdrawn. An overdraft account can be a perfectly good class, just not a subtype of an account that promises a non-negative balance.

Rule 3 — Invariants must be preserved. An invariant is a condition that holds throughout the object's life. `Square` broke "width and height are independent". (The same bug can equally be described as a weakened postcondition: `Rectangle.setWidth` implicitly guarantees "the height is unchanged", and `Square.setWidth` drops that guarantee. Martin describes it that way; either description leads to the same conclusion.) Invariants are the hardest rule to follow because they are rarely written down: a subclass inherits every promise of its parent, including the ones nobody documented.

Rule 4 — No new exceptions callers cannot handle. An override may throw the exceptions the parent already declares (or subtypes of them), nothing new: a new exception type arrives at callers that have no `catch` for it. `ElectricCar.refuel()` broke this rule. Note that Java's compiler enforces this only for **checked** exceptions; `UnsupportedOperationException` is unchecked, so the violating code compiled without complaint. This is the most visible rule to break and the least harmful, because the failure is immediate and loud.

Rule 5 — The history constraint. Liskov and Wing added a rule about how state changes over time: **a subtype must not allow state changes that the parent type does not allow.** In the example, `Price` promises that its value never changes after creation. `AdjustablePrice` adds an `adjust()` method. Each call looks innocent, but a caller that captured a quote of 100 is later charged 120 on "the same" price. This is why immutable Java classes such as `String` and `Integer` are declared `final`: a mutable subclass could break the immutability callers rely on.

Here are all of them in runnable code:

```java
// File: code/lld/ch04/lsp/rules/Discount.java
package lld.ch04.lsp.rules;

import java.math.BigDecimal;
import java.math.RoundingMode;

/** Contract -- precondition: any amount >= 0. Postcondition: result = 90% of amount. */
public class Discount {
    private static final BigDecimal RATE = new BigDecimal("0.90");

    public BigDecimal apply(BigDecimal amount) {
        if (amount.signum() < 0) {
            throw new IllegalArgumentException("amount must be >= 0");
        }
        return amount.multiply(RATE).setScale(2, RoundingMode.HALF_EVEN);
    }
}
```

```java
// File: code/lld/ch04/lsp/rules/SmallOrderDiscount.java
package lld.ch04.lsp.rules;

import java.math.BigDecimal;

/** RULE 1 BROKEN: demands MORE than the parent (amount must also be under 100). */
public class SmallOrderDiscount extends Discount {
    private static final BigDecimal LIMIT = new BigDecimal("100");

    @Override
    public BigDecimal apply(BigDecimal amount) {
        if (amount.compareTo(LIMIT) >= 0) {
            throw new IllegalArgumentException("only for orders under 100");
        }
        return super.apply(amount);
    }
}
```

```java
// File: code/lld/ch04/lsp/rules/Account.java
package lld.ch04.lsp.rules;

import java.math.BigDecimal;

/** Contract -- postcondition of withdraw: the balance is never negative. */
public class Account {
    protected BigDecimal balance;

    public Account(BigDecimal balance) {
        this.balance = balance;
    }

    /** Withdraws only if funds allow; returns the balance afterwards (always >= 0). */
    public BigDecimal withdraw(BigDecimal amount) {
        if (amount.compareTo(balance) <= 0) {
            balance = balance.subtract(amount);
        }
        return balance;
    }
}
```

```java
// File: code/lld/ch04/lsp/rules/OverdraftAccount.java
package lld.ch04.lsp.rules;

import java.math.BigDecimal;

/** RULE 2 BROKEN: promises LESS -- the balance may now go negative. Nothing throws. */
public class OverdraftAccount extends Account {
    public OverdraftAccount(BigDecimal balance) {
        super(balance);
    }

    @Override
    public BigDecimal withdraw(BigDecimal amount) {
        balance = balance.subtract(amount);
        return balance;
    }
}
```

```java
// File: code/lld/ch04/lsp/rules/Rectangle.java
package lld.ch04.lsp.rules;

/** Unwritten invariant: width and height can be set independently. */
public class Rectangle {
    protected int width;
    protected int height;

    public Rectangle(int width, int height) {
        this.width = width;
        this.height = height;
    }

    public void setWidth(int width) {
        this.width = width;
    }

    public void setHeight(int height) {
        this.height = height;
    }

    public int area() {
        return width * height;
    }
}
```

```java
// File: code/lld/ch04/lsp/rules/Square.java
package lld.ch04.lsp.rules;

/** RULE 3 BROKEN: to stay square it couples the two setters -- a silent wrong answer. */
public class Square extends Rectangle {
    public Square(int side) {
        super(side, side);
    }

    @Override
    public void setWidth(int width) {
        this.width = width;
        this.height = width;
    }

    @Override
    public void setHeight(int height) {
        this.width = height;
        this.height = height;
    }
}
```

```java
// File: code/lld/ch04/lsp/rules/Price.java
package lld.ch04.lsp.rules;

import java.math.BigDecimal;

/** Contract (history): once created, a Price never changes value. */
public class Price {
    private final BigDecimal value;

    public Price(BigDecimal value) {
        this.value = value;
    }

    public BigDecimal value() {
        return value;
    }
}
```

```java
// File: code/lld/ch04/lsp/rules/AdjustablePrice.java
package lld.ch04.lsp.rules;

import java.math.BigDecimal;

/**
 * HISTORY CONSTRAINT BROKEN: adds a mutator, so an object the caller believed was
 * frozen can now change between two reads. No single method call looks wrong.
 */
public class AdjustablePrice extends Price {
    private BigDecimal current;

    public AdjustablePrice(BigDecimal value) {
        super(value);
        this.current = value;
    }

    @Override
    public BigDecimal value() {
        return current;
    }

    public void adjust(BigDecimal newValue) {
        this.current = newValue;
    }
}
```

Why it matters so much. Code written against a base type has no way to protect itself: it cannot see which subclass it received, and it should not need to. Only the subclass author can keep the contract.

The fix for Rectangle/Square. Do not share a mutable promise that one of them cannot keep. Make both **immutable** and give them a common interface that promises only what both can do (`area()`). "Resizing" returns a new object, so no setter can break an invariant. Java 21 `record`s and a `sealed` interface make this short:

```java
// File: code/lld/ch04/lsp/shapes/Shape.java
package lld.ch04.lsp.shapes;

/** The fix for Rectangle/Square: promise only what every shape can keep. */
public sealed interface Shape permits Rect, Sq {
    int area();
}
```

```java
// File: code/lld/ch04/lsp/shapes/Rect.java
package lld.ch04.lsp.shapes;

/** Immutable: "resizing" returns a new object, so no setter can break an invariant. */
public record Rect(int width, int height) implements Shape {
    public Rect withWidth(int w) {
        return new Rect(w, height);
    }

    @Override
    public int area() {
        return width * height;
    }
}
```

```java
// File: code/lld/ch04/lsp/shapes/Sq.java
package lld.ch04.lsp.shapes;

public record Sq(int side) implements Shape {
    @Override
    public int area() {
        return side * side;
    }
}
```

And the driver that runs every example, with each client method written only against the base type:

```java
// File: code/lld/ch04/lsp/Main.java
package lld.ch04.lsp;

import java.math.BigDecimal;
import java.util.List;
import lld.ch04.lsp.birds.composition.Bird;
import lld.ch04.lsp.birds.composition.Flying;
import lld.ch04.lsp.birds.composition.Running;
import lld.ch04.lsp.birds.composition.Swimming;
import lld.ch04.lsp.car.after.Vehicle;
import lld.ch04.lsp.rules.Account;
import lld.ch04.lsp.rules.AdjustablePrice;
import lld.ch04.lsp.rules.Discount;
import lld.ch04.lsp.rules.OverdraftAccount;
import lld.ch04.lsp.rules.Price;
import lld.ch04.lsp.rules.Rectangle;
import lld.ch04.lsp.rules.SmallOrderDiscount;
import lld.ch04.lsp.rules.Square;
import lld.ch04.lsp.shapes.Rect;
import lld.ch04.lsp.shapes.Shape;
import lld.ch04.lsp.shapes.Sq;

public class Main {

    // ---- Client code: every method below knows ONLY the base type ----

    static void service(lld.ch04.lsp.car.before.Car car) {
        car.start();
        car.refuel();
        car.stop();
    }

    static void takeATrip(Vehicle v) {
        v.start();
        v.replenishEnergy();
        v.stop();
    }

    static void checkout(Discount d, BigDecimal amount) {
        try {
            System.out.println("  Charged: " + d.apply(amount));
        } catch (IllegalArgumentException e) {
            System.out.println("  Broke on a valid amount: " + e.getMessage());
        }
    }

    static void paySupplier(Account account, BigDecimal amount, String label) {
        BigDecimal balance = account.withdraw(amount);
        System.out.println("  " + label + " balance after payment: " + balance
                + (balance.signum() < 0 ? "   <-- the caller never expected this" : ""));
    }

    /** A contract test written against Rectangle -- run it for every subclass. */
    static void resizeAndCheck(Rectangle r, String label) {
        r.setWidth(5);
        r.setHeight(4);
        System.out.println("  " + label + " expected area 20, got " + r.area());
    }

    static void quoteThenCharge(Price p, Runnable somethingElseHappens) {
        BigDecimal quoted = p.value();
        somethingElseHappens.run();
        System.out.println("  quoted " + quoted + ", charged " + p.value());
    }

    public static void main(String[] args) {
        System.out.println("=== 1. Car / ElectricCar (exception rule) ===");
        service(new lld.ch04.lsp.car.before.Car());
        try {
            service(new lld.ch04.lsp.car.before.ElectricCar());
```

```
        } catch (UnsupportedOperationException e) {
            System.out.println("  substitution broke: " + e.getMessage());
        }

        System.out.println("=== 2. Fixed hierarchy: Vehicle -> Car | ElectricCar ===");
        List<Vehicle> fleet = List.of(new lld.ch04.lsp.car.after.Car(),
                new lld.ch04.lsp.car.after.ElectricCar());
        fleet.forEach(Main::takeATrip);

        System.out.println("=== 3. Precondition strengthened ===");
        checkout(new Discount(), new BigDecimal("250.00"));
        checkout(new SmallOrderDiscount(), new BigDecimal("250.00"));

        System.out.println("=== 4. Postcondition weakened ===");
        paySupplier(new Account(new BigDecimal("50.00")), new BigDecimal("80.00"), "Account:         ");
        paySupplier(new OverdraftAccount(new BigDecimal("50.00")), new BigDecimal("80.00"),
                "OverdraftAccount:");

        System.out.println("=== 5. Invariant broken (Rectangle / Square) ===");
        resizeAndCheck(new Rectangle(1, 1), "Rectangle:");
        resizeAndCheck(new Square(1), "Square:   ");

        System.out.println("=== 6. History constraint broken ===");
        quoteThenCharge(new Price(new BigDecimal("100.00")), () -> { });
        AdjustablePrice sneaky = new AdjustablePrice(new BigDecimal("100.00"));
        quoteThenCharge(sneaky, () -> sneaky.adjust(new BigDecimal("120.00")));

        System.out.println("=== 7. Fix for shapes: no shared mutable promise ===");
        List<Shape> shapes = List.of(new Rect(5, 4), new Sq(4), new Rect(2, 3).withWidth(5));
        for (Shape s : shapes) {
            System.out.println("  " + s + " area=" + s.area());
        }

        System.out.println("=== 8. Birds: inheritance vs composition ===");
        List<lld.ch04.lsp.birds.inheritance.Bird> old = List.of(
                new lld.ch04.lsp.birds.inheritance.Sparrow(),
                new lld.ch04.lsp.birds.inheritance.Penguin());
        for (var b : old) {
            try {
                b.fly();
            } catch (UnsupportedOperationException e) {
                System.out.println("  Substitution broke: " + e.getMessage());
            }
        }
        List<Bird> birds = List.of(
                new Bird("Sparrow", new Flying()),
                new Bird("Penguin", new Swimming()),
                new Bird("Ostrich", new Running()));
        birds.forEach(Bird::travel);
        Bird sparrow = birds.get(0);
        sparrow.changeMovement(new Running()); // behaviour swapped at run time
        sparrow.travel();
    }
}
```

Output:

```
=== 1. Car / ElectricCar (exception rule) ===
Car is starting
Car is refueling
Car is stopping
Car is starting
  substitution broke: Electric cars don't need refueling
=== 2. Fixed hierarchy: Vehicle -> Car | ElectricCar ===
Car is starting
Car is pumping gasoline
Car is stopping
ElectricCar is starting
Electric car is charging its battery
ElectricCar is stopping
=== 3. Precondition strengthened ===
  Charged: 225.00
  Broke on a valid amount: only for orders under 100
=== 4. Postcondition weakened ===
  Account:          balance after payment: 50.00
  OverdraftAccount: balance after payment: -30.00   <-- the caller never expected this
=== 5. Invariant broken (Rectangle / Square) ===
  Rectangle: expected area 20, got 20
  Square:    expected area 20, got 16
=== 6. History constraint broken ===
  quoted 100.00, charged 100.00
  quoted 100.00, charged 120.00
=== 7. Fix for shapes: no shared mutable promise ===
  Rect[width=5, height=4] area=20
  Sq[side=4] area=16
  Rect[width=5, height=3] area=15
=== 8. Birds: inheritance vs composition ===
Sparrow is flying
  Substitution broke: Penguins can't fly
Sparrow is flying through the air
Penguin is swimming through the water
Ostrich is running along the ground
Sparrow is running along the ground
```

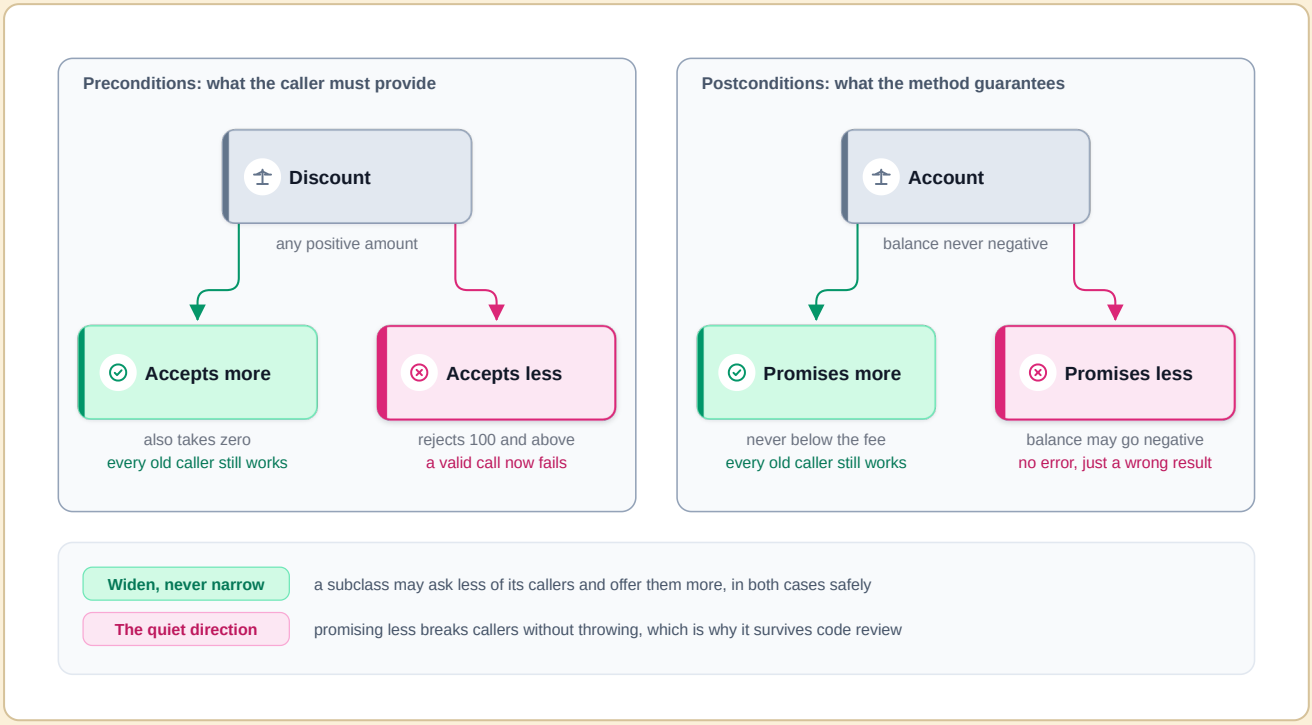


Figure — preconditions and postconditions: accepting more or promising more is safe; accepting less or promising less breaks callers (course diagram)

The figure above sums up the direction: **widen, never narrow.** A subclass may ask less of its callers and offer them more. (In the figure the parent `Discount` accepts positive amounts and the safe subclass also accepts zero; in this chapter's code the parent already accepts zero, but the direction of the

rule is the same. Likewise the figure's "promises more" account keeps the balance above a fee, a stronger guarantee than "never negative".)

The contract in one table:

Rule	The subclass may not…	How it usually shows up
Preconditions	demand more from the caller	valid input is rejected
Postconditions	guarantee less to the caller	a returned value breaks a caller's assumption
Invariants	break a rule that always held	a silently wrong result (the square)
Exceptions	throw something new	a crash in code with no matching `catch`
History	allow state changes the parent forbids	a "fixed" value changes between two reads

Signature rules (the part the compiler checks). Behavioural subtyping also has type-level rules: parameter types may be *wider* (contravariant), return types may be *narrower* (covariant), and exceptions may only be *narrower*. Java enforces a version of this for you:

- an overriding method may return a **subtype** of the parent's return type (covariant returns),
- it may **not** declare new or broader **checked** exceptions,
- it may **not** reduce visibility (a `public` method stays `public`),
- parameter types must match exactly (a different parameter type is an **overload**, not an override; use `@Override` so the compiler catches this).

Java checks none of the **behavioural** rules. Unchecked exceptions, weakened postconditions and broken invariants all compile happily. That part is on you.

How to check a subclass

1. **List the parent's promises before writing the subclass.** Promises, not method signatures: "the balance never drops below zero", "width and height change independently". Most violations slip in because the promise was never stated, so its breach went unnoticed.
2. **Write one test against the parent type and run it for every subclass.** `resizeAndCheck(Rectangle)` is such a test: it never mentions `Square`, yet `Square` fails it. Whenever a base-type test fails for some subclass, that subclass violates LSP, regardless of how reasonable its code looks. (In JUnit, the standard Java testing library, this is an *abstract contract test* that each subclass's test class extends.)

Why LSP is important

- **Prevents fragile code** — callers are not surprised by a subclass.
- **Keeps polymorphism usable** — new subclasses plug in without `instanceof` checks, which is what makes OCP work.
- **Correctness** — results stay right, not just "no crash".
- **Reuse** — base types can be extended safely.

Trade-offs

Following LSP gains	Costs
Callers trust the base type; no type checks	You must think about and document contracts
New subclasses are safe to add	Some "natural" hierarchies (Square/Rectangle) must be abandoned
Composition gives run-time flexibility	More small objects; behaviour is spread out

When not to use it

LSP is not optional where you use polymorphism. What you can skip is inheritance itself: if a subclass cannot keep the parent's promises, **do not make it a subclass**.

Common mistakes

- Modelling the real world ("a square is a rectangle", "a penguin is a bird") instead of the code's promises.
- "Fixing" a violation by making the method do nothing silently. That just changes a loud failure into a quiet wrong result.
- Thinking LSP is only about exceptions. The silent rules (postconditions, invariants, history) are more dangerous.
- Believing that siblings must substitute for each other. Substitution is only child → parent.

Refactorings and smells

- Smell: **Refused Bequest** (a subclass does not want what it inherits).
- Fixes: **Replace Subclass with Delegate**, **Replace Superclass with Delegate** (composition), **Extract Superclass** (raise a general parent), **Push Down Method** (move a method only some subclasses support out of the parent).

Real-world examples

- **`java.sql.Date extends java.util.Date`**: its time methods such as `getHours()` throw `IllegalArgumentException`, because an SQL date has no time part. Code written against `java.util.Date` can break.
- **`java.sql.Timestamp`**: its Javadoc (the official documentation generated from code comments) warns that its `equals` is not symmetric with `java.util.Date.equals`, and says the inheritance is implementation inheritance, not type inheritance. A textbook LSP warning in the JDK itself.
- **`java.util.Stack extends Vector`**: a stack inherits `insertElementAt` and `get(int)`, which let callers break stack order. The Javadoc now recommends `Deque` (a double-ended queue that adds and removes only at its ends) instead.
- **`List.of(...)` and `Collections.unmodifiableList(...)`** throw `UnsupportedOperationException` from `add`. The `List` interface marks `add` as an *optional operation*, so this is written into the contract, but it is a known design compromise: a caller holding a `List` cannot know whether `add` works.

In the interview

> **30-second answer:** "LSP says a subclass must work anywhere its parent is expected, giving correct results, not just no crash. Concretely: it cannot demand more input, promise less output, break an invariant, throw new exceptions, or allow state changes the parent forbids. Square extending a mutable Rectangle is the classic silent break: set width 5 and height 4 and you get 16. I test it by writing one test against the base type and running it for every subclass. When a subclass must refuse a method, I either raise a better parent or move the behaviour into a composed object."

Follow-ups:

- *"Give an LSP violation you've seen."* — The Rectangle/Square case, or `java.sql.Date` throwing from `getHours()`. Then state the rule: demand no more, promise no less, keep the invariants.
- *"Composition over inheritance, always?"* — No. Inheritance is fine when the child truly honours every parent method. The signal to switch is a subclass refusing an inherited method.
- *"Is `ElectricCar` a substitute for `Car` after your fix?"* — No, and it should not be. Both substitute for `Vehicle`; siblings do not substitute for each other.

Self-check

1. **Q:** A subclass accepts **fewer** inputs than its parent. Which rule? **A:** Preconditions strengthened (rule 1).
2. **Q:** Why is the Square/Rectangle bug more dangerous than `ElectricCar.refuel()` throwing? **A:** It gives a silent wrong answer; the exception fails loudly and is found at once.
3. **Q:** What is the history constraint? **A:** A subtype must not allow state changes the parent does not allow, e.g. a mutable subclass of an immutable class.

4. **Q:** What does `@Override` protect you from? **A:** Accidentally overloading (different parameter type) instead of overriding.
5. **Q:** What is "Tell, Don't Ask"? **A:** Tell an object to do the job (`bird.travel()`), instead of inspecting its type or state and deciding for it.

TL;DR

A subclass must keep every promise of its parent: accept at least as much, deliver at least as much, keep invariants and history, throw nothing new. Inheritance follows promises, not real-world categories. When a subclass must refuse something, raise a better parent or use composition.

4.9 Interface Segregation Principle (ISP)

What it is

Clients should not be forced to depend on methods they do not use. Instead of one large, general interface, offer smaller interfaces shaped around what each kind of caller needs.

Where it comes from. Robert C. Martin describes meeting the problem while consulting for Xerox. A printer system had one big `Job` class used by every task (printing, stapling and so on), so a change for one task forced every other task's code to be rebuilt. Splitting the class's services into separate interfaces, one per kind of client, fixed it.

> **Analogy:** a restaurant that hands every guest the full 40-page menu, including the wine list, the kids' menu and the catering price list, even to someone who only wants a coffee. A coffee card is enough for them.

A **fat interface** is an interface with many methods, usually because it grew one feature at a time. A **role interface** is a small interface describing the role an object plays for one kind of caller.

The problem it solves

- Implementers are forced to write methods they cannot honour, so they throw or do nothing (which also breaks LSP).
- Callers depend on methods they never call, so a change to those methods still reaches them (recompiles, redeploys, retests).
- Test doubles become long, because a fake must implement every method.

Example: a bloated `Printer` interface

One interface for everything a printer might do: print, scan, fax, staple.

```java
// File: code/lld/ch04/isp/fat/Printer.java
package lld.ch04.isp.fat;

/** A FAT interface: one list of everything any printer might ever do. */
public interface Printer {
    void printDocument(String doc);

    void scanDocument(String doc);

    void faxDocument(String doc);

    void stapleDocument(String doc);
}
```

```java
// File: code/lld/ch04/isp/fat/BasicPrinter.java
package lld.ch04.isp.fat;

/** VIOLATION: forced to "implement" three methods it can only refuse. */
public class BasicPrinter implements Printer {
    @Override
    public void printDocument(String doc) {
        System.out.println("BasicPrinter printing " + doc);
    }

    @Override
    public void scanDocument(String doc) {
        throw new UnsupportedOperationException("BasicPrinter cannot scan");
    }

    @Override
    public void faxDocument(String doc) {
        throw new UnsupportedOperationException("BasicPrinter cannot fax");
    }

    @Override
    public void stapleDocument(String doc) {
        throw new UnsupportedOperationException("BasicPrinter cannot staple");
    }
}
```

```java
// File: code/lld/ch04/isp/fat/AdvancedPrinter.java
package lld.ch04.isp.fat;

/** The wide interface is honest for this device -- it really does all four jobs. */
public class AdvancedPrinter implements Printer {
    @Override
    public void printDocument(String doc) {
        System.out.println("AdvancedPrinter printing " + doc);
    }

    @Override
    public void scanDocument(String doc) {
        System.out.println("AdvancedPrinter scanning " + doc);
    }

    @Override
    public void faxDocument(String doc) {
        System.out.println("AdvancedPrinter faxing " + doc);
    }

    @Override
    public void stapleDocument(String doc) {
        System.out.println("AdvancedPrinter stapling " + doc);
    }
}
```

`BasicPrinter` only prints, but it must "implement" scanning, faxing and stapling, and the only honest thing it can do is throw. `AdvancedPrinter` really does all four, so for it the wide interface is fine.

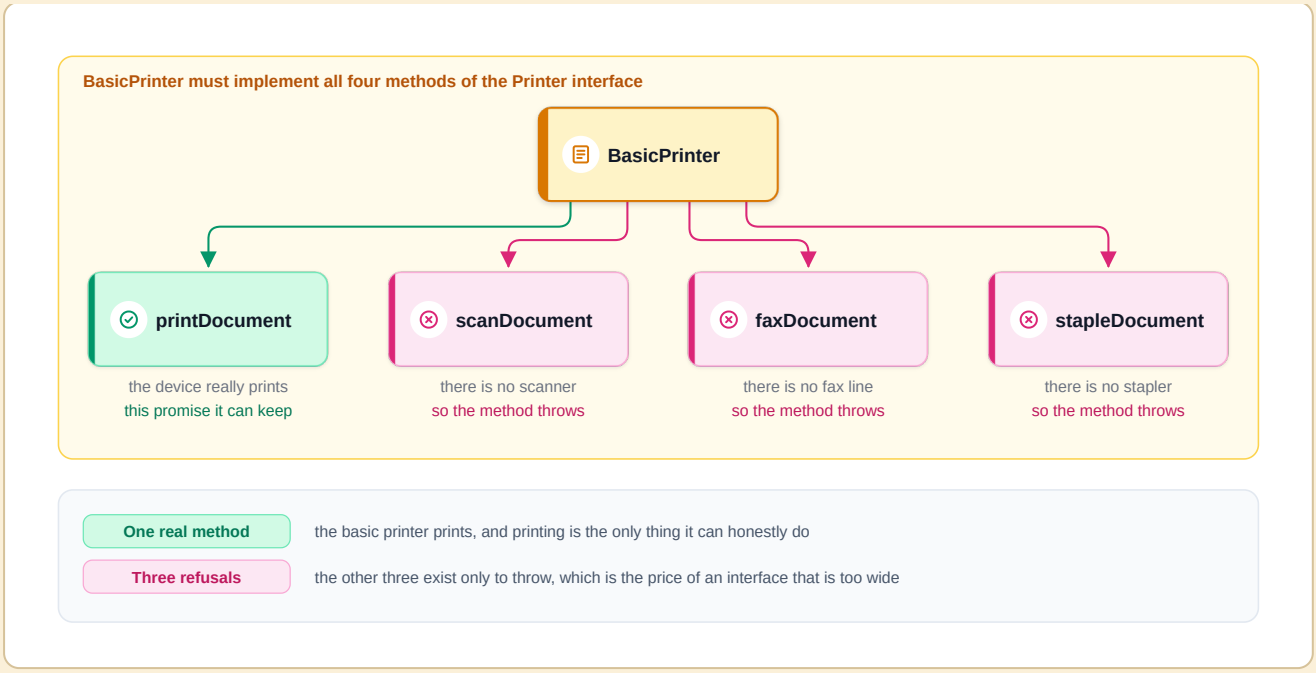


Figure — BasicPrinter keeps one real method and three that exist only to throw (course diagram)

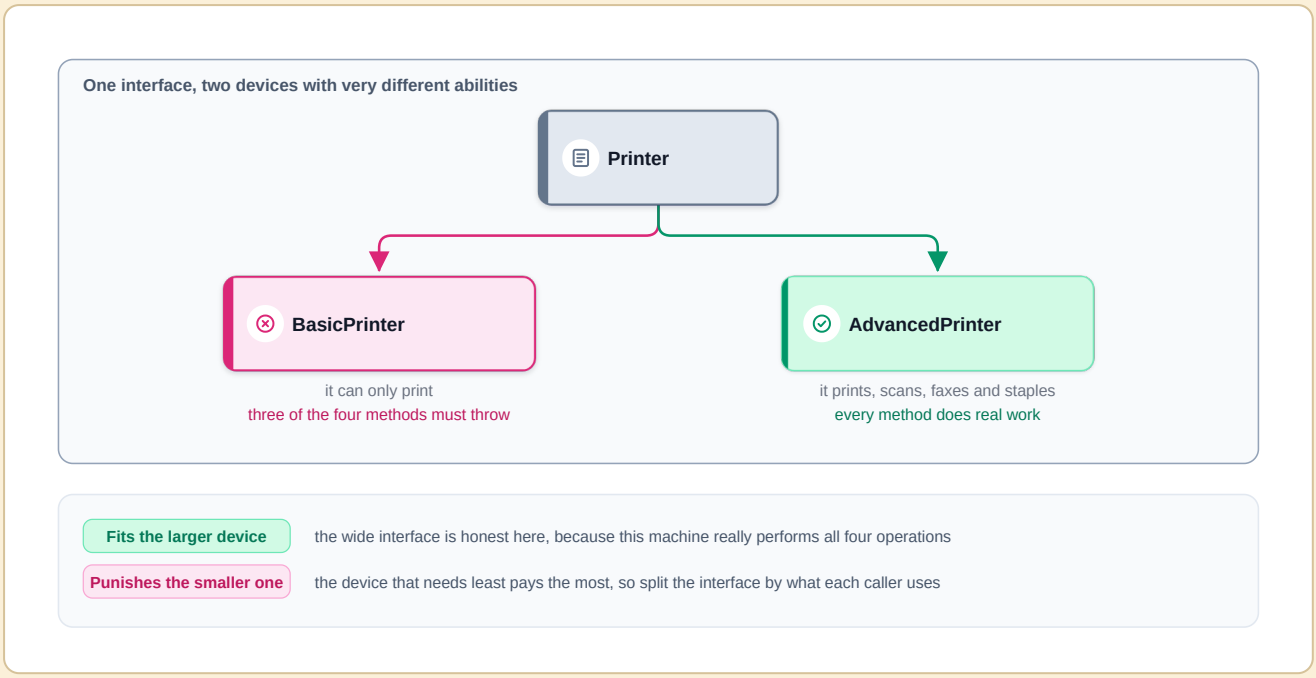


Figure — one interface, two devices: it fits the larger device and punishes the smaller one (course diagram)

The two figures above capture ISP in one picture: the simplest device carries the heaviest burden. Split the interface according to what each caller actually uses.

Restructuring: first attempt, one interface per capability

Split the big interface into `Printer`, `Scanner`, `Fax` and `Stapler`. `BasicPrinter` implements only `Printer`; `AdvancedPrinter` implements all four. No class implements anything it cannot do.

```java
// File: code/lld/ch04/isp/split/Printer.java
package lld.ch04.isp.split;

public interface Printer {
    void printDocument(String doc);
}
```

```java
// File: code/lld/ch04/isp/split/Scanner.java
package lld.ch04.isp.split;

public interface Scanner {
    void scanDocument(String doc);
}
```

```java
// File: code/lld/ch04/isp/split/Fax.java
package lld.ch04.isp.split;

public interface Fax {
    void faxDocument(String doc);
}
```

```java
// File: code/lld/ch04/isp/split/Stapler.java
package lld.ch04.isp.split;

public interface Stapler {
    void stapleDocument(String doc);
}
```

```java
// File: code/lld/ch04/isp/split/BasicPrinter.java
package lld.ch04.isp.split;

/** Implements only what it can honour. */
public class BasicPrinter implements Printer {
    @Override
    public void printDocument(String doc) {
        System.out.println("BasicPrinter printing " + doc);
    }
}
```

```java
// File: code/lld/ch04/isp/split/AdvancedPrinter.java
package lld.ch04.isp.split;

/** Opts in to every capability it really has. */
public class AdvancedPrinter implements Printer, Scanner, Fax, Stapler {
    @Override
    public void printDocument(String doc) {
        System.out.println("AdvancedPrinter printing " + doc);
    }

    @Override
    public void scanDocument(String doc) {
        System.out.println("AdvancedPrinter scanning " + doc);
    }

    @Override
    public void faxDocument(String doc) {
        System.out.println("AdvancedPrinter faxing " + doc);
    }

    @Override
    public void stapleDocument(String doc) {
        System.out.println("AdvancedPrinter stapling " + doc);
    }
}
```

This satisfies ISP and makes the mechanics easy to see. But **one interface per method is not the goal.** If every scanning device can also fax, and every caller uses both, putting them in separate interfaces adds names while removing no dependency at all.

How small should an interface be?

Group by caller, not by method: an interface should carry the methods that one kind of caller needs. The size is right when both answers below are "no":

1. **Is any implementer forced to supply a method it cannot honour?** If yes, the interface is too wide.
2. **Does any caller depend on methods it never calls?** If yes, it is too wide for that caller.

It might have one method or six. The count was never the measure.

What over-splitting costs. Now add a caller to the one-method-per-interface design. `OfficeWorkflow` scans, prints and staples, so it needs three constructor parameters, and whoever creates it hands over **one device three times**:

```java
// File: code/lld/ch04/isp/split/OfficeWorkflow.java
package lld.ch04.isp.split;

/** OVER-SPLIT: needs all three, so it takes three dependencies -- and gains nothing. */
public class OfficeWorkflow {
    private final Printer printer;
    private final Scanner scanner;
    private final Stapler stapler;

    public OfficeWorkflow(Printer printer, Scanner scanner, Stapler stapler) {
        this.printer = printer;
        this.scanner = scanner;
        this.stapler = stapler;
    }

    public void run(String doc) {
        scanner.scanDocument(doc);
        printer.printDocument(doc);
        stapler.stapleDocument(doc);
    }
}
```

The code runs and technically follows ISP, yet nothing was gained: `OfficeWorkflow` calls all three methods, so the split took away no dependency. It only added two extra type names and two extra parameters.

Grouping by role. `OfficeWorkflow` wants one device that can scan, print and staple, so that is a single role, `DocumentProcessor`. `PrintQueue` only ever prints, which is a different role, `Printer`. The reason two interfaces are justified is that a `BasicPrinter` can play the second role but not the first. In Java, `DocumentProcessor extends Printer`, so any document processor can also be placed in a print queue.

```java
// File: code/lld/ch04/isp/roles/Printer.java
package lld.ch04.isp.roles;

/** Role interface for callers that only print (e.g. PrintQueue). */
public interface Printer {
    void printDocument(String doc);
}
```

```java
// File: code/lld/ch04/isp/roles/DocumentProcessor.java
package lld.ch04.isp.roles;

/** Role interface for callers that scan, print AND staple (e.g. OfficeWorkflow). */
public interface DocumentProcessor extends Printer {
    void scanDocument(String doc);

    void stapleDocument(String doc);
}
```

```java
// File: code/lld/ch04/isp/roles/BasicPrinter.java
package lld.ch04.isp.roles;

public class BasicPrinter implements Printer {
    @Override
    public void printDocument(String doc) {
        System.out.println("Basic printer printing " + doc);
    }
}
```

```java
// File: code/lld/ch04/isp/roles/OfficeDevice.java
package lld.ch04.isp.roles;

public class OfficeDevice implements DocumentProcessor {
    @Override
    public void printDocument(String doc) {
        System.out.println("Printing " + doc);
    }

    @Override
    public void scanDocument(String doc) {
        System.out.println("Scanning " + doc);
    }

    @Override
    public void stapleDocument(String doc) {
        System.out.println("Stapling " + doc);
    }
}
```

```java
// File: code/lld/ch04/isp/roles/PrintQueue.java
package lld.ch04.isp.roles;

/** Depends on printing only, so a basic printer is enough. */
public class PrintQueue {
    private final Printer printer;

    public PrintQueue(Printer printer) {
        this.printer = printer;
    }

    public void submit(String doc) {
        printer.printDocument(doc);
    }
}
```

```java
// File: code/lld/ch04/isp/roles/OfficeWorkflow.java
package lld.ch04.isp.roles;

/** One dependency, because one role covers everything it needs. */
public class OfficeWorkflow {
    private final DocumentProcessor processor;

    public OfficeWorkflow(DocumentProcessor processor) {
        this.processor = processor;
    }

    public void run(String doc) {
        processor.scanDocument(doc);
        processor.printDocument(doc);
        processor.stapleDocument(doc);
    }
}
```

```java
// File: code/lld/ch04/isp/Main.java
package lld.ch04.isp;

import java.util.List;
import lld.ch04.isp.roles.BasicPrinter;
import lld.ch04.isp.roles.OfficeDevice;
import lld.ch04.isp.roles.OfficeWorkflow;
import lld.ch04.isp.roles.PrintQueue;

public class Main {
    public static void main(String[] args) {
        System.out.println("=== 1. Fat interface ===");
        List<lld.ch04.isp.fat.Printer> printers = List.of(
                new lld.ch04.isp.fat.AdvancedPrinter(), new lld.ch04.isp.fat.BasicPrinter());
        for (var p : printers) {
            p.printDocument("report.pdf");
            try {
                p.scanDocument("report.pdf");
            } catch (UnsupportedOperationException e) {
                System.out.println("  forced to implement: " + e.getMessage());
            }
        }

        System.out.println("=== 2. One interface per method ===");
        new lld.ch04.isp.split.BasicPrinter().printDocument("memo.pdf");
        var adv = new lld.ch04.isp.split.AdvancedPrinter();
        adv.faxDocument("memo.pdf");
        // The same object passed three times -- a sign the split went too far.
        new lld.ch04.isp.split.OfficeWorkflow(adv, adv, adv).run("contract.pdf");

        System.out.println("=== 3. Role interfaces (grouped by caller) ===");
        new PrintQueue(new BasicPrinter()).submit("receipt.pdf");
        new PrintQueue(new OfficeDevice()).submit("receipt.pdf"); // bigger device fits the small role
        new OfficeWorkflow(new OfficeDevice()).run("contract.pdf");
    }
}
```

Output:

```
=== 1. Fat interface ===
AdvancedPrinter printing report.pdf
AdvancedPrinter scanning report.pdf
BasicPrinter printing report.pdf
  forced to implement: BasicPrinter cannot scan
=== 2. One interface per method ===
BasicPrinter printing memo.pdf
AdvancedPrinter faxing memo.pdf
AdvancedPrinter scanning contract.pdf
AdvancedPrinter printing contract.pdf
AdvancedPrinter stapling contract.pdf
=== 3. Role interfaces (grouped by caller) ===
Basic printer printing receipt.pdf
Printing receipt.pdf
Scanning contract.pdf
Printing contract.pdf
Stapling contract.pdf
```

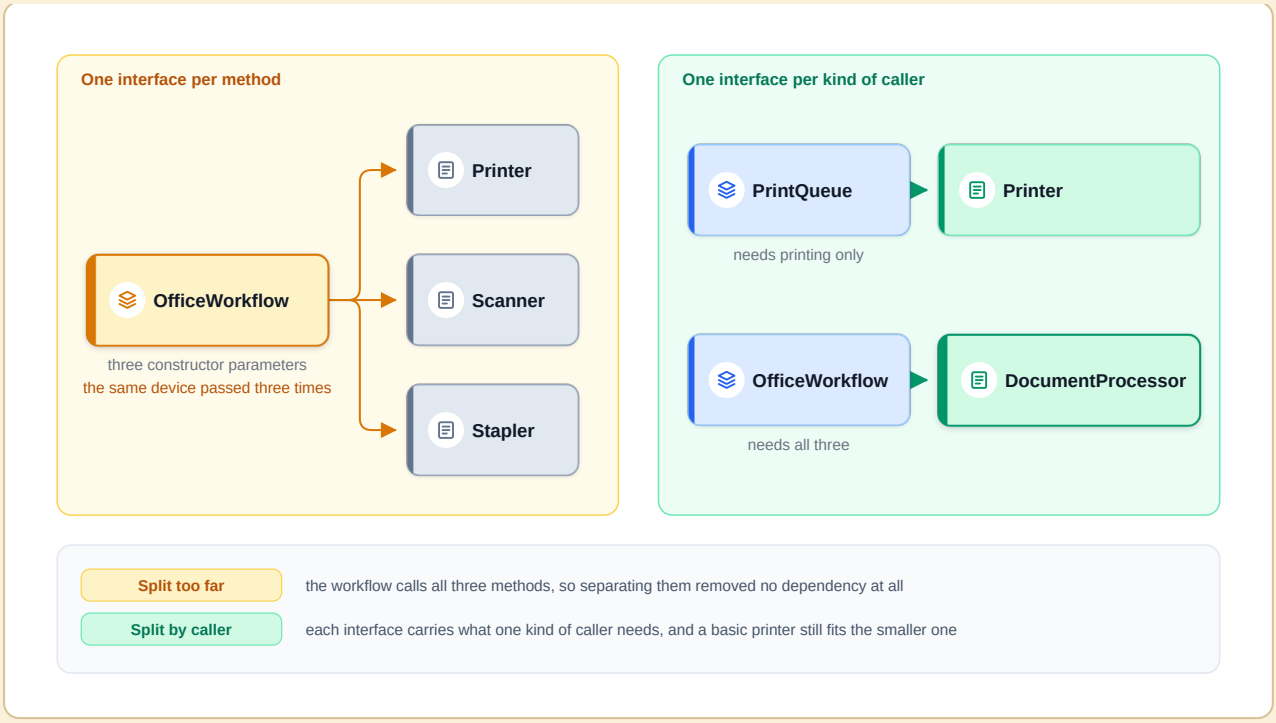
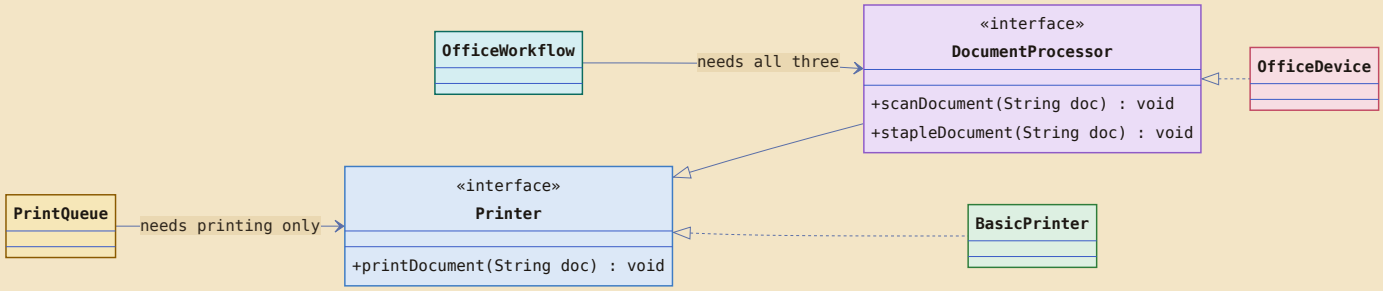


Figure — split too far (three interfaces, the same device passed three times) vs split by caller (one role per kind of caller) (course diagram)



	Four interfaces, one method each	Two role interfaces
Implementer forced to supply what it cannot do?	No	No
Caller depending on unused methods?	No	No
Dependencies for `OfficeWorkflow`	3	1
Names to learn	4	2

Both designs satisfy ISP. The role-based one is leaner because its interfaces were shaped around callers instead of around methods.

Rule of thumb: only split an interface when you can **point to the caller** that needs the smaller piece; without such a caller the split is speculation. The converse also holds: two interfaces that are always implemented by the same classes and always consumed by the same callers should probably be merged.

Languages without an `interface` keyword. Python and JavaScript have no `interface` keyword, but the interface is still there, just invisible: it is **the set of methods a caller actually calls**. Handing a 20-method object to a function that calls one of them creates exactly the same needless dependency, only no compiler points it out. Testing hides it further: in Java a wide interface forces a long stub (a fake class with every method filled in), while in Python you can pass a tiny object with one method and the test passes, which is convenient but masks the problem. So check deliberately: look at what a function calls on its argument and ask whether the argument is far bigger than that. Python's `abc` module and `typing.Protocol` (Python's two ways to declare an interface) let you write the role down when it is worth it; for a role used in only one place, a comment listing the methods the function needs is often enough.

Five signals of an ISP violation

1. Low cohesion in the interface. The methods belong to unrelated groups.

```
interface MediaDevice { void playAudio(); void playVideo(); void recordVideo(); }
// A Speaker can only playAudio(); playVideo() and recordVideo() must throw.
```

2. A fat interface. A long method list that grew one feature at a time. Method names that name a specific mechanism are a tell: some implementer will not use them.

```
interface PaymentProcessor { void processCreditCard(BigDecimal a); void processPayPal(BigDecimal a); void
processBankTransfer(BigDecimal a); }
// A shop that only takes PayPal must still supply the other two.
// Better: interface PaymentMethod { void pay(BigDecimal amount); } with one class per method.
```

3. Empty or throwing methods. The clearest signal: the interface asks for something the class cannot provide.

```
interface Vehicle { void drive(); void fly(); void sail(); }
// A Car drives; fly() and sail() throw UnsupportedOperationException.
```

4. One interface serving several kinds of client. It describes a whole category ("everything an employee can do") instead of one caller's needs, so every role implements the union of all roles.

```
interface Employee { void calculateSalary(); void handleCustomerQueries(); void manageTeam(); }
// A support agent handles queries; the other two methods must throw.
```

5. Tests that need long stand-ins. This one appears while writing tests rather than reading code. A fake for a wide interface must implement every method, even if the test exercises just one; dozens of empty method bodies for a single call mean the interface is too wide **for that caller**. What this signal really measures is the distance between what a caller uses and what it is forced to depend on.

Signals 1 and 2 show up when you read the interface, 3 and 4 when you read its implementers, and 5 when you write a test.

Signal	When it shows up	Typical cause
Low cohesion	Reading the interface	Groups of unrelated methods should be split into their own interfaces
Fat interface	Reading the interface	It grew feature by feature and was never split
Empty / throwing methods	Reading an implementer	The implementer was forced to accept what it cannot do
One interface, many roles	Comparing implementers	It describes a category, not a caller's needs
Long test stand-ins	Writing a test	Callers are tied to much more than they actually call

How it works (step by step)

1. For each caller of an interface, list the methods it actually calls.
2. Group callers whose lists are the same; each group is a **role**.
3. Create one interface per role (interfaces may extend each other, like `DocumentProcessor extends Printer`).
4. Let each class implement every role it can honestly fill.
5. Change each caller's parameter type to the narrowest role it needs.

Importance of ISP

- **No bloated classes** forced to implement unneeded methods.
- **Maintainability** — a change to one role does not ripple to callers of another.

- **Flexibility** — small roles are easy to implement in new ways.
- **Simpler tests** — fakes implement one or two methods.
- **Clearer design** — each interface documents one relationship.

How ISP relates to the other principles

- **SRP**: ISP is SRP for interfaces. Each interface has one focused purpose.
- **OCP**: new capabilities arrive as new small interfaces instead of edits to a big one.
- **LSP**: implementers of small interfaces can honour every method, so substitution is safe. A fat interface **forces** LSP violations (the throwing `BasicPrinter`).
- **DIP**: high-level code depends on the smallest abstraction it needs.

Trade-offs

Small role interfaces gain	Costs
No forced, throwing methods	More interface names
Callers insulated from unrelated changes	Over-splitting adds wiring with no benefit
Tiny test doubles; lambdas for one-method roles	Harder to see "everything this device can do" in one place

When not to use it

- When every implementer supports every method and every caller uses them all (the interface is already right-sized).
- When you cannot name the caller that wants the smaller piece.

Common mistakes

- **One interface per method everywhere.** The most common mistake among people who just learned ISP. A fat interface at least makes its cost visible; hundreds of tiny ones cost you slowly, in names and wiring, and are harder to undo.
- Splitting by the implementer's features instead of the caller's needs.
- Keeping the fat interface and adding an "adapter" base class with empty methods (like AWT's `MouseAdapter`; AWT is Java's original desktop user-interface toolkit): convenient, but a symptom that the interface is too wide.

Real-world examples

- **Java's tiny role interfaces**: `Runnable`, `Comparable`, `AutoCloseable`, `Iterable`, `CharSequence`, and the whole `java.util.function` package. `try-with-resources` needs only `AutoCloseable`, not "a file".
- **AWT** `MouseListener` has five methods; most listeners need one, so the JDK added `MouseAdapter` with empty bodies. Later APIs (JavaFX event handlers) use one-method handlers.
- **Collections**: `List.of(...)` and unmodifiable views throw on `add`, because `Collection` bundles reading and writing. Kotlin (another language that runs on the JVM), by contrast, separates `List` (read) and `MutableList` (write).
- **Spring Data** (a Spring library that generates repository code for you): `CrudRepository`, `PagingAndSortingRepository`, and the bare `Repository` marker let you expose only the operations a repository needs.

In the interview

> **30-second answer:** "ISP says no caller should depend on methods it does not use. The symptom is a class implementing an interface method by throwing, or a test fake full of empty methods. I split interfaces by caller: one role interface per kind of caller, not one per method. For printers, a print queue needs `Printer`, an office workflow needs `DocumentProcessor`. I split only when I can name the caller that wants the smaller piece."

Follow-ups:

- *"Difference between SRP and ISP?"* — SRP protects the **class** (one reason to change); ISP protects the **caller** (no unused dependencies). SRP splits classes, ISP splits interfaces; you can break either without the other.
- *"How small is too small?"* — When a caller has to take several interfaces that are always the same object. Always implemented together and always used together means one interface.

Self-check

1. **Q:** What two questions decide if an interface is the right size? **A:** Is any implementer forced to supply something it cannot do? Does any caller depend on methods it never calls?
2. **Q:** Name three signals of an ISP violation. **A:** Throwing/empty methods, a fat interface with mechanism-named methods, long test stand-ins (also: low cohesion, one interface for several roles).
3. **Q:** Why did the four-interface version not help `OfficeWorkflow`? **A:** It uses all three methods, so no dependency was removed; only names and parameters were added.

TL;DR

One interface per kind of caller. Throwing methods and long fakes mean it is too wide; the same object passed three times means it is too thin.

4.10 Dependency Inversion Principle (DIP)

What it is

Robert C. Martin's two statements, paraphrased:

1. **High-level modules should not depend on low-level modules. Both should depend on abstractions.**
2. **Abstractions should not depend on details. Details should depend on abstractions.**

Terms:

- A **high-level module** holds the business policy: *what* the application does (e.g. "notify the user when an order ships").
- A **low-level module** handles a technical detail: *how* it is done (SMTP email, SMTP being the protocol for sending mail; SMS gateway; MySQL).
- An **abstraction** is the interface in between.

Why "inversion"? In traditional layered code, high-level code calls low-level code **and** its source code depends on it (`import com.mail.SmtpClient`). DIP keeps the direction of the call (policy still calls the detail at run time) but **reverses the direction of the source-code dependency**: the detail now imports and implements an interface that belongs to the policy.

> **Analogy: a wall socket.** Your lamp does not wire itself directly into the power station. It plugs into a standard socket. The building's wiring (the detail) was built to the socket standard too. Either side can be replaced without touching the other.

The problem it solves

- Business logic is tied to one vendor or technology; switching (email → SMS) means editing the business code.
- Business logic cannot be tested without the real infrastructure (the servers and services the program runs on: a mail server, a database).
- A change in a low-level library ripples up into the policy.

Example: notifications

Before. `NotificationService` (high level) creates and names `EmailService` (low level):

```java
// File: code/lld/ch04/dip/before/EmailService.java
package lld.ch04.dip.before;

/** Low-level detail. */
public class EmailService {
    public void sendEmail(String message) {
        System.out.println("Sending email: " + message);
    }
}
```

```java
// File: code/lld/ch04/dip/before/NotificationService.java
package lld.ch04.dip.before;

/** VIOLATION: high-level policy creates and names a low-level detail. */
public class NotificationService {
    private final EmailService emailService = new EmailService();

    public void notifyUser(String message) {
        emailService.sendEmail(message);
    }
}
```

It is tightly coupled, and switching to SMS or push means editing `NotificationService`.

After, in four steps.

1. **Introduce an abstraction**, `NotificationSender`.
2. **Low-level modules implement it** (`EmailSender`, `SmsSender`): details depend on the abstraction.
3. **The high-level module depends only on the abstraction**, received through its constructor.
4. **Test it** with different senders, including a recording fake.

Look at the packages. The interface lives in `app`, **next to the policy that uses it**, not in `infra` next to the implementations. That is the part of DIP most people miss: **the high-level side owns the abstraction.** It is shaped by what the policy needs, not by what email happens to offer. `app` imports nothing from `infra`; `infra` imports `app`.

```java
// File: code/lld/ch04/dip/app/NotificationSender.java
package lld.ch04.dip.app;

/**
 * The abstraction. Note the PACKAGE: it lives with the high-level policy (app),
 * not with the implementations (infra). The policy owns the contract.
 */
public interface NotificationSender {
    void send(String recipient, String message);
}
```

```java
// File: code/lld/ch04/dip/app/NotificationService.java
package lld.ch04.dip.app;

import java.util.Objects;

/** High-level policy: depends only on an abstraction it owns. Imports nothing from infra. */
public class NotificationService {
    private final NotificationSender sender;

    public NotificationService(NotificationSender sender) { // constructor injection
        this.sender = Objects.requireNonNull(sender);
    }

    public void notifyUser(String userId, String message) {
        if (message.isBlank()) {
            throw new IllegalArgumentException("empty notification");
        }
        sender.send(userId, message);
    }
}
```

```java
// File: code/lld/ch04/dip/infra/EmailSender.java
package lld.ch04.dip.infra;

import lld.ch04.dip.app.NotificationSender;

/** Detail depends on the abstraction -- the arrow now points UP into app. */
public class EmailSender implements NotificationSender {
    @Override
    public void send(String recipient, String message) {
        System.out.println("[email -> " + recipient + "] " + message);
    }
}
```

```java
// File: code/lld/ch04/dip/infra/SmsSender.java
package lld.ch04.dip.infra;

import lld.ch04.dip.app.NotificationSender;

public class SmsSender implements NotificationSender {
    @Override
    public void send(String recipient, String message) {
        System.out.println("[sms -> " + recipient + "] " + message);
    }
}
```

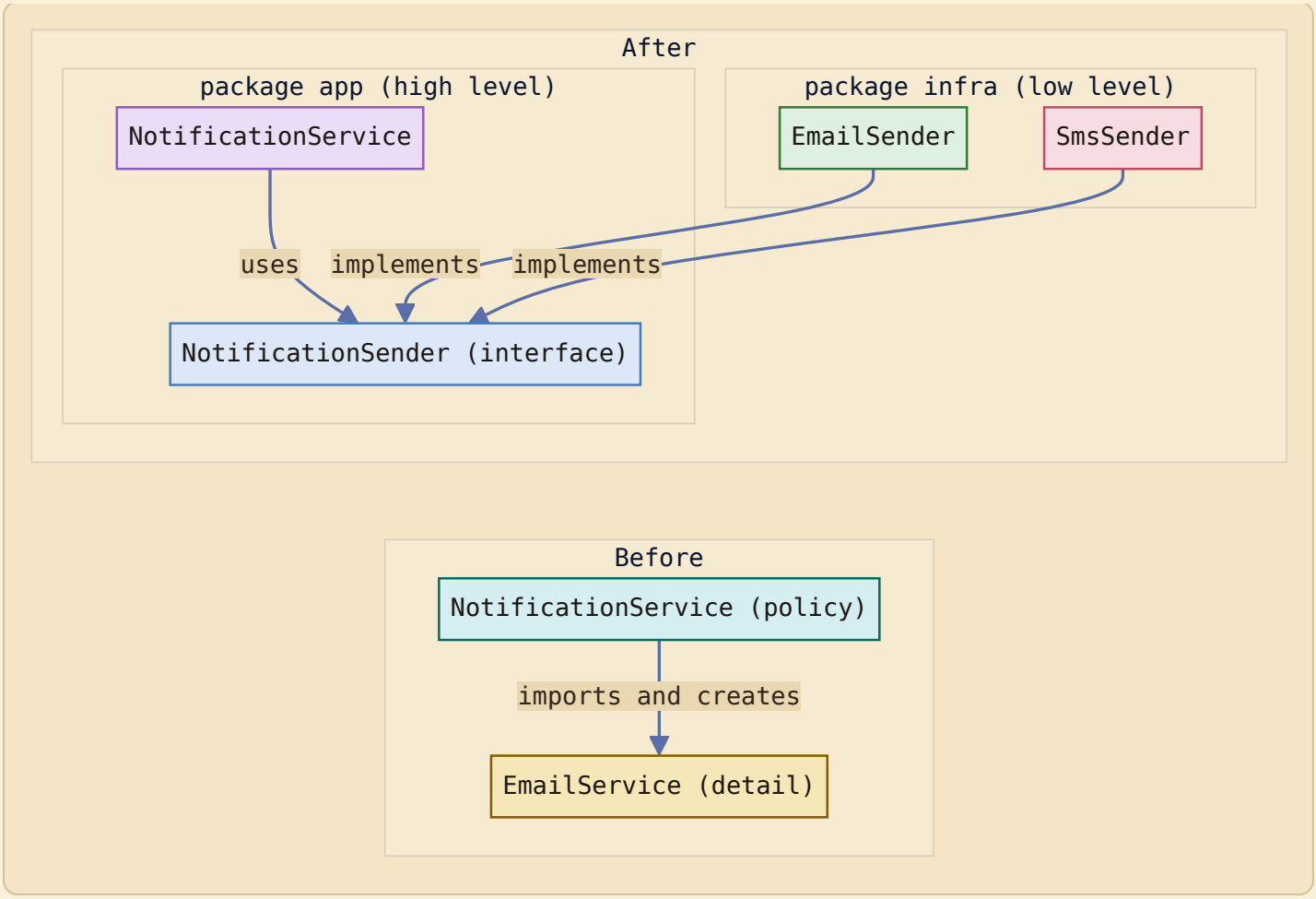
```java
// File: code/lld/ch04/dip/infra/RecordingSender.java
package lld.ch04.dip.infra;

import java.util.ArrayList;
import java.util.List;
import lld.ch04.dip.app.NotificationSender;

/** A test double: proves NotificationService can be tested with no mail server. */
public class RecordingSender implements NotificationSender {
    private final List<String> sent = new ArrayList<>();

    @Override
    public void send(String recipient, String message) {
        sent.add(recipient + ":" + message);
    }

    public List<String> sent() {
        return List.copyOf(sent);
    }
}
```



In the "After" box, every arrow between packages points **from the detail into the policy**. That is the inversion.

DIP in practice: four real-world techniques

1. Dependency Injection (DI). Give a class its dependencies from outside instead of letting it create them. Three flavours:

Kind	How	Use when
Constructor injection	Pass dependencies to the constructor	Default choice: required dependencies, immutable fields, obvious in the signature
Setter injection	Call `setX(...)` after construction	Optional dependencies with a sensible default
Method (parameter) injection	Pass the dependency to the one method that needs it	It varies per call (e.g. if the format changed on every call, `invoice.generate(generator)` instead of the constructor injection used in 4.7)

A typical web-application case: a `DataService` needs database access, so it receives a `DataRepository` interface in its constructor instead of building a MySQL repository itself; moving to another database then means passing in a different implementation, with no edit to `DataService`.

The class that does all the `new`-ing is called the **composition root**. In this chapter it is each `Main`: the one place that knows the concrete classes.

2. Service Locator. A central registry hands out services by name or type. The high-level class depends on a `Locator` abstraction and asks it for what it needs at run time.

```java
// File: code/lld/ch04/dip/locator/PaymentService.java
package lld.ch04.dip.locator;

import java.math.BigDecimal;

public interface PaymentService {
    void processPayment(BigDecimal amount);
}
```

```java
// File: code/lld/ch04/dip/locator/PayPalService.java
package lld.ch04.dip.locator;

import java.math.BigDecimal;

public class PayPalService implements PaymentService {
    @Override
    public void processPayment(BigDecimal amount) {
        System.out.println("Processing payment via PayPal: $" + amount);
    }
}
```

```java
// File: code/lld/ch04/dip/locator/Locator.java
package lld.ch04.dip.locator;

/** The high-level code depends on this abstraction, not on a concrete registry. */
public interface Locator {
    PaymentService paymentService(String name);
}
```

```java
// File: code/lld/ch04/dip/locator/ServiceLocator.java
package lld.ch04.dip.locator;

import java.util.Map;
import java.util.NoSuchElementException;
import java.util.concurrent.ConcurrentHashMap;

/** A central registry that hands out services by name. */
public class ServiceLocator implements Locator {
    private final Map<String, PaymentService> services = new ConcurrentHashMap<>();

    public void register(String name, PaymentService service) {
        services.put(name, service);
    }

    @Override
    public PaymentService paymentService(String name) {
        PaymentService s = services.get(name);
        if (s == null) {
            throw new NoSuchElementException("No payment service named " + name);
        }
        return s;
    }
}
```

```java
// File: code/lld/ch04/dip/locator/CheckoutFlow.java
package lld.ch04.dip.locator;

import java.math.BigDecimal;

/** Asks the locator at run time. Its real dependency (PaymentService) is hidden inside. */
public class CheckoutFlow {
    private final Locator locator;

    public CheckoutFlow(Locator locator) {
        this.locator = locator;
    }

    public void pay(String serviceName, BigDecimal amount) {
        locator.paymentService(serviceName).processPayment(amount);
    }
}
```

This does follow DIP: `CheckoutFlow` names no concrete payment class, and it receives the `Locator` *interface* through its constructor, so even the registry itself can be replaced (for example by a test locator) without touching `CheckoutFlow`. But be aware of the criticism (Martin Fowler compares the two approaches; Mark Seemann calls Service Locator an anti-pattern, a common solution that causes more problems than it solves): the class's **real** dependencies are hidden inside method bodies, so you cannot see from the constructor what it needs, and a missing registration fails only at run time. Prefer constructor injection; use a locator mainly at framework boundaries.

3. Inversion of Control (IoC) containers. Frameworks such as **Spring** (Java) and the built-in container in **.NET** (Microsoft's application platform) create objects and inject dependencies for you. In Spring, a class with a single constructor gets its dependencies injected automatically (the `@Autowired` annotation, an `@` tag that marks where Spring should inject, is optional for a single constructor since Spring 4.3; a **bean** is an object Spring creates and manages, and the **classpath** is the list of places Java searches for compiled classes and libraries):

```java
// Spring version -- NOT part of this chapter's compiled code (needs Spring on the classpath).
@Service
public class OrderService {
    private final PaymentService payments;          // an interface

    public OrderService(PaymentService payments) {  // Spring finds the one PaymentService bean
        this.payments = payments;
    }

    public void placeOrder(BigDecimal amount) { payments.processPayment(amount); }
}
```

Without Spring, the same wiring is one line in `main`: `new OrderService(new StripePayments())`. **The framework is a convenience for wiring, not the principle.** `OrderService` was already inverted the moment it accepted an abstraction instead of creating a concrete class.

4. Plug-in architecture. The core defines an abstraction; plug-ins implement it and are installed without changing the core. A media player defines `AudioDecoder`; MP3 and FLAC support arrive as plug-ins.

```java
// File: code/lld/ch04/dip/plugin/AudioDecoder.java
package lld.ch04.dip.plugin;

/** The plug-in contract, owned by the core media player. */
public interface AudioDecoder {
    String extension();

    void decode(String fileName);
}
```

```java
// File: code/lld/ch04/dip/plugin/Mp3Decoder.java
package lld.ch04.dip.plugin;

public class Mp3Decoder implements AudioDecoder {
    @Override
    public String extension() {
        return "mp3";
    }

    @Override
    public void decode(String fileName) {
        System.out.println("Decoding MP3 file: " + fileName);
    }
}
```

```java
// File: code/lld/ch04/dip/plugin/FlacDecoder.java
package lld.ch04.dip.plugin;

public class FlacDecoder implements AudioDecoder {
    @Override
    public String extension() {
        return "flac";
    }

    @Override
    public void decode(String fileName) {
        System.out.println("Decoding FLAC file: " + fileName);
    }
}
```

```java
// File: code/lld/ch04/dip/plugin/MediaPlayer.java
package lld.ch04.dip.plugin;

import java.util.HashMap;
import java.util.Map;

/** The core. New formats arrive as plug-ins; this class is never edited for them. */
public class MediaPlayer {
    private final Map<String, AudioDecoder> decoders = new HashMap<>();

    public void install(AudioDecoder decoder) {
        decoders.put(decoder.extension(), decoder);
    }

    public void play(String fileName) {
        String ext = fileName.substring(fileName.lastIndexOf('.') + 1);
        AudioDecoder decoder = decoders.get(ext);
        if (decoder == null) {
            System.out.println("No plug-in installed for ." + ext);
            return;
        }
        decoder.decode(fileName);
        System.out.println("Playing audio...");
    }
}
```

```java
// File: code/lld/ch04/dip/Main.java
package lld.ch04.dip;

import java.math.BigDecimal;
import lld.ch04.dip.app.NotificationService;
import lld.ch04.dip.infra.EmailSender;
import lld.ch04.dip.infra.RecordingSender;
import lld.ch04.dip.infra.SmsSender;
import lld.ch04.dip.locator.CheckoutFlow;
import lld.ch04.dip.locator.PayPalService;
import lld.ch04.dip.locator.ServiceLocator;
import lld.ch04.dip.plugin.FlacDecoder;
import lld.ch04.dip.plugin.MediaPlayer;
import lld.ch04.dip.plugin.Mp3Decoder;

/** This class is the COMPOSITION ROOT: the one place that knows the concrete classes. */
public class Main {
    public static void main(String[] args) {
        System.out.println("=== 1. Before: NotificationService builds EmailService itself ===");
        new lld.ch04.dip.before.NotificationService().notifyUser("Your order shipped");

        System.out.println("=== 2. After: inject any NotificationSender ===");
        new NotificationService(new EmailSender()).notifyUser("u-42", "Your order shipped");
        new NotificationService(new SmsSender()).notifyUser("u-42", "Your order shipped");
        var recorder = new RecordingSender();
        new NotificationService(recorder).notifyUser("u-7", "Test message");
        System.out.println("test double captured: " + recorder.sent());

        System.out.println("=== 3. Service locator ===");
        var locator = new ServiceLocator();
        locator.register("PayPal", new PayPalService());
        new CheckoutFlow(locator).pay("PayPal", new BigDecimal("100.00"));

        System.out.println("=== 4. Plug-in architecture ===");
        var player = new MediaPlayer();
        player.install(new Mp3Decoder());
        player.install(new FlacDecoder());
        player.play("song.mp3");
        player.play("concert.flac");
        player.play("voice.ogg");
    }
}
```

Output:

```
=== 1. Before: NotificationService builds EmailService itself ===
Sending email: Your order shipped
=== 2. After: inject any NotificationSender ===
[email -> u-42] Your order shipped
[sms -> u-42] Your order shipped
test double captured: [u-7:Test message]
=== 3. Service locator ===
Processing payment via PayPal: $100.00
=== 4. Plug-in architecture ===
Decoding MP3 file: song.mp3
Playing audio...
Decoding FLAC file: concert.flac
Playing audio...
No plug-in installed for .ogg
```

DI vs plug-in architecture. At first sight they are the same thing (an implementation of an interface is handed to a core), but they work at different scales. **DI** is a small-scale mechanism: how one class obtains its collaborators. **Plug-in architecture** is a system-level design for extensibility: whole modules, often built by other teams, are added independently and frequently discovered while the program runs (Java's `ServiceLoader`, OSGi, a Java module system built for plug-ins, IDE plug-in APIs).

DIP vs DI vs IoC — do not mix them up

Term	What it is	Kind	Example
Dependency Inversion (DIP)	Depend on abstractions; the policy owns the abstraction	A **principle** (what good structure looks like)	`NotificationService` depends on `NotificationSender`
Dependency Injection (DI)	Pass dependencies in from outside instead of creating them	A **technique** (one way to achieve DIP)	`new NotificationService(new SmsSender())`
Inversion of Control (IoC)	The framework calls your code, not the other way round ("don't call us, we'll call you" — the Hollywood Principle)	A broad **idea**; DI is one form of it	Spring creating beans; a servlet container (a Java web server such as Tomcat) calling `doGet`; JUnit calling your `@Test` methods
IoC container	A framework that performs DI for you	A **tool**	Spring `ApplicationContext`, Google Guice (a lightweight DI library)

Two combinations prove they are different:

- **Injection without inversion**: `new OrderService(new MySqlDatabase())` where the parameter type is the concrete `MySqlDatabase`. It is injected, but the policy still depends on a detail.
- **Inversion without a framework**: everything in this chapter. Constructor + interface + a `main` that wires it. No container needed.

Stable vs volatile dependencies

DIP targets **volatile** dependencies: things that change often or that you want to fake in tests (databases, networks, clocks, third-party APIs, i.e. services run by other companies that your code calls, business rules that vary). Depending directly on **stable** concrete classes is fine: `String`, `List`, `LocalDate`, `Math`. Nobody wraps `String` in an interface. (Head First's version of DIP gives guidelines like "no variable should hold a reference to a concrete class", and then admits they are ideals you will break; the stable/volatile distinction is how you decide when.)

Importance of DIP

- **Flexibility** — swap implementations without touching business logic.
- **Maintainability** — detail changes stay in the detail.
- **Testability** — pass fakes and mocks (fakes that also record how they were called, so a test can check it); no real database or mail server in unit tests.

How DIP relates to the other principles

- **SRP**: injected collaborators keep each class focused on its own job; creating and configuring dependencies becomes the composition root's job.
- **OCP**: high-level code depends on abstractions, so new implementations extend it without edits.
- **LSP**: all implementations of the abstraction must be proper substitutes, or injection becomes dangerous.
- **ISP**: the abstraction the high-level module owns should be the **smallest** role it needs.

How it works (step by step)

1. Find a high-level class that names or creates a volatile low-level class.
2. Write down what the high-level class **needs** from it, in its own words (`send(recipient, message)`).
3. Make that an interface **in the high-level package**.
4. Make the low-level class implement it.
5. Inject the implementation (constructor by default).
6. Move all `new` calls for concrete classes to the composition root (`main`, a factory, or a DI container).

Trade-offs

You gain	You pay
Policy independent of technology	An interface per volatile dependency
Easy unit tests with fakes	Wiring code (or a framework and its learning curve)
Swappable implementations	Indirection: "go to implementation" takes an extra hop

When not to use it

- For stable classes (`String`, collections, value objects).
- For an interface that would have exactly one implementation forever and no test double.
- In tiny scripts.

Common mistakes

- Putting the interface in the **low-level** package (e.g. `mysql.MySqlRepository` and `mysql.Repository`). The policy then still depends on the detail's package.
- Designing the interface around one vendor's API (`sendSmtpMessage(...)`) instead of the policy's need (`send(...)`). Changing vendors then changes the interface too.
- Breaking the **second** clause ("abstractions should not depend on details"): an interface whose methods return `java.sql.ResultSet` or throw `javax.mail.MessagingException` is technically an abstraction, but it drags the detail's types into the policy. Use the policy's own types (`Optional<Order>`, a box that may or may not hold an order; a domain exception, an error type named in business terms) instead.
- Believing "we use Spring, so we follow DIP". Spring can inject concrete classes just as happily.
- Calling `new ConcreteThing()` inside a class "just this once".

Real-world examples

- **JDBC**: your code depends on `java.sql.Connection` / `DataSource` interfaces; drivers from MySQL, PostgreSQL, Oracle implement them. `DriverManager` finds drivers through `ServiceLoader`.
- **SLF4J**: the application depends on the logging facade; Logback or Log4j 2 is the plugged-in detail.
- **Spring** and **Jakarta EE CDI** (the enterprise-Java standard's built-in injection): IoC containers doing constructor injection.
- **JPA** (Java's standard API for saving objects to SQL databases): code uses `EntityManager`; Hibernate or EclipseLink implements it.
- **Hexagonal architecture (ports and adapters)**: the core defines *ports* (interfaces); *adapters* for HTTP, databases and queues (message queues, see System Design Fundamentals) implement them. This is DIP applied to a whole service.

In the interview

> **30-second answer:** "DIP says business logic should not depend on technical details; both depend on an abstraction, and the business side owns that abstraction. So `NotificationService` depends on a `NotificationSender` interface in its own package, and email and SMS senders implement it. I inject the implementation through the constructor and wire everything in `main`. That makes the service testable with a fake and lets me swap providers. DI is the technique; DIP is the principle; Spring is just a tool that does the wiring."

Follow-ups:

- *"Difference between dependency inversion and dependency injection?"* — Inversion is the principle (depend on abstractions); injection is one technique (pass dependencies in). You can inject a concrete class, which is injection without inversion.
- *"Who should own the interface?"* — The high-level module that uses it.

- *"Is Service Locator OK?"* — It satisfies DIP but hides dependencies and fails late. I prefer constructor injection.

Self-check

1. **Q:** What is "inverted" in DIP? **A:** The source-code dependency: the detail depends on (implements) an abstraction owned by the policy, instead of the policy importing the detail.
2. **Q:** Is `new OrderService(new MySqlDatabase())` with a `MySqlDatabase` parameter DIP? **A:** No. It is injection of a concrete class; the policy still depends on the detail.
3. **Q:** What is a composition root? **A:** The one place (usually `main` or the container config) where concrete classes are created and wired.
4. **Q:** Should you put `String` behind an interface? **A:** No. It is stable; DIP is for volatile dependencies.

TL;DR

Business code depends on interfaces it owns; details implement them; `main` (or a container) wires them. DI is the how, DIP is the why, IoC containers are the tool.

4.11 Refactoring one class through all five principles

In real code the principles seldom show up one at a time; they tend to appear **in a fixed sequence**, each exposing the need for the next. This section refactors a single order-checkout class five times. The class checks the order, prices it, saves it and emails the customer, the sort of class that has been growing unnoticed for a couple of years.

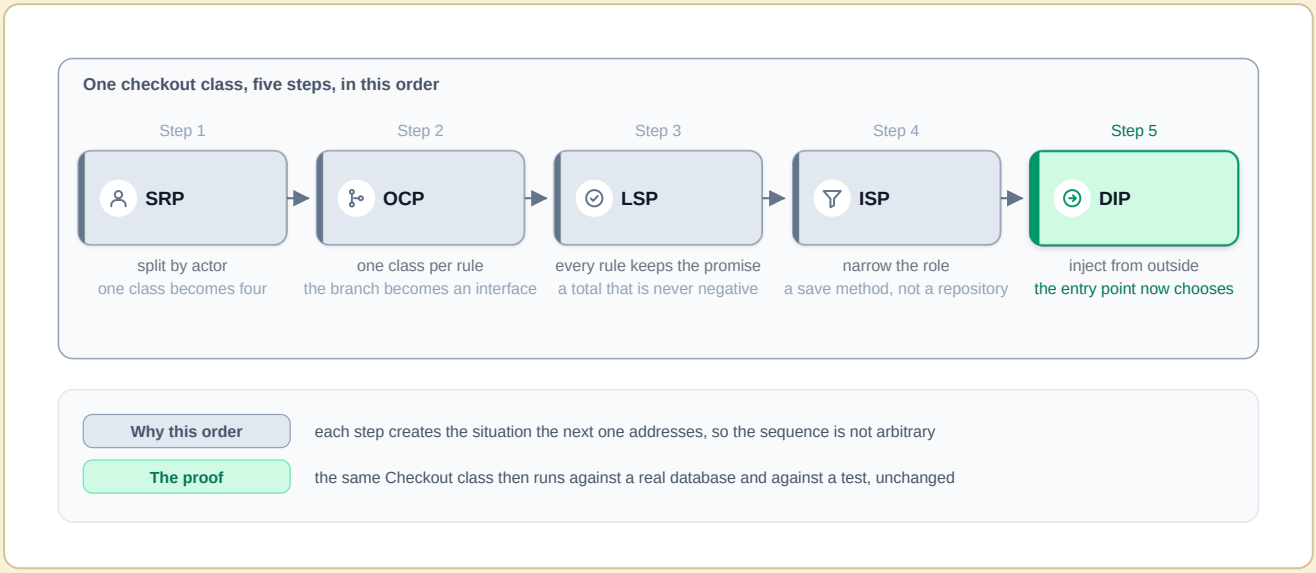


Figure — one checkout class, five steps: SRP, OCP, LSP, ISP, DIP, each creating the situation the next addresses (course diagram)

Step 0: the starting class

The code runs correctly, and yet each of the five principles finds something to object to.

```java
// File: code/lld/ch04/checkout/step0/CustomerOrder.java
package lld.ch04.checkout.step0;

import java.math.BigDecimal;

/** The original order shape: the customer type travels with the order as a string. */
public record CustomerOrder(String orderId, String customerType, BigDecimal amount, String email) { }
```

```java
// File: code/lld/ch04/checkout/step0/OrderService.java
package lld.ch04.checkout.step0;

import java.math.BigDecimal;
import java.math.RoundingMode;

/** STEP 0 -- the class that grew quietly for two years. It works. */
public class OrderService {
    public void process(CustomerOrder order) {
        // 1. Validation  (business-rules owner)
        if (order.amount().signum() <= 0) {
            System.out.println("Rejected: amount must be positive");
            return;
        }
        // 2. Pricing, one branch per customer type  (finance)
        BigDecimal total = order.amount();
        if (order.customerType().equals("member")) {
            total = total.multiply(new BigDecimal("0.90"));
        } else if (order.customerType().equals("staff")) {
            total = total.multiply(new BigDecimal("0.70"));
        }
        total = total.setScale(2, RoundingMode.HALF_EVEN); // money: always 2 decimals
        // 3. Storage  (database team)
        System.out.println("INSERT INTO orders VALUES ('" + order.orderId() + "', " + total + ")");
        // 4. Telling the customer  (marketing)
        System.out.println("Emailing " + order.email() + ": your order total is " + total);
    }
}
```

Step 1: Single Responsibility — split by actor

List everyone who might request a change to `process`:

Section of `process`	Requested by
Validation	The business-rules owner
Pricing	Finance
The `INSERT` statement	The database team
The email wording	Marketing

One method answers to four different groups, so rewording the email forces a redeploy of the pricing logic. Split it by actor into `OrderValidator`, `PriceCalculator`, `OrderRepository` and `Notifier`:

```java
// File: code/lld/ch04/checkout/step1/OrderValidator.java
package lld.ch04.checkout.step1;

import lld.ch04.checkout.step0.CustomerOrder;

/** Actor: business-rules owner. */
public class OrderValidator {
    public boolean isValid(CustomerOrder order) {
        return order.amount().signum() > 0;
    }
}
```

```java
// File: code/lld/ch04/checkout/step1/PriceCalculator.java
package lld.ch04.checkout.step1;

import java.math.BigDecimal;
import java.math.RoundingMode;
import lld.ch04.checkout.step0.CustomerOrder;

/** Actor: finance. SRP is satisfied, but the branch is still here -> OCP problem next. */
public class PriceCalculator {
    public BigDecimal total(CustomerOrder order) {
        BigDecimal total = switch (order.customerType()) {
            case "member" -> order.amount().multiply(new BigDecimal("0.90"));
            case "staff" -> order.amount().multiply(new BigDecimal("0.70"));
            default -> order.amount();
        };
        return total.setScale(2, RoundingMode.HALF_EVEN);
    }
}
```

```java
// File: code/lld/ch04/checkout/step1/OrderRepository.java
package lld.ch04.checkout.step1;

import java.math.BigDecimal;
import lld.ch04.checkout.step0.CustomerOrder;

/** Actor: database team. */
public class OrderRepository {
    public void save(CustomerOrder order, BigDecimal total) {
        System.out.println("INSERT INTO orders VALUES ('" + order.orderId() + "', " + total + ")");
    }
}
```

```java
// File: code/lld/ch04/checkout/step1/Notifier.java
package lld.ch04.checkout.step1;

import java.math.BigDecimal;
import lld.ch04.checkout.step0.CustomerOrder;

/** Actor: marketing (owns the wording). */
public class Notifier {
    public void send(CustomerOrder order, BigDecimal total) {
        System.out.println("Emailing " + order.email() + ": your order total is " + total);
    }
}
```

```java
// File: code/lld/ch04/checkout/step1/Checkout.java
package lld.ch04.checkout.step1;

import java.math.BigDecimal;
import lld.ch04.checkout.step0.CustomerOrder;

/** STEP 1 (SRP): coordinates four single-actor classes. Still news them up itself. */
public class Checkout {
    private final OrderValidator validator = new OrderValidator();
    private final PriceCalculator calculator = new PriceCalculator();
    private final OrderRepository repository = new OrderRepository();
    private final Notifier notifier = new Notifier();

    public void process(CustomerOrder order) {
        if (!validator.isValid(order)) {
            System.out.println("Rejected: amount must be positive");
            return;
        }
        BigDecimal total = calculator.total(order);
        repository.save(order, total);
        notifier.send(order, total);
    }
}
```

Step 2: Open/Closed — one class per pricing rule

Inside `PriceCalculator` the branch on customer type survived the split, so a new "student" rate would still mean editing working code. Give each rate its own class behind a `PricingRule` interface. From here on the order stops carrying a type string; whoever calls the checkout picks the rule:

```java
// File: code/lld/ch04/checkout/Order.java
package lld.ch04.checkout;

import java.math.BigDecimal;

/** From step 2 on, the order no longer carries a customer-type string: the rule is chosen outside. */
public record Order(String orderId, BigDecimal amount, String email) { }
```

```java
// File: code/lld/ch04/checkout/pricing/PricingRule.java
package lld.ch04.checkout.pricing;

import java.math.BigDecimal;

/**
 * STEP 2 (OCP): one class per rate; a new rate is a new class.
 * CONTRACT every implementation must keep (STEP 3, LSP):
 *   precondition : amount >= 0
 *   postcondition: result >= 0, rounded to 2 decimals   (a total is never negative)
 */
@FunctionalInterface
public interface PricingRule {
    BigDecimal apply(BigDecimal amount);
}
```

```java
// File: code/lld/ch04/checkout/pricing/RegularPrice.java
package lld.ch04.checkout.pricing;

import java.math.BigDecimal;
import java.math.RoundingMode;

public class RegularPrice implements PricingRule {
    @Override
    public BigDecimal apply(BigDecimal amount) {
        return amount.setScale(2, RoundingMode.HALF_EVEN);
    }
}
```

```java
// File: code/lld/ch04/checkout/pricing/MemberDiscount.java
package lld.ch04.checkout.pricing;

import java.math.BigDecimal;
import java.math.RoundingMode;

public class MemberDiscount implements PricingRule {
    private static final BigDecimal RATE = new BigDecimal("0.90");

    @Override
    public BigDecimal apply(BigDecimal amount) {
        return amount.multiply(RATE).setScale(2, RoundingMode.HALF_EVEN);
    }
}
```

```java
// File: code/lld/ch04/checkout/pricing/StaffDiscount.java
package lld.ch04.checkout.pricing;

import java.math.BigDecimal;
import java.math.RoundingMode;

public class StaffDiscount implements PricingRule {
    private static final BigDecimal RATE = new BigDecimal("0.70");

    @Override
    public BigDecimal apply(BigDecimal amount) {
        return amount.multiply(RATE).setScale(2, RoundingMode.HALF_EVEN);
    }
}
```

```java
// File: code/lld/ch04/checkout/step2/PriceCalculator.java
package lld.ch04.checkout.step2;

import java.math.BigDecimal;
import lld.ch04.checkout.Order;
import lld.ch04.checkout.pricing.PricingRule;

/** STEP 2: no branch left. The calculator never learns how many rates exist. */
public class PriceCalculator {
    public BigDecimal total(Order order, PricingRule rule) {
        return rule.apply(order.amount());
    }
}
```

A student rate is now one new class and no edits.

Step 3: Liskov Substitution — every rule keeps the promise

Once one interface has several implementations, you have to ask whether each of them can really stand in for the others. Finance now wants a **fixed** discount of 20 rather than a percentage. A naive version turns a 10 order into a total of **−10**, with no exception at all: the negative total is saved, and later the customer may be credited money they never paid. This is the postcondition/invariant rule from 4.8: every pricing rule must guarantee **a total that is never negative**, and a rule that does not is not a valid substitute. The fix is to state the promise explicitly (it is in the `PricingRule` Javadoc), make every rule honour it, and verify that with one contract test applied to every implementation:

```java
// File: code/lld/ch04/checkout/pricing/CarelessFixedDiscount.java
package lld.ch04.checkout.pricing;

import java.math.BigDecimal;
import java.math.RoundingMode;

/** LSP VIOLATION kept for the demo: 20 off a 10 order gives -10. Nothing throws. */
public class CarelessFixedDiscount implements PricingRule {
    private final BigDecimal off;

    public CarelessFixedDiscount(BigDecimal off) {
        this.off = off;
    }

    @Override
    public BigDecimal apply(BigDecimal amount) {
        return amount.subtract(off).setScale(2, RoundingMode.HALF_EVEN);
    }
}
```

```java
// File: code/lld/ch04/checkout/pricing/FixedDiscount.java
package lld.ch04.checkout.pricing;

import java.math.BigDecimal;
import java.math.RoundingMode;

/** STEP 3 fix: keeps the promise that a total is never negative. */
public class FixedDiscount implements PricingRule {
    private final BigDecimal off;

    public FixedDiscount(BigDecimal off) {
        if (off.signum() < 0) {
            throw new IllegalArgumentException("discount must be >= 0");
        }
        this.off = off;
    }

    @Override
    public BigDecimal apply(BigDecimal amount) {
        return amount.subtract(off).max(BigDecimal.ZERO).setScale(2, RoundingMode.HALF_EVEN);
    }
}
```

```java
// File: code/lld/ch04/checkout/step3/PricingRuleContract.java
package lld.ch04.checkout.step3;

import java.math.BigDecimal;
import java.util.List;
import lld.ch04.checkout.pricing.PricingRule;

/** STEP 3: ONE test written against the interface, run for EVERY implementation. */
public final class PricingRuleContract {
    private static final List<BigDecimal> SAMPLE_AMOUNTS = List.of(
            new BigDecimal("0.00"), new BigDecimal("10.00"),
            new BigDecimal("100.00"), new BigDecimal("250.00"));

    private PricingRuleContract() { }

    public static boolean holds(PricingRule rule) {
        for (BigDecimal amount : SAMPLE_AMOUNTS) {
            BigDecimal total = rule.apply(amount);
            if (total.signum() < 0) {
                System.out.println("  " + rule.getClass().getSimpleName()
                        + " broke the contract: apply(" + amount + ") = " + total);
                return false;
            }
        }
        System.out.println("  " + rule.getClass().getSimpleName() + " keeps the contract");
        return true;
    }
}
```

Step 4: Interface Segregation — the checkout needs a save method, not a repository

Repositories tend to accumulate methods (`save`, `findById`, `delete`, `listAll`) as other parts of the system ask for them. The checkout, however, only ever **saves**. If it is typed against the full repository, each new query added for reporting becomes one more method it depends on without calling. So describe the **role** the checkout needs instead of the class it is handed: an order saver, exactly one method wide. The real repository is free to play that role and others:

```java
// File: code/lld/ch04/checkout/fin/OrderSaver.java
package lld.ch04.checkout.fin;

import java.math.BigDecimal;
import lld.ch04.checkout.Order;

/** STEP 4 (ISP): the role the checkout needs -- one method wide, owned by the checkout. */
@FunctionalInterface
public interface OrderSaver {
    void save(Order order, BigDecimal total);
}
```

```java
// File: code/lld/ch04/checkout/fin/OrderFinder.java
package lld.ch04.checkout.fin;

import java.util.List;
import java.util.Optional;
import lld.ch04.checkout.Order;

/** A different role, for reporting callers. The checkout never sees it. */
public interface OrderFinder {
    Optional<Order> findById(String orderId);

    List<Order> listAll();
}
```

```java
// File: code/lld/ch04/checkout/fin/OrderNotifier.java
package lld.ch04.checkout.fin;

import java.math.BigDecimal;
import lld.ch04.checkout.Order;

@FunctionalInterface
public interface OrderNotifier {
    void send(Order order, BigDecimal total);
}
```

```java
// File: code/lld/ch04/checkout/fin/SqlOrderRepository.java
package lld.ch04.checkout.fin;

import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;
import lld.ch04.checkout.Order;

/** The full repository grows (save, find, list...) but plays two separate roles. */
public class SqlOrderRepository implements OrderSaver, OrderFinder {
    private final List<Order> table = new ArrayList<>(); // stands in for a real DB table

    @Override
    public void save(Order order, BigDecimal total) {
        table.add(order);
        System.out.println("  INSERT INTO orders VALUES ('" + order.orderId() + "', " + total + ")");
    }

    @Override
    public Optional<Order> findById(String orderId) {
        return table.stream().filter(o -> o.orderId().equals(orderId)).findFirst();
    }

    @Override
    public List<Order> listAll() {
        return List.copyOf(table);
    }
}
```

Step 5: Dependency Inversion — the entry point decides

A last problem remains: the step-1 `Checkout` constructs its collaborators itself, so it refers to concrete classes (a SQL repository, an email notifier) and cannot run in a test without a database and a mail server. Instead, have the abstractions passed in. The checkout then mentions no concrete class, and the program's entry point chooses the implementations:

```java
// File: code/lld/ch04/checkout/fin/InMemoryOrders.java
package lld.ch04.checkout.fin;

import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.List;
import lld.ch04.checkout.Order;

/** Test double: one method to implement, because the role is one method wide. */
public class InMemoryOrders implements OrderSaver {
    private final List<String> saved = new ArrayList<>();

    @Override
    public void save(Order order, BigDecimal total) {
        saved.add(order.orderId() + "=" + total);
        System.out.println("  kept " + order.orderId() + " in memory");
    }

    public List<String> saved() {
        return List.copyOf(saved);
    }
}
```

```java
// File: code/lld/ch04/checkout/fin/EmailNotifier.java
package lld.ch04.checkout.fin;

import java.math.BigDecimal;
import lld.ch04.checkout.Order;

public class EmailNotifier implements OrderNotifier {
    @Override
    public void send(Order order, BigDecimal total) {
        System.out.println("  emailing " + order.email() + ": total " + total);
    }
}
```

```java
// File: code/lld/ch04/checkout/fin/SilentNotifier.java
package lld.ch04.checkout.fin;

import java.math.BigDecimal;
import lld.ch04.checkout.Order;

public class SilentNotifier implements OrderNotifier {
    @Override
    public void send(Order order, BigDecimal total) {
        System.out.println("  (test run, no email sent)");
    }
}
```

```java
// File: code/lld/ch04/checkout/fin/Checkout.java
package lld.ch04.checkout.fin;

import java.math.BigDecimal;
import java.util.Objects;
import lld.ch04.checkout.Order;
import lld.ch04.checkout.pricing.PricingRule;

/**
 * STEP 5 (DIP): every volatile collaborator arrives from outside.
 * No concrete database, mail or pricing class is named here.
 */
public class Checkout {
    private final PricingRule rule;
    private final OrderSaver orders;
    private final OrderNotifier notifier;

    public Checkout(PricingRule rule, OrderSaver orders, OrderNotifier notifier) {
        this.rule = Objects.requireNonNull(rule);
        this.orders = Objects.requireNonNull(orders);
        this.notifier = Objects.requireNonNull(notifier);
    }

    public boolean process(Order order) {
        // Validation has not varied yet, so it stays a plain check (see "What we did not do").
        if (order.amount().signum() <= 0) {
            System.out.println("  rejected: amount must be positive");
            return false;
        }
        BigDecimal total = rule.apply(order.amount());
        orders.save(order, total);
        notifier.send(order, total);
        return true;
    }
}
```

```java
// File: code/lld/ch04/checkout/Main.java
package lld.ch04.checkout;

import java.math.BigDecimal;
import java.util.List;
import lld.ch04.checkout.fin.Checkout;
import lld.ch04.checkout.fin.EmailNotifier;
import lld.ch04.checkout.fin.InMemoryOrders;
import lld.ch04.checkout.fin.SilentNotifier;
import lld.ch04.checkout.fin.SqlOrderRepository;
import lld.ch04.checkout.pricing.CarelessFixedDiscount;
import lld.ch04.checkout.pricing.FixedDiscount;
import lld.ch04.checkout.pricing.MemberDiscount;
import lld.ch04.checkout.pricing.PricingRule;
import lld.ch04.checkout.pricing.RegularPrice;
import lld.ch04.checkout.pricing.StaffDiscount;
import lld.ch04.checkout.step0.CustomerOrder;
import lld.ch04.checkout.step0.OrderService;
import lld.ch04.checkout.step2.PriceCalculator;
import lld.ch04.checkout.step3.PricingRuleContract;

public class Main {
    private static final BigDecimal TWENTY = new BigDecimal("20.00");

    public static void main(String[] args) {
        System.out.println("=== Step 0: the original class ===");
        var legacy = new OrderService();
        legacy.process(new CustomerOrder("A-1", "regular", new BigDecimal("100.00"), "ada@example.com"));
        legacy.process(new CustomerOrder("A-2", "staff", new BigDecimal("100.00"), "grace@example.com"));

        System.out.println("=== Step 1: SRP -- split by actor (same output) ===");
        new lld.ch04.checkout.step1.Checkout()
                .process(new CustomerOrder("A-2", "staff", new BigDecimal("100.00"), "grace@example.com"));

        System.out.println("=== Step 2: OCP -- one class per pricing rule ===");
        var calc = new PriceCalculator();
        var order = new Order("A-3", new BigDecimal("100.00"), "ada@example.com");
        for (PricingRule r : List.of(new RegularPrice(), new MemberDiscount(), new StaffDiscount())) {
            System.out.println("  " + r.getClass().getSimpleName() + " -> " + calc.total(order, r));
        }

        System.out.println("=== Step 3: LSP -- every rule must keep 'total >= 0' ===");
        var small = new Order("A-4", new BigDecimal("10.00"), "ada@example.com");
        System.out.println("  careless FixedDiscount(20) on 10 -> "
                + calc.total(small, new CarelessFixedDiscount(TWENTY)));
        System.out.println("  correct  FixedDiscount(20) on 10 -> "
                + calc.total(small, new FixedDiscount(TWENTY)));
        List<PricingRule> all = List.of(new RegularPrice(), new MemberDiscount(),
                new StaffDiscount(), new FixedDiscount(TWENTY), new CarelessFixedDiscount(TWENTY));
        long passing = all.stream().filter(PricingRuleContract::holds).count();
        System.out.println("  " + passing + "/" + all.size() + " rules are safe substitutes");

        System.out.println("=== Steps 4+5: ISP + DIP -- same Checkout, two wirings ===");
        var a9 = new Order("A-9", new BigDecimal("200.00"), "ada@example.com");
        System.out.println("Production wiring:");
        var db = new SqlOrderRepository();
        new Checkout(new MemberDiscount(), db, new EmailNotifier()).process(a9);
        System.out.println("  reporting side can still find it: " + db.findById("A-9").isPresent());

        System.out.println("Test wiring, same Checkout class:");
        var fake = new InMemoryOrders();
        new Checkout(new MemberDiscount(), fake, new SilentNotifier()).process(a9);
        System.out.println("  test double recorded " + fake.saved());

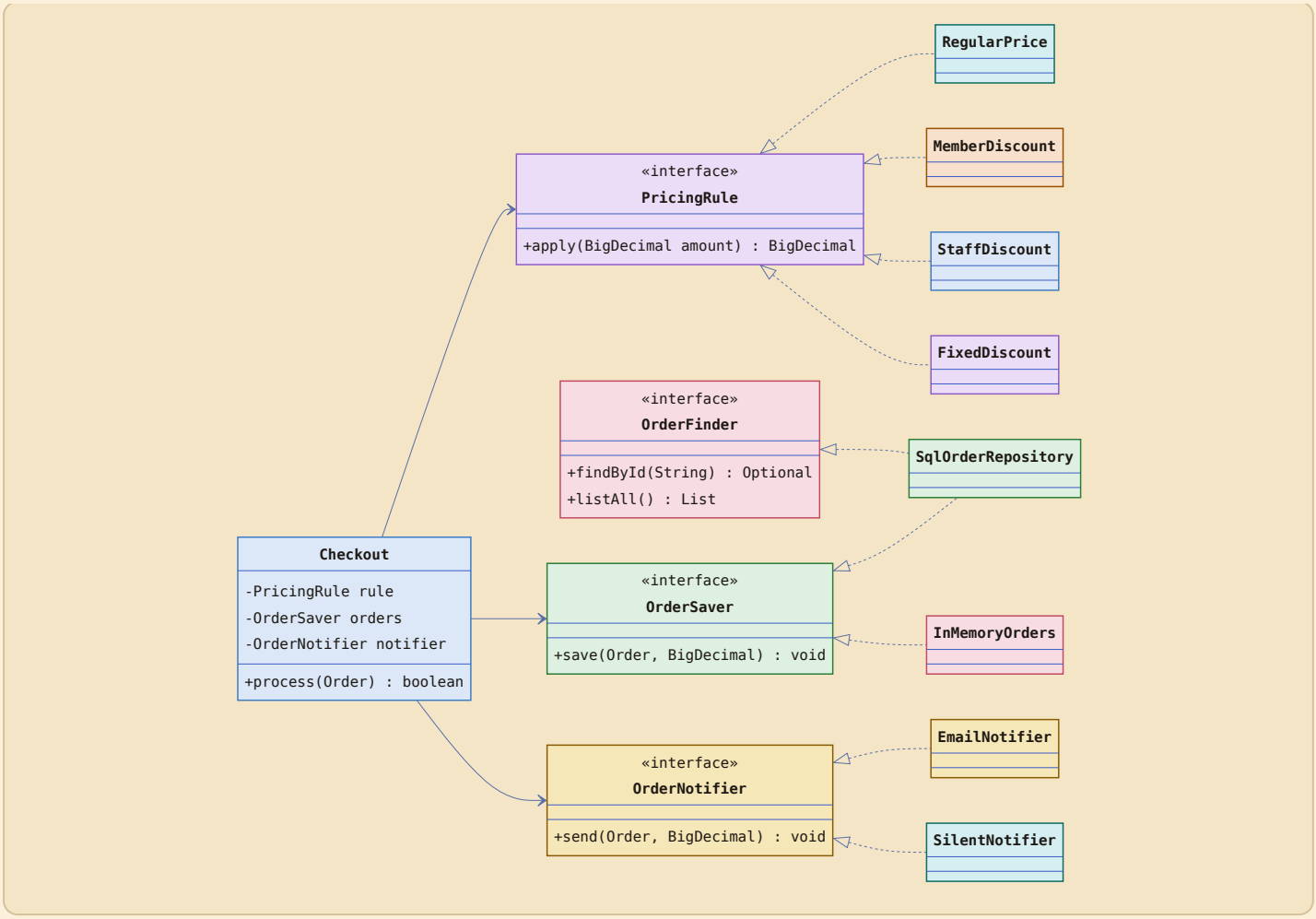
        System.out.println("Lambda wiring (roles are functional interfaces):");
        new Checkout(amount -> amount, (o, t) -> System.out.println("  saved " + o.orderId()),
                (o, t) -> System.out.println("  notified " + o.email())).process(a9);
        new Checkout(new RegularPrice(), fake, new SilentNotifier())
                .process(new Order("A-10", BigDecimal.ZERO, "x@example.com"));
    }
}
```

```
    }
}
```

Output:

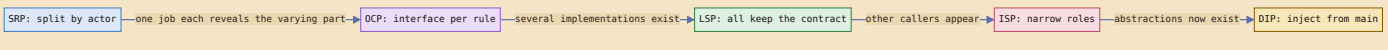
```
=== Step 0: the original class ===
INSERT INTO orders VALUES ('A-1', 100.00)
Emailing ada@example.com: your order total is 100.00
INSERT INTO orders VALUES ('A-2', 70.00)
Emailing grace@example.com: your order total is 70.00
=== Step 1: SRP -- split by actor (same output) ===
INSERT INTO orders VALUES ('A-2', 70.00)
Emailing grace@example.com: your order total is 70.00
=== Step 2: OCP -- one class per pricing rule ===
  RegularPrice -> 100.00
  MemberDiscount -> 90.00
  StaffDiscount -> 70.00
=== Step 3: LSP -- every rule must keep 'total >= 0' ===
  careless FixedDiscount(20) on 10 -> -10.00
  correct  FixedDiscount(20) on 10 -> 0.00
  RegularPrice keeps the contract
  MemberDiscount keeps the contract
  StaffDiscount keeps the contract
  FixedDiscount keeps the contract
  CarelessFixedDiscount broke the contract: apply(0.00) = -20.00
  4/5 rules are safe substitutes
=== Steps 4+5: ISP + DIP -- same Checkout, two wirings ===
Production wiring:
  INSERT INTO orders VALUES ('A-9', 180.00)
  emailing ada@example.com: total 180.00
  reporting side can still find it: true
Test wiring, same Checkout class:
  kept A-9 in memory
  (test run, no email sent)
  test double recorded [A-9=180.00]
Lambda wiring (roles are functional interfaces):
  saved A-9
  notified ada@example.com
  rejected: amount must be positive
```

One unmodified `Checkout` class ran against production collaborators, against test doubles and even against lambdas, **without being edited or subclassed**. That is what the five steps bought.



Reading the five steps back

Step	The question it asked	What changed
SRP	Who might request changes here?	The single class split into four, one per actor
OCP	What breaks when a new case shows up?	Each pricing branch turned into its own class
LSP	Does each rule work when called through the interface?	All rules guarantee the total never drops below zero
ISP	Is the caller tied to more than it calls?	Checkout depends on one save method rather than a whole repository
DIP	Who decides which concrete class is used?	The entry point decides, not the checkout



The sequence has a reason. SRP goes first because the point where behaviour varies stays hidden until each class has a single job. OCP then introduces an interface. LSP becomes relevant as soon as that interface has a second implementation. ISP becomes relevant when new callers (reporting) show up. DIP is last because inversion needs abstractions to point at. Every step sets up the problem the next step solves, which is why the five are normally learned as a group.

What we did not do

- **Not every principle was applied to every class.** `Order` remains a simple record without an interface, because no requirement has made it vary.
- **Not every class was closed against every change.** Adding a validation rule still means editing `Checkout`. That is fine while validation stays stable; the day a third validation rule is requested

is the day validation deserves the treatment pricing received.

- **Not everything was injected.** `Order` and the `BigDecimal` arithmetic are stable, so they are used directly.

Pre-emptively applying all five principles to every class turns a small, clear program into a confusing one. **Each step above was triggered by an actual change request, never by a checklist.**

Self-check

1. **Q:** Why does SRP come before OCP here? **A:** Until pricing lived in its own class, the varying part (the branch on customer type) was buried in a method that did four things.
2. **Q:** What contract did step 3 add? **A:** Every `PricingRule` returns a total ≥ 0 for any amount ≥ 0.
3. **Q:** Why is `OrderSaver` one method wide? **A:** The checkout only saves; depending on the full repository would tie it to reporting methods it never calls.
4. **Q:** What proves the design worked? **A:** One `Checkout` class ran with three different wirings (production, test doubles, lambdas) without being edited.

TL;DR

SRP → OCP → LSP → ISP → DIP, each triggered by a real change. Proof: one class, several wirings, zero edits.

4.12 SOLID and design patterns

Most classic design patterns (Chapters 8–11) exist because of these principles. Learning the principles first makes the patterns look inevitable rather than arbitrary: **the principles say what good structure looks like; the patterns are named, reusable ways to produce it.**

Pattern	Principle(s) it serves	How
Strategy	OCP, DIP	A varying algorithm behind an interface (`InvoiceGenerator`, `PricingRule`)
Template Method	OCP	A fixed skeleton in a base class; subclasses fill in hook steps
Decorator	OCP, SRP	Adds behaviour by wrapping, without modifying the original class
Adapter	DIP, ISP	Lets a caller keep depending on the interface it already owns while a foreign class is plugged in
Factory Method / Abstract Factory	DIP, OCP	Keeps callers away from concrete constructors; the `switch` lives in one place
Observer	OCP, DIP	New reactions are new subscribers; the subject depends only on a listener interface
Composite	LSP, OCP	Leaves and groups share one interface, so callers treat them the same
Proxy	SRP, OCP	Adds access control, caching or laziness (creating things only when first needed) in a separate class
Command	SRP, OCP	Each request is its own object; new commands are new classes
Visitor	OCP (for operations)	Easy to add operations, hard to add element types: OCP along a different axis
Null Object	LSP	A do-nothing implementation that still keeps the contract, instead of `null` checks

Head First Design Patterns states several of the same ideas as its "OO design principles": **encapsulate what varies**, **favour composition over inheritance**, **program to an interface, not an implementation**, **classes should be open for extension but closed for modification** (its Decorator chapter), **depend upon**

abstractions, do not depend upon concrete classes (its Factory chapter's name for DIP), and **a class should have only one reason to change** (its Iterator/Composite chapter). *Clean Code* covers SRP and "organising for change" in its chapter on classes, and "isolating from change" is DIP by another name.

4.13 SOLID in the design interview

Keeping the design honest: five checks over your class diagram

A complete class diagram can still be a poor design. In an OOD interview, SOLID is most useful as **five quick checks over the diagram you just drew**, taking about five minutes. Run them in this order:

1. **Single responsibility.** Say each class's responsibility in one sentence. If the sentence needs an "and", the class has two jobs. A `ParkingLot` that assigns spots **and** prints receipts changes when pricing changes and again when the layout changes: two classes. *Quick check:* name the change that would force you to edit the class; if you can name two unrelated changes, split it.
2. **Open/closed.** Look for a `switch` or `if` chain over a type. In a parking lot, pricing as a switch over rate types means a weekend rate edits the class that prices everything. Put pricing behind a small interface and the weekend rate is a new class. *Quick check:* pick one likely feature; if adding it edits three existing classes, the design is closed to the wrong thing.
3. **Liskov substitution.** Read each inheritance arrow: can the child be handed to code expecting the parent? If any parent method would throw, do nothing or refuse, the arrow is wrong. Check this again at the end: inheritance arrows get added late, when time is short.
4. **Interface segregation.** Look for a big interface where implementers leave methods empty. Split until every implementer uses everything it promised. A library `Search` interface with four methods is fine if the catalog genuinely implements all four.
5. **Dependency inversion.** Find high-level classes naming low-level ones. A `ParkingLot` that constructs a `CreditCardTransaction` cannot be tested without a payment provider and cannot take cash without an edit. Make it depend on `Payment` and let something else decide which kind arrives.

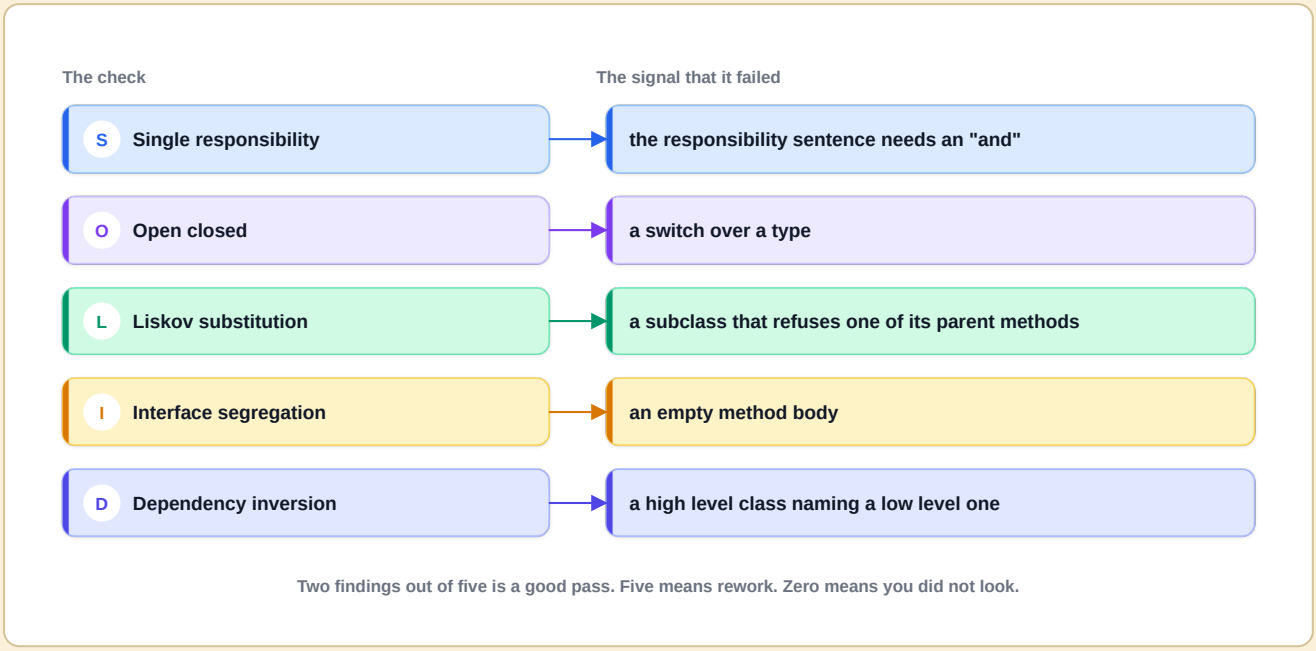


Figure — each check and the signal that it failed: an "and" in the sentence, a switch over a type, a refused parent method, an empty method body, a high-level class naming a low-level one (course diagram)

In the figure above, the right-hand column is what to scan for. The note at the bottom is the calibration: **two findings out of five is a good pass; five means rework; zero means you did not look.**

The two that earn the most credit. SRP improves your class list: twelve well-named classes beat six that each do three things. **OCP** is the one interviewers probe, usually as a late requirement ("what if trucks are charged a different rate?"). "Add a class" is a stronger answer than "add a branch in this switch".

A five-minute pass over a parking lot design:

Check	Finding
Single responsibility	`ParkingLot` issues tickets and finds spots: two reasons to change. Split if there is time; name it out loud if not.
Open/closed	Pricing sits behind `ParkingRate`, so a new rate is a new class. Passes.
Liskov substitution	Five spot subclasses add no behaviour. Collapse them into one class with a `type` field.
Interface segregation	No wide interfaces. Passes.
Dependency inversion	The lot depends on `Payment`, not on the card class. Passes.

Two real findings, each explainable in a sentence. That is what a good pass looks like.

Where SOLID is not the answer. It is a set of checks, not a scoring system. Do **not** add an interface for everything: an interface with one implementer, added because a rule "seemed to ask for it", is indirection with no benefit; add it when a second implementation exists or is clearly coming. Do **not** split classes further than a reader can follow: thirty classes with two methods each cannot be held in anyone's head. The one-sentence test is the boundary: if the responsibility fits in one sentence, the class is small enough.

Naming a design problem out loud

Interviewers seldom ask for a definition of OCP. They ask for a parking lot, a rate limiter (a component that caps how many requests are allowed per time window) or a file store, and then judge whether you can **explain the reasons behind the shape of your design**, which separates someone who memorised five acronyms from someone who uses them. Three things are weighed, most important first:

1. **Can you spot and name a problem in the code in front of you?** Shown a class with an ever-growing `if/else` on a type field, can you state the flaw in one crisp sentence?
2. **Can you defend a choice?** "SOLID says so" earns nothing. "Pricing rules change every quarter, and I don't want every change to reopen the checkout" earns a lot.
3. **Do you know when to hold back?** This is what marks a senior candidate. Adding abstractions is easy; choosing which ones to skip is the rarer skill.

A sentence pattern that works — say three things, in this order:

1. **What you see** — a specific fact about the code.
2. **What it will cost** — the future change that this makes painful.
3. **What you would do, and what that costs you** — the remedy and its price. This third part is the one most people leave out, and it is exactly what makes an answer sound like it comes from experience rather than from memory.

Example: a `ReportGenerator` that formats a report, writes it to disk and emails it.

- Weak: "It breaks SRP since it does three things; I'd make three classes." Correct, but it sounds like a definition recalled from last night's reading.
- Strong: "Three different owners can request changes here: marketing controls the wording, the platform team controls where files land, and the email owners control delivery. That means a copy edit redeploys the file-writing code. I'd pull formatting apart from delivery, likely into a formatter and a sink (the destination the output is sent to). It costs me two more classes and a little wiring, which pays off because the wording changes frequently." It names the actors, the pain, the fix and its price, and ends in a decision.

Say the observation before the principle. "`Reporting` implements nine methods but calls only two of them, which is an interface segregation problem" shows that you noticed something. Opening with "This violates ISP because…" sounds like you went looking for a place to use the acronym. Do still name the principle at the end, because it hands the interviewer a label for the write-up.

"Would you always do that?" This follow-up checks whether you understand the price of your own suggestion; it is rarely a trap, and the right answer is almost never "yes". Keep honest limits ready:

- *On splitting classes:* "No. If the two parts would always be edited together, I've just created an extra file. I split when two different people would ask for two different changes."
- *On extension points:* "No. I keep the `if/else` until a second or third case actually appears. An extension point built in advance is usually wasted effort and makes the code harder to follow in the meantime."
- *On interfaces:* "Only where behaviour varies or a test needs a substitute. An interface with a single implementation and no fake is just one more file."

Treating a principle as unconditional is the quickest way to show you have never actually applied it.

Three things that cost marks:

- **Reciting definitions** without tying them to the code on the whiteboard.
- **Applying everything at once.** Five interfaces for a forty-line problem looks inexperienced, not thorough; real systems earn their abstractions.
- **Naming the wrong principle confidently.** Mixing up cohesion and coupling reveals that the vocabulary is second-hand. When unsure, describe the issue in plain words and leave the label out: an accurate unnamed description beats a confidently wrong name.

A one-minute drill for any class you own: (1) Who can ask me to change this class, and which parts belong to whom? (2) What is the next feature likely to be, and would I have to edit this class for it? (3) Which dependencies could I not replace in a test, and does that matter? Practising this on your own code prepares you better than rereading definitions, and the interview will ask you to do it on code you have never seen.

> `In the interview:` describe before you label, and end every recommendation with its cost, for example: "I'd move delivery into its own class; that costs a file and some wiring, and in return a wording change no longer touches file handling."

4.14 Common questions about SOLID (with short answers)

Give the two-line version and stop; expand only if asked. Long unprompted answers about definitions eat the time you need for the design itself.

1. What is the difference between SRP and ISP? The difference is *whom* each one guards. **SRP guards the class** that does the work: does it have more than one reason to change? **ISP guards the caller** at a boundary: must it depend on methods it never calls? A `Printer` interface with `print`, `scan` and `fax` is an ISP issue for a basic printer, which has to supply two operations it cannot perform. A `ReportService` that both formats and emails is an SRP issue with no interface in sight. *Short answer:* SRP splits classes, ISP splits interfaces, and either can be violated without the other.

2. When would you not apply these principles? This question separates candidates, so have a concrete answer. Skip them when nothing has varied yet (an extension point for a case that never arrives adds indirection and returns nothing); in small or throwaway code (a one-off script needs no interfaces); when the split would be arbitrary (you cannot name the two groups who would each ask for changes); and when it would hurt readability (five four-line files can be harder to follow than one twenty-line file). *Framing:* each principle buys **flexibility later** with **simplicity now**; if "later" never comes, the price was paid for nothing.

3. Is SOLID still relevant, given the criticism? Yes, and being aware of the criticism counts in your favour. The objections are reasonable: the principles came from large object-oriented systems and are frequently taught as absolute rules to people building small ones, which yields layers of interfaces with a single implementation each. Some engineers argue that plainer advice (keep functions small, make dependencies explicit) captures most of the benefit. What outlasts the criticism is the **vocabulary and the diagnosis**: a compact way to explain why a piece of code will resist change, in any language or

paradigm. *Short answer:* valuable as diagnostic tools, misleading as rules; treat them as names for problems, not a checklist.

4. Dependency inversion vs dependency injection? Candidates mix these up constantly, so a crisp answer stands out. Inversion is the **principle** (depend on abstractions); injection is one **technique** for reaching it (supply dependencies from outside rather than constructing them inside). Injecting a concrete class is injection without inversion, and a hand-written constructor with no framework is inversion without a container. Spring only automates the wiring; it is not the principle.

5. Give an example of an LSP violation. Prefer one from your own work; failing that, Square/Rectangle, but explain the underlying rule rather than just the story: `Rectangle` lets callers set width and height separately, `Square` cannot permit that and remain a square, so code written for `Rectangle` computes a wrong area without any error. The general rule: a subclass must not require more, guarantee less, or break an assumption callers depend on; an exception is merely the most obvious way to break it.

6. Does OCP mean I can never edit a file? No, and saying so plainly is a good answer. It means one more variation *of a kind you already support* can be added without touching existing code; the file is not frozen. Bug fixes, refactorings and changes to the extension point itself all involve edits. Closure is also relative to a kind of change: the invoice design in 4.7 was closed to new formats yet fully open to new discount rules, and that was the right trade at the time.

7. Composition over inheritance, always? No; it is a guideline, not a rule. Inherit when the child really is a kind of the parent and keeps every inherited promise. Compose when a behaviour differs between objects or a subclass would have to reject an inherited method. That rejection is the thing to watch for: an override that throws, does nothing, or is documented as "don't call this on X" belongs in a separate object.

8. How do these principles relate to design patterns? Many patterns are these principles in concrete form, which is why learning the principles first makes the patterns feel natural. **Strategy** applies OCP to an algorithm that varies; **Adapter** lets a caller go on using the interface it already knows; **Decorator** adds behaviour without editing the wrapped class; **Factory** hides concrete constructors from callers, which serves DIP. The principles describe good structure; the patterns are named, reusable recipes for producing it (see 4.12).

9. Which principle is violated most often? There is no single right answer, but two defensible ones. **SRP**, because classes grow by small accumulations: no one sets out to write a two-thousand-line service; it builds up one sensible-looking addition at a time. **ISP in reverse** (over-splitting): engineers fresh from learning it create an interface per method, meeting the wording of the rule while adding names and removing no dependency. Citing the second shows you have watched the principles being misused, not merely ignored.

> **Interview tip:** if the interviewer invites you to go deeper, the two questions worth expanding are "when would you not apply this?" and "dependency inversion vs dependency injection", since most candidates stay vague on both and a precise answer stands out. The strongest answers on SOLID always **admit a limit**: anyone can praise a principle; saying where it stops paying off is what sounds like real maintenance experience.

4.15 Over-engineering: the warnings in one place

Warning sign	Principle misapplied	What to do instead
Classes named `XHelper`, `XUtils`, `XManager2` after a split	SRP	Split by actor; each new class needs a name that stands alone
Six small classes that always change in the same commit	SRP (over-split) — *Shotgun Surgery*	Merge them back
An interface + factory + registry for a single implementation	OCP — *Speculative Generality*	Write the `if`; refactor at the second or third case (rule of three)
A subclass that "works" by silently doing nothing	LSP	Raise a better parent or use composition
One interface per method; the same object passed three times	ISP (over-split)	Group by caller into role interfaces
An interface in front of `String`-like stable classes	DIP	Invert only volatile dependencies
A DI container for a 200-line program	DIP	Wire by hand in `main`

YAGNI ("You Aren't Gonna Need It", from Extreme Programming, an agile method built on small frequent releases and heavy testing) is the counterweight to all of SOLID: do not build for requirements you do not have. SOLID tells you **what** a good seam looks like; YAGNI tells you **when** to cut it: when a real change asks for it.

Chapter self-check

1. **Q:** State each SOLID principle in one line. **A:** SRP: one reason (actor) to change. OCP: add behaviour by adding code, not editing. LSP: subtypes must be substitutable for their base types. ISP: no caller depends on methods it does not use. DIP: policy and details both depend on abstractions owned by the policy.
2. **Q:** A class has a `switch (vehicleType)` that computes parking fees. Which principle, and what fix? **A:** OCP. Put fees behind a `ParkingRate` interface with one class per rate (Strategy), chosen at the edge.
3. **Q:** `ReadOnlyList extends ArrayList` and throws on `add`. Which principle? **A:** LSP (a new exception / refused bequest). Prefer composition or a read-only interface.
4. **Q:** A `Robot` class implements a `Worker` interface with `work()` and `eat()`. What is wrong? **A:** ISP: the robot must implement `eat()`. Split into `Workable` and `Eater` roles based on callers.
5. **Q:** `OrderService` has `private final MySqlOrderDao dao = new MySqlOrderDao();`. What two problems? **A:** Tight coupling / DIP violation (names and creates a volatile detail) and untestable without MySQL. Inject an `OrderSaver` interface owned by the service.
6. **Q:** Why is Square/Rectangle a worse bug than `Penguin.fly()` throwing? **A:** It is silent: the program continues with a wrong result.
7. **Q:** Coupling vs cohesion in one sentence each. **A:** Cohesion is how well the insides of one class belong together; coupling is how much one class depends on another.
8. **Q:** In what order do the principles usually appear during a refactor, and why? **A:** SRP → OCP → LSP → ISP → DIP; each creates the situation the next one addresses.
9. **Q:** What is the difference between DIP, DI and IoC? **A:** DIP is the principle (depend on abstractions); DI is a technique (pass dependencies in); IoC is the broader idea that a framework calls your code, and IoC containers perform DI.
10. **Q:** When should you not add an interface? **A:** When there is one implementation, no test double, and nothing has varied; or when the dependency is stable.

TL;DR

- **SOLID** = five names for the ways code becomes painful to change. Use them to diagnose, not as a checklist.
- **SRP**: one actor per class. Split by *who asks for changes*, not by method count. High cohesion inside, and check coupling after every split.
- **OCP**: put the part that *has* varied behind an interface; add variants as classes (Strategy). A class is closed against a specific kind of change.
- **LSP**: a subclass must accept at least as much, promise at least as much, keep invariants and history, and throw nothing new. Inheritance follows promises, not real-world categories. Prefer composition when a subclass would refuse a method.
- **ISP**: one role interface per kind of caller. Throwing methods mean too wide; the same object passed three times means too thin.
- **DIP**: policy owns the abstraction; details implement it; `main` wires them. DI is the technique, IoC containers are the tool.
- **Order**: SRP → OCP → LSP → ISP → DIP, each triggered by a real change.
- **In the interview**: observation → cost → fix → price of the fix. Always know when *not* to apply a principle.

Chapter 5 — More Design Principles

SOLID (Chapter 4) is only five of the rules good designers use. This chapter covers the rest of the everyday toolkit: short rules of thumb (DRY, KISS, YAGNI), rules about how objects talk to each other (Law of Demeter, Tell-Don't-Ask, loose coupling, the Hollywood Principle), the object-oriented principles that *Head First Design Patterns* and the Gang of Four book (*Design Patterns*, 1994, by Gamma, Helm, Johnson and Vlissides — "GoF" for short) build every pattern on (encapsulate what varies, favour composition, program to an interface), rules about correctness (fail fast, immutability, Design by Contract) **GRASP** — nine guidelines for deciding *which class should do what* — and, finally, the essentials of **Domain-Driven Design**, which decides *which classes exist and where their boundaries are* by starting from the business.

Key terms used throughout (defined once here):

Term	Meaning
Coupling	How much one class depends on (knows about) another. High coupling = a change in one forces changes in the other.
Cohesion	How closely the things inside one class belong together. High cohesion = the class does one focused job.
Abstraction	A simplified view (usually an interface or abstract class) that hides details that callers do not need.
Encapsulation	Keeping an object's data private and letting other code change it only through the object's methods.
Invariant	A fact about an object that must always be true (e.g. "balance is never negative").
Side effect	Anything a method changes besides returning a value: fields, files, the database, the screen.
Responsibility	Something a class *knows* (its data) or *does* (its behaviour).

Chapter map

#	Principle	One-line idea	Comes from
1	DRY	Each piece of knowledge has one authoritative home	Hunt & Thomas, *The Pragmatic Programmer* (1999)
2	KISS	Choose the simplest design that works	US Navy (1960); popularised by Kelly Johnson, Lockheed Skunk Works
3	YAGNI	Don't build features before they are needed	Extreme Programming, late 1990s (usually credited to Ron Jeffries)
4	Law of Demeter	Only talk to your immediate friends	Ian Holland / Karl Lieberherr, Demeter project, Northeastern University (1987)
5	Tell, Don't Ask	Tell objects what to do; don't pull their data out and decide for them	Andy Hunt & Dave Thomas (Pragmatic Programmers), building on Alec Sharp's *Smalltalk by Example* (1997); explained by Martin Fowler
6	Composition over inheritance	Prefer "has-a" to "is-a" for reuse	GoF book (1994), *Effective Java* (Joshua Bloch)
7	Program to an interface	Depend on supertypes, not concrete classes	GoF book
8	Encapsulate what varies	Isolate the part that changes	GoF book, *Head First Design Patterns*
9	Loose coupling	Interacting objects should know little about each other	*Head First Design Patterns*
10	Hollywood Principle / IoC	"Don't call us, we'll call you"	*Head First*; frameworks, IoC containers
11	Separation of Concerns	Each part of the program handles one concern	Edsger Dijkstra (1974)
12	Least Astonishment	Behave the way readers expect	User-interface and programming-language design folklore (no single author)
13	Fail fast	Detect problems early and loudly	Popularised by Jim Shore's "Fail Fast" article, *IEEE Software* (2004)
14	Immutability	Objects that never change after creation	*Effective Java* ("minimize mutability")
15	Design by Contract	Preconditions, postconditions, invariants	Bertrand Meyer, Eiffel language (term coined 1986)
16	GRASP (9 patterns)	Guidelines for assigning responsibilities	Craig Larman, *Applying UML and Patterns* (1st ed. 1997; today's nine from the 2nd ed. 2001)
17	Domain-Driven Design essentials	Model the business: bounded contexts, entities vs value objects, aggregates, domain events, repositories	Eric Evans, *Domain-Driven Design* (2003); Vaughn Vernon, *Implementing Domain-Driven Design* (2013)

How to read each section. Every principle follows the same path: what it is → the problem → how to apply it → a Java *before/after* program you can run → trade-offs → when not to use it → mistakes → real-world examples → interview answer → self-check → TL;DR. All code lives in `code/lld/ch05/<topic>/Main.java` and was compiled with `javac --release 21` (`javac` is the Java compiler, which turns source code into runnable bytecode; `--release 21` targets Java 21) and run; the output shown is the real output.

---

1. DRY — Don't Repeat Yourself

What it is

DRY says: every piece of *knowledge* in a system — a business rule, a formula, a validation, a constant — should have **a single, unambiguous, authoritative representation** — one place where it is defined. Everything else refers to that place. The principle comes from Andy Hunt and Dave Thomas's *The Pragmatic Programmer* (1999).

Analogy: a company keeps one master price list. Shops look prices up from it. If each shop kept its own hand-written copy, a price change would reach some shops and not others.

Important: DRY is about **knowledge**, not about lines of text that happen to look the same. Two pieces of code that look identical but represent *different* rules (which may change for different reasons) are *not* a DRY violation.

The problem it solves

When a rule is copied into several places, the copies eventually **drift**: someone updates one copy and forgets the others. The system then gives two different answers to the same question — a bug that is hard to spot because each copy looks reasonable on its own. Duplication also multiplies the effort of every change.

How it works

1. Notice a rule that appears in more than one place (a regex, a tax rate, a status check, a SQL fragment). A *regex* (regular expression) is a compact pattern that text must match, such as "something@something"; *SQL* is the standard language for querying relational databases.
2. Ask: *"If this rule changes, must all copies change together?"* If yes, it is one piece of knowledge.
3. Give it one home: a constant, a method, a value object, an enum, a config entry or a shared module.
4. Replace every copy with a reference to that home.
5. Apply the **Rule of Three** for code-level duplication (a heuristic credited to Don Roberts and popularised by Martin Fowler's *Refactoring* — refactoring means improving code's structure without changing what it does, see Chapter 7): the first time, just write it; the second time, notice the duplication but tolerate it; the third time, extract.

Java example — before/after

The *before* has two services that each carry their own copy of "what a valid email is" and "what the tax is". The copies have already drifted. The *after* puts the email rule inside an `EmailAddress` value object (a small immutable type — one whose value can never change after it is created — whose constructor validates its value) and the tax rule inside `TaxPolicy`.

```java
// File: lld/ch05/dry/Main.java
package lld.ch05.dry;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.regex.Pattern;

/** DRY: one piece of knowledge (what a valid email is, what the tax rate is) lives in ONE place. */
public class Main {

    // ---------------- BEFORE: the same rule is written twice and the copies have drifted ----------------
    static class Before {
        static class SignupService {
            boolean signup(String email) {
                // copy #1 of the rule (someone relaxed it later)
                return email != null && email.contains("@");
            }
            BigDecimal priceWithTax(BigDecimal price) {
                return price.multiply(new BigDecimal("1.18")).setScale(2, RoundingMode.HALF_UP); // 18% hard-coded
            }
        }
        static class ProfileService {
            boolean changeEmail(String email) {
                // copy #2 of the rule (the stricter original)
                return email != null && email.matches("[^@\\s]+@[^@\\s]+\\.[a-z]{2,}");
            }
            BigDecimal priceWithTax(BigDecimal price) {
                return price.multiply(new BigDecimal("1.18")).setScale(2, RoundingMode.HALF_UP); // 18% again
            }
        }
    }

    // ---------------- AFTER: each rule has a single, authoritative home ----------------
    static class After {
        /** The ONLY definition of "valid email" in the system. */
        record EmailAddress(String value) {
            private static final Pattern FORMAT = Pattern.compile("[^@\\s]+@[^@\\s]+\\.[a-z]{2,}");
            EmailAddress {
                if (value == null || !FORMAT.matcher(value).matches())
                    throw new IllegalArgumentException("invalid email: " + value);
            }
        }

        /** The ONLY definition of the tax rule. */
        static final class TaxPolicy {
            private static final BigDecimal RATE = new BigDecimal("0.18");
            private TaxPolicy() {}
            static BigDecimal withTax(BigDecimal price) {
                return price.add(price.multiply(RATE)).setScale(2, RoundingMode.HALF_UP);
            }
        }

        static class SignupService {
            String signup(String rawEmail) { return "signed up " + new EmailAddress(rawEmail).value(); }
        }
        static class ProfileService {
            String changeEmail(String rawEmail) { return "email changed to " + new EmailAddress(rawEmail).value();
}
        }
    }

    public static void main(String[] args) {
        System.out.println("== BEFORE ==");
        var signup = new Before.SignupService();
        var profile = new Before.ProfileService();
        String sloppy = "bob@localhost";
        System.out.println("signup accepts '" + sloppy + "'? " + signup.signup(sloppy));
        System.out.println("profile accepts '" + sloppy + "'? " + profile.changeEmail(sloppy) + "   <-- same rule,
two answers");
```

```
        System.out.println("== AFTER ==");
        var s = new After.SignupService();
        var p = new After.ProfileService();
        System.out.println(s.signup("bob@example.com"));
        System.out.println(p.changeEmail("bob@example.com"));
        for (String candidate : new String[]{sloppy}) {
            try { s.signup(candidate); } catch (IllegalArgumentException e) { System.out.println("signup rejected:
" + e.getMessage()); }
            try { p.changeEmail(candidate); } catch (IllegalArgumentException e) { System.out.println("profile
rejected: " + e.getMessage()); }
        }
        System.out.println("price 100.00 with tax = " + After.TaxPolicy.withTax(new BigDecimal("100.00")));
    }
}
```

Output:

```
== BEFORE ==
signup accepts 'bob@localhost'? true
profile accepts 'bob@localhost'? false   <-- same rule, two answers
== AFTER ==
signed up bob@example.com
email changed to bob@example.com
signup rejected: invalid email: bob@localhost
profile rejected: invalid email: bob@localhost
price 100.00 with tax = 118.00
```

What each part does:

- `Before.SignupService` and `Before.ProfileService` each re-implement the email rule; the output shows them disagreeing about `bob@localhost`.
- `After.EmailAddress` is a Java **record** (a compact, immutable data class). Its *compact constructor* (a record constructor written without a parameter list, used only to check or normalise the incoming values) runs the check, so an invalid email simply cannot exist as an `EmailAddress`.
- `After.TaxPolicy` holds the rate in one private constant; changing GST from 18% means editing one line.

Trade-offs

Benefit	Cost
One change point per rule; no drift	A shared abstraction couples its users together
Less code to read and test	A premature or wrong abstraction is worse than duplication
Rules get a name (self-documenting)	Extra indirection to find the real code

When not to use it

- When two snippets look alike **by coincidence** and will evolve separately (e.g. "max username length 30" and "max tag length 30"). Merging them creates false coupling.
- Across service boundaries in **microservices** (an architecture where one application is split into many small services, each deployed on its own and talking over the network): sharing a domain library between independently deployed services can couple their release cycles. Some duplication is the lesser evil.
- In tests, a little repetition often makes each test easier to read on its own (this is sometimes called DAMP — "descriptive and meaningful phrases").

Common mistakes

- Treating DRY as "never have two similar lines" and building one monster function with flags for every caller.
- Extracting after the first repetition, before you understand what really varies.
- Forgetting non-code duplication: the same rule in code *and* in a database constraint *and* in docs.

Real-world examples

- **Spring's `@Value` / `application.yml`** (Spring is the most widely used Java framework for server applications; `@Value` is an *annotation* — a `@`-prefixed tag on code that tools and frameworks read — and `application.yml` is its configuration file): configuration values live in one file and are injected wherever needed.
- **Database normalisation** is DRY for data: a customer's address is stored once and referenced by key (a unique ID column that other tables point to).
- **Bean Validation (`jakarta.validation`) annotations** such as `@Email` on a field define the rule once and reuse it for REST input (data arriving through a web API built on HTTP's standard verbs and URLs), forms and persistence (saving data so it outlives the program, usually in a database).
- **OpenAPI / Protobuf schemas** (machine-readable descriptions of an API's messages — OpenAPI for HTTP APIs that exchange JSON, a plain-text key–value data format; Protobuf for Google's compact binary format) generate client and server code from one contract, so the message format is not hand-copied.

In the interview

> **30-second answer:** "DRY means each piece of knowledge has one authoritative representation. It is about rules, not about identical-looking text. I extract a rule when copies must change together — for example, validation into a value object — and I use the Rule of Three so I don't abstract too early."

Follow-ups:

- *Q: Is duplicated code always a DRY violation?* A: No. If the two copies represent different rules that change for different reasons, merging them adds wrong coupling.
- *Q: DRY vs. SRP?* A: They cooperate (SRP is the Single Responsibility Principle from Chapter 4: a class should have only one reason to change). DRY gives each rule one home; SRP makes sure that home has only one reason to change.
- *Q: How do you spot duplication of knowledge?* A: Ask "if this business rule changes, how many files do I edit?" More than one is a smell.

Self-check

1. Q: What exactly must not be repeated? → A: Knowledge (rules, formulas, decisions), not merely similar-looking lines.
2. Q: What is the Rule of Three? → A: Tolerate a second copy; extract a shared abstraction when a third appears.
3. Q: What bug does duplication cause? → A: Drift — copies change independently and start disagreeing.

TL;DR

One rule, one home. Extract when copies must change together; leave coincidental look-alikes alone.

---

2. KISS — Keep It Simple, Stupid

What it is

KISS says most systems work best when they are kept simple. The phrase was used by the US Navy around 1960 and is associated with Kelly Johnson, lead engineer of Lockheed's Skunk Works, who wanted jet aircraft a mechanic could repair in the field with basic tools. Choose the plainest design that fully solves the real problem. *Simple* means easy to understand and change — not necessarily the fewest lines.

Analogy: to hang one picture you use a nail, not a motorised track system with remote control.

The problem it solves

Engineers love clever solutions: home-made frameworks, deep inheritance trees, generic engines configured by lambdas (short unnamed functions written inline, e.g. `x -> x * 2`). Every extra moving part must be read, tested, debugged and explained to new team members. Accidental complexity (complexity we add, as opposed to complexity the problem truly has) is the biggest source of bugs and slow delivery.

How it works

1. State the actual requirement in one sentence.
2. Write the most direct solution using standard language features and libraries.
3. Ask: "Could a new team member understand this in two minutes?"
4. Add structure (patterns, abstractions) only when a concrete need — a second variant, a test seam, a real performance issue — shows up.
5. Delete cleverness that no longer earns its keep.

Java example — before/after

Three fixed customer tiers get three fixed discounts. The *before* builds a priority-sorted rule engine of predicates and functions. The *after* is a `switch` expression, which the compiler also checks for completeness: a `switch` *expression* over an enum (a type with a fixed list of named constants) with no `default` must cover every constant, so adding a new tier makes it fail to compile until you handle the new case. (Old-style `switch` *statements* are not checked this way.) Money is `BigDecimal` (Java's exact decimal-number class), never `double` (binary floating point cannot represent most decimal amounts exactly).

```java
// File: lld/ch05/kiss/Main.java
package lld.ch05.kiss;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
import java.util.function.Function;
import java.util.function.Predicate;

/** KISS: the simplest design that fully solves today's problem wins. Money is BigDecimal, never double. */
public class Main {

    enum Tier { REGULAR, SILVER, GOLD }

    static final BigDecimal SILVER_FACTOR = new BigDecimal("0.90");
    static final BigDecimal GOLD_FACTOR = new BigDecimal("0.80");

    static BigDecimal times(BigDecimal amount, BigDecimal factor) {
        return amount.multiply(factor).setScale(2, RoundingMode.HALF_UP);
    }

    // ---------------- BEFORE: a home-made "rule engine" for three fixed discounts ----------------
    static class Before {
        record Rule(int priority, Predicate<Tier> when, Function<BigDecimal, BigDecimal> apply) {}

        static class DiscountRuleEngine {
            private final List<Rule> rules = new ArrayList<>();
            DiscountRuleEngine register(int priority, Predicate<Tier> when, Function<BigDecimal, BigDecimal>
apply) {
                rules.add(new Rule(priority, when, apply));
                rules.sort(Comparator.comparingInt(Rule::priority).reversed());
                return this;
            }
            BigDecimal evaluate(Tier tier, BigDecimal amount) {
                return rules.stream().filter(r -> r.when().test(tier)).findFirst()
                        .map(r -> r.apply().apply(amount)).orElse(amount);
            }
        }

        static DiscountRuleEngine build() {
            return new DiscountRuleEngine()
                    .register(10, t -> t == Tier.GOLD, a -> times(a, GOLD_FACTOR))
                    .register(5, t -> t == Tier.SILVER, a -> times(a, SILVER_FACTOR))
                    .register(0, t -> true, a -> a);
        }
    }

    // ---------------- AFTER: a plain switch expression; the compiler checks every tier is handled --------------
--
    static class After {
        static BigDecimal discounted(Tier tier, BigDecimal amount) {
            return switch (tier) {
                case GOLD -> times(amount, GOLD_FACTOR);
                case SILVER -> times(amount, SILVER_FACTOR);
                case REGULAR -> amount;
            };
        }
    }

    public static void main(String[] args) {
        BigDecimal price = new BigDecimal("100.00");
        var engine = Before.build();
        System.out.println("== BEFORE (rule engine) ==");
        for (Tier t : Tier.values()) System.out.printf("%-7s -> %s%n", t, engine.evaluate(t, price));
        System.out.println("== AFTER (switch) ==");
        for (Tier t : Tier.values()) System.out.printf("%-7s -> %s%n", t, After.discounted(t, price));
    }
}
```

Output:

```
== BEFORE (rule engine) ==
REGULAR -> 100.00
SILVER  -> 90.00
GOLD    -> 80.00
== AFTER (switch) ==
REGULAR -> 100.00
SILVER  -> 90.00
GOLD    -> 80.00
```

Both produce identical results, but the *after* has no ordering bugs to worry about (in the engine, forgetting a priority would silently change behaviour), no mutable list, and reads like the business rule.

Trade-offs

Simple design	"Flexible" complex design
Fast to read, change, test	Handles hypothetical cases
May need refactoring when real variation arrives	Costs you every day, even if variation never arrives
Uses standard, well-known constructs	Custom mechanisms need docs and training

When not to use it

KISS is never "wrong", but it is not an excuse to ignore the real complexity of the problem. A payment system *needs* idempotency (repeating the same request — say, after a timeout — has the same effect as doing it once, so nobody is charged twice; see System Design Fundamentals), retries and auditing; leaving them out is not simplicity, it is incompleteness. Also, when a rule-engine really is the requirement (business users edit rules at runtime — while the program is running), build or buy one.

Common mistakes

- Confusing *simple* with *easy* or *short*: a one-line stream with five nested lambdas may be shorter but harder to read than a loop.
- Using KISS to justify skipping tests, error handling or validation.
- Copy-pasting instead of simplifying (that violates DRY).

Real-world examples

- **Unix philosophy** — small tools that do one thing and compose through pipes (`|`, which feeds one program's output into the next program's input).
- **Go language design** deliberately left out many features to keep the language simple.
- `java.util.Optional` was intentionally kept small (no `Serializable`, not meant as a field type) to steer people toward its one purpose: return values that might be absent.

In the interview

30-second answer: "KISS means picking the simplest design that fully meets the requirements. In an interview I start with a straightforward model and only introduce patterns when a requirement demands variation or extension — and I say why."

Follow-ups:

- *Q: Isn't a design pattern always better?* A: No. A pattern adds classes and indirection; it's worth it only if it solves a problem you actually have.
- *Q: How do KISS and YAGNI differ?* A: KISS is about *how* you build (the simplest form); YAGNI is about *what* you build (only needed features).

Self-check

1. Q: What is accidental complexity? → A: Complexity introduced by our solution rather than required by the problem.
2. Q: Why did the `switch` version win? → A: Same behaviour, fewer moving parts, compiler-checked exhaustiveness.

TL;DR

Solve the real problem in the plainest way; add machinery only when a real need appears.

---

3. YAGNI — You Aren't Gonna Need It

What it is

YAGNI says: do not build a capability until you actually need it. It comes from Extreme Programming (XP), the agile method (a way of building software in short cycles with frequent feedback) created by Kent Beck in the late 1990s; the slogan is usually credited to XP co-founder Ron Jeffries. The opposite habit — building hooks, options and layers "just in case" — is called **speculative generality**, a **code smell** (a visible sign in the code that usually points to a deeper design problem) in Martin Fowler's *Refactoring*.

Analogy: you don't buy a 7-seater car because you *might* have five children one day.

The problem it solves

Features built early cost four times: the time to build them, the time to maintain and test them, the complexity they add to everything around them, and the cost of being wrong (the real future requirement usually looks different from the guess, so the speculative code must be rewritten anyway).

How it works

1. Implement the stories/requirements you have **now** (a *user story* is a short feature request written from the user's point of view: "As a shopper I can …").
2. Keep the code clean (tested, well-factored) so that it is cheap to change later — YAGNI depends on refactoring being easy.
3. When a new requirement actually arrives, extend the design then.
4. Delete unused options, parameters and classes you discover.

Java example — before/after

The requirement: *export orders as CSV* (comma-separated values — a plain-text table with one row per line). The *before* builds a universal exporter with five formats, encodings and compression — four formats and compression throw `UnsupportedOperationException`. The *after* is one method.

```java
// File: lld/ch05/yagni/Main.java
package lld.ch05.yagni;

import java.math.BigDecimal;
import java.util.List;
import java.util.stream.Collectors;

/** YAGNI: build what the current requirement needs ("export orders as CSV"), not what might be asked one day. */
public class Main {

    record Order(String id, String customer, BigDecimal total) {}   // money as BigDecimal, never double

    // ---------------- BEFORE: speculative generality ----------------
    static class Before {
        enum Format { CSV, PDF, XML, EXCEL, YAML }

        /** Five formats, pluggable encodings, compression... only CSV was requested. */
        interface ReportExporter {
            String export(List<Order> orders, Format format, String encoding, boolean compress);
        }

        static class UniversalExporter implements ReportExporter {
            @Override
            public String export(List<Order> orders, Format format, String encoding, boolean compress) {
                if (compress) throw new UnsupportedOperationException("compression not implemented yet");
                return switch (format) {
                    case CSV -> orders.stream().map(o -> o.id() + "," + o.customer() + "," +
o.total().toPlainString())
                            .collect(Collectors.joining("\n", "id,customer,total\n", ""));
                    case PDF, XML, EXCEL, YAML -> throw new UnsupportedOperationException(format + " not
implemented yet");
                };
            }
        }
    }

    // ---------------- AFTER: exactly what is needed today ----------------
    static class After {
        static String toCsv(List<Order> orders) {
            return orders.stream()
                    .map(o -> String.join(",", o.id(), o.customer(), o.total().toPlainString()))
                    .collect(Collectors.joining("\n", "id,customer,total\n", ""));
        }
    }

    public static void main(String[] args) {
        List<Order> orders = List.of(new Order("o1", "asha", new BigDecimal("250.00")),
                                     new Order("o2", "ravi", new BigDecimal("99.50")));
        System.out.println("== BEFORE ==");
        var exporter = new Before.UniversalExporter();
        System.out.println(exporter.export(orders, Before.Format.CSV, "UTF-8", false));
        try {
            exporter.export(orders, Before.Format.PDF, "UTF-8", false);
        } catch (UnsupportedOperationException e) {
            System.out.println("dead option: " + e.getMessage());
        }
        System.out.println("== AFTER ==");
        System.out.println(After.toCsv(orders));
    }
}
```

Output:

```
== BEFORE ==
id,customer,total
o1,asha,250.00
o2,ravi,99.50
dead option: PDF not implemented yet
== AFTER ==
id,customer,total
o1,asha,250.00
o2,ravi,99.50
```

Every caller of the *before* API (Application Programming Interface — the methods and types that other code calls) must pass three meaningless arguments, and every reader wonders which formats really work.

Trade-offs

YAGNI	Build ahead
Less code, faster delivery	Might save time if the guess is right
Design fits real needs	Guesses are usually wrong in detail
Needs good tests + refactoring discipline	Leaves dead options that confuse readers

When not to use it

- **Hard-to-change decisions** — public API formats, database schemas (the structure of tables and columns) with huge data, security, data retention laws. Think ahead there because changing later is very expensive.
- Architectural qualities that are known requirements (e.g. "must scale to 10k requests/s" — a *throughput* target, i.e. how much work per second the system must handle) are not speculation; they are requirements.

Common mistakes

- Using YAGNI as an excuse for messy code ("we'll never need clean code").
- Ignoring a requirement the interviewer *did* mention because "YAGNI".
- Adding `interface` + `Impl` pairs for every class "in case we swap it" — only add the interface when there is a second implementation or a test seam need (a *test seam* is a place where a test can swap in a fake collaborator).

Real-world examples

- **Minimum Viable Product (MVP)** thinking in startups is YAGNI at the product level.
- **Feature flags** (on/off switches in configuration that enable a code path without redeploying; e.g. LaunchDarkly, Unleash) let teams ship only what is needed now and turn on more later, rather than building every variant up front.
- XP teams (and many agile teams) plan per iteration so that only near-term work is built.

In the interview

30-second answer: "YAGNI means I build what the requirements need today and keep the design easy to extend. In an LLD (low-level design — designing the classes, interfaces and their interactions) interview I'll mention an extension point — say a `PaymentStrategy` interface — only if the interviewer hinted at multiple variants; otherwise I note it verbally as a possible extension."

Follow-ups:

- *Q: Doesn't YAGNI conflict with OCP?* A: Not really. OCP (the Open/Closed Principle from Chapter 4: open for extension, closed for modification) is applied where variation is known or likely; YAGNI stops you from inventing variation that isn't there.
- *Q: What makes YAGNI safe?* A: A good test suite (the full set of automated tests for the code) and clean code, so adding the feature later is cheap.

Self-check

1. Q: Name the smell YAGNI prevents. → A: Speculative generality.
2. Q: When should you design ahead anyway? → A: For expensive-to-reverse decisions (public APIs, schemas, security).

TL;DR

Build for today's requirements with clean, changeable code; tomorrow's features get built tomorrow.

---

4. Law of Demeter (Principle of Least Knowledge)

What it is

The **Law of Demeter (LoD)**, also called the **Principle of Least Knowledge**, says a method should only talk to its **immediate friends**, not to strangers it reaches through other objects. Formally, a method `m` of object `O` should only call methods on:

1. `O` itself (`this`);
2. objects passed to `m` as parameters;
3. objects `m` creates;
4. `O`'s own fields (its direct components).

(The original formulation by Ian Holland and Karl Lieberherr at Northeastern University, 1987, also allows global objects accessible to `O`; modern summaries usually list the four above.)

The famous slogan: **"use only one dot"** — e.g. `a.getB().getC().doIt()` is a **train wreck** (a chain of calls that looks like connected railway cars).

Analogy: when a paperboy collects money, he asks the customer to pay. He doesn't reach into the customer's pocket, pull out the wallet and take the cash himself.

The problem it solves

A train wreck couples the caller to the **whole chain's structure**. In `customer.getWallet().getBalance()`, the paperboy now depends on `Customer` *and* `Wallet`. If the customer starts paying by UPI (India's instant bank-to-bank mobile payment system) instead of a wallet, the paperboy breaks. Knowledge of internal structure leaks everywhere.

How it works

1. Spot a chain that navigates through other objects' internals.
2. Ask: *what does the caller really want?* ("get paid", not "get the wallet").
3. Add a method on the first object that does that job (`customer.pay(amount)`), delegating internally.
4. Keep the internal object private.



Java example — before/after

```java
// File: lld/ch05/demeter/Main.java
package lld.ch05.demeter;

import java.math.BigDecimal;
import java.util.List;

/** Law of Demeter ("only talk to your immediate friends"): the paperboy should not reach into the customer's
wallet. */
public class Main {

    // ---------------- BEFORE: a train wreck -- paperboy -> customer -> wallet -> balance ----------------
    static class Before {
        static class Wallet {
            private BigDecimal balance;
            Wallet(BigDecimal balance) { this.balance = balance; }
            BigDecimal getBalance() { return balance; }
            void setBalance(BigDecimal b) { balance = b; }
        }
        static class Customer {
            private final String name;
            private final Wallet wallet;
            Customer(String name, Wallet wallet) { this.name = name; this.wallet = wallet; }
            String getName() { return name; }
            Wallet getWallet() { return wallet; }
        }
        static class Paperboy {
            BigDecimal collected = BigDecimal.ZERO;
            void collect(Customer c, BigDecimal due) {
                // knows Customer has a Wallet, knows Wallet has a balance, changes it directly
                if (c.getWallet().getBalance().compareTo(due) >= 0) {
                    c.getWallet().setBalance(c.getWallet().getBalance().subtract(due));
                    collected = collected.add(due);
                    System.out.println("collected " + due + " from " + c.getName());
                } else {
                    System.out.println(c.getName() + " cannot pay");
                }
            }
        }
    }

    // ---------------- AFTER: the paperboy asks the customer (a direct friend) to pay ----------------
    static class After {
        static class Wallet {
            private BigDecimal balance;
            Wallet(BigDecimal balance) { this.balance = balance; }
            boolean take(BigDecimal amount) {
                if (amount.compareTo(balance) > 0) return false;
                balance = balance.subtract(amount);
                return true;
            }
        }
        static class Customer {
            private final String name;
            private final Wallet wallet;   // private detail -- could become a card or UPI tomorrow
            Customer(String name, Wallet wallet) { this.name = name; this.wallet = wallet; }
            String name() { return name; }
            /** Customer decides HOW to pay. */
            boolean pay(BigDecimal amount) { return wallet.take(amount); }
        }
        static class Paperboy {
            BigDecimal collected = BigDecimal.ZERO;
            void collect(Customer c, BigDecimal due) {
                if (c.pay(due)) {                       // talk only to a parameter: allowed
                    collected = collected.add(due);
                    System.out.println("collected " + due + " from " + c.name());
                } else {
                    System.out.println(c.name() + " cannot pay");
                }
            }
```

```
        }
    }

    public static void main(String[] args) {
        BigDecimal due = new BigDecimal("4.00");
        System.out.println("== BEFORE ==");
        var boy1 = new Before.Paperboy();
        boy1.collect(new Before.Customer("Meera", new Before.Wallet(new BigDecimal("10.00"))), due);
        boy1.collect(new Before.Customer("Kabir", new Before.Wallet(new BigDecimal("2.00"))), due);

        System.out.println("== AFTER ==");
        var boy2 = new After.Paperboy();
        boy2.collect(new After.Customer("Meera", new After.Wallet(new BigDecimal("10.00"))), due);
        boy2.collect(new After.Customer("Kabir", new After.Wallet(new BigDecimal("2.00"))), due);
        System.out.println("total collected = " + boy2.collected);

        // Chained calls on a fluent API / stream are NOT Demeter violations: each call returns the same kind of
thing.
        long longNames = List.of("Meera", "Kabir", "Jo").stream().filter(n -> n.length() > 2).count();
        System.out.println("stream chain is fine: " + longNames + " long names");
    }
}
```

Output:

```
== BEFORE ==
collected 4.00 from Meera
Kabir cannot pay
== AFTER ==
collected 4.00 from Meera
Kabir cannot pay
total collected = 4.00
stream chain is fine: 2 long names
```

What changed: in *before*, `Paperboy` knows three classes and even sets the balance directly. In *after*, `Paperboy` knows only `Customer.pay`, and `Customer` is free to change how it pays.

Trade-offs

Following LoD	Ignoring LoD
Changes to internals stay local	Structural changes ripple to callers
Easier mocking (mock = fake object used in tests) — one collaborator	Tests must build whole object graphs
May add small **wrapper/delegate methods**	No extra methods
Too many wrappers can bloat the middle class	—

When not to use it

- **Fluent interfaces and builders** (APIs whose methods return an object so calls can be chained; a builder assembles an object step by step — `StringBuilder.append(..).append(..)`, `stream().filter().map()`) return the same object or a sibling of the same abstraction — they are *not* violations.
- **Data structures / DTOs** (Data Transfer Objects — plain data with no behaviour, see Chapter 6) are meant to expose their data; `order.shippingAddress().city()` on a record is fine.
- Don't mechanically add dozens of pass-through methods — that is the **Middle Man** smell. Rethink responsibilities instead.

Common mistakes

- Counting dots instead of thinking about *knowledge*: splitting a chain into temp variables (`var w = c.getWallet(); w.getBalance()`) still violates the law.
- Adding getters (methods like `getWallet()` that just return a field) for everything, which invites train wrecks.

Real-world examples

- *Head First Design Patterns* presents it as "talk only to your immediate friends" in the Facade chapter — a **Facade** (one simple class placed in front of a group of complex classes) reduces how many classes a client talks to.
- **Spring Data** repositories (a *repository* is an object that loads and saves domain objects while hiding the database): services call `orderRepository.findByCustomerId(id)` rather than navigating `entityManager.getSession().getConnection()...`.
- `java.nio.file.Files.readString(path)` hides channels, buffers and decoders behind a single call.

In the interview

> **30-second answer:** "The Law of Demeter says a method should only call methods on itself, its parameters, objects it creates and its own fields. Long chains like `a.getB().getC()` couple the caller to internal structure. I fix them by adding a method that expresses intent — `customer.pay()` instead of reaching for the wallet."

Follow-ups:

- *Q: Is `list.stream().map(...).collect(...)` a violation?* A: No — each call returns a stream/collection abstraction meant for chaining (a Java *stream* is a pipeline that processes a sequence of elements step by step); you're not navigating an object's internals.
- *Q: What's the cost of following it strictly?* A: Wrapper methods can pile up (Middle Man). Balance by moving behaviour to where the data is (Tell-Don't-Ask).

Self-check

1. Q: List the four "friends" of a method. → A: `this`, parameters, objects it creates, its own fields.
2. Q: What is a train wreck? → A: A chain of getters navigating through several objects.
3. Q: Do records/DTOs have to obey it? → A: Not really; they are data structures meant to expose data.

TL;DR

Only talk to your friends. Replace `a.getB().getC().do()` with a method on `a` that says what you want.

---

5. Tell, Don't Ask

What it is

Tell, Don't Ask (TDA) says: instead of *asking* an object for its data and then making a decision for it, *tell* the object what you want and let it decide using its own data. Behaviour should live with the data it needs. The idea was popularised by Andy Hunt and Dave Thomas (the Pragmatic Programmers), who built on Alec Sharp's observation in *Smalltalk by Example* (1997) that procedural code (code organised as functions acting on separate data) fetches data and then decides, while object-oriented code tells objects to act.

Analogy: you tell a taxi driver "take me to the airport". You don't ask for the steering wheel angle and the speed every second and decide the turns yourself.

The problem it solves

When callers ask-then-act (`if (acc.getBalance() >= x) acc.setBalance(acc.getBalance() - x)`), the business rule is spread over every caller. Soon one caller forgets part of the rule (e.g. "frozen accounts can't withdraw") and the object's **invariant** breaks. The object becomes a passive data bag — the **Anemic Domain Model** anti-pattern.

How it works

1. Look for code that reads an object's fields, decides something, then writes back.
2. Move that decision into a method on the object (`withdraw(amount)`).

3. Make the object reject illegal requests (exceptions or result types).
4. Remove setters (methods like `setBalance(x)` that just overwrite a field) that are no longer needed.

Java example — before/after

```
// File: lld/ch05/telldontask/Main.java
package lld.ch05.telldontask;

/** Tell, Don't Ask: tell an object what you want; don't pull its data out, decide, and push data back in. */
public class Main {

    // ---------------- BEFORE: callers ask for state and make the decision themselves ----------------
    static class Before {
        static class Account {
            private long balancePaise;
            private boolean frozen;
            Account(long balance) { this.balancePaise = balance; }
            long getBalance() { return balancePaise; }
            void setBalance(long b) { balancePaise = b; }
            boolean isFrozen() { return frozen; }
            void setFrozen(boolean f) { frozen = f; }
        }
        // ATM remembered the "frozen" rule...
        static void atmWithdraw(Account a, long amt) {
            if (!a.isFrozen() && a.getBalance() >= amt) a.setBalance(a.getBalance() - amt);
        }
        // ...the mobile app forgot it. The business rule is scattered and now inconsistent.
        static void appWithdraw(Account a, long amt) {
            if (a.getBalance() >= amt) a.setBalance(a.getBalance() - amt);
        }
    }

    // ---------------- AFTER: the rule lives with the data it protects ----------------
    static class After {
        static final class Account {
            private long balancePaise;
            private boolean frozen;
            Account(long balance) { this.balancePaise = balance; }
            void freeze() { frozen = true; }
            void withdraw(long amt) {
                if (amt <= 0) throw new IllegalArgumentException("amount must be positive");
                if (frozen) throw new IllegalStateException("account is frozen");
                if (amt > balancePaise) throw new IllegalStateException("insufficient funds");
                balancePaise -= amt;
            }
            long balance() { return balancePaise; }   // queries are fine for display -- just don't decide outside
        }
    }

    public static void main(String[] args) {
        System.out.println("== BEFORE ==");
        var a = new Before.Account(1_000);
        a.setFrozen(true);
        Before.atmWithdraw(a, 300);
        System.out.println("after ATM on frozen account: " + a.getBalance());
        Before.appWithdraw(a, 300);
        System.out.println("after APP on frozen account: " + a.getBalance() + "   <-- rule bypassed");

        System.out.println("== AFTER ==");
        var b = new After.Account(1_000);
        b.withdraw(300);
        System.out.println("withdrew 300, balance " + b.balance());
        b.freeze();
        try { b.withdraw(300); } catch (IllegalStateException e) { System.out.println("rejected: " +
e.getMessage()); }
        System.out.println("balance still " + b.balance());
    }
}
```

Output:

```
== BEFORE ==
after ATM on frozen account: 1000
after APP on frozen account: 700   <-- rule bypassed
== AFTER ==
withdrew 300, balance 700
rejected: account is frozen
balance still 700
```

In *before*, the ATM and the app each reimplement the rule and the app forgets `frozen`. In *after*, `withdraw` is the only way to take money, so the rule cannot be bypassed.

Trade-offs

Tell	Ask
Invariants protected in one place	Rules duplicated across callers
Richer, more expressive objects	Simple data objects
Risk: object grows too many responsibilities	Fine for pure data (DTOs, reports)

When not to use it

- For **queries whose purpose is display or reporting** — asking for a balance to show it on screen is fine. The rule is about *deciding* outside.
- When the decision genuinely belongs to another object that combines data from several sources (then that other object is the Information Expert — see GRASP).

Common mistakes

- Adding a getter "just for this one check" — that is where the rule starts leaking.
- Pushing *every* behaviour into the entity, including emails and database calls (breaks SRP; use services for cross-cutting work).

Real-world examples

- `java.util.List.removeIf(predicate)` — you tell the list what to remove instead of iterating and removing yourself.
- `Map.computeIfAbsent`, `Map.merge` — tell the map the update you want; it handles the check-then-act.
- `AtomicInteger.incrementAndGet()` — the object performs the read-modify-write atomically (as one indivisible step that no other thread can interrupt); asking then setting would be a **race condition** (a bug where the result depends on the unlucky timing of two *threads* — independent paths of execution running at the same time — touching the same data).

In the interview

30-second answer: "Tell, Don't Ask means sending commands to the object that owns the data, e.g. `account.withdraw(100)`, rather than reading its balance and setting a new one. It keeps invariants in one place and avoids anemic models."

Follow-ups:

- *Q: How is it related to the Law of Demeter?* A: Both push behaviour to where the data is; LoD focuses on *who* you talk to, TDA on *what* you ask for.
- *Q: Concurrency angle?* A: Ask-then-act across two calls is a check-then-act race; a single `withdraw` method can be made atomic.

Self-check

1. Q: What anti-pattern does TDA fight? → A: The Anemic Domain Model (data bags with logic outside).
2. Q: Are getters forbidden? → A: No; they're fine for display. Avoid deciding outside the object.

TL;DR

Don't pull data out to make the object's decisions — tell the object what you want.

---

6. Composition over Inheritance

What it is

Favour object composition over class inheritance (from the GoF book). *Inheritance* ("is-a") reuses code by making a subclass of another class. *Composition* ("has-a") reuses code by holding a reference to another object and **delegating** (forwarding) calls to it.

Analogy: a car *has* an engine. You can swap a petrol engine for an electric one. If "car" *inherited from* "petrol engine", you could never swap it.

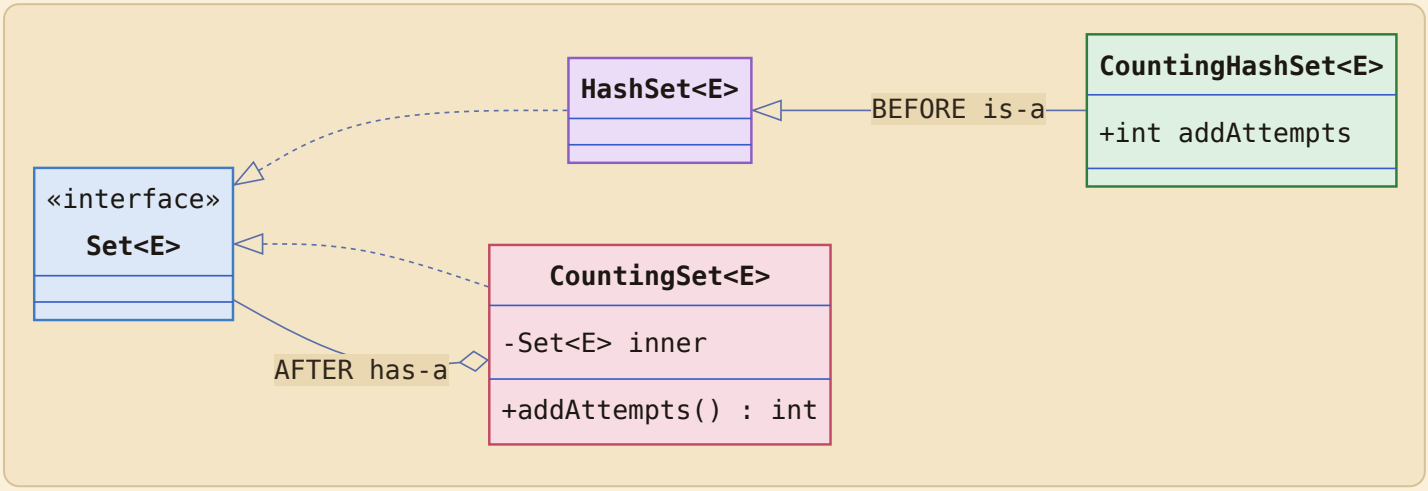
The problem it solves

Inheritance is the tightest coupling in object-oriented programming:

- **Fragile base class problem** — a subclass can break when the parent's *internal* implementation changes, even if the parent's public API is unchanged. The subclass depends on details like "does `addAll` call `add`?".
- **Fixed at compile time** (decided when the code is compiled, before it ever runs) — you cannot change the parent at runtime.
- **Class explosion** — combining features by subclassing (`LoggingCachingBufferedStream`) multiplies classes.
- It exposes all parent methods, even those that make no sense for the child.

How it works

1. Instead of `class B extends A`, give `B` a private field of type `A` (preferably an interface type).
2. Implement `B`'s methods by adding your behaviour and **forwarding** to the wrapped object.
3. Receive the wrapped object through the constructor, so any implementation can be plugged in.
4. Keep inheritance for genuine "is-a" relationships where the subclass truly is substitutable (Liskov Substitution, Chapter 4) and the parent was **designed for extension** (documented hooks, often abstract).



Java example — before/after

We want to count how many elements were ever added to a set. This is the classic trap described in *Effective Java*.

```java
// File: lld/ch05/composition/Main.java
package lld.ch05.composition;

import java.util.Collection;
import java.util.HashSet;
import java.util.Iterator;
import java.util.List;
import java.util.Set;
import java.util.AbstractSet;

/** Composition over inheritance: extending a concrete class couples you to its private implementation details. */
public class Main {

    // ---------------- BEFORE: inherit from HashSet to count insert attempts ----------------
    static class Before {
        @SuppressWarnings("serial")
        static class CountingHashSet<E> extends HashSet<E> {
            int addAttempts;
            @Override public boolean add(E e) { addAttempts++; return super.add(e); }
            @Override public boolean addAll(Collection<? extends E> c) {
                addAttempts += c.size();
                return super.addAll(c);   // HashSet's inherited addAll calls add() for each element -> counted
twice!
            }
        }
    }

    // ---------------- AFTER: wrap ANY Set and forward calls (a decorator) ----------------
    static class After {
        static final class CountingSet<E> extends AbstractSet<E> {
            private final Set<E> inner;   // "has-a", not "is-a"
            private int addAttempts;
            CountingSet(Set<E> inner) { this.inner = inner; }
            @Override public boolean add(E e) { addAttempts++; return inner.add(e); }
            @Override public boolean addAll(Collection<? extends E> c) { addAttempts += c.size(); return
inner.addAll(c); }
            @Override public boolean contains(Object o) { return inner.contains(o); }   // keep the inner set's
O(1)/O(log n) lookup
            @Override public boolean remove(Object o) { return inner.remove(o); }
            @Override public Iterator<E> iterator() { return inner.iterator(); }
            @Override public int size() { return inner.size(); }
            int addAttempts() { return addAttempts; }
        }
    }

    public static void main(String[] args) {
        List<String> words = List.of("red", "green", "blue");

        var before = new Before.CountingHashSet<String>();
        before.addAll(words);
        System.out.println("BEFORE: added 3 items, counter says " + before.addAttempts);

        var after = new After.CountingSet<String>(new HashSet<>());
        after.addAll(words);
        System.out.println("AFTER : added 3 items, counter says " + after.addAttempts());

        // Bonus: composition works with any Set implementation, chosen at runtime.
        var sorted = new After.CountingSet<String>(new java.util.TreeSet<>());
        sorted.addAll(words);
        System.out.println("AFTER over a TreeSet: " + sorted + " attempts=" + sorted.addAttempts());
    }
}
```

Output:

```
BEFORE: added 3 items, counter says 6
AFTER : added 3 items, counter says 3
AFTER over a TreeSet: [blue, green, red] attempts=3
```

Why the *before* prints 6: `HashSet` does not override `addAll`; it inherits `AbstractCollection.addAll`, which loops and calls `add` for every element. Our `addAll` adds 3, then the parent's loop calls *our overridden* `add` three more times. We depended on an implementation detail ("self-use") that `HashSet` does not promise to keep — the fragile base class problem. The *after* wraps any `Set` and forwards; the inner set's internal calls never reach our counter. It also forwards `contains` and `remove`: `AbstractSet` would otherwise implement them by scanning the iterator (an object that walks through a collection one element at a time) — O(n), i.e. time grows in step with the number of elements *n* (*Big-O notation*) — instead of using the inner set's fast lookup. It also works with a `TreeSet` chosen at runtime.

Trade-offs

Composition	Inheritance
Loose coupling to the public API only	Coupled to parent internals
Swap behaviour at runtime	Fixed at compile time
Combine behaviours freely (decorators)	Class explosion for combinations
More boilerplate (forwarding methods)	Free reuse of all parent methods
No automatic polymorphism (one variable working with many types) — need an interface	Subclass is automatically a subtype

When not to use it

- True "is-a" hierarchies with a base class **designed for extension**: e.g. extending `AbstractList` (implement `get` and `size`) or a Template Method base (see Hollywood Principle below).
- Sealed type hierarchies (a `sealed` type lists exactly which subtypes may exist) modelling a closed set of variants (`sealed interface Shape permits Circle, Square`).
- When you control both classes and they are in the same package and evolve together.

Common mistakes

- Inheriting just to reuse a few helper methods ("`OrderService extends BaseDao`").
- Composition without an interface — then you can't substitute implementations.
- Exposing the wrapped object with a getter, defeating encapsulation.

Real-world examples

- **JDK mistakes admitted by its own authors:** `java.util.Stack extends Vector` (so a stack lets you insert in the middle) and `java.util.Properties extends Hashtable` (so you can put non-String values). *Effective Java* cites both.
- **`java.io` streams** — `new BufferedInputStream(new FileInputStream(f))` composes behaviour (Decorator pattern, Chapter 9).
- **`Collections.unmodifiableList(list)` / `synchronizedList(list)`** wrap a list instead of subclassing each list type.
- **Game engines (Unity)** use an Entity–Component model: a game object *has* components (physics, renderer) instead of deep inheritance trees.

In the interview

> **30-second answer:** "Inheritance couples a subclass to its parent's implementation, is fixed at compile time and explodes combinatorially. Composition holds a reference to an interface and delegates, so behaviour can be swapped at runtime. I use inheritance only for true is-a relationships with classes designed for extension."

Follow-ups:

- *Q: Which patterns are built on composition?* A: Strategy, Decorator, Composite, State, Bridge, Proxy, Adapter (object form) — most GoF patterns.

- *Q: Give a concrete inheritance failure.* A: The counting `HashSet` above, or `Stack extends Vector` exposing `insertElementAt`.
- *Q: When is inheritance OK?* A: When substitutable (LSP holds) and the parent documents its self-use or provides abstract hooks.

Self-check

1. Q: What is the fragile base class problem? → A: Subclasses breaking because the parent's internal implementation changed.
2. Q: What does "forwarding" mean? → A: The wrapper's method calls the same method on the wrapped object.
3. Q: Why did `CountingHashSet` count 6? → A: The inherited `addAll` calls the overridden `add`.

TL;DR

Reuse by holding and delegating ("has-a"); reserve "is-a" for real substitutable subtypes designed for extension.

---

7. Program to an Interface, Not an Implementation

What it is

Declare variables, fields, parameters and return types using the **most general supertype** (a parent type higher up the hierarchy) that offers what you need (an interface or abstract class), not a concrete class. "Interface" here means *supertype*, not only the Java `interface` keyword.

Analogy: a phone charger is designed for the USB-C *standard*, not for one specific phone model. Any USB-C phone works.

The problem it solves

Code written against a concrete class (`ArrayList`, `MySqlUserStore`) can only ever work with that class. You cannot test it without the real thing (e.g. a MySQL server), cannot swap implementations and cannot reuse it with other types. Changing the implementation means editing every caller.

How it works

1. Identify the **role** a collaborator plays ("stores users", "a collection of names").
2. Express the role as an interface with only the needed methods (`UserRepository.findById`).
3. Type fields/parameters by that interface (`List<String>`, `Collection<String>`, `UserRepository`).
4. Create concrete objects only at the edges — in `main`, a factory (an object or method whose job is creating other objects), or a DI container (a framework component, such as Spring's, that builds objects and wires their dependencies automatically) — and pass them in (**Dependency Injection**: giving an object its collaborators from outside instead of letting it `new` them).
5. For parameters, accept the widest type you can (`Collection` beats `List` beats `ArrayList`); for return types, return an interface but a specific enough one to be useful.

Java example — before/after

```java
// File: lld/ch05/interfaces/Main.java
package lld.ch05.interfaces;

import java.util.ArrayList;
import java.util.Collection;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.TreeSet;

/** Program to an interface (a supertype), not an implementation. */
public class Main {

    record User(String id, String name) {}

    // ---------------- BEFORE: every signature names a concrete class ----------------
    static class Before {
        /** Pretend this talks to a real MySQL server -- impossible to use in a unit test. */
        static class MySqlUserStore {
            User load(String id) { throw new IllegalStateException("no MySQL server on this laptop"); }
        }
        static class GreetingService {
            private final MySqlUserStore store = new MySqlUserStore();   // hard-wired concrete dependency
            String greet(String id) { return "Hello, " + store.load(id).name(); }
        }
        static int countLongNames(ArrayList<String> names) {         // only accepts ArrayList
            int n = 0;
            for (String s : names) if (s.length() > 4) n++;
            return n;
        }
    }

    // ---------------- AFTER: depend on the supertype; pick the implementation at the edge ----------------
    static class After {
        interface UserRepository { Optional<User> findById(String id); }

        static final class InMemoryUserRepository implements UserRepository {
            private final Map<String, User> data = new HashMap<>();
            InMemoryUserRepository save(User u) { data.put(u.id(), u); return this; }
            @Override public Optional<User> findById(String id) { return Optional.ofNullable(data.get(id)); }
        }

        static final class GreetingService {
            private final UserRepository users;                  // any implementation
            GreetingService(UserRepository users) { this.users = users; }
            String greet(String id) {
                return users.findById(id).map(u -> "Hello, " + u.name()).orElse("Hello, stranger");
            }
        }

        static int countLongNames(Collection<String> names) {   // works for List, Set, Queue...
            return (int) names.stream().filter(s -> s.length() > 4).count();
        }
    }

    public static void main(String[] args) {
        System.out.println("== BEFORE ==");
        try {
            System.out.println(new Before.GreetingService().greet("u1"));
        } catch (IllegalStateException e) {
            System.out.println("cannot even run it: " + e.getMessage());
        }
        System.out.println("long names in ArrayList: " + Before.countLongNames(new ArrayList<>(List.of("Anand",
"Li", "Priya"))));
        // Before.countLongNames(new TreeSet<>(...)) would not compile.

        System.out.println("== AFTER ==");
        After.UserRepository repo = new After.InMemoryUserRepository().save(new User("u1", "Asha"));
```

```
        var service = new After.GreetingService(repo);
        System.out.println(service.greet("u1"));
        System.out.println(service.greet("u2"));
        System.out.println("long names in List:    " + After.countLongNames(List.of("Anand", "Li", "Priya")));
        System.out.println("long names in TreeSet: " + After.countLongNames(new TreeSet<>(List.of("Anand", "Li",
"Priya"))));
    }
}
```

Output:

```
== BEFORE ==
cannot even run it: no MySQL server on this laptop
long names in ArrayList: 2
== AFTER ==
Hello, Asha
Hello, stranger
long names in List:    2
long names in TreeSet: 2
```

The *before* service creates its own `MySqlUserStore`, so it cannot even run on a laptop, and `countLongNames` accepts only `ArrayList`. The *after* depends on `UserRepository` and `Collection`; an in-memory implementation makes it runnable and testable, and any collection works.

Trade-offs

Interface-typed	Concrete-typed
Swappable, testable (fakes/mocks)	Simple, one less type
Callers can't use implementation-only methods	Access to everything (e.g. `ArrayList.trimToSize`)
Extra interface to maintain	Changes ripple to callers

When not to use it

- **Value types / data classes** (`String`, `LocalDate`, records) — no need for an interface.
- A class with exactly one implementation and no testing seam need — creating `FooService` + `FooServiceImpl` for everything is ceremony (see YAGNI). Add the interface when the second implementation or test double (any stand-in object used in tests: fake, stub or mock) appears.
- When you truly need implementation-specific behaviour (e.g. `LinkedHashMap` access order for an LRU cache (a *cache* keeps recently used data close at hand for fast reuse; LRU = evict the Least Recently Used entry when full) — then declare it as such).

Common mistakes

- Interfaces that mirror a class one-to-one with 30 methods (violates Interface Segregation, Chapter 4).
- Declaring `List` but then casting to `ArrayList` inside.
- Still calling `new ConcreteThing()` inside the class that uses the interface — the dependency is typed abstractly but still hard-wired.

Real-world examples

- **Java Collections Framework**: code uses `List`, `Map`, `Set`; `Collections.sort(List)` works on any list.
- **JDBC** (Java Database Connectivity — the standard Java API for talking to SQL databases): your code uses `java.sql.Connection`, `PreparedStatement`, `ResultSet` interfaces; the driver (MySQL, PostgreSQL) supplies the implementation.
- **SLF4J** (a logging *facade*: one logging API in front of many logging libraries): code logs through the `org.slf4j.Logger` interface; Logback or Log4j2 is plugged in at deployment.
- **Spring** injects beans (objects created and managed by the Spring container) by interface type (`@Autowired PaymentGateway gateway`).

In the interview

> **30-second answer:** "I type dependencies by roles — interfaces — and inject concrete implementations from the outside. It lets me swap implementations, e.g. an in-memory repository for tests, and keeps callers stable when implementations change. It's the practical form of the Dependency Inversion Principle."

Follow-ups:

- *Q: Relationship to DIP?* A: DIP says high-level modules depend on abstractions; programming to an interface is how you do it at each variable/field.
- *Q: Should every class have an interface?* A: No — only roles with multiple implementations or a need for test doubles.

Self-check

1. Q: What does "interface" mean in this principle? → A: Any supertype (interface or abstract class).
2. Q: Where should `new ConcreteClass()` appear? → A: At the edges: `main`, factories, DI configuration.

TL;DR

Depend on roles (`List`, `UserRepository`), not on concrete classes; choose the concrete class once, at the edge.

---

8. Encapsulate What Varies

What it is

Find the parts of your application that **change** (or differ between cases) and **separate them** from the parts that stay the same. Put the varying part behind its own abstraction so it can change without touching stable code. *Head First Design Patterns* calls this the first design principle; the GoF book phrases it as "encapsulate the concept that varies".

Analogy: a camera body stays the same while you swap lenses. The lens mount is the stable interface; lenses are the varying part.

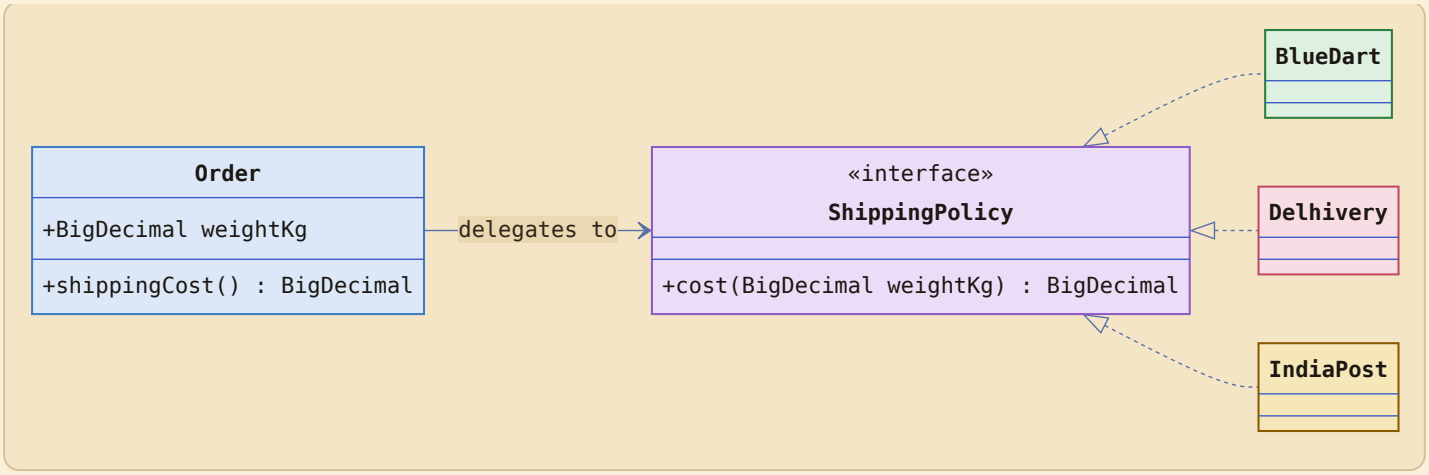
The problem it solves

When a volatile rule (shipping prices, tax, discounts, file formats) is written inline in a stable class, every change to the rule edits that class, re-tests it and risks breaking unrelated behaviour. Long `if/else` or `switch` chains on a "type" string grow forever (violating the Open/Closed Principle).

How it works

1. Ask: "Which parts of this class changed in the last few releases, or differ per customer/region/type?"
2. Extract that part behind an interface (or a functional interface — an interface with exactly one abstract method, which a lambda can implement).
3. Give the stable class a field of that interface type and delegate to it.
4. Add new variants as new implementations; the stable class does not change.

Almost every design pattern (a named, reusable solution to a recurring design problem; Chapters 8–11) is an application of this principle: Strategy (varying algorithm), State (varying state-dependent behaviour), Factory (varying creation), Observer (varying set of listeners), Template Method (varying steps).


Order
+BigDecimal weightKg
+shippingCost() : BigDecimal
delegates to
«interface»
ShippingPolicy
+cost(BigDecimal weightKg) : BigDecimal
BlueDart
Delhivery
IndiaPost

Java example — before/after

```java
// File: lld/ch05/encapsulatevaries/Main.java
package lld.ch05.encapsulatevaries;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.Map;

/** Encapsulate what varies: find the part that keeps changing (shipping rules) and put it behind its own
abstraction. */
public class Main {

    /** Rupee amount = base + perKg * weight, exact decimal arithmetic (money is never a double). */
    static BigDecimal linear(String base, String perKg, BigDecimal weightKg) {
        return new BigDecimal(base).add(new BigDecimal(perKg).multiply(weightKg)).setScale(2,
RoundingMode.HALF_UP);
    }

    // ---------------- BEFORE: the changing part is tangled into a stable class ----------------
    static class Before {
        static class Order {
            final BigDecimal weightKg;
            final String carrier;
            Order(BigDecimal weightKg, String carrier) { this.weightKg = weightKg; this.carrier = carrier; }

            BigDecimal shippingCost() {
                // every new carrier or price change edits Order -- a class that should be stable
                if (carrier.equals("BLUEDART")) return linear("50", "20", weightKg);
                else if (carrier.equals("DELHIVERY")) return linear("40", "25", weightKg);
                else if (carrier.equals("INDIA_POST")) return linear("30", "10", weightKg);
                throw new IllegalArgumentException("unknown carrier " + carrier);
            }
        }
    }

    // ---------------- AFTER: the varying part is a separate, swappable object ----------------
    static class After {
        @FunctionalInterface
        interface ShippingPolicy { BigDecimal cost(BigDecimal weightKg); }

        /** Stable catalogue of today's policies. Adding one = adding one entry; Order never changes. */
        static final Map<String, ShippingPolicy> POLICIES = Map.of(
                "BLUEDART", w -> linear("50", "20", w),
                "DELHIVERY", w -> linear("40", "25", w),
                "INDIA_POST", w -> linear("30", "10", w));

        record Order(BigDecimal weightKg, ShippingPolicy shipping) {
            BigDecimal shippingCost() { return shipping.cost(weightKg); }
        }
    }

    public static void main(String[] args) {
        BigDecimal twoKg = new BigDecimal("2");
        System.out.println("== BEFORE ==");
        for (String c : new String[]{"BLUEDART", "DELHIVERY", "INDIA_POST"})
            System.out.printf("%-10s %s%n", c, new Before.Order(twoKg, c).shippingCost());

        System.out.println("== AFTER ==");
        for (String c : new String[]{"BLUEDART", "DELHIVERY", "INDIA_POST"})
            System.out.printf("%-10s %s%n", c, new After.Order(twoKg, After.POLICIES.get(c)).shippingCost());
        // A brand new flat-rate promo, with zero edits to Order:
        System.out.printf("%-10s %s%n", "PROMO", new After.Order(twoKg, w -> new
BigDecimal("0.00")).shippingCost());
    }
}
```

Output:

```
== BEFORE ==
BLUEDART   90.00
DELHIVERY  90.00
INDIA_POST 50.00
== AFTER ==
BLUEDART   90.00
DELHIVERY  90.00
INDIA_POST 50.00
PROMO      0.00
```

In *after*, `Order` never changes when a carrier is added or re-priced; a brand-new policy (the promo) was added with zero edits to `Order`. The diagram draws named classes for clarity; the code uses lambdas implementing the same interface.

Trade-offs

Encapsulated	Inline
New variants without touching stable code	Everything in one place, easy to see
Each variant testable alone	Simple for 2 variants that never change
More types / indirection	Grows into long conditionals

When not to use it

When the "varying" thing has never varied and there is no sign it will (YAGNI). Two fixed branches are fine inline; extract when the third arrives or change becomes frequent.

Common mistakes

- Guessing the wrong axis of variation (extracting something that never changes while the real volatile part stays inline).
- Leaving the `switch` on type in the client code after introducing the strategy (just moving the conditional).

Real-world examples

- `java.util.Comparator` — sorting algorithm stable, comparison varies.
- `java.util.concurrent.ThreadPoolExecutor` takes a `RejectedExecutionHandler` (what to do when full varies) and a `ThreadFactory` (how threads are made varies).
- **Payment gateways** in e-commerce apps (Razorpay, Stripe, PayPal) behind one `PaymentGateway` interface.

In the interview

30-second answer: "I identify what is likely to change — say pricing or payment method — and hide it behind an interface so the core classes stay closed to modification. It's the principle behind Strategy, State and most other patterns."

Follow-ups:

- *Q: How do you know what varies?* A: From requirements ("multiple payment methods"), from history (what changed often), and from the interviewer's hints.
- *Q: Relation to OCP?* A: Encapsulating the variation is how you make a class open for extension and closed for modification.

Self-check

1. Q: Which pattern encapsulates a varying algorithm? → A: Strategy.
2. Q: What code smell signals a missing encapsulation? → A: A growing `if/else`/`switch` on a type code.

TL;DR

Separate what changes from what stays the same; put the changing part behind an interface.

---

9. Strive for Loosely Coupled Designs

What it is

Loose coupling means objects that interact know as little as possible about each other — ideally only a small interface. *Head First Design Patterns* states it as: strive for loosely coupled designs between objects that interact.

Analogy: a radio station broadcasts; it doesn't know who is listening or what brand of radio they own. Listeners tune in or out freely.

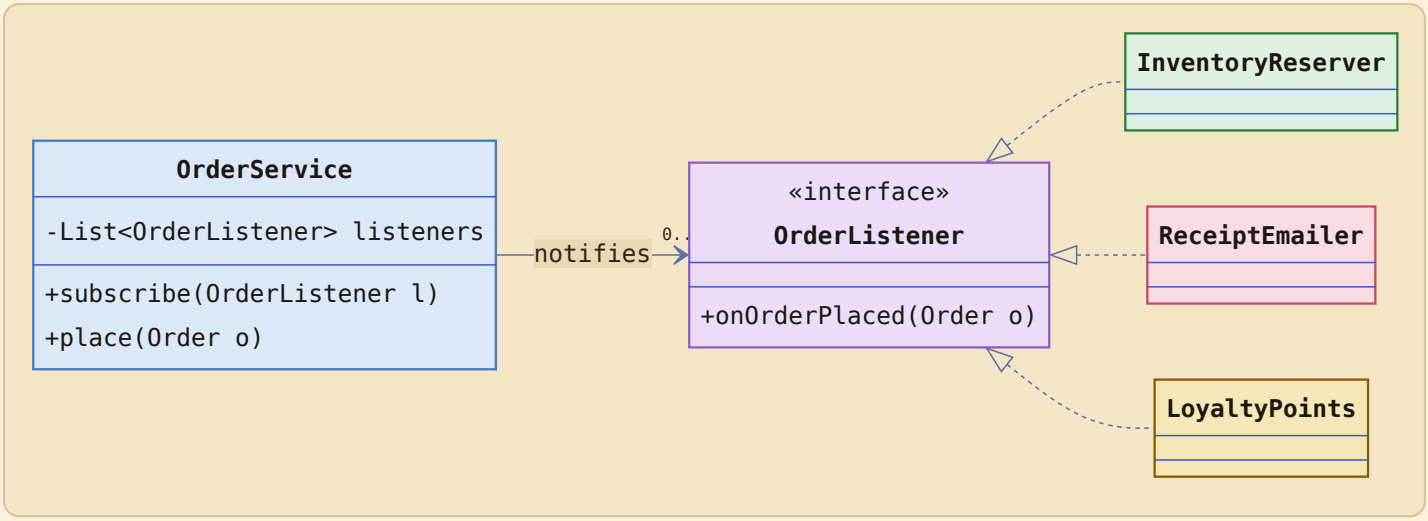
The problem it solves

In a **tightly coupled** design, a class creates and calls many concrete collaborators directly. Adding, removing or changing any collaborator edits that class. You can't reuse it without dragging all its dependencies along, and you can't test it in isolation.

How it works

1. Depend on **interfaces**, not concrete classes.
2. **Inject** dependencies instead of creating them inside.
3. Communicate through **events or messages** when the sender shouldn't know the receivers (Observer / publish-subscribe — senders publish events and any number of subscribers receive them without knowing each other).
4. Keep interfaces **narrow** (a single method is ideal for listeners).
5. Pass data as immutable values (records) rather than shared mutable objects.

Kinds of coupling from tightest to loosest (a common classification): *content* (reaching into another's internals) → *common* (shared global data) → *external* (sharing an externally imposed format, protocol or device interface) → *control* (passing flags that steer the other's logic) → *stamp* (passing a big structure when only part is needed) → *data* (passing only needed values) → *message* (communicating only through messages/events).



Java example — before/after

```java
// File: lld/ch05/loosecoupling/Main.java
package lld.ch05.loosecoupling;

import java.util.List;
import java.util.concurrent.CopyOnWriteArrayList;

/** Strive for loosely coupled designs between objects that interact. */
public class Main {

    record Order(String id, String email, int items) {}

    // ---------------- BEFORE: OrderService knows every class that cares about an order ----------------
    static class Before {
        static class EmailSender { void sendReceipt(Order o) { System.out.println("  email receipt to " +
o.email()); } }
        static class Inventory { void reserve(Order o) { System.out.println("  reserve " + o.items() + " items");
} }
        static class Analytics { void track(Order o) { System.out.println("  track order " + o.id()); } }

        static class OrderService {
            private final EmailSender email = new EmailSender();   // concrete, created inside
            private final Inventory inventory = new Inventory();
            private final Analytics analytics = new Analytics();
            void place(Order o) {
                System.out.println("placed " + o.id());
                inventory.reserve(o);
                email.sendReceipt(o);
                analytics.track(o);          // adding "loyalty points" means editing this class
            }
        }
    }

    // ---------------- AFTER: OrderService only knows a tiny listener interface ----------------
    static class After {
        @FunctionalInterface
        interface OrderListener { void onOrderPlaced(Order o); }

        static final class OrderService {
            private final List<OrderListener> listeners = new CopyOnWriteArrayList<>();
            void subscribe(OrderListener l) { listeners.add(l); }
            void place(Order o) {
                System.out.println("placed " + o.id());
                listeners.forEach(l -> l.onOrderPlaced(o));   // doesn't know who listens or what they do
            }
        }
    }

    public static void main(String[] args) {
        System.out.println("== BEFORE ==");
        new Before.OrderService().place(new Order("o-1", "asha@example.com", 2));

        System.out.println("== AFTER ==");
        var service = new After.OrderService();
        service.subscribe(o -> System.out.println("  reserve " + o.items() + " items"));
        service.subscribe(o -> System.out.println("  email receipt to " + o.email()));
        service.subscribe(o -> System.out.println("  track order " + o.id()));
        service.subscribe(o -> System.out.println("  +" + o.items() * 10 + " loyalty points"));  // new, no edits
        service.place(new Order("o-2", "ravi@example.com", 3));
    }
}
```

Output:

```
== BEFORE ==
placed o-1
  reserve 2 items
  email receipt to asha@example.com
  track order o-1
== AFTER ==
placed o-2
  reserve 3 items
  email receipt to ravi@example.com
  track order o-2
  +30 loyalty points
```

The *after* `OrderService` depends on one tiny interface. Adding "loyalty points" needed zero edits to it. A `CopyOnWriteArrayList` (a thread-safe list that copies its internal array on every write) is used so listeners can subscribe safely even while notifications run on other threads.

Trade-offs

Loose	Tight
Change and test parts independently	Easy to follow control flow
Add features by adding listeners	Direct calls are faster to trace in a debugger
Flow is less visible ("who handles this event?")	Every change edits the central class
Error handling/ordering between listeners must be designed	Order is explicit

When not to use it

- Inside a small, cohesive module where classes always change together — decoupling them adds ceremony.
- When the order and atomicity (all steps happen, or none do) of steps is critical (e.g. debit then credit in a transfer) — explicit orchestration is clearer and safer than fire-and-forget events.

Common mistakes

- "Decoupling" with a global event bus (one shared channel that every component publishes to and listens on) for everything, making the flow impossible to follow.
- Hidden coupling via shared mutable static state.
- Listeners that throw and stop the other listeners from running (decide a policy: catch and log per listener).

Real-world examples

- **Swing / JavaFX / Android** event listeners (`ActionListener`, `OnClickListener`).
- **Spring** `ApplicationEventPublisher` and `@EventListener`.
- **Apache Kafka** (a distributed *message broker*: producers append messages to named *topics*, and *consumer groups* read them at their own pace; see System Design Fundamentals) — producers write to topics without knowing which consumer groups read them; an order service at an e-commerce company publishes `OrderPlaced`, and inventory, notifications and analytics each consume independently.

In the interview

30-second answer: "Loose coupling means collaborators know only small interfaces about each other. I achieve it with interfaces, dependency injection and events. In the order example, the order service just publishes 'order placed' and new features subscribe without touching it."

Follow-ups:

- *Q: Coupling vs cohesion?* A: Coupling is between modules (want low); cohesion is within a module (want high).

- *Q: Downside of event-driven decoupling?* A: Harder to trace flows, ordering and failure handling; needs good logging and clear contracts.

Self-check

1. Q: Which pattern is the classic loose-coupling tool for one-to-many notifications? → A: Observer.
2. Q: Name two techniques for loose coupling. → A: Depend on interfaces; inject dependencies (also: events, narrow interfaces).

TL;DR

Let interacting objects share only small interfaces; inject them and use events so parts change independently.

---

10. The Hollywood Principle (Inversion of Control)

What it is

"Don't call us, we'll call you." High-level components (a framework or base class) control the flow and **call** low-level components at the right moments. Low-level components **hook into** the system but never call the high-level components directly. This reversal of who-calls-whom is called **Inversion of Control (IoC)**.

Analogy: actors audition and leave their number; the studio calls them when there's a role. Actors don't phone the studio every hour.

Library vs framework: you call a library; a framework calls you. That is IoC.

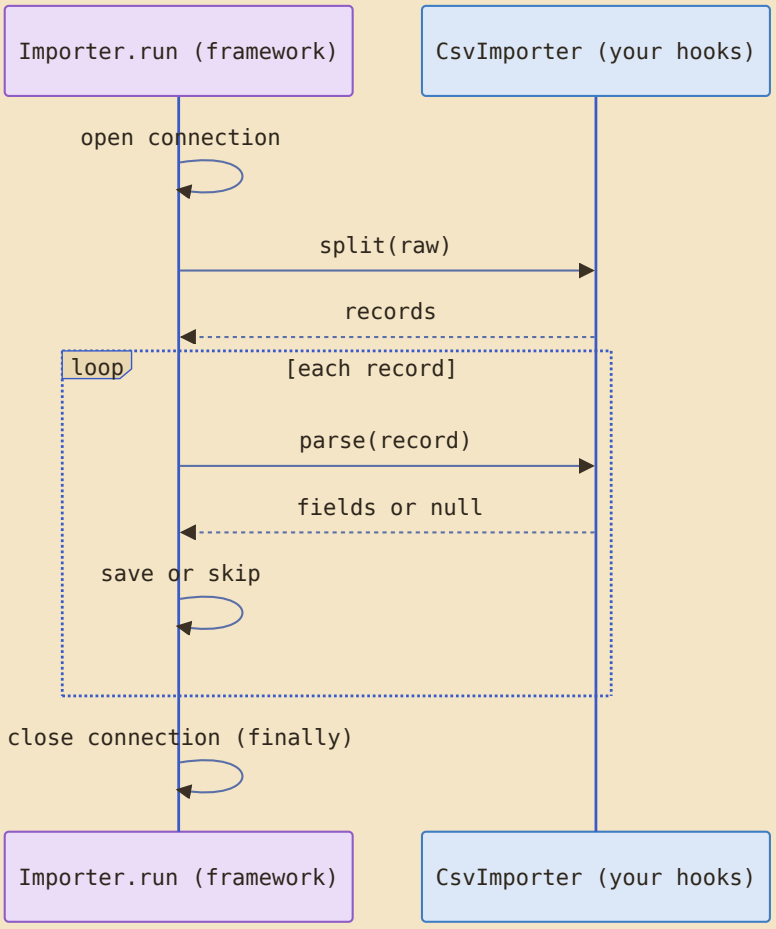
The problem it solves

Without it, every component drives the whole process itself: it re-implements the algorithm's skeleton, decides the order of steps and calls shared services directly. You get duplicated control flow, forgotten steps (like closing a database connection — the open session a program holds with a database) and **dependency rot** — high-level and low-level components depending on each other in tangled ways.

How it works

There are several ways to invert control:

1. **Template Method** — a base class defines a `final` algorithm and calls abstract "hook" methods that subclasses fill in.
2. **Callbacks / listeners** — you register a function; the system calls it on an event (Observer).
3. **Dependency Injection** — a container creates your objects and passes them their dependencies; your class never looks them up.
4. **Lifecycle management** — a container starts/stops your components (`@PostConstruct`, servlet `init`/`destroy`; a *servlet* is a Java class that handles web requests inside a server such as Tomcat).


Importer.run (framework)
CsvImporter (your hooks)
open connection
split(raw)
records
loop
[each record]
parse(record)
fields or null
save or skip
close connection (finally)
Importer.run (framework)
CsvImporter (your hooks)

Java example — before/after

```java
// File: lld/ch05/hollywood/Main.java
package lld.ch05.hollywood;

import java.util.ArrayList;
import java.util.List;

/** Hollywood Principle: "Don't call us, we'll call you." The high-level framework owns the flow and calls your
hooks. */
public class Main {

    // ---------------- BEFORE: each importer re-implements the whole flow and drives it itself ----------------
    static class Before {
        static class CsvImporter {
            void run(String raw) {
                System.out.println("  open connection");              // step order copied into every importer
                for (String row : raw.split(";")) {
                    String[] f = row.split(",");
                    if (f.length != 2) { System.out.println("  skip bad row '" + row + "'"); continue; }
                    System.out.println("  save " + f[0] + "=" + f[1]);
                }
                // forgot to close the connection -- nothing forced it
            }
        }
    }

    // ---------------- AFTER: Template Method -- the framework calls you ----------------
    static class After {
        abstract static class Importer {
            /** The fixed algorithm. final = subclasses cannot reorder or skip steps. */
            public final void run(String raw) {
                System.out.println("  open connection");
                try {
                    for (String record : split(raw)) {
                        String[] fields = parse(record);
                        if (fields == null) { System.out.println("  skip bad record '" + record + "'"); continue;
}
                        System.out.println("  save " + fields[0] + "=" + fields[1]);
                    }
                } finally {
                    System.out.println("  close connection");       // guaranteed by the framework
                }
            }
            protected abstract List<String> split(String raw);       // hooks: WE call YOU
            protected abstract String[] parse(String record);
        }

        static final class CsvImporter extends Importer {
            @Override protected List<String> split(String raw) { return List.of(raw.split(";")); }
            @Override protected String[] parse(String r) { String[] f = r.split(","); return f.length == 2 ? f :
null; }
        }
        static final class KeyValueImporter extends Importer {
            @Override protected List<String> split(String raw) { return List.of(raw.split("\n")); }
            @Override protected String[] parse(String r) { String[] f = r.split("="); return f.length == 2 ? f :
null; }
        }

        /** A tiny IoC "container": components register, the container decides when to start/stop them. */
        interface Lifecycle { void start(); void stop(); }
        static final class Container {
            private final List<Lifecycle> components = new ArrayList<>();
            void register(Lifecycle c) { components.add(c); }
            void boot() { components.forEach(Lifecycle::start); }
            void shutdown() { components.reversed().forEach(Lifecycle::stop); }  // reverse order, like Spring
        }
    }

    public static void main(String[] args) {
        System.out.println("== BEFORE ==");
```

```
        new Before.CsvImporter().run("a,1;b,2;oops");
        System.out.println("== AFTER ==");
        new After.CsvImporter().run("a,1;b,2;oops");
        new After.KeyValueImporter().run("x=9\ny=8");

        var container = new After.Container();
        for (String name : List.of("database", "cache", "web")) {
            container.register(new After.Lifecycle() {
                public void start() { System.out.println("  start " + name); }
                public void stop() { System.out.println("  stop " + name); }
            });
        }
        container.boot();
        container.shutdown();
    }
}
```

Output:

```
== BEFORE ==
  open connection
  save a=1
  save b=2
  skip bad row 'oops'
== AFTER ==
  open connection
  save a=1
  save b=2
  skip bad record 'oops'
  close connection
  open connection
  save x=9
  save y=8
  close connection
  start database
  start cache
  start web
  stop web
  stop cache
  stop database
```

The *before* importer drives the flow and forgot to close the connection. In *after*, `Importer.run` is `final`: subclasses only supply `split` and `parse`, and the framework guarantees the close. The tiny `Container` shows IoC for lifecycles: components never start themselves; the container starts them in order and stops them in reverse order (the same idea Spring uses when destroying singletons — objects of which only one instance exists in the application).

Trade-offs

IoC	Direct control
Framework guarantees flow, ordering, cleanup	You see the whole flow in your code
Less duplication across components	Duplicated skeletons
"Magic": harder to debug who calls what	Explicit, simple for tiny programs
Components tied to the framework's hooks	Independent

When not to use it

For small scripts or one-off logic where there is no repeated skeleton — a framework or template adds indirection with nothing to share.

Common mistakes

- Letting low-level hooks call back up into the framework (e.g. a subclass calling `run()` recursively).

- Forgetting `final` on the template method, so subclasses override the skeleton.
- Confusing IoC (the general idea) with DI (one specific form of it).

Real-world examples

- **JUnit** (the standard Java unit-testing framework) discovers and calls your `@Test` methods; you never call them.
- **Servlets**: the container calls `doGet` / `doPost` (handlers for HTTP GET and POST requests); **Spring MVC** calls your `@GetMapping` methods.
- **Spring IoC container** creates beans, injects dependencies and calls `@PostConstruct` / `@PreDestroy`.
- **Android** calls `Activity.onCreate`, `onResume`, `onDestroy`.
- `java.util.AbstractList` — you implement `get` and `size`; the base class implements iteration, `indexOf`, `equals` by calling your methods.
- **Spring** `JdbcTemplate` runs the connection/statement/cleanup skeleton and calls your `RowMapper`.

In the interview

> **30-second answer:** "The Hollywood Principle — 'don't call us, we'll call you' — means the high-level component controls the flow and calls low-level hooks. It's what separates a framework from a library. Template Method, callbacks and dependency injection are all ways to apply it, and it prevents low-level parts from depending on high-level ones."

Follow-ups:

- *Q: IoC vs DI?* A: IoC is the general reversal of control; DI is IoC applied to obtaining dependencies (they're given to you rather than looked up).
- *Q: Hollywood vs Dependency Inversion?* A: DIP is about depending on abstractions; Hollywood is about who calls whom. They often appear together.

Self-check

1. Q: In Template Method, who calls whom? → A: The base class's template method calls the subclass hooks.
2. Q: Why make the template method `final`? → A: So subclasses cannot reorder or skip steps.
3. Q: Library or framework: which one calls your code? → A: Framework.

TL;DR

Let the framework own the flow and call your hooks; low-level parts plug in and never call up.

---

11. Separation of Concerns (SoC)

What it is

A **concern** is one aspect of what a program does: reading input, business rules, storage, presentation, security, logging. **Separation of Concerns** means each part of the program deals with one concern, so you can think about, change and test it without thinking about the others. Edsger Dijkstra coined the phrase in his 1974 essay "On the role of scientific thought" (EWD 447).

Analogy: in a restaurant the waiter takes orders, the chef cooks, the cashier bills. If one person did all three at once, every change (a new menu, a new payment method) would disrupt everything.

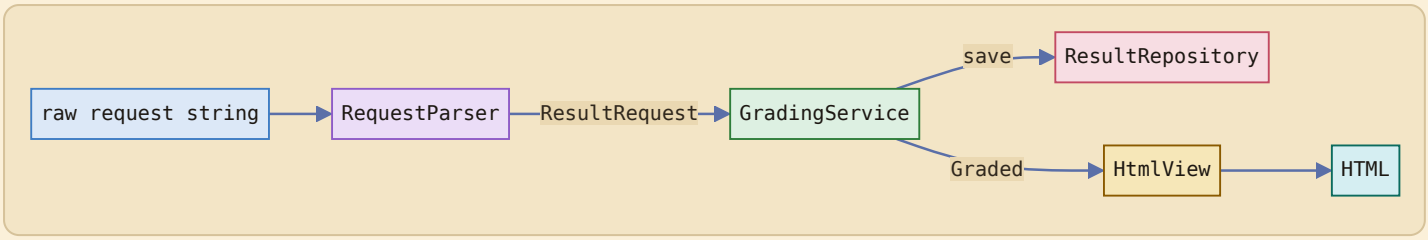
SoC is the **big-picture** version of the Single Responsibility Principle: SRP is about one class; SoC applies at every level — functions, classes, layers (MVC — Model–View–Controller, which keeps data/rules, display and input handling apart; controller/service/repository — a common backend layering where the controller receives web requests, the service holds business rules and the repository talks to the database), services and even languages (HTML for structure, CSS for style, JS for behaviour).

The problem it solves

Mixed concerns create code where you can't change the HTML without risking the grading rule, can't test the rule without parsing strings, and can't switch storage without editing presentation. Everything is tangled — *spaghetti code*.

How it works

1. List the concerns in the feature (input parsing, validation/rules, persistence, output format).
2. Give each its own unit (function → class → layer), with clear inputs and outputs (typed objects, not raw strings).
3. Let data flow in one direction: parse → apply rules → store → present.
4. Make lower layers not know about higher ones (the service doesn't know about HTML).
5. Handle **cross-cutting concerns** (logging, security, transactions — a *transaction* is a group of database changes that succeed or fail as one) with dedicated mechanisms — filters, decorators, proxies (stand-in objects that intercept calls to the real one) or aspects (AOP, aspect-oriented programming: write code such as logging once and have it applied around many methods automatically) — rather than sprinkling them everywhere (see Chapter 6, *Systems*).



Java example — before/after

```java
// File: lld/ch05/soc/Main.java
package lld.ch05.soc;

import java.util.HashMap;
import java.util.Map;

/** Separation of Concerns: parsing, business rules, storage and presentation each live in their own place. */
public class Main {

    // ---------------- BEFORE: one method does input parsing, rules, persistence AND HTML ----------------
    static class Before {
        static final Map<String, Integer> DB = new HashMap<>();
        static String handle(String request) {                    // "name=Asha&marks=87"
            String name = null; int marks = -1;
            for (String kv : request.split("&")) {
                String[] p = kv.split("=");
                if (p[0].equals("name")) name = p[1];
                if (p[0].equals("marks")) marks = Integer.parseInt(p[1]);
            }
            if (marks < 0 || marks > 100) return "<p style='color:red'>bad marks</p>";
            String grade = marks >= 90 ? "A" : marks >= 75 ? "B" : marks >= 50 ? "C" : "F";
            DB.put(name, marks);
            return "<p>" + name + " got " + grade + "</p>";
        }
    }

    // ---------------- AFTER: each concern is a separate class with one job ----------------
    static class After {
        record ResultRequest(String name, int marks) {}
        record Graded(String name, int marks, String grade) {}

        /** Concern 1: turn raw input into a typed request. */
        static final class RequestParser {
            ResultRequest parse(String raw) {
                Map<String, String> m = new HashMap<>();
                for (String kv : raw.split("&")) { String[] p = kv.split("=", 2); m.put(p[0], p[1]); }
                return new ResultRequest(m.get("name"), Integer.parseInt(m.get("marks")));
            }
        }
        /** Concern 2: business rules only. No HTML, no storage. */
        static final class GradingService {
            private final ResultRepository repo;
            GradingService(ResultRepository repo) { this.repo = repo; }
            Graded record(ResultRequest r) {
                if (r.marks() < 0 || r.marks() > 100) throw new IllegalArgumentException("marks must be 0..100");
                String grade = r.marks() >= 90 ? "A" : r.marks() >= 75 ? "B" : r.marks() >= 50 ? "C" : "F";
                repo.save(r.name(), r.marks());
                return new Graded(r.name(), r.marks(), grade);
            }
        }
        /** Concern 3: storage. */
        interface ResultRepository { void save(String name, int marks); int count(); }
        static final class InMemoryResultRepository implements ResultRepository {
            private final Map<String, Integer> rows = new HashMap<>();
            public void save(String name, int marks) { rows.put(name, marks); }
            public int count() { return rows.size(); }
        }
        /** Concern 4: presentation. Could be swapped for JSON without touching rules. */
        static final class HtmlView {
            String render(Graded g) { return "<p>" + g.name() + " got " + g.grade() + "</p>"; }
            String error(String msg) { return "<p style='color:red'>" + msg + "</p>"; }
        }
    }

    public static void main(String[] args) {
        System.out.println("== BEFORE ==");
        System.out.println(Before.handle("name=Asha&marks=87"));
        System.out.println(Before.handle("name=Ravi&marks=140"));
```

```
        System.out.println("== AFTER ==");
        var repo = new After.InMemoryResultRepository();
        var parser = new After.RequestParser();
        var service = new After.GradingService(repo);
        var view = new After.HtmlView();
        for (String raw : new String[]{"name=Asha&marks=87", "name=Ravi&marks=140"}) {
            try {
                System.out.println(view.render(service.record(parser.parse(raw))));
            } catch (IllegalArgumentException e) {
                System.out.println(view.error(e.getMessage()));
            }
        }
        System.out.println("rows stored: " + repo.count());
    }
}
```

Output:

```
== BEFORE ==
<p>Asha got B</p>
<p style='color:red'>bad marks</p>
== AFTER ==
<p>Asha got B</p>
<p style='color:red'>marks must be 0..100</p>
rows stored: 1
```

Each *after* class can now be tested alone: `GradingService` with an in-memory repository and no strings to parse; `HtmlView` could be replaced with a JSON view without touching the rules.

Trade-offs

Separated	Mixed
Change one concern safely	Quick to write for a one-off
Parallel team work (UI vs backend)	Everything visible in one place
More types and wiring	Tangled, hard to test

When not to use it

Tiny throwaway scripts. Also avoid over-layering (five layers that just pass data through — "lasagna code").

Common mistakes

- Business rules leaking into controllers or SQL.
- Separating by *technical type* only (all DTOs in one package) while one feature's logic is scattered — prefer separating by concern and cohesive feature packages.

Real-world examples

- **MVC** (Model–View–Controller) in Spring MVC, Rails, ASP.NET.
- **HTML / CSS / JavaScript** separation on the web.
- **TCP/IP network layers** (the stack of protocols the internet runs on, from physical links up through IP addressing, TCP's reliable delivery and HTTP): each layer handles one concern and talks only to its neighbours.
- **Spring AOP / servlet filters** separate cross-cutting concerns (security, transactions, logging) from business code.

In the interview

30-second answer: "Separation of Concerns means each module handles one aspect — parsing, rules, storage, presentation — so each can change and be tested independently. In LLD I show it as

> layers: controller, service, repository, with domain objects in the middle."

Follow-ups:

- *Q: SoC vs SRP?* A: SRP is SoC applied to a single class — one reason to change.
- *Q: What is a cross-cutting concern?* A: A concern that touches many modules (logging, auth, transactions); handled by AOP, proxies, decorators or filters.

Self-check

1. Q: Name four concerns in a typical web request. → A: Parsing input, business rules, persistence, presentation.
2. Q: Who coined the term? → A: Edsger Dijkstra (1974).

TL;DR

Give each concern its own home and let them communicate through clear, typed interfaces.

---

12. Principle of Least Astonishment (POLA)

What it is

Also called the **Principle of Least Surprise**: a component should behave the way most of its users *expect* from its name, signature and context. If a behaviour would surprise a reasonable reader, change the behaviour or the name.

Analogy: a light switch that also locks the front door would be "correct" according to its designer, but it surprises everybody.

The problem it solves

Surprising code causes bugs even when it "works as designed", because callers use it based on expectations. Typical surprises: methods named like queries that change state, methods that mutate and return `this` so aliases (other variables pointing to the same object) change, inconsistent naming (`size()` vs `length()` vs `count()` in one API), 0-based vs 1-based confusion, and silently swallowed errors.

How it works

1. Make names tell the truth: `find` / `get` / `is` must not modify state; if it creates, say so (`getOrCreate`).
2. Follow language and platform conventions (the `equals`/`hashCode` contract — equal objects must return the same *hash code*, the number hash-based collections like `HashMap` use to locate them — Java naming, `close()` via `AutoCloseable`).
3. Be consistent within your API — same word for same concept.
4. Prefer immutable results and no hidden side effects.
5. Signal "maybe absent" in the type (`Optional`, a box that either holds a value or is empty), not with a surprise `null` or a magic value.
6. Document any unavoidable surprise prominently.

Java example — before/after

```java
// File: lld/ch05/leastastonishment/Main.java
package lld.ch05.leastastonishment;

import java.math.BigDecimal;
import java.util.Arrays;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Optional;

/** Principle of Least Astonishment: code should behave the way its name and its readers expect. */
public class Main {

    // ---------------- BEFORE: surprising behaviour hidden behind innocent names ----------------
    static class Before {
        static class Money {
            long paise;
            Money(long paise) { this.paise = paise; }
            Money add(Money other) { this.paise += other.paise; return this; }   // looks pure, actually mutates
            @Override public String toString() { return "Rs " + BigDecimal.valueOf(paise, 2); }   // exact, no
double
        }
        static class UserDirectory {
            final Map<String, String> users = new HashMap<>();
            String findUser(String id) {                        // "find" silently CREATES a user
                return users.computeIfAbsent(id, k -> "new-user-" + k);
            }
        }
    }

    // ---------------- AFTER: names tell the truth; no hidden side effects ----------------
    static class After {
        record Money(long paise) {
            Money plus(Money other) { return new Money(paise + other.paise); }   // returns a NEW value
            @Override public String toString() { return "Rs " + BigDecimal.valueOf(paise, 2); }   // exact, no
double
        }
        static class UserDirectory {
            private final Map<String, String> users = new HashMap<>();
            Optional<String> findUser(String id) { return Optional.ofNullable(users.get(id)); }
            String getOrCreateUser(String id) { return users.computeIfAbsent(id, k -> "new-user-" + k); }
            int size() { return users.size(); }
        }
    }

    public static void main(String[] args) {
        System.out.println("== BEFORE ==");
        var price = new Before.Money(10_000);
        var withTip = price.add(new Before.Money(1_000));
        System.out.println("price=" + price + " withTip=" + withTip + "   <-- price changed too!");
        var dir = new Before.UserDirectory();
        dir.findUser("ghost");
        System.out.println("users after a *find*: " + dir.users.size());

        System.out.println("== AFTER ==");
        var p = new After.Money(10_000);
        var t = p.plus(new After.Money(1_000));
        System.out.println("price=" + p + " withTip=" + t);
        var d = new After.UserDirectory();
        System.out.println("find ghost -> " + d.findUser("ghost") + ", users=" + d.size());
        System.out.println("getOrCreate ghost -> " + d.getOrCreateUser("ghost") + ", users=" + d.size());

        System.out.println("== Real JDK surprises (good to know) ==");
        System.out.println("new BigDecimal(\"2.0\").equals(new BigDecimal(\"2.00\")) = "
                + new BigDecimal("2.0").equals(new BigDecimal("2.00")));
        System.out.println("new BigDecimal(\"2.0\").compareTo(new BigDecimal(\"2.00\")) = "
                + new BigDecimal("2.0").compareTo(new BigDecimal("2.00")));
        List<String> fixed = Arrays.asList("a", "b");
        try { fixed.add("c"); } catch (UnsupportedOperationException e) {
```

```
            System.out.println("Arrays.asList(...).add -> UnsupportedOperationException (fixed-size list)");
        }
    }
}
```

Output:

```
== BEFORE ==
price=Rs 110.00 withTip=Rs 110.00   <-- price changed too!
users after a *find*: 1
== AFTER ==
price=Rs 100.00 withTip=Rs 110.00
find ghost -> Optional.empty, users=0
getOrCreate ghost -> new-user-ghost, users=1
== Real JDK surprises (good to know) ==
new BigDecimal("2.0").equals(new BigDecimal("2.00")) = false
new BigDecimal("2.0").compareTo(new BigDecimal("2.00")) = 0
Arrays.asList(...).add -> UnsupportedOperationException (fixed-size list)
```

The last block shows *real* JDK behaviour that surprises many developers: `BigDecimal.equals` compares scale as well as value (`2.0` ≠ `2.00`), while `compareTo` says they are equal; and `Arrays.asList` returns a fixed-size list backed by the array. These are documented, but they are classic interview gotchas — use `compareTo` for numeric equality and `new ArrayList<>(Arrays.asList(...))` or `List.of(...)` depending on whether you need mutability.

Trade-offs

Following POLA sometimes means a longer name (`getOrCreateUser`) or an extra method. That's cheap compared to the bugs surprises cause. The only real tension: what's "expected" depends on the audience — follow the conventions of *your* platform and team.

When not to use it

There's no case where surprising users is good. But don't twist a design to match a *wrong* expectation; instead, fix the name so expectations become correct.

Common mistakes

- Getters with side effects (lazy creation, logging to DB, incrementing counters).
- Overloading `equals` (writing `equals(Money m)`) instead of overriding `equals(Object o)` — the collection classes silently ignore it.
- Methods whose behaviour changes based on a hidden global flag.

Real-world examples

- **java.time** was designed to fix the surprises of `java.util.Date`/`Calendar` (mutable dates, months numbered 0–11).
- `java.net.URL.equals` may perform DNS resolution (a network lookup that turns a host name into an IP address) (documented) — a notorious surprise; `java.net.URI` is usually preferred as a map key.
- **Unix** `rm` has no undo; tools like `trash` exist to reduce astonishment for new users.
- UI design: "Save" always in the same place; "Cancel" never saves.

In the interview

30-second answer: "Least astonishment means an API should do what its name and conventions suggest. Queries don't mutate, creation is named explicitly, absence is typed with Optional, and value objects are immutable. Surprises like `BigDecimal.equals` considering scale are exactly what we try not to create."

Follow-ups:

- *Q: Example of violating it in Java?* A: A `get` method that lazily writes to the database, or `Money.add` mutating `this`.

- *Q: How does CQS help?* A: Command-Query Separation (Chapter 6) makes it obvious which methods change state.

Self-check

1. Q: What should `findUser` do if the user doesn't exist? → A: Return an empty `Optional` (or throw a documented exception), never silently create one.
2. Q: Why is `a.add(b)` returning a mutated `this` surprising? → A: Other references to `a` change unexpectedly (aliasing).

TL;DR

Do what the name promises and nothing more; follow conventions; make absence and side effects visible.

---

13. Fail Fast

What it is

A **fail-fast** system detects an invalid state or input **immediately**, at the point where it arises, and stops with a clear error — rather than carrying on and failing mysteriously later (**fail slow / fail silently**).

Analogy: a smoke alarm that beeps at the first smoke, not after the house is on fire.

The problem it solves

If a `null` endpoint (the URL of a service to call) or a negative retry count is accepted in a constructor, the crash happens hours later, deep in unrelated code, with a stack trace (the list of method calls that were active when the error happened) that points to the symptom, not the cause. Bad data may also be saved and corrupt other data before anyone notices. The longer the gap between cause and failure, the more expensive the bug.

How it works

1. **Validate at boundaries**: constructors, public method entry, API/controller input, configuration load at startup.
2. Use **guard clauses** (checks at the top of a method that throw or return immediately on bad input) — `Objects.requireNonNull`, `if (bad) throw new IllegalArgumentException(...)` at the top of the method.
3. Throw specific exceptions with a helpful message (what was wrong, what value, what was expected).
4. Make invalid objects **unconstructable** (validation in constructor/factory + immutability).
5. Fail at **startup** for misconfiguration rather than on the first request.
6. Don't swallow exceptions (`catch (Exception e) {}`).

Fail fast vs fail safe: *fail-safe* means a failure leaves the system in a safe state (e.g. a lift's brakes engage on power loss). Interview literature (not the JDK documentation, which says "snapshot" or "weakly consistent") also calls **iterators** "fail-safe" when they never throw `ConcurrentModificationException` — `CopyOnWriteArrayList` iterates over a snapshot, `ConcurrentHashMap` iterators are weakly consistent — in contrast to *fail-fast* iterators (e.g. `ArrayList`) that throw `ConcurrentModificationException` when the collection is structurally modified during iteration. Note the JDK documents fail-fast iterators as **best-effort** — you must not rely on them for correctness.

Java example — before/after

```java
// File: lld/ch05/failfast/Main.java
package lld.ch05.failfast;

import java.time.Duration;
import java.util.ArrayList;
import java.util.ConcurrentModificationException;
import java.util.List;
import java.util.Objects;

/** Fail fast: detect a bad state at the earliest point, loudly, close to its cause. */
public class Main {

    // ---------------- BEFORE: garbage is accepted and blows up far away, much later ----------------
    static class Before {
        static class RetryConfig {
            final String endpoint; final int maxAttempts; final Duration backoff;
            RetryConfig(String endpoint, int maxAttempts, Duration backoff) {
                this.endpoint = endpoint; this.maxAttempts = maxAttempts; this.backoff = backoff;  // no checks
            }
        }
        static String callWithRetry(RetryConfig c) {
            // Hours later, deep inside business code:
            return "calling " + c.endpoint.toUpperCase() + " up to " + c.maxAttempts + " times, backoff "
                    + c.backoff.toMillis() + "ms";
        }
    }

    // ---------------- AFTER: validate at the boundary -- the object can never exist in a bad state ----------
---
    static class After {
        record RetryConfig(String endpoint, int maxAttempts, Duration backoff) {
            RetryConfig {
                Objects.requireNonNull(endpoint, "endpoint");
                Objects.requireNonNull(backoff, "backoff");
                if (maxAttempts < 1) throw new IllegalArgumentException("maxAttempts must be >= 1, was " +
maxAttempts);
                if (backoff.isNegative()) throw new IllegalArgumentException("backoff must not be negative");
            }
        }
    }

    public static void main(String[] args) {
        System.out.println("== BEFORE ==");
        var bad = new Before.RetryConfig(null, 0, null);   // constructed "successfully"
        System.out.println("config created, nothing complained...");
        try { Before.callWithRetry(bad); }
        catch (NullPointerException e) { System.out.println("much later: NullPointerException far from the real
mistake"); }

        System.out.println("== AFTER ==");
        try { new After.RetryConfig(null, 3, Duration.ofMillis(100)); }
        catch (NullPointerException e) { System.out.println("rejected at construction: " + e.getMessage()); }
        try { new After.RetryConfig("https://pay", 0, Duration.ofMillis(100)); }
        catch (IllegalArgumentException e) { System.out.println("rejected at construction: " + e.getMessage()); }
        System.out.println("ok: " + new After.RetryConfig("https://pay", 3, Duration.ofMillis(100)));

        System.out.println("== JDK fail-fast iterator ==");
        List<String> list = new ArrayList<>(List.of("a", "b", "c"));
        try {
            for (String s : list) if (s.equals("a")) list.remove(s);   // structural change during iteration
        } catch (ConcurrentModificationException e) {
            System.out.println("ArrayList iterator threw ConcurrentModificationException");
        }
        list.removeIf(s -> s.equals("b"));                              // the safe way
        System.out.println("after removeIf: " + list);
    }
}
```

Output:

```
== BEFORE ==
config created, nothing complained...
much later: NullPointerException far from the real mistake
== AFTER ==
rejected at construction: endpoint
rejected at construction: maxAttempts must be >= 1, was 0
ok: RetryConfig[endpoint=https://pay, maxAttempts=3, backoff=PT0.1S]
== JDK fail-fast iterator ==
ArrayList iterator threw ConcurrentModificationException
after removeIf: [c]
```

`RetryConfig` in *after* cannot exist with bad values; the error message names exactly what's wrong. The last block shows the JDK's own fail-fast iterator and the safe alternative `removeIf`. (Note: `list.remove("a")` itself succeeded before the iterator noticed the change on its next step, which is why only `"b"` was left for `removeIf` to delete.)

Trade-offs

Fail fast	Fail slow / silently
Bugs found early, near their cause	System "keeps running"
Clear diagnostics	Data corruption, confusing symptoms
Some upfront validation code	Less code, more debugging
May stop a whole process for a minor issue	—

When not to use it

Fail fast *inside* a component, but think about **resilience at system boundaries**: a UI shouldn't crash because one widget failed to load, and a batch job (a program that processes a large set of records in one run, often on a schedule) may skip and log a bad record rather than abort millions of good ones. The rule is: detect early, then decide deliberately how to handle (reject, retry, degrade) — never ignore.

Common mistakes

- Catching and ignoring exceptions, or returning `null` / `-1` on error.
- Validating deep inside business logic instead of at entry.
- Using `assert` for argument validation in public methods — Java assertions are **disabled by default** (enabled with `-ea`), so they can't be the only guard.

Real-world examples

- `Objects.requireNonNull`, `List.of(null)` throws `NullPointerException` immediately.
- **Spring Boot** (Spring's ready-made setup for quickly building runnable applications) fails at startup if a required bean or property is missing, and `@Validated` configuration properties fail on bad values.
- **Circuit breakers** (a wrapper around remote calls that "trips" after repeated failures and rejects further calls for a while; libraries: Resilience4j; earlier Netflix Hystrix, now in maintenance mode) fail fast: when a downstream service (one your service depends on and calls) is unhealthy, calls are rejected immediately instead of waiting for timeouts.
- **Erlang/OTP "let it crash"**: processes fail fast and supervisors restart them.
- **CI pipelines** (Continuous Integration: an automated build-and-test run on every code change) stop on the first failing stage.

In the interview

> **30-second answer:** "Fail fast means validating at the boundary and throwing a clear error immediately, so the failure is close to its cause. In LLD I put guard clauses in constructors and public methods so invalid objects can't exist. At the system level, circuit breakers fail fast to avoid piling up on slow dependencies."

Follow-ups:

- *Q: Fail-fast vs fail-safe iterators?* A: Fail-fast (`ArrayList`, `HashMap`) throw `ConcurrentModificationException` on structural modification during iteration (best-effort); fail-safe/weakly consistent ones (`CopyOnWriteArrayList`, `ConcurrentHashMap`) iterate over a snapshot or tolerate concurrent changes and never throw it.
- *Q: Isn't crashing bad for users?* A: Fail fast inside; handle gracefully at the edge (show an error, retry, degrade).

Self-check

1. Q: Where should validation go? → A: At boundaries: constructors, public method entry, startup config, API input.
2. Q: Why not rely on `assert` for public argument checks? → A: Assertions are disabled by default at runtime.

TL;DR

Check early, throw clearly, never swallow; make invalid states impossible to construct.

---

14. Immutability

What it is

An **immutable object** cannot change its observable state after it is constructed. To "change" it, you create a new object. `String`, `Integer`, `LocalDate`, `BigDecimal` and records with immutable components are examples.

Analogy: a printed receipt. If the amount is wrong, you print a new receipt; you don't edit the old one in everyone's hands.

The problem it solves

Mutable shared objects cause:

- **Aliasing bugs** — two references to the same object; one changes it, the other is surprised.
- **Broken invariants** — someone calls a setter and leaves the object half-valid.
- **Thread-safety problems** — concurrent reads and writes need locks (a mechanism that lets only one thread at a time run a protected piece of code).
- **Hash-based collection bugs** — changing a field used by `hashCode` after putting the object in a `HashSet` makes it "disappear".

Immutable objects are **inherently thread-safe** (can be shared freely without locks), can be cached and reused, are safe as map keys, and are easy to reason about.

How it works

The five rules from *Effective Java*'s item "Minimize mutability" (Joshua Bloch):

1. Don't provide methods that modify the object's state (no setters, no mutators).
2. Ensure the class can't be extended (`final` class, or private constructors + static factories — `static` methods such as `of(...)` that create instances). Records are implicitly final.
3. Make all fields `final`.
4. Make all fields `private`.

5. Ensure exclusive access to any mutable components: **defensively copy** mutable inputs in the constructor (`List.copyOf`) and never hand out references to mutable internals (return copies or unmodifiable views).

A common extra convenience: "wither" methods that return a new instance with one thing changed (`withStart`, `plusCity`).

Unmodifiable vs immutable: `Collections.unmodifiableList(list)` is a read-only *view* — if someone still holds the original list and changes it, the view shows the change. `List.copyOf` / `List.of` create an independent unmodifiable list. Records are **shallowly** immutable: a `List` component could still be mutable unless you copy it.

Java example — before/after

```
// File: lld/ch05/immutability/Main.java
package lld.ch05.immutability;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;

/** Immutability: once created, an object's observable state never changes. */
public class Main {

    // ---------------- BEFORE: a mutable class that leaks its internals ----------------
    static class Before {
        static class Itinerary {
            private final List<String> cities;              // 'final' reference, but the LIST is mutable
            private LocalDate start;
            Itinerary(List<String> cities, LocalDate start) { this.cities = cities; this.start = start; }
            List<String> getCities() { return cities; }     // hands out the internal list
            void setStart(LocalDate d) { start = d; }
            @Override public String toString() { return start + " " + cities; }
        }
    }

    // ---------------- AFTER: final class, final fields, defensive copies, "wither" methods ----------------
    static class After {
        /** A record is shallowly immutable; List.copyOf makes the list part immutable too. */
        record Itinerary(List<String> cities, LocalDate start) {
            Itinerary {
                cities = List.copyOf(cities);   // copy in: caller's later edits cannot reach us
            }
            Itinerary withStart(LocalDate newStart) { return new Itinerary(cities, newStart); }
            Itinerary plusCity(String c) {
                List<String> next = new ArrayList<>(cities);
                next.add(c);
                return new Itinerary(next, start);
            }
        }
    }

    public static void main(String[] args) {
        System.out.println("== BEFORE ==");
        List<String> input = new ArrayList<>(List.of("Delhi", "Agra"));
        var trip = new Before.Itinerary(input, LocalDate.of(2026, 10, 1));
        input.add("Jaipur");                        // caller edits its own list...
        trip.getCities().clear();                   // ...another caller wipes ours
        System.out.println("shared trip is now: " + trip + "   <-- changed by outsiders");

        System.out.println("== AFTER ==");
        List<String> input2 = new ArrayList<>(List.of("Delhi", "Agra"));
        var t1 = new After.Itinerary(input2, LocalDate.of(2026, 10, 1));
        input2.add("Jaipur");
        try { t1.cities().clear(); } catch (UnsupportedOperationException e) { System.out.println("cannot clear:
unmodifiable"); }
        var t2 = t1.plusCity("Jaipur").withStart(LocalDate.of(2026, 11, 5));
        System.out.println("t1 = " + t1);
        System.out.println("t2 = " + t2);

        System.out.println("== JDK immutables ==");
        String s = "hello";
        String upper = s.toUpperCase();
        LocalDate d = LocalDate.of(2026, 1, 31);
        System.out.println(s + " / " + upper + " ; " + d + " plusMonths(1) = " + d.plusMonths(1) + " (d unchanged:
" + d + ")");
    }
}
```

Output:

```
== BEFORE ==
shared trip is now: 2026-10-01 []   <-- changed by outsiders
== AFTER ==
cannot clear: unmodifiable
t1 = Itinerary[cities=[Delhi, Agra], start=2026-10-01]
t2 = Itinerary[cities=[Delhi, Agra, Jaipur], start=2026-11-05]
== JDK immutables ==
hello / HELLO ; 2026-01-31 plusMonths(1) = 2026-02-28 (d unchanged: 2026-01-31)
```

In *before*, `final List` only stops the *reference* from changing; outsiders both added to and cleared the list. In *after*, the compact constructor copies the list, the accessor returns an unmodifiable list, and "changes" produce new `Itinerary` values while `t1` stays intact. The JDK line shows `LocalDate.plusMonths` returning a new date (and clamping Jan 31 + 1 month to Feb 28).

Trade-offs

Immutable	Mutable
Thread-safe, no locks	Needs synchronisation when shared
Safe map keys, cacheable, easy reasoning	In-place updates, no allocation
Allocation for every "change" (usually cheap on modern JVMs — Java Virtual Machines, which run Java code)	Aliasing and invariant bugs
Large objects: copying cost (use builders or persistent data structures, which share unchanged parts between versions)	—

When not to use it

- Hot loops building big results — use a mutable builder locally (`StringBuilder`, then `toString()`), and expose the immutable result.
- Entities whose identity persists while state changes (a `BankAccount` balance) — keep them mutable but encapsulated; make their *values* (Money, Address) immutable.
- ORM entities (ORM = Object-Relational Mapping, a library that maps classes to database tables; JPA is Java's standard ORM API) usually need no-arg constructors and setters.

Common mistakes

- Thinking `final` fields make an object immutable (not if the field refers to a mutable object).
- Returning the internal list from a getter.
- Using `Collections.unmodifiableList` over a list the caller still holds.
- Letting `this` escape during construction (e.g. registering a listener in the constructor) — other threads may see a half-built object.

Real-world examples

- `String`, wrapper types, `BigDecimal`, `java.time.*`, `List.of`, `Map.of`, records.
- **Git commits** (saved versions in Git, the version-control tool) are immutable snapshots identified by hash (a fingerprint-like ID computed from the content).
- **Kafka log records** are immutable once appended.
- **Redux / React state** is treated as immutable; updates produce new state objects so change detection is a cheap reference check.

In the interview

30-second answer: "Immutable objects never change after creation: final class, private final fields, no setters, defensive copies in and out. They're thread-safe without locks and safe as map

keys. In designs I make value objects like Money, Address and configuration immutable, and keep mutable state encapsulated inside entities."

Follow-ups:

- *Q: Are records immutable?* A: Shallowly — fields are final, but a mutable component (like an `ArrayList`) must be defensively copied.
- *Q: `unmodifiableList` vs `List.copyOf`?* A: The first is a live read-only view; the second is an independent unmodifiable copy.
- *Q: Why must immutable classes be final?* A: A subclass could add mutable state or override methods to break the guarantees.

Self-check

1. Q: Does `private final List<String> items` make a class immutable? → A: No; the list itself can still be modified.
2. Q: What is a "wither" method? → A: A method returning a new instance with one field changed.
3. Q: Why are immutable objects thread-safe? → A: No thread can modify them, so there are no data races on their state (given safe construction).

TL;DR

No setters, final everything, copy mutable inputs and outputs; "changing" means creating a new value.

---

15. Design by Contract (DbC)

What it is

Design by Contract, from Bertrand Meyer and the Eiffel language, treats a method like a business contract between a **client** (caller) and a **supplier** (the method):

- **Precondition** — what must be true *before* the call. The caller's obligation. (e.g. `pop` requires a non-empty stack.)
- **Postcondition** — what the method guarantees *after* the call if the precondition held. The supplier's obligation. (e.g. after `push(x)`, size grew by one and the top is `x`.)
- **Class invariant** — what is always true about every object between public calls. (e.g. `0 ≤ size ≤ capacity`.)

Analogy: a courier contract: you must pack the parcel and pay (precondition); they promise delivery within 2 days (postcondition); the parcel is never opened in transit (invariant).

The problem it solves

Without explicit contracts, each side guesses: the caller doesn't know what inputs are allowed; the method either checks everything defensively (duplicated checks everywhere) or checks nothing and silently corrupts state. Errors show up as ambiguous return values (`-1`) or lost data.

How it works

1. For each public method, write down the precondition, postcondition and the class invariant (in Javadoc — the structured `/** ... */` comments that generate Java API docs — at least).
2. Check preconditions at entry and throw a clear exception when violated (`IllegalArgumentException`, `IllegalStateException`).
3. Check postconditions and invariants during development/testing (assertions or internal checks).
4. Give callers **query methods** to check preconditions themselves (`isEmpty()`, `isFull()`), so they can avoid violations.
5. With inheritance, obey the **contract rules of subtyping** (which is what the Liskov Substitution Principle formalises): a subclass may **weaken** (accept more) preconditions and **strengthen** (promise more) postconditions, never the reverse, and must preserve the invariant.

Java example — before/after

```java
// File: lld/ch05/contract/Main.java
package lld.ch05.contract;

import java.util.Arrays;

/** Design by Contract: preconditions (caller's duty), postconditions (method's promise), invariants (always true). */
public class Main {

    // ---------------- BEFORE: no stated contract; misuse silently corrupts state ----------------
    static class Before {
        static class BoundedStack {
            private final int[] items;
            private int size;
            BoundedStack(int capacity) { items = new int[Math.max(capacity, 0)]; }
            void push(int x) { if (size < items.length) items[size++] = x; }        // silently drops when full
            int pop() { return size > 0 ? items[--size] : -1; }                    // -1: is that data or an error?
        }
    }

    // ---------------- AFTER: every public method states and checks its contract ----------------
    static class After {
        static final class BoundedStack {
            private final int[] items;
            private int size;

            BoundedStack(int capacity) {
                require(capacity > 0, "capacity > 0");
                items = new int[capacity];
                checkInvariant();
            }
            /** pre: !isFull(). post: size() == old size + 1 and peek() == x. */
            void push(int x) {
                require(!isFull(), "push requires !isFull()");
                int oldSize = size;
                items[size++] = x;
                ensure(size == oldSize + 1 && peek() == x, "push postcondition");
                checkInvariant();
            }
            /** pre: !isEmpty(). post: size() == old size - 1. */
            int pop() {
                require(!isEmpty(), "pop requires !isEmpty()");
                int oldSize = size;
                int top = items[--size];
                ensure(size == oldSize - 1, "pop postcondition");
                checkInvariant();
                return top;
            }
            int peek() { require(!isEmpty(), "peek requires !isEmpty()"); return items[size - 1]; }
            boolean isEmpty() { return size == 0; }
            boolean isFull() { return size == items.length; }
            int size() { return size; }
            @Override public String toString() { return Arrays.toString(Arrays.copyOf(items, size)); }

            // invariant: 0 <= size <= capacity
            private void checkInvariant() { if (size < 0 || size > items.length) throw new AssertionError("invariant broken"); }
            private static void require(boolean ok, String what) { if (!ok) throw new IllegalStateException("precondition failed: " + what); }
            private static void ensure(boolean ok, String what) { if (!ok) throw new AssertionError("postcondition failed: " + what); }
        }
    }

    public static void main(String[] args) {
        System.out.println("== BEFORE ==");
        var s = new Before.BoundedStack(2);
        s.push(1); s.push(2); s.push(3);                              // 3 vanishes
```

```
        System.out.println("pop=" + s.pop() + " pop=" + s.pop() + " pop=" + s.pop() + "   <-- 3 lost, -1
ambiguous");

        System.out.println("== AFTER ==");
        var t = new After.BoundedStack(2);
        t.push(1); t.push(2);
        System.out.println("stack " + t + " full=" + t.isFull());
        try { t.push(3); } catch (IllegalStateException e) { System.out.println(e.getMessage()); }
        System.out.println("pop=" + t.pop() + " pop=" + t.pop());
        try { t.pop(); } catch (IllegalStateException e) { System.out.println(e.getMessage()); }
    }
}
```

Output:

```
== BEFORE ==
pop=2 pop=1 pop=-1   <-- 3 lost, -1 ambiguous
== AFTER ==
stack [1, 2] full=true
precondition failed: push requires !isFull()
pop=2 pop=1
precondition failed: pop requires !isEmpty()
```

The *before* stack silently drops pushes when full and returns `-1` when empty (is `-1` a value or an error?). The *after* stack documents and enforces its contract: violations are reported immediately and precisely, and callers can check `isFull()`/`isEmpty()` first.

Trade-offs

Explicit contracts	Implicit
Clear responsibilities, precise errors	Less code
Documentation that can be executed	Ambiguity, defensive checks everywhere
Runtime cost of checks (can be disabled for postconditions/invariants in production)	Silent corruption

When not to use it

Full postcondition/invariant checking in performance-critical inner loops — keep precondition checks on public APIs, and move expensive postcondition checks into tests or `assert` statements.

Common mistakes

- Using `assert` for public preconditions (disabled by default).
- Subclasses strengthening preconditions (e.g. a `SavingsAccount.withdraw` that rejects amounts the base `Account.withdraw` contract allowed) — an LSP violation. (The classic `Square extends Rectangle` problem is the mirror case: `Square.setWidth` breaks the *postcondition* that the height stays unchanged.)
- Returning special values instead of stating a precondition.

Real-world examples

- **Java's own API docs** are contracts: `List.get(int)` states it throws `IndexOutOfBoundsException` if the index is out of range; `Comparable.compareTo` and `Object.equals`/`hashCode` specify contracts every implementation must honour (e.g. equal objects must have equal hash codes).
- **Guava** `Preconditions.checkArgument/checkState` and Spring's `Assert` utilities are precondition helpers.
- **Kotlin** `require()` / `check()` mirror preconditions and state checks.
- **Eiffel** has `require`, `ensure` and `invariant` built into the language; **Ada 2012** has `Pre`/`Post` aspects.

In the interview

> **30-second answer:** "Design by Contract defines preconditions the caller must satisfy, postconditions the method guarantees, and invariants always true for the object. I enforce preconditions with guard clauses, expose query methods like `isEmpty`, and remember that subclasses can only weaken preconditions and strengthen postconditions — that's LSP."

Follow-ups:

- *Q: DbC vs defensive programming?* A: Defensive programming checks everything everywhere; DbC assigns each check to one party — preconditions to the caller (verified at entry), postconditions to the supplier.
- *Q: Which exceptions fit?* A: `IllegalArgumentException` for bad arguments, `IllegalStateException` when the object's state forbids the call, `NullPointerException` for nulls (via `requireNonNull`).

Self-check

1. Q: Whose duty is a precondition? → A: The caller's.
2. Q: What can a subclass do to preconditions? → A: Weaken (accept more), never strengthen.
3. Q: State the invariant of a bounded stack. → A: `0 ≤ size ≤ capacity`.

TL;DR

State and enforce who promises what: preconditions (caller), postconditions (method), invariants (always).

---

16. GRASP — General Responsibility Assignment Software Patterns

What it is

GRASP is a set of nine patterns (Larman calls them patterns; many people say principles or guidelines) by Craig Larman, from *Applying UML and Patterns* (UML, the Unified Modeling Language, is the standard diagram notation for classes and their interactions), for the most frequent question in object design: **"Which class should be responsible for this?"** A *responsibility* is either *knowing* something (data, related objects, derived values) or *doing* something (computing, creating, coordinating). GRASP is a thinking aid — a vocabulary for justifying design decisions — rather than a set of code templates.

History note: GRASP first appeared in the book's 1st edition (1997), where the ninth item was "Don't Talk to Strangers" (the Law of Demeter). From the 2nd edition (2001) it was replaced by the more general **Protected Variations**, giving the nine listed below.

#	GRASP guideline	Question it answers	Short answer
1	Information Expert	Who does this job?	The class that has the information needed
2	Creator	Who creates an object A?	A class that contains, records, closely uses or has the data to initialise A
3	Controller	Who first handles a system event from the UI?	A non-UI object representing the use case (one user goal, e.g. checkout) or the whole system
4	Low Coupling	How do we reduce the impact of change?	Assign responsibilities to keep dependencies few
5	High Cohesion	How do we keep classes focused and understandable?	Assign responsibilities so each class does related work
6	Polymorphism	How do we handle behaviour that varies by type?	Give each type its own implementation of a shared operation
7	Pure Fabrication	What if no domain class fits without hurting cohesion/coupling?	Invent a helper class (e.g. a repository)
8	Indirection	How do we avoid direct coupling between two things?	Put an intermediate object between them
9	Protected Variations	How do we shield elements from changes in others?	Wrap points of instability behind a stable interface

Analogy: staffing a hospital. The person who has the patient's chart (Expert) writes the summary; the maternity ward registers newborns (Creator); the front desk routes every visitor to the right department (Controller); nobody needs to know everyone (Low Coupling); each department has a focused speciality (High Cohesion).

The problem it solves

Beginners often end up with one "god" class (one class that knows and does far too much — often the UI screen or a `Manager`) that knows everything, and many anemic data classes. Or they spread logic randomly so that no one knows where anything lives. GRASP gives repeatable reasons for placing each responsibility, which also makes your interview explanations concrete ("I put `total()` on `Sale` because it's the Information Expert").

How it works

1. List the system events/use cases (e.g. `scan item`, `pay`).
2. For each event, choose a **Controller** to receive it.
3. For each thing that must be *known* or *computed*, find the **Information Expert**.
4. For each object that must be *created*, find the **Creator**.
5. Check **Low Coupling** and **High Cohesion** as evaluative forces; if a choice hurts them, reconsider.
6. Where behaviour varies by type, use **Polymorphism**; where an external or unstable thing touches you, use **Protected Variations** via **Indirection** (an interface or adapter — a class that converts one interface into the one your code expects).
7. When no domain object is a good home (persistence, I/O — input/output such as files and network calls), create a **Pure Fabrication**.

Java example — before/after (all nine in one checkout)

The *before* is a UI `CheckoutScreen` that creates line items, reaches into them to total the sale, hard-codes tax rules and builds SQL. The *after* assigns each responsibility using GRASP; each class's Javadoc names the guideline it illustrates.

```java
// File: lld/ch05/grasp/Main.java
package lld.ch05.grasp;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * GRASP on a tiny checkout: who should do what?
 * Information Expert, Creator, Controller, Low Coupling, High Cohesion,
 * Polymorphism, Pure Fabrication, Indirection, Protected Variations.
 */
public class Main {

    record Product(String sku, String name, BigDecimal price) {}

    // ---------------- BEFORE: a "god" screen class does everything ----------------
    static class Before {
        static class LineItem { Product product; int qty; }
        static class Sale { List<LineItem> items = new ArrayList<>(); }

        /** UI class that computes totals, calls the tax vendor directly and builds SQL. */
        static class CheckoutScreen {
            final Sale sale = new Sale();
            final List<String> sqlLog = new ArrayList<>();
            void onScan(Product p, int qty) {
                LineItem li = new LineItem();              // UI creates domain parts
                li.product = p; li.qty = qty;
                sale.items.add(li);
            }
            void onPayClicked(String region) {
                BigDecimal total = BigDecimal.ZERO;        // UI reaches into Sale -> LineItem -> Product
                for (LineItem li : sale.items) total =
total.add(li.product.price().multiply(BigDecimal.valueOf(li.qty)));
                BigDecimal tax = region.equals("IN") ? total.multiply(new BigDecimal("0.18"))   // vendor rule
inline
                        : total.multiply(new BigDecimal("0.07"));
                sqlLog.add("INSERT INTO sales VALUES(" + total.add(tax) + ")");
                System.out.println("screen shows total " + total.add(tax).setScale(2, RoundingMode.HALF_UP));
            }
        }
    }

    // ---------------- AFTER: responsibilities assigned with GRASP ----------------
    static class After {

        /** INFORMATION EXPERT: a line item knows its product and quantity, so it computes its own subtotal. */
        record LineItem(Product product, int qty) {
            BigDecimal subtotal() { return product.price().multiply(BigDecimal.valueOf(qty)); }
        }

        /** EXPERT for the total + CREATOR of LineItems (a Sale contains/aggregates them). HIGH COHESION: only
sale logic. */
        static final class Sale {
            private final String id;
            private final List<LineItem> items = new ArrayList<>();
            Sale(String id) { this.id = id; }
            void addItem(Product p, int qty) { items.add(new LineItem(p, qty)); }        // Creator
            BigDecimal total() { return items.stream().map(LineItem::subtotal).reduce(BigDecimal.ZERO,
BigDecimal::add); }
            String id() { return id; }
            List<LineItem> items() { return List.copyOf(items); }
        }

        /** PROTECTED VARIATIONS: tax rules vary by region/vendor -> hide them behind a stable interface. */
        interface TaxCalculator { BigDecimal taxFor(Sale sale); }
```

```
        /** POLYMORPHISM: each variant is its own type; no if/else on region in the domain. */
        static final class IndiaGst implements TaxCalculator {
            public BigDecimal taxFor(Sale s) { return s.total().multiply(new BigDecimal("0.18")); }
        }
        static final class FlatUsSalesTax implements TaxCalculator {
            public BigDecimal taxFor(Sale s) { return s.total().multiply(new BigDecimal("0.07")); }
        }

        /** A third-party library with its own, awkward API (we don't control it): it even returns tax as a
double. */
        static final class LegacyTaxEngine {
            double computeTaxCents(long amountCents, String countryCode) { return amountCents *
("GB".equals(countryCode) ? 0.20 : 0.0); }
        }
        /** INDIRECTION: an adapter sits between our domain and the vendor, so neither knows the other. */
        static final class LegacyTaxAdapter implements TaxCalculator {
            private final LegacyTaxEngine engine; private final String country;
            LegacyTaxAdapter(LegacyTaxEngine engine, String country) { this.engine = engine; this.country =
country; }
            public BigDecimal taxFor(Sale s) {   // converts to/from the vendor's types at the boundary only
                long cents = s.total().movePointRight(2).longValueExact();
                return BigDecimal.valueOf(Math.round(engine.computeTaxCents(cents, country))).movePointLeft(2);
// whole cents
            }
        }

        /** PURE FABRICATION: not a domain concept -- invented to keep persistence out of Sale (cohesion +
coupling). */
        interface SaleRepository { String nextId(); void save(Sale sale, BigDecimal grandTotal); int count(); }
        static final class InMemorySaleRepository implements SaleRepository {
            private final Map<String, BigDecimal> rows = new HashMap<>();
            private int seq;
            public String nextId() { return "S" + (++seq); }
            public void save(Sale s, BigDecimal g) { rows.put(s.id(), g); }
            public int count() { return rows.size(); }
        }

        /** CONTROLLER: the first non-UI object that receives system events (scan, pay) and delegates. LOW
COUPLING: depends on interfaces. */
        static final class CheckoutController {
            private final TaxCalculator tax; private final SaleRepository repo;
            private Sale current;
            CheckoutController(TaxCalculator tax, SaleRepository repo) { this.tax = tax; this.repo = repo; }
            void startSale() { current = new Sale(repo.nextId()); }
            void scan(Product p, int qty) { current.addItem(p, qty); }
            BigDecimal pay() {
                BigDecimal grand = current.total().add(tax.taxFor(current)).setScale(2, RoundingMode.HALF_UP);
                repo.save(current, grand);
                return grand;
            }
        }
    }

    public static void main(String[] args) {
        Product tea = new Product("T1", "Tea", new BigDecimal("120.00"));
        Product cup = new Product("C1", "Cup", new BigDecimal("80.00"));

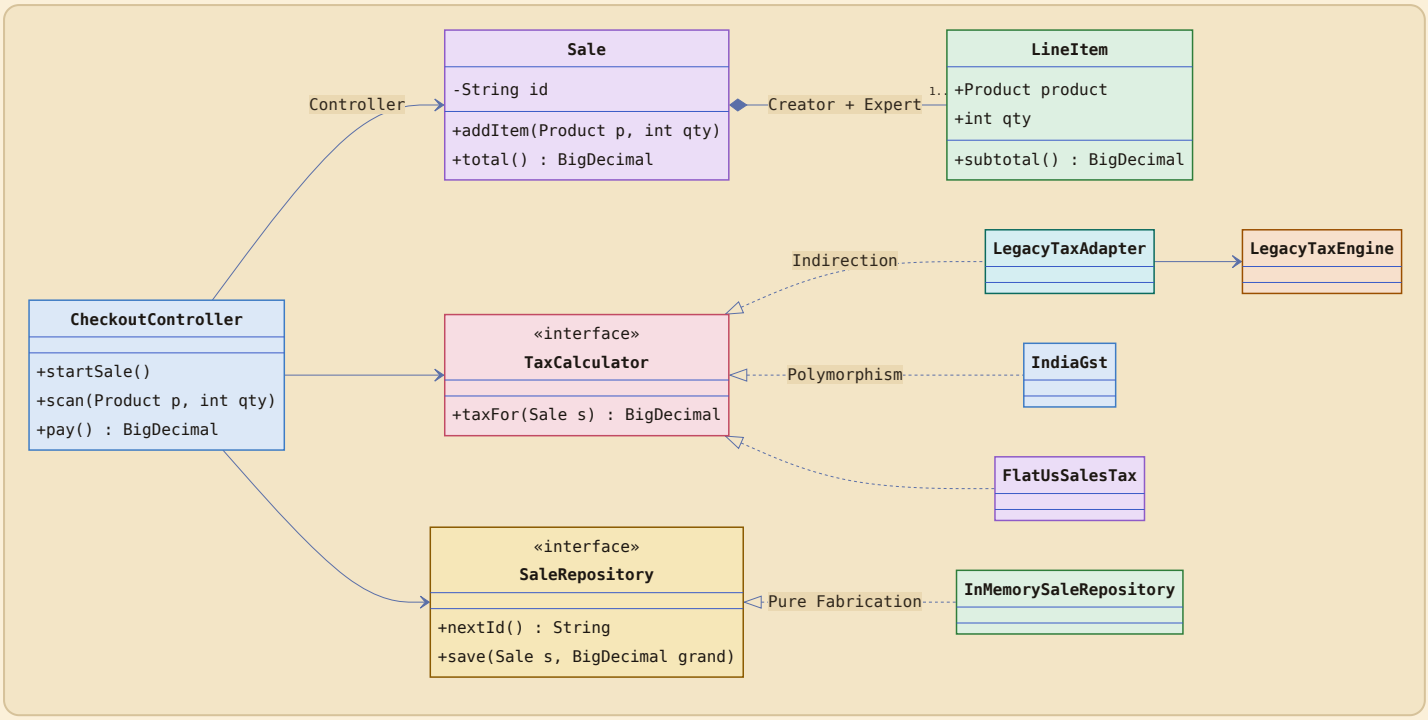
        System.out.println("== BEFORE ==");
        var screen = new Before.CheckoutScreen();
        screen.onScan(tea, 2); screen.onScan(cup, 1);
        screen.onPayClicked("IN");
        System.out.println("sql: " + screen.sqlLog);

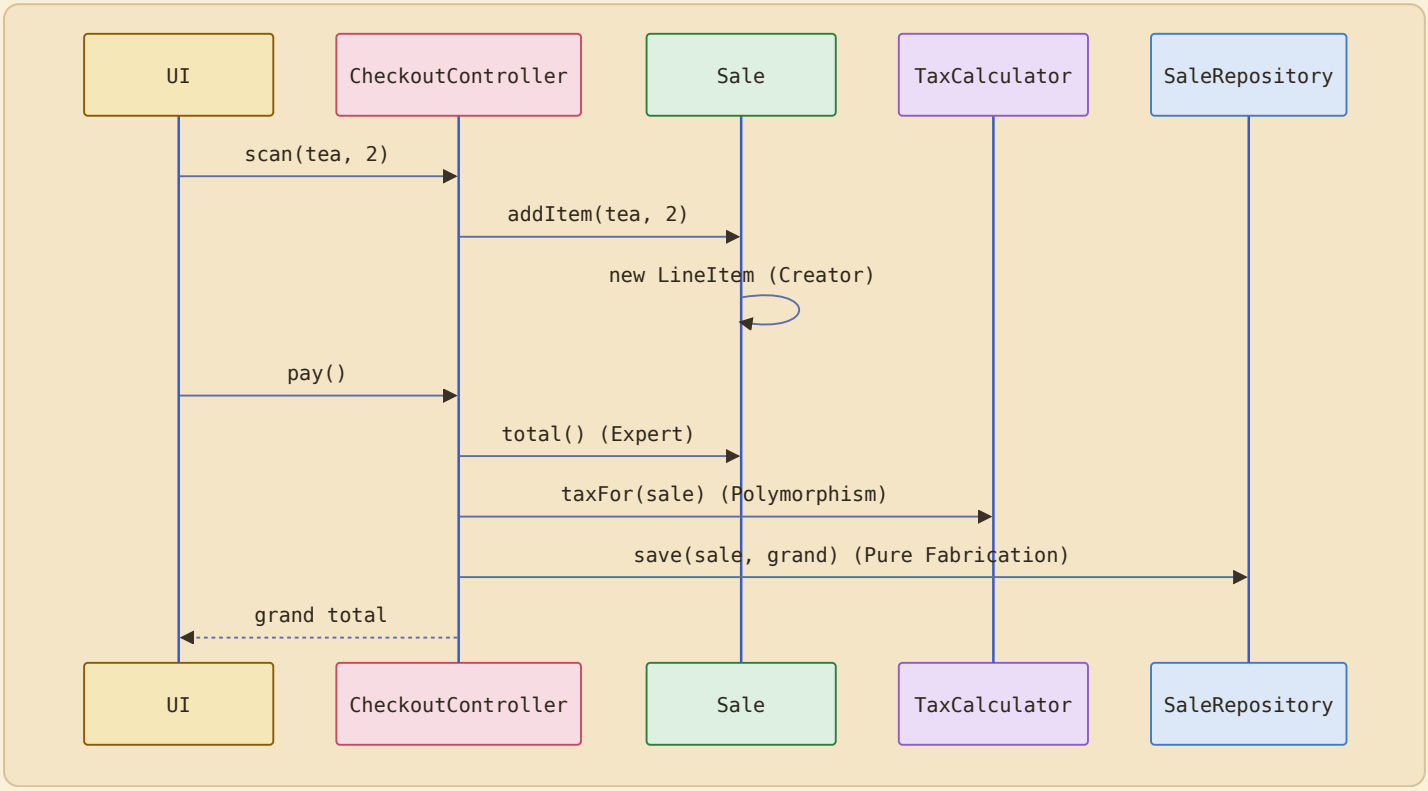
        System.out.println("== AFTER ==");
        var repo = new After.InMemorySaleRepository();
        for (After.TaxCalculator calc : List.of(new After.IndiaGst(), new After.FlatUsSalesTax(),
                new After.LegacyTaxAdapter(new After.LegacyTaxEngine(), "GB"))) {
            var controller = new After.CheckoutController(calc, repo);
            controller.startSale();
            controller.scan(tea, 2);
            controller.scan(cup, 1);
```

```
            System.out.println(calc.getClass().getSimpleName() + " -> grand total " + controller.pay());
        }
        System.out.println("sales saved: " + repo.count());
    }
}
```

Output:

```
== BEFORE ==
screen shows total 377.60
sql: [INSERT INTO sales VALUES(377.6000)]
== AFTER ==
IndiaGst -> grand total 377.60
FlatUsSalesTax -> grand total 342.40
LegacyTaxAdapter -> grand total 384.00
sales saved: 3
```





The rest of this section explains each guideline in the book's standard format.

16.1 Information Expert

What it is

Give a responsibility to the class that **has the information** needed to fulfil it. `LineItem` knows price and quantity → it computes `subtotal()`. `Sale` knows its line items → it computes `total()`.

The problem it solves

Prevents "manager" classes that pull data out of other objects to compute things (the *Feature Envy* smell), which duplicates logic and couples the manager to everyone's internals.

How it works

1. State the responsibility ("compute sale total").
2. Ask what information is needed (line item subtotals).
3. Find who holds it (Sale holds line items; each LineItem holds price × qty).
4. Split the work along those lines — partial experts cooperate.

Trade-offs

Keeps behaviour with data (Tell-Don't-Ask). But blindly applying it can hurt cohesion: `Sale` has the data to save itself, yet putting SQL into `Sale` mixes concerns — that's why Pure Fabrication exists.

When not to use it

When following it would put technical concerns (persistence, UI, network) into domain objects.

Common mistakes

Computing everything in a service with getters; or the opposite, making an entity save/print/email itself.

Real-world examples

`String.length()`, `List.size()`, `LocalDate.isLeapYear()` — the object with the data answers. In a Swiggy/Zomato-style cart, the `Cart` computes its own item total.

In the interview

"`total()` lives on `Order` because it owns the line items — Information Expert. Each `LineItem` computes its own subtotal." *Follow-up: when does Expert conflict with other principles?* When it would add persistence or UI to a domain class; then use Pure Fabrication.

Self-check

Q: Who should compute a line item's subtotal? → A: `LineItem` — it has price and quantity.

TL;DR

The one with the data does the work.

16.2 Creator

What it is

Class **B** should create instances of class **A** if one or more is true: B *contains* or aggregates A; B *records* A; B *closely uses* A; B *has the initialising data* for A. Here `Sale` creates `LineItem`s.

The problem it solves

Who calls `new`? Random placement spreads coupling to A's constructor everywhere. Choosing a class that already relates to A adds no new coupling.

How it works

1. Find the class that aggregates/contains A (strongest reason).
2. Otherwise one that records or closely uses it, or has its initial data.
3. If creation is complex or varies (families, pooling, choosing a subclass), use a **Factory** or **Builder** instead (Chapter 8).

Trade-offs

Simple and low-coupling for plain objects; not suitable when creation logic is complex or conditional.

When not to use it

Complex creation → Factory/Builder; framework-managed objects → DI container.

Common mistakes

UI or controller code doing `new LineItem(...)` and pushing it into the sale's list (the *before* code).

Real-world examples

`HashMap` creating its own internal node objects for the entries it contains; a `Document` creating its `Paragraph`s; `Thread` objects created by an `ExecutorService` through its `ThreadFactory` (a factory, because creation varies).

In the interview

"`Sale.addItem(product, qty)` creates the `LineItem` because the sale contains them — GRASP Creator. If creation got complex I'd move it to a factory." *Follow-up: Creator vs Factory?* Creator picks an existing class that already relates to A (no new coupling); a Factory is a Pure Fabrication you add when creation is complex or must choose among subclasses.

Self-check

Q: Strongest reason for B to create A? → A: B contains/aggregates A.

TL;DR

The container (or the one with the initial data) creates.

16.3 Controller

What it is

The first object **beyond the UI layer** that receives and coordinates a **system operation** (an event from an actor — a person or external system using ours — such as "scan item" or "pay"). Two kinds: a **facade controller** representing the whole system/device (e.g. `Register`, `ParkingLotSystem`) or a **use-case controller** for one scenario (e.g. `CheckoutController`, `BookTicketHandler`).

The problem it solves

Keeps business logic out of UI classes, so the same logic works from a web page, a mobile app or a test, and the UI can change freely.

How it works

1. List system events from use cases.
2. Assign each to a controller that represents the system or the use case.

3. The controller **delegates** — it coordinates, it doesn't do the domain work itself.

Trade-offs

Clean UI/domain separation; but a controller that does real work becomes **bloated** (low cohesion, many responsibilities) — split into use-case controllers and push work to experts.

When not to use it

Very small programs where the UI and logic are trivial.

Common mistakes

Fat controllers containing pricing, validation and SQL. Controllers that are UI classes (e.g. a Swing `JFrame` computing totals).

Real-world examples

Spring `@RestController` → `@Service` (the service often plays GRASP Controller for a use case); `DispatcherServlet` is a front controller (a single entry point that receives every web request and routes it); in an Uber-like app a `TripService.requestRide()` handles the rider's "request ride" event.

In the interview

"`ParkingLotController.park(vehicle)` receives the system event and delegates to `ParkingFloor` and `SpotAllocator` — GRASP Controller, kept thin." *Follow-up: facade or use-case controller?* A facade controller (one per system) is fine while there are few system events; when it grows bloated, split into use-case controllers (one per scenario).

Self-check

Q: Should a controller calculate tax? → A: No — it delegates to the expert or a tax strategy.

TL;DR

A thin non-UI coordinator receives each system event and delegates.

16.4 Low Coupling

What it is

An **evaluative** guideline: among design choices, prefer the one that keeps **dependencies between classes low**. Here the controller depends only on interfaces (`TaxCalculator`, `SaleRepository`), not on vendors or databases.

The problem it solves

High coupling means changes ripple, classes can't be reused alone, and understanding one class requires understanding many.

How it works

Compare alternatives by counting and weighing dependencies: who knows about whom? Prefer interfaces, fewer collaborators, one-directional dependencies, no cycles. Use DI.

Trade-offs

Coupling to *stable* things (JDK, `String`, `List`) is harmless. Striving for zero coupling produces bloated indirection.

When not to use it

Don't decouple from stable, standard libraries — focus on unstable or external dependencies.

Common mistakes

Circular dependencies between packages; UI knowing the database; everything depending on a god class.

Real-world examples

Microservices communicating over well-defined APIs/events; JDBC drivers decoupled from application code by interfaces.

In the interview

"My service depends on `PaymentGateway`, not on Stripe's SDK (Software Development Kit — the vendor's client library), so switching provider touches one adapter." *Follow-up: can coupling ever be zero?* No — objects must collaborate. Aim for few dependencies, pointing at stable abstractions, with no cycles.

Self-check

Q: Is depending on `java.util.List` bad coupling? → A: No — it is stable.

TL;DR

Choose the design with fewer, more stable dependencies.

16.5 High Cohesion

What it is

The other **evaluative** guideline: keep each class's responsibilities **focused and related**, so it's easy to understand and maintain. `Sale` does sale maths; persistence is in the repository; tax is in the calculator.

The problem it solves

Low-cohesion classes (e.g. `Utils`, `Manager`, a screen doing everything) are hard to understand, reuse and change; they change for many reasons (violating SRP).

How it works

When adding a responsibility, ask "is this related to what the class already does?" If not, place it elsewhere or create a new class. Larman describes four levels, from worst to best: *very low* (one class responsible for many things in very different areas) → *low* (one class alone responsible for a big, complex job in one area) → *moderate* (a few light responsibilities in different areas that relate to the class concept but not to each other) → *high* (a moderate number of related responsibilities in one area, collaborating with other objects for the rest — the goal).

Trade-offs

Many small focused classes vs a few big ones — more files, but each is simple. Coupling and cohesion usually improve together.

When not to use it

Rarely; sometimes grouping for performance (e.g. a remote API returning coarse-grained data) lowers cohesion on purpose.

Common mistakes

"Helper" and "Utils" dumping grounds; entities that also send emails.

Real-world examples

`java.time.LocalDate` (dates only), `java.util.regex.Pattern` (regex only). Spring Data repositories focus purely on persistence.

In the interview

"I split `BookingService` into seat locking, pricing and payment so each class has one focused job — high cohesion." *Follow-up: how do cohesion and coupling interact?* They usually improve together: moving unrelated work out of a class raises its cohesion and removes the dependencies that work dragged in.

Self-check

Q: Name a common low-cohesion smell. → A: A `Utils` / `Manager` class with unrelated methods.

TL;DR

Each class does one related set of things.

16.6 Polymorphism

What it is

When behaviour **varies by type**, give each type its own implementation of a shared operation (an interface method), instead of branching on the type with `if/else` or `switch`. `IndiaGst`, `FlatUsSalesTax` and `LegacyTaxAdapter` each implement `taxFor`.

The problem it solves

Type-checking conditionals repeated across the code; adding a type means finding and editing every branch (violates OCP).

How it works

1. Identify the operation that varies (`taxFor`).
2. Declare it on an interface/abstract class.
3. Implement it per variant.
4. Callers invoke it without knowing the concrete type.

Trade-offs

Easy to add new types; harder to add new operations across all types (you must touch each class — the Visitor pattern, Chapter 11, flips this trade-off).

When not to use it

For closed sets of types where you often add *operations*, a `switch` over a `sealed` hierarchy (Java 21 pattern matching — a `switch` that tests an object's type and unpacks it in one step) can be clearer.

Common mistakes

Creating polymorphic classes for variation that doesn't exist (YAGNI); `instanceof` chains after introducing the interface.

Real-world examples

`Comparator` implementations, `InputStream` subclasses, `Runnable` / `Callable`, payment methods in any checkout (card, UPI, wallet).

In the interview

"Instead of `if (vehicle instanceof Car)`, each `Vehicle` type answers `requiredSpotSize()` — GRASP Polymorphism." *Follow-up: what if I need a new operation across all types?* Adding it touches every class; if that happens often, consider a `sealed` hierarchy with pattern-matching `switch`, or the Visitor pattern.

Self-check

Q: Which kind of change does polymorphism make easy? → A: Adding a new type/variant.

TL;DR

Let each type implement the varying behaviour; don't switch on types.

16.7 Pure Fabrication

What it is

A class that **doesn't represent a real-world domain concept**, invented purely to keep high cohesion and low coupling when Information Expert would give a bad result. `SaleRepository` is a fabrication: the shop has no "repository", but putting SQL into `Sale` would ruin cohesion.

The problem it solves

Some responsibilities (persistence, logging, ID generation, messaging) don't fit any domain class without dragging technical concerns into it.

How it works

1. Notice the Expert choice would make a domain class do technical work.
2. Create a service class whose name describes the job (`SaleRepository`, `NotificationSender`, `IdGenerator`).
3. Keep it focused and reusable.

Trade-offs

Great for cohesion/reuse; overusing it produces an anemic domain where all behaviour sits in "services".

When not to use it

When a real domain object is a natural home — prefer the Expert first.

Common mistakes

Moving domain logic (pricing, rules) into fabricated "Manager" services.

Real-world examples

DAOs (Data Access Objects — classes that hide database queries)/repositories, `ObjectMapper` (Jackson, the common Java JSON library), `Clock` providers, Spring `@Service` classes for infrastructure, most GoF helper objects (Strategy, Adapter, Factory objects are fabrications).

In the interview

"`TicketRepository` isn't a domain thing — it's a pure fabrication that keeps persistence out of `Ticket`." *Follow-up: risk of too many fabrications?* An anemic domain: all rules drift into "service" classes. Keep domain rules on domain objects (Expert) and fabricate only for technical or cross-cutting work.

Self-check

Q: Is `Sale` a pure fabrication? → A: No — it's a domain concept. `SaleRepository` is.

TL;DR

Invent a helper class when no domain class is a clean home.

16.8 Indirection

What it is

Assign a responsibility to an **intermediate object** that mediates between two components so they aren't directly coupled. `LegacyTaxAdapter` sits between our `CheckoutController` and the vendor's `LegacyTaxEngine`.

The problem it solves

Direct dependencies on external or volatile components (vendor SDKs, hardware, other subsystems).

How it works

Introduce an interface plus an adapter, a facade, a mediator (a hub that objects talk through instead of to each other) or a proxy; the two sides talk through it.

Trade-offs

Decoupling vs an extra hop (small performance cost and one more class to read). A famous saying, usually attributed to David Wheeler, is that any problem in computer science can be solved by another level of indirection; the usual retort is "except the problem of too many levels of indirection".

When not to use it

Between stable, cohesive classes that always change together.

Common mistakes

Layers that add nothing (pure pass-through), making navigation painful.

Real-world examples

Adapter/Facade/Mediator/Proxy patterns; JDBC's `DriverManager`; API gateways in microservices (a single front door that routes client calls to many services); message brokers between producers and consumers.

In the interview

"I don't call the SMS vendor directly — an `SmsSender` interface with a `TwilioSmsSender` adapter provides indirection." *Follow-up: Indirection vs Protected Variations?* Indirection is the mechanism (an intermediate object); Protected Variations is the goal (shield the rest of the system from a point of change). Indirection is one common way to achieve it.

Self-check

Q: Which GoF patterns are forms of indirection? → A: Adapter, Facade, Mediator, Proxy (and Bridge).

TL;DR

Put a middleman between things that shouldn't know each other.

16.9 Protected Variations

What it is

Identify **points of predicted variation or instability** and wrap them with a **stable interface**, so variation doesn't ripple to other elements. `TaxCalculator` protects the checkout from tax-rule and vendor changes. Larman notes it is essentially David Parnas's *information hiding* (keeping design decisions that may change hidden inside one module) and Meyer's Open/Closed Principle stated as a responsibility guideline (and close to "encapsulate what varies"); the Law of Demeter — GRASP's "Don't Talk to Strangers" in the 1st edition — is one mechanism for it.

The problem it solves

Changes in one place (a vendor, a regulation, a format) forcing edits throughout the system.

How it works

1. Find variation points (external systems, rules that change, platforms).
2. Define a stable interface around each.
3. Put variants behind it (Polymorphism, Indirection, data-driven config).

Trade-offs

Protection has a cost; over-protecting things that never vary ("speculative generality") adds complexity. Protect *predicted* variation.

When not to use it

Stable, unlikely-to-change code (YAGNI).

Common mistakes

Leaky interfaces that expose vendor-specific types (e.g. returning a `StripeCharge` from `PaymentGateway`).

Real-world examples

JDBC, JPA, SLF4J, Java's `ServiceLoader` for pluggable providers; `java.nio.file.FileSystem` abstraction for different file systems.

In the interview

"Payment providers are a variation point, so `PaymentGateway` is my protected variation — the booking flow never sees provider details." *Follow-up: which variation points should you protect?* Predicted ones — external vendors, regulations, formats, platforms — not every class (that is speculative generality).

Self-check

Q: Which SOLID principle is closest to Protected Variations? → A: Open/Closed (also Dependency Inversion).

TL;DR

Wrap what's unstable behind a stable interface.

GRASP in the interview (whole set)

> **30-second answer:** "GRASP is Larman's set of nine guidelines for assigning responsibilities. I use Information Expert and Creator to place most behaviour, a thin Controller for system events, Polymorphism and Protected Variations for things that vary, Indirection and Pure Fabrication to keep technical concerns out of the domain, and I judge every choice by Low Coupling and High Cohesion."

Follow-ups:

- *Q: How does GRASP relate to GoF patterns?* A: GRASP is more fundamental: patterns like Strategy (Polymorphism + Protected Variations), Adapter (Indirection) and Repository (Pure Fabrication) are specific applications.
- *Q: Which two GRASP items are evaluative rather than assignment rules?* A: Low Coupling and High Cohesion — you use them to compare alternatives.
- *Q: Expert says `Sale` should save itself — why not?* A: It hurts cohesion and couples the domain to storage; use a Pure Fabrication (repository).

Self-check (GRASP)

1. Q: Who handles the "pay" button event after the UI? → A: A Controller (use-case or facade).
2. Q: Which guideline justifies an adapter around a vendor library? → A: Indirection (serving Protected Variations).
3. Q: Name the guideline for inventing `IdGenerator`. → A: Pure Fabrication.

TL;DR

Put work where the data is, let containers create, keep controllers thin, vary by polymorphism behind stable interfaces, invent helpers when needed — and judge it all by coupling and cohesion.

---

17. Domain-Driven Design (DDD) Essentials

What it is

Domain-Driven Design is an approach, named by Eric Evans in *Domain-Driven Design: Tackling Complexity in the Heart of Software* (2003), for building software whose code mirrors the business it serves. The **domain** is the subject area the software is about — lending, shipping, grocery retail. DDD has two halves:

- **Strategic design** — how to split a large business into parts, each with its own model: **ubiquitous language**, **bounded contexts**, **context maps** and the **anti-corruption layer**.
- **Tactical design** — the building blocks *inside* one part: **entities**, **value objects**, **aggregates**, **domain events**, **repositories**, plus domain services and factories.

The principles earlier in this chapter tell you how to shape classes; DDD tells you **which classes to have and where their boundaries are**, starting from the business rather than the database.

Analogy: a hospital. The word "patient" means one thing at the reception desk (name, insurance, appointment), another in the operating theatre (blood type, allergies, vitals) and another in billing (procedures, amounts owed). Each department keeps its own file with its own vocabulary; they exchange only what they need. Forcing one giant "Patient" form on all three would please nobody. Each department is a bounded context.

Key terms (defined once):

Term	Meaning
Ubiquitous language	The shared vocabulary that domain experts and developers both use — in conversation, in documents *and in the code*. If the business says "place an order", the method is `place()`, not `setStatus(2)`.
Bounded context	A boundary inside which one model and one language apply consistently. The same word may mean something different in another context (a Sales "order" vs a Shipping "consignment"). Often one team and one service.
Context map	A picture of the bounded contexts and how they relate (who depends on whom, who translates).
Anti-corruption layer (ACL)	A translation layer at a context's border that converts another context's (or a legacy — old, still-running — system's) model into yours, so theirs cannot leak in.
Entity	An object defined by its **identity**, which stays the same while its attributes change (an `Order` with id 1001 is the same order after items are added). Equality = same id.
Value object	An object defined only by its **attributes**, with no identity; immutable; interchangeable when equal (`Money(₹350)`, an `Address`, a `DateRange`). Equality = same values.
Aggregate	A cluster of entities and value objects treated as **one unit for changes**, with one entity as its **aggregate root** — the only object outsiders may hold or call. The root enforces the cluster's invariants.
Domain event	A record of something that happened that the business cares about, named in the past tense (`OrderPlaced`, `PaymentFailed`). Other parts react to it.
Repository	A collection-like interface for loading and saving **aggregates** (one repository per aggregate root), hiding the database (Chapter 11, §11.7.3).
Domain service	An operation that belongs to the domain but not naturally to one entity (e.g. "transfer money between two accounts"). Stateless.

The problem it solves

Big business systems fail less from bad algorithms than from **muddled models**:

- **One model for everything.** A single `Customer` or `Product` class grows fields for every department until nobody can change it safely (a "big ball of mud").
- **Translation errors.** Developers call it `status=3`, the business calls it "on hold"; requirements get lost between the two vocabularies.
- **Anemic domain model** (Martin Fowler's term): classes are data bags with getters and setters, and every rule lives in services. Rules get duplicated, and anyone can put an object into an invalid state (Tell, Don't Ask — section 5 — violated at scale).
- **No consistency boundary.** Without aggregates, code updates any object anywhere, locking and transactions sprawl, and invariants ("an order's total never exceeds the credit limit") are checked in some paths but not others.
- **Tight coupling between teams.** If Shipping reads Sales' tables directly, Sales cannot change its schema without breaking Shipping.

How it works

Strategic design — split by meaning, not by table

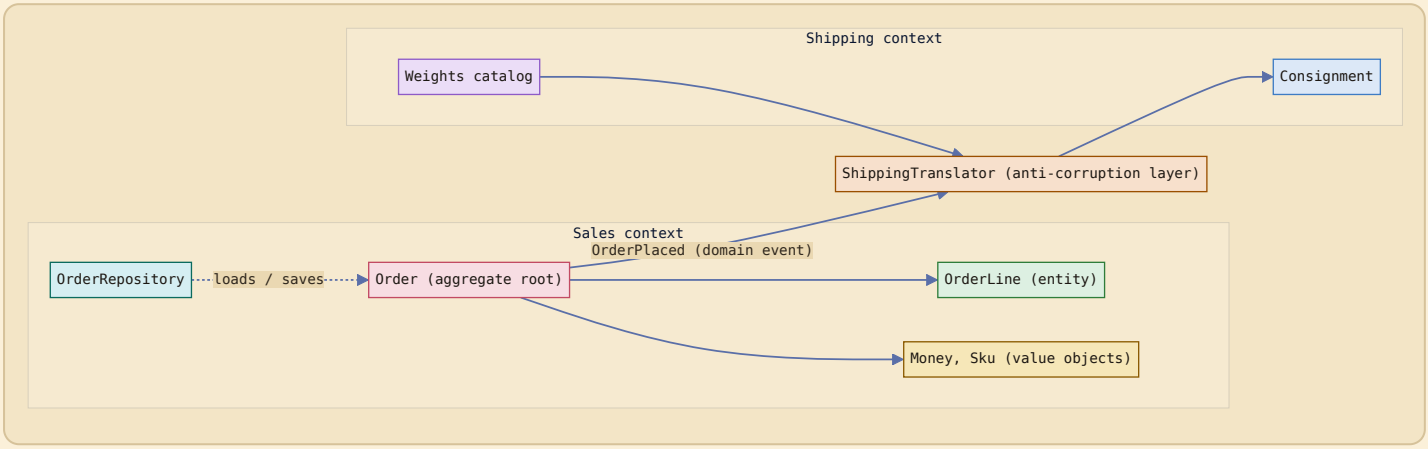
1. **Talk to domain experts and write down the language.** Nouns become candidate entities and value objects, verbs become methods, "when X happens" becomes a domain event.
2. **Draw the bounded contexts.** Where the same word starts meaning different things, you have a boundary. In an e-commerce business: *Catalog* (product descriptions, images), *Sales* (orders, prices, promotions), *Payments*, *Shipping* (parcels, weights, carriers), *Support*.
3. **Decide how contexts talk (the context map).** The common relationships: one side publishes events or an API and the other consumes it; a **shared kernel** (a small shared library both own — use

sparingly); **conformist** (you simply accept the other model); and the **anti-corruption layer** (you translate the other model into yours — the safest choice against a legacy or third-party system).

4. **Map contexts to teams and services.** A bounded context is the most natural boundary for a microservice; that is why DDD is the usual starting point for splitting a monolith (an application built and deployed as one single unit) (see the Strangler Fig notes for migrating piece by piece).

Tactical design — the building blocks inside one context

- **Prefer value objects.** Wrap primitives (raw built-in types such as `double` or `String`) that carry rules: `Money` (amount + currency, no negatives), `Email`, `Quantity`, `DateRange`. They are immutable (section 14), validate themselves (Fail fast, section 13) and make method signatures honest (`transfer(Money)` instead of `transfer(double)`).
- **Entities have identity and a lifecycle.** Give them an id value object (`OrderId`), and methods named in the ubiquitous language that protect their invariants.
- **Aggregate design rules** (from Vaughn Vernon's *Implementing Domain-Driven Design*, 2013):
  1. Protect **true invariants** inside the aggregate boundary — the things that must be consistent *immediately*.
  2. Keep aggregates **small** (often the root plus a few value objects or child entities).
  3. **Reference other aggregates by id**, not by object (`CustomerId`, not `Customer`).
  4. **Change one aggregate per transaction**; keep other aggregates in step through domain events (**eventual consistency** — the other aggregates catch up shortly afterwards instead of instantly; see System Design Fundamentals).
- **Domain events** are raised inside the aggregate when a state change happens, and published **after** the change is saved. In a database-backed system the reliable way is the **transactional outbox**: write the event to an outbox table in the same transaction as the aggregate, and let a relay publish it (see the Saga notes and Event-Driven Architecture notes).
- **Repositories** exist only for aggregate roots (`OrderRepository`, never `OrderLineRepository`), because outsiders may only reach an `OrderLine` through its `Order`.
- **Application services** (a thin layer above the domain) load an aggregate from its repository, call one domain method, save it — they orchestrate, the aggregate decides (GRASP Controller, section 16.3).
- **Reads can use a different model.** When screens need data joined across aggregates, a separate read model fed by domain events is often simpler — that is **CQRS** (Command Query Responsibility Segregation — separate models for writing and reading; see the CQRS notes); storing the events themselves as the source of truth is **Event Sourcing** (Chapter 11, §11.7.6).



Java example — before/after

```java
// File: lld/ch05/ddd/Main.java
package lld.ch05.ddd;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Objects;
import java.util.Optional;
import java.util.function.Consumer;

/** Domain-Driven Design, tactical building blocks: value objects, entities, an aggregate, domain events, a repository, and two bounded contexts. */
public class Main {

    // ---------------- BEFORE: an anemic model — data bags with setters, rules scattered in services ----------
    static class Before {
        static class OrderLine { public String sku; public int qty; public double price; }
        static class Order {
            public List<OrderLine> lines = new ArrayList<>();
            public String status = "NEW";
            public double total;
        }
        static class OrderService {                       // the only place that "knows" the rules... in theory
            void addLine(Order o, String sku, int qty, double price) {
                OrderLine l = new OrderLine(); l.sku = sku; l.qty = qty; l.price = price;
                o.lines.add(l);
                o.total += qty * price;
            }
        }
    }

    // ---------------- AFTER: the rules live inside the model, in the business's own words ----------------
    static class After {
        /** Value object: no identity, immutable, equal when all fields are equal, validates itself. */
        record Money(long paise) {
            Money { if (paise < 0) throw new IllegalArgumentException("Money cannot be negative"); }
            static Money rupees(long r) { return new Money(r * 100); }
            Money plus(Money o) { return new Money(paise + o.paise); }
            Money times(int n) { return new Money(paise * n); }
            @Override public String toString() { return "Rs %d.%02d".formatted(paise / 100, paise % 100); }
        }
        record OrderId(String value) { }
        record CustomerId(String value) { }               // another aggregate is referenced by ID, not by object
        record Sku(String code) { }

        /** Domain events: facts, named in the past tense, that other parts of the system react to. */
        sealed interface DomainEvent permits OrderPlaced, OrderCancelled { }
        record OrderPlaced(OrderId orderId, CustomerId customer, Map<Sku, Integer> quantities, Money total) implements DomainEvent { }
        record OrderCancelled(OrderId orderId, String reason) implements DomainEvent { }

        enum OrderStatus { DRAFT, PLACED, CANCELLED }

        /** Entity inside the aggregate: identity (lineNo) that is unique only within its Order. */
        static final class OrderLine {
            final int lineNo; final Sku sku; final Money unitPrice; private int quantity;
            OrderLine(int lineNo, Sku sku, Money unitPrice, int quantity) {
                this.lineNo = lineNo; this.sku = sku; this.unitPrice = unitPrice; this.quantity = quantity;
            }
            Money subtotal() { return unitPrice.times(quantity); }
        }

        /** Aggregate root: the only door into the cluster. Every invariant is checked here, in one place. */
        static final class Order {
            static final int MAX_UNITS_PER_SKU = 10;
            private final OrderId id;
            private final CustomerId customer;
```

```
            private final List<OrderLine> lines = new ArrayList<>();
            private final List<DomainEvent> events = new ArrayList<>();
            private OrderStatus status = OrderStatus.DRAFT;

            Order(OrderId id, CustomerId customer) { this.id = id; this.customer = customer; }

            OrderId id() { return id; }
            OrderStatus status() { return status; }

            void addItem(Sku sku, Money unitPrice, int quantity) {
                requireDraft("add items");
                if (quantity <= 0) throw new IllegalArgumentException("Quantity must be positive");
                int already = lines.stream().filter(l -> l.sku.equals(sku)).mapToInt(l -> l.quantity).sum();
                if (already + quantity > MAX_UNITS_PER_SKU)
                    throw new IllegalStateException("At most " + MAX_UNITS_PER_SKU + " units of " + sku.code() + "
per order");
                lines.add(new OrderLine(lines.size() + 1, sku, unitPrice, quantity));
            }

            Money total() { return lines.stream().map(OrderLine::subtotal).reduce(new Money(0), Money::plus); }

            /** Ubiquitous language: "place" an order, not setStatus("PLACED"). */
            void place() {
                requireDraft("place");
                if (lines.isEmpty()) throw new IllegalStateException("Cannot place an empty order");
                status = OrderStatus.PLACED;
                Map<Sku, Integer> q = new HashMap<>();
                lines.forEach(l -> q.merge(l.sku, l.quantity, Integer::sum));
                events.add(new OrderPlaced(id, customer, Map.copyOf(q), total()));
            }

            void cancel(String reason) {
                if (status == OrderStatus.CANCELLED) return;              // idempotent
                status = OrderStatus.CANCELLED;
                events.add(new OrderCancelled(id, reason));
            }

            List<DomainEvent> pullEvents() { List<DomainEvent> e = List.copyOf(events); events.clear(); return e;
}

            private void requireDraft(String action) {
                if (status != OrderStatus.DRAFT) throw new IllegalStateException("Cannot " + action + ": order is
" + status);
            }

            @Override public boolean equals(Object o) { return o instanceof Order other && other.id.equals(id); }
// entity: by identity
            @Override public int hashCode() { return Objects.hash(id); }
        }

        /** Repository: a collection-like interface per AGGREGATE ROOT (no OrderLineRepository). */
        interface OrderRepository {
            Optional<Order> byId(OrderId id);
            void save(Order order);
        }

        /** Saves the aggregate, then publishes its events (in production: outbox table in the same transaction).
*/
        static final class InMemoryOrderRepository implements OrderRepository {
            private final Map<OrderId, Order> store = new HashMap<>();
            private final List<Consumer<DomainEvent>> subscribers = new ArrayList<>();
            void subscribe(Consumer<DomainEvent> s) { subscribers.add(s); }
            @Override public Optional<Order> byId(OrderId id) { return Optional.ofNullable(store.get(id)); }
            @Override public void save(Order order) {
                store.put(order.id(), order);
                for (DomainEvent e : order.pullEvents()) subscribers.forEach(s -> s.accept(e));
            }
        }

        // ---- A second bounded context: Shipping has its OWN model and words ("consignment", weight), not Sales'
Order ----
```

```
        record Consignment(String reference, int parcels, int weightGrams) { }

        /** Anti-corruption layer: translates a Sales event into Shipping's language, so Sales can change freely.
*/
        static final class ShippingTranslator implements Consumer<DomainEvent> {
            private final Map<String, Integer> gramsPerItem;          // Shipping's own catalog: weight, not price
            final List<Consignment> created = new ArrayList<>();
            ShippingTranslator(Map<String, Integer> gramsPerItem) { this.gramsPerItem = gramsPerItem; }
            @Override public void accept(DomainEvent e) {
                if (e instanceof OrderPlaced p) {
                    int grams = p.quantities().entrySet().stream()
                            .mapToInt(en -> gramsPerItem.getOrDefault(en.getKey().code(), 500) *
en.getValue()).sum();
                    Consignment c = new Consignment("SO-" + p.orderId().value(), (grams + 9_999) / 10_000, grams);
                    created.add(c);
                    System.out.println("   shipping context: created " + c);
                } else if (e instanceof OrderCancelled c) {
                    System.out.println("   shipping context: stop consignment SO-" + c.orderId().value() + " (" +
c.reason() + ")");
                }
            }
        }
    }

    public static void main(String[] args) {
        System.out.println("== BEFORE: anemic model ==");
        var svc = new Before.OrderService();
        var o = new Before.Order();
        svc.addLine(o, "TEA-250", 1, 0.1);
        svc.addLine(o, "MUG-01", 1, 0.2);
        o.status = "SHIPPED";
        o.lines.add(new Before.OrderLine());                 // bypasses the service: nothing stops it
        o.lines.get(2).qty = -5;
        System.out.println("total=" + o.total + " status=" + o.status + " lines=" + o.lines.size() + " (a line was
added after shipping, with qty -5)");

        System.out.println("== AFTER: aggregate + value objects + events ==");
        var repo = new After.InMemoryOrderRepository();
        var shipping = new After.ShippingTranslator(Map.of("TEA-250", 260, "MUG-01", 400));
        repo.subscribe(shipping);
        repo.subscribe(e -> System.out.println("   event published: " + e.getClass().getSimpleName()));

        var order = new After.Order(new After.OrderId("1001"), new After.CustomerId("C-42"));
        order.addItem(new After.Sku("TEA-250"), After.Money.rupees(180), 2);
        order.addItem(new After.Sku("MUG-01"), new After.Money(34950), 1);
        try { order.addItem(new After.Sku("TEA-250"), After.Money.rupees(180), 9); }
        catch (IllegalStateException e) { System.out.println("rejected: " + e.getMessage()); }
        System.out.println("total " + order.total() + " (money in paise: 0.1 + 0.2 problems cannot happen)");
        order.place();
        repo.save(order);
        try { order.addItem(new After.Sku("MUG-01"), new After.Money(34950), 1); }
        catch (IllegalStateException e) { System.out.println("rejected: " + e.getMessage()); }

        System.out.println("== identity vs value equality ==");
        System.out.println("Money(100) equals Money(100)? " + new After.Money(100).equals(new After.Money(100)) +
"   (value object)");
        var sameId = new After.Order(new After.OrderId("1001"), new After.CustomerId("someone-else"));
        System.out.println("two Order objects with id 1001 equal? " + order.equals(sameId) + "   (entity: identity
decides)");
        System.out.println("loaded from repository: " + repo.byId(new
After.OrderId("1001")).map(After.Order::status).orElseThrow());
        order.cancel("customer request");
        repo.save(order);
    }
}
```

Output:

```
== BEFORE: anemic model ==
total=0.30000000000000004 status=SHIPPED lines=3 (a line was added after shipping, with qty -5)
== AFTER: aggregate + value objects + events ==
rejected: At most 10 units of TEA-250 per order
total Rs 709.50 (money in paise: 0.1 + 0.2 problems cannot happen)
   shipping context: created Consignment[reference=SO-1001, parcels=1, weightGrams=920]
   event published: OrderPlaced
rejected: Cannot add items: order is PLACED
== identity vs value equality ==
Money(100) equals Money(100)? true   (value object)
two Order objects with id 1001 equal? true   (entity: identity decides)
loaded from repository: PLACED
   shipping context: stop consignment SO-1001 (customer request)
   event published: OrderCancelled
```

Before, the order is a bag of public fields: `double` money prints `0.30000000000000004`, the status is a free-form string, and code outside the service added a line with quantity −5 *after* the order "shipped" — nothing could stop it. *After*, `Money` is a value object (paise — 1/100 of a rupee — in a `long`, never negative, equal by value), `Order` is the aggregate root that owns its `OrderLine` entities and refuses changes once placed or beyond 10 units per SKU (Stock Keeping Unit — a product's unique code), `place()` speaks the business language and records an `OrderPlaced` event, and the repository publishes events only after saving. The Shipping context never sees Sales' `Order`: its translator turns the event into a `Consignment` with Shipping's own concepts (weight, parcels), using Shipping's own weight catalog. Two `Order` objects with the same id are equal (entity), two `Money(100)` values are equal (value object).

Trade-offs

DDD brings	DDD costs
Code that reads like the business; rules in one place	Time with domain experts; learning the vocabulary
Clear boundaries for teams and services	More types (value objects, events, translators)
Aggregates give a clear transaction and locking unit	Eventual consistency between aggregates must be designed and explained
Models can change independently per context	Translation code at every context border
Rich model is easy to unit-test (test one class in isolation) without a database	ORMs sometimes fight rich models (private setters, value object mapping)

When not to use it

- **CRUD-shaped apps** (Create, Read, Update, Delete — apps that mostly store and show records: forms over tables, an admin panel) with few rules — a simple layered design is enough; tactical DDD would be ceremony (KISS, YAGNI).
- **Small teams with one small model** — strategic splitting pays off when there are several teams or sub-domains.
- **Technical problems** (a compiler, a video encoder) where the complexity is algorithmic, not in business rules.
- Still worth borrowing even then: **value objects** and **ubiquitous naming** cost little and help everywhere.

Common mistakes

- **Anemic aggregates** — entities with public setters and "services" holding the rules. That is the *before* code with DDD names on it.
- **Huge aggregates** ("Customer" containing all orders, addresses and payments) — every change locks everything; keep aggregates small and reference others by id.
- **One repository per table** instead of per aggregate root.
- **Publishing events before the transaction commits** — consumers react to changes that may be rolled back (undone instead of committed, i.e. made permanent); use an outbox.

- **Sharing one domain model across services** "to avoid duplication" — it couples the teams; duplicate the small bits each context needs (DRY applies *within* a context, not across).
- **Treating DDD as folders** (`entities/`, `valueobjects/`) rather than as modelling with domain experts.
- Using **primitives for domain concepts** (`String email`, `double price`) — "primitive obsession" (Chapter 7).

Real-world examples

- **Microservice boundaries** at many companies are drawn along bounded contexts (orders, payments, shipping, catalog) — the default advice in Sam Newman's *Building Microservices* and in Microsoft's .NET microservices architecture guide, which uses DDD aggregates and domain events in its reference application *eShopOnContainers*.
- **Payments**: a `Payment` aggregate with an `Amount` value object and events such as `PaymentAuthorized`, `PaymentCaptured`, `PaymentRefunded`, consumed by ledger and notification contexts.
- **Java libraries**: `java.time` types (`LocalDate`, `Duration`) and `BigDecimal` are value objects in all but name; frameworks such as Axon (used in the CQRS notes' example) are built around aggregates and events.
- In this book: the grocery POS (point-of-sale checkout system, Chapter 19) keeps `Money` and tenders as value types and a `Sale` that guards its own states; the shipping locker (Chapter 14, §14.7) separates `Compartment` (reusable door) from `Booking` (one parcel's lifecycle).

In the interview

> **30-second answer:** "DDD models the software on the business. Strategically, I split the domain into bounded contexts — each with its own model and ubiquitous language, like Sales and Shipping — and connect them through events or APIs, translating at the border with an anti-corruption layer. Tactically, inside a context I use entities for things with identity, immutable value objects for things like Money, and small aggregates whose root enforces the invariants; one aggregate changes per transaction, others follow through domain events, and each aggregate root has a repository."

Follow-ups:

- *Q: Entity or value object — how do you decide?* A: Ask whether you track it over time by identity. An `Order` is (id 1001 stays 1001); an `Address` or `Money` is not — you replace it with a new value.
- *Q: How big should an aggregate be?* A: As small as the invariants allow; reference other aggregates by id and use events for rules that can be eventually consistent.
- *Q: How do bounded contexts relate to microservices?* A: A context is a good candidate boundary for a service (often one service per context), but a context can also be a module inside a monolith.
- *Q: How do you keep two aggregates consistent?* A: Change one per transaction, publish a domain event via an outbox, let a handler (or a saga — a chain of local transactions with compensating "undo" steps — for multi-step processes) update the other.

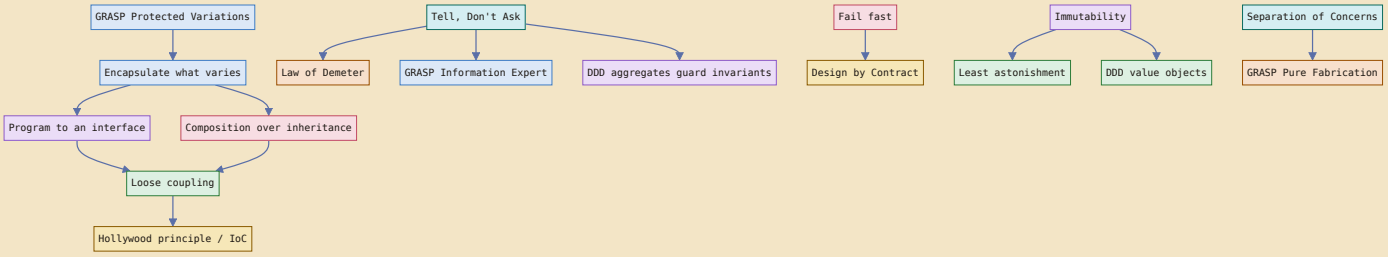
Self-check

1. Q: Two `Money(₹100)` objects — equal? Two customers named "Asha Rao" — equal? → A: Money yes (value object); customers only if they have the same id (entities).
2. Q: Why must outsiders not hold a reference to an `OrderLine`? → A: They could change it without the `Order` root checking its invariants.
3. Q: Why reference `CustomerId` instead of `Customer` inside `Order`? → A: Keeps aggregates small and independent; avoids loading or locking the customer when changing an order.
4. Q: What is an anti-corruption layer for? → A: Translating another context's or a legacy system's model into ours so its concepts don't leak in.
5. Q: When do you publish a domain event? → A: After the aggregate's change is committed — reliably via a transactional outbox.

TL;DR

Speak the business's language in code; split the domain into bounded contexts with their own models and translate at the borders; inside one context use entities (identity), immutable value objects (values), small aggregates whose root guards the invariants, domain events for everything that can happen later, and one repository per aggregate root.

Chapter summary — how the principles fit together



Principle	Main goal	Typical smell it fixes	Key Java tool
DRY	One home per rule	Duplicated rules drifting	Value objects, constants, methods
KISS	Simplicity	Over-engineering	Plain code, `switch` expressions
YAGNI	Build only what's needed	Speculative generality	Delete unused code
Law of Demeter	Hide structure	Train wrecks	Intent-revealing delegate methods
Tell, Don't Ask	Behaviour with data	Anemic model, feature envy	Command methods on the owner
Composition over inheritance	Flexible reuse	Fragile base class	Forwarding/decorators
Program to an interface	Substitutability	Hard-wired concrete deps	Interfaces + DI
Encapsulate what varies	Isolate change	Growing type-switches	Strategy, lambdas
Loose coupling	Independent change	God orchestrators	Observer, events, DI
Hollywood / IoC	Framework owns flow	Duplicated skeletons	Template Method, callbacks, DI
Separation of Concerns	One concern per unit	Spaghetti code	Layers, MVC
Least Astonishment	Predictability	Hidden side effects	Honest names, `Optional`
Fail fast	Early, clear errors	Distant NPEs (NullPointerExceptions)	Guard clauses, `requireNonNull`
Immutability	Safe sharing	Aliasing, races	Records, `List.copyOf`
Design by Contract	Clear obligations	Magic error values	Pre/postconditions, invariants
GRASP	Place responsibilities	God classes	Expert, Creator, Controller, ...
DDD	Model the business; clear boundaries	Anemic model, big ball of mud, primitive obsession	Records as value objects, aggregate roots, domain events, repositories

Tensions to remember (and mention in interviews):

- DRY vs loose coupling — sharing code couples its users; sometimes duplicate across boundaries.
- YAGNI vs Encapsulate-what-varies/OCP — only abstract variation that is real or clearly predicted.
- Law of Demeter vs cohesion — too many delegate methods create a Middle Man; move behaviour instead.
- Information Expert vs Separation of Concerns — domain objects compute, fabrications persist.
- DRY vs bounded contexts — share nothing but events and ids across contexts; a little duplication beats a shared model that couples teams.

Chapter self-check

1. Q: Which principle is violated by `order.getCustomer().getAddress().getCity()` inside a service that then decides shipping? → A: Law of Demeter (and Tell-Don't-Ask); move the decision to where the data is or ask `order.shippingZone()`.
2. Q: A `find` method that inserts a row violates? → A: Least Astonishment (and Command-Query Separation).
3. Q: A `HashSet` subclass double-counts in `addAll` — which principle? → A: Composition over inheritance (fragile base class).
4. Q: Which GRASP guideline says "put `total()` on `Order`"? → A: Information Expert.
5. Q: What makes a record *truly* immutable when it holds a list? → A: Defensively copy it with `List.copyOf` in the compact constructor.
6. Q: In DDD, why is `Money` a value object but `Order` an entity? → A: Money is defined by its amount and currency and is replaced, not changed; an order keeps its identity while its lines and status change.

TL;DR (chapter)

Keep rules in one place, keep things simple, build only what's needed, let objects own their behaviour and talk only to friends, depend on small interfaces and compose, let frameworks call you, separate concerns, don't surprise people, fail early, prefer immutable values, state your contracts, use GRASP to decide who does what — and let DDD's bounded contexts, aggregates and value objects decide which classes exist.

Chapter 6 — Clean Code

Design principles (Chapters 4–5) decide *which classes exist and how they relate*. **Clean code** is about what happens *inside* those classes: names, functions, comments, error handling, tests. In machine-coding rounds (interview rounds where you build a small, working program in about 60–120 minutes), interviewers read your code line by line — clean code is often what separates "hire" from "no hire" when two candidates reach similar designs.

This chapter covers every major topic of Robert C. Martin's *Clean Code* (2008), in the book's order (Chapters 1–17; the two long case studies, Chapters 15–16, are summarised in §13), explained in our own words with runnable Java 21 before/after programs. It finishes with three practical best-practice topics: a **code review checklist**, an **error-handling strategy** for real services and **testing strategies** (unit vs integration tests, test doubles, mocks).

Key terms (defined once here):

Term	Meaning
Clean code	Code that is easy to read, understand and change by someone other than its author. It does one thing well, has tests, and has no surprises.
Code smell	A surface symptom that *usually* signals a deeper design problem (e.g. a 200-line method). Not a bug by itself.
Refactoring	Changing the internal structure of code without changing its observable behaviour (Chapter 7).
Level of abstraction	How "high-level" a statement is. `processPayroll()` is high; `sb.append(',')` is low.
Side effect	Any change a function makes besides returning a value (fields, files, globals, output).
Test double	Any object that stands in for a real dependency during a test (dummy, stub, spy, mock, fake).
TDD	Test-Driven Development: write a failing test, make it pass, refactor — in very short cycles.

What is clean code, and why bother?

What it is

Well-known programmers describe clean code in similar ways: it reads like well-written prose, it does one thing, it is elegant and efficient, it has tests, it contains no duplication, it expresses its intent, and it looks like it was written by someone who **cares**. A simple working definition: *code whose intent is obvious to the next reader, and which is easy to change safely.*

Analogy: a well-organised kitchen. Anyone can walk in, find the knives, cook, and clean up. A messy kitchen still "works" for its owner — until someone else has to cook in it.

The problem it solves

Messy code slows teams down more and more over time. Every change takes longer because nobody understands the code, and each change adds more mess. Eventually productivity approaches zero and the team asks for a "grand redesign", which often reproduces the same mess. We read code far more than we write it (the ratio is often quoted as more than 10:1), so making code easy to read makes writing faster too. "I'll clean it up later" rarely happens — *later equals never* (a saying known as LeBlanc's law).

How it works

1. Treat readability as a first-class requirement, not decoration.

2. Follow the **Boy Scout Rule**: *always leave the code a little cleaner than you found it* — rename one variable, split one long function, delete one dead comment each time you touch a file.
3. Write tests so you can clean without fear.
4. Clean in small steps (successive refinement): first make it work, then make it right. Never ship "it works, but it's a mess".

Trade-offs

Clean code takes slightly longer *the first time* you write it; it saves time on every later read and change. For truly throwaway code (a one-off script) the balance is different.

When not to use it

There is no case where unreadable code is a goal. But avoid **gold-plating** — endlessly polishing code that is already clear and not changing.

Common mistakes

- Thinking "working" means "done".
- Blaming deadlines: going fast by making a mess is an illusion; the mess slows you down immediately.
- Big-bang rewrites instead of continuous small improvements.

Real-world examples

- **Google's code review culture** requires reviewer approval for readability, and it has a "readability" certification process per language.
- **Linters and formatters** (tools that automatically flag suspicious code or rewrite its layout) running in CI — Continuous Integration, a server that builds and tests every change automatically — (Checkstyle, SpotBugs, SonarQube, google-java-format) enforce parts of clean code automatically.

In the interview

> **30-second answer:** "Clean code is code whose intent is obvious and which is cheap to change. In a machine-coding round I use intention-revealing names, small single-purpose functions, clear exceptions instead of error codes, immutable value objects (small objects such as `Money` that never change after creation) and a few tests — and I leave time to refactor once it works."

Self-check

1. Q: What is the Boy Scout Rule? → A: Leave the code a little cleaner than you found it.
2. Q: Why does readability matter more than writing speed? → A: Code is read far more often than it is written.

TL;DR

Clean code = obvious intent + safe to change. Clean a little every time you touch code.

---

1. Meaningful Names

What it is

Names are everywhere — variables, functions, parameters, classes, packages. A good name answers three questions: **why does it exist, what does it do, how is it used**. If a name needs a comment to explain it, the name is not good enough.

Analogy: labels on moving boxes. "Kitchen — glasses — fragile" helps; "Box 7" doesn't.

The problem it solves

Bad names force readers to keep a **mental map** ("`d` is elapsed days, `i1` is status, 4 means done"). Every reader pays that cost every time, and misunderstanding causes bugs.

How it works — the rules

Rule	Bad	Good
Reveal intention	`int d;`	`int elapsedTimeInDays;`
Avoid disinformation (names that suggest something false)	`accountList` for a `Set`; `l` / `O` that look like 1/0	`accounts`
Make meaningful distinctions — no number series or noise words	`a1, a2`; `ProductInfo` vs `ProductData`; `nameString`	`source, destination`; one `Product`
Use pronounceable names	`crtdTs`	`createdAt`
Use searchable names — avoid single letters and bare numbers except tiny local scopes	`7`, `e`	`MAX_LOGIN_ATTEMPTS`, `employee`
Avoid encodings — no Hungarian notation (type codes in names), no `m_` prefixes, no `I` prefix on interfaces	`strName`, `m_desc`, `IShapeFactory`	`name`, `description`, `ShapeFactory` (+ `ShapeFactoryImpl` or better, a specific name)
Avoid mental mapping	`r` for "lower-cased URL without host"	`normalizedPath`
Class names = nouns	`Manager`, `Processor`, `Data`, `Info` (vague)	`Customer`, `Invoice`, `AddressParser`
Method names = verbs; accessors `get` / `is` / `has`	`data()`	`postPayment()`, `isPosted()`
Prefer named static factories (static creation methods like `of...`) to overloaded constructors (several constructors with different parameter lists)	`new Money(23)` (rupees? paise, 1/100 of a rupee?)	`Money.ofRupees(23)`
Don't be cute	`yeetAll()`, `nuke()`	`deleteAll()`, `terminate()`
One word per concept	`fetch` / `retrieve` / `get` in sibling classes	pick one, e.g. `get`
Don't pun (same word, different meaning)	`add` for both "sum" and "append to collection"	`append`, `insert`
Use solution-domain names for technical concepts	—	`OrderRepository`, `RetryPolicy`, `EventQueue` (programmers know these)
Use problem-domain names for business concepts	—	`Premium`, `Policy`, `Claim`
Add meaningful context	lone `state`	`shippingState` or better, an `Address` class with `state`
Don't add gratuitous context	`AcmeShopCustomerAddress` in an app called "Acme Shop"	`Address`

Length rule of thumb: the **longer the scope, the longer the name**. `i` is fine for a 3-line loop; a field used across a class deserves a full name.

Java example — before/after

```java
// File: lld/ch06/names/Main.java
package lld.ch06.names;

import java.time.Duration;
import java.util.ArrayList;
import java.util.List;

/** Meaningful names: the same logic, before and after renaming. */
public class Main {

    // ---------------- BEFORE ----------------
    static class Before {
        static class DtaRcrd102 {                       // unpronounceable, encoded, meaningless
            String strNm; int i1; int i2; boolean flg;
        }
        static class Mgr {                               // vague noun
            List<DtaRcrd102> lst = new ArrayList<>();    // "lst" of what?
            // what is 4? what is 18000? why flg?
            List<DtaRcrd102> getThem() {
                List<DtaRcrd102> l1 = new ArrayList<>();
                for (DtaRcrd102 x : lst) if (x.i1 == 4 && x.i2 > 18000 && !x.flg) l1.add(x);
                return l1;
            }
            int d;                                       // elapsed time in days? seconds? since what?
        }
    }

    // ---------------- AFTER ----------------
    static class After {
        enum TaskStatus { OPEN, IN_PROGRESS, BLOCKED, DONE }

        /** Class name = noun. Field names = what they hold, with units where it matters. */
        record Task(String title, TaskStatus status, Duration timeSpent, boolean archived) {
            boolean isCompleted() { return status == TaskStatus.DONE; }          // boolean method reads as a
question
            boolean tookLongerThan(Duration limit) { return timeSpent.compareTo(limit) > 0; }
        }

        static final class TaskBoard {
            private static final Duration LONG_RUNNING = Duration.ofHours(5);  // named constant, not a magic
number
            private final List<Task> tasks = new ArrayList<>();
            void add(Task task) { tasks.add(task); }                             // method name = verb
            List<Task> findCompletedLongRunningTasks() {                         // says exactly what it returns
                List<Task> result = new ArrayList<>();
                for (Task task : tasks)
                    if (task.isCompleted() && task.tookLongerThan(LONG_RUNNING) && !task.archived())
result.add(task);
                return result;
            }
        }

        /** Static factory methods with descriptive names beat overloaded constructors. */
        record Temperature(double kelvin) {
            static Temperature ofCelsius(double c) { return new Temperature(c + 273.15); }
            static Temperature ofFahrenheit(double f) { return new Temperature((f - 32) * 5 / 9 + 273.15); }
        }
    }

    public static void main(String[] args) {
        var mgr = new Before.Mgr();
        var r = new Before.DtaRcrd102(); r.strNm = "Deploy"; r.i1 = 4; r.i2 = 20000; mgr.lst.add(r);
        System.out.println("BEFORE getThem(): " + mgr.getThem().size() + " record(s) -- but what are they?");

        var board = new After.TaskBoard();
        board.add(new After.Task("Deploy", After.TaskStatus.DONE, Duration.ofHours(6), false));
        board.add(new After.Task("Write docs", After.TaskStatus.DONE, Duration.ofHours(1), false));
        System.out.println("AFTER findCompletedLongRunningTasks(): " + board.findCompletedLongRunningTasks());
        System.out.printf("Temperature.ofCelsius(25) = %.2f K, ofFahrenheit(212) = %.2f K%n",
```

```
                After.Temperature.ofCelsius(25).kelvin(), After.Temperature.ofFahrenheit(212).kelvin());
    }
}
```

Output:

```
BEFORE getThem(): 1 record(s) -- but what are they?
AFTER findCompletedLongRunningTasks(): [Task[title=Deploy, status=DONE, timeSpent=PT6H, archived=false]]
Temperature.ofCelsius(25) = 298.15 K, ofFahrenheit(212) = 373.15 K
```

The logic is the same shape — filter a list by three conditions — but *after* you can read the rule aloud: "completed tasks that took longer than five hours and aren't archived". Magic numbers (unexplained literal values dropped into code) became a named constant, a status code became an enum (a type with a fixed set of named values), and `Temperature.ofCelsius(25)` is unambiguous where `new Temperature(25)` would not be.

Trade-offs

Longer names cost a few keystrokes (IDEs — Integrated Development Environments such as IntelliJ IDEA — autocomplete them). Very long names can hint the function does too much — split it rather than abbreviate.

When not to use it

Conventional short names are fine where universally understood: `i`/`j` loop indexes in tiny loops, `e` for a caught exception, `x`/`y` for coordinates, `T`/`K`/`V` for generics (type parameters, as in `List<T>`, that let one class work with many types).

Common mistakes

- Renaming is cheap — not doing it because "everyone knows what `tmp2` means".
- Abbreviations only the author understands (`calcAmtPrc`).
- Inconsistent vocabulary across a codebase (`customer`, `client`, `user` for the same concept).

Real-world examples

- The JDK's naming: `isEmpty()`, `getOrDefault()`, `computeIfAbsent()`, `List.of()`, `Path.of()`, `Duration.ofMinutes()` — verbs for methods, named static factories.
- **Google Java Style Guide** and Oracle's code conventions standardise casing (`UpperCamelCase` classes, `lowerCamelCase` methods, `UPPER_SNAKE_CASE` constants).

In the interview

> **30-second answer:** "I name things so the code reads like the requirement: nouns for classes, verbs for methods, booleans as questions, named constants instead of magic numbers, enums instead of status codes, and one consistent word per concept."

Follow-ups:

- *Q: Should interfaces start with `I`?* A: Not in Java convention; name the role (`PaymentGateway`) and give implementations specific names (`RazorpayGateway`).
- *Q: When is a short name okay?* A: When the scope is tiny and the meaning conventional (`i` in a short loop).

Self-check

1. Q: What's wrong with `ProductInfo` and `ProductData` in one codebase? → A: Noise words; no meaningful distinction.
2. Q: Why prefer `Temperature.ofCelsius(x)` over a constructor? → A: The name says what the argument means.
3. Q: What's a pun in naming? → A: Using one word for two different meanings.

TL;DR

Names should reveal intent, be pronounceable and searchable, use one word per concept, and let the code read like the requirement.

---

2. Functions

What it is

Functions (methods) are the first line of organisation in any program. Clean functions are **small**, **do one thing**, stay at **one level of abstraction**, take **few arguments**, have **no hidden side effects**, and either **do something or answer something** — never both.

Analogy: a recipe with numbered steps ("make the sauce", "boil pasta", "combine") where each step has its own sub-recipe. You'd never write one 400-line paragraph.

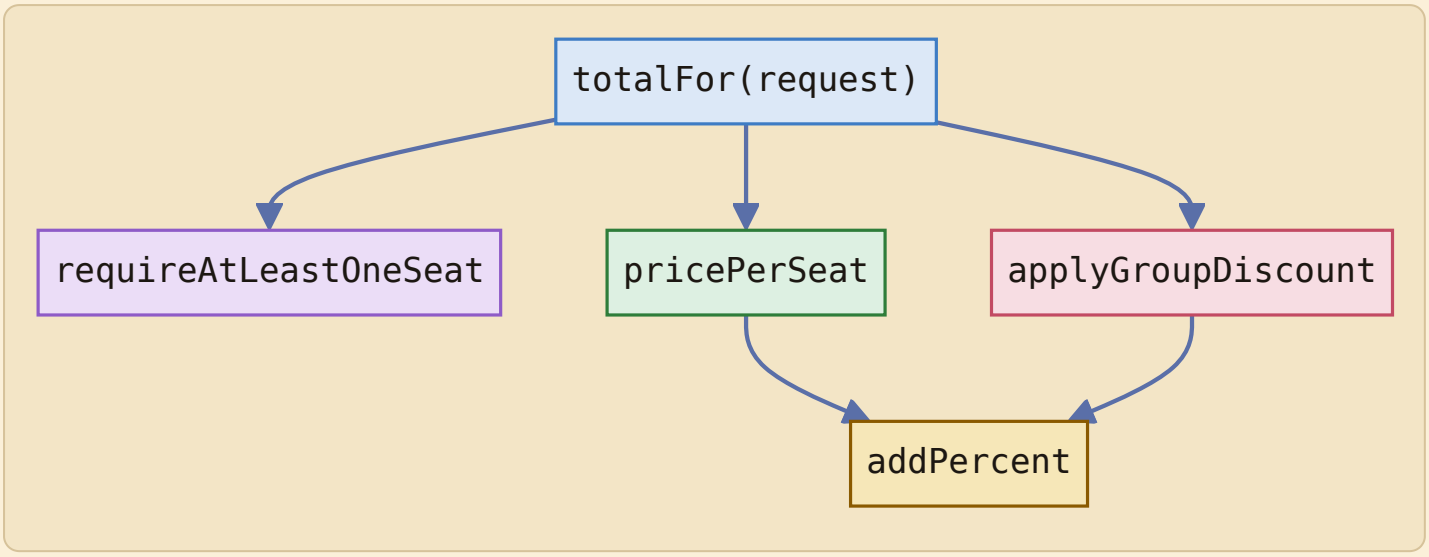
The problem it solves

Long functions mix levels of detail, hide bugs, can't be reused, can't be named well, and can't be tested in pieces. Flag arguments and output arguments confuse callers; hidden side effects create surprising bugs; error codes clutter every caller with `if` checks.

How it works — the rules

1. **Small, then smaller.** The book says functions should hardly ever reach 20 lines, and praises functions of just two to four lines. Blocks inside `if` / `else` / `while` should ideally be one line — usually a call to a well-named function. Indentation should rarely exceed one or two levels.
2. **Do one thing.** A function does one thing if you can't extract another function from it with a name that isn't just a restatement of its body. If a function has "sections" (declarations, initialisation, processing), it's doing several things.
3. **One level of abstraction per function.** Don't mix `getHtml()` (high) with `.append("\n")` (low) in the same body.
4. **The Stepdown Rule.** Code should read top-down like a narrative: each function is followed by the functions it calls, one level of abstraction lower. "To do X, we do A, then B. To do A, we …".
5. **Switch statements.** A `switch` on a type code tends to be repeated in many places and to grow. Tolerate one only if it appears **once**, is buried in a low-level class (e.g. inside a factory — a class or method whose job is creating objects — that creates polymorphic objects, i.e. objects of different classes used through one shared interface), and is never repeated. Otherwise replace it with polymorphism (or enum data/behaviour).
6. **Use descriptive names.** A long descriptive name beats a short enigmatic one and a long comment. Small functions are easier to name.
7. **Arguments: fewer is better.** Zero (niladic) is ideal, then one (monadic), then two (dyadic); three (triadic) should be avoided where possible; more than three (polyadic) requires very special justification — and then shouldn't be used anyway. Arguments are hard to read and each one multiplies test cases.
    - Common one-argument forms: asking a question (`isValid(email)`), transforming (`Path resolve(name)`), or an event (`onLoginFailed(attempts)`) — make the kind obvious.
    - **Flag arguments** (`render(true)`) are ugly: they announce the function does two things. Split into two functions or pass an enum.
    - **Argument objects**: when several arguments travel together, wrap them (`book(BookingRequest request)` instead of `book(show, seatType, count, day)`).
    - **Argument lists** (varargs — a parameter like `Object... args` that accepts any number of values) count as one argument (`String.format(fmt, args...)`).
    - **Verbs and keywords**: encode argument roles in the name (`copyFromTo(source, target)`, `renameFileTo(newName)`).

8. **Have no side effects.** A function named `checkPassword` must not also initialise the session (the server-side record of a logged-in user). Hidden side effects create **temporal coupling** (it only works if called at the right time). If a side effect is necessary, put it in the name — or better, separate it.
9. **Avoid output arguments.** `addSignature(letter)` — does it add a signature *to* the letter, or the letter *to* something? Prefer `letter.addSignature()` (change the state of the owning object).
10. **Command-Query Separation (CQS).** A function should either *change state* (command, returns `void`) or *return information* (query, no state change), not both. `if (enable("darkMode"))` is confusing: is it asking whether it's enabled, or enabling it?
11. **Prefer exceptions to returning error codes.** Error codes force immediate `if` checks at every call site (each place in the code that calls the function) and deeply nested code; exceptions separate the happy path (the normal, no-error flow) from error handling.
12. **Extract try/catch blocks.** Put the body of a `try` into its own function, so the function that handles errors does *only* error handling ("error handling is one thing": if `try` appears, it should be the first word in the function and nothing should follow the `catch`/`finally` blocks).
13. **Avoid an "error code magnet".** A central `enum Error` that everything imports forces recompiles and discourages new codes; exception subclasses can be added without touching existing code.
14. **Don't Repeat Yourself** — duplication is the root of much evil in software (see Chapter 5).
15. **Structured programming** (Dijkstra's one-entry, one-exit rule: one `return`, no `break`/`continue` in loops, never `goto`) matters mostly for large functions; small functions can use an occasional early `return`, `break` or `continue` when that makes them clearer (but still never `goto`).
16. **How to get there:** nobody writes clean functions the first time. Write a working rough draft with tests, then refine — split, rename, remove duplication — keeping the tests green (green = all tests passing).



Java example — before/after

```java
// File: lld/ch06/functions/Main.java
package lld.ch06.functions;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/** Clean functions: small, one thing, one level of abstraction, few args, no flags, no hidden side effects, CQS, exceptions. */
@SuppressWarnings("serial") // the nested exception classes are never serialized
public class Main {

    // ---------------- BEFORE ----------------
    static class Before {
        static final Map<String, Object> SESSION = new HashMap<>();

        /**
         * Returns total in rupees, or -1 for bad input.
         * Does validation, pricing, surcharges, discounts, logging and session updates -- all in one place.
         */
        static int book(String seatType, int count, boolean weekend, List<String> log) {   // flag arg + output arg
            if (count <= 0) return -1;                                                      // error code
            int price;
            switch (seatType) {                                                             // switch on a type code
                case "SILVER": price = 150; break;
                case "GOLD": price = 250; break;
                case "RECLINER": price = 450; break;
                default: return -1;
            }
            if (weekend) price = price + price * 20 / 100;
            int total = price * count;
            if (count >= 4) total = total - total * 10 / 100;
            log.add("booked " + count + " " + seatType);                                    // mutates the caller's list
            SESSION.put("lastTotal", total);                                                // hidden side effect
            return total;
        }
    }

    // ---------------- AFTER ----------------
    static class After {
        /** The type-code switch is replaced by data on the enum; parsing happens once, at the boundary. */
        enum SeatType {
            SILVER(150), GOLD(250), RECLINER(450);
            final int basePrice;
            SeatType(int basePrice) { this.basePrice = basePrice; }
            static SeatType parse(String raw) {
                try { return valueOf(raw); }
                catch (IllegalArgumentException e) { throw new InvalidBookingException("unknown seat type '" + raw + "'"); }
            }
        }
        enum DayType {                          // replaces the boolean flag: callers write DayType.WEEKEND, not 'true'
            WEEKDAY(0), WEEKEND(20);
            final int surchargePercent;
            DayType(int p) { surchargePercent = p; }
        }
        /** Argument object: three related arguments become one concept. */
        record BookingRequest(SeatType seat, int seats, DayType day) {}

        static final class InvalidBookingException extends RuntimeException {
            InvalidBookingException(String message) { super(message); }
        }

        /** Stepdown rule: read top to bottom, each function one level more detailed than the one above. */
        static final class TicketPricer {
```

```
            private static final int GROUP_SIZE = 4;
            private static final int GROUP_DISCOUNT_PERCENT = 10;

            int totalFor(BookingRequest request) {                      // level 1: the story
                requireAtLeastOneSeat(request);
                int total = pricePerSeat(request) * request.seats();
                return applyGroupDiscount(total, request.seats());
            }

            private void requireAtLeastOneSeat(BookingRequest r) {      // level 2: the details
                if (r.seats() <= 0) throw new InvalidBookingException("seats must be positive, was " + r.seats());
            }
            private int pricePerSeat(BookingRequest r) {
                return addPercent(r.seat().basePrice, r.day().surchargePercent);
            }
            private int applyGroupDiscount(int total, int seats) {
                return seats >= GROUP_SIZE ? addPercent(total, -GROUP_DISCOUNT_PERCENT) : total;
            }
            private static int addPercent(int amount, int percent) {    // level 3: arithmetic
                return amount + amount * percent / 100;
            }
        }

        /** Command-Query Separation: book() changes state and returns nothing; history() answers and changes
    nothing. */
        static final class BookingService {
            private final TicketPricer pricer = new TicketPricer();
            private final List<String> history = new ArrayList<>();

            void book(BookingRequest request) {
                int total = pricer.totalFor(request);
                history.add(request.seats() + " x " + request.seat() + " on " + request.day() + " = Rs " + total);
            }
            List<String> history() { return List.copyOf(history); }
        }

        /** Error handling is "one thing": this function only translates failures for the UI. */
        static String bookFromForm(BookingService service, String seat, int seats, DayType day) {
            try {
                service.book(new BookingRequest(SeatType.parse(seat), seats, day));
                return "OK";
            } catch (InvalidBookingException e) {
                return "Sorry: " + e.getMessage();
            }
        }
    }

    public static void main(String[] args) {
        System.out.println("== BEFORE ==");
        List<String> log = new ArrayList<>();
        int result = Before.book("GOLD", 4, true, log);           // what does 'true' mean here?
        System.out.println("total=" + result + " log=" + log + " session=" + Before.SESSION);
        int bad = Before.book("PLATINUM", 2, false, log);
        System.out.println("bad seat returns " + bad + " -- caller must remember to check for -1");

        System.out.println("== AFTER ==");
        var service = new After.BookingService();
        System.out.println(After.bookFromForm(service, "GOLD", 4, After.DayType.WEEKEND));
        System.out.println(After.bookFromForm(service, "SILVER", 2, After.DayType.WEEKDAY));
        System.out.println(After.bookFromForm(service, "PLATINUM", 2, After.DayType.WEEKDAY));
        System.out.println(After.bookFromForm(service, "GOLD", 0, After.DayType.WEEKDAY));
        service.history().forEach(h -> System.out.println("  history: " + h));
    }
}
```

Output:

```
== BEFORE ==
total=1080 log=[booked 4 GOLD] session={lastTotal=1080}
bad seat returns -1 -- caller must remember to check for -1
== AFTER ==
OK
OK
Sorry: unknown seat type 'PLATINUM'
Sorry: seats must be positive, was 0
  history: 4 x GOLD on WEEKEND = Rs 1080
  history: 2 x SILVER on WEEKDAY = Rs 300
```

What changed and why:

- The **boolean flag** `weekend` became `DayType`, so call sites read `DayType.WEEKEND`, not a mysterious `true`.
- The **type-code switch** disappeared: each `SeatType` carries its base price; string parsing happens once at the boundary (`SeatType.parse`).
- The **error codes** (`-1`) became `InvalidBookingException` with a message saying exactly what was wrong.
- The **output argument** (`log`) and the **hidden side effect** (`SESSION`) became explicit state owned by `BookingService`.
- **CQS**: `book` is a command (returns `void`); `history` is a query (returns a copy, changes nothing). (Many real APIs pragmatically let a command return the created ID — that's a widely accepted exception.)
- **Stepdown**: `totalFor` reads as the story; the helpers below it are one level more detailed.
- **Error handling is one thing**: `bookFromForm` only translates failures into a UI message.

Trade-offs

Many small functions	Few large functions
Each is nameable, testable, reusable	Everything visible in one place
Need good names to navigate	Hard to understand, test, change
Too-tiny splitting can scatter logic	Mixed abstraction levels

When not to use it

Don't split a genuinely simple 8-line function into 8 one-liners just to hit a number; the goal is clarity, not a line count. Performance-critical inner loops rarely need manual inlining (copying a small method's body into its caller) in Java — the JIT (the Just-In-Time compiler inside the JVM, the Java Virtual Machine that runs Java programs, which turns frequently run bytecode into machine code while the program runs) inlines small methods — so size is almost never a real performance excuse.

Common mistakes

- Boolean parameters in public APIs (the methods and classes other code is meant to call).
- Getters that change state (breaking CQS and least astonishment).
- Catching exceptions deep inside and returning `null`.
- Functions whose name says one thing and body does three.

Real-world examples

- `java.util.Collections.sort(list)` vs `list.sort(comparator)` — command methods return `void`.
- `Iterator.next()` (an `Iterator` walks through a collection one element at a time) is a well-known pragmatic CQS exception (it both advances and returns); `Queue.poll()` likewise. Know them, but don't design new APIs that way without reason.
- `Files.readString`, `Files.writeString` — tiny, single-purpose APIs.

In the interview

> **30-second answer:** "I keep functions small and single-purpose, at one level of abstraction, top-down by the stepdown rule. I avoid flag arguments with enums or separate methods, keep argument counts low with parameter objects, separate commands from queries, and throw descriptive exceptions instead of returning error codes."

Follow-ups:

- *Q: How small is small?* A: Small enough to have one clear name and one level of abstraction — often under ~20 lines, frequently 3–10.
- *Q: What's wrong with* `boolean` *parameters?* A: The call site is unreadable and the function does two things.
- *Q: What's CQS?* A: Commands change state and return nothing; queries return data and change nothing.

Self-check

1. Q: How do you know a function does "one thing"? → A: You can't extract another meaningful function from it that isn't a mere restatement.
2. Q: When is a `switch` acceptable? → A: Once, buried in a factory/low level, never repeated.
3. Q: What replaces 4+ related arguments? → A: An argument (parameter) object.

TL;DR

Small, one thing, one level, top-down, ≤2–3 args, no flags, no hidden side effects, commands or queries, exceptions not codes.

---

3. Comments

What it is

Comments are, at best, a necessary evil: we write them to make up for our failure to express ourselves in code. Code changes; comments often don't, so they drift and **lie**. The only truly accurate description of behaviour is the code itself. So: **explain yourself in code first**, and keep only comments that code *cannot* express.

Analogy: a road sign that says "Bridge closed" long after the bridge was reopened is worse than no sign.

The problem it solves

Comments used as a crutch for unclear code, redundant comments that repeat code, and outdated comments that mislead.

How it works

Replace a comment with a well-named function or variable whenever possible:

```
// free shipping if the order is big enough and domestic
if (order.total() >= 499 && !order.isInternational())
```

becomes `if (order.qualifiesForFreeShipping())` — the comment is no longer needed.

Good comments (keep these):

Kind	Example
Legal	Copyright/licence header
Informative	What a regex (text-matching pattern) matches, the format of a returned string
Explanation of intent	*Why* a decision was made ("we copy because callers may pass an unmodifiable list")
Clarification	Translating an obscure argument or return value from a library you can't change
Warning of consequences	"Not thread-safe" (unsafe to use from several threads at once), "takes 10 minutes to run", "billed per call"
TODO	Work that should be done but can't be now — with an owner; review them regularly
Amplification	Stressing the importance of something that looks trivial ("the trim is critical: …")
Javadoc in public APIs	Contracts callers depend on: params, return, exceptions

Bad comments (remove or replace):

Kind	What it looks like
Mumbling	A vague comment only the author understands
Redundant	`i++; // increment i`
Misleading	Subtly inaccurate description
Mandated	Javadoc on every function "because the rule says so" (`@param a the a`)
Journal	Change logs at the top of the file — version control (a tool such as Git that records every change) does this
Noise	`/** Default constructor. */`
Scary noise	Copy-pasted Javadoc noise with errors
Comment instead of a function/variable	Explaining an expression that could be named
Position markers	`// ********** Actions **********` banners
Closing-brace comments	`} // end while` — make the function shorter instead
Attributions and bylines	`/* Added by Rick */` — version control knows
Commented-out code	Others are afraid to delete it; it rots. Delete it — git remembers
HTML comments	Markup inside comments, unreadable in the editor
Nonlocal information	Commenting a system-wide default next to an unrelated function
Too much information	Historical discussions, RFC (internet standards document) excerpts
Inobvious connection	A comment that itself needs explaining
Function headers	Short, well-named functions don't need them
Javadoc in non-public code	Formal docs for internal code is usually noise

Java example — before/after

```java
// File: lld/ch06/comments/Main.java
/*
 * Copyright (c) 2026 Example Corp. Licensed under the Apache License, Version 2.0.   <-- GOOD: legal comment
 */
package lld.ch06.comments;

import java.util.List;
import java.util.regex.Pattern;

/** Comments: explain yourself in code first; keep only comments that code cannot express. */
public class Main {

    record Applicant(String name, int age, int creditScore, long monthlyIncomePaise, boolean hasDefaulted) {}

    // ---------------- BEFORE: comments used to excuse unclear code ----------------
    static class Before {
        // Changes:
        //   2019-03-01 RK: created
        //   2021-07-12 AB: fixed bug                                  <-- BAD: journal comment (git does this)
        /**
         * The eligible method.
         * @param a the a                                              <-- BAD: noise / mandated Javadoc
         * @return the boolean
         */
        static boolean eligible(Applicant a) {
            // check if applicant is eligible for a premium loan      <-- BAD: comment instead of a good name
            if (a.age() >= 21 && a.age() <= 60 && a.creditScore() > 750 && !a.hasDefaulted()
                    && a.monthlyIncomePaise() >= 5_000_000) {
                return true;   // return true                         <-- BAD: redundant
            }
            // if (a.age() > 65) return false;                        <-- BAD: commented-out code
            return false;
        } // end eligible                                             <-- BAD: closing-brace comment
    }

    // ---------------- AFTER: the code explains itself; comments add what code can't ----------------
    static class After {
        private static final int MIN_AGE = 21, MAX_AGE = 60;
        private static final int GOOD_CREDIT_SCORE = 750;
        private static final long MIN_MONTHLY_INCOME_PAISE = 5_000_000;   // Rs 50,000

        static boolean isEligibleForPremiumLoan(Applicant a) {
            return isWorkingAge(a) && hasGoodCredit(a) && earnsEnough(a);
        }
        private static boolean isWorkingAge(Applicant a) { return a.age() >= MIN_AGE && a.age() <= MAX_AGE; }
        private static boolean hasGoodCredit(Applicant a) { return a.creditScore() > GOOD_CREDIT_SCORE &&
!a.hasDefaulted(); }
        private static boolean earnsEnough(Applicant a) { return a.monthlyIncomePaise() >=
MIN_MONTHLY_INCOME_PAISE; }

        // GOOD: informative comment -- clarifies what a regex matches (format: 10-character PAN, e.g. ABCDE1234F)
        private static final Pattern PAN_FORMAT = Pattern.compile("[A-Z]{5}[0-9]{4}[A-Z]");

        static boolean looksLikePan(String s) { return PAN_FORMAT.matcher(s).matches(); }

        /**
         * GOOD: Javadoc on a public API -- states the contract callers rely on.
         * @param scores credit scores in the range 300..900
         * @return the median score; for an even count, the lower of the two middle values
         * @throws IllegalArgumentException if {@code scores} is empty
         */
        static int medianScore(List<Integer> scores) {
            if (scores.isEmpty()) throw new IllegalArgumentException("no scores");
            // GOOD: explanation of intent -- we copy because callers may pass an unmodifiable list, and sorting
in place would fail
            List<Integer> sorted = scores.stream().sorted().toList();
            return sorted.get((sorted.size() - 1) / 2);
        }
```

```
        // GOOD: warning of consequences -- this call hits the credit bureau and is billed per request.
        // TODO(loans-team): cache bureau responses for 24h once the bureau contract allows it.   <-- GOOD: TODO
        static int fetchBureauScore(String pan) { return 780; }
    }

    public static void main(String[] args) {
        var asha = new Applicant("Asha", 30, 790, 6_000_000, false);
        var ravi = new Applicant("Ravi", 30, 790, 6_000_000, true);
        System.out.println("BEFORE eligible(asha)=" + Before.eligible(asha) + ", eligible(ravi)=" +
Before.eligible(ravi));
        System.out.println("AFTER  isEligibleForPremiumLoan(asha)=" + After.isEligibleForPremiumLoan(asha)
                + ", (ravi)=" + After.isEligibleForPremiumLoan(ravi));
        System.out.println("looksLikePan(ABCDE1234F)=" + After.looksLikePan("ABCDE1234F"));
        System.out.println("medianScore([700, 820, 650, 900])=" + After.medianScore(List.of(700, 820, 650, 900)));
        System.out.println("bureau score=" + After.fetchBureauScore("ABCDE1234F"));
    }
}
```

Output:

```
BEFORE eligible(asha)=true, eligible(ravi)=false
AFTER  isEligibleForPremiumLoan(asha)=true, (ravi)=false
looksLikePan(ABCDE1234F)=true
medianScore([700, 820, 650, 900])=700
bureau score=780
```

In *before*, every comment is a smell: journal, mandated noise, a comment used instead of a name, redundant, commented-out code and a closing-brace marker. In *after*, the long condition became three named predicates and named constants; the remaining comments are the good kinds — legal header, informative (regex format), intent, public-API Javadoc, warning and a TODO with an owner.

Trade-offs

Comments are cheap to write and expensive to maintain. A few precise "why" comments are valuable; many "what" comments are a liability.

When not to use it

"No comments" is not the goal. Public libraries need Javadoc; complex algorithms (a published paper's method, a tricky concurrency protocol) deserve a comment explaining the approach and why.

Common mistakes

- Updating code but not the comment next to it.
- Keeping commented-out code "just in case".
- Using comments as a to-do list that no one ever reviews.

Real-world examples

- The **JDK's Javadoc** is a model of contract documentation (e.g. `HashMap` explains iteration order is not guaranteed; `ConcurrentHashMap` explains weakly consistent iterators).
- **SonarQube** (a server that scans code for problems) flags commented-out code; many teams' linters reject `TODO` without a ticket reference (an ID in the team's issue tracker).

In the interview

30-second answer: "I prefer expressive code over comments: named functions and constants replace most 'what' comments. I keep comments for why, warnings, legal headers, TODOs with owners and public API contracts, and I delete commented-out code because version control remembers it."

Self-check

1. Q: Name three good kinds of comment. → A: Legal, explanation of intent, warning of consequences (also informative, TODO, amplification, public Javadoc).

2. Q: What should you do with commented-out code? → A: Delete it.
3. Q: Why do comments lie? → A: Code changes and comments aren't updated.

TL;DR

Say it in code; comment only the *why*, warnings, legal text and public contracts; delete dead comments and commented-out code.

---

4. Formatting

What it is

Formatting is about **communication**: the layout of code tells the reader what belongs together and what matters most. A team should agree on one set of formatting rules and apply them automatically everywhere.

Analogy: a newspaper. The headline tells you the story, the first paragraph gives the summary, details come further down. You can stop reading at any depth and still understand the gist.

The problem it solves

Inconsistent, cramped or random layout makes readers work to understand structure before they can understand logic. It also produces noisy diffs (the line-by-line view of what changed) in code review (a teammate reading your change before it is accepted; §15).

How it works

Vertical formatting (top-to-bottom):

1. **File size** — small files are easier to understand. Many well-known Java projects keep most files around 200 lines, with very few over 500.
2. **Newspaper metaphor** — the class name and the top of the file give the high-level picture; details increase as you move down.
3. **Vertical openness** — separate distinct concepts (package, imports, each method) with blank lines.
4. **Vertical density** — lines that are tightly related appear together, without blank lines or useless comments between them.
5. **Vertical distance** — keep related concepts close:
   - declare local variables close to their first use;
   - instance variables in one well-known place (Java convention: top of the class);
   - **dependent functions**: the caller sits *above* the callee, close by;
   - **conceptual affinity**: functions that do similar things (overloads, related operations) sit near each other.
6. **Vertical ordering** — high-level functions first, details later (the stepdown rule again).

Horizontal formatting (left-to-right):

1. **Line width** — keep lines short enough to read without scrolling. Martin sets his own limit at 120 characters; Google's Java style uses 100.
2. **Horizontal openness and density** — spaces around assignment operators and between arguments; no space between a function name and its `(`; use spacing to show precedence where it helps.
3. **No horizontal alignment** of declarations in columns — it draws the eye to the wrong thing (types vs names) and breaks with every rename.
4. **Indentation** — shows scope hierarchy; never collapse a short `if` or loop body onto one line if it hides structure.
5. **Dummy scopes** — if a `while` or `for` has an empty body, make it visible (put the semicolon on its own line or use `{}`), because a trailing `;` is easy to miss.

Team rules — a codebase should look like one person wrote it. Encode the rules in an automatic formatter (IDE settings, `google-java-format`, Spotless, Checkstyle) and let CI enforce them.

Java example — before/after

```java
// File: lld/ch06/formatting/Main.java
package lld.ch06.formatting;

import java.util.ArrayList;
import java.util.List;

/** Formatting: same code, formatted two ways. The compiler doesn't care; readers do. */
public class Main {

    // ---------------- BEFORE: cramped, random order, aligned columns, no vertical openness ----------------
    static class Before {
        static class Playlist{private int cursor;
            private String helper(int i){return songs.get(i).toUpperCase();}
            public String next(){if(songs.isEmpty())return null;String s=helper(cursor);cursor=
(cursor+1)%songs.size();return s;}
            private final List<String>   songs   =   new ArrayList<>();
            private final String         name;
            public void add(String s){songs.add(s);}Playlist(String n){name=n;}
            public String name(){return name;}}
    }

    // ---------------- AFTER: newspaper order -- headline first, details below ----------------
    static class After {
        /** A looping playlist. (Headline: the class name and this one line tell you what it is.) */
        static final class Playlist {
            // 1. Fields at the top, together (vertical density for closely related lines).
            private final String name;
            private final List<String> songs = new ArrayList<>();
            private int cursor;

            // 2. Constructor next.
            Playlist(String name) {
                this.name = name;
            }

            // 3. Public API -- each concept separated by a blank line (vertical openness).
            public String name() {
                return name;
            }

            public void add(String song) {
                songs.add(song);
            }

            public String next() {
                if (songs.isEmpty()) {
                    return null;
                }
                String song = displayTitle(cursor);    // caller ABOVE callee (dependent functions close together)
                cursor = (cursor + 1) % songs.size();
                return song;
            }

            // 4. Private helpers just below the functions that use them.
            private String displayTitle(int index) {
                return songs.get(index).toUpperCase();
            }
        }
    }

    public static void main(String[] args) {
        var before = new Before.Playlist("Road trip");
        var after = new After.Playlist("Road trip");
        for (String s : List.of("Kesariya", "Yellow", "Tum Hi Ho")) { before.add(s); after.add(s); }
        StringBuilder b = new StringBuilder(), a = new StringBuilder();
        for (int i = 0; i < 4; i++) { b.append(before.next()).append(' '); a.append(after.next()).append(' '); }
        System.out.println("BEFORE " + before.name() + ": " + b.toString().trim());
        System.out.println("AFTER  " + after.name() + ": " + a.toString().trim());
        System.out.println("Same behaviour -- formatting only changes how fast a human understands it.");
```

```
    }
}
```

Output:

```
BEFORE Road trip: KESARIYA YELLOW TUM HI HO KESARIYA
AFTER  Road trip: KESARIYA YELLOW TUM HI HO KESARIYA
Same behaviour -- formatting only changes how fast a human understands it.
```

Both classes behave identically. The *after* uses the newspaper order (fields → constructor → public API → private helpers, with each helper right under its caller), blank lines between concepts, no column alignment and consistent indentation.

Trade-offs

Almost none — formatting is cheap when automated. The only cost is agreeing on the rules once.

When not to use it

Generated code (e.g. from Protobuf, Google's schema-based message format, whose compiler writes classes for you) doesn't need hand formatting; exclude it from formatters.

Common mistakes

- Mixing formatting changes with logic changes in the same commit (one saved, described change in version control) — it makes the diffs unreadable.
- Personal styles inside a team codebase.
- Aligning declarations in columns.

Real-world examples

- **google-java-format**, **Spotless** (Gradle/Maven — Java build tools), **Prettier** (JS), **gofmt** (Go formats all Go code identically — style debates disappear), **Black** (Python).
- **EditorConfig** files share indentation settings across editors.

In the interview

30-second answer: "Formatting communicates structure. I keep related lines together, separate concepts with blank lines, put callers above callees, fields at the top, and let an automatic formatter enforce one team style."

Self-check

1. Q: What is the newspaper metaphor? → A: High-level summary at the top, increasing detail below.
2. Q: Where should a private helper go? → A: Just below the function that calls it.

TL;DR

Format for the reader: headline first, related things close, concepts separated, one automated team style.

---

5. Objects and Data Structures

What it is

There are two fundamentally different kinds of types:

- An **object** *hides* its data behind abstractions and *exposes behaviour* (functions that operate on that data). Example: an `Order` with `isMetroDelivery()`.
- A **data structure** *exposes* its data and has *no meaningful behaviour*. Example: a record `Address(city, pincode)`, a DTO (Data Transfer Object — a plain data carrier).

Data abstraction: hiding implementation is not just putting getters and setters on private fields. It means exposing abstract interfaces that let users manipulate the *essence* of the data without knowing its representation (e.g. `fuelPercentRemaining()` rather than `getFuelTankCapacityInLitres()` and `getLitresOfFuel()`).

Analogy: a vending machine (object) — you press buttons and get results; you can't touch its coin box. A spreadsheet (data structure) — all values are visible; formulas live elsewhere.

The problem it solves

Confusion between the two creates **hybrids** — half object, half data structure — with public fields *and* significant behaviour. Hybrids get the worst of both worlds: hard to add new functions *and* hard to add new types. Knowing the difference also tells you which design makes your likely changes easy.

How it works — the data/object anti-symmetry

	Procedural code (data structures + functions)	**OO code (objects)**
Add a new **function** (e.g. `perimeter`)	**Easy**: add one function; data types untouched	**Hard**: add a method to every class
Add a new **type** (e.g. `Triangle`)	**Hard**: edit every function	**Easy**: add one class; existing code untouched

So: *everything is an object* is a myth. When you expect **new types**, choose objects and polymorphism. When you expect **new operations** over a stable set of types, data structures with functions (or the Visitor pattern, Chapter 11, which adds operations to a set of classes without editing them) may be better. Java 21's `sealed` interfaces (which list exactly which classes may implement them) with records and pattern-matching `switch` (a `switch` that branches on an object's type) make the procedural style safe: the compiler tells you if a `switch` misses a type.

The Law of Demeter (Chapter 5, §4) applies to objects: a method should not navigate an object's internals (`order.getCustomer().getAddress().city()`), because that exposes their structure — a **train wreck**. It does *not* strictly apply to data structures, whose whole job is to expose data. The fix for objects is to **hide structure**: ask the object to do the job (`order.isMetroDelivery()`).

Data Transfer Objects (DTOs): the purest data structure — public (or record) fields, no behaviour. Used to move data across boundaries: database rows, JSON (a plain-text key–value data format) request/response bodies, messages. *Bean* form (private fields + getters/setters) gives no real encapsulation benefit but is required by some frameworks.

Active Record: a special DTO that also has navigation methods like `save()` and `find()` and maps to one database row (Rails' ActiveRecord, some Java ORMs — Object-Relational Mappers, libraries that map classes to database tables). Common mistake: putting business rules into active records, making them hybrids. Keep business rules in separate objects that *use* the active record.

Java example — before/after

```java
// File: lld/ch06/objects/Main.java
package lld.ch06.objects;

import java.util.List;

/**
 * Objects vs data structures.
 * Objects hide data and expose behaviour; data structures expose data and have no behaviour.
 * Data/object anti-symmetry: each makes a different kind of change easy.
 */
public class Main {

    // ---------------- STYLE 1: data structures + procedures (procedural) ----------------
    static class Procedural {
        sealed interface Shape permits Square, Circle {}
        record Square(double side) implements Shape {}
        record Circle(double radius) implements Shape {}

        /** All behaviour lives here. Adding perimeter() = one new method, shapes untouched. Adding Triangle =
edit every method. */
        static final class Geometry {
            static double area(Shape s) {
                return switch (s) {
                    case Square sq -> sq.side() * sq.side();
                    case Circle c -> Math.PI * c.radius() * c.radius();
                };
            }
            static double perimeter(Shape s) {                // NEW FUNCTION: easy -- no shape class changes
                return switch (s) {
                    case Square sq -> 4 * sq.side();
                    case Circle c -> 2 * Math.PI * c.radius();
                };
            }
        }
    }

    // ---------------- STYLE 2: objects (polymorphic) ----------------
    static class ObjectOriented {
        interface Shape { double area(); }
        record Square(double side) implements Shape { public double area() { return side * side; } }
        record Circle(double radius) implements Shape { public double area() { return Math.PI * radius * radius; }
}
        record Triangle(double base, double height) implements Shape {   // NEW TYPE: easy -- nothing else changes
            public double area() { return 0.5 * base * height; }
        }
        // ...but adding perimeter() would mean editing Shape and EVERY implementation.
    }

    // ---------------- Hiding structure (Law of Demeter) ----------------
    static class Before {
        record Address(String city, String pincode) {}
        static class Customer { Address address = new Address("Pune", "411001"); Address getAddress() { return
address; } }
        static class Order { Customer customer = new Customer(); Customer getCustomer() { return customer; } }
        static boolean isMetroDelivery(Order o) {
            return List.of("Mumbai", "Delhi", "Pune").contains(o.getCustomer().getAddress().city());  // train
wreck
        }
    }
    static class After {
        record Address(String city, String pincode) {                    // a data structure: exposing data is its
job
            boolean isMetro() { return List.of("Mumbai", "Delhi", "Pune").contains(city); }
        }
        static final class Order {                                         // an object: hides its structure
            private final Address shipTo;
            Order(Address shipTo) { this.shipTo = shipTo; }
            boolean isMetroDelivery() { return shipTo.isMetro(); }        // tell the order what you need
        }
```

```
    }

    // ---------------- DTO: pure data for crossing a boundary ----------------
    /** A Data Transfer Object: public data, no behaviour; e.g. JSON returned by a REST endpoint. */
    record OrderDto(String orderId, String city, long totalPaise) {}

    public static void main(String[] args) {
        var shapes = List.<Procedural.Shape>of(new Procedural.Square(2), new Procedural.Circle(1));
        for (var s : shapes)
            System.out.printf("procedural %-22s area=%.2f perimeter=%.2f%n", s, Procedural.Geometry.area(s),
Procedural.Geometry.perimeter(s));

        var objects = List.<ObjectOriented.Shape>of(new ObjectOriented.Square(2), new ObjectOriented.Circle(1),
                new ObjectOriented.Triangle(3, 4));
        for (var s : objects) System.out.printf("object     %-22s area=%.2f%n", s, s.area());

        System.out.println("BEFORE train wreck says metro? " + Before.isMetroDelivery(new Before.Order()));
        System.out.println("AFTER  order.isMetroDelivery()? " + new After.Order(new After.Address("Pune",
"411001")).isMetroDelivery());
        System.out.println("DTO: " + new OrderDto("o-9", "Pune", 45_000));
    }
}
```

Output:

```
procedural Square[side=2.0]       area=4.00 perimeter=8.00
procedural Circle[radius=1.0]     area=3.14 perimeter=6.28
object     Square[side=2.0]       area=4.00
object     Circle[radius=1.0]     area=3.14
object     Triangle[base=3.0, height=4.0] area=6.00
BEFORE train wreck says metro? true
AFTER  order.isMetroDelivery()? true
DTO: OrderDto[orderId=o-9, city=Pune, totalPaise=45000]
```

The procedural `Geometry` gained `perimeter()` without touching the shapes; the OO version gained `Triangle` without touching anything. The train wreck `o.getCustomer().getAddress().city()` became `order.isMetroDelivery()`, while the `Address` record (a data structure) freely exposes `city`. `OrderDto` is a pure data carrier.

Trade-offs

Summarised in the anti-symmetry table above. Choose based on which change you expect more.

When not to use it

Don't force behaviour into DTOs that cross process boundaries (travel between separately running programs, e.g. over the network) — keep them dumb and version them carefully.

Common mistakes

- Adding getters and setters for every private field by reflex (that's not encapsulation).
- Hybrids: public mutable fields plus business methods.
- Business logic inside JPA (Java's standard ORM API) entities that also serve as API response bodies.

Real-world examples

- Records are Java's official data carriers; `java.awt.Point` has public fields (a data structure).
- `java.time.LocalDate` is an object: you call `plusDays`, `isLeapYear`; you never see its internal fields.
- DTOs in Spring (the main Java server framework) REST controllers (classes that handle HTTP API requests) (`@RequestBody CreateOrderRequest`); Rails ActiveRecord / Spring Data JPA entities.

In the interview

30-second answer: "Objects hide data and expose behaviour; data structures expose data with no behaviour. OO makes adding types easy and functions hard; procedural does the opposite. I use

> records for DTOs and value data, rich objects for domain logic, and avoid hybrids and train wrecks."

Follow-ups:

- *Q: Is `a.getB().getC()` always wrong?* A: For objects, yes (Demeter). For pure data structures like records it's acceptable.
- *Q: Which pattern lets OO code add operations easily?* A: Visitor (Chapter 11), at the cost of making new types harder.

Self-check

1. Q: Which style makes adding a new shape type easy? → A: OO (polymorphic objects).
2. Q: What is a hybrid? → A: A type with both exposed data and significant behaviour — to be avoided.
3. Q: What's an Active Record? → A: A DTO with persistence/navigation methods mapped to a DB row.

TL;DR

Objects hide data and expose behaviour; data structures expose data. Pick based on whether you'll add types or operations. No hybrids, no train wrecks through objects.

---

6. Error Handling

What it is

Error handling is important, but if it **obscures the logic** it's wrong. Clean error handling keeps the happy path readable and makes failures explicit, informative and hard to ignore.

Analogy: a fire exit plan — clearly marked, separate from normal corridors, and never blocking daily traffic.

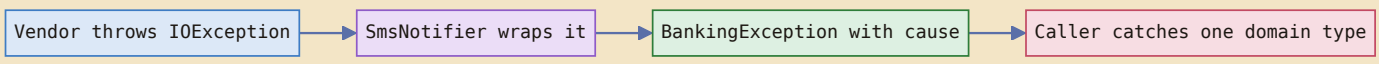
The problem it solves

Error codes clutter every caller with `if` checks and are easy to forget; `null` returns cause `NullPointerException`s far away; checked exceptions (ones the Java compiler forces every caller to catch or declare) from low-level libraries leak into every signature; vague messages make debugging slow.

How it works — the rules

1. **Use exceptions rather than return codes.** The caller's logic stays clean; errors can't be silently ignored.
2. **Write your `try-catch-finally` statement first** when writing code that can throw. The `try` block is like a transaction (a unit of work that either fully happens or is fully undone): whatever happens inside, the `catch` must leave the program consistent. In Java, prefer **try-with-resources** for anything `AutoCloseable` (any class with a `close()` method, such as a file or connection) — the resource is closed even if an exception is thrown.
3. **Use unchecked exceptions.** Checked exceptions violate the Open/Closed Principle: if a low-level method starts throwing a new checked exception, every method between it and the handler must change its signature. Modern languages (C#, Kotlin, Python, Scala) have no checked exceptions. Use checked exceptions only when a caller can and must realistically recover (a rare, deliberate choice).
4. **Provide context with exceptions.** Message should say what failed, with which values, and why (`account 'B' has 100 but 900 was requested`). Keep the original exception as the **cause** when wrapping.
5. **Define exception classes in terms of the caller's needs.** How will the caller *catch* it? Often one exception type (per area) is enough. **Wrap third-party APIs** so their exceptions are translated into yours; this minimises dependency on the vendor and makes swapping libraries easy.
6. **Define the normal flow — the Special Case pattern** (Martin Fowler): instead of throwing or returning `null` for an expected "absence" case, return an object that handles that case (`NoMembership` with 0 points, `GuestCustomer`, an empty list). Callers need no special handling.

7. **Don't return `null`.** Return an empty collection, `Optional` (a container that holds a value or is empty), a special-case object, or throw. Every `null` return is a future `NullPointerException`.
8. **Don't pass `null`.** Unless an API expects it, reject `null` arguments immediately (`Objects.requireNonNull`) — fail fast (Chapter 5).



Java example — before/after

```java
// File: lld/ch06/errors/Main.java
package lld.ch06.errors;

import java.io.IOException;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Objects;
import java.util.Optional;

/** Error handling: exceptions not codes, unchecked with context, wrap third-party errors, special case, no null.
*/
@SuppressWarnings("serial") // the nested exception classes are never serialized
public class Main {

    // ---------------- BEFORE ----------------
    static class Before {
        static final int OK = 0, E_NO_ACCOUNT = 1, E_NO_MONEY = 2;
        static final Map<String, Long> BALANCES = new HashMap<>(Map.of("A", 500L, "B", 100L));
        static final Map<String, Integer> POINTS = new HashMap<>(Map.of("A", 40));

        static int transfer(String from, String to, long amount) {        // caller must check an int
            Long f = BALANCES.get(from), t = BALANCES.get(to);
            if (f == null || t == null) return E_NO_ACCOUNT;               // which one? no context
            if (f < amount) return E_NO_MONEY;                             // how much was missing?
            BALANCES.put(from, f - amount); BALANCES.put(to, t + amount);
            return OK;
        }
        static List<String> history(String id) { return null; }          // "no history" = null
        static Integer loyaltyPoints(String id) { return POINTS.get(id); } // null for non-members
    }

    // ---------------- AFTER ----------------
    static class After {
        /** Unchecked, domain-specific, with context. Defined by what the CALLER needs to handle. */
        static class BankingException extends RuntimeException {
            BankingException(String msg) { super(msg); }
            BankingException(String msg, Throwable cause) { super(msg, cause); }
        }
        static final class AccountNotFoundException extends BankingException {
            AccountNotFoundException(String id) { super("account '" + id + "' does not exist"); }
        }
        static final class InsufficientFundsException extends BankingException {
            InsufficientFundsException(String id, long requested, long available) {
                super("account '" + id + "' has " + available + " but " + requested + " was requested");
            }
        }

        /** SPECIAL CASE pattern: an object that represents "no loyalty membership" -- callers need no null check.
*/
        interface Loyalty { int points(); String tier(); }
        record Member(int points) implements Loyalty { public String tier() { return points >= 100 ? "GOLD" :
"SILVER"; } }
        enum NoMembership implements Loyalty { INSTANCE; public int points() { return 0; } public String tier() {
return "NONE"; } }

        /** Third-party library that throws its own checked exceptions (we can't change it). */
        static final class VendorSmsClient {
            void push(String phone, String text) throws IOException {
                if (phone.startsWith("000")) throw new IOException("socket timeout to sms-gw-3");
            }
        }
        /** Wrapper: one exception type our code understands; original kept as the cause. */
        static final class SmsNotifier {
            private final VendorSmsClient client = new VendorSmsClient();
            void send(String phone, String text) {
                try { client.push(phone, text); }
                catch (IOException e) { throw new BankingException("could not send SMS to " + phone, e); }
            }
```

```
        }

        static final class Bank {
            private final Map<String, Long> balances = new HashMap<>(Map.of("A", 500L, "B", 100L));
            private final Map<String, Integer> points = new HashMap<>(Map.of("A", 40));

            void transfer(String from, String to, long amount) {
                Objects.requireNonNull(from, "from"); Objects.requireNonNull(to, "to");     // don't accept null
                long available = balanceOf(from);
                balanceOf(to);                                                              // validates 'to'
exists
                if (available < amount) throw new InsufficientFundsException(from, amount, available);
                balances.merge(from, -amount, Long::sum);
                balances.merge(to, amount, Long::sum);
            }
            long balanceOf(String id) {
                Long b = balances.get(id);
                if (b == null) throw new AccountNotFoundException(id);
                return b;
            }
            List<String> history(String id) { return List.of(); }                          // empty, never null
            Optional<Long> findBalance(String id) { return Optional.ofNullable(balances.get(id)); }
            Loyalty loyaltyOf(String id) {
                Integer p = points.get(id);
                return p == null ? NoMembership.INSTANCE : new Member(p);
            }
        }

        /** "Write the try first": a resource that is always released, via try-with-resources. */
        static final class AuditFile implements AutoCloseable {
            AuditFile() { System.out.println("  audit file opened"); }
            void write(String line) { System.out.println("  audit: " + line); }
            @Override public void close() { System.out.println("  audit file closed"); }
        }
    }

    public static void main(String[] args) {
        System.out.println("== BEFORE ==");
        int code = Before.transfer("A", "Z", 50);
        System.out.println("transfer returned " + code + " (which account? caller has to decode)");
        try {
            System.out.println("history size " + Before.history("A").size());
        } catch (NullPointerException e) { System.out.println("history(): NullPointerException -- someone forgot
the null check"); }
        try {
            int pts = Before.loyaltyPoints("B");            // auto-unboxing a null Integer
            System.out.println(pts);
        } catch (NullPointerException e) { System.out.println("loyaltyPoints(B): NullPointerException on
unboxing"); }

        System.out.println("== AFTER ==");
        var bank = new After.Bank();
        for (String[] t : new String[][]{{"A", "Z", "50"}, {"B", "A", "900"}, {"A", "B", "200"}}) {
            try {
                bank.transfer(t[0], t[1], Long.parseLong(t[2]));
                System.out.println("transfer " + t[0] + "->" + t[1] + " of " + t[2] + " ok; A=" +
bank.balanceOf("A") + " B=" + bank.balanceOf("B"));
            } catch (After.BankingException e) {
                System.out.println("transfer failed: " + e.getMessage());
            }
        }
        System.out.println("history(A) size = " + bank.history("A").size());
        System.out.println("findBalance(Z) = " + bank.findBalance("Z"));
        for (String id : List.of("A", "B")) {
            var l = bank.loyaltyOf(id);                     // no null check needed
            System.out.println("loyalty " + id + ": " + l.points() + " pts, tier " + l.tier());
        }
        try {
            new After.SmsNotifier().send("000123", "hi");
        } catch (After.BankingException e) {
            System.out.println(e.getMessage() + " (cause: " + e.getCause().getMessage() + ")");
```

```
        }
        try (var audit = new After.AuditFile()) {
            audit.write("transfer A->B 200");
            throw new IllegalStateException("simulated crash");
        } catch (IllegalStateException e) {
            System.out.println("  caught: " + e.getMessage() + " -- file was still closed first");
        }
    }
}
```

Output:

```
== BEFORE ==
transfer returned 1 (which account? caller has to decode)
history(): NullPointerException -- someone forgot the null check
loyaltyPoints(B): NullPointerException on unboxing
== AFTER ==
transfer failed: account 'Z' does not exist
transfer failed: account 'B' has 100 but 900 was requested
transfer A->B of 200 ok; A=300 B=300
history(A) size = 0
findBalance(Z) = Optional.empty
loyalty A: 40 pts, tier SILVER
loyalty B: 0 pts, tier NONE
could not send SMS to 000123 (cause: socket timeout to sms-gw-3)
  audit file opened
  audit: transfer A->B 200
  audit file closed
  caught: simulated crash -- file was still closed first
```

What each part shows:

- *Before*: `transfer` returns `1` without saying which account; `history` returns `null`; `loyaltyPoints` returns `null` that crashes on auto-unboxing (Java's automatic conversion from `Integer` to `int`, which throws on `null`).
- *After*: an unchecked hierarchy under `BankingException`, messages with context; `history` returns an empty list; `findBalance` returns `Optional`; `loyaltyOf` returns a **special case** (`NoMembership.INSTANCE`) so callers never check for `null`; the vendor's checked `IOException` is **wrapped** with the original as its cause; `try-with-resources` closes the audit file even when an exception is thrown.

Trade-offs

Choice	Pro	Con
Unchecked exceptions	Clean signatures, OCP-friendly	Compiler doesn't remind callers; document them
Checked exceptions	Compiler-enforced handling	Signature ripple; often caught and swallowed
`Optional` return	Absence visible in the type	Not for fields/parameters; small allocation
Special Case object	No checks at all in callers	Can hide a real problem if absence *should* be noticed
Result/Either type (holds either a success value or an error)	Expected failures as values, composable	More verbose in Java (see Error-handling strategy)

When not to use it

Exceptions are for exceptional or error situations, not normal control flow — e.g. don't throw to exit a loop. For *expected, frequent* outcomes (validation of user input rows), returning a result value can be cleaner (see §16).

Common mistakes

- `catch (Exception e) {}` — swallowing.
- Logging and rethrowing at every layer (the same error logged 5 times).
- Losing the cause when wrapping (`new MyException(e.getMessage())` instead of `new MyException(msg, e)`).
- Returning `null` from collections-returning methods.

Real-world examples

- **Spring's** `DataAccessException` hierarchy translates vendor-specific `SQLException`s into unchecked, meaningful exceptions (e.g. `DuplicateKeyException`) — exactly "wrap third-party APIs, define exceptions by caller's needs".
- `Collections.emptyList()`, `Optional.empty()` — JDK support for "don't return null".
- `java.util.Objects.requireNonNull` — "don't pass null".
- `UncheckedIOException` (Java 8) wraps `IOException` for use in lambdas/streams (inline functions and Java's element-processing pipelines, whose standard interfaces cannot throw checked exceptions).

In the interview

> **30-second answer:** "I throw unchecked, domain-specific exceptions with context instead of returning codes or null. I wrap third-party exceptions at the boundary, return empty collections or Optional for absence, use a special-case object when absence is normal, validate nulls at entry, and use try-with-resources for cleanup."

Follow-ups:

- *Q: Checked vs unchecked?* A: Checked forces handling but ripples through signatures and breaks OCP; I default to unchecked and reserve checked for recoverable conditions callers must handle.
- *Q: What is the Special Case pattern?* A: A subclass/object representing a special situation (e.g. no membership) that behaves sensibly, so callers need no null checks.

Self-check

1. Q: Why are error codes bad? → A: They clutter callers, are easy to ignore, and lack context.
2. Q: What should a collection-returning method return when there's nothing? → A: An empty collection.
3. Q: Why keep the cause when wrapping? → A: So the original stack trace and message aren't lost.

TL;DR

Exceptions not codes; unchecked with context; wrap vendors; special cases, empty collections or Optional instead of null; never pass null; always clean up.

---

7. Boundaries

What it is

A **boundary** is where your code meets code you don't control: third-party libraries, frameworks, other teams' services, even very general JDK types like `Map`. Clean boundaries keep that foreign code at the edge, behind interfaces shaped for *your* needs.

Analogy: a power adapter when travelling. Your laptop doesn't care about each country's socket; the adapter handles it.

The problem it solves

Providers of libraries want broad applicability; users want interfaces focused on their needs. That tension leaks: a raw `Map<String, Session>` passed around lets any code `clear()` it, put wrong values in, and forces everyone to know the key format. When the library's API changes, every usage site breaks.

How it works

1. **Wrap boundary types.** Don't pass `Map`, vendor clients or framework types around your application. Hide them inside a class that exposes only what your app needs (`SessionRegistry.open/activeSession/close`). Keep the boundary type **private** or at least out of public APIs.
2. **Write learning tests.** When adopting a library, write small tests that call its API the way you intend to use it. They document your understanding, cost nothing extra (you had to learn the API anyway), and — re-run on each upgrade — tell you immediately if the library's behaviour changed.
3. **Use code that does not exist yet.** When another team's API isn't ready, define the interface *you wish you had* (`FraudCheck`), code and test against it with a fake, and later write an **Adapter** (Chapter 9) that bridges it to the real API. This also gives you a clean seam (a place where a test can plug in a substitute) for testing.
4. **Keep boundary knowledge in very few places** so a vendor change touches one adapter.

Java example — before/after

```
// File: lld/ch06/boundaries/Main.java
package lld.ch06.boundaries;

import java.time.Instant;
import java.util.HashMap;
import java.util.Map;
import java.util.Optional;
import java.util.concurrent.ConcurrentHashMap;

import lld.ch06.testing.MiniTest;

/** Boundaries: wrap third-party / general-purpose APIs, learn them with tests, and define interfaces for code
that doesn't exist yet. */
public class Main {

    record Session(String userId, Instant expiresAt) {}

    // ---------------- BEFORE: a raw Map passed all over the codebase ----------------
    static class Before {
        // Every caller can clear(), put(null, ...), put wrong types... and every caller must know the key format.
        static void loginHandler(Map<String, Session> sessions, String token, String user) {
            sessions.put(token, new Session(user, Instant.parse("2026-09-28T12:00:00Z")));
        }
        static void buggyAdminTool(Map<String, Session> sessions) { sessions.clear(); }   // oops: everyone logged
out
    }

    // ---------------- AFTER: the Map is hidden behind an interface shaped for OUR needs ----------------
    static class After {
        static final class SessionRegistry {
            private final Map<String, Session> byToken = new ConcurrentHashMap<>();   // boundary type stays
private

            void open(String token, Session session) { byToken.put(token, session); }
            Optional<Session> activeSession(String token, Instant now) {
                return Optional.ofNullable(byToken.get(token)).filter(s -> now.isBefore(s.expiresAt()));
            }
            void close(String token) { byToken.remove(token); }
            int size() { return byToken.size(); }
        }

        /** The fraud team's API isn't written yet: we define the interface WE want and code against it. */
        interface FraudCheck { boolean isSuspicious(String userId, long amountPaise); }

        /** Fake used until the real client exists; later an Adapter will wrap the real API behind the same
interface. */
        static final class FakeFraudCheck implements FraudCheck {
            public boolean isSuspicious(String userId, long amountPaise) { return amountPaise > 10_000_000; }
        }
    }

    /** LEARNING TESTS: executable notes about how third-party (here: JDK) APIs really behave. Re-run them on
every upgrade. */
    public static class JdkLearningTest {
        @lld.ch06.testing.Test void hashmap_allows_one_null_key() {
            Map<String, Integer> m = new HashMap<>();
            m.put(null, 1);
            MiniTest.assertEquals(1, m.get(null));
        }
        @lld.ch06.testing.Test void concurrenthashmap_rejects_null_keys() {
            MiniTest.assertThrows(NullPointerException.class, () -> new ConcurrentHashMap<String, Integer>
().put(null, 1));
        }
        @lld.ch06.testing.Test void map_of_rejects_duplicate_keys() {
            MiniTest.assertThrows(IllegalArgumentException.class, () -> Map.of("k", 1, "k", 2));
        }
        @lld.ch06.testing.Test void string_split_drops_trailing_empty_strings() {
            MiniTest.assertEquals(2, "a,b,,".split(",").length);
            MiniTest.assertEquals(4, "a,b,,".split(",", -1).length);
```

```
        }
    }

    public static void main(String[] args) {
        System.out.println("== BEFORE ==");
        Map<String, Session> raw = new HashMap<>();
        Before.loginHandler(raw, "t1", "asha");
        Before.buggyAdminTool(raw);
        System.out.println("sessions after admin tool: " + raw.size() + "  <-- nothing stopped the clear()");

        System.out.println("== AFTER ==");
        var registry = new After.SessionRegistry();
        registry.open("t1", new Session("asha", Instant.parse("2026-09-28T12:00:00Z")));
        System.out.println("active at 11:00? " + registry.activeSession("t1", Instant.parse("2026-09-
28T11:00:00Z")).isPresent());
        System.out.println("active at 13:00? " + registry.activeSession("t1", Instant.parse("2026-09-
28T13:00:00Z")).isPresent());
        After.FraudCheck fraud = new After.FakeFraudCheck();
        System.out.println("fraud(asha, Rs 2,00,000)? " + fraud.isSuspicious("asha", 20_000_000));

        System.out.println("== Learning tests ==");
        var summary = MiniTest.run(JdkLearningTest.class);
        System.out.println(summary.passed() + " passed, " + summary.failed() + " failed");
    }
}
```

Output:

```
== BEFORE ==
sessions after admin tool: 0  <-- nothing stopped the clear()
== AFTER ==
active at 11:00? true
active at 13:00? false
fraud(asha, Rs 2,00,000)? true
== Learning tests ==
JdkLearningTest
  PASS  concurrenthashmap rejects null keys
  PASS  hashmap allows one null key
  PASS  map of rejects duplicate keys
  PASS  string split drops trailing empty strings
4 passed, 0 failed
```

- The raw `Map` let an admin tool wipe every session. `SessionRegistry` exposes only open/lookup/close and adds the "not expired" rule in one place.
- `FraudCheck` is the interface we want; `FakeFraudCheck` unblocks us until the real client exists.
- `JdkLearningTest` captures surprising but real API facts (`HashMap` allows one `null` key, `ConcurrentHashMap` rejects it, `Map.of` rejects duplicate keys, `String.split` drops trailing empty strings unless you pass a negative limit).

Trade-offs

Wrapping costs a class and some delegation. It pays off whenever the boundary type is used in more than one place or might change.

When not to use it

Don't wrap stable, universal types used locally (a `Map` used inside one method is fine). The rule is about *passing* boundary types around your system.

Common mistakes

- Returning vendor types from your interfaces (`PaymentGateway.charge()` returning a `StripeCharge`) — leaks the vendor.
- Skipping learning tests and discovering library behaviour in production (the live system that real users use).

Real-world examples

- **Anti-Corruption Layer** in Domain-Driven Design: a translation layer between your model and a legacy (old, still-in-use)/external system's model.
- **SLF4J** (Simple Logging Facade for Java) wraps the actual logging library; **Spring's** `RestClient` / `WebClient` abstractions let you swap the HTTP engine.
- Contract tests (e.g. Pact) between microservices (small, independently deployed services) are a team-level version of learning tests.

In the interview

> **30-second answer:** "I keep third-party code at the edges: wrap it behind an interface shaped for my domain, don't pass raw library types around, write learning tests when adopting a library, and if a dependency doesn't exist yet I define the interface I need and use a fake until an adapter can bridge to the real API."

Self-check

1. Q: What is a learning test? → A: A test that exercises a third-party API to learn and pin down its behaviour.
2. Q: How do you handle an API that isn't built yet? → A: Define your own interface, use a fake, adapt later.

TL;DR

Keep foreign code at the edge behind your own interfaces; learn it with tests; adapt, don't leak.

---

8. Unit Tests

What it is

Unit tests are small, automated tests that check one unit of behaviour (usually a class or a few collaborating objects) in isolation. *Clean Code* argues test code is **as important as production code**: it must be clean, because it's what lets you change production code without fear.

Analogy: the safety net under a trapeze artist. Without it, every new move is dangerous; with it, you can try, fall and adjust.

The problem it solves

Without tests, every change risks breaking something, so people stop cleaning code and the design rots. With *dirty* tests, the tests themselves become a burden: they break with every change, take forever to update, and are eventually abandoned — and then production code rots too.

How it works

The three laws of TDD (the book says they lock you into a cycle perhaps thirty seconds long, with the tests just a few seconds ahead of the production code):

1. You may not write production code until you have written a failing unit test.
2. You may not write more of a unit test than is sufficient to fail (and not compiling counts as failing).
3. You may not write more production code than is sufficient to pass the currently failing test.

A related, coarser description (from Kent Beck's TDD, not from these three laws) is **Red → Green → Refactor**: write a failing test (red), make it pass in the simplest way (green), then clean up both code and tests (refactor).

Keep tests clean — they enable the "-ilities": tests make code flexible, maintainable and reusable, because you can change anything as long as the tests pass.

Clean tests are readable tests:

- Use the **Build–Operate–Check** pattern (also called **Arrange–Act–Assert** or **Given–When–Then**): build the test data, operate on it, check the results.
- Build a small **domain-specific testing language**: helper functions like `cart("TEA", 3)` and `checkoutWith(coupons, audit)` hide setup noise.
- **Dual standard:** test code must be clean and readable, but it need not be as efficient as production code (e.g. building strings for assertions is fine).

One assert per test? The book suggests minimising the number of asserts, and — more importantly — **one concept per test**. Several asserts are fine if they all check the same concept.

F.I.R.S.T. — clean tests are:

Letter	Meaning	How our code does it
Fast	Run in milliseconds so you run them constantly	In-memory doubles, no network/DB
Independent	No test depends on another or on order	`MiniTest` creates a fresh instance per test
Repeatable	Same result in any environment, any time	Fixed `Clock` instead of `now()`
Self-validating	Pass/fail automatically, no reading logs	Assertions, not `println`
Timely	Written just before (or with) the production code	TDD cycle

Java example — a JUnit-free runner, clean tests and a dirty one

Instead of JUnit (the standard Java testing framework), our tiny test framework (`@Test` + `MiniTest`) uses reflection (Java's ability to inspect classes and call methods by name at runtime) to find annotated methods (methods tagged with `@Test`), runs each on a new instance and reports PASS/FAIL. The production class `Checkout` has four collaborators behind interfaces, so tests can replace them with doubles.

```
// File: lld/ch06/testing/BeforeStyleTest.java
package lld.ch06.testing;

import java.time.Clock;
import java.util.List;
import java.util.Optional;

/**
 * BEFORE: how NOT to write tests. It "passes", but:
 *   - tests several concepts at once, so a failure doesn't say what broke
 *   - uses the real system clock (not Repeatable)
 *   - prints instead of asserting (not Self-validating: a human must read the output)
 *   - cryptic name and magic numbers
 */
public class BeforeStyleTest {
    @Test void test1() {
        var c = new Checkout(sku -> 12_000L, code -> Optional.empty(), r -> {}, e -> {},
Clock.systemDefaultZone());
        long a = c.quote(List.of(new Checkout.Line("X", 3)), null);
        long b = c.quote(List.of(new Checkout.Line("X", 3)), "NOPE");
        var r = c.checkout(List.of(new Checkout.Line("X", 1)), null, "z");
        System.out.println("        a=" + a + " b=" + b + " r=" + r + "  (is this right? someone must check by
eye)");
    }
}
```

```java
// File: lld/ch06/testing/Checkout.java
package lld.ch06.testing;

import java.time.Clock;
import java.time.LocalDate;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CopyOnWriteArrayList;

/** The production code under test: a small checkout with four collaborators behind interfaces. */
public final class Checkout {

    public record Line(String sku, int qty) {}
    public record Coupon(String code, int percentOff, LocalDate expiresOn) {}
    public record Receipt(String email, long totalPaise) {}

    public interface PriceCatalog { long pricePaise(String sku); }
    public interface CouponRepository { Optional<Coupon> find(String code); }
    public interface ReceiptSender { void send(Receipt receipt); }
    public interface AuditLog { void record(String event); }

    static final long HIGH_VALUE_PAISE = 1_000_000;   // Rs 10,000

    private final PriceCatalog catalog;
    private final CouponRepository coupons;
    private final ReceiptSender sender;
    private final AuditLog audit;
    private final Clock clock;

    public Checkout(PriceCatalog catalog, CouponRepository coupons, ReceiptSender sender, AuditLog audit, Clock
clock) {
        this.catalog = catalog; this.coupons = coupons; this.sender = sender; this.audit = audit; this.clock =
clock;
    }

    /**
     * Query: computes the total, changes nothing.
     * {@code couponCode} may be {@code null}, meaning "no coupon" -- a documented part of this API's contract
     * (the "don't pass null" rule allows null only where an API explicitly expects it).
     */
    public long quote(List<Line> lines, String couponCode) {
        long subtotal = lines.stream().mapToLong(l -> catalog.pricePaise(l.sku()) * l.qty()).sum();
        return subtotal - discount(subtotal, couponCode);
    }

    /** Command: charges, sends the receipt, audits big orders. {@code couponCode} may be {@code null} (no
coupon), as in {@link #quote}. */
    public Receipt checkout(List<Line> lines, String couponCode, String email) {
        if (lines.isEmpty()) throw new IllegalArgumentException("cart is empty");
        Receipt receipt = new Receipt(email, quote(lines, couponCode));
        sender.send(receipt);
        if (receipt.totalPaise() >= HIGH_VALUE_PAISE) audit.record("high-value order for " + email);
        return receipt;
    }

    private long discount(long subtotal, String couponCode) {
        if (couponCode == null) return 0;
        return coupons.find(couponCode)
                .filter(c -> !LocalDate.now(clock).isAfter(c.expiresOn()))
                .map(c -> subtotal * c.percentOff() / 100)
                .orElse(0L);
    }

    // ---------- simple real implementations, used by the integration test ----------
    public static final class InMemoryCatalog implements PriceCatalog {
        private final Map<String, Long> prices = new ConcurrentHashMap<>();
        public InMemoryCatalog put(String sku, long paise) { prices.put(sku, paise); return this; }
        public long pricePaise(String sku) {
```

```
            Long p = prices.get(sku);
            if (p == null) throw new IllegalArgumentException("unknown sku " + sku);
            return p;
        }
    }
    public static final class InMemoryCoupons implements CouponRepository {
        private final Map<String, Coupon> byCode = new ConcurrentHashMap<>();
        public InMemoryCoupons save(Coupon c) { byCode.put(c.code(), c); return this; }
        public Optional<Coupon> find(String code) { return Optional.ofNullable(byCode.get(code)); }
    }
    public static final class OutboxSender implements ReceiptSender {
        private final List<Receipt> outbox = new CopyOnWriteArrayList<>();
        public void send(Receipt r) { outbox.add(r); }
        public List<Receipt> outbox() { return List.copyOf(outbox); }
    }
}
```

```
// File: lld/ch06/testing/CheckoutIntegrationTest.java
package lld.ch06.testing;

import static lld.ch06.testing.MiniTest.assertEquals;

import java.time.Clock;
import java.time.Instant;
import java.time.LocalDate;
import java.time.ZoneOffset;
import java.util.List;

/** INTEGRATION test: real collaborating implementations wired together, no doubles for our own code. */
public class CheckoutIntegrationTest {

    @Test void full_checkout_flow_with_real_components() {
        var catalog = new Checkout.InMemoryCatalog().put("TEA", 12_000).put("CUP", 8_000);
        var coupons = new Checkout.InMemoryCoupons().save(new Checkout.Coupon("WELCOME", 25, LocalDate.of(2030, 1,
1)));
        var outbox = new Checkout.OutboxSender();
        var checkout = new Checkout(catalog, coupons, outbox, event -> {}, Clock.fixed(Instant.parse("2026-09-
28T10:00:00Z"), ZoneOffset.UTC));

        checkout.checkout(List.of(new Checkout.Line("TEA", 2), new Checkout.Line("CUP", 1)), "WELCOME",
"meera@example.com");

        assertEquals(List.of(new Checkout.Receipt("meera@example.com", 24_000L)), outbox.outbox());
    }
}
```

```java
// File: lld/ch06/testing/CheckoutTest.java
package lld.ch06.testing;

import static lld.ch06.testing.MiniTest.assertEquals;
import static lld.ch06.testing.MiniTest.assertThrows;
import static lld.ch06.testing.MiniTest.assertTrue;

import java.time.Clock;
import java.time.Instant;
import java.time.LocalDate;
import java.time.ZoneOffset;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.Optional;

/** Clean UNIT tests: one concept each, Given/When/Then, every collaborator replaced by a test double. */
public class CheckoutTest {

    // ===== The five kinds of test double =====

    /** DUMMY: passed only to satisfy a parameter list; must never be used. */
    static final class DummyAuditLog implements Checkout.AuditLog {
        public void record(String event) { throw new AssertionError("dummy must not be called"); }
    }
    /** STUB: returns canned answers to the calls made during the test. */
    static final class StubCatalog implements Checkout.PriceCatalog {
        private final Map<String, Long> prices;
        StubCatalog(Map<String, Long> prices) { this.prices = prices; }
        public long pricePaise(String sku) { return prices.get(sku); }
    }
    /** FAKE: a working but simplified implementation (in-memory instead of a database). */
    static final class FakeCoupons implements Checkout.CouponRepository {
        private final List<Checkout.Coupon> rows = new ArrayList<>();
        FakeCoupons add(Checkout.Coupon c) { rows.add(c); return this; }
        public Optional<Checkout.Coupon> find(String code) { return rows.stream().filter(c ->
c.code().equals(code)).findFirst(); }
    }
    /** SPY: a stub that also RECORDS how it was called, so the test can inspect it afterwards. */
    static final class SpySender implements Checkout.ReceiptSender {
        final List<Checkout.Receipt> sent = new ArrayList<>();
        public void send(Checkout.Receipt r) { sent.add(r); }
    }
    /** MOCK: pre-programmed with EXPECTATIONS and verifies them itself. */
    static final class MockAuditLog implements Checkout.AuditLog {
        private final List<String> expected = new ArrayList<>();
        private final List<String> actual = new ArrayList<>();
        MockAuditLog expect(String event) { expected.add(event); return this; }
        public void record(String event) { actual.add(event); }
        void verify() { assertEquals(expected, actual); }
    }

    // ===== Fixture helpers (a tiny domain-specific testing language) =====
    private static final LocalDate TODAY = LocalDate.of(2026, 9, 28);
    private static final Clock FIXED_CLOCK = Clock.fixed(Instant.parse("2026-09-28T10:00:00Z"), ZoneOffset.UTC);
// Repeatable

    private final StubCatalog catalog = new StubCatalog(Map.of("TEA", 12_000L, "TV", 5_000_000L));
    private final SpySender sender = new SpySender();

    private Checkout checkoutWith(Checkout.CouponRepository coupons, Checkout.AuditLog audit) {
        return new Checkout(catalog, coupons, sender, audit, FIXED_CLOCK);
    }
    private static List<Checkout.Line> cart(Object... skuQtyPairs) {
        List<Checkout.Line> lines = new ArrayList<>();
        for (int i = 0; i < skuQtyPairs.length; i += 2) lines.add(new Checkout.Line((String) skuQtyPairs[i],
(Integer) skuQtyPairs[i + 1]));
        return lines;
    }
```

```
    // ===== Tests =====
    @Test void quote_sums_price_times_quantity() {
        Checkout checkout = checkoutWith(new FakeCoupons(), new DummyAuditLog());   // Given
        long total = checkout.quote(cart("TEA", 3), null);                          // When
        assertEquals(36_000L, total);                                                // Then
    }

    @Test void valid_coupon_takes_percentage_off() {
        var coupons = new FakeCoupons().add(new Checkout.Coupon("DIWALI10", 10, TODAY));
        long total = checkoutWith(coupons, new DummyAuditLog()).quote(cart("TEA", 10), "DIWALI10");
        assertEquals(108_000L, total);
    }

    @Test void expired_coupon_is_ignored() {
        var coupons = new FakeCoupons().add(new Checkout.Coupon("OLD", 50, TODAY.minusDays(1)));
        long total = checkoutWith(coupons, new DummyAuditLog()).quote(cart("TEA", 1), "OLD");
        assertEquals(12_000L, total);
    }

    @Test void checkout_sends_exactly_one_receipt() {
        checkoutWith(new FakeCoupons(), new DummyAuditLog()).checkout(cart("TEA", 1), null, "asha@example.com");
        assertEquals(List.of(new Checkout.Receipt("asha@example.com", 12_000L)), sender.sent);   // inspect the
spy
    }

    @Test void high_value_order_is_audited() {
        var audit = new MockAuditLog().expect("high-value order for ravi@example.com");
        checkoutWith(new FakeCoupons(), audit).checkout(cart("TV", 1), null, "ravi@example.com");
        audit.verify();                                                              // the mock checks itself
    }

    @Test void empty_cart_is_rejected() {
        var checkout = checkoutWith(new FakeCoupons(), new DummyAuditLog());
        var e = assertThrows(IllegalArgumentException.class, () -> checkout.checkout(List.of(), null,
"x@example.com"));
        assertTrue(e.getMessage().contains("empty"), "message should explain the problem");
        assertTrue(sender.sent.isEmpty(), "nothing must be sent");
    }
}
```

```
// File: lld/ch06/testing/Main.java
package lld.ch06.testing;

/** Runs all test classes with the MiniTest runner and prints a summary. */
public class Main {
    public static void main(String[] args) {
        MiniTest.Summary total = new MiniTest.Summary(0, 0);
        for (Class<?> c : new Class<?>[]{BeforeStyleTest.class, CheckoutTest.class,
CheckoutIntegrationTest.class}) {
            total = total.plus(MiniTest.run(c));
        }
        System.out.println("TOTAL: " + total.passed() + " passed, " + total.failed() + " failed");
        if (total.failed() > 0) System.exit(1);
    }
}
```

```java
// File: lld/ch06/testing/MiniTest.java
package lld.ch06.testing;

import java.lang.reflect.InvocationTargetException;
import java.lang.reflect.Method;
import java.util.Arrays;
import java.util.Comparator;
import java.util.Objects;

/**
 * A ~60-line, JUnit-free test runner.
 * - finds methods annotated with @Test (reflection)
 * - creates a FRESH instance of the test class for every test (tests stay Independent)
 * - reports PASS / FAIL / ERROR by itself (Self-validating)
 */
public final class MiniTest {
    private MiniTest() {}

    public record Summary(int passed, int failed) {
        public Summary plus(Summary o) { return new Summary(passed + o.passed, failed + o.failed); }
    }

    public static Summary run(Class<?> testClass) {
        System.out.println(testClass.getSimpleName());
        int passed = 0, failed = 0;
        Method[] tests = Arrays.stream(testClass.getDeclaredMethods())
                .filter(m -> m.isAnnotationPresent(Test.class))
                .sorted(Comparator.comparing(Method::getName))      // stable output; tests must not depend on
order
                .toArray(Method[]::new);
        for (Method m : tests) {
            String name = m.getName().replace('_', ' ');
            try {
                Object instance = testClass.getDeclaredConstructor().newInstance();
                m.setAccessible(true);
                m.invoke(instance);
                passed++;
                System.out.println("  PASS  " + name);
            } catch (InvocationTargetException e) {
                failed++;
                Throwable cause = e.getCause();
                String kind = cause instanceof AssertionError ? "FAIL " : "ERROR";
                System.out.println("  " + kind + " " + name + " -> " + cause);
            } catch (ReflectiveOperationException e) {
                failed++;
                System.out.println("  ERROR " + name + " -> cannot run: " + e);
            }
        }
        return new Summary(passed, failed);
    }

    // ---------- assertions ----------
    public static void assertEquals(Object expected, Object actual) {
        if (!Objects.equals(expected, actual))
            throw new AssertionError("expected <" + expected + "> but was <" + actual + ">");
    }
    public static void assertTrue(boolean condition, String message) {
        if (!condition) throw new AssertionError(message);
    }
    public static <T extends Throwable> T assertThrows(Class<T> type, Runnable action) {
        try {
            action.run();
        } catch (Throwable t) {
            if (type.isInstance(t)) return type.cast(t);
            throw new AssertionError("expected " + type.getSimpleName() + " but got " + t);
        }
        throw new AssertionError("expected " + type.getSimpleName() + " but nothing was thrown");
    }
}
```

```java
// File: lld/ch06/testing/Test.java
package lld.ch06.testing;

import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

/** Marks a method as a test for {@link MiniTest} -- our tiny stand-in for JUnit's @Test. */
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface Test {}
```

Output:

```
BeforeStyleTest
        a=36000 b=36000 r=Receipt[email=z, totalPaise=12000]  (is this right? someone must check by eye)
  PASS  test1
CheckoutTest
  PASS  checkout sends exactly one receipt
  PASS  empty cart is rejected
  PASS  expired coupon is ignored
  PASS  high value order is audited
  PASS  quote sums price times quantity
  PASS  valid coupon takes percentage off
CheckoutIntegrationTest
  PASS  full checkout flow with real components
TOTAL: 8 passed, 0 failed
```

Reading the output: `BeforeStyleTest.test1` "passes" but proves nothing — it checks three concepts, uses the real clock, prints values for a human to judge and has a meaningless name. The `CheckoutTest` methods each check **one concept**, are named as behaviour specifications (`expired_coupon_is_ignored`), use Given/When/Then, a fixed clock and self-validating assertions. `CheckoutIntegrationTest` wires the real in-memory components together (see §17 for unit vs integration testing and the five kinds of test double used here).

Trade-offs

Well-tested code	Untested code
Fearless refactoring	Fear of change → rot
Tests are executable documentation	Knowledge in people's heads
Test code must be maintained too	Faster to write the first time

When not to use it

TDD is less useful for exploratory spikes (quick experiments to learn something — throw them away afterwards) and for pure UI layout. Don't unit-test trivial getters or framework code you don't own (use learning tests for that instead).

Common mistakes

- Testing implementation details (private methods, exact call order) so tests break on every refactor.
- Shared mutable fixtures (the data and objects set up before tests) that make tests order-dependent.
- Using `Thread.sleep` or the real clock/network in unit tests.
- Test names like `test1`, `testCheckout`.

Real-world examples

- **JUnit 5** (`@Test`, `@BeforeEach`, `assertThrows`, `@DisplayName`), **AssertJ** for fluent assertions, **Mockito** for doubles.

- JUnit 5's default lifecycle creates a **new test-class instance per test method** — the same independence choice `MiniTest` makes.
- `java.time.Clock` exists partly so time-dependent code can be tested with `Clock.fixed(...)`.

In the interview

> **30-second answer:** "I write small, fast, independent tests — one concept each, named as behaviour, arranged Given/When/Then. I inject collaborators behind interfaces so I can use stubs, fakes or mocks, and inject a `Clock` for time. In a machine-coding round I at least write a `main` that exercises key flows, and a few unit tests for the core logic if time allows."

Follow-ups:

- *Q: What are the three laws of TDD?* A: No production code without a failing test; write only enough test to fail; write only enough code to pass.
- *Q: What does F.I.R.S.T. stand for?* A: Fast, Independent, Repeatable, Self-validating, Timely.
- *Q: One assert per test?* A: The real rule is one concept per test; several asserts on one concept are fine.

Self-check

1. Q: What's wrong with printing results in a test? → A: Not self-validating — a human has to check.
2. Q: How do you make a time-based test repeatable? → A: Inject a `Clock` and use `Clock.fixed` in tests.
3. Q: What is Build–Operate–Check? → A: Arrange data, act on it, assert results (Given/When/Then).

TL;DR

Tests are first-class code: TDD cycle, one concept per test, readable Given/When/Then, and F.I.R.S.T.

---

9. Classes

What it is

Rules for clean classes: follow the standard **organisation**, keep classes **small** (measured in responsibilities, not lines), give each **one reason to change** (SRP), keep them **cohesive**, and **organise them for change** so new features mean new classes rather than edits to old ones.

Analogy: a well-run office has departments with clear jobs. When a new service is added, you open a new desk; you don't make accounting also handle legal.

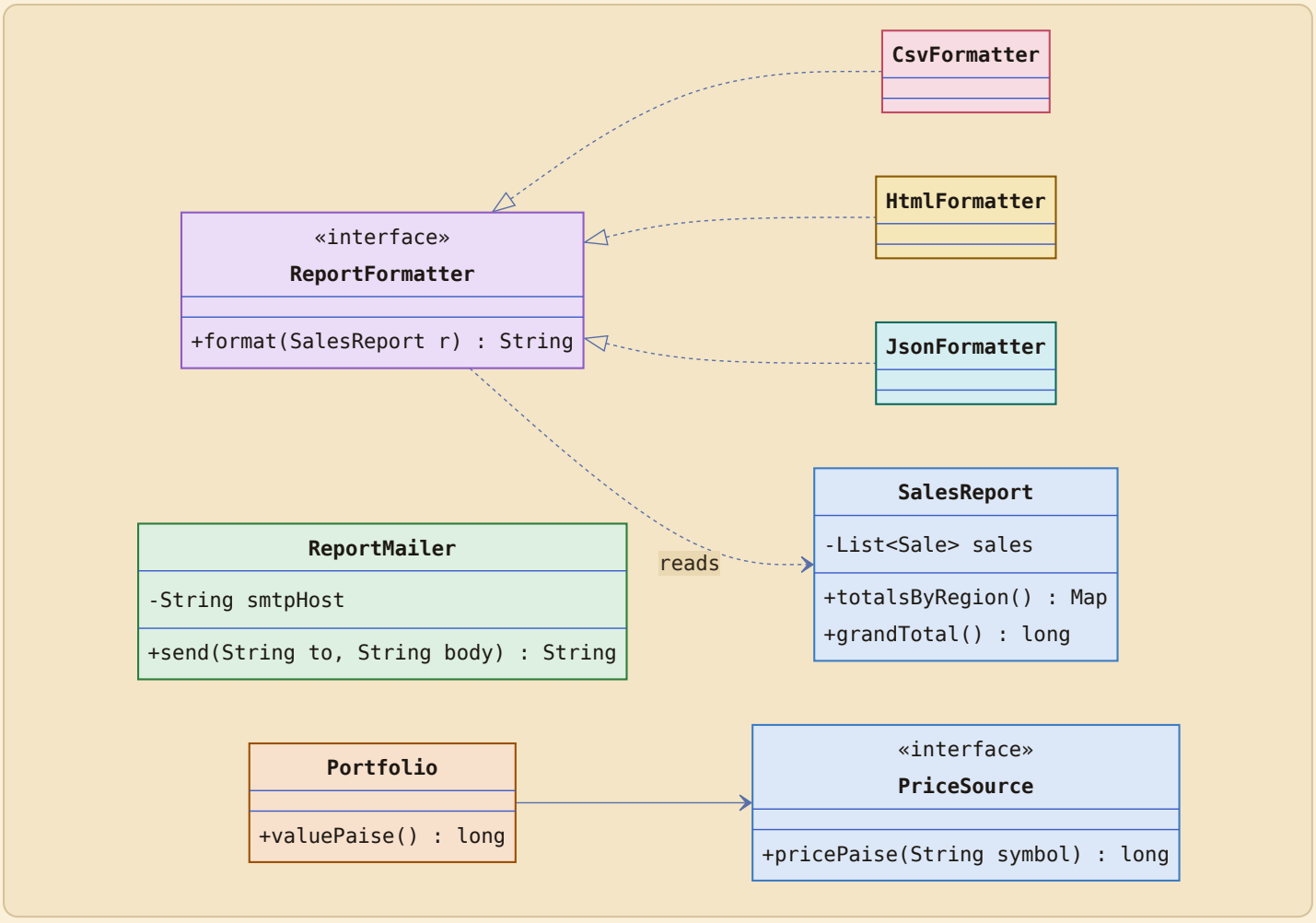
The problem it solves

Big "god" classes with many responsibilities are hard to understand, change for many reasons, and every change risks breaking unrelated features. A long list of `if/else` branches for formats or types means reopening the same class repeatedly.

How it works

1. **Class organisation** (Java convention): public static constants → private static variables → private instance variables → public functions → private helpers right after the public function that uses them (stepdown rule). Public instance variables are rarely justified.
2. **Encapsulation**: keep fields and helpers private. Loosening to package-private/protected (visible only within the same package, or also to subclasses) *for tests* is acceptable as a last resort, but look for another way first.
3. **Classes should be small** — measured by *responsibilities*. The class name should describe them; if you can't name it concisely, or the name needs words like *Manager*, *Processor*, *Super*, or you describe it with "and"/"or"/"but", it does too much. You should be able to describe the class in about 25 words.

4. **Single Responsibility Principle**: a class should have **one reason to change** (Chapter 4). Many small classes are better than a few large ones — the total amount of code is the same; small classes are just better organised, like tools in labelled drawers instead of one big box.
5. **Cohesion**: a class is maximally cohesive if every method uses every instance variable. Aim for high cohesion. When a subset of methods uses a subset of fields, that subset wants to be its own class — **maintaining cohesion results in many small classes**.
6. **Organise for change**: when adding a feature means *opening* a class to edit it (e.g. adding a branch to `render()`), consider splitting so that new features are new classes (Open/Closed Principle). Each report format became its own `ReportFormatter`.
7. **Isolate from change**: depend on abstractions (interfaces) rather than volatile concrete details. `Portfolio` depends on `PriceSource`, not a live stock-exchange client, so it can be tested with fixed prices (Dependency Inversion Principle).



Java example — before/after

```java
// File: lld/ch06/classes/Main.java
package lld.ch06.classes;

import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;

/** Classes: small (few responsibilities), cohesive, organised for change, isolated from change. */
public class Main {

    record Sale(String region, long amountPaise) {}

    // ---------------- BEFORE: one class, many reasons to change ----------------
    static class Before {
        static class ReportManager {
            private final List<Sale> sales;
            private String smtpHost = "smtp.example.com";          // email concern
            private String outputFormat = "CSV";                   // formatting concern
            ReportManager(List<Sale> sales) { this.sales = sales; }
            void setOutputFormat(String f) { outputFormat = f; }
            Map<String, Long> totalsByRegion() {                   // business concern
                return sales.stream().collect(Collectors.groupingBy(Sale::region, java.util.TreeMap::new,
                        Collectors.summingLong(Sale::amountPaise)));
            }
            String render() {                                      // grows a new branch for every format
                if (outputFormat.equals("CSV")) {
                    StringBuilder sb = new StringBuilder("region,total\n");
                    totalsByRegion().forEach((r, t) -> sb.append(r).append(',').append(t).append('\n'));
                    return sb.toString().trim();
                } else if (outputFormat.equals("HTML")) {
                    StringBuilder sb = new StringBuilder("<table>");
                    totalsByRegion().forEach((r, t) -> sb.append("<tr><td>").append(r).append("</td>
<td>").append(t).append("</td></tr>"));
                    return sb.append("</table>").toString();
                }
                throw new IllegalStateException("unknown format");
            }
            String email(String to) { return "SMTP " + smtpHost + " -> " + to + ": " + render(); }
        }
    }

    // ---------------- AFTER ----------------
    static class After {
        /** Cohesive: every method uses the same field (sales). One reason to change: the report's business rules.
*/
        static final class SalesReport {
            private final List<Sale> sales;
            SalesReport(List<Sale> sales) { this.sales = List.copyOf(sales); }
            Map<String, Long> totalsByRegion() {
                return sales.stream().collect(Collectors.groupingBy(Sale::region, java.util.TreeMap::new,
                        Collectors.summingLong(Sale::amountPaise)));
            }
            long grandTotal() { return sales.stream().mapToLong(Sale::amountPaise).sum(); }
        }

        /** Organised for change: a new format is a NEW class; existing formatters are never reopened (OCP). */
        interface ReportFormatter { String format(SalesReport report); }

        static final class CsvFormatter implements ReportFormatter {
            public String format(SalesReport r) {
                StringBuilder sb = new StringBuilder("region,total\n");
                r.totalsByRegion().forEach((region, total) ->
sb.append(region).append(',').append(total).append('\n'));
                return sb.toString().trim();
            }
        }
        static final class HtmlFormatter implements ReportFormatter {
            public String format(SalesReport r) {
                StringBuilder sb = new StringBuilder("<table>");
```

```
                r.totalsByRegion().forEach((region, total) ->
                        sb.append("<tr><td>").append(region).append("</td><td>").append(total).append("</td>
</tr>"));
                return sb.append("</table>").toString();
            }
        }
        static final class JsonFormatter implements ReportFormatter {   // added later, nothing else edited
            public String format(SalesReport r) {
                return r.totalsByRegion().entrySet().stream()
                        .map(e -> "\"" + e.getKey() + "\":" + e.getValue())
                        .collect(Collectors.joining(",", "{", "}"));
            }
        }

        /** Separate reason to change: delivery. */
        static final class ReportMailer {
            private final String smtpHost;
            ReportMailer(String smtpHost) { this.smtpHost = smtpHost; }
            String send(String to, String body) { return "SMTP " + smtpHost + " -> " + to + ": " + body; }
        }

        /** Isolating from change: Portfolio depends on an abstraction, not on a live stock-exchange client. */
        interface PriceSource { long pricePaise(String symbol); }
        static final class Portfolio {
            private final PriceSource prices;
            private final Map<String, Integer> holdings;
            Portfolio(PriceSource prices, Map<String, Integer> holdings) { this.prices = prices; this.holdings =
Map.copyOf(holdings); }
            long valuePaise() {
                return holdings.entrySet().stream().mapToLong(h -> prices.pricePaise(h.getKey()) *
h.getValue()).sum();
            }
        }
    }

    public static void main(String[] args) {
        List<Sale> sales = List.of(new Sale("North", 5_000), new Sale("South", 7_000), new Sale("North", 1_000));

        System.out.println("== BEFORE ==");
        var manager = new Before.ReportManager(sales);
        System.out.println(manager.render().replace('\n', ' '));
        manager.setOutputFormat("HTML");
        System.out.println(manager.email("boss@example.com"));

        System.out.println("== AFTER ==");
        var report = new After.SalesReport(sales);
        for (After.ReportFormatter f : List.of(new After.CsvFormatter(), new After.HtmlFormatter(), new
After.JsonFormatter()))
            System.out.println(f.getClass().getSimpleName() + ": " + f.format(report).replace('\n', ' '));
        System.out.println(new After.ReportMailer("smtp.example.com").send("boss@example.com", new
After.JsonFormatter().format(report)));

        After.PriceSource fixedPrices = symbol -> Map.of("INFY", 150_000L, "TCS", 400_000L).get(symbol);  // test-
friendly stub
        var portfolio = new After.Portfolio(fixedPrices, Map.of("INFY", 10, "TCS", 2));
        System.out.println("portfolio value (paise) = " + portfolio.valuePaise());
    }
}
```

Output:

```
== BEFORE ==
region,total North,6000 South,7000
SMTP smtp.example.com -> boss@example.com: <table><tr><td>North</td><td>6000</td></tr><tr><td>South</td>
<td>7000</td></tr></table>
== AFTER ==
CsvFormatter: region,total North,6000 South,7000
HtmlFormatter: <table><tr><td>North</td><td>6000</td></tr><tr><td>South</td><td>7000</td></tr></table>
JsonFormatter: {"North":6000,"South":7000}
SMTP smtp.example.com -> boss@example.com: {"North":6000,"South":7000}
portfolio value (paise) = 2300000
```

`ReportManager` had three reasons to change (business totals, output format, email delivery) and an `if/else` that grows with every format. After the split, `JsonFormatter` was added without editing any existing class, and `Portfolio` is testable because it depends on the `PriceSource` abstraction.

Trade-offs

Many small classes	Few big classes
Each is easy to understand and test	Fewer files to navigate
Changes are localised	Every change touches the same class
Need good names and packages to navigate	Merge conflicts (clashing edits to the same lines), fear of change

When not to use it

Don't split a small cohesive class into many micro-classes that always change together — that's fragmentation, not SRP.

Common mistakes

- Classes named `*Manager`, `*Helper`, `*Utils` collecting unrelated methods.
- Splitting by *technical layer* only, leaving a god service.
- Making everything `public` "for tests".

Real-world examples

- JDK's `java.time` splits responsibilities across `LocalDate`, `LocalTime`, `ZonedDateTime`, `Duration`, `Period`, `DateTimeFormatter` — versus the old `java.util.Date`/`Calendar`, which mixed many concerns.
- Spring's `JdbcTemplate` delegates row mapping to `RowMapper` and parameter setting to `PreparedStatementSetter`.

In the interview

30-second answer: "I keep classes small by responsibility — one reason to change — and highly cohesive, so methods use most of the fields. When a feature would mean adding another branch to an existing class, I split so the new feature is a new class, and I depend on interfaces to isolate volatile details."

Self-check

1. Q: How do you measure a class's size? → A: By its responsibilities, not lines.
2. Q: What is maximal cohesion? → A: Every method uses every instance variable.
3. Q: What does "organise for change" mean? → A: Structure classes so new features are added as new classes, not edits.

TL;DR

Small (one responsibility), cohesive, open for extension, dependent on abstractions.

---

10. Systems

What it is

At the system level, clean code means **separating the startup process (constructing and wiring objects) from the runtime logic that uses them**, using **Dependency Injection**, handling **cross-cutting concerns** in one place, and letting the architecture **grow incrementally**.

Analogy: building a city. Nobody designs every street up front, and no single person manages everything. Cities work because of abstraction and modularity — water, power, traffic are separate systems with clear interfaces, and they grow over time.

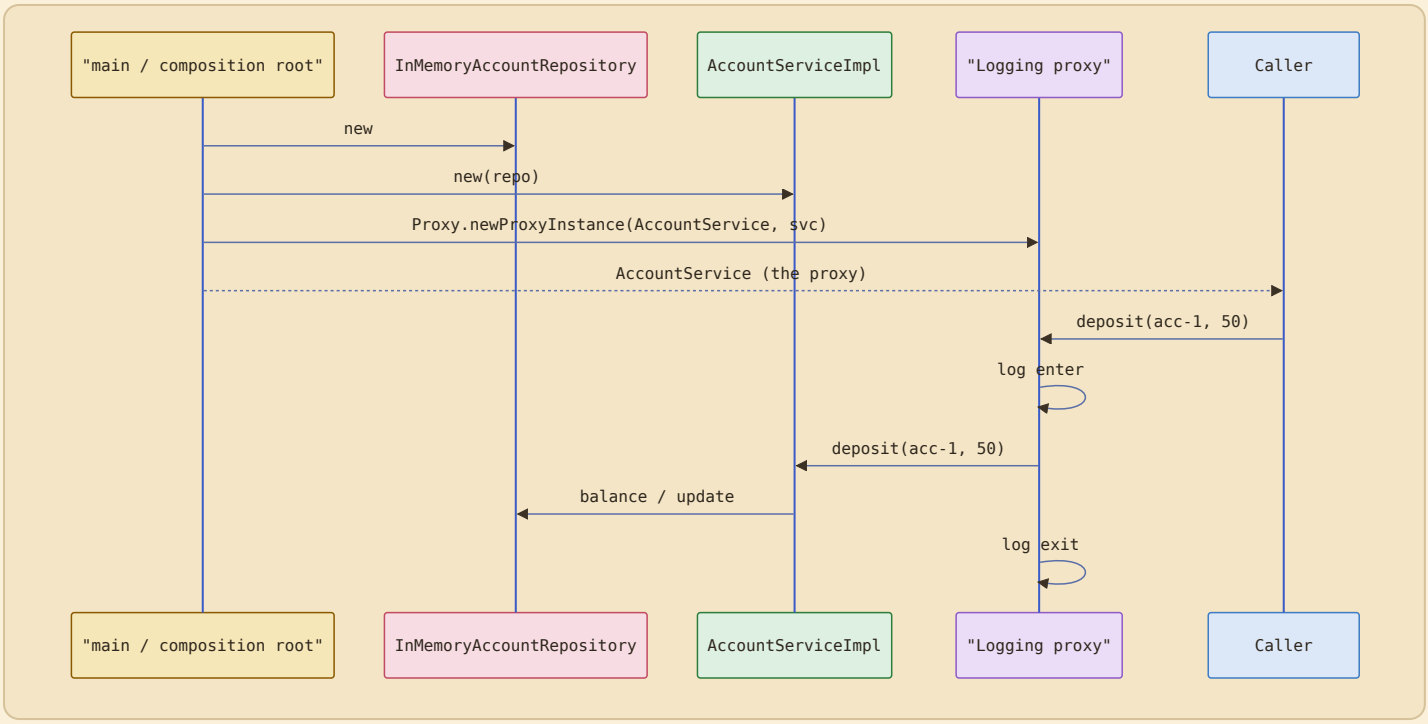
The problem it solves

When objects create their own dependencies (`if (service == null) service = new MyServiceImpl(...)` — lazy initialisation, i.e. creating something only the first time it is needed, inside business code), the business code is hard-wired to concrete classes, can't be tested in isolation, mixes construction with use, and violates SRP. When logging, security and transactions are copied into every method, every business method becomes noisy and inconsistent.

How it works

1. **Separate construction from use.**
   - **Separation of `main`**: build all objects in `main` (or a module called at startup — the **composition root**) and pass them to the application. The application knows nothing about how its objects were built; dependencies point from `main` toward the application, never back.
   - **Factories**: when the *application* must decide *when* to create something (e.g. a `LineItem` for each scan), give it an **Abstract Factory** interface (an interface whose methods create objects, so the caller never names the concrete classes); the concrete factory still lives on the `main` side.
   - **Dependency Injection (DI)**: an object receives its dependencies (constructor, setter or method parameter) instead of creating or looking them up. DI is the application of Inversion of Control (outside code, often a framework, takes charge of creating objects and driving the flow instead of your class) to dependency management. Constructor injection is preferred: dependencies are explicit, final, and the object is complete after construction.
2. **Scale up incrementally.** Build today's system well-modularised; well-separated concerns let the architecture grow (e.g. from a monolith — one application deployed as a single unit — module to a separate service) without a rewrite.
3. **Cross-cutting concerns** (persistence, transactions, security, logging, caching — keeping recent results in fast storage for reuse; see System Design Fundamentals) cut across many objects. Handle them modularly:
   - **Java dynamic proxies** (`java.lang.reflect.Proxy`) wrap an interface and intercept every call — shown in our code.
   - **Pure-Java AOP frameworks** (AOP = aspect-oriented programming: write a concern such as logging once, as an *aspect*, and apply it to many methods; e.g. Spring AOP) generate proxies for you from annotations (`@`-tags on code)/config (`@Transactional`, `@Cacheable`, `@PreAuthorize`).
   - **AspectJ** — a full aspect-oriented language extension that weaves advice (the aspect's code) into bytecode (the compiled form of Java classes that the JVM runs).
   - Also: **decorators** and **servlet filters / interceptors** (code that runs before/after every web request) for simpler cases.
4. **Test-drive the architecture**: with business logic in plain objects (POJOs — Plain Old Java Objects, with no framework dependencies) and concerns decoupled, you can start simple and evolve the architecture as needs become clear.
5. **Optimise decision making**: postpone decisions until the last responsible moment, when you have the most information; modularity makes postponement possible.
6. **Use standards wisely** — only when they add demonstrable value (heavyweight standards can bury simple logic).

7. **Systems need domain-specific languages**: a good DSL (domain-specific language — a mini-language tailored to one problem) (or fluent API — methods designed to chain into readable sentences) lets code read like the domain (e.g. a test-fixture builder or a routing DSL).



Java example — before/after

```java
// File: lld/ch06/systems/Main.java
package lld.ch06.systems;

import java.lang.reflect.InvocationHandler;
import java.lang.reflect.InvocationTargetException;
import java.lang.reflect.Proxy;
import java.util.HashMap;
import java.util.Map;

/** Systems: separate construction from use, inject dependencies, handle cross-cutting concerns in one place. */
public class Main {

    interface AccountRepository { long balance(String id); void update(String id, long balance); }
    interface AccountService { long balance(String id); void deposit(String id, long amount); }

    static final class InMemoryAccountRepository implements AccountRepository {
        private final Map<String, Long> rows = new HashMap<>(Map.of("acc-1", 100L));
        public long balance(String id) { return rows.getOrDefault(id, 0L); }
        public void update(String id, long b) { rows.put(id, b); }
    }

    // ---------------- BEFORE: the service builds its own dependency (lazy init) and mixes in logging/timing ----------
    static class Before {
        static final class AccountServiceImpl implements AccountService {
            private AccountRepository repo;
            private AccountRepository repo() {
                if (repo == null) repo = new InMemoryAccountRepository();   // construction mixed with use; hard-coded impl
                return repo;
            }
            public long balance(String id) {
                System.out.println("  LOG enter balance " + id);          // cross-cutting code copy #1
                long b = repo().balance(id);
                System.out.println("  LOG exit balance = " + b);
                return b;
            }
            public void deposit(String id, long amount) {
                System.out.println("  LOG enter deposit " + id);          // cross-cutting code copy #2
                if (amount <= 0) throw new IllegalArgumentException("amount must be positive");
                repo().update(id, repo().balance(id) + amount);
                System.out.println("  LOG exit deposit");
            }
        }
    }

    // ---------------- AFTER ----------------
    static class After {
        /** Pure business logic. Gets its dependency from outside (constructor injection). No logging code. */
        static final class AccountServiceImpl implements AccountService {
            private final AccountRepository repo;
            AccountServiceImpl(AccountRepository repo) { this.repo = repo; }
            public long balance(String id) { return repo.balance(id); }
            public void deposit(String id, long amount) {
                if (amount <= 0) throw new IllegalArgumentException("amount must be positive");
                repo.update(id, repo.balance(id) + amount);
            }
        }

        /** Cross-cutting concern written ONCE and applied to any interface via a JDK dynamic proxy (how Spring AOP works for interfaces). */
        @SuppressWarnings("unchecked")
        static <T> T withLogging(Class<T> type, T target) {
            InvocationHandler handler = (proxy, method, args) -> {
                String argText = args == null ? "" : String.join(", ", java.util.Arrays.stream(args).map(String::valueOf).toList());
                System.out.println("  LOG -> " + method.getName() + "(" + argText + ")");
                try {
                    Object result = method.invoke(target, args);
```

```
                    System.out.println("  LOG <- " + method.getName() + (result == null ? "" : " = " + result));
                    return result;
                } catch (InvocationTargetException e) {
                    System.out.println("  LOG !! " + method.getName() + " threw " + e.getCause().getMessage());
                    throw e.getCause();                         // rethrow the real exception, not the reflection
wrapper
                }
            };
            return (T) Proxy.newProxyInstance(type.getClassLoader(), new Class<?>[]{type}, handler);
        }

        /** Composition root: the ONLY place that knows concrete classes and wires them together (the "main"
side). */
        static AccountService buildApplication() {
            AccountRepository repo = new InMemoryAccountRepository();
            AccountService core = new AccountServiceImpl(repo);
            return withLogging(AccountService.class, core);
        }
    }

    public static void main(String[] args) {
        System.out.println("== BEFORE ==");
        var before = new Before.AccountServiceImpl();
        before.deposit("acc-1", 50);
        System.out.println("balance = " + before.balance("acc-1"));

        System.out.println("== AFTER ==");
        AccountService service = After.buildApplication();      // construction happens here, once
        service.deposit("acc-1", 50);                            // use happens everywhere else
        service.balance("acc-1");
        try { service.deposit("acc-1", -5); } catch (IllegalArgumentException e) { System.out.println("caller
sees: " + e.getMessage()); }

        // In a unit test, construct the core with a different repository -- no logging, no real storage.
        var testService = new After.AccountServiceImpl(new InMemoryAccountRepository());
        testService.deposit("acc-1", 1);
        System.out.println("test double wiring works: " + testService.balance("acc-1"));
    }
}
```

Output:

```
== BEFORE ==
  LOG enter deposit acc-1
  LOG exit deposit
  LOG enter balance acc-1
  LOG exit balance = 150
balance = 150
== AFTER ==
  LOG -> deposit(acc-1, 50)
  LOG <- deposit
  LOG -> balance(acc-1)
  LOG <- balance = 150
  LOG -> deposit(acc-1, -5)
  LOG !! deposit threw amount must be positive
caller sees: amount must be positive
test double wiring works: 101
```

In *before*, the service builds its own repository and repeats logging in every method. In *after*, `buildApplication()` (the composition root) is the only place that knows concrete classes; `AccountServiceImpl` receives its repository via its constructor; logging is written **once** in an `InvocationHandler` and applied through a JDK dynamic proxy. The last lines show that tests can wire the core differently. The proxy rethrows the *original* exception (`e.getCause()`), not the reflection wrapper.

Trade-offs

DI + composition root + proxies	Self-constructing objects
Testable, swappable, one place for wiring	Simple for tiny programs
Cross-cutting code in one place	Duplicated, inconsistent concerns
"Magic" with frameworks; proxies only intercept calls *through* the proxy	Explicit and visible

A real Spring gotcha that follows from the proxy mechanism: calling a `@Transactional` method *from another method of the same bean* (an object managed by Spring) (self-invocation) bypasses the proxy, so no transaction starts.

When not to use it

Small scripts and tiny interview programs don't need a DI framework — manual constructor injection in `main` is enough (and is what interviewers expect).

Common mistakes

- Service Locator (a global registry that code asks for its dependencies) calls (`Registry.get(Foo.class)`) scattered in business code — hides dependencies.
- Field injection with `@Autowired` on private fields — hides dependencies and blocks immutability; prefer constructor injection.
- Business logic depending on framework annotations everywhere.

Real-world examples

- **Spring IoC container / Spring Boot**, **Google Guice**, **Dagger** (compile-time DI, common on Android), **Jakarta CDI**.
- **Spring AOP** implements `@Transactional`, `@Cacheable`, `@Async` via JDK dynamic proxies (for interfaces) or CGLIB subclass proxies (generated subclasses that wrap the real class).
- Servlet filters and Spring `HandlerInterceptor`s for authentication and request logging.

In the interview

> **30-second answer:** "I separate construction from use: a composition root in `main` builds and wires everything, and classes receive dependencies through constructors. Cross-cutting concerns like logging or transactions go in one place — a decorator, proxy or aspect — not in every method. That keeps the domain in plain, testable objects."

Follow-ups:

- *Q: DI vs Service Locator?* A: DI pushes dependencies in (explicit in the constructor); a locator pulls them (hidden, harder to test).
- *Q: How does Spring implement `@Transactional`?* A: With a proxy around the bean that begins/commits/rolls back around the method call.

Self-check

1. Q: What is the composition root? → A: The one place (near `main`) where objects are constructed and wired.
2. Q: Name three ways to handle cross-cutting concerns in Java. → A: Dynamic proxies, AOP frameworks/AspectJ, decorators (also filters/interceptors).

TL;DR

Build objects in one place and inject them; put cross-cutting concerns in proxies/aspects; grow the architecture incrementally.

---

11. Emergence — Kent Beck's Four Rules of Simple Design

What it is

Good design can **emerge** from following four simple rules. In priority order, a design is simple if it:

1. **Runs all the tests** (passes them).
2. **Contains no duplication.**
3. **Expresses the intent of the programmer.**
4. **Minimises the number of classes and methods.**

(*Clean Code*'s Chapter 12, written by Jeff Langr, lists them in this order. **Kent Beck's own usual wording and order** — as summarised in Martin Fowler's "BeckDesignRules" article — is: **1. Passes the tests, 2. Reveals intention, 3. No duplication, 4. Fewest elements.** Fowler notes that the middle two are the ones people swap, and that it rarely matters because they reinforce each other. Rules 1 and 4 are always first and last. In an interview, quote Beck's order and mention that *Clean Code* swaps 2 and 3.)

Analogy: sculpting — first make sure the statue stands (tests), then remove what's repeated, then refine the expression, then trim anything unnecessary.

The problem it solves

Design decisions can feel overwhelming. These four rules give a simple, ordered checklist that consistently pushes code toward SRP, DIP and good structure.

How it works

1. **Runs all the tests** — a system you can't verify shouldn't be deployed. Making code testable pushes you toward small, single-purpose, loosely coupled classes (you need DI and interfaces to test in isolation). So rule 1 alone drives much good design.
2. **Refactor, guided by rules 2–4.** With tests in place, clean up without fear after every few lines:
   - **No duplication** — duplicate lines, and duplicate *implementation* (e.g. `isEmpty()` could be implemented as `size() == 0`). Extract methods, and use patterns like **Template Method** to remove higher-level duplication (the shared algorithm with varying steps).
   - **Expressive** — good names, small functions and classes, standard pattern names in class names (`...Command`, `...Visitor`), well-written tests as documentation. Most importantly: *care*.
   - **Minimal classes and methods** — don't create an interface for every class or split fields from behaviour dogmatically. This rule has the lowest priority: SRP and no-duplication win when they conflict.

Java example — before/after

```
// File: lld/ch06/emergence/Main.java
package lld.ch06.emergence;

import lld.ch06.testing.MiniTest;
import lld.ch06.testing.Test;

/**
 * Kent Beck's four rules of simple design, in the priority order Clean Code gives:
 *  1. runs all the tests   2. no duplication   3. expresses intent   4. fewest classes and methods
 */
public class Main {

    record Employee(String name, int monthsWorked, boolean onProbation) {}

    // ---------------- BEFORE: duplicated algorithm, magic numbers, needless layers ----------------
    static class Before {
        interface LeaveCalcStrategyProvider { LeaveCalc get(); }            // extra element with no purpose
        static class LeaveCalc {
            int calcIN(Employee e) {                                         // unclear name
                if (e.onProbation()) return 0;
                int d = e.monthsWorked() * 18 / 12;                          // 18 per year? magic
                if (d > 45) d = 45;                                          // 45 cap? magic
                return d;
            }
            int calcUS(Employee e) {                                         // the same algorithm, copied
                if (e.onProbation()) return 0;
                int d = e.monthsWorked() * 15 / 12;
                if (d > 30) d = 30;
                return d;
            }
        }
    }

    // ---------------- AFTER: one algorithm (Template Method), named rules, only needed elements ----------------
    static class After {
        abstract static class LeavePolicy {
            /** The shared algorithm, written once (rule 2: no duplication). */
            final int earnedLeaveDays(Employee e) {
                if (e.onProbation()) return 0;
                return Math.min(daysPerYear() * e.monthsWorked() / 12, maxCarryForwardDays());
            }
            protected abstract int daysPerYear();              // what varies, with intention-revealing names
(rule 3)
            protected abstract int maxCarryForwardDays();
        }
        static final class IndiaLeavePolicy extends LeavePolicy {
            protected int daysPerYear() { return 18; }
            protected int maxCarryForwardDays() { return 45; }
        }
        static final class UsLeavePolicy extends LeavePolicy {
            protected int daysPerYear() { return 15; }
            protected int maxCarryForwardDays() { return 30; }
        }
    }

    /** Rule 1 first: the refactoring is only safe because these tests pass before AND after. */
    public static class LeavePolicyTest {
        private final Employee veteran = new Employee("Asha", 48, false);
        private final Employee newHire = new Employee("Ravi", 8, false);
        private final Employee probationer = new Employee("Kabir", 3, true);

        @Test void india_policy_matches_old_behaviour() {
            var old = new Before.LeaveCalc();
            var policy = new After.IndiaLeavePolicy();
            for (Employee e : new Employee[]{veteran, newHire, probationer})
                MiniTest.assertEquals(old.calcIN(e), policy.earnedLeaveDays(e));
        }
        @Test void us_policy_matches_old_behaviour() {
            var old = new Before.LeaveCalc();
```

```
            var policy = new After.UsLeavePolicy();
            for (Employee e : new Employee[]{veteran, newHire, probationer})
                MiniTest.assertEquals(old.calcUS(e), policy.earnedLeaveDays(e));
        }
        @Test void carry_forward_is_capped() {
            MiniTest.assertEquals(45, new After.IndiaLeavePolicy().earnedLeaveDays(veteran));
        }
        @Test void probationers_earn_nothing() {
            MiniTest.assertEquals(0, new After.UsLeavePolicy().earnedLeaveDays(probationer));
        }
    }

    public static void main(String[] args) {
        var summary = MiniTest.run(LeavePolicyTest.class);
        System.out.println(summary.passed() + " passed, " + summary.failed() + " failed");
        var in = new After.IndiaLeavePolicy();
        var us = new After.UsLeavePolicy();
        for (Employee e : new Employee[]{new Employee("Asha", 48, false), new Employee("Ravi", 8, false)})
            System.out.println(e.name() + ": India " + in.earnedLeaveDays(e) + " days, US " +
us.earnedLeaveDays(e) + " days");
    }
}
```

Output:

```
LeavePolicyTest
  PASS  carry forward is capped
  PASS  india policy matches old behaviour
  PASS  probationers earn nothing
  PASS  us policy matches old behaviour
4 passed, 0 failed
Asha: India 45 days, US 30 days
Ravi: India 12 days, US 10 days
```

The tests (rule 1) prove the refactoring kept behaviour identical. Duplicate algorithms became one Template Method (rule 2). Magic numbers became `daysPerYear()`/`maxCarryForwardDays()` (rule 3). The purposeless `LeaveCalcStrategyProvider` interface was dropped (rule 4).

Trade-offs

Rules 2–4 can pull against each other (removing duplication can add a class; expressiveness can add a method). Resolve by priority, and by looking for a better design that satisfies both.

When not to use it

Always applicable; it's a lightweight mindset, not a process.

Common mistakes

- Refactoring without tests (breaking rule 1 to satisfy the others).
- Dogmatically minimising classes at the expense of SRP.

Real-world examples

Extreme Programming teams apply these rules continuously. The Template Method refactoring in our example mirrors how `java.util.AbstractList` removes duplication across list implementations.

In the interview

30-second answer: "Kent Beck's rules of simple design, in priority order: passes the tests, reveals intention, no duplication, fewest elements. (*Clean Code* lists no-duplication before intention; the middle two reinforce each other.) I get the core working with a quick test or demo, then refactor out duplication and improve names, and I don't add classes that don't earn their place."

Self-check

1. Q: Which rule comes first and why? → A: Runs all the tests — without them you can't refactor safely, and testability itself improves design.
2. Q: Which rule has the lowest priority? → A: Minimal classes and methods.

TL;DR

Tests pass → no duplication → clear intent → nothing extra.

---

12. Concurrency

What it is

Concurrency — having several tasks in progress at once, usually on separate *threads* (independent paths of execution inside one program) — is a **decoupling strategy**: it separates *what* gets done from *when* it gets done. It can improve throughput (work completed per second; see System Design Fundamentals) and structure, but concurrent code is hard to get right. Clean concurrency means keeping concurrency code small, separate and built on well-tested library tools.

Analogy: a restaurant kitchen. Several cooks work at once. It works because each cook has their own station and ingredients; trouble starts when two cooks grab the same pan.

The problem it solves

Myths lead to bugs: *concurrency always improves performance* (only when there's waiting or multiple cores — CPU units that can each run a thread at the same moment — to exploit), *design doesn't change* (it changes a lot), *understanding concurrency doesn't matter with a container like a web server* (you still share state between requests). In reality, concurrency adds overhead, bugs are often not repeatable, and it requires a different design strategy.

How it works — defence principles

1. **Single Responsibility**: keep concurrency-related code separate from other code. Our `countWords` is pure single-threaded logic; the executor code (an executor runs tasks on a pool of reusable threads) is separate.
2. **Limit the scope of shared data**: encapsulate it and minimise the places that touch it; guard critical sections (code that touches shared data and must be run by one thread at a time, usually enforced with a *lock*).
3. **Use copies of data**: share immutable objects or give each thread its own copy and merge results at the end.
4. **Threads should be as independent as possible**: each thread processes its own data with no sharing (like each request having its own local variables).
5. **Know your library**: use thread-safe collections (`ConcurrentHashMap`), the executor framework (`ExecutorService`), non-blocking atomics (`AtomicInteger`, `LongAdder` — classes that update a value in one indivisible step without locks), and synchronisers (`CountDownLatch`, `Semaphore`, `CyclicBarrier`) instead of hand-made `wait/notify` code.
6. **Know your execution models** (Chapter 13 covers them in depth): **producer–consumer** (bounded queue between producers and consumers), **readers–writers** (many readers, exclusive writers; balance throughput vs starvation), **dining philosophers** (competing for multiple resources → deadlock, livelock, starvation). *Deadlock*: threads wait for each other forever; *starvation*: a thread never gets its turn.
7. **Beware dependencies between synchronised methods** (`synchronized` methods, which only one thread at a time can run on a given object) on the same shared object: if you must call several, use client-based locking, server-based locking (one method that does it all under its lock) or an adapted server.
8. **Keep synchronised sections small** — locks create contention (threads queuing for the same lock); lock as little code as possible, but correctness first.

9. **Writing correct shutdown code is hard** — plan graceful shutdown (letting running tasks finish before the program stops) from the start (`shutdown()` + `awaitTermination()`, then `shutdownNow()`), beware deadlocks where threads wait for signals that never come.
10. **Testing threaded code:**
    - Treat spurious, one-off failures as potential threading issues — don't dismiss them as "cosmic rays".
    - Get non-threaded code working first.
    - Make threaded code **pluggable** (run with 1, 2, many threads) and **tunable** (thread counts configurable).
    - Run with **more threads than processors** to force more task switching.
    - Run on different platforms.
    - **Instrument** code to try to force failures (hand-coded or automated "jiggling" — inserting yields/sleeps to change interleavings — the order in which different threads' steps happen to mix; tools like jcstress exist for this).

Key vocabulary: **bound resources** (fixed-size resources like connection pools — a reusable set of open database connections), **mutual exclusion** (only one thread at a time may use a resource), **starvation**, **deadlock**, **livelock** (threads keep reacting to each other and make no progress). These are covered in Chapters 12–13. (*Clean Code*'s **Appendix A, "Concurrency II"**, goes deeper into the same ideas: client- vs server-based locking, deadlock's four conditions and how to break them, and testing multithreaded code.)

Java example — before/after

```java
// File: lld/ch06/concurrency/Main.java
package lld.ch06.concurrency;

import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.concurrent.Callable;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;
import java.util.concurrent.TimeUnit;

/** Clean concurrency: separate concurrency from logic, limit shared data, prefer copies and independence, use the library. */
public class Main {

    static final List<String> PAGES = List.of(
            "the cat sat on the mat", "the dog sat", "a cat and a dog", "the end");

    /** Pure, single-threaded logic -- testable without any threads (SRP: concurrency kept separate). */
    static Map<String, Integer> countWords(String page) {
        Map<String, Integer> counts = new HashMap<>();
        for (String w : page.split(" ")) counts.merge(w, 1, Integer::sum);
        return counts;
    }

    // ---------------- BEFORE: threads share an unsynchronised counter ----------------
    static class Before {
        static int theCount = 0;                                        // shared mutable state, no synchronisation
        static int run(int repeats) throws InterruptedException {
            Thread[] threads = new Thread[4];
            for (int t = 0; t < threads.length; t++) {
                threads[t] = new Thread(() -> {
                    for (int r = 0; r < repeats; r++)
                        for (String page : PAGES)
                            for (String w : page.split(" "))
                                if (w.equals("the")) theCount++;      // read-modify-write race: updates get lost
                });
                threads[t].start();
            }
            for (Thread t : threads) t.join();
            return theCount;
        }
        // (Sharing a plain HashMap this way is even worse: concurrent resizes can corrupt it.)
    }

    // ---------------- AFTER ----------------
    static class After {
        /** Each task works on its OWN data and returns a result; results are combined once, at the end. */
        static Map<String, Integer> run(int repeats, int threads) throws Exception {
            ExecutorService pool = Executors.newFixedThreadPool(threads);   // know your library
            try {
                List<Callable<Map<String, Integer>>> tasks = new java.util.ArrayList<>();
                for (int t = 0; t < threads; t++) {
                    tasks.add(() -> {
                        Map<String, Integer> local = new HashMap<>();       // thread-confined: no sharing, no locks
                        for (int r = 0; r < repeats; r++)
                            for (String page : PAGES) countWords(page).forEach((w, c) -> local.merge(w, c, Integer::sum));
                        return local;
                    });
                }
                Map<String, Integer> total = new HashMap<>();               // combined by ONE thread: no sharing
                for (Future<Map<String, Integer>> f : pool.invokeAll(tasks))
                    f.get().forEach((w, c) -> total.merge(w, c, Integer::sum));   // Future.get() safely hands over each result
                return Map.copyOf(total);                                   // publish an immutable copy
            } finally {
```

```
                pool.shutdown();                                                    // clean shutdown is part of
the design
                if (!pool.awaitTermination(5, TimeUnit.SECONDS)) pool.shutdownNow();
            }
        }
    }

    public static void main(String[] args) throws Exception {
        int repeats = 20_000;
        int expectedThe = 4 * repeats * 4;   // "the" appears 4 times per pass over PAGES, 4 threads
        System.out.println("pure logic: " + new java.util.TreeMap<>(countWords("the cat sat on the mat")));

        System.out.println("== BEFORE (racy; the number varies from run to run) ==");
        int racy = Before.run(repeats);
        System.out.println("count(the) = " + racy + " expected " + expectedThe + " -> lost " + (expectedThe -
racy) + " updates");

        System.out.println("== AFTER (deterministic) ==");
        for (int threads : new int[]{1, 4, 16}) {                    // more threads than cores is a good stress
test
            Map<String, Integer> safe = After.run(repeats, threads);
            System.out.println(threads + " thread(s): count(the) = " + safe.get("the") + " expected " + (4 *
repeats * threads));
        }
    }
}
```

Output:

```
pure logic: {cat=1, mat=1, on=1, sat=1, the=2}
== BEFORE (racy; the number varies from run to run) ==
count(the) = 182526 expected 320000 -> lost 137474 updates
== AFTER (deterministic) ==
1 thread(s): count(the) = 80000 expected 80000
4 thread(s): count(the) = 320000 expected 320000
16 thread(s): count(the) = 1280000 expected 1280000
```

The *before* shares one `int` among four threads; `theCount++` is a read-modify-write, so concurrent increments overwrite each other and updates are **lost** (the exact number varies from run to run — and on rare runs it may even be correct, which is exactly why such bugs escape testing). The *after* follows the principles: pure logic separate from threading; each task owns its data; results are handed back through `Future.get()` (a `Future` is a handle to a result that a task will produce later) (which guarantees the result is safely visible to the caller) and merged by one thread; the final map is an immutable copy; the pool is shut down properly; and it is tested with 1, 4 and 16 threads (more threads than cores).

Trade-offs

Confinement + merge (after)	Shared state + locks
No locks, no races, scales well	Simple mental model for small cases
Extra memory for per-thread copies	Contention, deadlock risk
Needs a merge step	Every access must be correctly synchronised

When not to use it

Don't add concurrency without a measured need; a single thread is simpler and often fast enough. Prefer higher-level tools (parallel streams for pure CPU work, virtual threads in Java 21 — lightweight threads managed by the JVM, cheap enough to create one per task — for many blocking I/O tasks, i.e. input/output where the thread just waits, such as a network call) over manual thread management.

Common mistakes

- Sharing a `HashMap` / `ArrayList` between threads without synchronisation.

- Check-then-act races (`if (!map.containsKey(k)) map.put(k, v)`) — use `putIfAbsent`/`computeIfAbsent`.
- Forgetting to shut down executors (the JVM may not exit).
- Holding locks while calling unknown code (callbacks — functions passed in to be called later) → deadlock risk.

Real-world examples

- **`java.util.concurrent`** (Doug Lea): `ExecutorService`, `ConcurrentHashMap`, `CompletableFuture`, `ForkJoinPool` (work-stealing — idle threads take queued tasks from busy ones; used by parallel streams).
- **Map-reduce / Hadoop / Spark** (frameworks that split big-data jobs across many machines): independent workers on partitions + a merge step — the same confinement idea at cluster scale.
- **Node.js and Redis** (a JavaScript server runtime and an in-memory data store) use single-threaded event loops (one thread that handles ready events one after another) for their main processing, avoiding shared-memory races by design.

In the interview

> **30-second answer:** "I keep concurrency separate from business logic, minimise shared mutable state — preferring thread confinement (each piece of data is used by only one thread), immutable data and merging results — and use `java.util.concurrent` rather than raw threads. I plan shutdown and test with more threads than cores."

Follow-ups:

- *Q: Why is `count++` not thread-safe?* A: It's read, add, write — two threads can read the same value and one update is lost. Use `AtomicInteger`/`LongAdder` or a lock.
- *Q: Name three concurrency execution models.* A: Producer–consumer, readers–writers, dining philosophers.

Self-check

1. Q: What does concurrency decouple? → A: What gets done from when it gets done.
2. Q: How do you avoid sharing data between threads? → A: Give each task its own data (confinement) and merge results, or share only immutable data.
3. Q: How should you treat a one-off test failure in concurrent code? → A: As a likely real threading bug.

TL;DR

Separate concurrency code, share as little as possible (copies, immutability, confinement), use the library, keep locks small, plan shutdown, test hard.

---

13. Successive Refinement and the Case Studies (Chapters 14–16: how clean code is actually written)

Chapters 14–16 of *Clean Code* are long case studies rather than new rules: the author takes real code and cleans it step by step. They matter because they show the *process*.

Chapter 14 — Successive Refinement refines a command-line argument parser (`Args`) — code that reads the options typed after a program's name, like `-l -p 8080`. The lessons, in brief:

1. **Write it dirty first, then clean it.** Nobody writes clean code in one pass; a working first draft is normal. What's not acceptable is *stopping* there.
2. **Stop when it starts to rot.** When adding each new feature (a new argument type) makes the code noticeably messier, stop adding features and restructure.
3. **Refactor in tiny steps with tests** — after every small change, run the tests; never leave the system broken for long. Large "big-bang" restructurings tend to fail.

4. **Use design to absorb variation**: in the case study, each argument type became its own class behind a common interface (a polymorphic "marshaler"), so new types didn't touch the parser — the same Open/Closed move as our report formatters.
5. **Clean the error handling too** — collect error codes and messages into one exception class instead of scattering them.

Chapter 15 — JUnit Internals critiques a small class from JUnit (`ComparisonCompactor`, by Kent Beck and Erich Gamma), which was already good and well tested, and still improves it. Lessons (codes such as N6 and G28 refer to the smells-and-heuristics catalogue in §14):

- Remove encoding prefixes on fields (`fExpected` → `expected`) — N6.
- Encapsulate conditionals and prefer positive ones (`if (shouldNotCompact())` → `if (canBeCompacted())`) — G28, G29.
- Rename a function whose name hides a side effect or a second job (a `compact` method that also formats → `formatCompactedComparison`) — N7, G30.
- Make hidden temporal coupling explicit (finding the common suffix depends on first finding the common prefix, so make the order visible) — G31.
- Tidy `+1`/`-1` boundary arithmetic into well-named variables — G33.
- The point: even good code can be left a little cleaner — the Boy Scout Rule in action.

Chapter 16 — Refactoring SerialDate cleans up a date class from the open-source (published for anyone to use and modify) JCommon library. The process is the lesson:

1. **First, make it work** — write more tests: coverage (the share of code lines the tests actually execute) rose from about 50% to about 92%, and the new tests exposed real bugs, which were fixed.
2. **Then make it right** — rename (`SerialDate` → `DayDate`), replace integer constants with enums (a `Month` enum; J3), move methods to the classes that own the data (G14/G17), introduce an **Abstract Factory** so the base class no longer knows its concrete subclass (G7), and delete dead code, duplication and noise comments (G9, G5, C3).
3. Every step keeps the tests green.

TL;DR: make it work, then make it right, in small tested steps — and don't let a mess grow.

---

14. Smells and Heuristics (the checklist)

What it is

Clean Code ends with a catalogue of **smells and heuristics** — short rules, each with a code (C1, G5, N1, …), grouped by area. It extends the smells from Fowler's *Refactoring* (Chapter 7). Use it as a review checklist: each item is a *hint* that something could be cleaner, not an absolute law.

The problem it solves

Knowing principles is not enough; you need to *recognise* problems quickly while reading code. A named catalogue gives you and your reviewers a shared vocabulary ("this is G28 — encapsulate that conditional").

How it works — the catalogue, grouped (names listed; meaning in our words)

Comments (C)

Code	Heuristic	Meaning
C1	Inappropriate information	Change history, authors, ticket metadata belong in version control / issue tracker
C2	Obsolete comment	Old, irrelevant or wrong comments — update or delete
C3	Redundant comment	Says what the code already says clearly
C4	Poorly written comment	If worth writing, write it well: brief, correct grammar, precise
C5	Commented-out code	Delete it; version control remembers

Environment (E)

Code	Heuristic	Meaning
E1	Build requires more than one step	Check out, run one command, get a build (compiled, packaged program)
E2	Tests require more than one step	One command (or one click) runs all tests

Functions (F)

Code	Heuristic	Meaning
F1	Too many arguments	Zero best, then one, two, three; more needs strong justification
F2	Output arguments	Surprising; change the owning object's state instead
F3	Flag arguments	A boolean argument means the function does two things
F4	Dead function	Never called? Delete it

General (G) — the largest group, clustered by theme:

Correctness and safety

Code	Heuristic	Meaning
G2	Obvious behaviour is unimplemented	Honour the principle of least surprise; implement what users would expect from the name
G3	Incorrect behaviour at the boundaries	Test edge and corner cases (inputs at the limits: empty, zero, maximum); don't rely on intuition
G4	Overridden safeties	Don't turn off compiler warnings, skip failing tests, or ignore errors
G21	Understand the algorithm	Don't fiddle with ifs until tests pass; know *why* it works
G26	Be precise	e.g. don't use floating point (`double`, inexact for decimals) for currency; handle concurrency, nulls and ties deliberately
G31	Hidden temporal couplings	If calls must happen in order, make the order explicit (each step returns what the next needs)
G33	Encapsulate boundary conditions	Put `+1`/`-1` edge logic in one named place (`int lastIndex = size - 1;`)

Duplication and dead weight

Code	Heuristic	Meaning
G5	Duplication	"One of the most important rules" in the book: every duplication is a missed opportunity for abstraction (methods, polymorphism, Template Method, Strategy)
G9	Dead code	Unreachable/unused code — delete
G12	Clutter	Unused variables, empty default constructors, meaningless comments

Structure and placement

Code	Heuristic	Meaning
G1	Multiple languages in one source file	Minimise mixing (e.g. Java + SQL + HTML + JS in one file)
G6	Code at wrong level of abstraction	Keep high-level concepts in base classes/interfaces and details in implementations
G7	Base classes depending on their derivatives	A base class shouldn't know its subclasses
G8	Too much information	Small, focused interfaces; hide data, utilities, constants
G10	Vertical separation	Define variables and private functions close to where they're used
G13	Artificial coupling	Don't put things together that don't depend on each other (e.g. a general enum inside a specific class)
G14	Feature envy	A method more interested in another class's data than its own should move there
G17	Misplaced responsibility	Put code where a reader would expect to find it
G18	Inappropriate static	Make a method static only if it will never need polymorphic behaviour
G22	Make logical dependencies physical	If a module depends on another's assumption, ask it explicitly instead of assuming
G35	Keep configurable data at high levels	Defaults/constants that are configuration belong near the top (e.g. in `main`/config), passed down
G36	Avoid transitive navigation	Law of Demeter — don't write `a.getB().getC()`

Readability and intent

Code	Heuristic	Meaning
G11	Inconsistency	Do similar things the same way throughout
G15	Selector arguments	Arguments (booleans, enums, ints) that select behaviour → split into functions
G16	Obscured intent	Dense, clever expressions and magic hex numbers hide meaning
G19	Use explanatory variables	Break calculations into named intermediate values
G20	Function names should say what they do	`invoice.adjust(5)` — five what? up or down? Name it `applyDiscountPercent(5)`
G23	Prefer polymorphism to if/else or switch/case	"One switch" rule: at most one switch per type selection, used to create polymorphic objects
G24	Follow standard conventions	Team/industry coding standard, applied consistently
G25	Replace magic numbers with named constants	Also applies to magic strings and tokens
G27	Structure over convention	Enforce design decisions with structure (e.g. abstract methods) rather than naming conventions
G28	Encapsulate conditionals	`if (isRefundable(order))` beats `if (order.isDelivered() && order.daysSinceDelivery() <= 7)`
G29	Avoid negative conditionals	`if (user.isActive())` beats `if (!user.isNotActive())`
G30	Functions should do one thing	Split multi-step functions
G32	Don't be arbitrary	Structure code for a reason and let the reason show
G34	Functions should descend only one level of abstraction	Statements in a function sit one level below the function's name

Java (J)

Code	Heuristic	Meaning
J1	Avoid long import lists by using wildcards (`import java.util.*;`)	The book's advice; note many modern style guides (e.g. Google Java Style) forbid wildcard imports — follow your team's standard
J2	Don't inherit constants	Don't implement an interface just to get its constants; use static imports
J3	Constants versus enums	Prefer `enum` (type-safe, can carry data and behaviour) over `public static final int` codes

Names (N)

Code	Heuristic	Meaning
N1	Choose descriptive names	Names make up most of readability
N2	Choose names at the appropriate level of abstraction	Name for the concept, not the implementation (`notify(recipient)`, not `sendSms(phone)` if it may later be email or push)
N3	Use standard nomenclature where possible	Pattern names, domain terms (`...Decorator`, `...Factory`)
N4	Unambiguous names	Name should make the function's work clear
N5	Use long names for long scopes	Short names only in tiny scopes
N6	Avoid encodings	No type/scope prefixes
N7	Names should describe side effects	`getOrCreateUser`, not `getUser`, if it creates

Tests (T)

Code	Heuristic	Meaning
T1	Insufficient tests	Test everything that could break
T2	Use a coverage tool	Find untested lines and branches (e.g. JaCoCo)
T3	Don't skip trivial tests	They're cheap and document behaviour
T4	An ignored test is a question about an ambiguity	`@Disabled` tests mark unclear requirements — resolve them
T5	Test boundary conditions	Off-by-one errors (one too many or too few) live at the edges
T6	Exhaustively test near bugs	Bugs cluster; where you find one, look for more
T7	Patterns of failure are revealing	Which tests fail together often points to the cause
T8	Test coverage patterns can be revealing	Code not executed by passing tests hints at why failures happen
T9	Tests should be fast	Slow tests don't get run

Java example — several heuristics on one function

```java
// File: lld/ch06/smells/Main.java
package lld.ch06.smells;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.List;

/**
 * A handful of Clean Code heuristics applied to one function:
 * G25 magic numbers, G19 explanatory variables, G16 obscured intent, G28 encapsulate conditionals,
 * G29 avoid negative conditionals, F3 flag arguments, G26 be precise (no floating point for money),
 * N1/G20 descriptive names that say what the function does.
 */
public class Main {

    enum Membership { NONE, PRIME }
    record Parcel(double weightKg, int distanceKm, Membership membership, boolean fragile) {}

    // ---------------- BEFORE ----------------
    static class Before {
        static double calc(Parcel p, boolean express) {                                 // double for money
(G26)
            double c;
            if (!(p.membership() != Membership.PRIME)) c = 0; else c = 40;           // double negative
            if (p.weightKg() > 20 || p.distanceKm() > 1500 && !p.fragile() == false) c += 250;   // what does this
mean?
            c += p.weightKg() * 12.5;
            if (express) c = c * 1.5;                                                   // flag argument
            if (express && p.membership() == Membership.PRIME) c = c * 0.9;
            return Math.round(c * 100) / 100.0;
        }
    }

    // ---------------- AFTER ----------------
    static class After {
        enum Speed {                                            // replaces the flag; each speed knows its
multiplier
            STANDARD("1.0"), EXPRESS("1.5");
            final BigDecimal multiplier;
            Speed(String m) { multiplier = new BigDecimal(m); }
        }

        private static final BigDecimal BASE_FEE = new BigDecimal("40");          // rupees, exact decimal (G26)
        private static final BigDecimal RATE_PER_KG = new BigDecimal("12.5");
        private static final double HEAVY_KG = 20;
        private static final int LONG_HAUL_KM = 1500;
        private static final BigDecimal SPECIAL_HANDLING_FEE = new BigDecimal("250");
        private static final BigDecimal PRIME_EXPRESS_DISCOUNT = new BigDecimal("0.9");

        static BigDecimal shippingCost(Parcel p, Speed speed) {
            BigDecimal baseFee = isPrime(p) ? BigDecimal.ZERO : BASE_FEE;
            BigDecimal handlingFee = needsSpecialHandling(p) ? SPECIAL_HANDLING_FEE : BigDecimal.ZERO;
            BigDecimal weightCharge = BigDecimal.valueOf(p.weightKg()).multiply(RATE_PER_KG);   // explanatory
variables
            BigDecimal cost = baseFee.add(handlingFee).add(weightCharge).multiply(speed.multiplier);
            if (isPrime(p) && speed == Speed.EXPRESS) cost = cost.multiply(PRIME_EXPRESS_DISCOUNT);
            return cost.setScale(2, RoundingMode.HALF_UP);                          // round once, explicitly, to
paise
        }

        private static boolean isPrime(Parcel p) { return p.membership() == Membership.PRIME; }   // positive
conditional
        private static boolean needsSpecialHandling(Parcel p) {                                 // encapsulated
conditional
            boolean heavy = p.weightKg() > HEAVY_KG;
            boolean fragileLongHaul = p.distanceKm() > LONG_HAUL_KM && p.fragile();
            return heavy || fragileLongHaul;
        }
    }
```

```
    public static void main(String[] args) {
        List<Parcel> parcels = List.of(
                new Parcel(2, 300, Membership.NONE, false),
                new Parcel(25, 100, Membership.PRIME, false),
                new Parcel(3, 2000, Membership.NONE, true),
                new Parcel(3, 2000, Membership.PRIME, false));
        for (Parcel p : parcels) {
            for (boolean express : new boolean[]{false, true}) {
                double before = Before.calc(p, express);
                BigDecimal after = After.shippingCost(p, express ? After.Speed.EXPRESS : After.Speed.STANDARD);
                boolean same = BigDecimal.valueOf(before).compareTo(after) == 0;
                System.out.printf("%-55s express=%-5s before=%7.2f after=%7s %s%n",
                        p, express, before, after, same ? "same" : "DIFFERENT");
            }
        }
    }
}
```

Output:

```
Parcel[weightKg=2.0, distanceKm=300, membership=NONE, fragile=false] express=false before=  65.00 after=  65.00 same
Parcel[weightKg=2.0, distanceKm=300, membership=NONE, fragile=false] express=true  before=  97.50 after=  97.50 same
Parcel[weightKg=25.0, distanceKm=100, membership=PRIME, fragile=false] express=false before= 562.50 after= 562.50 same
Parcel[weightKg=25.0, distanceKm=100, membership=PRIME, fragile=false] express=true  before= 759.38 after= 759.38 same
Parcel[weightKg=3.0, distanceKm=2000, membership=NONE, fragile=true] express=false before= 327.50 after= 327.50 same
Parcel[weightKg=3.0, distanceKm=2000, membership=NONE, fragile=true] express=true  before= 491.25 after= 491.25 same
Parcel[weightKg=3.0, distanceKm=2000, membership=PRIME, fragile=false] express=false before=  37.50 after=  37.50 same
Parcel[weightKg=3.0, distanceKm=2000, membership=PRIME, fragile=false] express=true  before=  50.63 after=  50.63 same
```

The "same" column proves behaviour didn't change. Fixed smells: double negative (G29), unreadable mixed `&&` / `||` / `== false` condition (G16, G28 → `needsSpecialHandling`), magic numbers (G25), flag argument (F3 → `Speed` enum), unnamed intermediate values (G19), a cryptic name `calc` (N1/G20 → `shippingCost`), and floating-point money (G26 → `BigDecimal` with one explicit `HALF_UP` rounding to paise). The *before* deliberately keeps `double` so you can see the smell; never do that in real money code.

Trade-offs

A checklist can be applied dogmatically. Each item is a heuristic: when two conflict, favour readability and correctness.

When not to use it

Don't block a critical fix on style nits; log them as follow-ups.

Common mistakes

- Treating smell counts as a metric to game.
- Ignoring the **Java** notes that are dated (J1): follow your team's style guide.

Real-world examples

Static analysers (tools that inspect code without running it) encode many of these: **SonarQube** (cognitive complexity — a score for how hard code is to follow, duplicated blocks, commented-out code), **PMD** (unused code, god class), **Checkstyle** (naming, magic numbers), **SpotBugs** (bug patterns), **Error Prone** (Google).

In the interview

> **30-second answer:** "When reviewing I look for duplication, long functions, flag and output arguments, magic numbers, negative or complex conditionals, feature envy, train wrecks, misplaced responsibilities, dead code, and weak tests. Most fixes are small refactorings: extract method, introduce constant, replace conditional with polymorphism."

Self-check

1. Q: Which heuristic does the book call one of its most important rules? → A: G5, Duplication.
2. Q: What does G28 ask you to do? → A: Encapsulate conditionals in a well-named function.
3. Q: What does T4 say about a disabled test? → A: It signals an unresolved ambiguity in requirements.

TL;DR

Use the C/E/F/G/J/N/T catalogue as a shared review vocabulary; fix with small, tested refactorings.

---

15. Code Review Checklist (best practices)

What it is

A **code review** is a peer reading a change before it's merged (combined into the team's main branch), to catch defects, share knowledge and keep the codebase consistent. A checklist makes reviews thorough and fair — and in interviews, it's what you mentally run over your own code before saying "done".

Analogy: a pilot's pre-flight checklist — experienced pilots still use it because it catches what memory misses.

The problem it solves

Unreviewed changes introduce bugs, security holes and inconsistent styles; unfocused reviews waste time on nits while missing real issues.

How it works — the checklist (from most to least important)

1. **Correctness**
   - Does it do what the requirement/ticket says? Are edge cases handled (empty, null, zero, max, duplicates, time zones, rounding)?
   - Are errors handled — no swallowed exceptions, meaningful messages, resources closed?
   - Concurrency: shared mutable state guarded? check-then-act races? thread-safe collections?
2. **Design**
   - Does it fit the existing architecture? Right class/layer for each responsibility (SRP, GRASP)?
   - Dependencies on abstractions (DIP)? Easy to extend without modification (OCP)?
   - Any over-engineering (YAGNI) or duplication (DRY)?
3. **Readability**
   - Intention-revealing names; small functions; no magic numbers; no flag arguments.
   - Comments explain *why*, not *what*; no commented-out code.
4. **Tests**
   - New behaviour covered by tests? Bug fixes include a regression test (one that makes sure the fixed bug can't return)?
   - Tests are F.I.R.S.T., one concept each, readable names?
5. **Security**
   - Input validation; no SQL/command injection (an attack where user input gets run as part of a database query or shell command; prevented by parameterised queries, which send input separately

as plain data); output encoding (escaping data before putting it into HTML so it can't run as script).
    - No secrets or credentials in code or logs; sensitive data (PII — personally identifiable information such as names, phone numbers, emails) masked in logs.
    - Authorisation checks (is *this* user allowed to do *this* action?) on every sensitive operation.
6. **Performance and resources**
    - Obvious inefficiencies: N+1 queries (one query to load a list, then one more query per item), repeated work in loops, unbounded collections/caches, missing pagination (returning results page by page instead of all at once).
    - Timeouts (a maximum wait, after which the call gives up) on remote calls; connection/stream closing.
7. **Operability**
    - Useful logs at the right level (and not too many); metrics (counters and timings collected for dashboards and alerts) for important events; feature flags (switches that turn a feature on or off without redeploying) for risky changes.
    - Backwards compatibility (old clients keep working after the change) of APIs, messages and database migrations (scripted changes to the database's table structure).
8. **Style and consistency** — ideally automated by formatter/linter so humans focus on the above.

How to review well (and be reviewed well):

- Keep changes **small** (a few hundred lines at most); large diffs get rubber-stamped (approved without real reading).
- Author: write a clear description (what, why, how tested), self-review first.
- Reviewer: be specific and kind; explain *why*; separate must-fix from optional suggestions ("nit:"); ask questions rather than accuse.
- Automate the mechanical (formatting, static analysis, tests in CI) so humans review design and logic.

Java example

No separate program: every before/after in this chapter is a review finding. For example, reviewing `Before.book` in §2 you would flag: flag argument (F3), error codes, output argument (F2), hidden side effect (SESSION), switch on type code (G23), magic numbers (G25) — exactly the fixes shown in its *after* version.

Trade-offs

Reviews slow merging slightly but catch defects early, spread knowledge and keep style consistent. Too many reviewers or nit-heavy reviews slow teams without adding quality.

When not to use it

Emergency hotfixes (urgent fixes deployed straight to production) may be reviewed after merge (with a follow-up), but should still be reviewed.

Common mistakes

- Reviewing only style while missing logic and security bugs.
- Huge pull requests (a pull request is a proposed change submitted for review and merging).
- Personal preferences presented as rules.

Real-world examples

- **GitHub pull requests / GitLab merge requests / Gerrit** (used by Android and Chromium) implement review workflows.
- Google's engineering practices publish guidance for reviewers and authors (focus on design, functionality, complexity, tests, naming, comments, style).
- **CODEOWNERS** files route reviews to responsible teams.

In the interview

> **30-second answer:** "I review for correctness first — requirements, edge cases, errors, concurrency — then design, readability, tests, security and performance, and leave formatting to tools. I keep feedback specific, explain why, and mark nits as optional."

Self-check

1. Q: What should a reviewer check first? → A: Correctness (including edge cases and error handling).
2. Q: Why keep pull requests small? → A: Large diffs get shallow reviews.

TL;DR

Correctness → design → readability → tests → security → performance → operability; automate style; keep PRs small and feedback kind.

---

16. Error-Handling Strategy (best practices for real services)

What it is

Section 6 covered error-handling *rules* inside code. An **error-handling strategy** is the *system-wide policy*: which exception types exist, where errors are translated, which are retried, how they're logged, and what callers (users/other services) see.

Analogy: a hospital triage system — every problem is classified (critical, urgent, minor), routed to the right place, and recorded once in the patient chart.

The problem it solves

Without a strategy, each developer invents their own: some return `null`, some throw checked exceptions, some log and rethrow at every layer (same error logged five times), some expose stack traces to users, and some retry everything — including permanent failures, which overloads already-struggling services.

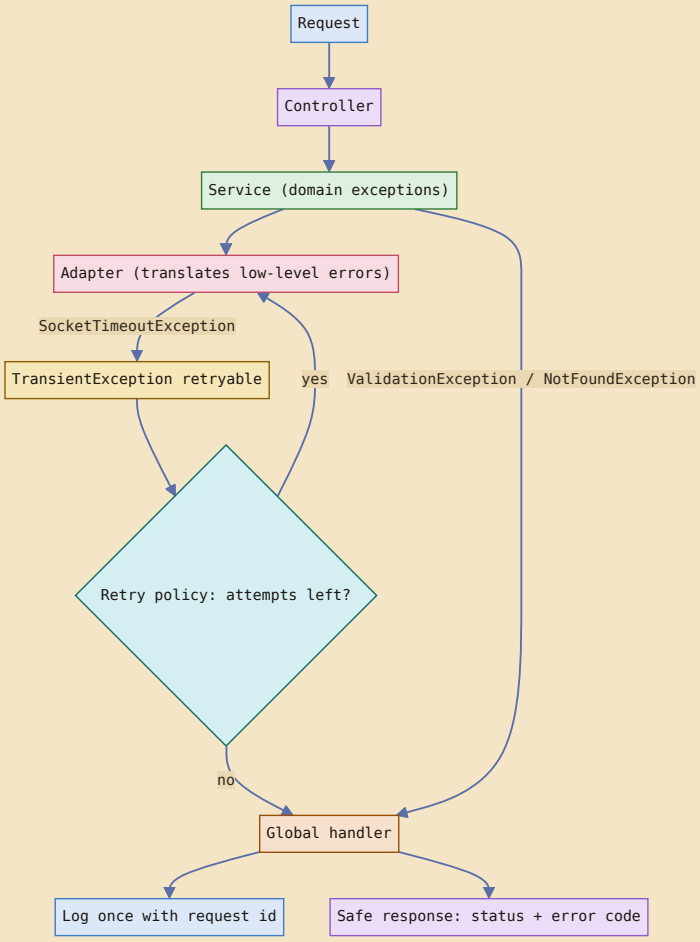
How it works

> **Key term — HTTP status codes:** every reply from a web API carries a three-digit status code. **2xx** means success (200 OK); **4xx** means the *client* sent something wrong (400 bad request, 404 not found, 409 conflict with the current state); **5xx** means the *server* failed (500 internal error, 503 service unavailable — often temporary). Retrying usually only makes sense for some 5xx errors and timeouts. See System Design Fundamentals.

1. **Classify errors:**

Class	Examples	Handling
Validation / client errors	bad input, missing field	Reject immediately, 4xx, clear message, no retry
Business rule violations	insufficient funds, seat already booked	Domain exception or result value, 4xx (e.g. 409 conflict)
Not found	unknown id	404 / empty `Optional`
Transient infrastructure	timeouts, 503, connection reset, deadlock victim	Retry with backoff — waiting longer between attempts (only if the operation is **idempotent** — safe to repeat)
Permanent infrastructure	auth failure to DB, misconfiguration	Fail fast, alert
Programming bugs	NPE, index out of bounds	Don't catch locally; generic 500 at the edge, log with details, fix the bug

2. **One exception hierarchy** with an error code and a *retryable* flag, so handlers don't need `instanceof` chains on vendor exceptions.
3. **Translate at boundaries**: adapters convert low-level exceptions (`SocketTimeoutException`, `SQLException`) into domain/infrastructure exceptions, keeping the cause.
4. **Retry only transient errors**, with a maximum number of attempts, **exponential backoff** (each wait doubles: 10 ms, 20 ms, 40 ms…) and **jitter** (randomness to avoid many clients retrying in sync). Combine with **timeouts** and **circuit breakers** (a guard that stops calling a failing dependency for a while after repeated errors) for dependencies.
5. **Use result values for expected, per-item failures** (e.g. validating 10,000 imported rows — collect all errors instead of stopping at the first).
6. **Handle globally at the edge**: one handler maps exceptions to responses (status code, stable error code, safe message) and **logs once** with a correlation/request id (a unique ID attached to one request and stamped on all its log lines, so they can be found together). Never leak stack traces or internals to clients.
7. **Keep the system consistent**: use transactions or compensating actions (follow-up steps that undo an earlier one, e.g. a refund) so partial failures don't leave corrupt state; make handlers idempotent.


Request
Controller
Service (domain exceptions)
Adapter (translates low-level errors)
SocketTimeoutException
TransientException retryable
yes
ValidationException / NotFoundException
Retry policy: attempts left?
no
Global handler
Log once with request id
Safe response: status + error code

Java example

```java
// File: lld/ch06/errorstrategy/Main.java
package lld.ch06.errorstrategy;

import java.net.SocketTimeoutException;
import java.util.ArrayList;
import java.util.List;
import java.util.function.Function;
import java.util.function.Supplier;

/**
 * An end-to-end error-handling STRATEGY for a service:
 *  1. one exception hierarchy with error codes (+ "retryable" flag)
 *  2. translate low-level errors at the infrastructure boundary
 *  3. retry only transient errors, with backoff and a limit
 *  4. use a Result type for EXPECTED failures (e.g. validating many rows)
 *  5. one global handler maps exceptions to responses and logs ONCE, hiding internals
 */
@SuppressWarnings("serial") // the nested exception classes are never serialized
public class Main {

    // ---------- 1. exception hierarchy ----------
    enum ErrorCode {
        VALIDATION_FAILED(400), NOT_FOUND(404), CONFLICT(409), DEPENDENCY_UNAVAILABLE(503);
        final int httpStatus;
        ErrorCode(int s) { httpStatus = s; }
    }
    static class AppException extends RuntimeException {
        final ErrorCode code;
        final boolean retryable;
        AppException(ErrorCode code, String message, boolean retryable, Throwable cause) {
            super(message, cause); this.code = code; this.retryable = retryable;
        }
    }
    static final class ValidationException extends AppException {
        ValidationException(String m) { super(ErrorCode.VALIDATION_FAILED, m, false, null); }
    }
    static final class NotFoundException extends AppException {
        NotFoundException(String m) { super(ErrorCode.NOT_FOUND, m, false, null); }
    }
    static final class TransientException extends AppException {
        TransientException(String m, Throwable cause) { super(ErrorCode.DEPENDENCY_UNAVAILABLE, m, true, cause); }
    }

    // ---------- 2. translate at the infrastructure boundary ----------
    static final class InventoryClient {
        private int callsUntilHealthy;
        InventoryClient(int failFirst) { this.callsUntilHealthy = failFirst; }
        int stockOf(String sku) {
            try {
                return rawHttpCall(sku);
            } catch (SocketTimeoutException e) {
                throw new TransientException("inventory service timed out", e);   // low-level -> domain meaning
            }
        }
        private int rawHttpCall(String sku) throws SocketTimeoutException {
            if (callsUntilHealthy-- > 0) throw new SocketTimeoutException("read timed out after 2000ms");
            if (sku.startsWith("X")) throw new NotFoundException("sku " + sku + " not found");
            return 7;
        }
    }

    // ---------- 3. retry transient errors only ----------
    static <T> T withRetry(Supplier<T> action, int maxAttempts, long initialBackoffMs) {
        long backoff = initialBackoffMs;
        for (int attempt = 1; ; attempt++) {
            try {
                return action.get();
            } catch (AppException e) {
                if (!e.retryable || attempt == maxAttempts) throw e;
```

```
                System.out.println("    attempt " + attempt + " failed (" + e.getMessage() + "), retrying in " +
backoff + "ms");
                sleep(backoff);
                backoff *= 2;                                   // exponential backoff (add jitter in production)
            }
        }
    }
    private static void sleep(long ms) {
        try { Thread.sleep(ms); } catch (InterruptedException e) { Thread.currentThread().interrupt(); throw new
IllegalStateException(e); }
    }

    // ---------- 4. Result type for expected, per-item failures ----------
    sealed interface Result<T> {
        record Ok<T>(T value) implements Result<T> {}
        record Err<T>(String error) implements Result<T> {}
    }
    record Row(int line, String sku, int qty) {}
    static Result<Row> parseRow(int line, String csv) {
        String[] f = csv.split(",");
        if (f.length != 2) return new Result.Err<>("line " + line + ": expected 2 fields");
        try {
            int qty = Integer.parseInt(f[1].trim());
            if (qty <= 0) return new Result.Err<>("line " + line + ": qty must be positive");
            return new Result.Ok<>(new Row(line, f[0].trim(), qty));
        } catch (NumberFormatException e) {
            return new Result.Err<>("line " + line + ": qty '" + f[1].trim() + "' is not a number");
        }
    }

    // ---------- 5. global handler at the edge ----------
    record Response(int status, String body) {}
    static <T> Response handle(String requestId, Supplier<T> endpoint, Function<T, String> render) {
        try {
            return new Response(200, render.apply(endpoint.get()));
        } catch (AppException e) {
            System.out.println("    [log once] " + requestId + " " + e.code + ": " + e.getMessage());
            return new Response(e.code.httpStatus, "{\"error\":\"" + e.code + "\",\"message\":\"" + e.getMessage()
+ "\"}");
        } catch (RuntimeException e) {
            System.out.println("    [log once] " + requestId + " UNEXPECTED " + e);   // full details only in logs
            return new Response(500, "{\"error\":\"INTERNAL\",\"requestId\":\"" + requestId + "\"}");
        }
    }

    public static void main(String[] args) {
        System.out.println("Retry a transient failure, then succeed:");
        var flaky = new InventoryClient(2);
        System.out.println("  " + handle("req-1", () -> withRetry(() -> flaky.stockOf("TEA"), 3, 10), n -> "
{\"stock\":" + n + "}"));

        System.out.println("Do NOT retry a permanent failure:");
        var healthy = new InventoryClient(0);
        System.out.println("  " + handle("req-2", () -> withRetry(() -> healthy.stockOf("X-404"), 3, 10), n -> "
{\"stock\":" + n + "}"));

        System.out.println("Give up after max attempts:");
        var down = new InventoryClient(99);
        System.out.println("  " + handle("req-3", () -> withRetry(() -> down.stockOf("TEA"), 3, 10), n -> "
{\"stock\":" + n + "}"));

        System.out.println("Unexpected bug -> generic 500, no internals leaked:");
        System.out.println("  " + handle("req-4", () -> List.<String>of().get(0), s -> s));

        System.out.println("Bulk import: collect ALL row errors instead of stopping at the first:");
        List<String> csv = List.of("TEA,2", "CUP", "PEN,abc", "INK,-1", "MUG,1");
        List<Row> ok = new ArrayList<>();
        List<String> errors = new ArrayList<>();
        for (int i = 0; i < csv.size(); i++) {
            switch (parseRow(i + 1, csv.get(i))) {
                case Result.Ok<Row> r -> ok.add(r.value());
```

```
                case Result.Err<Row> e -> errors.add(e.error());
            }
        }
        System.out.println("  imported " + ok);
        errors.forEach(e -> System.out.println("  rejected " + e));
        if (!errors.isEmpty())
            System.out.println("  " + handle("req-5", () -> { throw new ValidationException(errors.size() + "
invalid rows"); }, x -> "" + x));
    }
}
```

Output:

```
Retry a transient failure, then succeed:
    attempt 1 failed (inventory service timed out), retrying in 10ms
    attempt 2 failed (inventory service timed out), retrying in 20ms
  Response[status=200, body={"stock":7}]
Do NOT retry a permanent failure:
    [log once] req-2 NOT_FOUND: sku X-404 not found
  Response[status=404, body={"error":"NOT_FOUND","message":"sku X-404 not found"}]
Give up after max attempts:
    attempt 1 failed (inventory service timed out), retrying in 10ms
    attempt 2 failed (inventory service timed out), retrying in 20ms
    [log once] req-3 DEPENDENCY_UNAVAILABLE: inventory service timed out
  Response[status=503, body={"error":"DEPENDENCY_UNAVAILABLE","message":"inventory service timed out"}]
Unexpected bug -> generic 500, no internals leaked:
    [log once] req-4 UNEXPECTED java.lang.ArrayIndexOutOfBoundsException: Index 0 out of bounds for length 0
  Response[status=500, body={"error":"INTERNAL","requestId":"req-4"}]
Bulk import: collect ALL row errors instead of stopping at the first:
  imported [Row[line=1, sku=TEA, qty=2], Row[line=5, sku=MUG, qty=1]]
  rejected line 2: expected 2 fields
  rejected line 3: qty 'abc' is not a number
  rejected line 4: qty must be positive
    [log once] req-5 VALIDATION_FAILED: 3 invalid rows
  Response[status=400, body={"error":"VALIDATION_FAILED","message":"3 invalid rows"}]
```

- **req-1**: a transient timeout is retried with exponential backoff (10 ms, 20 ms) and then succeeds.
- **req-2**: `NotFoundException` is *not* retryable, so it fails immediately with 404.
- **req-3**: the dependency stays down; after 3 attempts we give up with 503.
- **req-4**: a programming bug becomes a generic 500 with only a request id; details go to the log.
- **Bulk import**: a sealed `Result` type (`Ok` / `Err`) with pattern-matching `switch` collects all row errors; the caller then decides to reject with 400.

Trade-offs

Choice	Benefit	Cost
Retries	Survive blips	Duplicate side effects unless idempotent; can amplify load
Global handler	Consistent responses, log once	Must avoid swallowing context
Result types	Explicit expected failures	Verbose in Java; mixing with exceptions needs a clear rule

When not to use it

Don't retry non-idempotent operations (e.g. "charge card") unless you use an **idempotency key** (a unique ID the client sends with a request so the server can recognise and ignore repeats), and never retry validation or authorisation failures.

Common mistakes

- Log-and-rethrow at every layer.
- Catching `Exception` in business code.
- Retrying immediately in a tight loop without backoff.
- Returning 200 with an error body.

Real-world examples

- **Spring** `@ControllerAdvice` + `@ExceptionHandler` — a global exception-to-response mapper; **RFC 9457 "Problem Details"** (successor of RFC 7807) standardises JSON error bodies, supported by Spring's `ProblemDetail`.
- **Resilience4j** (successor to Netflix Hystrix, which is in maintenance mode): retry, circuit breaker, rate limiter (caps calls per second), bulkhead (isolates resources so one failing part can't exhaust them all).
- **AWS SDKs** (Amazon Web Services' client libraries) retry throttling ("slow down" responses)/transient errors with exponential backoff and jitter by default.
- **Stripe's API** supports idempotency keys so clients can safely retry payment requests.

In the interview

> **30-second answer:** "I classify errors into validation, business, not-found, transient and bugs; use one exception hierarchy with codes and a retryable flag; translate low-level errors in adapters; retry only transient, idempotent operations with backoff and jitter; and map everything to safe responses in one global handler that logs once with a request id."

Follow-ups:

- *Q: Why jitter?* A: Without it, many clients retry at the same moments and overload the recovering service (thundering herd).
- *Q: How do you make payment retries safe?* A: Idempotency keys — the server stores the result per key and returns it for repeats.

Self-check

1. Q: Which errors should be retried? → A: Transient ones, and only for idempotent operations.
2. Q: Where should errors be logged? → A: Once, at the boundary/global handler (with a correlation id).

TL;DR

Classify, translate at boundaries, retry only transient + idempotent with backoff, result values for expected failures, one global handler that logs once and hides internals.

---

17. Testing Strategies (unit vs integration, test doubles, mocks)

What it is

A **testing strategy** decides *what kinds* of tests to write, *how many* of each, and *how to isolate* code under test. It builds on §8 (clean unit tests).

Analogy: car manufacturing — each part is tested alone (unit), assembled systems are tested together (integration), and finally the whole car is test-driven (end-to-end).

The problem it solves

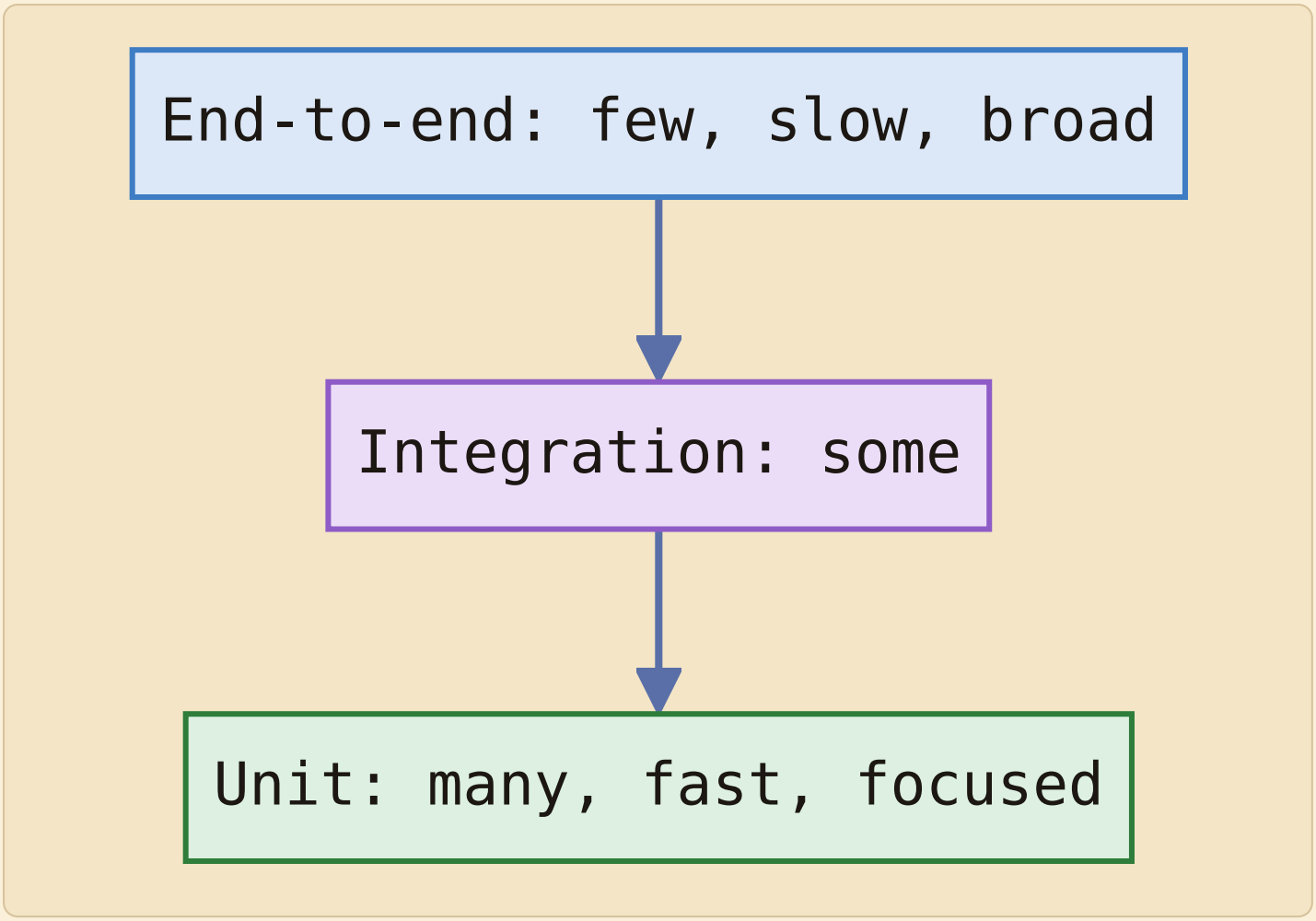
Too few tests → bugs reach production. The wrong mix (mostly slow end-to-end UI tests) → slow, flaky (sometimes passing, sometimes failing with no code change) pipelines (the automated build-and-test runs in CI) that people stop trusting. Over-mocking → tests that pass while the real system is broken, and break on every refactor.

How it works

Kinds of tests:

Kind	Scope	Speed	Catches
Unit	One class/function, collaborators replaced by doubles	Milliseconds	Logic errors
Integration	Several real components together (DB, message broker — a server that passes messages between services, your modules)	Seconds	Wiring, queries, serialisation (turning objects into bytes/text and back), config
Contract	Agreement between a consumer and a provider service	Fast	API mismatches between services
End-to-end (E2E)	The whole system through its UI/API	Slow	User-journey breakage
Others	Performance/load, security, exploratory, property-based, mutation	Varies	Non-functional issues, weak tests

The Test Pyramid (popularised by Mike Cohn): many fast unit tests at the base, fewer integration tests in the middle, very few E2E tests at the top. Avoid the "ice-cream cone" (mostly manual/E2E tests).



Test doubles (Gerard Meszaros' vocabulary, popularised by Martin Fowler):

Double	What it does	In our code
Dummy	Passed to fill a parameter; never used	`DummyAuditLog` (throws if called)
Stub	Returns canned answers to calls	`StubCatalog`
Spy	A stub that records calls for later assertions	`SpySender.sent`
Mock	Pre-programmed with expectations; verifies the interaction itself	`MockAuditLog.expect(...).verify()`
Fake	A working, simplified implementation	`FakeCoupons`, in-memory repositories

State vs behaviour verification: stubs/fakes support *state verification* (check the result or the resulting state); mocks support *behaviour verification* (check that the right calls happened). Prefer state verification; use mocks for outgoing side effects you can't observe otherwise (emails sent, audit events).

What to test: public behaviour, business rules, edge cases (empty, null, boundaries, duplicates), error paths, and every bug fix (regression tests). Don't test private methods directly, trivial accessors, or the framework itself.

Mocking rules of thumb: don't mock types you don't own (wrap them first and mock your wrapper — see Boundaries); don't mock value objects; one or two mocks per test at most; if a test needs many mocks, the class has too many dependencies.

Coverage: a coverage tool (e.g. JaCoCo) shows untested code; high coverage is *necessary but not sufficient* — mutation testing (e.g. PIT) checks whether tests actually detect changes.

Java example

The code is in §8 (`lld.ch06.testing`): `CheckoutTest` uses all five double types for fast unit tests; `CheckoutIntegrationTest` wires real in-memory implementations together; `BeforeStyleTest` shows what to avoid. The output there shows all eight tests passing.

Trade-offs

More unit tests	More integration/E2E tests
Fast, precise failure location	Higher confidence the pieces fit
May miss wiring/config bugs	Slow, flakier, harder to debug
Over-mocking couples tests to implementation	Need environments (DB, containers — isolated runtime packages such as Docker)

When not to use it

Don't insist on unit tests for thin glue code that only wires frameworks — an integration test covers it better. Don't write E2E tests for every edge case — push those down to unit tests.

Common mistakes

- Mocking everything, including value objects and the class under test.
- Flaky tests (time, randomness, order dependence, shared DB state) left unfixed — they destroy trust.
- Testing implementation details (exact internal calls) instead of outcomes.
- No tests for error paths.

Real-world examples

- **JUnit 5**, **Mockito** (mocks/spies/stubs), **AssertJ**, **Testcontainers** (real PostgreSQL/Kafka in Docker for integration tests), **WireMock** (fake HTTP services), **Pact** (consumer-driven contract tests), **PIT** (mutation testing), **JaCoCo** (coverage).

- Spring Boot test slices (`@WebMvcTest`, `@DataJpaTest`) for focused integration tests.

In the interview

> **30-second answer:** "I follow the test pyramid: many fast unit tests with test doubles, fewer integration tests with real components like the database via Testcontainers, and a handful of end-to-end tests. I prefer state verification with stubs and fakes, use mocks only for outgoing side effects, and never mock types I don't own."

Follow-ups:

- *Q: Mock vs stub?* A: A stub supplies canned answers so the test can check state; a mock is set up with expectations and verifies the calls themselves.
- *Q: Spy vs mock?* A: A spy records calls for the test to assert afterwards; a mock checks expectations itself.
- *Q: What's a fake?* A: A lightweight working implementation, e.g. an in-memory repository.

Self-check

1. Q: Name the five test doubles. → A: Dummy, stub, spy, mock, fake.
2. Q: Why a pyramid shape? → A: Lower-level tests are faster, cheaper and more precise; top-level tests are slow and brittle, so keep few.
3. Q: Is 100% coverage proof of good tests? → A: No — it shows code ran, not that assertions would catch bugs (mutation testing checks that).

TL;DR

Many unit tests, some integration, few E2E; pick the right double (dummy/stub/spy/mock/fake); verify state over interactions; don't mock what you don't own.

Chapter summary

Topic	Remember this
Names	Reveal intent, pronounceable, searchable, one word per concept
Functions	Small, one thing, one level, stepdown, ≤2–3 args, no flags, CQS, exceptions
Comments	Explain in code; comment only why/warnings/legal/public API
Formatting	Newspaper order, related things close, automated team style
Objects vs data	Objects hide data; data structures expose it; anti-symmetry decides
Error handling	Unchecked exceptions with context, wrap vendors, no null in or out
Boundaries	Wrap third-party code; learning tests; define interfaces you need
Unit tests	TDD laws, one concept per test, F.I.R.S.T.
Classes	Small by responsibility, cohesive, organised for change
Systems	Composition root + DI; cross-cutting via proxies/aspects
Emergence	Beck: tests pass → reveals intent → no duplication → fewest elements (*Clean Code* swaps the middle two)
Concurrency	Separate it, share little, use the library, test hard
Smells	C/E/F/G/J/N/T checklist
Code review	Correctness → design → readability → tests → security → performance
Error strategy	Classify, translate, retry transient, handle globally, log once
Testing strategy	Pyramid; right test double; state over behaviour verification

Chapter self-check

1. Q: A method `boolean saveAndValidate(Order o, boolean sendEmail)` breaks which rules? → A: Does two things, flag argument (F3), likely violates CQS (returns a boolean while changing state), name reveals multiple responsibilities.
2. Q: What should `findOrders(customerId)` return when there are none? → A: An empty list, never `null`.
3. Q: How do you test code that calls `LocalDate.now()`? → A: Inject a `Clock`; use `Clock.fixed` in tests.
4. Q: What does a dynamic proxy let you do cleanly? → A: Apply a cross-cutting concern (logging, transactions) to every method of an interface in one place.
5. Q: Which of Beck's rules has the highest priority? → A: Passes all the tests.

TL;DR (chapter)

Write code for the next reader: honest names, tiny single-purpose functions, few comments that explain why, consistent layout, clear exceptions instead of codes or nulls, foreign code behind your own interfaces, clean fast tests, small cohesive classes wired from one place, simple design that emerges through refactoring, careful concurrency — and review everything against a shared checklist.

Chapter 7 — Refactoring

> **What you will learn.** What refactoring is (and is not), when to do it, how tests make it safe, the full catalogue of **code smells** (the signals that tell you *where* to refactor), the full catalogue of **refactorings** from Martin Fowler's *Refactoring* (2nd edition, 2018), a smell → refactoring lookup table, a complete worked example in Java that goes from messy to clean in six compiling steps, and how refactoring leads you toward design patterns (named, reusable solutions to recurring design problems; Chapters 8–11).
>
> **Why it matters for LLD interviews.** Machine-coding rounds (interview rounds where you build a small working program in about 60–120 minutes; a core part of LLD — low-level design — interviews) often end with "now add feature X" or "what would you clean up?". Interviewers want to hear you name a smell ("this switch is repeated in three places — a Repeated Switches smell") and name the fix ("Replace Conditional with Polymorphism, which gives us a Strategy" — the Strategy pattern puts each interchangeable algorithm in its own class). This chapter gives you that vocabulary.

Chapter map

Part	Content
7.1	Refactoring fundamentals — definition, two hats, why, when, when not, tests, performance, rewrite vs refactor, IDE (Integrated Development Environment, e.g. IntelliJ IDEA) automation, legacy code
7.2	Code smells — the 24 smells of the 2nd edition, grouped the refactoring.guru way
7.3	The refactoring catalogue — all 61 named refactorings of the 2nd edition, by book chapter, plus the extra ones on refactoring.com
7.4	Smell → refactorings lookup table
7.5	Worked example — a messy invoice class refactored in six compiling steps
7.6	Refactoring toward patterns — Strategy, State, Special Case, Template Method, Factory
7.7	Trade-offs, mistakes, real-world examples, interview answers, self-check, TL;DR
7.8	Full source code and program output

7.1 Refactoring fundamentals

What it is

Refactoring (noun) is a change made to the internal structure of software to make it easier to understand and cheaper to modify, *without changing its observable behaviour*.

Refactoring (verb) is restructuring software by applying a *series* of such small, behaviour-preserving changes.

Two words in that definition do all the work:

- **Behaviour-preserving.** After the change, the program does the same thing from the outside. Same outputs for the same inputs, same side effects (changes made besides returning a value, such as writing a file), same errors. The only things that may change are *internal* — names, where code lives, how it is split up. (Performance may change a little; see 7.1.9.)
- **Small steps.** Each step is so small that it is almost impossible to get wrong, and the code compiles and the tests pass after *every* step. You chain many tiny steps to get a big change. If you ever sit with broken code for an hour, you were not refactoring — you were rewriting.

Everyday analogy. Think of reorganising a kitchen while the restaurant stays open. You move one shelf at a time. After each move, the cooks can still make every dish. You never tear the whole kitchen down on a Friday night.

> **Term — observable behaviour.** What a user (or calling code) can see: return values, printed output, data written, exceptions thrown. The *call stack* (the chain of method calls currently in progress), the *class structure*, and the *names of private methods* are not observable behaviour.

What refactoring is NOT.

Activity	Is it refactoring?	Why
Renaming a method with the IDE	Yes	Behaviour unchanged, tiny step
Fixing a bug	No	Behaviour changes (on purpose)
Adding a feature	No	Behaviour changes
Performance tuning	No	Goal is speed, not clarity (though behaviour is preserved)
"I spent two days restructuring and nothing worked until the end"	No	That is restructuring/rewriting, not refactoring
Reformatting code	Borderline	Behaviour-preserving, but it does not improve design

> **Term — restructuring.** The general word for reorganising code. Refactoring is a *specific kind* of restructuring: small, behaviour-preserving steps.

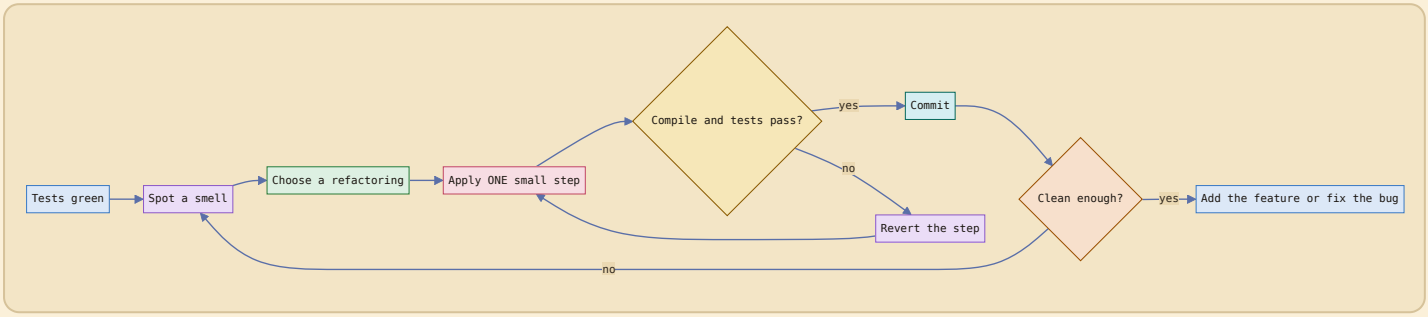
The problem it solves

Code decays. Every quick fix, every "just add another `if`", every copy-paste makes the next change a little harder. This is called **design decay** or **technical debt** — the extra cost you pay on every future change because the code is harder to work with than it needs to be.

Refactoring pays down that debt. It keeps the design good *continuously*, instead of hoping to get the design right up front (which never fully works, because requirements change).

How it works

1. **Make sure you have tests** that pin down current behaviour (7.1.7). Run them — they must be green (all passing).
2. **Spot a smell** — a surface sign that the design could be better (7.2).
3. **Pick a named refactoring** that cures the smell (7.3, 7.4).
4. **Apply one small step** of it (ideally with the IDE's automated refactoring).
5. **Compile and run the tests.** Green? Commit (locally) — i.e. save a snapshot of the code in version control such as Git. Red? Undo the step (don't debug) and take a smaller step.
6. **Repeat** until the code is clean enough for the job at hand.



7.1.1 The two hats

Kent Beck's metaphor: when you program you wear **one of two hats** at any moment.

Hat	You are allowed to	You are NOT allowed to
Adding function	Add new behaviour, add new tests	Restructure existing code
Refactoring	Restructure code	Add behaviour or tests for new behaviour (you may add tests for *existing* behaviour you forgot to cover)

You swap hats often — maybe every few minutes — but you always know which one you are wearing. Mixing them ("I'll just fix this bug while I extract this method") is how refactoring becomes risky: when a test fails you cannot tell which change caused it.

7.1.2 Why refactor

- **It improves the design.** Without it, the internal design (architecture) of a program decays as people change code to meet short-term goals. Regular refactoring keeps the code in shape. A big part of this is **removing duplication** — making sure each fact lives in exactly one place.
- **It makes software easier to understand.** Code is read far more often than it is written. Refactoring lets you move your understanding *out of your head and into the code*, so the next reader (including you in six months) does not need to rediscover it.
- **It helps you find bugs.** When you clarify the structure, assumptions become visible and bugs have fewer places to hide.
- **It helps you program faster.** This is the surprising one. Fowler calls it the **Design Stamina Hypothesis**: a team that keeps its internal quality high will, after a short while, add features *faster* than a team that does not, because each feature builds on clean code instead of fighting a mess. Speed is the economic argument for refactoring — not "clean code for its own sake".

7.1.3 When to refactor

Trigger	What it means	Example
Rule of Three (Don Roberts)	First time, just do it. Second time you do something similar, wince but duplicate. Third time, refactor.	Third copy of the same validation code → extract a validator
Preparatory refactoring	Refactor *just before* adding a feature, to make the feature easy to add. "Make the change easy, then make the easy change."	Before adding HTML output, split calculation from text formatting (see 7.5, step 3)
Comprehension refactoring	Refactor to understand code you are reading. Rename a variable once you understand it; extract a block once you know what it does. The understanding moves into the code.	Rename `d` to `daysOverdue` after working out what it holds
Litter-pickup refactoring	You notice mess while working on something else. Fix it if it is quick; if not, note it and come back. The **Boy Scout Rule** — leave the campground cleaner than you found it.	Remove an unused parameter while passing by
Planned vs opportunistic	Most refactoring should be *opportunistic* — woven into feature work and bug fixes, not scheduled separately. Planned refactoring episodes are sometimes needed when a team has neglected the code, but they should be rare.	Good: every feature PR (pull request — a change submitted for review) contains a few small refactoring commits. Warning sign: a "refactoring sprint" (a fixed 1–4-week work period) every quarter
Long-term refactoring	Big changes (replacing a library, splitting a module) done gradually over weeks, keeping the system working the whole time — e.g. with **Branch by Abstraction** (introduce an abstraction layer, switch callers to it, swap the implementation behind it, delete the old one).	Moving from one ORM (library mapping objects to database tables) to another behind a repository interface
During code review	Reviewing is a natural moment to suggest (or pair on) refactorings. Fowler prefers doing it *with the author*, trying the refactoring ideas as you go, rather than only leaving comments.	"This method has four flag parameters — can we Remove Flag Argument?"

> **Term — Rule of Three.** A heuristic: do not abstract on the first or second occurrence of a pattern (you do not yet know what varies); abstract on the third, when the shape is clear.

Should you tell your manager? Fowler's position: for a manager who does not understand technical work, do not ask permission for refactoring any more than you ask permission to write tests. It is part of how professionals deliver features quickly. (For large, planned efforts, of course, discuss them.)

7.1.4 When NOT to refactor

- **Code you never need to touch or understand.** Messy code hidden behind a clean API (the set of methods other code calls), that works and never changes, can stay messy. Refactor only what gets in your way.
- **When rewriting is cheaper** than refactoring (see 7.1.10). This is a judgement call and usually needs experience.
- **Without a safety net** — if you cannot test it and cannot use safe automated refactorings, first add tests (7.1.11 shows how for legacy code).
- **On a deadline that truly cannot move**, a big refactoring may wait — but note that the Design Stamina Hypothesis says skipping it usually makes you *slower*, even in the short term. Deadlines are rarely a good excuse for small refactorings.
- **Published interfaces** that you do not control the callers of (a public library API, a REST endpoint — a URL of a web API, see System Design Fundamentals — used by other teams). You can still refactor

them, but you must keep the old version working for a while and deprecate it (mark it as outdated and due for removal) (see *Change Function Declaration*, migration mechanics).

7.1.5 Problems and costs of refactoring

Problem	What goes wrong	Mitigation
Slowing down new features	Refactoring "takes time away" from features	Refactor *because* it speeds features up; focus on code you are about to change
Code ownership	You cannot rename a method whose callers are in another team's repo (repository — a project's version-controlled code store)	Keep the old name as a delegating, `@Deprecated` method; migrate callers later; prefer weak (team-wide) code ownership
Long-lived feature branches (lines of work kept apart from the main code for a long time)	Refactoring renames and moves code, which makes merges painful	**Continuous Integration** (merge into the shared main line often, with automatic builds and tests) / trunk-based development (everyone works on one main branch): integrate at least daily, so refactorings conflict less
Missing tests	Refactoring without tests is risky	Add tests first; use only IDE-automated, provably-safe refactorings until tests exist
Legacy code	Big, untested, tangled	Feathers' techniques (7.1.11)
Databases	Schema (table structure) changes are hard to reverse	**Evolutionary database design**: small migration scripts (scripts that change the database structure), versioned with the code; use **parallel change** (add new column, copy, switch readers, remove old)

> **Term — parallel change (expand / contract).** Make a breaking change in three phases: *expand* (add the new thing alongside the old), *migrate* (move all users to the new thing), *contract* (remove the old thing). At no point is anything broken.

7.1.6 Refactoring, architecture and YAGNI

In the past people tried to get the architecture "right" before writing code. Refactoring changes that: because you can reshape code safely later, you can build **only what today's requirements need** and evolve the design when new needs appear. This is **YAGNI** ("You Aren't Gonna Need It") — do not add flexibility for imagined future requirements. YAGNI does not mean "never think about architecture"; it means you trust refactoring to add the flexibility when the need becomes real (and you think harder up front only about things that are *truly* hard to change later, like a public API or a database schema).

7.1.7 Tests as the safety net

Refactoring is safe only if you can detect a behaviour change immediately. That is what **self-testing code** gives you: a suite of automated tests that checks itself (pass/fail, no human reading output) and runs in seconds.

- Run tests after **every** small step. With fast tests, a failure points at the few lines you just changed.
- If a test fails, **revert** — do not start debugging a refactoring. Then redo it in smaller steps.
- Tests should focus on **behaviour** (public API, outputs), not on private structure. Tests that check private methods break on every refactoring and become a cost instead of a safety net.
- **Characterisation tests** (a term from Michael Feathers): for code without tests, write tests that record what the code *currently does* — even if that looks wrong — so you notice if a refactoring changes it. The worked example in 7.5 uses exactly this: a checker that compares every step's output with the original, byte for byte.
- Consider **mutation testing** (e.g. PIT for Java) to check your tests actually catch changes — it deliberately introduces small bugs and reports any your tests fail to detect.

> **Term — safety net.** Anything that tells you quickly and reliably that you broke something: unit tests (one class tested in isolation), integration tests (several real parts tested together), the compiler's type checks, and the IDE's automated refactorings (which are proven safe by the tool).

7.1.8 IDE automation

Modern IDEs perform many refactorings *automatically and safely*, using the compiler's knowledge of the code (the **syntax tree** and **symbol resolution** — the parsed structure of the code, and knowing exactly which declaration each name refers to), not text search. A text search-and-replace of `name` would also hit comments, strings and unrelated fields; the IDE's Rename only changes that one symbol and all its real references.

History in one line: Bill Opdyke's 1992 PhD thesis formalised refactorings; John Brant and Don Roberts built the Smalltalk (an early object-oriented language) *Refactoring Browser*, the first automated tool; Java IDEs (IntelliJ IDEA, Eclipse) popularised it.

Refactoring	IntelliJ IDEA (Windows/Linux · macOS)	Eclipse (Windows/Linux)
Rename	Shift+F6 · Shift+F6	Alt+Shift+R
Extract Method (Extract Function)	Ctrl+Alt+M · Cmd+Opt+M	Alt+Shift+M
Extract Variable	Ctrl+Alt+V · Cmd+Opt+V	Alt+Shift+L
Extract Field / Constant / Parameter	Ctrl+Alt+F / C / P · Cmd+Opt+F / C / P	(Refactor menu)
Inline	Ctrl+Alt+N · Cmd+Opt+N	Alt+Shift+I
Change Signature	Ctrl+F6 · Cmd+F6	Alt+Shift+C
Move	F6 · F6	Alt+Shift+V
"Refactor This" menu	Ctrl+Alt+Shift+T · Ctrl+T	Alt+Shift+T

(Shortcuts are the default keymaps; check your IDE if you use a custom keymap.)

Beyond the IDE there are **large-scale automated refactoring tools**, such as OpenRewrite (recipes that, for example, migrate a whole codebase between framework versions) and Google's Refaster templates in Error Prone (Google's bug-finding plugin for the Java compiler). These let a company apply the same refactoring across thousands of files.

> **Caution.** Automated refactorings are safe for statically known calls. They can miss **reflection** (`Class.forName("…")` — code that looks up classes or methods by name at runtime), names used in **configuration files**, **serialised field names** (serialising = turning objects into text or bytes, e.g. JSON, a plain-text data format; JSON/Jackson, JPA — Java's standard ORM API — column names inferred from field names), and code in other repositories. After an IDE rename of a field that is serialised, the JSON your API returns changes — that *is* an observable behaviour change.

7.1.9 Performance and refactoring

A common worry: "refactoring adds method calls and small objects, so the program gets slower." Fowler's answer has three parts:

1. **Most refactorings have negligible performance effect.** The JIT (Just-In-Time) compiler — the part of the JVM, Java's runtime, that compiles frequently run code to machine code while the program runs — inlines small methods (copies their body into the caller); extra method calls are nearly free in modern Java.
2. **Programs spend most of their time in a small fraction of the code.** Optimising everywhere wastes effort on code that is rarely run. (The book retells a story from Ron Jeffries about Chrysler's slow payroll system: nobody's guesses about the cause were right; profiling (measuring where a program actually spends its time) found the time going to one unexpected spot, and a small fix there made it fast.)

3. **Well-factored code is easier to tune.** When you do need speed, you profile, find the hot spot (the small piece of code where most time goes), and — because the code is well-structured — the hot spot is small and isolated, so it is easy to optimise.

So the recommended order is: **make it work → make it clean (refactor) → make it fast (profile, then optimise only the hot spots)**. Sometimes a refactoring makes code slightly slower; that is fine if it makes the later, targeted optimisation easier.

Example from this chapter. In 7.5 step 2 we replace a summing loop with a stream (Java's pipeline for processing a sequence of elements). `DoubleStream.sum()` is allowed to use *compensated summation* (a technique that tracks rounding errors to make floating-point sums more accurate; OpenJDK, the reference Java implementation, does), so it can give a result that differs in the last bit from a plain loop — a tiny *behaviour change*. To stay a true refactoring we used `reduce(0.0, Double::sum)`, which adds left to right exactly like the loop. Refactoring demands this kind of care.

7.1.10 Refactoring vs rewriting

	Refactoring	Rewriting
Unit of change	Many tiny, safe steps	One big replacement
System works during the change?	Always	Usually not until the end
Risk	Low	High — known bugs fixed years ago get reintroduced; the old system keeps evolving while you rewrite
Delivers value	Continuously	Only at the end
Good when	Code mostly works, needs reshaping	Code is so broken it does not work at all, or the technology must change completely

Even when you must replace a system, you rarely do it in one go. The **Strangler Fig** approach (named by Fowler after a vine that slowly grows around a tree) builds the new system *around* the old one, routing one feature at a time to the new code until the old code can be switched off. It is the rewrite equivalent of "small steps".

7.1.11 Legacy code

Michael Feathers (*Working Effectively with Legacy Code*, 2004) defines **legacy code as code without tests**. Such code is hard to refactor because you have no safety net, and hard to test because it was not designed for it. The way out is a loop:

1. **Identify change points** — where you need to change code.
2. **Find test points** — where you can observe behaviour.
3. **Break dependencies** — using a **seam**: a place where you can change behaviour without editing the code there, e.g. by passing in an interface instead of calling `new Database()` inside a method (constructor injection), or overriding a method in a test subclass.
4. **Write characterisation tests** that pin down current behaviour.
5. **Make the change and refactor** under that safety net.

Other useful techniques from that book:

- **Sprout Method / Sprout Class** — write the new feature as a *new*, fully tested method or class, and call it from the old code with a one-line change.
- **Wrap Method / Wrap Class** — rename the old method and create a new method with the old name that calls the old one plus the new behaviour (a Decorator-like move — a Decorator wraps an object to add behaviour).
- Use **safe automated refactorings only** (Rename, Extract Method with the IDE) until tests exist.
- Do not try to clean everything. Refactor the areas you touch; over time the most-changed code becomes the cleanest.

7.2 Code smells

What it is

A **code smell** (a term coined by Kent Beck) is a *surface sign* in the code that usually points to a deeper design problem. A smell is not a bug — the code works. It is a hint that the code will be hard to understand or change. Like a smell in a fridge, it tells you *where to look*, not exactly what is wrong.

Smells are **heuristics, not rules**. A long method is not always bad; a switch statement is not always bad. The smell tells you to look closer and use judgement.

The problem it solves

Knowing *how* to refactor is not enough; you must also know *when* and *where*. Fowler and Beck deliberately give no precise metrics ("no method longer than 10 lines"). Instead they give a catalogue of named smells, each with the refactorings that usually cure it. Names matter: in a code review, "Feature Envy" communicates in two words what would otherwise take a paragraph.

How it works

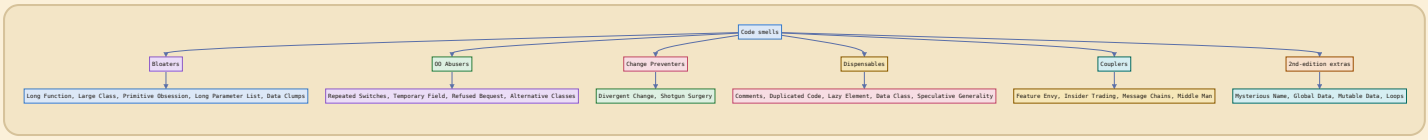
1. Read the code and notice discomfort: "I had to scroll", "I changed five files for one feature", "I can't tell what `x2` is".
2. Match the discomfort to a named smell (the table below).
3. Look up the refactorings that cure it (each smell below, and 7.4).
4. Apply them in small steps under tests.

Smell groups (refactoring.guru classification)

refactoring.guru (a popular website on refactoring and design patterns) groups smells into five families. It uses mostly the 1st-edition names, so the table maps them to the 2nd-edition names used in this chapter.

Group	Idea	Smells (2nd-ed name; refactoring.guru name in brackets if different)
Bloaters	Code, methods and classes that have grown so big they are hard to work with. They usually grow slowly.	Long Function *(Long Method)*, Large Class, Primitive Obsession, Long Parameter List, Data Clumps
Object-Orientation Abusers	Incomplete or wrong use of OO (object-oriented) principles.	Repeated Switches *(Switch Statements)*, Temporary Field, Refused Bequest, Alternative Classes with Different Interfaces
Change Preventers	One change forces many other changes.	Divergent Change, Shotgun Surgery *(also: Parallel Inheritance Hierarchies — a 1st-ed smell)*
Dispensables	Something pointless whose removal makes the code cleaner.	Comments, Duplicated Code *(Duplicate Code)*, Lazy Element *(Lazy Class)*, Data Class, Speculative Generality *(also: Dead Code — in the 2nd ed. this is the refactoring Remove Dead Code)*
Couplers	Too much coupling between classes, or too much delegation (handing work off to another object).	Feature Envy, Insider Trading *(Inappropriate Intimacy)*, Message Chains, Middle Man *(also: Incomplete Library Class — a 1st-ed smell)*
New in the 2nd edition (not in refactoring.guru's five groups)		Mysterious Name (a comprehension smell), Global Data and Mutable Data (both about uncontrolled shared state — closest to Couplers/Change Preventers), Loops (closest to Bloaters)

> **Term — coupling.** How much one piece of code depends on another. High coupling means a change in one place forces changes in others. **Cohesion** is the opposite idea inside one unit: how strongly the things inside a class or module belong together. Good design: low coupling, high cohesion.



Below are all 24 smells from the 2nd edition, in the book's order. Each has: **what it is**, **why it hurts**, a small **Java example** (smelly → cured), and **refactorings that fix it** (the ones Fowler names for that smell; anything extra is marked). The names match the catalogue in 7.3.

> **Note on the snippets.** Several short snippets use `double` for prices or balances to stay tiny. Real money should use `BigDecimal` (Java's exact decimal type) or a `Money` value object (a small immutable object compared by its value) — see 7.2.11 and `code/smells/Money.java`.

7.2.1 Mysterious Name

What it is. A function, variable, field, class or module whose name does not tell you what it does or holds: `data`, `tmp`, `proc()`, `x2`, `Manager`, `handle()`.

Why it hurts. Names are the main tool for making code understandable. A bad name forces every reader to decode the implementation. And if you *cannot* think of a good name, that is often a sign of a deeper design problem — the thing does too much or has no clear purpose.

```
// Smelly
double calc(List<Double> l, int d) { ... }

// Cured
double averageDailyRevenue(List<Double> dailyRevenues, int daysInPeriod) { ... }
```

Refactorings. Change Function Declaration (rename), Rename Variable, Rename Field.

7.2.2 Duplicated Code

What it is. The same (or nearly the same) code structure in more than one place.

Why it hurts. Every change must be made in every copy, and you must *find* every copy and check whether the small differences matter. Sooner or later one copy is missed and a bug appears.

```
// Smelly: two methods repeat the same tax rule
double invoiceTotal(double net)  { return net + net * 0.18; }
double quoteTotal(double net)    { return net + net * 0.18; }

// Cured: one place owns the rule
private static final double GST_RATE = 0.18;
double withTax(double net)       { return net + net * GST_RATE; }
double invoiceTotal(double net)  { return withTax(net); }
double quoteTotal(double net)    { return withTax(net); }
```

Refactorings. Fowler lists: Extract Function; Slide Statements (to bring similar code together so it can be extracted); Pull Up Method (if the copies are in sibling subclasses). Also useful (not in Fowler's list for this smell): Replace Inline Code with Function Call (if a library already does it); Substitute Algorithm (if two different algorithms do the same thing).

7.2.3 Long Function

What it is. A function that does too much and is too long to understand at a glance. Fowler's heuristic: whenever you feel you need a **comment** to explain a block, extract that block into a function named after the *intent* of the block.

Why it hurts. Long functions are hard to understand, hard to reuse partially, and hard to test. Short, well-named functions are self-documenting: you read the name and skip the body.

```
// Smelly
void printOwing(Invoice inv) {
    System.out.println("*** Customer Owes ***");
    double outstanding = 0;
    for (Order o : inv.orders()) outstanding += o.amount();      // calculate outstanding
    inv.setDueDate(LocalDate.now().plusDays(30));               // record due date
    System.out.println("name: " + inv.customer());               // print details
    System.out.println("amount: " + outstanding);
}

// Cured: each block became a named function
void printOwing(Invoice inv) {
    printBanner();
    double outstanding = outstandingOf(inv);
    recordDueDate(inv);
    printDetails(inv, outstanding);
}
```

Refactorings. Extract Function (the main one, ~99% of the time), Replace Temp with Query, Introduce Parameter Object, Preserve Whole Object, Replace Function with Command, Decompose Conditional, Replace Conditional with Polymorphism, Split Loop.

7.2.4 Long Parameter List

What it is. A function that takes many parameters (often more than three or four), especially several of the same type, or boolean flags.

Why it hurts. Callers mix up the order (`move(10, 20, 5, 3)` — which is which?), every new parameter changes every caller, and long lists often hide a missing concept.

```
// Smelly
Booking book(String hotelId, LocalDate from, LocalDate to, int adults, int children, boolean breakfast) { ... }

// Cured: parameter objects for the clumps, and an explicit enum instead of an unreadable boolean flag
record Stay(LocalDate from, LocalDate to) { }
record Guests(int adults, int children) { }
Booking book(Hotel hotel, Stay stay, Guests guests, MealPlan mealPlan) { ... }
```

Refactorings. Replace Parameter with Query, Preserve Whole Object, Introduce Parameter Object, Remove Flag Argument, Combine Functions into Class (several functions share the same parameters → make them fields of a class).

7.2.5 Global Data

What it is. Data that can be read and modified from anywhere: global variables, public static mutable fields (`static` fields belong to the class and are shared by the whole program), and mutable singletons (classes with exactly one shared instance).

Why it hurts. Any code anywhere can change it, and there is no way to find *who* changed it. Bugs appear far away from their cause. Fowler calls it one of the most pungent smells; the problems grow with the size of the program. (Class variables and singletons count too.)

```java
// Smelly
public class Config { public static double discountRate = 0.1; }      // anyone can write it

// Cured: Encapsulate Variable - one controlled access point, easy to find every writer.
// (The range check is a NEW rule added afterwards, wearing the "adding function" hat - it is a behaviour change,
not part of the refactoring.)
public final class Config {
    private static volatile double discountRate = 0.1;
    public static double discountRate() { return discountRate; }
    public static void setDiscountRate(double rate) {
        if (rate < 0 || rate > 0.5) throw new IllegalArgumentException("rate out of range");
        discountRate = rate;
    }
}
// Even better: pass the value in (dependency injection) instead of reading a static at all.
```

Refactorings. Encapsulate Variable (first step, always), then narrow its scope — ideally move it into a class or module so only a few functions can see it.

7.2.6 Mutable Data

What it is. Data that is changed after creation, especially when it is shared, or when the same variable is updated for different purposes.

Why it hurts. A change in one place can break code in another place that assumed the value stayed the same. These bugs are hard to find, especially with threads (several paths of execution running at the same time). Functional programming is built on the idea that data should never change; you do not need to go that far, but you should limit where mutation can happen.

```java
// Smelly: callers can change the list inside the object, and total is a stored copy that can go stale
class Cart {
    public List<Item> items = new ArrayList<>();
    public double total;                          // must be updated by hand after every change
}

// Cured
final class Cart {
    private final List<Item> items = new ArrayList<>();
    void add(Item item) { items.add(item); }
    List<Item> items() { return List.copyOf(items); }                    // Encapsulate Collection
    double total() { return items.stream().mapToDouble(Item::price).sum(); } // Replace Derived Variable with
Query
}
```

Refactorings. Encapsulate Variable, Split Variable, Slide Statements, Extract Function, Separate Query from Modifier, Remove Setting Method, Replace Derived Variable with Query, Combine Functions into Class, Combine Functions into Transform, Change Reference to Value (use immutable value objects such as Java records — compact, immutable data classes).

7.2.7 Divergent Change

What it is. **One class is changed for many different reasons.** "I have to edit this class when we add a database, *and* when we add a financial instrument, *and* when we change the report format."

Why it hurts. The class mixes several responsibilities (it breaks the **Single Responsibility Principle**, Chapter 4). Each kind of change risks breaking the others, and different people edit the same file for unrelated reasons.

```java
// Smelly: storage, pricing and formatting in one class
class ProductService {
    void saveToMySql(Product p) { ... }         // changes when the database changes
    double priceWithTax(Product p) { ... }      // changes when tax rules change
    String toCsv(Product p) { ... }             // changes when the export format changes
}

// Cured: one reason to change per class
class ProductRepository { void save(Product p) { ... } }
class PricingService    { double priceWithTax(Product p) { ... } }
class ProductCsvWriter  { String toCsv(Product p) { ... } }
```

Refactorings. Split Phase (when the changes fall into sequential phases, e.g. parse then compute), Move Function, Extract Function, Extract Class.

7.2.8 Shotgun Surgery

What it is. The opposite of Divergent Change: **one change forces many small edits in many classes.**

Why it hurts. You have to find all the places, and it is easy to miss one. The logic for one concept is scattered.

```java
// Smelly: the "free-shipping threshold" rule appears in 4 classes
class CartView      { boolean showBanner(double t)  { return t >= 500; } }
class CheckoutCtrl  { double shipping(double t)     { return t >= 500 ? 0 : 40; } }
class EmailBuilder  { String line(double t)         { return t >= 500 ? "Free shipping!" : ""; } }
class ReportJob     { long freeOrders(List<Double> ts) { return ts.stream().filter(t -> t >= 500).count(); } }

// Cured: the concept lives in one class, and the four callers use it
final class ShippingPolicy {
    static final double FREE_SHIPPING_THRESHOLD = 500;
    static boolean isFree(double total)   { return total >= FREE_SHIPPING_THRESHOLD; }
    static double charge(double total)    { return isFree(total) ? 0 : 40; }
}
```

Refactorings. Move Function and Move Field (gather everything into one module), Combine Functions into Class, Combine Functions into Transform, Split Phase; sometimes Inline Function / Inline Class first to pull the scattered bits together, then extract a better-shaped unit.

	Divergent Change	Shotgun Surgery
Shape	One class, many reasons to change	One reason to change, many classes
Fix	Split the class	Gather the scattered code

7.2.9 Feature Envy

What it is. A method that is more interested in *another* class's data than in its own — it calls many getters (methods that just return a field) on another object to compute something.

Why it hurts. The behaviour lives away from the data it uses, so a change to that data needs changes elsewhere. The basic OO rule: **put behaviour next to the data it uses**. (Exception: patterns like Strategy and Visitor — an object that carries one operation across a group of classes — deliberately separate them, to allow swapping behaviour.)

```java
// Smelly: Order computes a line total using only Line's data
class Order {
    double lineTotal(Line line) {
        return line.getUnitPrice() * line.getQuantity() * (1 - line.getDiscount());
    }
}

// Cured: Move Function - Line computes its own total
record Line(double unitPrice, int quantity, double discount) {
    double total() { return unitPrice * quantity * (1 - discount); }
}
```

Refactorings. Move Function; Extract Function first if only *part* of the method is envious, then move that part.

7.2.10 Data Clumps

What it is. The same group of data items always appear together — as fields in several classes, or as parameters in many signatures: `(street, city, zip)`, `(startDate, endDate)`, `(x, y)`, `(amount, currency)`.

Why it hurts. The group is really a concept that has not been named. Without a class, behaviour about it (e.g. "does this range contain a date?") gets duplicated everywhere. **Test:** delete one item of the clump — if the others no longer make sense alone, they belong in an object.

```java
// Smelly
boolean isAvailable(LocalDate start, LocalDate end) { ... }
double price(LocalDate start, LocalDate end, double nightly) { ... }

// Cured (see code/smells/DateRange.java)
record DateRange(LocalDate start, LocalDate end) {
    boolean contains(LocalDate day) { return !day.isBefore(start) && !day.isAfter(end); }
}
boolean isAvailable(DateRange stay) { ... }
```

Refactorings. Extract Class (for clumps of fields), Introduce Parameter Object and Preserve Whole Object (for clumps of parameters). Then look for Feature Envy — behaviour will want to move onto the new class.

7.2.11 Primitive Obsession

What it is. Using primitive types (`int`, `double`, `String`, `boolean`) for domain concepts that deserve their own type: money as `double`, phone numbers as `String`, a status as `"ACTIVE"`, a temperature as a bare `int` without a unit.

Why it hurts. No validation (`"banana"` is a valid `String` phone number), no behaviour (formatting and comparing logic is duplicated), easy mix-ups (`transfer(double amount, double fee)` — swap them and nothing complains). Money as `double` also suffers from floating-point error (binary `double` numbers cannot store most decimal fractions exactly): `0.1 + 0.2` prints `0.30000000000000004` (see the output in 7.8).

```java
// Smelly
void charge(double amount, String currency) { ... }

// Cured (see code/smells/Money.java): validation and arithmetic live in one type
record Money(BigDecimal amount, Currency currency) {
    Money plus(Money other) { /* checks same currency, then adds */ }
}
void charge(Money amount) { ... }
```

Refactorings. Replace Primitive with Object, Replace Type Code with Subclasses, Replace Conditional with Polymorphism (for type codes that drive behaviour), Extract Class, Introduce Parameter Object.

7.2.12 Repeated Switches

What it is. The *same* `switch` (or `if/else if` chain) on the same type code (a field, such as a string or int, that says which kind of thing an object is) appears in several places. (In the 1st edition this smell was "Switch Statements" — *any* switch. In the 2nd edition Fowler narrowed it: a single switch is fine; the problem is the *repetition*.)

Why it hurts. Adding a new case means finding and updating every switch — Shotgun Surgery. Miss one and you have a bug.

```
// Smelly: three switches on the same type
double speed(Bird b)    { switch (b.type) { case "EUROPEAN": ...; case "AFRICAN": ...; case "PARROT": ...; } }
String plumage(Bird b)  { switch (b.type) { case "EUROPEAN": ...; case "AFRICAN": ...; case "PARROT": ...; } }
int    lifespan(Bird b) { switch (b.type) { ... } }

// Cured: Replace Conditional with Polymorphism
sealed interface Bird permits EuropeanSwallow, AfricanSwallow, NorwegianBlueParrot {
    double speed();
    String plumage();
}
```

Refactorings. Replace Conditional with Polymorphism. (Java note: with a `sealed` type (one that lists all its permitted subtypes) or an `enum` (a fixed set of named constants), a *switch expression* is checked for exhaustiveness — covering every case — by the compiler, which removes much of the risk. A few such switches may be acceptable; polymorphism is still better when the behaviour per case is large.)

7.2.13 Loops

What it is. A classic `for`/`while` loop that filters, maps or aggregates a collection, where a pipeline would state the intent more clearly. (New in the 2nd edition — languages now have good first-class functions (functions that can be stored in variables and passed around); in Java, the Stream API since Java 8.)

Why it hurts. A loop makes the reader simulate the steps to see what it does. A pipeline names each step: *filter*, *map*, *collect*.

```
// Smelly
List<String> names = new ArrayList<>();
for (Employee e : employees) {
    if (e.department().equals("IT")) {
        names.add(e.name());
    }
}

// Cured: Replace Loop with Pipeline
List<String> names = employees.stream()
        .filter(e -> e.department().equals("IT"))
        .map(Employee::name)
        .toList();                      // Java 16+: returns an unmodifiable list
```

Refactorings. Replace Loop with Pipeline. (Note: `Stream.toList()` returns an *unmodifiable* list, while the loop built a mutable `ArrayList`. If a caller later adds to the list, that is a behaviour change — use `collect(Collectors.toCollection(ArrayList::new))` in that case.)

7.2.14 Lazy Element

What it is. A class, function or other structure that does not pull its weight — for example a class with one trivial method, a function whose body is as clear as its name, or a subclass that adds nothing. (1st edition: "Lazy Class".)

Why it hurts. Every element costs something: a name to learn, a file to open, an indirection to follow. If it adds nothing, it is pure cost. Lazy elements are often left behind after other refactorings, or

created for growth that never happened.

```
// Smelly: a wrapper that adds nothing
class NameFormatter { String format(String s) { return s.trim(); } }

// Cured: Inline Class / Inline Function
String cleanName = rawName.trim();
```

Refactorings. Inline Function, Inline Class, Collapse Hierarchy.

7.2.15 Speculative Generality

What it is. Hooks, abstract classes, parameters and special cases added "because we might need them someday" — but nobody uses them. Typical signs: an abstract class with only one subclass, unused parameters, methods only called by tests, a plug-in system with one plug-in.

Why it hurts. It makes the code harder to understand and maintain for no benefit. It violates **YAGNI**. If the need does come, refactoring can add the flexibility then — usually in a better shape, because you then know the real requirement.

```
// Smelly
interface PaymentProcessorFactoryProvider { PaymentProcessorFactory get(); }   // one implementation, ever
void pay(Money m, Map<String, Object> futureOptions) { ... }                   // futureOptions never used

// Cured
void pay(Money m) { processor.process(m); }
```

Refactorings. Collapse Hierarchy (useless abstract class), Inline Function and Inline Class (useless delegation), Change Function Declaration (remove unused parameters), Remove Dead Code (for code used only by its own tests — delete the tests too).

7.2.16 Temporary Field

What it is. A field that is set and used only in some situations (e.g. only during one complex calculation) and is empty/null the rest of the time.

Why it hurts. Readers expect an object to need all its fields. A field that is usually `null` is confusing — "why is this null? is that a bug?"

```
// Smelly: these fields are only meaningful while computeRoute() runs
class Planner {
    private Graph graph;
    private Map<Node, Integer> tmpDistances;   // null except inside computeRoute
    private Node tmpTarget;                    // null except inside computeRoute
}

// Cured: Extract Class - the temporary state gets its own short-lived object
class Planner {
    private Graph graph;
    Route computeRoute(Node from, Node to) { return new RouteSearch(graph, from, to).run(); }
}
final class RouteSearch {                      // lives only for one search
    private final Map<Node, Integer> distances = new HashMap<>();
    ...
}
```

Refactorings. Extract Class (put the fields and the code that uses them in their own class), Move Function, Introduce Special Case (for the conditional code that checks whether the field is set).

7.2.17 Message Chains

What it is. A client asks one object for another object, then asks that for another, and so on: `order.getCustomer().getAddress().getCity().getName()`. Also a long chain of temporary variables doing the

same thing.

Why it hurts. The client is coupled to the *whole structure* of the navigation. Any change in an intermediate relationship breaks the client. It violates the **Law of Demeter** ("talk only to your immediate friends", Chapter 5).

```
// Smelly
String city = order.getCustomer().getAddress().getCity();

// Cured: Hide Delegate (see code/smells/Order.java)
String city = order.shippingCity();
```

Caution: do not blindly hide every delegate — that creates Middle Men. Often better: look at what the final object is *used for*, and Extract Function + Move Function that usage down the chain.

Refactorings. Hide Delegate, Extract Function, Move Function.

7.2.18 Middle Man

What it is. A class whose methods mostly just forward calls to another class, adding nothing. (Some delegation is good — that is encapsulation. The smell is when *most* of the class is pass-through.)

Why it hurts. An extra layer to read and maintain, with no value. Every new feature of the delegate requires a new forwarding method.

```
// Smelly
class Person {
    private Department dept;
    Person getManager()          { return dept.getManager(); }
    String getDepartmentCode()   { return dept.getCode(); }
    Budget getDepartmentBudget() { return dept.getBudget(); }
    // ... ten more forwarding methods
}

// Cured: Remove Middle Man - let clients talk to the Department directly
Department department() { return dept; }
```

Refactorings. Remove Middle Man, Inline Function (for a few forwarders), Replace Superclass with Delegate or Replace Subclass with Delegate (if the middle man is tied up in inheritance).

Message Chains and Middle Man pull in opposite directions — Hide Delegate cures one and can cause the other. Use judgement to find the balance.

7.2.19 Insider Trading

What it is. Two modules (classes, packages) that exchange too much private information — they poke at each other's internals. Subclasses that know too much about their parents' internals are a common case. (1st edition: "Inappropriate Intimacy".)

Why it hurts. The modules are tightly coupled: you cannot change one without changing the other. Some trading is necessary; the goal is to keep it small and visible.

```
// Smelly: Invoice reaches into Customer's internal map
double discount = customer.internalLoyaltyTable.get(customer.tierCode) * 0.01;

// Cured: Customer owns its own rule, Invoice asks
double discount = customer.loyaltyDiscount();
```

Refactorings. Move Function and Move Field (reduce the need to talk), Hide Delegate (make a third module the go-between), Replace Subclass with Delegate or Replace Superclass with Delegate (when inheritance creates the intimacy).

7.2.20 Large Class

What it is. A class trying to do too much — too many fields, too many methods, too much code. Often called a **God Class**.

Why it hurts. It becomes a magnet for duplication and a bottleneck for changes (everyone edits it). It usually violates the Single Responsibility Principle.

```
// Smelly
class Employee {
    String name; String email;                    // identity
    String street, city, zip;                     // address clump
    double baseSalary, bonus, taxRate;            // payroll
    List<Review> reviews; int rating;             // performance
    ...
}

// Cured: Extract Class for each cohesive subset (look for fields with common prefixes/suffixes,
// or fields used together by the same methods)
class Employee {
    String name; String email;
    Address address;
    Compensation compensation;
    PerformanceRecord performance;
}
```

Refactorings. Extract Class, Extract Superclass, Replace Type Code with Subclasses (Extract Subclass). Tip: look at how *clients* use the class — distinct groups of clients using distinct subsets of methods suggest how to split it.

7.2.21 Alternative Classes with Different Interfaces

What it is. Two classes that do the same kind of job but have different method names or signatures, so you cannot substitute one for the other: `EmailSender.sendMail(to, body)` and `SmsGateway.dispatch(msg, phone)`.

Why it hurts. You cannot treat them polymorphically (through one shared interface, without caring which class you have), so callers write `if email ... else if sms ...`. It also often hides duplicated code.

```
// Smelly
class EmailSender { void sendMail(String to, String body) { ... } }
class SmsGateway  { void dispatch(String body, String phone) { ... } }

// Cured: rename and align signatures, then extract a common interface
interface Notifier { void send(String recipient, String message); }
class EmailNotifier implements Notifier { ... }
class SmsNotifier   implements Notifier { ... }
```

Refactorings. Change Function Declaration (make signatures match), Move Function (until the protocols match), Extract Superclass (if the implementations then share code). If you cannot edit one of the classes (third-party), wrap it in an **Adapter** (Chapter 9) — a class that converts one interface into the one you expect.

7.2.22 Data Class

What it is. A class with only fields and getters/setters (setters are methods that just overwrite a field), and no behaviour. Other classes manipulate its data in detail.

Why it hurts. The behaviour that belongs with the data lives elsewhere (usually as Feature Envy in its clients), often duplicated across several clients. Public setters also allow anyone to break the object's invariants (rules that must always hold, e.g. "balance is never negative").

Important exceptions. Immutable data carriers are fine and often good: a Java **record** that is the *result* of a computation (e.g. the output of Split Phase), a DTO (Data Transfer Object) crossing a network

boundary, or a value object. The smell is about *mutable* data classes whose logic is scattered around them.

```
// Smelly
class Account { private double balance; public double getBalance() {...} public void setBalance(double b) {...} }
// ...and in a service:
if (acc.getBalance() >= amt) acc.setBalance(acc.getBalance() - amt); else throw ...;

// Cured: Remove Setting Method + Move Function - the rule lives with the data
class Account {
    private double balance;
    void withdraw(double amount) {
        if (amount > balance) throw new IllegalStateException("insufficient funds");
        balance -= amount;
    }
}
```

Refactorings. Encapsulate Record, Remove Setting Method (for fields that should not change), Move Function and Extract Function (bring behaviour in from the clients), Split Phase (when the data class is the result of a phase — then it is fine as an immutable record). Also useful (not in Fowler's list for this smell): Encapsulate Collection for public collection fields.

7.2.23 Refused Bequest

What it is. A subclass inherits methods and data from its parent but does not want or need them — or worse, refuses the parent's *interface* (overrides methods to throw `UnsupportedOperationException`).

Why it hurts. The inheritance relationship is wrong. Refusing *implementation* is a mild smell (Fowler says it is often not worth fixing). Refusing the *interface* is serious: it breaks the **Liskov Substitution Principle** (Chapter 4: a subclass must work anywhere its parent is expected) — code that uses the parent type will fail with this subclass.

```
// Smelly: Stack extends a list type, and so inherits add(index, e), remove(index) ... which break stack rules
class Stack<T> extends ArrayList<T> {
    void push(T t) { add(t); }
    T pop() { return remove(size() - 1); }
}
// (java.util.Stack extends Vector - the JDK's own famous example of this smell.)

// Cured: Replace Superclass with Delegate - use a list, don't be one
final class Stack<T> {
    private final Deque<T> items = new ArrayDeque<>();
    void push(T t) { items.push(t); }
    T pop()        { return items.pop(); }
}
// Behaviour check: pop() on an empty stack now throws NoSuchElementException instead of
// IndexOutOfBoundsException, and ArrayDeque rejects null elements. If callers rely on either,
// keep the old behaviour (e.g. an ArrayList delegate) - a refactoring must not change it.
```

Refactorings. Push Down Method and Push Down Field (move unused parent members into a sibling that does use them, leaving the parent with only what is shared), Replace Subclass with Delegate, Replace Superclass with Delegate.

7.2.24 Comments

What it is. Comments are not bad in themselves — Fowler calls them a *sweet* smell. The smell is comments used as **deodorant**: a long comment explaining *what* a confusing block does, when the block should have been made clear instead.

Why it hurts. Comments go stale — the code changes and the comment does not. A comment that explains *what* code does is a sign the code does not explain itself.

Good comments (keep them): *why* something is done a particular way, a warning, a reference to an algorithm or bug ticket, a note about something you are unsure of, public API documentation (Javadoc).

```java
// Smelly
// check if the customer is eligible for the senior discount
if (customer.getAge() >= 60 && customer.getMembershipYears() > 2 && !customer.isBlocked()) { ... }

// Cured: Extract Function - the name replaces the comment
if (isEligibleForSeniorDiscount(customer)) { ... }

// Smelly: comment states an assumption
// rate must never be negative here
// Cured: Introduce Assertion
assert rate >= 0 : "rate must not be negative";
```

Refactorings. Extract Function (comment explains a block), Change Function Declaration (comment explains what a function does — rename it), Introduce Assertion (comment states a required condition).

1st-edition smell names (renamed or dropped in the 2nd edition)

refactoring.guru still uses the 1st-edition (1999) names. You may hear them in interviews:

1st-edition smell	Meaning	Where it went / fix
Long Method	Method too long to understand	Renamed **Long Function**
Switch Statements	Any complex switch	Narrowed to **Repeated Switches**
Lazy Class	Class that does too little	Generalised to **Lazy Element**
Inappropriate Intimacy	Classes too involved in each other's privates	Renamed **Insider Trading**
Parallel Inheritance Hierarchies	Every time you add a subclass in one hierarchy you must add one in another	A special case of Shotgun Surgery; fix with Move Function / Move Field so one hierarchy disappears
Incomplete Library Class	A library lacks a method you need and you cannot change it	Add a wrapper or utility (Introduce Foreign Method / Introduce Local Extension in the 1st ed.), or an Adapter
Dead Code	Unused variables, parameters, methods, classes	The 2nd ed. treats it as the refactoring **Remove Dead Code**

7.3 The refactoring catalogue

What it is

A **named refactoring** is a small, well-defined transformation with a name, a *motivation* (when and why to use it), *mechanics* (the safe steps), and examples. Fowler's 2nd edition catalogues 61 of them in seven chapters (Chapters 6–12 of the book). The same catalogue, with a few extra entries, is online at refactoring.com/catalog. Many 1st-edition names (1999) were changed to be language-neutral ("function" instead of "method") — the old names are shown as *aliases*.

Many refactorings come in **inverse pairs** (Extract Function ↔ Inline Function). There is no "right direction" — you move whichever way makes the code clearer *for the change you are about to make*.

How to read each entry

- **Motivation** — when and why.
- **Before → After** — a short Java sketch. `// ...` marks irrelevant code. Sketches are illustrative fragments; the fully compiling examples are in `code/` (7.5, 7.6, 7.8).
- **Inverse** — the opposite refactoring, if one exists.

Catalogue at a glance

Book chapter	Refactorings
6 — A First Set	Extract Function, Inline Function, Extract Variable, Inline Variable, Change Function Declaration, Encapsulate Variable, Rename Variable, Introduce Parameter Object, Combine Functions into Class, Combine Functions into Transform, Split Phase
7 — Encapsulation	Encapsulate Record, Encapsulate Collection, Replace Primitive with Object, Replace Temp with Query, Extract Class, Inline Class, Hide Delegate, Remove Middle Man, Substitute Algorithm
8 — Moving Features	Move Function, Move Field, Move Statements into Function, Move Statements to Callers, Replace Inline Code with Function Call, Slide Statements, Split Loop, Replace Loop with Pipeline, Remove Dead Code
9 — Organizing Data	Split Variable, Rename Field, Replace Derived Variable with Query, Change Reference to Value, Change Value to Reference
10 — Simplifying Conditional Logic	Decompose Conditional, Consolidate Conditional Expression, Replace Nested Conditional with Guard Clauses, Replace Conditional with Polymorphism, Introduce Special Case, Introduce Assertion
11 — Refactoring APIs	Separate Query from Modifier, Parameterize Function, Remove Flag Argument, Preserve Whole Object, Replace Parameter with Query, Replace Query with Parameter, Remove Setting Method, Replace Constructor with Factory Function, Replace Function with Command, Replace Command with Function
12 — Dealing with Inheritance	Pull Up Method, Pull Up Field, Pull Up Constructor Body, Push Down Method, Push Down Field, Replace Type Code with Subclasses, Remove Subclass, Extract Superclass, Collapse Hierarchy, Replace Subclass with Delegate, Replace Superclass with Delegate
refactoring.com extras (web edition)	Replace Control Flag with Break, Replace Error Code with Exception, Replace Exception with Precheck, Replace Magic Literal, Return Modified Value

11 + 9 + 9 + 5 + 6 + 10 + 11 = **61** book refactorings.

7.3.1 A First Set of Refactorings (book chapter 6)

These are the refactorings you will use most — every day.

Extract Function *(alias: Extract Method)* · inverse: Inline Function

Motivation. Separate *intention* from *implementation*. If you have to spend effort looking at a fragment of code to work out *what* it does, extract it into a function named after that "what". Even a one-line function is worth it if the name says more than the code. Short functions are cheap in Java (the JIT inlines them).

```
// Before
void printOwing(Invoice invoice) {
    double outstanding = invoice.outstanding();
    System.out.println("name: " + invoice.customer());
    System.out.println("amount: " + outstanding);
}
// After
void printOwing(Invoice invoice) {
    printDetails(invoice, invoice.outstanding());
}
private void printDetails(Invoice invoice, double outstanding) {
    System.out.println("name: " + invoice.customer());
    System.out.println("amount: " + outstanding);
}
```

Inline Function *(alias: Inline Method)* · inverse: Extract Function

Motivation. When a function's body is as clear as its name, or when you have a tangle of badly factored small functions, put them back together — then extract again in better places. Also useful to remove needless indirection (a function that only delegates).

```
// Before
int rating(Driver d) { return moreThanFiveLateDeliveries(d) ? 2 : 1; }
boolean moreThanFiveLateDeliveries(Driver d) { return d.lateDeliveries() > 5; }
// After
int rating(Driver d) { return d.lateDeliveries() > 5 ? 2 : 1; }
```

Extract Variable ***(alias: Introduce Explaining Variable)*** **· inverse: Inline Variable**

Motivation. Complex expressions are hard to read. Name the parts with local variables. If the name makes sense beyond this function, prefer Extract Function (or Replace Temp with Query) instead so others can reuse it.

```
// Before
return order.quantity() * order.itemPrice()
     - Math.max(0, order.quantity() - 500) * order.itemPrice() * 0.05
     + Math.min(order.quantity() * order.itemPrice() * 0.1, 100);
// After
double basePrice = order.quantity() * order.itemPrice();
double quantityDiscount = Math.max(0, order.quantity() - 500) * order.itemPrice() * 0.05;
double shipping = Math.min(basePrice * 0.1, 100);
return basePrice - quantityDiscount + shipping;
```

Inline Variable ***(alias: Inline Temp)*** **· inverse: Extract Variable**

Motivation. A variable whose name says no more than the expression, or which gets in the way of another refactoring (e.g. Extract Function), should be removed.

```
// Before
double basePrice = order.basePrice();
return basePrice > 1000;
// After
return order.basePrice() > 1000;
```

Change Function Declaration ***(aliases: Rename Function, Rename Method, Add Parameter, Remove Parameter, Change Signature)***

Motivation. Function names and parameters are the joints of the system; bad ones make everything harder. Rename a function the moment you understand it better; add or remove parameters to change what the function depends on. Two ways: the **simple mechanics** (change the declaration and all callers at once — fine when you can see all callers, e.g. with the IDE) and the **migration mechanics** (create the new function, make the old one delegate to it and mark it `@Deprecated`, migrate callers gradually, then delete the old one — needed for published APIs).

```
// Before
double circum(double radius) { return 2 * Math.PI * radius; }
// After (migration mechanics)
double circumference(double radius) { return 2 * Math.PI * radius; }
@Deprecated(forRemoval = true)
double circum(double radius) { return circumference(radius); }   // delete when no callers remain
```

Encapsulate Variable ***(aliases: Encapsulate Field, Self-Encapsulate Field)***

Motivation. Data is harder to refactor than functions, because every reference must change at once. Wrapping widely accessed data in accessor functions (getter/setter methods) turns "reorganise data" into "reorganise functions" and gives you one place to add validation, logging or change notifications. Always the first step for Global Data.

```
// Before
public static Owner defaultOwner = new Owner("Martin", "Fowler");
// After
private static Owner defaultOwner = new Owner("Martin", "Fowler");
public static Owner defaultOwner() { return defaultOwner; }
public static void setDefaultOwner(Owner owner) { defaultOwner = owner; }   // add checks later, as a separate change
```

Rename Variable

Motivation. Good names are the heart of clear code. The wider the scope of a variable, the more its name matters. A short name is fine in a three-line lambda; a field needs a descriptive name.

```
// Before
double a = height * width;
// After
double area = height * width;
```

Introduce Parameter Object

Motivation. A group of parameters that always travel together (a Data Clump) becomes one object. Signatures shrink, the relationship between the values is made explicit, and behaviour about them now has a home — often the start of a useful domain class.

```
// Before
List<Reading> readingsOutsideRange(Station s, int min, int max) { ... }
// After
record NumberRange(int min, int max) {
    boolean contains(int v) { return v >= min && v <= max; }
}
List<Reading> readingsOutsideRange(Station s, NumberRange range) { ... }
```

Combine Functions into Class

Motivation. When several functions work on the same data (often passed as the same argument to all of them), create a class that holds the data as fields and turns the functions into methods. The shared data and the common environment become explicit, and clients no longer pass the data around.

```
// Before
double base(Reading r) { ... }
double taxableCharge(Reading r) { ... }
double calculateBaseCharge(Reading r) { ... }
// After
final class ReadingCalculator {
    private final Reading reading;
    ReadingCalculator(Reading reading) { this.reading = reading; }
    double base() { ... }
    double taxableCharge() { ... }
}
```

Combine Functions into Transform

Motivation. An alternative to Combine Functions into Class when the source data is read-only: a *transform* function takes the raw data and returns an enriched copy with all derived values computed. All derivation logic lives in one place. (Use a class instead if the source data can change, since a transform's copy would go stale.)

```
// Before (derived values computed in several places)
double base = baseRate(r.month(), r.year()) * r.quantity();
double taxable = Math.max(0, base - taxThreshold(r.year()));
// After
record EnrichedReading(Reading raw, double baseCharge, double taxableCharge) { }
static EnrichedReading enrichReading(Reading r) {
    double base = baseRate(r.month(), r.year()) * r.quantity();
    return new EnrichedReading(r, base, Math.max(0, base - taxThreshold(r.year())));
}
```

Split Phase

Motivation. When code deals with two different things in sequence (e.g. parse input, then compute; or compute, then format), split it into two phases connected by an intermediate data structure. Each phase can then be understood and changed separately — a classic cure for Divergent Change. Compilers are the textbook example (tokenise — split text into words and symbols → parse — build a tree of its structure → generate code). See the worked example, step 3.

```
// Before
String priceOrder(String orderLine) {
    String[] parts = orderLine.split("\\s+");
    Product p = catalog.get(parts[0].split("-")[1]);
    int qty = Integer.parseInt(parts[1]);
    return "Total: " + qty * p.price();
}
// After
record OrderRequest(String productId, int quantity) { }           // the intermediate data structure
OrderRequest parse(String line) {                                  // phase 1
    String[] parts = line.split("\\s+");
    return new OrderRequest(parts[0].split("-")[1], Integer.parseInt(parts[1]));
}
String price(OrderRequest req) {                                   // phase 2
    return "Total: " + req.quantity() * catalog.get(req.productId()).price();
}
String priceOrder(String orderLine) { return price(parse(orderLine)); }
```

7.3.2 Encapsulation (book chapter 7)

The key to modular design is deciding what each module *hides*. These refactorings hide data structures, temporary variables, and relationships between classes.

Encapsulate Record ***(alias: Replace Record with Data Class)***

Motivation. A raw record-like structure (a `Map<String,Object>`, an array, a struct with public fields) forces clients to know its layout and lets anyone change it. Replace it with a class (or a Java `record`) whose accessors hide the storage and can compute derived values.

```
// Before
Map<String, Object> organization = Map.of("name", "Acme", "country", "GB");
String name = (String) organization.get("name");
// After
record Organization(String name, String country) { }
Organization organization = new Organization("Acme", "GB");
String name = organization.name();
```

Encapsulate Collection

Motivation. If a getter returns the internal collection, clients can modify it without the owner knowing. Offer `add`/`remove` methods on the owner, and have the getter return a read-only view or a copy.

```
// Before
class Person {
    private List<Course> courses = new ArrayList<>();
    List<Course> getCourses() { return courses; }                  // caller can add/clear!
    void setCourses(List<Course> c) { courses = c; }
}
// After
class Person {
    private final List<Course> courses = new ArrayList<>();
    List<Course> courses()        { return Collections.unmodifiableList(courses); }
    void addCourse(Course c)      { courses.add(c); }
    void removeCourse(Course c)   { courses.remove(c); }
}
```

Replace Primitive with Object ***(aliases: Replace Data Value with Object, Replace Type Code with Class)***

Motivation. A simple value (a phone number string, a priority `"high"`) soon needs behaviour: validation, formatting, comparison. Wrap it in its own type as soon as you see that. A cure for Primitive Obsession.

```
// Before
List<Order> urgent = orders.stream().filter(o -> o.priority().equals("high") ||
o.priority().equals("rush")).toList();
// After
enum Priority { LOW, NORMAL, HIGH, RUSH;
    boolean higherThan(Priority other) { return compareTo(other) > 0; } }
List<Order> urgent = orders.stream().filter(o -> o.priority().higherThan(Priority.NORMAL)).toList();
```

Replace Temp with Query

Motivation. A temporary variable holding a computed value can only be used inside its function. Turn the computation into a method, and every other method of the class can use it too; it also makes Extract Function easier (fewer locals to pass). Works best when the temp is assigned once and its computation has no side effects.

```
// Before
double price() {
    double basePrice = quantity * itemPrice;
    return basePrice > 1000 ? basePrice * 0.95 : basePrice * 0.98;
}
// After
double price()     { return basePrice() > 1000 ? basePrice() * 0.95 : basePrice() * 0.98; }
double basePrice() { return quantity * itemPrice; }
```

Extract Class · inverse: Inline Class

Motivation. Classes grow. When a subset of fields and methods clearly go together (and change together), move them to a new class. A cure for Large Class and Divergent Change.

```
// Before
class Person {
    String name; String officeAreaCode; String officeNumber;
    String telephoneNumber() { return "(" + officeAreaCode + ") " + officeNumber; }
}
// After
class Person {
    String name; TelephoneNumber officeTelephone;
    String telephoneNumber() { return officeTelephone.toString(); }
}
record TelephoneNumber(String areaCode, String number) {
    @Override public String toString() { return "(" + areaCode + ") " + number; }
}
```

Inline Class · inverse: Extract Class

Motivation. A class that no longer does enough (often after its responsibilities moved away) should be folded into its main user. Also a useful first step when re-splitting two classes differently: merge them, then extract again.

```
// Before
class Shipment { TrackingInformation trackingInfo; }
class TrackingInformation { String shippingCompany; String trackingNumber; }
// After
class Shipment { String shippingCompany; String trackingNumber; }
```

Hide Delegate · inverse: Remove Middle Man

Motivation. If a client calls `a.getB().doSomething()`, it depends on `B`. Adding a delegating method `a.doSomething()` hides `B`; if `B` changes, only `A` changes. A cure for Message Chains and the Law of Demeter.

```
// Before
Person manager = person.getDepartment().getManager();
// After
class Person { Person manager() { return department.getManager(); } }
Person manager = person.manager();
```

Remove Middle Man · inverse: Hide Delegate

Motivation. When a class has so many delegating methods that it is just a pass-through, let clients call the delegate directly. The right amount of hiding changes as the system changes.

```
// Before
Person manager = person.manager();              // manager() only forwards
// After
Person manager = person.department().getManager();
```

Substitute Algorithm

Motivation. If you find a clearer way to do something, replace the whole body with the clearer algorithm (after decomposing the old one as far as possible, and with tests proving they agree). Also used to replace hand-written code with a library call.

```
// Before
String foundPerson(List<String> people) {
    for (String p : people) {
        if (p.equals("Don")) return "Don";
        if (p.equals("John")) return "John";
        if (p.equals("Kent")) return "Kent";
    }
    return "";
}
// After
String foundPerson(List<String> people) {
    Set<String> candidates = Set.of("Don", "John", "Kent");
    return people.stream().filter(candidates::contains).findFirst().orElse("");
}
```

7.3.3 Moving Features (book chapter 8)

These refactorings move code between contexts — between classes, between functions, and around inside a function.

Move Function ***(alias: Move Method)***

Motivation. Put a function in the context (class/module) whose data and functions it references most. A cure for Feature Envy and Shotgun Surgery. Good modules keep related things together.

```
// Before
class Account {
    double overdraftCharge() {
        if (type.isPremium()) { double base = 10; return daysOverdrawn > 7 ? base + (daysOverdrawn - 7) * 0.85 :
base; }
        return daysOverdrawn * 1.75;
    }
}
// After - the rule depends mostly on AccountType, so it moves there
class AccountType {
    double overdraftCharge(int daysOverdrawn) { /* same logic, using isPremium() */ }
}
class Account { double overdraftCharge() { return type.overdraftCharge(daysOverdrawn); } }
```

Move Field

Motivation. Data structures are the backbone of a program. If you always pass a field of A alongside B, or a change in B forces a change to A's field, the field probably belongs in B. Encapsulate it first (Encapsulate Variable) to make the move easy.

```
// Before
class Customer { Plan plan; double discountRate; }
class Plan { }
// After - the discount rate is a property of the plan, not of each customer
class Customer { Plan plan; double discountRate() { return plan.discountRate(); } }
class Plan { double discountRate; double discountRate() { return discountRate; } }
```

Move Statements into Function · inverse: Move Statements to Callers

Motivation. If the same statements run every time a function is called, move them into the function, removing duplication at every call site.

```
// Before
result.add("<p>title: " + photo.title() + "</p>");
result.addAll(photoData(photo));
List<String> photoData(Photo p) { return List.of("<p>location: " + p.location() + "</p>"); }
// After
result.addAll(photoData(photo));
List<String> photoData(Photo p) {
    return List.of("<p>title: " + p.title() + "</p>", "<p>location: " + p.location() + "</p>");
}
```

Move Statements to Callers · inverse: Move Statements into Function

Motivation. When a function that was shared starts needing *different* behaviour for different callers, move the varying statements out to the callers.

```
// Before
void emitPhotoData(Writer out, Photo p) { out.write(p.title()); out.write(p.location()); }
// After - the location line moves out to every caller; behaviour is still identical
void emitPhotoData(Writer out, Photo p) { out.write(p.title()); }
// caller A:
emitPhotoData(out, p); out.write(p.location());
// caller B:
emitPhotoData(out, p); out.write(p.location());
// Now caller B can change its own line later (e.g. hide the location) - as a separate, deliberate behaviour
change.
```

Replace Inline Code with Function Call

Motivation. If code does what an existing function (yours or a library's) already does, call the function. You gain a name and one place to fix bugs.

```
// Before
boolean appliesToMass = false;
for (String s : states) { if (s.equals("MA")) appliesToMass = true; }
// After
boolean appliesToMass = states.contains("MA");
```

Slide Statements *(alias: Consolidate Duplicate Conditional Fragments)*

Motivation. Code is easier to understand when related statements are next to each other — e.g. declare a variable right before its first use. Sliding is often a preparation for Extract Function. The 1st-edition alias covers one case: a statement repeated in every branch of a conditional can slide out of the conditional. Be careful: you cannot slide past code that the statement interferes with (shared variables, side effects).

```
// Before
String pricingPlan = retrievePricingPlan();
Order order = retrieveOrder();
double baseCharge = pricingPlan.base();
// After - declare each value next to its use
Order order = retrieveOrder();
String pricingPlan = retrievePricingPlan();
double baseCharge = pricingPlan.base();
```

Split Loop

Motivation. A loop that does two things forces you to understand both whenever you change either. Split it into two loops, each doing one thing; then each can be extracted or turned into a pipeline. Worried about performance? Iterating twice is rarely a bottleneck — refactor first, optimise later if the profiler says so.

```
// Before
int youngest = Integer.MAX_VALUE; double totalSalary = 0;
for (Person p : people) {
    if (p.age() < youngest) youngest = p.age();
    totalSalary += p.salary();
}
// After
double totalSalary = 0;
for (Person p : people) totalSalary += p.salary();
int youngest = Integer.MAX_VALUE;
for (Person p : people) if (p.age() < youngest) youngest = p.age();
```

Replace Loop with Pipeline

Motivation. Collection pipelines (`filter`, `map`, `reduce`) describe the processing as a series of named operations, which is easier to read than a loop. See the Loops smell.

```
// Before
List<String> result = new ArrayList<>();
for (String line : lines) {
    if (line.isBlank() || line.startsWith("#")) continue;
    result.add(line.split(",")[1].trim());
}
// After
List<String> result = lines.stream()
        .filter(l -> !l.isBlank() && !l.startsWith("#"))
        .map(l -> l.split(",")[1].trim())
        .toList();
```

Remove Dead Code

Motivation. Unused code still costs readers time ("is this called somewhere?"). Delete it — version control remembers it if you ever need it back. Do not comment it out.

```
// Before
if (false) { legacyDiscount(); }
private void legacyDiscount() { /* unused since 2019 */ }
// After
// (gone)
```

7.3.4 Organizing Data (book chapter 9)

Data structures play an important role; these refactorings improve how data is named and shared.

Split Variable *(aliases: Remove Assignments to Parameters, Split Temp)*

Motivation. A variable (other than a loop counter or an accumulator) assigned more than once usually has more than one responsibility. Give each responsibility its own variable (ideally `final`). The 1st-ed "Remove Assignments to Parameters" is a special case: do not reuse a parameter as a local.

```
// Before
double temp = 2 * (height + width);
System.out.println(temp);
temp = height * width;
System.out.println(temp);
// After
final double perimeter = 2 * (height + width);
System.out.println(perimeter);
final double area = height * width;
System.out.println(area);
```

Rename Field

Motivation. Field names explain the data structure; they deserve as much care as function names. Encapsulate first if the field is widely used, then rename. (For records, the accessor name changes too.)

```
// Before
record Organization(String nm, String country) { }
// After
record Organization(String name, String country) { }
```

Replace Derived Variable with Query

Motivation. Mutable data that is *computed from* other data can drift out of sync. Compute it on demand instead. Exception: if the source data is immutable, a stored derived value is safe (e.g. a record computed once).

```
// Before
class ProductionPlan {
    private double production;
    private final List<Adjustment> adjustments = new ArrayList<>();
    void apply(Adjustment a) { adjustments.add(a); production += a.amount(); }  // two updates to keep in sync
    double production() { return production; }
}
// After
class ProductionPlan {
    private final List<Adjustment> adjustments = new ArrayList<>();
    void apply(Adjustment a) { adjustments.add(a); }
    double production() { return adjustments.stream().mapToDouble(Adjustment::amount).sum(); }
}
```

Change Reference to Value · inverse: Change Value to Reference

Motivation. If an inner object does not need identity (being tracked as one specific object) (two telephone numbers with the same digits are "the same"), make it an immutable **value object**: replace it wholesale instead of mutating it, and compare by value (`equals`/`hashCode`). Values are safe to share and to use across threads. Java `record`s are ideal.

```
// Before
class TelephoneNumber { String areaCode; String number; void setAreaCode(String a) { areaCode = a; } }
person.telephone().setAreaCode("020");
// After
record TelephoneNumber(String areaCode, String number) { }                  // immutable, equals by value
person.setTelephone(new TelephoneNumber("020", person.telephone().number()));
```

Change Value to Reference · inverse: Change Reference to Value

Motivation. If many objects hold *copies* of the same logical entity (every order holds its own `Customer` copy) and that entity needs to be updated, a change to one copy is not seen by the others. Share **one** reference instead, usually via a **repository** (an object that stores domain objects and looks them up, usually by ID) that hands out the single instance per ID.

```
// Before
Order(String customerId) { this.customer = new Customer(customerId); }      // each order its own copy
// After
Order(String customerId, CustomerRepository repo) { this.customer = repo.findOrCreate(customerId); }
```

7.3.5 Simplifying Conditional Logic (book chapter 10)

Conditionals are where much of a program's complexity hides.

Decompose Conditional

Motivation. A complex `if`/`else` mixes *why* (the condition) and *what* (each branch). Extract the condition and each branch into well-named functions. A special case of Extract Function.

```
// Before
if (!date.isBefore(plan.summerStart()) && !date.isAfter(plan.summerEnd()))
    charge = quantity * plan.summerRate();
else
    charge = quantity * plan.regularRate() + plan.regularServiceCharge();
// After
charge = isSummer(date) ? summerCharge() : regularCharge();
```

Consolidate Conditional Expression

Motivation. Several checks that all lead to the same result are really one condition. Combine them with `&&`/`||` and extract the result into a named function. (Only when the checks have no side effects and truly form one decision.)

```
// Before
if (employee.seniority() < 2) return 0;
if (employee.monthsDisabled() > 12) return 0;
if (employee.isPartTime()) return 0;
// After
if (isNotEligibleForDisability(employee)) return 0;
boolean isNotEligibleForDisability(Employee e) {
    return e.seniority() < 2 || e.monthsDisabled() > 12 || e.isPartTime();
}
```

Replace Nested Conditional with Guard Clauses

Motivation. When one branch is the normal case and the others are unusual, check the unusual cases first and return immediately (**guard clauses**). The normal path then reads straight down, without deep nesting. ("One exit point" is not a rule worth keeping if it hurts clarity.)

```
// Before
double payAmount(Employee e) {
    double result;
    if (e.isSeparated()) result = 0;
    else { if (e.isRetired()) result = 0; else { result = normalPay(e); } }
    return result;
}
// After
double payAmount(Employee e) {
    if (e.isSeparated()) return 0;
    if (e.isRetired()) return 0;
    return normalPay(e);
}
```

Replace Conditional with Polymorphism

Motivation. When a switch on a type appears in several places (Repeated Switches), or when there is a base behaviour with variants, create a class per case and let dynamic dispatch (the runtime choosing which class's method to run from the object's actual type) pick the behaviour. Adding a new case means adding a class, not editing every switch. This is how you arrive at **Strategy** or **State** (7.6) — State

gives each state of an object its own class, which supplies the behaviour for that state. The worked example's step 5 does exactly this.

```
// Before
double speed(Bird b) {
    return switch (b.type()) {
        case "EUROPEAN" -> 35;
        case "AFRICAN"  -> 40 - 2 * b.numberOfCoconuts();
        case "PARROT"   -> b.isNailed() ? 0 : 10 + b.voltage() / 10.0;
        default -> throw new IllegalArgumentException(b.type());
    };
}
// After
sealed interface Bird permits EuropeanSwallow, AfricanSwallow, NorwegianBlueParrot { double speed(); }
record EuropeanSwallow() implements Bird { public double speed() { return 35; } }
record AfricanSwallow(int numberOfCoconuts) implements Bird { public double speed() { return 40 - 2 *
numberOfCoconuts; } }
record NorwegianBlueParrot(boolean isNailed, int voltage) implements Bird {
    public double speed() { return isNailed ? 0 : 10 + voltage / 10.0; }
}
```

Introduce Special Case ***(alias: Introduce Null Object)***

Motivation. Many clients check for the same special value (usually `null`, or an "unknown customer") and do the same thing. Create a **special-case object** that answers those questions itself. The **Null Object** pattern is the most common form. See 7.6 for a runnable example.

```
// Before
String name = (customer == null) ? "occupant" : customer.name();
Plan plan  = (customer == null) ? Plan.BASIC : customer.plan();
// After
final class UnknownCustomer implements Customer {
    public String name() { return "occupant"; }
    public Plan plan()   { return Plan.BASIC; }
}
String name = customer.name();          // repository returns UnknownCustomer instead of null
```

(Java's `Optional<T>` is a related tool: it makes "may be absent" visible in the type, but callers still decide what to do. A special case object *centralises* that decision.)

Introduce Assertion

Motivation. Some code only works if a condition holds (e.g. a discount rate is never negative). Make the assumption explicit with an assertion. An assertion documents the assumption and catches programmer errors early; the program must be correct *without* it. In Java, `assert` is disabled by default (enable with `java -ea`), so do not use it to validate external input — use exceptions for that.

```
// Before
double applyDiscount(double amount) {
    // discountRate should never be negative here
    return amount - discountRate * amount;
}
// After
double applyDiscount(double amount) {
    assert discountRate >= 0 : "discountRate must not be negative";
    return amount - discountRate * amount;
}
```

7.3.6 Refactoring APIs (book chapter 11)

Modules and their functions are the building blocks; APIs are the joints. These make APIs easier to understand and use.

Separate Query from Modifier

Motivation. A function that returns a value should have no observable side effects — then you can call it anywhere, as often as you like, safely. This is **Command–Query Separation** (Bertrand Meyer). If a function both returns something and changes state, split it into a query and a modifier.

```
// Before
String alertForMiscreant(List<String> people) {
    for (String p : people) {
        if (p.equals("Don") || p.equals("John")) { setOffAlarms(); return p; }
    }
    return "";
}
// After
String findMiscreant(List<String> people) {                    // query - no side effects
    return people.stream().filter(p -> p.equals("Don") || p.equals("John")).findFirst().orElse("");
}
void alertForMiscreant(List<String> people) {                   // modifier
    if (!findMiscreant(people).isEmpty()) setOffAlarms();
}
```

Parameterize Function ***(alias: Parameterize Method)***

Motivation. Several functions with very similar logic that differ only in some literal values can become one function with a parameter.

```
// Before
void tenPercentRaise(Person p)  { p.setSalary(p.salary() * 1.10); }
void fivePercentRaise(Person p) { p.setSalary(p.salary() * 1.05); }
// After
void raise(Person p, double factor) { p.setSalary(p.salary() * (1 + factor)); }
```

Remove Flag Argument ***(alias: Replace Parameter with Explicit Methods)***

Motivation. A **flag argument** (a boolean or enum literal that the caller passes to choose which code path runs) makes calls unreadable: what does `book(customer, true)` mean? Provide an explicit function per path.

```
// Before
void setDimension(String name, double value) { if (name.equals("height")) height = value; else if
(name.equals("width")) width = value; }
LocalDate deliveryDate(Order o, boolean isRush) { ... }
deliveryDate(order, true);
// After
void setHeight(double value) { height = value; }
void setWidth(double value)  { width = value; }
LocalDate rushDeliveryDate(Order o)    { ... }
LocalDate regularDeliveryDate(Order o) { ... }
```

Preserve Whole Object

Motivation. If you pull several values out of one object only to pass them to a function, pass the whole object. The function can then take whatever it needs, and signatures stay stable when needs change. (Don't do it if you want to keep the function independent of that object's type.)

```
// Before
int low = room.daysTempRange().low();
int high = room.daysTempRange().high();
if (!plan.withinRange(low, high)) alerts.add("room temperature out of range");
// After
if (!plan.withinRange(room.daysTempRange())) alerts.add("room temperature out of range");
```

Replace Parameter with Query ***(alias: Replace Parameter with Method)*** **· inverse: Replace Query with Parameter**

Motivation. If the function can easily get a value itself (from its own object, or from another parameter), the caller should not have to pass it. Removing the parameter simplifies every call. (Don't

do it if it would add an unwanted dependency.)

```
// Before
double finalPrice() { return discountedPrice(basePrice(), discountLevel()); }
double discountedPrice(double base, int discountLevel) { return discountLevel == 2 ? base * 0.9 : base * 0.95; }
// After
double finalPrice() { return discountedPrice(basePrice()); }
double discountedPrice(double base) { return discountLevel() == 2 ? base * 0.9 : base * 0.95; }
```

Replace Query with Parameter · inverse: Replace Parameter with Query

Motivation. If a function reaches out to a global, a singleton or a mutable field, it is harder to test and not **referentially transparent** (same inputs do not always give the same output). Pass the value in as a parameter instead — the caller takes on the dependency.

```
// Before
double targetTemperature(HeatingPlan plan) {
    double current = Thermostat.INSTANCE.currentTemperature();        // hidden global dependency
    return Math.clamp(current, plan.min(), plan.max());
}
// After
double targetTemperature(HeatingPlan plan, double currentTemperature) {
    return Math.clamp(currentTemperature, plan.min(), plan.max());      // Math.clamp: Java 21+
}
```

Remove Setting Method

Motivation. If a field should not change after construction, remove its setter and set it in the constructor (make it `final`). The class then clearly communicates immutability.

```
// Before
class Person { private String id; void setId(String id) { this.id = id; } }
// After
class Person { private final String id; Person(String id) { this.id = id; } String id() { return id; } }
```

Replace Constructor with Factory Function *(alias: Replace Constructor with Factory Method)*

Motivation. Constructors have limitations: they must return a new instance of exactly that class, their name is fixed, and `new` is needed. A static factory method (a `static` method that creates and returns an instance) can have an intention-revealing name, return a subclass or cached instance, and hide construction details. (Joshua Bloch's *Effective Java*, Item 1, makes the same argument. JDK examples: `List.of`, `Integer.valueOf`, `LocalDate.of`.)

```
// Before
Employee leadEngineer = new Employee(document.leadEngineer(), "E");
// After
static Employee createEngineer(String name) { return new Employee(name, "E"); }
Employee leadEngineer = Employee.createEngineer(document.leadEngineer());
```

Replace Function with Command *(alias: Replace Method with Method Object)* · inverse: Replace Command with Function

Motivation. Wrap a function in its own object (a **command**, with usually one `execute()` method). The object can hold state across steps, support undo, be queued, and — most commonly in refactoring — lets you break a big function with many local variables into small methods that share those variables as fields.

```
// Before
int score(Candidate c, MedicalExam exam, ScoringGuide guide) {
    int result = 0; int healthLevel = 0; boolean highMedicalRisk = false;
    // ... 60 lines using result, healthLevel, highMedicalRisk
    return result;
}
// After
final class Scorer {
    private final Candidate candidate; private final MedicalExam exam; private final ScoringGuide guide;
    private int result; private int healthLevel; private boolean highMedicalRisk;
    Scorer(Candidate c, MedicalExam e, ScoringGuide g) { candidate = c; exam = e; guide = g; }
    int execute() { scoreSmoking(); scoreCertification(); return result - Math.max(healthLevel - 5, 0); }
    private void scoreSmoking() { /* uses fields, no parameter passing */ }
    private void scoreCertification() { /* ... */ }
}
int score(Candidate c, MedicalExam exam, ScoringGuide guide) { return new Scorer(c, exam, guide).execute(); }
```

Replace Command with Function · inverse: Replace Function with Command

Motivation. If a command object is simple and adds nothing a function couldn't do, turn it back into a function. Commands are powerful but cost extra complexity.

```
// Before
final class ChargeCalculator {
    private final Customer customer; private final int usage;
    ChargeCalculator(Customer c, int usage) { this.customer = c; this.usage = usage; }
    double charge() { return customer.rate() * usage; }
}
double monthCharge = new ChargeCalculator(customer, usage).charge();
// After
static double charge(Customer customer, int usage) { return customer.rate() * usage; }
double monthCharge = charge(customer, usage);
```

7.3.7 Dealing with Inheritance (book chapter 12)

Inheritance is powerful and easy to misuse. These refactorings move features up and down a hierarchy and replace inheritance with delegation (composition — one object holding another and forwarding work to it) where needed.

Pull Up Method · inverse: Push Down Method

Motivation. Duplicate methods in sibling subclasses belong in the superclass. If they are similar but not identical, first make them identical (Parameterize Function), or use a Template Method (7.6) — a base-class method that fixes an algorithm's steps and lets subclasses fill some in.

```
// Before
class Salesman extends Employee { String name() { return name; } }
class Engineer extends Employee { String name() { return name; } }
// After
class Employee { String name() { return name; } }
```

Pull Up Field · inverse: Push Down Field

Motivation. Subclasses developed independently often have the same field (maybe with different names). Rename to match, then move the field to the superclass.

```
// Before
class Salesman extends Employee { protected String name; }
class Engineer extends Employee { protected String name; }
// After
class Employee { protected String name; }
```

Pull Up Constructor Body

Motivation. Constructors in subclasses that repeat the same initialisation should call a superclass constructor that does it. In Java 21 `super(...)` must be the first statement of a constructor (Java 25 relaxed this with flexible constructor bodies, JEP 513 — a JDK Enhancement Proposal), so the common code goes into the superclass constructor.

```
// Before
class Manager extends Employee {
    Manager(String name, String id, int grade) { this.name = name; this.id = id; this.grade = grade; }
}
// After
class Employee { Employee(String name, String id) { this.name = name; this.id = id; } }
class Manager extends Employee {
    Manager(String name, String id, int grade) { super(name, id); this.grade = grade; }
}
```

Push Down Method · inverse: Pull Up Method

Motivation. If a superclass method is only relevant to one (or a few) subclasses, move it down to them. Callers must then know they have that subclass — only do it when they do.

```
// Before
class Employee { int quota() { ... } }                    // only salesmen have a quota
// After
class Salesman extends Employee { int quota() { ... } }
```

Push Down Field · inverse: Pull Up Field

Motivation. A field used by only one subclass should live in that subclass.

```
// Before
class Employee { protected int quota; }
// After
class Salesman extends Employee { private int quota; }
```

Replace Type Code with Subclasses ***(aliases: Extract Subclass, Replace Type Code with State/Strategy)*** **· inverse: Remove Subclass**

Motivation. A type code field (`"engineer"`, `"manager"`) that drives behaviour is a hint that you need subclasses; then conditionals can become polymorphism. Two ways: **direct inheritance** (subclass the class itself) or **indirect** (the class holds a separate type object, e.g. `EmployeeType`, which is subclassed — needed if the type can change at runtime or the class is already subclassed for another reason; this is the old "Replace Type Code with State/Strategy").

```
// Before
class Employee { String type; Employee(String name, String type) { ... } }
// After (direct)
abstract sealed class Employee permits Engineer, Manager, Salesman { ... }
static Employee create(String name, String type) {
    return switch (type) {
        case "engineer" -> new Engineer(name);
        case "manager"  -> new Manager(name);
        case "salesman" -> new Salesman(name);
        default -> throw new IllegalArgumentException("Employee cannot be of type " + type);
    };
}
```

Remove Subclass ***(alias: Replace Subclass with Fields)*** **· inverse: Replace Type Code with Subclasses**

Motivation. Subclasses that no longer differ in behaviour (e.g. they differ only in a constant) are not worth their cost. Replace them with a field in the superclass.

```
// Before
abstract class Person { abstract String genderCode(); }
class Male extends Person   { String genderCode() { return "M"; } }
class Female extends Person { String genderCode() { return "F"; } }
// After
class Person {
    private final String genderCode;
    Person(String name, String genderCode) { ...; this.genderCode = genderCode; }
    String genderCode() { return genderCode; }
}
```

Extract Superclass

Motivation. Two classes doing similar things have duplicated fields and methods. Create a common superclass and pull the shared parts up. (Alternative: Extract Class and use delegation. Fowler notes you can start with inheritance and switch to delegation later with Replace Superclass with Delegate if it becomes a problem.)

```
// Before
class Department { String name; double totalAnnualCost() { ... } int headCount() { ... } }
class Employee   { String name; double annualCost() { ... } String id() { ... } }
// After
abstract class Party { protected String name; abstract double annualCost(); }
class Department extends Party { ... }
class Employee extends Party { ... }
```

Collapse Hierarchy

Motivation. When a subclass and its superclass are no longer different enough to be worth keeping separate, merge them. A cure for Lazy Element and Speculative Generality.

```
// Before
class Employee { ... }
class Salesman extends Employee { }          // adds nothing any more
// After
class Employee { ... }
```

Replace Subclass with Delegate

Motivation. Inheritance has two limits: you can vary only **one** axis (a person cannot be subclassed by both age-group and income-level), and subclasses are tightly coupled to their parent. Delegation (composition) handles both: the varying behaviour moves to a separate delegate object that can be changed at runtime. This is "favour object composition over class inheritance" (GoF — the "Gang of Four" authors of the 1994 *Design Patterns* book), and the core of the **State** and **Strategy** patterns.

```
// Before
class Booking { double basePrice() { return show.price(); } }
class PremiumBooking extends Booking { @Override double basePrice() { return super.basePrice() +
extras.premiumFee(); } }
// After
class Booking {
    private PremiumDelegate premiumDelegate;           // null unless premium - can be set at runtime
    void bePremium(Extras extras) { premiumDelegate = new PremiumDelegate(this, extras); }
    double basePrice() {
        double result = show.price();
        return premiumDelegate == null ? result : premiumDelegate.extendBasePrice(result);
    }
}
```

Replace Superclass with Delegate ***(alias: Replace Inheritance with Delegation)***

Motivation. If a subclass uses only part of its superclass, or the superclass's methods do not make sense for it (Refused Bequest, broken Liskov substitution), stop inheriting and hold an instance instead, forwarding only what you need. Classic example: a `Stack` that extends a list type (the JDK's `java.util.Stack extends Vector` is often cited).

```
// Before
class Scroll extends CatalogItem { ... }     // but a scroll is a physical copy, not a catalogue entry
// After
class Scroll {
    private final CatalogItem catalogItem;   // delegate
    String title() { return catalogItem.title(); }
    boolean hasTag(String tag) { return catalogItem.hasTag(tag); }
    LocalDate lastCleaned;                   // scroll's own data
}
```

7.3.8 Extra refactorings on refactoring.com

The online catalogue (web edition) lists a few more refactorings that are not among the 61 book chapters' entries. They are worth knowing:

Replace Control Flag with Break ***(alias: Remove Control Flag)***

Motivation. A boolean "found" flag that only exists to stop a loop is clumsy. Use `break`, `continue` or `return`.

```
// Before
boolean found = false;
for (String p : people) {
    if (!found && p.equals("Don")) { sendAlert(); found = true; }
}
// After
for (String p : people) {
    if (p.equals("Don")) { sendAlert(); break; }
}
```

Replace Error Code with Exception

Motivation. Returning special codes (`-1`, `null`) for errors makes every caller check them — and callers forget. Throw an exception so errors cannot be silently ignored and the normal path stays clean.

```
// Before
int withdraw(double amount) { if (amount > balance) return -1; balance -= amount; return 0; }
// After
void withdraw(double amount) {
    if (amount > balance) throw new InsufficientFundsException(amount, balance);
    balance -= amount;
}
```

Replace Exception with Precheck ***(alias: Replace Exception with Test)***

Motivation. Exceptions are for *exceptional* behaviour. If the caller can easily check the condition first, check it instead of catching an exception.

```
// Before
double valueAt(int i) { try { return values[i]; } catch (ArrayIndexOutOfBoundsException e) { return 0; } }
// After
double valueAt(int i) { return (i >= 0 && i < values.length) ? values[i] : 0; }
```

Replace Magic Literal ***(alias: Replace Magic Number with Symbolic Constant)***

Motivation. A literal like `9.81` or `"STD"` with special meaning should get a name, so readers know what it means and there is one place to change it. (Don't bother when the meaning is obvious, e.g. `0` as an initial sum.) The worked example's step 4 does this.

```
// Before
double potentialEnergy(double mass, double height) { return mass * 9.81 * height; }
// After
static final double STANDARD_GRAVITY = 9.81;
double potentialEnergy(double mass, double height) { return mass * STANDARD_GRAVITY * height; }
```

Return Modified Value

Motivation. When a function modifies a variable passed in or shared, make it *return* the new value instead and let the caller assign it. The data flow becomes visible.

```
// Before
double totalAscent = 0;
calculateAscent();                      // secretly updates the field totalAscent
// After
double totalAscent = calculateAscent();
double calculateAscent() {
    double result = 0;
    for (int i = 1; i < points.size(); i++) {
        double diff = points.get(i).elevation() - points.get(i - 1).elevation();
        result += Math.max(diff, 0);
    }
    return result;
}
```

1st-edition names you may still hear

1st-edition name	2nd-edition name
Extract Method / Inline Method	Extract Function / Inline Function
Introduce Explaining Variable / Inline Temp	Extract Variable / Inline Variable
Rename Method, Add Parameter, Remove Parameter	Change Function Declaration
Self-Encapsulate Field / Encapsulate Field	Encapsulate Variable
Replace Record with Data Class	Encapsulate Record
Replace Data Value with Object, Replace Type Code with Class	Replace Primitive with Object
Move Method	Move Function
Consolidate Duplicate Conditional Fragments	Slide Statements
Split Temp, Remove Assignments to Parameters	Split Variable
Introduce Null Object	Introduce Special Case
Parameterize Method	Parameterize Function
Replace Parameter with Explicit Methods	Remove Flag Argument
Replace Parameter with Method	Replace Parameter with Query
Replace Constructor with Factory Method	Replace Constructor with Factory Function
Replace Method with Method Object	Replace Function with Command
Extract Subclass, Replace Type Code with State/Strategy	Replace Type Code with Subclasses
Replace Subclass with Fields	Remove Subclass
Replace Inheritance with Delegation	Replace Superclass with Delegate
Remove Control Flag	Replace Control Flag with Break (web)
Replace Magic Number with Symbolic Constant	Replace Magic Literal (web)
Replace Exception with Test	Replace Exception with Precheck (web)

7.4 Smell → refactorings lookup table

Use this when you have named a smell and need to choose a cure. **Bold** = the usual first move.

Smell	Group	Refactorings that cure it
Mysterious Name	2nd-ed extra	**Change Function Declaration**, Rename Variable, Rename Field
Duplicated Code	Dispensables	**Extract Function**, Slide Statements, Pull Up Method *(also useful, not in Fowler's list: Replace Inline Code with Function Call, Substitute Algorithm)*
Long Function	Bloaters	**Extract Function**, Replace Temp with Query, Introduce Parameter Object, Preserve Whole Object, Replace Function with Command, Decompose Conditional, Replace Conditional with Polymorphism, Split Loop
Long Parameter List	Bloaters	Replace Parameter with Query, **Preserve Whole Object**, **Introduce Parameter Object**, Remove Flag Argument, Combine Functions into Class
Global Data	2nd-ed extra	**Encapsulate Variable** (then shrink its scope)
Mutable Data	2nd-ed extra	**Encapsulate Variable**, Split Variable, Slide Statements, Extract Function, Separate Query from Modifier, Remove Setting Method, Replace Derived Variable with Query, Combine Functions into Class, Combine Functions into Transform, Change Reference to Value
Divergent Change	Change Preventers	**Split Phase**, Move Function, Extract Function, **Extract Class**
Shotgun Surgery	Change Preventers	**Move Function**, **Move Field**, Combine Functions into Class, Combine Functions into Transform, Split Phase, Inline Function, Inline Class
Feature Envy	Couplers	**Move Function**, Extract Function
Data Clumps	Bloaters	**Extract Class**, **Introduce Parameter Object**, Preserve Whole Object
Primitive Obsession	Bloaters	**Replace Primitive with Object**, Replace Type Code with Subclasses, Replace Conditional with Polymorphism, Extract Class, Introduce Parameter Object
Repeated Switches	OO Abusers	**Replace Conditional with Polymorphism**
Loops	2nd-ed extra	**Replace Loop with Pipeline**
Lazy Element	Dispensables	**Inline Function**, **Inline Class**, Collapse Hierarchy
Speculative Generality	Dispensables	Collapse Hierarchy, Inline Function, Inline Class, Change Function Declaration, Remove Dead Code
Temporary Field	OO Abusers	**Extract Class**, Move Function, Introduce Special Case
Message Chains	Couplers	**Hide Delegate**, Extract Function, Move Function
Middle Man	Couplers	**Remove Middle Man**, Inline Function, Replace Superclass with Delegate, Replace Subclass with Delegate
Insider Trading	Couplers	**Move Function**, **Move Field**, Hide Delegate, Replace Subclass with Delegate, Replace Superclass with Delegate
Large Class	Bloaters	**Extract Class**, Extract Superclass, Replace Type Code with Subclasses
Alternative Classes with Different Interfaces	OO Abusers	**Change Function Declaration**, Move Function, Extract Superclass
Data Class	Dispensables	**Encapsulate Record**, Remove Setting Method, Move Function, Extract Function, Split Phase
Refused Bequest	OO Abusers	Push Down Method, Push Down Field, **Replace Subclass with Delegate**, **Replace Superclass with Delegate**
Comments	Dispensables	**Extract Function**, Change Function Declaration, Introduce Assertion

And the reverse view — which **design principle** each family of smells violates (links to Chapters 4–6):

Smell family	Principle it violates
Divergent Change, Large Class	Single Responsibility Principle
Repeated Switches, Shotgun Surgery	Open/Closed Principle
Refused Bequest	Liskov Substitution Principle
Long Parameter List with flags, fat interfaces	Interface Segregation / clean function design
Global Data, Mutable Data	Encapsulation / information hiding (control who can change what)
Hard-wired `new` of concrete dependencies	Dependency Inversion
Duplicated Code	DRY (Don't Repeat Yourself)
Speculative Generality	YAGNI
Message Chains, Insider Trading	Law of Demeter / encapsulation
Feature Envy, Data Class	"Tell, Don't Ask" — put behaviour with data

---

7.5 Worked example — from messy to clean in six compiling steps

The starting point

A courier company prints invoices. Each shipment has a **service type** (`STD` standard, `EXP` express, `FRG` fragile), a weight and a distance.

Tariffs:

Type	Charge	Loyalty points
STD	5 + 1.2 per kg, plus 0.05 per km beyond 100 km	1 per full 100 km
EXP	12 + 2.0 per kg, plus 0.10 per km beyond 50 km	1 per full 100 km, +2 bonus
FRG	8 + 1.5 per kg + 3 handling; ×1.2 if heavier than 10 kg	1 per full 100 km

The new feature request: "We also need an HTML invoice for the web portal, and next quarter we are adding an OVERNIGHT service."

The current code (step 0, abridged — full file in 7.8):

```java
static String print(Inv inv) {
    double tot = 0;
    int pts = 0;
    int unused = 0;                                  // dead code
    StringBuilder r = new StringBuilder("Invoice for " + inv.cust + "\n");
    for (Shp s : inv.list) {
        double amt = 0;
        switch (s.t) {
            case "STD": amt = 5 + s.kg * 1.2;       if (s.km > 100) amt += (s.km - 100) * 0.05; break;
            case "EXP": amt = 12 + s.kg * 2.0;      if (s.km > 50)  amt += (s.km - 50) * 0.10;  break;
            case "FRG": amt = 8 + s.kg * 1.5 + 3;   if (s.kg > 10)  amt = amt * 1.2;            break;
            default: throw new IllegalArgumentException("unknown type: " + s.t);
        }
        pts += s.km / 100;
        if ("EXP".equals(s.t)) pts += 2;
        r.append("  ").append(s.t).append(" ").append(s.kg).append("kg ")
         .append(s.km).append("km: ").append(String.format(Locale.ROOT, "%.2f", amt)).append("\n");
        tot += amt;
    }
    r.append("Amount owed: ").append(String.format(Locale.ROOT, "%.2f", tot)).append("\n");
    r.append("Points earned: ").append(pts).append("\n");
    return r.toString();
}
```

Smells spotted.

Smell	Where
Mysterious Name	`Shp`, `Inv`, `t`, `kg`, `km`, `amt`, `tot`, `pts`, `r`
Long Function	`print` calculates charges, calculates points, and formats — three jobs
Divergent Change	HTML output would force copying the whole function; a new tariff means editing the formatting function
Repeated Switches (waiting to happen)	The type code `"EXP"` is checked in two places already
Primitive Obsession	The service type is a `String`; any typo compiles
Magic literals	`5`, `1.2`, `100`, `0.05`, `12`, ...
Data Class / Mutable Data	Public mutable fields in `Shp` and `Inv`
Dead code	`int unused`

Adding HTML now would mean duplicating everything. So we do **preparatory refactoring** first: make the change easy, then make the easy change.

The safety net

Before touching anything we write a **characterisation check** — `lld.ch07.worked.Main`. It runs step 0 to record the "golden" output, then runs every later step and compares the output **byte for byte**. The fixture (the fixed test input data) includes boundary cases (exactly 100 km for STD, exactly 10 kg for FRG) because boundaries are where refactorings most often break behaviour. In a real project this would be a JUnit test (JUnit is the standard Java testing framework); here it is a plain `main` so it runs with the JDK (Java Development Kit — the compiler and runtime) alone.

Each step lives in its own package (`step0` … `step5`) so you can compile and diff them (compare them line by line). After *every* step the checker prints `SAME output`.

Step 1 — Extract Function, Rename Variable/Field, Remove Dead Code

- **Rename Field**: `t → type`, `kg → weightKg`, `km → distanceKm`; classes `Shp → Shipment`, `Inv → Invoice`.
- **Rename Variable**: `tot → totalAmount`, `pts → loyaltyPoints`, `r → result`.
- **Extract Function** ×3: the switch becomes `amountFor(shipment)`, the points logic becomes `pointsFor(shipment)`, the formatting becomes `money(amount)`.

- **Remove Dead Code**: `unused` is gone.

```
for (Shipment shipment : invoice.shipments) {
    double thisAmount = amountFor(shipment);
    loyaltyPoints += pointsFor(shipment);
    result.append(...).append(money(thisAmount)).append("\n");
    totalAmount += thisAmount;
}
```

The switch itself was *moved*, not changed. Moving before changing is a key habit: two small safe steps instead of one risky one.

Step 2 — Inline Variable, Split Loop, Replace Temp with Query, Replace Loop with Pipeline, Slide Statements

The loop still does three jobs (print a line, add to total, add points). We:

1. **Inline Variable** `thisAmount` — call `amountFor(shipment)` directly.
2. **Split Loop** — one loop for printing, one for the total, one for points.
3. **Replace Temp with Query** — the total and the points become `totalAmount(invoice)` and `totalPoints(invoice)`.
4. **Replace Loop with Pipeline** — those two queries become streams.
5. **Slide Statements** — the declarations disappear; each value is computed where it is used.

```
static double totalAmount(Invoice invoice) {
    return invoice.shipments.stream().map(Main::amountFor).reduce(0.0, Double::sum);
}
static int totalPoints(Invoice invoice) {
    return invoice.shipments.stream().mapToInt(Main::pointsFor).sum();
}
```

A subtle trap. The obvious pipeline is `mapToDouble(...).sum()`. But `DoubleStream.sum()` may use compensated summation, so it can differ from a plain left-to-right loop in the last bit of a `double`. That is a (tiny) behaviour change. `reduce(0.0, Double::sum)` on a sequential stream adds in encounter order — exactly like the loop. (For `int`, addition is exact, so `IntStream.sum()` is safe.) Performance note: we now call `amountFor` twice per shipment — the kind of cost Fowler says to ignore until a profiler shows it matters.

Step 3 — Split Phase, Encapsulate Record, Encapsulate Collection, Combine Functions into Transform

Now the key preparatory move. Calculation and formatting are two **phases**. We split them with an intermediate data structure:

- `StatementData` (a record) — everything the renderer needs: customer, one `Line` per shipment with its computed amount, total, points.
- `StatementCalculator.calculate(Invoice)` — phase 1, a **transform** (Combine Functions into Transform): raw invoice in, enriched data out.
- `TextRenderer.render(StatementData)` — phase 2. It knows nothing about tariffs.
- `Shipment` and `Invoice` become **records** (Encapsulate Record — no more public mutable fields), and `Invoice` copies its list with `List.copyOf` (Encapsulate Collection).

```
return new TextRenderer().render(new StatementCalculator().calculate(invoice));
```

Step 4 — Replace Primitive with Object, Replace Magic Literal

- **Replace Primitive with Object**: the `String` type code becomes `enum ServiceType { STANDARD("STD"), EXPRESS("EXP"), FRAGILE("FRG") }` with `fromCode(String)`. An unknown code now fails *at the boundary* (when the shipment is created) instead of deep inside the calculation. It is the same exception type and message, and in this program shipments are always built and printed together, so the observable behaviour is unchanged; if some caller created invoices without ever printing them, moving the

failure earlier would be a behaviour change to check first. The `"STD"` strings survive only where data enters the system and where it is printed.

- **Replace Magic Literal**: every tariff number gets a name (`STD_FREE_KM`, `FRG_HEAVY_FACTOR`, ...).
- The switch becomes a **switch expression** over the enum; with no `default` branch, the compiler checks it is exhaustive.

```
return switch (s.type()) {
    case STANDARD -> STD_BASE + s.weightKg() * STD_PER_KG
            + (s.distanceKm() > STD_FREE_KM ? (s.distanceKm() - STD_FREE_KM) * STD_PER_EXTRA_KM : 0);
    case EXPRESS  -> ...
    case FRAGILE  -> { double base = ...; yield s.weightKg() > FRG_HEAVY_KG ? base * FRG_HEAVY_FACTOR : base; }
};
```

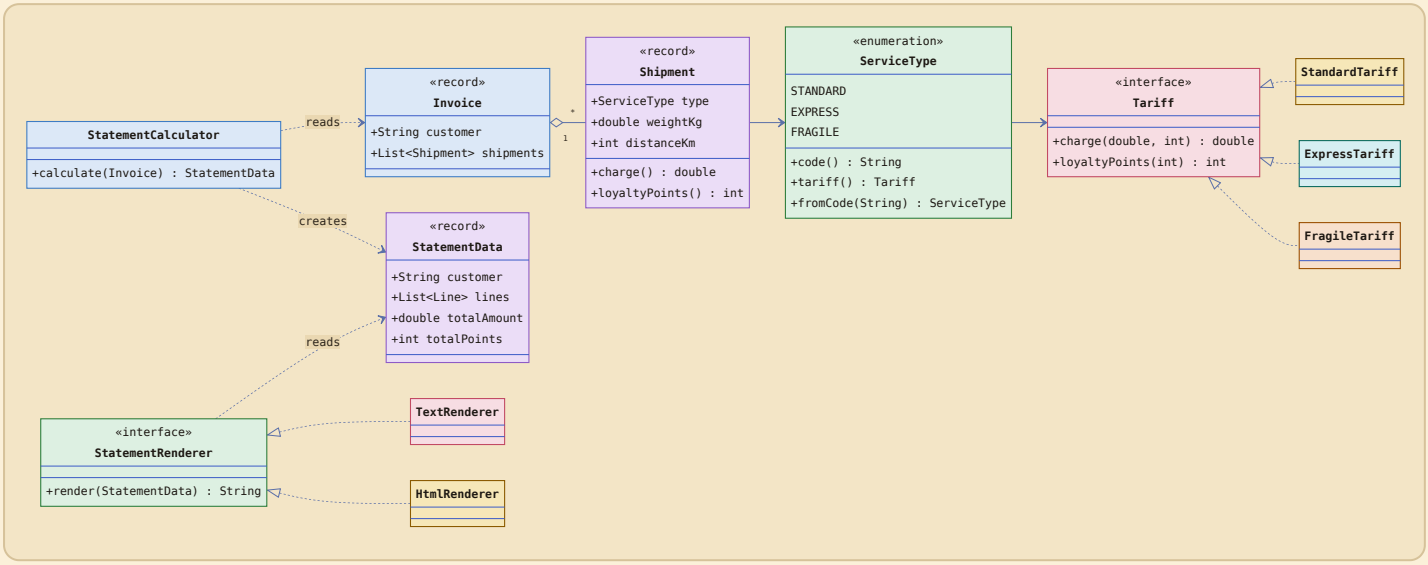
(Is `x + 0` the same as the old "`if` not taken"? For `double`, adding `+0.0` leaves any value unchanged except `-0.0`, which cannot occur here. The checker confirms it.)

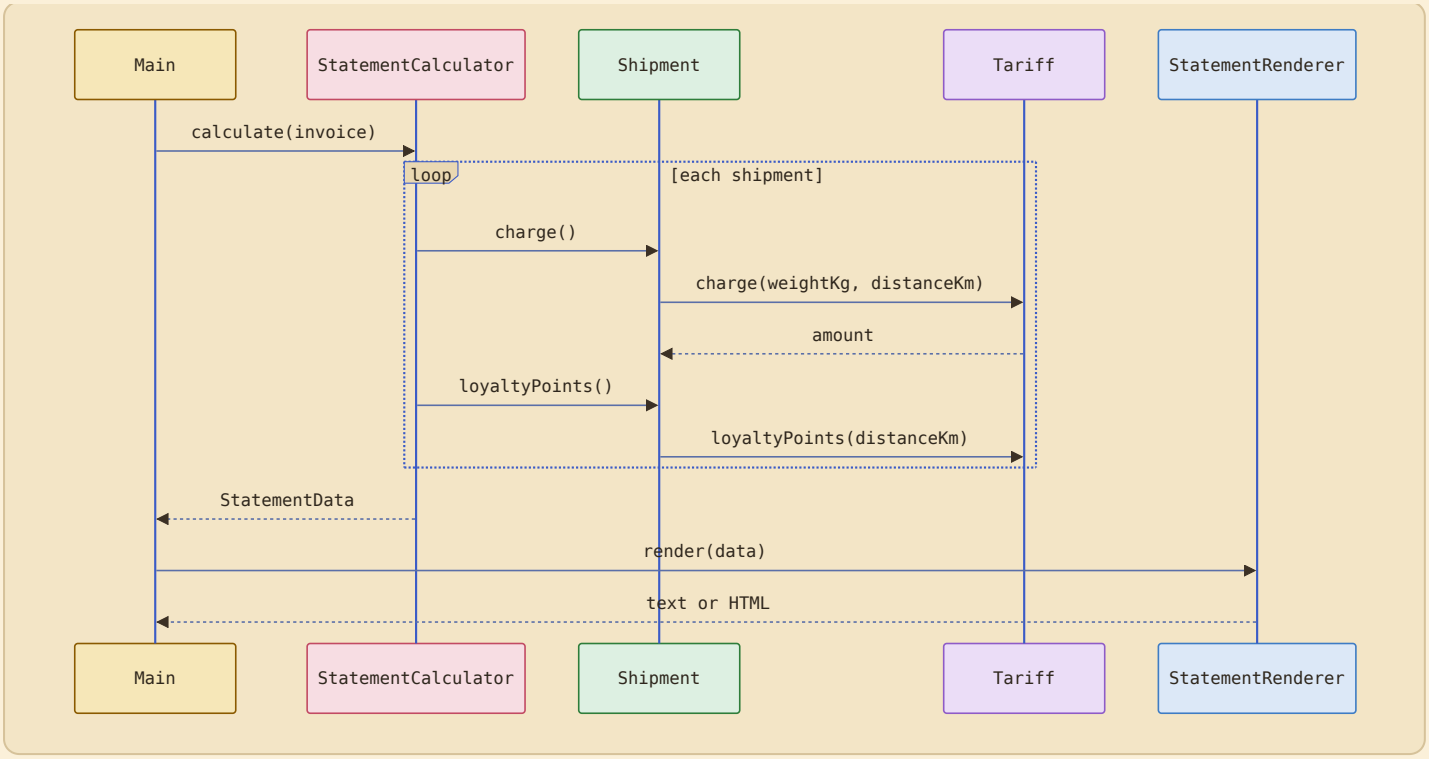
Step 5 — Replace Conditional with Polymorphism, Move Function, then the new feature

- **Replace Conditional with Polymorphism**: each switch branch becomes a class implementing `sealed interface Tariff { double charge(double weightKg, int distanceKm); default int loyaltyPoints(int distanceKm) }`. `ExpressTariff` overrides `loyaltyPoints` to add its bonus — the second `"EXP"` check disappears too. Each enum constant holds its `Tariff` (a **Strategy**).
- **Move Function**: `Shipment.charge()` and `Shipment.loyaltyPoints()` — the calculator no longer reaches into shipment data (Feature Envy cured).
- `StatementCalculator` has **no conditionals left**.

Then — wearing the *adding function* hat now — the new feature is easy:

- Extract a `StatementRenderer` interface; `TextRenderer` implements it.
- Add `HtmlRenderer` — about 15 lines, reusing phase 1 unchanged.
- Adding OVERNIGHT next quarter = one new `Tariff` record, one new enum constant, and one more name in `Tariff`'s `permits` list (a sealed interface must list every implementation). The calculator and the renderers do not change (Open/Closed Principle for the calculation and rendering code).





What changed, step by step

Step	Refactorings	Smells removed
1	Extract Function ×3, Rename Variable, Rename Field, Remove Dead Code	Mysterious Name, part of Long Function, dead code
2	Inline Variable, Split Loop, Replace Temp with Query, Replace Loop with Pipeline, Slide Statements	Long Function (loop doing 3 jobs), Loops
3	Split Phase, Combine Functions into Transform, Encapsulate Record, Encapsulate Collection	Divergent Change, Data Class, Mutable Data
4	Replace Primitive with Object, Replace Magic Literal	Primitive Obsession, magic numbers
5	Replace Conditional with Polymorphism, Move Function, (then Extract interface for renderers)	Repeated Switches, Feature Envy

Checker output (`java lld.ch07.worked.Main`):

```
Golden output from step 0:
Invoice for Asha Traders
  STD 4.0kg 250km: 17.30
  EXP 2.5kg 80km: 20.00
  FRG 12.0kg 40km: 34.80
  STD 1.0kg 90km: 6.20
  STD 0.5kg 100km: 5.60
  FRG 10.0kg 150km: 26.00
Amount owed: 109.90
Points earned: 6
---
step0 (messy original)         SAME output
step1 extract + rename         SAME output
step2 split loop + pipeline    SAME output
step3 split phase              SAME output
step4 primitive -> object      SAME output
step5 polymorphism (final)     SAME output
All steps preserve behaviour.
```

What we deliberately did NOT do. Money is still a `double`. Switching to `BigDecimal` (or the `Money` type from `code/smells`) would change rounding in some cases — that is a *behaviour change*, so it belongs under

the "adding function" hat, with its own tests and a product decision about rounding rules. Refactoring and behaviour changes are kept apart.

7.6 Refactoring toward patterns

What it is

Design patterns (Chapters 8–11) are not only designed up front. Very often you **arrive at a pattern by refactoring** when the code asks for it. Joshua Kerievsky's book *Refactoring to Patterns* (2004) catalogues these moves; many are compositions of Fowler's small refactorings. The idea: do not force patterns in early (Speculative Generality); refactor *toward* a pattern when a smell appears, and sometimes *away* from one when it is no longer pulling its weight.

How it works

> **Key term — patterns in this table not met earlier in this chapter** (all are covered in Chapters 8–11). **Factory**: an object or method whose only job is creating other objects; a **Builder** assembles one complex object step by step. **Observer**: subscribers register with an object and are notified automatically when it changes. **Bridge**: separates an abstraction from its implementation so the two can vary independently. **Composite**: a tree structure in which single items and groups of items are treated the same way.

Smell you see	Refactor toward	Via (Fowler's steps)
A switch/if-chain choosing an algorithm (e.g. discount by customer tier)	**Strategy**	Extract Function per branch → Replace Conditional with Polymorphism (Kerievsky: *Replace Conditional Logic with Strategy*)
Methods that check a status field and change it (`if status == PAID then ...`)	**State**	Replace Type Code with Subclasses (indirect, via a state object) → Replace Conditional with Polymorphism (Kerievsky: *Replace State-Altering Conditionals with State*)
`if (x == null)` repeated at many call sites	**Null Object / Special Case**	Introduce Special Case (Kerievsky: *Introduce Null Object*)
Sibling subclasses with the same algorithm skeleton but different steps	**Template Method**	Extract Function in each → Pull Up Method (Kerievsky: *Form Template Method*)
Many constructors with confusing boolean/int parameters	**Factory Method / Creation Methods**	Replace Constructor with Factory Function (Kerievsky: *Replace Constructors with Creation Methods*)
Many `new ConcreteX()` calls scattered; clients should not know which class	**Factory**	Replace Constructor with Factory Function + Move Function to a factory class (Kerievsky: *Encapsulate Classes with Factory*)
Complex object (in Kerievsky's refactoring, a Composite tree) built step by step with many calls in the right order	**Builder**	Kerievsky: *Encapsulate Composite with Builder*
A function too big to decompose because of many shared locals; or you need undo/queueing	**Command**	Replace Function with Command
Embellishment code (logging, caching) mixed into a core class	**Decorator**	Kerievsky: *Move Embellishment to Decorator*
Many objects polling one object for changes; hard-coded notification	**Observer**	Kerievsky: *Replace Hard-Coded Notifications with Observer*
Two classes with different interfaces doing the same job	**Adapter**	Change Function Declaration / Extract Superclass; wrap third-party code (Kerievsky: *Extract Adapter*, *Unify Interfaces with Adapter*)
A switch on a type code to create objects	**Factory + polymorphism**	Replace Type Code with Subclasses + Replace Constructor with Factory Function
Subclass explosion for combinations of features	**Strategy / Decorator / Bridge**	Replace Subclass with Delegate
A pattern that no longer earns its keep (one strategy, one decorator)	*Remove the pattern*	Inline Class, Collapse Hierarchy (Kerievsky: *Inline Singleton* and similar "away from" moves)

The runnable package `lld.ch07.towardpatterns` shows five of these with the **before** code and the **after** code side by side, and checks that both produce the same results. (The prices and loan amounts there are `double` only to keep the before/after comparison short; in production code use `BigDecimal` or a `Money` type.)

Replace Conditional Logic with Strategy

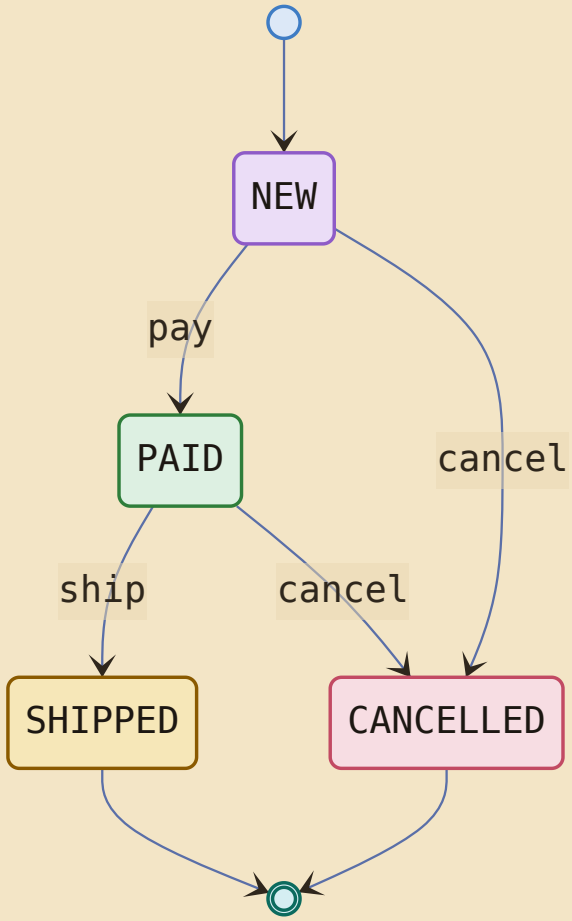
Before: a `switch (tier)` that every new customer tier must edit. **After:** a `DiscountPolicy` functional interface (an interface with a single abstract method, so a lambda — a short inline function — can implement it), one policy per tier (lambdas here; classes if a policy grows), stored in an `EnumMap` (a map whose keys are enum constants). New tiers are *registered*, not coded into a switch.

```java
@FunctionalInterface
interface DiscountPolicy { double apply(double amount); }

double price(Tier tier, double amount) {
    return policies.getOrDefault(tier, fallback).apply(amount);
}
```

Replace State-Altering Conditionals with State

Before: every method checks `status` strings and changes them — the rules of the lifecycle are scattered across methods. **After:** each state is an enum constant implementing `OrderState`; it overrides only the moves it allows. Default methods (interface methods with a built-in body) reject everything else. `Order` just delegates: `state = state.pay(this)`.




```java
enum States implements OrderState {
    NEW {
        @Override public OrderState pay(Order o)    { o.log("paid"); return PAID; }
        @Override public OrderState cancel(Order o) { o.log("cancelled"); return CANCELLED; }
    },
    PAID {
        @Override public OrderState ship(Order o)   { o.log("shipped"); return SHIPPED; }
        @Override public OrderState cancel(Order o) { o.log("cancelled"); return CANCELLED; }
    },
    SHIPPED, CANCELLED
}
```

Strategy vs State: they have the same class structure. The difference is intent — a **strategy** is usually chosen by the client and stays fixed; a **state** changes *itself* as the object moves through its lifecycle.

Introduce Special Case (Null Object)

Before: `(c == null) ? "occupant" : c.name()` in every client. **After:** `find(id)` returns an `UnknownCustomer` that answers `"occupant"` and `"basic"` itself. Clients have no null checks.

Form Template Method

Before: `legacyCsv` and `legacyMarkdown` repeat the same skeleton (header, one row per item, footer). **After:** an abstract `Exporter.export()` (declared `final`) holds the skeleton once and calls abstract `header()`, `row()`, `footer()` steps; `CsvExporter` and `MarkdownExporter` fill them in.

Replace Constructors with Creation Methods

Before: `new LegacyLoan(5000, false, 12)` — what does `false` mean? **After:** a private constructor plus `Loan.termLoan(5000, 12)` and `Loan.revolver(2000)`.

Output (`java lld.ch07.towardpatterns.Main`):

```
Strategy:        GOLD 250 -> 200.0, NEW 250 -> 240.0 | same as switch: true
State:           cannot ship when NEW; paid; cannot pay when PAID; shipped; cannot cancel when SHIPPED; | same as
if-chains: true
Special Case:    Hello Ravi on premium / Hello occupant on basic | same: true
Template Method: CSV and Markdown exporters | same: true
Factory:         term 5000.0/12, revolver 2000.0/0 | same: true
All pattern refactorings preserve behaviour.
```

Small smell cures (package `lld.ch07.smells`)

A separate runnable package shows the cures for several smells from 7.2 as real code: `Money` (Primitive Obsession), `DateRange` (Data Clumps), and `Order` with `shippingCity()` (Message Chains → Hide Delegate), `Line.total()` (Feature Envy → Move Function) and a read-only `lines()` view (Mutable Data → Encapsulate Collection).

Output (`java lld.ch07.smells.Main`):

```
double 0.1+0.2       = 0.30000000000000004
Money  0.10+0.20     = USD 0.30
mixing currencies    -> currency mismatch: USD vs EUR
booking contains 3rd = true, overlaps other = true
ship to Bengaluru, total INR 301.00
outsider tried lines().clear() -> UnsupportedOperationException (collection is encapsulated)
```

7.7 Putting it together

Trade-offs

Benefit	Cost / risk
Code easier to read and change; features get faster over time (Design Stamina)	Takes time now; the pay-off is later and hard to measure
Bugs become visible	Without tests, you can introduce bugs
Design can evolve with requirements (less up-front guessing)	Renames and moves cause merge conflicts on long-lived branches
Small steps are safe and can stop at any point	Small steps feel slow to beginners (they get fast with practice and IDE support)
More small classes and methods, each easy to understand	More indirection — too much creates Middle Men and Lazy Elements
Removes duplication	Premature abstraction (before the Rule of Three) can create the *wrong* abstraction, which is costlier than duplication

When not to use it

(Summary of 7.1.4.) Do not refactor code you do not need to touch; do not refactor without a safety net (add tests first); do not refactor instead of a rewrite when the code does not work at all; do not break published interfaces without a migration path; do not mix refactoring with behaviour changes in the same step.

Common mistakes

Mistake	Why it is a problem	Do this instead
Calling any code change "refactoring"	Behaviour changes hide inside "refactoring" commits; reviewers relax	Keep refactoring commits separate from feature/bug-fix commits
Big-bang refactoring (days with broken code)	High risk, merge pain, no way to stop halfway	Tiny steps, compile and test after each, commit often
Refactoring without tests	Silent behaviour changes	Write characterisation tests first; use IDE-automated refactorings
Debugging a failed refactoring step	Wastes time	Revert and redo in smaller steps
Tests coupled to private implementation	Every refactoring breaks tests	Test behaviour through public APIs
Text search-and-replace for renames	Changes strings, comments, unrelated symbols	IDE Rename (symbol-aware) — and check reflection/config/serialisation
Abstracting on the first duplication	Wrong abstraction; many flags later	Rule of Three
Changing rounding/ordering "while you're there" (e.g. `DoubleStream.sum()` vs loop, `toList()` vs mutable `ArrayList`)	Subtle behaviour changes	Know the exact semantics of the replacement API
Adding patterns everywhere "for flexibility"	Speculative Generality	Refactor toward a pattern when a smell demands it
Refactoring sprints as the main approach	Business sees refactoring as a separate, optional cost	Opportunistic refactoring inside every feature

Real-world examples

- **IntelliJ IDEA, Eclipse, VS Code (Java extension)** — automated Rename, Extract Method, Inline, Change Signature, Move, Pull Up / Push Down: these are direct implementations of catalogue entries.
- **OpenRewrite** — open-source engine for automated, large-scale source refactoring with "recipes"; used, for example, to migrate codebases between Spring Boot (a popular Java application framework) versions or from JUnit 4 to JUnit 5.
- **Google** — Error Prone with Refaster templates, used for large automated changes across its monorepo (one giant repository holding most of the company's code).
- **JDK history.** `java.util.Stack extends Vector` and `java.util.Properties extends Hashtable` are classic, often-cited cases of inheritance where delegation would have been better (Refused Bequest). For new code, the JDK docs recommend `ArrayDeque` (via the `Deque` interface) over `Stack`. `Date` → `java.time` (Java 8) shows Primitive Obsession/Mutable Data being replaced by immutable value types (`LocalDate`, `Instant`, `Duration`) — a new API alongside the old one rather than an in-place refactor, because the old API is published.
- **Deprecation as migration mechanics.** The JDK and Spring mark old methods `@Deprecated` (Java 9 added `forRemoval`) and keep them delegating for several releases — exactly *Change Function Declaration* with migration mechanics.
- **Java language features that support refactoring**: records (Encapsulate Record, value objects), `sealed` interfaces + switch pattern matching (safe type hierarchies), streams (Replace Loop with

Pipeline), `Optional` (making absence explicit), `List.copyOf` / `Collections.unmodifiableList` (Encapsulate Collection).

- **Strangler Fig in industry.** Many companies that split a monolith (an application built and deployed as one unit) into services (Fowler's own write-ups describe this pattern) route one feature at a time from the old system to the new, instead of a big-bang rewrite.

In the interview

30-second answer — "What is refactoring?" "Refactoring is changing the internal structure of code to make it easier to understand and modify, without changing its observable behaviour. You do it in very small steps — each one compiles and passes the tests — so the system is never broken. You use code smells like Long Function, Duplicated Code or Repeated Switches to find where to refactor, and named refactorings like Extract Function, Move Function or Replace Conditional with Polymorphism to fix them. Tests are the safety net. The goal is economic: clean code lets you add features faster."

Follow-ups (with answers)

- **Q: How do you refactor code that has no tests?** A: Treat it as legacy code. First write characterisation tests that record current behaviour. If the code is hard to test, break dependencies at a seam — for example, pass an interface through the constructor instead of calling `new` inside. Until tests exist, use only IDE-automated, symbol-aware refactorings. Sprout Method/Class lets me add new, tested code without first cleaning everything.
- **Q: Refactor or rewrite?** A: Default to refactoring — small safe steps and the system keeps working. Consider a rewrite only when the code really does not work or the platform must change, and even then use a Strangler Fig approach to replace it piece by piece.
- **Q: Doesn't refactoring hurt performance?** A: Rarely in a way that matters: small methods are inlined by the JIT, and most time is spent in a small part of the code. I refactor for clarity, then profile and optimise the real hot spots, which is easier in well-factored code.
- **Q: What would you refactor in this code?** (In a machine-coding round.) A: Name smells and cures: "`processOrder` is a Long Function — I'd Extract Function for validation, pricing and persistence. The switch on `vehicleType` appears in three places — Repeated Switches — I'd Replace Conditional with Polymorphism, giving a Strategy per vehicle type. `double price` is Primitive Obsession — a `Money` value object. And `getParkingLot().getFloor(1).getSpot(3)` is a Message Chain — Hide Delegate."
- **Q: When do you remove duplication?** A: Rule of Three. After three occurrences the shape of what varies is clear, so the abstraction is likely right. Earlier abstraction risks the wrong abstraction.
- **Q: Difference between Divergent Change and Shotgun Surgery?** A: Divergent Change: one class changes for many different reasons → split it. Shotgun Surgery: one reason to change touches many classes → gather the code into one place.
- **Q: How do you get from a switch statement to the Strategy or State pattern?** A: Extract each branch into a function, then Replace Conditional with Polymorphism: create an interface with one implementation per case and let the object hold the right implementation. If the choice is fixed by the client, it's Strategy; if the object moves between cases itself, it's State.
- **Q: Is a data class always bad?** A: No. Immutable records used as values, DTOs or the output of a Split Phase are fine. The smell is a *mutable* class with getters/setters whose behaviour lives in other classes.
- **Q: How do you rename a public API method used by other teams?** A: Migration mechanics of Change Function Declaration: add the new method, make the old one delegate to it and mark it `@Deprecated`, migrate callers over time, then remove the old one. For REST APIs, versioning or expand/contract.

Self-check

1. **Q:** Is fixing a bug while tidying the method one refactoring? **A:** No. The bug fix changes behaviour. Do the refactoring and the fix as separate steps (two hats), ideally separate commits.

2. **Q:** A method prints a banner, sums orders, sets a due date and prints details. Which smell and which refactoring? **A:** Long Function; Extract Function for each block (and maybe Split Loop / Replace Temp with Query).
3. **Q:** `book(hotel, from, to, 2, 1, true)` — which smells? **A:** Long Parameter List, Data Clumps (`from`, `to` and the two guest counts), a flag argument (`true`). Cures: Introduce Parameter Object (`Stay`, `Guests`), Remove Flag Argument (or replace the boolean with an explicit enum such as `MealPlan`).
4. **Q:** What is the inverse of Hide Delegate, and when do you use it? **A:** Remove Middle Man — when a class has become mostly forwarding methods.
5. **Q:** Why did the worked example use `reduce(0.0, Double::sum)` instead of `DoubleStream.sum()`? **A:** `DoubleStream.sum()` may use compensated summation and can differ from a plain loop in the last bit; `reduce` on a sequential stream adds left to right like the loop, so behaviour is preserved.
6. **Q:** What is a characterisation test? **A:** A test that records what existing code currently does (right or wrong), so a refactoring that changes behaviour is detected.
7. **Q:** Which smell does `class Stack extends ArrayList` have, and what is the cure? **A:** Refused Bequest (it inherits list operations that break stack rules, violating Liskov). Replace Superclass with Delegate — hold a `Deque` instead.
8. **Q:** Which refactoring turns a status-checking `if` chain into the State pattern? **A:** Replace Type Code with Subclasses (via a state object) followed by Replace Conditional with Polymorphism (Kerievsky: Replace State-Altering Conditionals with State).
9. **Q:** Name the five refactoring.guru smell groups. **A:** Bloaters, Object-Orientation Abusers, Change Preventers, Dispensables, Couplers.
10. **Q:** What is preparatory refactoring? **A:** Refactoring just before a feature so that the feature becomes easy to add — "make the change easy, then make the easy change". Step 3 (Split Phase) of the worked example did this for HTML output.
11. **Q:** When is a comment not a smell? **A:** When it explains *why* (a decision, a trade-off, a reference, a warning), not *what* the code does.
12. **Q:** What does Separate Query from Modifier enforce? **A:** Command–Query Separation: functions that return values have no observable side effects.
13. **Q:** A Java IDE rename of a field broke the JSON response. Was it a refactoring? **A:** It was meant to be, but the JSON field name (observable behaviour) changed, so behaviour was not preserved. Check serialisation, reflection and config when renaming.

TL;DR

- **Refactoring = small, behaviour-preserving changes** to internal structure, done continuously, with tests after every step. Two hats: never refactor and change behaviour at the same time.
- **When:** Rule of Three, preparatory ("make the change easy, then make the easy change"), comprehension, litter-pickup, during code review. Mostly opportunistic, rarely planned. **Not** for code you never touch, and not without a safety net.
- **Smells** tell you *where*: 24 in the 2nd edition — Mysterious Name, Duplicated Code, Long Function, Long Parameter List, Global Data, Mutable Data, Divergent Change, Shotgun Surgery, Feature Envy, Data Clumps, Primitive Obsession, Repeated Switches, Loops, Lazy Element, Speculative Generality, Temporary Field, Message Chains, Middle Man, Insider Trading, Large Class, Alternative Classes with Different Interfaces, Data Class, Refused Bequest, Comments. Groups: Bloaters, OO Abusers, Change Preventers, Dispensables, Couplers.
- **Refactorings** tell you *how*: 61 named ones in seven families; the everyday core is Extract/Inline Function, Rename, Move Function, Extract Class, Split Phase, Replace Conditional with Polymorphism.
- **Performance:** refactor first, then profile and optimise hot spots. **Rewrite** only rarely, and then with a Strangler Fig. **Legacy code** = code without tests → characterisation tests, seams, sprout/wrap.
- **Patterns** are often reached by refactoring: switch → Strategy/State, null checks → Null Object, duplicated skeleton → Template Method, confusing constructors → Factory methods.

7.8 Full source code and output

How to compile and run (Java 21, standard library only):

```
cd chapters/07-refactoring/code
javac --release 21 -d /tmp/lld_07 $(find . -name '*.java')
java -cp /tmp/lld_07 lld.ch07.worked.Main           # characterisation check over all six steps
java -cp /tmp/lld_07 lld.ch07.worked.step5.Main     # final design + new HTML feature
java -cp /tmp/lld_07 lld.ch07.towardpatterns.Main   # refactoring toward patterns
java -cp /tmp/lld_07 lld.ch07.smells.Main           # small smell cures
```

Packages: `lld.ch07.worked` (checker), `lld.ch07.worked.step0` … `step5` (each has its own `Main`; steps 0–4 print the text invoice only), `lld.ch07.towardpatterns`, `lld.ch07.smells`. All 9 `Main` classes compile and run.

Worked example — characterisation check

```java
// File: code/worked/Main.java
package lld.ch07.worked;

import java.util.LinkedHashMap;
import java.util.Map;
import java.util.function.Supplier;

/**
 * Characterisation check - the safety net for the whole worked example.
 * Every step must print exactly what the messy STEP 0 printed. If one byte differs,
 * that step was not a refactoring (it changed behaviour) and we exit with an error.
 */
public class Main {

    public static void main(String[] args) {
        Map<String, Supplier<String>> steps = new LinkedHashMap<>();
        steps.put("step0 (messy original)", lld.ch07.worked.step0.Main::run);
        steps.put("step1 extract + rename", lld.ch07.worked.step1.Main::run);
        steps.put("step2 split loop + pipeline", lld.ch07.worked.step2.Main::run);
        steps.put("step3 split phase", lld.ch07.worked.step3.Main::run);
        steps.put("step4 primitive -> object", lld.ch07.worked.step4.Main::run);
        steps.put("step5 polymorphism (final)", lld.ch07.worked.step5.Main::run);

        String golden = lld.ch07.worked.step0.Main.run();
        System.out.print("Golden output from step 0:\n" + golden);
        System.out.println("---");
        boolean allSame = true;
        for (Map.Entry<String, Supplier<String>> e : steps.entrySet()) {
            boolean same = golden.equals(e.getValue().get());
            allSame &= same;
            System.out.printf("%-30s %s%n", e.getKey(), same ? "SAME output" : "DIFFERENT output!");
        }
        if (!allSame) {
            System.err.println("A step changed behaviour - that was not a refactoring.");
            System.exit(1);
        }
        System.out.println("All steps preserve behaviour.");
    }
}
```

Worked example — step 0 (messy original)

```java
// File: code/worked/step0/Main.java
package lld.ch07.worked.step0;

import java.util.ArrayList;
import java.util.List;
import java.util.Locale;

/** STEP 0 - the messy starting point. It works, but it smells. */
public class Main {

    // Mutable public data class (smells: Data Class, Mysterious Name)
    static class Shp {
        public String t;   // "STD", "EXP" or "FRG"
        public double kg;
        public int km;
        Shp(String t, double kg, int km) { this.t = t; this.kg = kg; this.km = km; }
    }

    static class Inv {
        public String cust;
        public List<Shp> list = new ArrayList<>();
    }

    // One Long Function that calculates AND formats (smells: Long Function,
    // Repeated Switches waiting to happen, magic numbers, temps everywhere)
    static String print(Inv inv) {
        double tot = 0;
        int pts = 0;
        int unused = 0;                                  // dead code
        StringBuilder r = new StringBuilder("Invoice for " + inv.cust + "\n");
        for (Shp s : inv.list) {
            double amt = 0;
            switch (s.t) {
                case "STD":
                    amt = 5 + s.kg * 1.2;
                    if (s.km > 100) amt += (s.km - 100) * 0.05;
                    break;
                case "EXP":
                    amt = 12 + s.kg * 2.0;
                    if (s.km > 50) amt += (s.km - 50) * 0.10;
                    break;
                case "FRG":
                    amt = 8 + s.kg * 1.5 + 3;
                    if (s.kg > 10) amt = amt * 1.2;
                    break;
                default:
                    throw new IllegalArgumentException("unknown type: " + s.t);
            }
            // loyalty points
            pts += s.km / 100;
            if ("EXP".equals(s.t)) pts += 2;
            r.append("  ").append(s.t).append(" ").append(s.kg).append("kg ")
             .append(s.km).append("km: ")
             .append(String.format(Locale.ROOT, "%.2f", amt)).append("\n");
            tot += amt;
        }
        r.append("Amount owed: ").append(String.format(Locale.ROOT, "%.2f", tot)).append("\n");
        r.append("Points earned: ").append(pts).append("\n");
        return r.toString();
    }

    /** Used by the characterisation check in lld.ch07.worked.Main. */
    public static String run() {
        Inv inv = new Inv();
        inv.cust = "Asha Traders";
        inv.list.add(new Shp("STD", 4.0, 250));
        inv.list.add(new Shp("EXP", 2.5, 80));
        inv.list.add(new Shp("FRG", 12.0, 40));
        inv.list.add(new Shp("STD", 1.0, 90));
```

```
        inv.list.add(new Shp("STD", 0.5, 100));   // boundary: exactly 100 km
        inv.list.add(new Shp("FRG", 10.0, 150));  // boundary: exactly 10 kg
        return print(inv);
    }

    public static void main(String[] args) {
        System.out.print(run());
    }
}
```

Worked example — step 1

```java
// File: code/worked/step1/Main.java
package lld.ch07.worked.step1;

import java.util.ArrayList;
import java.util.List;
import java.util.Locale;

/**
 * STEP 1 - Extract Function + Rename Variable + Remove Dead Code.
 * The loop body is split into amountFor() and pointsFor(); names now say what they mean.
 */
public class Main {

    static class Shipment {
        public String type;          // Rename Field: t -> type
        public double weightKg;      // kg -> weightKg
        public int distanceKm;       // km -> distanceKm
        Shipment(String type, double weightKg, int distanceKm) {
            this.type = type; this.weightKg = weightKg; this.distanceKm = distanceKm;
        }
    }

    static class Invoice {
        public String customer;
        public List<Shipment> shipments = new ArrayList<>();
    }

    static String statement(Invoice invoice) {
        double totalAmount = 0;
        int loyaltyPoints = 0;
        StringBuilder result = new StringBuilder("Invoice for " + invoice.customer + "\n");
        for (Shipment shipment : invoice.shipments) {
            double thisAmount = amountFor(shipment);
            loyaltyPoints += pointsFor(shipment);
            result.append("  ").append(shipment.type).append(" ").append(shipment.weightKg).append("kg ")
                  .append(shipment.distanceKm).append("km: ").append(money(thisAmount)).append("\n");
            totalAmount += thisAmount;
        }
        result.append("Amount owed: ").append(money(totalAmount)).append("\n");
        result.append("Points earned: ").append(loyaltyPoints).append("\n");
        return result.toString();
    }

    /** Extracted from the loop body. The switch is unchanged - only moved. */
    static double amountFor(Shipment shipment) {
        double result;
        switch (shipment.type) {
            case "STD":
                result = 5 + shipment.weightKg * 1.2;
                if (shipment.distanceKm > 100) result += (shipment.distanceKm - 100) * 0.05;
                break;
            case "EXP":
                result = 12 + shipment.weightKg * 2.0;
                if (shipment.distanceKm > 50) result += (shipment.distanceKm - 50) * 0.10;
                break;
            case "FRG":
                result = 8 + shipment.weightKg * 1.5 + 3;
                if (shipment.weightKg > 10) result = result * 1.2;
                break;
            default:
                throw new IllegalArgumentException("unknown type: " + shipment.type);
        }
        return result;
    }

    static int pointsFor(Shipment shipment) {
        int result = shipment.distanceKm / 100;
        if ("EXP".equals(shipment.type)) result += 2;
        return result;
```

```
    }

    static String money(double amount) {
        return String.format(Locale.ROOT, "%.2f", amount);
    }

    public static String run() {
        Invoice invoice = new Invoice();
        invoice.customer = "Asha Traders";
        invoice.shipments.add(new Shipment("STD", 4.0, 250));
        invoice.shipments.add(new Shipment("EXP", 2.5, 80));
        invoice.shipments.add(new Shipment("FRG", 12.0, 40));
        invoice.shipments.add(new Shipment("STD", 1.0, 90));
        invoice.shipments.add(new Shipment("STD", 0.5, 100));
        invoice.shipments.add(new Shipment("FRG", 10.0, 150));
        return statement(invoice);
    }

    public static void main(String[] args) {
        System.out.print(run());
    }
}
```

Worked example — step 2

```java
// File: code/worked/step2/Main.java
package lld.ch07.worked.step2;

import java.util.ArrayList;
import java.util.List;
import java.util.Locale;

/**
 * STEP 2 - Inline Variable (thisAmount), Split Loop, Replace Temp with Query,
 * Replace Loop with Pipeline, Slide Statements.
 * The printing loop now only prints; totals come from their own small queries.
 */
public class Main {

    static class Shipment {
        public String type;
        public double weightKg;
        public int distanceKm;
        Shipment(String type, double weightKg, int distanceKm) {
            this.type = type; this.weightKg = weightKg; this.distanceKm = distanceKm;
        }
    }

    static class Invoice {
        public String customer;
        public List<Shipment> shipments = new ArrayList<>();
    }

    static String statement(Invoice invoice) {
        StringBuilder result = new StringBuilder("Invoice for " + invoice.customer + "\n");
        for (Shipment shipment : invoice.shipments) {
            result.append("  ").append(shipment.type).append(" ").append(shipment.weightKg).append("kg ")
                  .append(shipment.distanceKm).append("km: ").append(money(amountFor(shipment))).append("\n");
        }
        result.append("Amount owed: ").append(money(totalAmount(invoice))).append("\n");
        result.append("Points earned: ").append(totalPoints(invoice)).append("\n");
        return result.toString();
    }

    // reduce(0.0, Double::sum) is a plain left-to-right sum, exactly like the old loop.
    // DoubleStream.sum() may use compensated summation and could differ in the last bit,
    // which would NOT be a pure refactoring.
    static double totalAmount(Invoice invoice) {
        return invoice.shipments.stream().map(Main::amountFor).reduce(0.0, Double::sum);
    }

    static int totalPoints(Invoice invoice) {
        return invoice.shipments.stream().mapToInt(Main::pointsFor).sum();
    }

    static double amountFor(Shipment shipment) {
        double result;
        switch (shipment.type) {
            case "STD":
                result = 5 + shipment.weightKg * 1.2;
                if (shipment.distanceKm > 100) result += (shipment.distanceKm - 100) * 0.05;
                break;
            case "EXP":
                result = 12 + shipment.weightKg * 2.0;
                if (shipment.distanceKm > 50) result += (shipment.distanceKm - 50) * 0.10;
                break;
            case "FRG":
                result = 8 + shipment.weightKg * 1.5 + 3;
                if (shipment.weightKg > 10) result = result * 1.2;
                break;
            default:
                throw new IllegalArgumentException("unknown type: " + shipment.type);
        }
        return result;
```

```
    }

    static int pointsFor(Shipment shipment) {
        int result = shipment.distanceKm / 100;
        if ("EXP".equals(shipment.type)) result += 2;
        return result;
    }

    static String money(double amount) {
        return String.format(Locale.ROOT, "%.2f", amount);
    }

    public static String run() {
        Invoice invoice = new Invoice();
        invoice.customer = "Asha Traders";
        invoice.shipments.add(new Shipment("STD", 4.0, 250));
        invoice.shipments.add(new Shipment("EXP", 2.5, 80));
        invoice.shipments.add(new Shipment("FRG", 12.0, 40));
        invoice.shipments.add(new Shipment("STD", 1.0, 90));
        invoice.shipments.add(new Shipment("STD", 0.5, 100));
        invoice.shipments.add(new Shipment("FRG", 10.0, 150));
        return statement(invoice);
    }

    public static void main(String[] args) {
        System.out.print(run());
    }
}
```

Worked example — step 3

```
// File: code/worked/step3/Shipment.java
package lld.ch07.worked.step3;

/** Encapsulate Record: the public mutable class became an immutable record. */
public record Shipment(String type, double weightKg, int distanceKm) { }
```

```
// File: code/worked/step3/Invoice.java
package lld.ch07.worked.step3;

import java.util.List;

/** Encapsulate Collection: callers get an unmodifiable copy, never the live list. */
public record Invoice(String customer, List<Shipment> shipments) {
    public Invoice {
        shipments = List.copyOf(shipments);
    }
}
```

```
// File: code/worked/step3/StatementData.java
package lld.ch07.worked.step3;

import java.util.List;

/** The intermediate data structure produced by phase 1 and consumed by phase 2 (Split Phase). */
public record StatementData(String customer, List<Line> lines, double totalAmount, int totalPoints) {
    public record Line(String type, double weightKg, int distanceKm, double amount) { }
}
```

```java
// File: code/worked/step3/StatementCalculator.java
package lld.ch07.worked.step3;

import java.util.List;

/** Phase 1: pure calculation. Combine Functions into Transform - all derived values in one place. */
public final class StatementCalculator {

    public StatementData calculate(Invoice invoice) {
        List<StatementData.Line> lines = invoice.shipments().stream()
                .map(s -> new StatementData.Line(s.type(), s.weightKg(), s.distanceKm(), amountFor(s)))
                .toList();
        double total = lines.stream().map(StatementData.Line::amount).reduce(0.0, Double::sum);
        int points = invoice.shipments().stream().mapToInt(this::pointsFor).sum();
        return new StatementData(invoice.customer(), lines, total, points);
    }

    double amountFor(Shipment shipment) {
        double result;
        switch (shipment.type()) {
            case "STD":
                result = 5 + shipment.weightKg() * 1.2;
                if (shipment.distanceKm() > 100) result += (shipment.distanceKm() - 100) * 0.05;
                break;
            case "EXP":
                result = 12 + shipment.weightKg() * 2.0;
                if (shipment.distanceKm() > 50) result += (shipment.distanceKm() - 50) * 0.10;
                break;
            case "FRG":
                result = 8 + shipment.weightKg() * 1.5 + 3;
                if (shipment.weightKg() > 10) result = result * 1.2;
                break;
            default:
                throw new IllegalArgumentException("unknown type: " + shipment.type());
        }
        return result;
    }

    int pointsFor(Shipment shipment) {
        int result = shipment.distanceKm() / 100;
        if ("EXP".equals(shipment.type())) result += 2;
        return result;
    }
}
```

```java
// File: code/worked/step3/TextRenderer.java
package lld.ch07.worked.step3;

import java.util.Locale;

/** Phase 2: pure formatting. Knows nothing about tariffs. */
public final class TextRenderer {

    public String render(StatementData data) {
        StringBuilder result = new StringBuilder("Invoice for " + data.customer() + "\n");
        for (StatementData.Line line : data.lines()) {
            result.append("  ").append(line.type()).append(" ").append(line.weightKg()).append("kg ")
                  .append(line.distanceKm()).append("km: ").append(money(line.amount())).append("\n");
        }
        result.append("Amount owed: ").append(money(data.totalAmount())).append("\n");
        result.append("Points earned: ").append(data.totalPoints()).append("\n");
        return result.toString();
    }

    static String money(double amount) {
        return String.format(Locale.ROOT, "%.2f", amount);
    }
}
```

```java
// File: code/worked/step3/Main.java
package lld.ch07.worked.step3;

import java.util.List;

/** STEP 3 - Split Phase: calculate (StatementCalculator) then render (TextRenderer). */
public class Main {

    public static String run() {
        Invoice invoice = new Invoice("Asha Traders", List.of(
                new Shipment("STD", 4.0, 250),
                new Shipment("EXP", 2.5, 80),
                new Shipment("FRG", 12.0, 40),
                new Shipment("STD", 1.0, 90),
                new Shipment("STD", 0.5, 100),
                new Shipment("FRG", 10.0, 150)));
        return new TextRenderer().render(new StatementCalculator().calculate(invoice));
    }

    public static void main(String[] args) {
        System.out.print(run());
    }
}
```

Worked example — step 4

```java
// File: code/worked/step4/ServiceType.java
package lld.ch07.worked.step4;

/** Replace Primitive with Object (type code String -> enum). Invalid codes fail at the boundary. */
public enum ServiceType {
    STANDARD("STD"), EXPRESS("EXP"), FRAGILE("FRG");

    private final String code;

    ServiceType(String code) { this.code = code; }

    public String code() { return code; }

    public static ServiceType fromCode(String code) {
        for (ServiceType t : values()) {
            if (t.code.equals(code)) return t;
        }
        throw new IllegalArgumentException("unknown type: " + code);
    }
}
```

```java
// File: code/worked/step4/Shipment.java
package lld.ch07.worked.step4;

import java.util.Objects;

public record Shipment(ServiceType type, double weightKg, int distanceKm) {
    public Shipment {
        Objects.requireNonNull(type, "type");
    }
}
```

```java
// File: code/worked/step4/Invoice.java
package lld.ch07.worked.step4;

import java.util.List;

public record Invoice(String customer, List<Shipment> shipments) {
    public Invoice {
        shipments = List.copyOf(shipments);
    }
}
```

```java
// File: code/worked/step4/StatementData.java
package lld.ch07.worked.step4;

import java.util.List;

public record StatementData(String customer, List<Line> lines, double totalAmount, int totalPoints) {
    public record Line(ServiceType type, double weightKg, int distanceKm, double amount) { }
}
```

```java
// File: code/worked/step4/StatementCalculator.java
package lld.ch07.worked.step4;

import java.util.List;

/** Replace Magic Literal: every tariff number now has a name. The switch is exhaustive over the enum. */
public final class StatementCalculator {

    static final double STD_BASE = 5, STD_PER_KG = 1.2, STD_FREE_KM = 100, STD_PER_EXTRA_KM = 0.05;
    static final double EXP_BASE = 12, EXP_PER_KG = 2.0, EXP_FREE_KM = 50, EXP_PER_EXTRA_KM = 0.10;
    static final double FRG_BASE = 8, FRG_PER_KG = 1.5, FRG_HANDLING = 3, FRG_HEAVY_KG = 10, FRG_HEAVY_FACTOR =
1.2;
    static final int KM_PER_POINT = 100, EXPRESS_BONUS_POINTS = 2;

    public StatementData calculate(Invoice invoice) {
        List<StatementData.Line> lines = invoice.shipments().stream()
                .map(s -> new StatementData.Line(s.type(), s.weightKg(), s.distanceKm(), amountFor(s)))
                .toList();
        double total = lines.stream().map(StatementData.Line::amount).reduce(0.0, Double::sum);
        int points = invoice.shipments().stream().mapToInt(this::pointsFor).sum();
        return new StatementData(invoice.customer(), lines, total, points);
    }

    double amountFor(Shipment s) {
        return switch (s.type()) {
            case STANDARD -> STD_BASE + s.weightKg() * STD_PER_KG
                    + (s.distanceKm() > STD_FREE_KM ? (s.distanceKm() - STD_FREE_KM) * STD_PER_EXTRA_KM : 0);
            case EXPRESS -> EXP_BASE + s.weightKg() * EXP_PER_KG
                    + (s.distanceKm() > EXP_FREE_KM ? (s.distanceKm() - EXP_FREE_KM) * EXP_PER_EXTRA_KM : 0);
            case FRAGILE -> {
                double base = FRG_BASE + s.weightKg() * FRG_PER_KG + FRG_HANDLING;
                yield s.weightKg() > FRG_HEAVY_KG ? base * FRG_HEAVY_FACTOR : base;
            }
        };
    }

    int pointsFor(Shipment s) {
        int result = s.distanceKm() / KM_PER_POINT;
        if (s.type() == ServiceType.EXPRESS) result += EXPRESS_BONUS_POINTS;
        return result;
    }
}
```

```java
// File: code/worked/step4/TextRenderer.java
package lld.ch07.worked.step4;

import java.util.Locale;

public final class TextRenderer {

    public String render(StatementData data) {
        StringBuilder result = new StringBuilder("Invoice for " + data.customer() + "\n");
        for (StatementData.Line line : data.lines()) {
            result.append("  ").append(line.type().code()).append(" ").append(line.weightKg()).append("kg ")
                  .append(line.distanceKm()).append("km: ").append(money(line.amount())).append("\n");
        }
        result.append("Amount owed: ").append(money(data.totalAmount())).append("\n");
        result.append("Points earned: ").append(data.totalPoints()).append("\n");
        return result.toString();
    }

    static String money(double amount) {
        return String.format(Locale.ROOT, "%.2f", amount);
    }
}
```

```java
// File: code/worked/step4/Main.java
package lld.ch07.worked.step4;

import java.util.List;

/** STEP 4 - Replace Primitive with Object (ServiceType enum) + Replace Magic Literal. */
public class Main {

    public static String run() {
        Invoice invoice = new Invoice("Asha Traders", List.of(
                new Shipment(ServiceType.fromCode("STD"), 4.0, 250),
                new Shipment(ServiceType.fromCode("EXP"), 2.5, 80),
                new Shipment(ServiceType.fromCode("FRG"), 12.0, 40),
                new Shipment(ServiceType.fromCode("STD"), 1.0, 90),
                new Shipment(ServiceType.fromCode("STD"), 0.5, 100),
                new Shipment(ServiceType.fromCode("FRG"), 10.0, 150)));
        return new TextRenderer().render(new StatementCalculator().calculate(invoice));
    }

    public static void main(String[] args) {
        System.out.print(run());
    }
}
```

Worked example — step 5 (final)

```java
// File: code/worked/step5/Tariff.java
package lld.ch07.worked.step5;

/**
 * Replace Conditional with Polymorphism: each branch of the old switch became a class.
 * A sealed interface lists every permitted implementation, so the compiler knows the full set.
 */
public sealed interface Tariff permits StandardTariff, ExpressTariff, FragileTariff {

    double charge(double weightKg, int distanceKm);

    /** Default behaviour shared by most tariffs; ExpressTariff overrides it. */
    default int loyaltyPoints(int distanceKm) {
        return distanceKm / 100;
    }
}
```

```java
// File: code/worked/step5/StandardTariff.java
package lld.ch07.worked.step5;

public record StandardTariff() implements Tariff {
    private static final double BASE = 5, PER_KG = 1.2, FREE_KM = 100, PER_EXTRA_KM = 0.05;

    @Override
    public double charge(double weightKg, int distanceKm) {
        return BASE + weightKg * PER_KG + (distanceKm > FREE_KM ? (distanceKm - FREE_KM) * PER_EXTRA_KM : 0);
    }
}
```

```java
// File: code/worked/step5/ExpressTariff.java
package lld.ch07.worked.step5;

public record ExpressTariff() implements Tariff {
    private static final double BASE = 12, PER_KG = 2.0, FREE_KM = 50, PER_EXTRA_KM = 0.10;
    private static final int BONUS_POINTS = 2;

    @Override
    public double charge(double weightKg, int distanceKm) {
        return BASE + weightKg * PER_KG + (distanceKm > FREE_KM ? (distanceKm - FREE_KM) * PER_EXTRA_KM : 0);
    }

    @Override
    public int loyaltyPoints(int distanceKm) {
        return Tariff.super.loyaltyPoints(distanceKm) + BONUS_POINTS;
    }
}
```

```java
// File: code/worked/step5/FragileTariff.java
package lld.ch07.worked.step5;

public record FragileTariff() implements Tariff {
    private static final double BASE = 8, PER_KG = 1.5, HANDLING = 3, HEAVY_KG = 10, HEAVY_FACTOR = 1.2;

    @Override
    public double charge(double weightKg, int distanceKm) {
        double base = BASE + weightKg * PER_KG + HANDLING;
        return weightKg > HEAVY_KG ? base * HEAVY_FACTOR : base;
    }
}
```

```java
// File: code/worked/step5/ServiceType.java
package lld.ch07.worked.step5;

/** The enum now just names a service and points at its Tariff (Strategy). */
public enum ServiceType {
    STANDARD("STD", new StandardTariff()),
    EXPRESS("EXP", new ExpressTariff()),
    FRAGILE("FRG", new FragileTariff());

    private final String code;
    private final Tariff tariff;

    ServiceType(String code, Tariff tariff) { this.code = code; this.tariff = tariff; }

    public String code() { return code; }
    public Tariff tariff() { return tariff; }

    public static ServiceType fromCode(String code) {
        for (ServiceType t : values()) {
            if (t.code.equals(code)) return t;
        }
        throw new IllegalArgumentException("unknown type: " + code);
    }
}
```

```java
// File: code/worked/step5/Shipment.java
package lld.ch07.worked.step5;

import java.util.Objects;

/** Move Function: the shipment now answers its own charge and points (cures Feature Envy). */
public record Shipment(ServiceType type, double weightKg, int distanceKm) {
    public Shipment {
        Objects.requireNonNull(type, "type");
    }

    public double charge() { return type.tariff().charge(weightKg, distanceKm); }

    public int loyaltyPoints() { return type.tariff().loyaltyPoints(distanceKm); }
}
```

```java
// File: code/worked/step5/Invoice.java
package lld.ch07.worked.step5;

import java.util.List;

public record Invoice(String customer, List<Shipment> shipments) {
    public Invoice {
        shipments = List.copyOf(shipments);
    }
}
```

```java
// File: code/worked/step5/StatementData.java
package lld.ch07.worked.step5;

import java.util.List;

public record StatementData(String customer, List<Line> lines, double totalAmount, int totalPoints) {
    public record Line(ServiceType type, double weightKg, int distanceKm, double amount) { }
}
```

```java
// File: code/worked/step5/StatementCalculator.java
package lld.ch07.worked.step5;

import java.util.List;

/** Phase 1. No switch left: adding a tariff never touches this class (Open/Closed). */
public final class StatementCalculator {

    public StatementData calculate(Invoice invoice) {
        List<StatementData.Line> lines = invoice.shipments().stream()
                .map(s -> new StatementData.Line(s.type(), s.weightKg(), s.distanceKm(), s.charge()))
                .toList();
        double total = lines.stream().map(StatementData.Line::amount).reduce(0.0, Double::sum);
        int points = invoice.shipments().stream().mapToInt(Shipment::loyaltyPoints).sum();
        return new StatementData(invoice.customer(), lines, total, points);
    }
}
```

```java
// File: code/worked/step5/StatementRenderer.java
package lld.ch07.worked.step5;

import java.util.Locale;

/** Phase 2 abstraction. Extracted once a second renderer (HTML) was needed. */
public interface StatementRenderer {

    String render(StatementData data);

    static String money(double amount) {
        return String.format(Locale.ROOT, "%.2f", amount);
    }
}
```

```java
// File: code/worked/step5/TextRenderer.java
package lld.ch07.worked.step5;

import static lld.ch07.worked.step5.StatementRenderer.money;

public final class TextRenderer implements StatementRenderer {

    @Override
    public String render(StatementData data) {
        StringBuilder result = new StringBuilder("Invoice for " + data.customer() + "\n");
        for (StatementData.Line line : data.lines()) {
            result.append("  ").append(line.type().code()).append(" ").append(line.weightKg()).append("kg ")
                  .append(line.distanceKm()).append("km: ").append(money(line.amount())).append("\n");
        }
        result.append("Amount owed: ").append(money(data.totalAmount())).append("\n");
        result.append("Points earned: ").append(data.totalPoints()).append("\n");
        return result.toString();
    }
}
```

```java
// File: code/worked/step5/HtmlRenderer.java
package lld.ch07.worked.step5;

import static lld.ch07.worked.step5.StatementRenderer.money;

/** The NEW feature. Thanks to Split Phase it reuses the calculator untouched. */
public final class HtmlRenderer implements StatementRenderer {

    @Override
    public String render(StatementData data) {
        StringBuilder result = new StringBuilder("<h1>Invoice for " + data.customer() + "</h1>\n<table>\n");
        for (StatementData.Line line : data.lines()) {
            result.append("  <tr><td>").append(line.type().code()).append("</td><td>")
                  .append(line.weightKg()).append("kg</td><td>").append(line.distanceKm())
                  .append("km</td><td>").append(money(line.amount())).append("</td></tr>\n");
        }
        result.append("</table>\n<p>Amount owed: <b>").append(money(data.totalAmount())).append("</b></p>\n");
        result.append("<p>Points earned: <b>").append(data.totalPoints()).append("</b></p>\n");
        return result.toString();
    }
}
```

```java
// File: code/worked/step5/Main.java
package lld.ch07.worked.step5;

import java.util.List;

/** STEP 5 (final) - Replace Conditional with Polymorphism + Move Function, then add the HTML feature. */
public class Main {

    static Invoice sampleInvoice() {
        return new Invoice("Asha Traders", List.of(
                new Shipment(ServiceType.fromCode("STD"), 4.0, 250),
                new Shipment(ServiceType.fromCode("EXP"), 2.5, 80),
                new Shipment(ServiceType.fromCode("FRG"), 12.0, 40),
                new Shipment(ServiceType.fromCode("STD"), 1.0, 90),
                new Shipment(ServiceType.fromCode("STD"), 0.5, 100),
                new Shipment(ServiceType.fromCode("FRG"), 10.0, 150)));
    }

    public static String run() {
        return new TextRenderer().render(new StatementCalculator().calculate(sampleInvoice()));
    }

    public static void main(String[] args) {
        System.out.print(run());
        System.out.println("--- new feature: HTML statement ---");
        System.out.print(new HtmlRenderer().render(new StatementCalculator().calculate(sampleInvoice())));
    }
}
```

Output (`java -cp /tmp/lld_07 lld.ch07.worked.step5.Main`):

```
Invoice for Asha Traders
  STD 4.0kg 250km: 17.30
  EXP 2.5kg 80km: 20.00
  FRG 12.0kg 40km: 34.80
  STD 1.0kg 90km: 6.20
  STD 0.5kg 100km: 5.60
  FRG 10.0kg 150km: 26.00
Amount owed: 109.90
Points earned: 6
--- new feature: HTML statement ---
<h1>Invoice for Asha Traders</h1>
<table>
  <tr><td>STD</td><td>4.0kg</td><td>250km</td><td>17.30</td></tr>
  <tr><td>EXP</td><td>2.5kg</td><td>80km</td><td>20.00</td></tr>
  <tr><td>FRG</td><td>12.0kg</td><td>40km</td><td>34.80</td></tr>
  <tr><td>STD</td><td>1.0kg</td><td>90km</td><td>6.20</td></tr>
  <tr><td>STD</td><td>0.5kg</td><td>100km</td><td>5.60</td></tr>
  <tr><td>FRG</td><td>10.0kg</td><td>150km</td><td>26.00</td></tr>
</table>
<p>Amount owed: <b>109.90</b></p>
<p>Points earned: <b>6</b></p>
```

Output of `lld.ch07.worked.Main` is shown in 7.5. Steps 0–4 each print the same text invoice as the first nine lines above.

Refactoring toward patterns

```java
// File: code/towardpatterns/StrategyRefactoring.java
package lld.ch07.towardpatterns;

import java.util.EnumMap;
import java.util.Map;

/** Replace Conditional Logic with Strategy. */
public final class StrategyRefactoring {

    // ---------- BEFORE: a switch that every new tier must edit ----------
    static double legacyPrice(String tier, double amount) {
        switch (tier) {
            case "GOLD":   return amount * 0.80;
            case "SILVER": return amount * 0.90;
            case "NEW":    return amount > 100 ? amount - 10 : amount;
            default:       return amount;
        }
    }

    // ---------- AFTER: one small class (here a lambda) per rule ----------
    @FunctionalInterface
    interface DiscountPolicy {
        double apply(double amount);
    }

    enum Tier { GOLD, SILVER, NEW, REGULAR }

    static final class PriceCalculator {
        private final Map<Tier, DiscountPolicy> policies = new EnumMap<>(Tier.class);
        private final DiscountPolicy fallback = amount -> amount;

        PriceCalculator() {
            policies.put(Tier.GOLD, amount -> amount * 0.80);
            policies.put(Tier.SILVER, amount -> amount * 0.90);
            policies.put(Tier.NEW, amount -> amount > 100 ? amount - 10 : amount);
        }

        /** New tiers are registered, not coded into a switch (Open/Closed). */
        void register(Tier tier, DiscountPolicy policy) { policies.put(tier, policy); }

        double price(Tier tier, double amount) {
            return policies.getOrDefault(tier, fallback).apply(amount);
        }
    }

    static boolean demo() {
        PriceCalculator calc = new PriceCalculator();
        boolean same = true;
        for (Tier t : Tier.values()) {
            for (double amount : new double[] {50, 100, 250}) {
                double before = legacyPrice(t.name(), amount);
                double after = calc.price(t, amount);
                same &= before == after;
            }
        }
        System.out.println("Strategy:        GOLD 250 -> " + calc.price(Tier.GOLD, 250)
                + ", NEW 250 -> " + calc.price(Tier.NEW, 250) + " | same as switch: " + same);
        return same;
    }
}
```

```java
// File: code/towardpatterns/StateRefactoring.java
package lld.ch07.towardpatterns;

/** Replace State-Altering Conditionals with State. */
public final class StateRefactoring {

    // ---------- BEFORE: every method re-checks a status string ----------
    static final class LegacyOrder {
        String status = "NEW";

        String pay() {
            if (status.equals("NEW")) { status = "PAID"; return "paid"; }
            return "cannot pay when " + status;
        }
        String ship() {
            if (status.equals("PAID")) { status = "SHIPPED"; return "shipped"; }
            return "cannot ship when " + status;
        }
        String cancel() {
            if (status.equals("NEW") || status.equals("PAID")) { status = "CANCELLED"; return "cancelled"; }
            return "cannot cancel when " + status;
        }
    }

    // ---------- AFTER: each state is an object that knows its own legal moves ----------
    interface OrderState {
        String name();
        default OrderState pay(Order o)    { return o.reject("pay", this); }
        default OrderState ship(Order o)   { return o.reject("ship", this); }
        default OrderState cancel(Order o) { return o.reject("cancel", this); }
    }

    enum States implements OrderState {
        NEW {
            @Override public OrderState pay(Order o)    { o.log("paid"); return PAID; }
            @Override public OrderState cancel(Order o) { o.log("cancelled"); return CANCELLED; }
        },
        PAID {
            @Override public OrderState ship(Order o)   { o.log("shipped"); return SHIPPED; }
            @Override public OrderState cancel(Order o) { o.log("cancelled"); return CANCELLED; }
        },
        SHIPPED, CANCELLED
    }

    static final class Order {
        private OrderState state = States.NEW;
        private String lastMessage = "";

        String pay()    { state = state.pay(this);    return lastMessage; }
        String ship()   { state = state.ship(this);   return lastMessage; }
        String cancel() { state = state.cancel(this); return lastMessage; }

        void log(String msg) { lastMessage = msg; }

        OrderState reject(String action, OrderState current) {
            lastMessage = "cannot " + action + " when " + current.name();
            return current;                       // stay in the same state
        }
    }

    static boolean demo() {
        LegacyOrder before = new LegacyOrder();
        Order after = new Order();
        String[] script = {"ship", "pay", "pay", "ship", "cancel"};
        boolean same = true;
        StringBuilder trace = new StringBuilder();
        for (String step : script) {
            String b = switch (step) { case "pay" -> before.pay(); case "ship" -> before.ship(); default ->
before.cancel(); };
            String a = switch (step) { case "pay" -> after.pay();  case "ship" -> after.ship();  default ->
```

```
after.cancel(); };
            same &= a.equals(b);
            trace.append(a).append("; ");
        }
        System.out.println("State:           " + trace + "| same as if-chains: " + same);
        return same;
    }
}
```

```java
// File: code/towardpatterns/SpecialCaseRefactoring.java
package lld.ch07.towardpatterns;

import java.util.Map;

/** Introduce Special Case (Null Object): stop repeating "if (customer == null)" at every call site. */
public final class SpecialCaseRefactoring {

    // ---------- BEFORE ----------
    record LegacyCustomer(String name, String plan) { }

    static final Map<String, LegacyCustomer> LEGACY_DB = Map.of("c1", new LegacyCustomer("Ravi", "premium"));

    static String legacyGreeting(String id) {
        LegacyCustomer c = LEGACY_DB.get(id);
        String name = (c == null) ? "occupant" : c.name();       // null check #1
        String plan = (c == null) ? "basic" : c.plan();          // null check #2 ... and so on in every client
        return "Hello " + name + " on " + plan;
    }

    // ---------- AFTER ----------
    sealed interface Customer permits KnownCustomer, UnknownCustomer {
        String name();
        String plan();
    }
    record KnownCustomer(String name, String plan) implements Customer { }
    /** The special case: one place holds the default behaviour. */
    record UnknownCustomer() implements Customer {
        @Override public String name() { return "occupant"; }
        @Override public String plan() { return "basic"; }
    }

    static final Map<String, Customer> DB = Map.of("c1", new KnownCustomer("Ravi", "premium"));

    static Customer find(String id) {
        return DB.getOrDefault(id, new UnknownCustomer());
    }

    static String greeting(String id) {
        Customer c = find(id);                                   // no null checks at all
        return "Hello " + c.name() + " on " + c.plan();
    }

    static boolean demo() {
        boolean same = legacyGreeting("c1").equals(greeting("c1")) && legacyGreeting("zz").equals(greeting("zz"));
        System.out.println("Special Case:    " + greeting("c1") + " / " + greeting("zz") + " | same: " + same);
        return same;
    }
}
```

```java
// File: code/towardpatterns/TemplateMethodRefactoring.java
package lld.ch07.towardpatterns;

import java.util.List;

/** Form Template Method: two siblings with the same algorithm skeleton but different details. */
public final class TemplateMethodRefactoring {

    // ---------- BEFORE: duplicated skeleton ----------
    static String legacyCsv(List<String> rows) {
        StringBuilder sb = new StringBuilder();
        sb.append("name\n");                                  // header
        for (String r : rows) sb.append(r).append("\n");      // body
        sb.append("# ").append(rows.size()).append(" rows");  // footer
        return sb.toString();
    }
    static String legacyMarkdown(List<String> rows) {
        StringBuilder sb = new StringBuilder();
        sb.append("| name |\n|---|\n");
        for (String r : rows) sb.append("| ").append(r).append(" |\n");
        sb.append("_").append(rows.size()).append(" rows_");
        return sb.toString();
    }

    // ---------- AFTER: the skeleton lives once, subclasses fill in the steps ----------
    abstract static class Exporter {
        /** The template method - final so subclasses cannot change the order of steps. */
        final String export(List<String> rows) {
            StringBuilder sb = new StringBuilder(header());
            for (String r : rows) sb.append(row(r));
            return sb.append(footer(rows.size())).toString();
        }
        protected abstract String header();
        protected abstract String row(String value);
        protected abstract String footer(int count);
    }
    static final class CsvExporter extends Exporter {
        protected String header() { return "name\n"; }
        protected String row(String v) { return v + "\n"; }
        protected String footer(int n) { return "# " + n + " rows"; }
    }
    static final class MarkdownExporter extends Exporter {
        protected String header() { return "| name |\n|---|\n"; }
        protected String row(String v) { return "| " + v + " |\n"; }
        protected String footer(int n) { return "_" + n + " rows_"; }
    }

    static boolean demo() {
        List<String> rows = List.of("Asha", "Ravi");
        boolean same = legacyCsv(rows).equals(new CsvExporter().export(rows))
                && legacyMarkdown(rows).equals(new MarkdownExporter().export(rows));
        System.out.println("Template Method: CSV and Markdown exporters | same: " + same);
        return same;
    }
}
```

```java
// File: code/towardpatterns/FactoryRefactoring.java
package lld.ch07.towardpatterns;

/** Replace Constructor with Factory Function (Kerievsky: Replace Constructors with Creation Methods). */
public final class FactoryRefactoring {

    // ---------- BEFORE: which boolean means what? ----------
    static final class LegacyLoan {
        final double amount; final boolean revolving; final int months;
        LegacyLoan(double amount, boolean revolving, int months) {
            this.amount = amount; this.revolving = revolving; this.months = months;
        }
        @Override public String toString() { return (revolving ? "revolver" : "term") + " " + amount + "/" +
months; }
    }

    // ---------- AFTER: intention-revealing creation methods, private constructor ----------
    static final class Loan {
        private final double amount; private final boolean revolving; private final int months;
        private Loan(double amount, boolean revolving, int months) {
            this.amount = amount; this.revolving = revolving; this.months = months;
        }
        static Loan termLoan(double amount, int months) { return new Loan(amount, false, months); }
        static Loan revolver(double limit)              { return new Loan(limit, true, 0); }
        @Override public String toString() { return (revolving ? "revolver" : "term") + " " + amount + "/" +
months; }
    }

    static boolean demo() {
        boolean same = new LegacyLoan(5000, false, 12).toString().equals(Loan.termLoan(5000, 12).toString())
                && new LegacyLoan(2000, true, 0).toString().equals(Loan.revolver(2000).toString());
        System.out.println("Factory:         " + Loan.termLoan(5000, 12) + ", " + Loan.revolver(2000) + " | same:
" + same);
        return same;
    }
}
```

```java
// File: code/towardpatterns/Main.java
package lld.ch07.towardpatterns;

/** Refactoring toward patterns: each demo runs the BEFORE and AFTER code and checks they agree. */
public class Main {
    public static void main(String[] args) {
        boolean ok = StrategyRefactoring.demo()
                & StateRefactoring.demo()
                & SpecialCaseRefactoring.demo()
                & TemplateMethodRefactoring.demo()
                & FactoryRefactoring.demo();
        System.out.println(ok ? "All pattern refactorings preserve behaviour." : "BEHAVIOUR CHANGED!");
        if (!ok) System.exit(1);
    }
}
```

Output:

```
Strategy:        GOLD 250 -> 200.0, NEW 250 -> 240.0 | same as switch: true
State:           cannot ship when NEW; paid; cannot pay when PAID; shipped; cannot cancel when SHIPPED; | same as
if-chains: true
Special Case:    Hello Ravi on premium / Hello occupant on basic | same: true
Template Method: CSV and Markdown exporters | same: true
Factory:         term 5000.0/12, revolver 2000.0/0 | same: true
All pattern refactorings preserve behaviour.
```

Smell cures

```java
// File: code/smells/Money.java
package lld.ch07.smells;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.Currency;
import java.util.Objects;

/** Cure for Primitive Obsession: a value object instead of a bare double. */
public record Money(BigDecimal amount, Currency currency) {

    public Money {
        Objects.requireNonNull(amount, "amount");
        Objects.requireNonNull(currency, "currency");
        amount = amount.setScale(currency.getDefaultFractionDigits(), RoundingMode.HALF_EVEN);
    }

    public static Money of(String amount, String currencyCode) {
        return new Money(new BigDecimal(amount), Currency.getInstance(currencyCode));
    }

    public Money plus(Money other) {
        requireSameCurrency(other);
        return new Money(amount.add(other.amount), currency);
    }

    public Money times(int quantity) {
        return new Money(amount.multiply(BigDecimal.valueOf(quantity)), currency);
    }

    private void requireSameCurrency(Money other) {
        if (!currency.equals(other.currency)) {
            throw new IllegalArgumentException("currency mismatch: " + currency + " vs " + other.currency);
        }
    }

    @Override public String toString() { return currency.getCurrencyCode() + " " + amount.toPlainString(); }
}
```

```java
// File: code/smells/DateRange.java
package lld.ch07.smells;

import java.time.LocalDate;

/** Cure for Data Clumps: start and end always travelled together, so they become one object. */
public record DateRange(LocalDate start, LocalDate end) {

    public DateRange {
        if (end.isBefore(start)) throw new IllegalArgumentException("end before start");
    }

    public boolean contains(LocalDate day) {
        return !day.isBefore(start) && !day.isAfter(end);
    }

    public boolean overlaps(DateRange other) {
        return !other.end.isBefore(start) && !other.start.isAfter(end);
    }
}
```

```java
// File: code/smells/Order.java
package lld.ch07.smells;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

/**
 * Cures for Message Chains (Hide Delegate), Feature Envy (Move Function)
 * and Mutable Data (Encapsulate Collection).
 */
public final class Order {

    public record Address(String street, String city) { }
    public record Customer(String name, Address address) { }
    public record Line(String sku, Money unitPrice, int quantity) {
        /** Moved here from Order: it only used Line's data (Feature Envy cured). */
        public Money total() { return unitPrice.times(quantity); }
    }

    private final Customer customer;
    private final List<Line> lines = new ArrayList<>();

    public Order(Customer customer) { this.customer = customer; }

    /** Hide Delegate: callers ask the order, not order.customer().address().city(). */
    public String shippingCity() { return customer.address().city(); }

    /** Encapsulate Collection: modification goes through the owner... */
    public void add(Line line) { lines.add(line); }

    /** ...and readers get a read-only view. */
    public List<Line> lines() { return Collections.unmodifiableList(lines); }

    public Money total() {
        return lines.stream().map(Line::total).reduce(Money::plus).orElse(Money.of("0", "INR"));
    }
}
```

```java
// File: code/smells/Main.java
package lld.ch07.smells;

import java.time.LocalDate;

/** Small, runnable cures for common smells. */
public class Main {
    public static void main(String[] args) {
        // Primitive Obsession -> Money: 0.1 + 0.2 is exact, currencies cannot be mixed by accident
        System.out.println("double 0.1+0.2       = " + (0.1 + 0.2));
        System.out.println("Money  0.10+0.20     = " + Money.of("0.10", "USD").plus(Money.of("0.20", "USD")));
        try {
            Money.of("1", "USD").plus(Money.of("1", "EUR"));
        } catch (IllegalArgumentException e) {
            System.out.println("mixing currencies    -> " + e.getMessage());
        }

        // Data Clumps -> DateRange
        DateRange booking = new DateRange(LocalDate.of(2026, 10, 1), LocalDate.of(2026, 10, 5));
        DateRange other = new DateRange(LocalDate.of(2026, 10, 5), LocalDate.of(2026, 10, 8));
        System.out.println("booking contains 3rd = " + booking.contains(LocalDate.of(2026, 10, 3))
                + ", overlaps other = " + booking.overlaps(other));

        // Message Chains, Feature Envy, Mutable Data
        Order order = new Order(new Order.Customer("Asha", new Order.Address("MG Road", "Bengaluru")));
        order.add(new Order.Line("PEN", Money.of("25.50", "INR"), 4));
        order.add(new Order.Line("BOOK", Money.of("199", "INR"), 1));
        System.out.println("ship to " + order.shippingCity() + ", total " + order.total());
        try {
            order.lines().clear();
        } catch (UnsupportedOperationException e) {
            System.out.println("outsider tried lines().clear() -> UnsupportedOperationException (collection is
encapsulated)");
        }
    }
}
```

Output:

```
double 0.1+0.2       = 0.30000000000000004
Money  0.10+0.20     = USD 0.30
mixing currencies    -> currency mismatch: USD vs EUR
booking contains 3rd = true, overlaps other = true
ship to Bengaluru, total INR 301.00
outsider tried lines().clear() -> UnsupportedOperationException (collection is encapsulated)
```

Sources combined in this chapter

Martin Fowler, *Refactoring: Improving the Design of Existing Code*, 2nd ed. (2018) and the online catalogue at refactoring.com; refactoring.guru's smell groups; Michael Feathers, *Working Effectively with Legacy Code* (2004); Joshua Kerievsky, *Refactoring to Patterns* (2004). All explanations and code are original.